

208.5.0

Huawei LiteOS Developer Guide

Issue	6.0
Date	2023-05-15



Copyright © Huawei Technologies Co., Ltd. 2023. All rights reserved.

No part of this document may be reproduced or transmitted in any form or by any means without prior written consent of Huawei Technologies Co., Ltd.

Trademarks and Permissions



HUAWEI and other Huawei trademarks are trademarks of Huawei Technologies Co., Ltd.

All other trademarks and trade names mentioned in this document are the property of their respective holders.

Notice

The purchased products, services and features are stipulated by the contract made between Huawei and the customer. All or part of the products, services and features described in this document may not be within the purchase scope or the usage scope. Unless otherwise specified in the contract, all statements, information, and recommendations in this document are provided "AS IS" without warranties, guarantees or representations of any kind, either express or implied.

The information in this document is subject to change without notice. Every effort has been made in the preparation of this document to ensure accuracy of the contents, but all statements, information, and recommendations in this document do not constitute a warranty of any kind, express or implied.

Huawei Technologies Co., Ltd.

Address: Huawei Industrial Base
Bantian, Longgang
Shenzhen 518129
People's Republic of China

Website: <https://www.huawei.com>

Email: support@huawei.com

Contents

1 Preface.....	1
2 Overview.....	3
2.1 Background.....	3
2.2 Supported Cores.....	6
2.3 Constraints.....	7
3 Basic Kernel.....	8
3.1 Task.....	8
3.1.1 Overview.....	8
3.1.2 Development Guidelines.....	12
3.1.3 Precautions.....	24
3.1.4 Programming Example.....	25
3.1.5 Programming Example (SMP).....	28
3.2 Memory.....	30
3.2.1 Overview.....	30
3.2.2 Dynamic Memory.....	33
3.2.2.1 Development Guidelines.....	33
3.2.2.2 Precautions.....	39
3.2.2.3 Programming Example.....	40
3.2.2.3.1 Common Memory Usage Example.....	40
3.2.2.3.2 Multi-Module Memory Usage Example.....	41
3.2.3 Static Memory.....	42
3.2.3.1 Development Guidelines.....	42
3.2.3.2 Precautions.....	44
3.2.3.3 Programming Example.....	44
3.3 Interrupt.....	45
3.3.1 Overview.....	45
3.3.2 Development Guidelines.....	48
3.3.3 Precautions.....	55
3.3.4 Programming Example.....	56
3.4 Exception Management.....	58
3.4.1 Overview.....	58
3.4.2 User Guide.....	60

3.4.3 Precautions.....	60
3.4.4 Problem Locating Example.....	60
3.5 Error Handling.....	63
3.5.1 Overview.....	63
3.5.2 Development Guidelines.....	64
3.5.3 Precautions.....	66
3.5.4 Programming Example.....	66
3.6 Queue.....	66
3.6.1 Overview.....	67
3.6.2 Development Guidelines.....	70
3.6.3 Precautions.....	76
3.6.4 Programming Example.....	77
3.7 Event.....	79
3.7.1 Overview.....	79
3.7.2 Development Guidelines.....	81
3.7.3 Precautions.....	84
3.7.4 Programming Example.....	84
3.7.4.1 Read and Write Based on the Event Type.....	84
3.7.4.2 Read and Write Based on Event Conditions.....	86
3.8 Semaphore.....	89
3.8.1 Overview.....	89
3.8.2 Development Guidelines.....	91
3.8.3 Precautions.....	94
3.8.4 Programming Example.....	95
3.9 Read/Write Semaphore.....	97
3.9.1 Overview.....	97
3.9.2 Development Guidelines.....	99
3.9.3 Precautions.....	102
3.9.4 Programming Example.....	102
3.10 Mutex.....	105
3.10.1 Overview.....	106
3.10.2 Development Guidelines.....	107
3.10.3 Precautions.....	109
3.10.4 Programming Example.....	110
3.11 Software Timer.....	112
3.11.1 Overview.....	112
3.11.2 Development Guidelines.....	113
3.11.3 Precautions.....	117
3.11.4 Programming Example.....	118
3.12 Spinlock.....	119
3.12.1 Overview.....	119
3.12.2 Development Guidelines.....	120

3.12.3 Precautions.....	122
3.12.4 Programming Example.....	122
3.13 Levels of Initialization.....	124
3.13.1 Overview.....	125
3.13.2 Development Guidelines.....	125
3.13.3 Precautions.....	126
3.13.4 Programming Example.....	127
3.14 Others.....	128
3.14.1 Time Management.....	128
3.14.1.1 Overview.....	128
3.14.1.2 Development Guidelines.....	129
3.14.1.3 Precautions.....	130
3.14.1.4 Programming Example.....	130
3.14.2 Doubly Linked List.....	132
3.14.2.1 Overview.....	132
3.14.2.2 Development Guidelines.....	132
3.14.2.3 Precautions.....	134
3.14.2.4 Programming Example.....	134
3.14.3 Atomic Operation.....	136
3.14.3.1 Overview.....	136
3.14.3.2 Development Guidelines.....	137
3.14.3.3 Precautions.....	139
3.14.3.4 Programming Example.....	139
3.14.4 Bit Operation.....	140
3.14.4.1 Overview.....	140
3.14.4.2 Development Guidelines.....	140
3.14.4.3 Precautions.....	141
3.14.4.4 Programming Example.....	141
3.14.5 ringbuf.....	142
3.14.5.1 Overview.....	142
3.14.5.2 Development Guidelines.....	143
3.14.5.3 Precautions.....	145
3.14.5.4 Programming Example.....	145
3.14.6 Universal Memory Access.....	147
3.14.6.1 Overview.....	147
3.14.6.2 Development Guidelines.....	147
3.14.6.3 Precautions.....	149
4 Extended Kernels.....	150
4.1 Dynamic Loading.....	150
4.1.1 Overview.....	150
4.1.2 Development Guidelines.....	151
4.1.3 Precautions.....	160

4.1.4 Programming Example.....	161
4.2 Scatter Loading.....	162
4.2.1 Overview.....	162
4.2.2 Development Guidelines.....	163
4.2.3 Precautions.....	167
4.3 Low Power Consumption Framework.....	167
4.3.1 Overview.....	167
4.3.2 RunStop.....	167
4.3.2.1 Overview.....	168
4.3.2.2 Developer Guide.....	168
4.3.2.3 Precautions.....	175
4.3.3 System Sleep.....	175
4.3.3.1 Overview.....	176
4.3.3.2 Development Guidelines.....	177
4.3.3.3 Precautions.....	180
4.3.3.4 Programming Example.....	180
4.3.4 Tickless.....	182
4.3.4.1 Overview.....	182
4.3.4.2 Development Guidelines.....	183
4.3.4.3 Precautions.....	183
4.4 Access Permission Management.....	183
4.4.1 Overview.....	184
4.4.2 PMP.....	184
4.4.2.1 Overview.....	184
4.4.2.2 Development Guidelines.....	184
4.4.2.3 Precautions.....	188
4.4.2.4 Programming Example.....	189
4.4.3 MPU.....	189
4.4.3.1 Overview.....	189
4.4.3.2 Development Guidelines.....	190
4.4.3.3 Precautions.....	192
4.4.3.4 Programming Example.....	192
4.4.4 MMU.....	192
4.4.4.1 Overview.....	193
4.4.4.2 ARM32 MMU.....	193
4.4.4.2.1 Development Guidelines.....	195
4.4.4.2.2 Precautions.....	195
4.4.4.2.3 Programming Example.....	195
4.4.4.3 ARM64 MMU.....	198
4.4.4.3.1 Development Guidelines.....	199
4.4.4.3.2 Precautions.....	202
4.4.4.3.3 Programming Example.....	202

4.5 CPUP.....	203
4.5.1 Overview.....	203
4.5.2 Development Guidelines.....	203
4.5.3 Precautions.....	207
4.5.4 Programming Example.....	207
4.6 Trace.....	209
4.6.1 Overview.....	209
4.6.2 Development Guidelines.....	209
4.6.3 Precautions.....	215
4.6.4 Programming Example.....	215
4.7 LMS.....	217
4.7.1 Overview.....	217
4.7.2 Development Guidelines.....	220
4.7.3 Precautions.....	223
4.8 Perf.....	223
4.8.1 Overview.....	224
4.8.2 Development Guidelines.....	224
4.8.3 Precautions.....	228
4.8.4 Programming Example.....	229
4.9 C++ Support.....	232
4.9.1 Overview.....	232
4.9.2 Development Guidelines.....	232
4.9.3 Precautions.....	234
4.9.4 Programming Example.....	234
4.10 Dynamic Shared Memory.....	236
4.10.1 Overview.....	236
4.10.2 Development Guidelines.....	238
4.10.3 Precautions.....	241
4.11 Inter-core Communication.....	241
4.11.1 Overview.....	241
4.11.2 Development Guidelines.....	242
4.11.3 Precautions.....	242
4.11.4 Programming Example.....	242
5 File System.....	245
5.1 Functions Overview.....	245
5.2 VFS.....	246
5.2.1 Overview.....	247
5.2.2 Development Guidelines.....	248
5.2.3 Precautions.....	250
5.3 NFS.....	251
5.3.1 Overview.....	251
5.3.2 Development Guidelines.....	252

5.3.3 Precautions.....	254
5.4 JFFS2.....	255
5.4.1 Overview.....	255
5.4.2 Development Guidelines.....	255
5.4.3 Precautions.....	259
5.5 FAT.....	259
5.5.1 Overview.....	260
5.5.2 Development Guidelines.....	260
5.5.3 Compatibility of Virtual Partitions.....	271
5.5.4 Precautions.....	272
5.6 YAFFS2.....	273
5.6.1 Overview.....	273
5.6.2 Development Guidelines.....	274
5.6.3 Precautions.....	277
5.7 RAMFS.....	277
5.7.1 Overview.....	277
5.7.2 Development Guidelines.....	278
5.7.3 Precautions.....	278
5.8 PROC.....	279
5.8.1 Overview.....	279
5.8.2 Development Guidelines.....	279
5.8.3 Precautions.....	282
5.8.4 Programming Example.....	282
5.9 ROMFS.....	284
5.9.1 Overview.....	284
5.9.2 Development Guidelines.....	284
5.9.3 Precautions.....	286
5.10 LITTLEFS.....	286
5.10.1 Overview.....	286
5.10.2 Development Guidelines.....	287
5.10.3 Precautions.....	289
6 Compatible APIs.....	291
6.1 Overview.....	291
6.2 POSIX APIs.....	292
6.2.1 Supported POSIX APIs.....	292
6.2.2 Supported POSIX NP APIs.....	316
6.2.3 Unsupported POSIX APIs.....	316
6.3 Libc/Libm APIs.....	329
6.3.1 Supported Libc APIs.....	329
6.3.2 Supported Libm APIs.....	349
6.3.3 Unsupported Libc/Libm APIs.....	370
6.4 C++ Compatibility Specifications.....	385

6.5 CMSIS APIs.....	387
6.5.1 CMSIS v1.0.....	387
6.5.1.1 Supported CMSIS v1.0 APIs.....	387
6.5.1.2 Unsupported CMSIS v1.0 APIs.....	389
6.5.1.3 Differences from Standard CMSIS v1.0 APIs.....	389
6.5.2 CMSIS v2.0.....	390
6.5.2.1 Supported CMSIS v2.0 APIs.....	390
6.5.2.2 Unsupported CMSIS v2.0 APIs.....	393
6.5.2.3 Differences from Standard CMSIS v2.0 APIs.....	394
6.6 Adaption to Linux.....	395
6.6.1 Completion.....	395
6.6.1.1 Overview.....	395
6.6.1.2 Development Guidelines.....	395
6.6.1.3 Precautions.....	397
6.6.1.4 Programming Example.....	397
6.6.2 Work Queue.....	398
6.6.2.1 Overview.....	398
6.6.2.2 Development Guidelines.....	399
6.6.2.3 Precautions.....	402
6.6.2.4 Programming Example.....	402
6.6.3 Wait Queue.....	403
6.6.3.1 Overview.....	403
6.6.3.2 Development Guidelines.....	404
6.6.3.3 Precautions.....	406
6.6.3.4 Programming Example.....	406
6.6.4 Interrupt.....	407
6.6.4.1 Overview.....	407
6.6.4.2 Development Guidelines.....	408
6.6.4.3 Precautions.....	409
6.6.4.4 Programming Example.....	409
6.6.5 High Resolution Timer.....	410
6.6.5.1 Overview.....	410
6.6.5.2 Development Guidelines.....	410
6.6.5.3 Precautions.....	412
6.6.5.4 Programming Example.....	412
6.6.6 Supported Linux APIs.....	413
6.6.7 Overview of APIs Not Supported by Linux.....	434
7 Driver Development.....	438
7.1 Driver Framework.....	438
7.1.1 Overview.....	438
7.1.2 Development Guidelines.....	441
7.1.3 Precautions.....	443

7.1.4 Programming Example.....	443
7.2 File Adaptation.....	444
7.2.1 Overview.....	445
7.2.2 Development Guidelines.....	445
7.2.3 Precautions.....	448
7.3 MMC Driver Development.....	448
7.3.1 Overview.....	448
7.3.2 Development Guidelines.....	449
7.3.3 Precautions.....	456
7.4 USB Driver Development.....	457
7.4.1 Overview.....	457
7.4.2 Development Guidelines.....	458
7.4.2.1 Functions.....	458
7.4.2.2 Configuring the Memory of the USB Module.....	463
7.4.2.3 Dynamically Switching the USB Protocol.....	463
7.4.2.4 Compiling the Module.....	463
7.4.3 Programming Examples.....	466
7.4.3.1 Host.....	466
7.4.3.1.1 USB Flash Drive in Host Mode.....	466
7.4.3.1.2 Host USB-to-Ethernet Port and RNDIS.....	467
7.4.3.1.3 Host 4G-Modem.....	467
7.4.3.1.4 Host USB-WIFI.....	467
7.4.3.1.5 Host UVC.....	468
7.4.3.1.6 Host HID.....	469
7.4.3.2 Device.....	469
7.4.3.2.1 Virtual USB Flash Drive in Device Mode.....	469
7.4.3.2.2 Device UVC.....	473
7.4.3.2.3 Device UAC.....	474
7.4.3.2.4 Device UAV (Camera).....	474
7.4.3.2.5 Device Virtual Serial Port.....	474
7.4.3.2.6 Device Virtual Network Port.....	476
7.4.3.2.7 Composite Device (Serial Port and Network Port).....	477
7.4.3.2.8 Device DFU.....	478
7.4.3.2.9 Device HID.....	478
7.4.4 API Reference.....	480
7.4.4.1 usbd_set_device_info.....	482
7.4.4.2 usb_init.....	484
7.4.4.3 usb_deinit.....	486
7.4.4.4 usb_is_devicemode.....	486
7.4.4.5 fmass_register_notify.....	487
7.4.4.6 fmass_unregister_notify.....	488
7.4.4.7 fmass_partition_startup.....	489

7.4.4.8 ram_mass_storage_init.....	490
7.4.4.9 ram_mass_storage_deinit.....	490
7.4.4.10 fuvc_frame_descriptors_get.....	491
7.4.4.11 uvc_open_device.....	492
7.4.4.12 uvc_close_device.....	493
7.4.4.13 uvc_wait_host.....	493
7.4.4.14 uvc_get_state.....	495
7.4.4.15 uvc_rcv_pack.....	496
7.4.4.16 uvc_video_stop.....	496
7.4.4.17 uvc_format_info_get.....	497
7.4.4.18 uvc_format_status_check.....	498
7.4.4.19 hi_camera_register_cmd.....	499
7.4.4.20 uac_wait_host.....	500
7.4.4.21 fuac_send_message.....	501
7.4.4.22 fuac_receive_message.....	502
7.4.4.23 fuac_opts_set.....	503
7.4.4.24 fuac_get_opts.....	504
7.4.4.25 fuac_rate_get.....	505
7.4.4.26 fuac2_get_opts.....	505
7.4.4.27 fuac2_set_opts.....	506
7.4.4.28 usb_dfu_init_env_entities.....	507
7.4.4.29 usb_dfu_update_status.....	508
7.4.4.30 usb_dfu_update_size_get.....	508
7.4.4.31 usb_dfu_get_entity.....	509
7.4.4.32 usb_dfu_read.....	510
7.4.4.33 hid_report_descriptor_info.....	511
7.4.4.34 hid_add_report_descriptor.....	512
7.4.4.35 fhid_send_data.....	513
7.4.4.36 fhid_rcv_data.....	513
7.4.4.37 u3g_write_from_shell.....	514
7.4.4.38 uvc_get_frame_buf.....	515
7.4.4.39 usb_dfu_free_entities.....	516
7.5 UART Driver Development.....	517
7.5.1 Overview.....	517
7.5.2 Development Guidelines.....	518
7.5.2.1 Raw Read/Write UART.....	518
7.5.2.2 UART Read/Write in File Mode.....	518
7.5.2.3 Module Configuration.....	518
7.5.3 Programming Example.....	519
7.5.3.1 Simple UART.....	519
7.5.3.2 General UART.....	519
7.5.3.2.1 UART Raw Read/Write.....	519

7.5.3.2.2 UART Read/Write in File Mode.....	520
8 Configuration Tool.....	522
8.1 Overview.....	522
8.2 Installation Tools.....	522
8.3 Usage Guidelines.....	524
8.4 menuconfig.....	524
8.5 savemenuconfig.....	525
8.6 make defconfig.....	526
8.7 make allyesconfig.....	526
8.8 make allnoconfig.....	526
8.9 Configuration Item Description.....	526
8.9.1 Overview.....	526
8.9.2 Compiler.....	526
8.9.3 Targets.....	527
8.9.4 SSP.....	528
8.9.5 NX.....	529
9 Maintenance and Debugging.....	531
9.1 Telnet.....	531
9.1.1 Overview.....	531
9.1.2 Development Guidelines.....	532
9.1.3 Precautions.....	533
9.2 Shell.....	533
9.2.1 Overview.....	533
9.2.2 Development Guidelines.....	534
9.2.3 Precautions.....	537
9.2.4 Static Registration Programming Example.....	537
9.2.5 Dynamic Registration Programming Example.....	538
9.2.6 System Commands.....	539
9.2.6.1 Enabling System Commands.....	539
9.2.6.2 help.....	539
9.2.6.3 date.....	539
9.2.6.4 uname.....	540
9.2.6.5 task.....	541
9.2.6.6 free.....	545
9.2.6.7 memcheck.....	546
9.2.6.8 memused.....	548
9.2.6.9 hwi.....	549
9.2.6.10 queue.....	550
9.2.6.11 sem.....	551
9.2.6.12 mutex.....	553
9.2.6.13 swtmr.....	554
9.2.6.14 systeminfo.....	555

9.2.6.15 log.....	556
9.2.6.16 dmesg.....	557
9.2.6.17 stack.....	559
9.2.6.18 cpup.....	560
9.2.6.19 watch.....	561
9.2.6.20 schedstatstart.....	562
9.2.6.21 schedstatstop.....	563
9.2.6.22 schedstatinfo.....	563
9.2.7 File Commands.....	564
9.2.7.1 Command for Enabling the File System.....	564
9.2.7.2 cd.....	564
9.2.7.3 pwd.....	565
9.2.7.4 mkdir.....	565
9.2.7.5 rmdir.....	566
9.2.7.6 rm.....	567
9.2.7.7 touch.....	568
9.2.7.8 cp.....	569
9.2.7.9 dd.....	570
9.2.7.10 ls.....	572
9.2.7.11 cat.....	573
9.2.7.12 lsfd.....	573
9.2.7.13 format.....	574
9.2.7.14 mount.....	575
9.2.7.15 umount.....	576
9.2.7.16 partinfo.....	576
9.2.7.17 partition.....	577
9.2.7.18 statfs.....	578
9.2.7.19 virstatfs.....	579
9.2.7.20 sync.....	580
9.2.7.21 writeproc.....	581
9.2.8 Network Commands.....	582
9.2.8.1 Commands for Enabling the Network.....	582
9.2.8.2 ifconfig.....	582
9.2.8.3 dns.....	586
9.2.8.4 arp.....	587
9.2.8.5 ntpdate.....	588
9.2.8.6 ping.....	589
9.2.8.7 ping6.....	590
9.2.8.8 tftp.....	592
9.2.8.9 netstat.....	595
9.2.8.10 ipdebug.....	596
9.2.8.11 netdebug.....	597

9.2.8.12 telnet.....	602
9.2.8.13 iperf.....	603
9.2.9 Dynamic Loading Commands.....	606
9.2.9.1 Commands for Enabling Dynamic Loading.....	606
9.2.9.2 mopen.....	607
9.2.9.3 mclose.....	607
9.2.9.4 findsym.....	608
9.2.9.5 call.....	609
9.2.9.6 lddrop.....	610
9.2.10 Trace Commands.....	610
9.2.10.1 Enabling the Trace Function.....	610
9.2.10.2 trace_mask.....	611
9.2.10.3 trace_start.....	612
9.2.10.4 trace_stop.....	612
9.2.10.5 trace_dump.....	613
9.2.10.6 trace_reset.....	614
9.3 Scheduling Statistics.....	614
9.4 Memory Debugging.....	617
9.4.1 Overview.....	617
9.4.2 Multi-Memory Pool Mechanism.....	617
9.4.3 Memory Backup Mechanism.....	620
9.4.4 Memory Validity Check.....	621
9.4.5 Memory Size Check.....	623
9.4.6 Memory Leakage Check.....	626
9.5 MMC Debugging.....	627
9.5.1 Debugging for Command Failure.....	627
9.5.2 Debugging for DMA Transmission.....	627
9.6 Inter-Task Communication Debugging.....	628
9.6.1 Queue Debugging.....	628
9.6.2 Deadlock Debugging Mechanism.....	629
9.7 Exception Interaction.....	635
9.8 Low Power Consumption Debugging Methods.....	636
9.8.1 Enabling Low-power Commissioning.....	636
9.8.2 Querying System Status.....	636
9.8.3 Controlling Dynamic Low-power Consumption.....	637
9.8.4 Blocking System Deep Sleep Event Query.....	638
9.8.5 System RunStop Dotting.....	639
9.9 Usage of Dying Gasp.....	640
9.10 Usage of Magic Keys.....	642
9.11 Usage of the Stack Estimation Tool.....	643
9.12 Debugging Case.....	647
9.12.1 Methods for Locating Memory Overwriting.....	648

9.12.1.1 Exception Information.....	648
9.12.1.2 Memory Backup Mechanism.....	648
9.12.1.3 Memory Validity Check.....	648
9.12.1.4 Memory Size Check.....	648
9.12.1.5 Global Variable Check.....	648
9.12.1.6 Task Status Check.....	649
10 Security.....	650
10.1 Address Space Layout Randomization.....	650
10.1.1 Overview.....	650
10.1.2 Usage Guide.....	655
10.1.3 Precautions.....	664
11 Appendix.....	665
11.1 Startup Process of Huawei LiteOS Kernel.....	665
11.2 Key Modification Points for Platform Compatibility Adaptation.....	666

1 Preface

Purpose

This guide describes the structure of Huawei LiteOS Kernel. It also explains how to develop and debug the Kernel.

Intended Audience

This guide is primarily intended for Huawei LiteOS Kernel developers, and is also recommended for:

- Internet of Things (IoT) device software engineers
- IoT architects

Symbol Conventions

The symbols that may be found in this guide are defined as follows:

Symbol	Description
 CAUTION	Indicates a potentially hazardous situation that, if not avoided, could result in minor or moderate injury.
NOTICE	Indicates a device or environmental safety alert that, if not avoided, could result in equipment damage, data loss, performance deterioration, or unanticipated consequences. "Notice" cannot result in injury.
 NOTE	"Note" is not a safety alert, cannot result in personal, device or environmental injury.

Change History

Document changes are cumulative. The latest document issue contains all the changes made in earlier issues.

Date	Version	Description
2021-06-22	1.0	Release 206.X
2021-11-25	2.0	Release 206.1
2022-03-23	3.0	Release 206.2
2022-11-25	4.0	Release 207.0
2023-03-30	5.0	Release 207.1

2 Overview

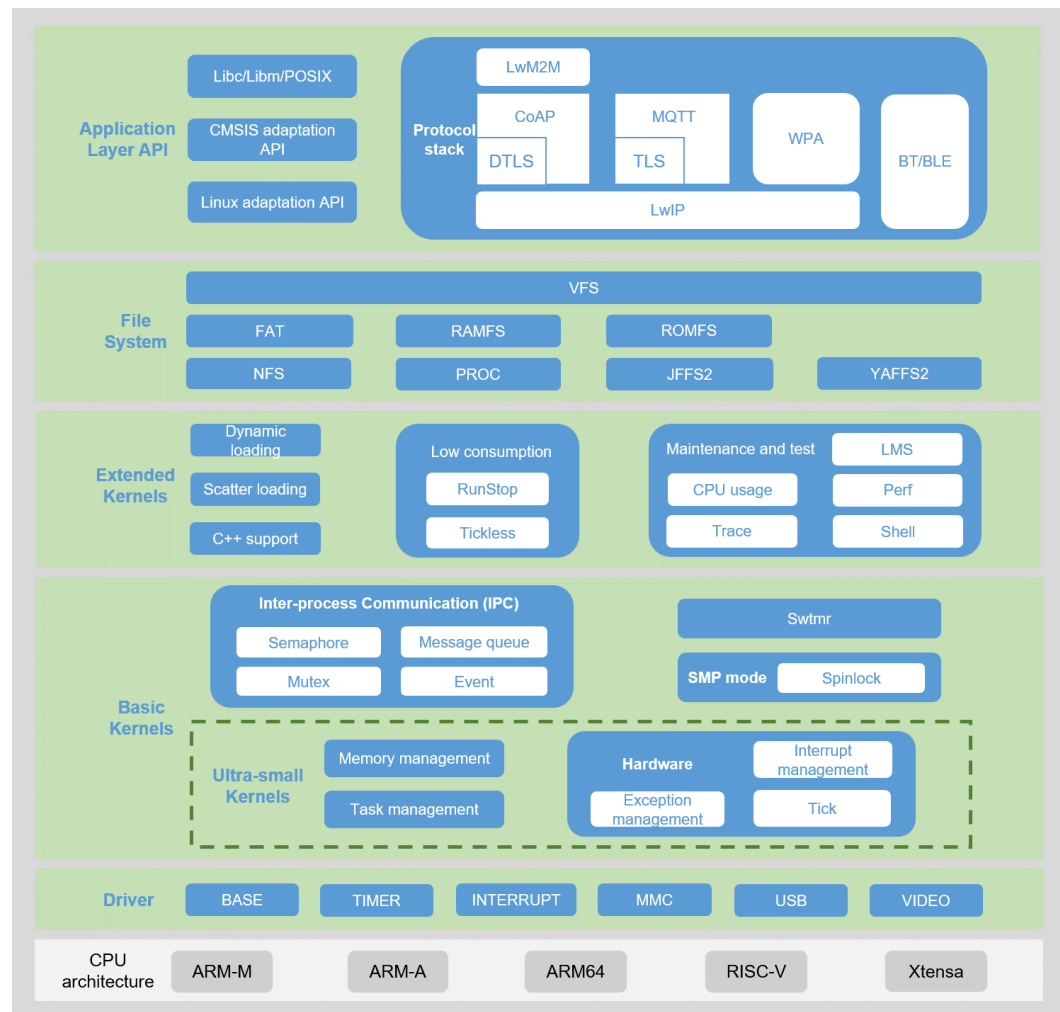
[2.1 Background](#)

[2.2 Supported Cores](#)

[2.3 Constraints](#)

2.1 Background

Huawei LiteOS is a real-time operating system (OS) with key capabilities such as lightweight, low power consumption, quick startup, and rich components.

Figure 2-1 Huawei LiteOS framework

The basic kernel of Huawei LiteOS includes an ultra-small kernel that is not tailorable and some tailorable modules. The ultra-small kernel provides basic functions related to task management and scheduling. The tailorable modules provide the following capabilities: semaphore, mutex, queue management, event management, software timer, memory management, interrupt management, exception management, and system clock tick.

Huawei LiteOS supports the uniprocessor (UP) and symmetric multiprocessor (SMP) modes. That is, Huawei LiteOS can run environments with one or multiple processors.

Highlights of Huawei LiteOS

- High real-time performance and stability
- Low power consumption
- Dynamic and scatter loading
- Static function tailoring
- Quick startup

Modules

Task management

Provides task creation, deletion, delay, suspension, and resumption. Supports locking and unlocking task scheduling. Supports adopting the preemptive scheduling mechanism for tasks with different priorities. Tasks with the same priority are scheduled in a round-robin way using time slices.

Memory management

- Provides static and dynamic memory algorithms and supports memory allocation and release. Currently, it provides the static memory management algorithm BOX as well as dynamic memory management algorithms bestfit and bestfit_little.
- Provides memory statistics and memory overwriting detection.

Hardware-related functions

Provides the following functions:

- Interrupt management: creates, deletes, enables, and disables interrupts; clears interrupt request flags.
- Exception management: prints the information stored in the call stack of the abnormal function or saves the current OS state when exceptions occur at runtime.
- Tick: the basic time unit used in OS scheduling. Duration is measured by the number of ticks per second, which is configurable.

Inter-process communication (IPC)

Provides the following functions:

- Message queue: creates, deletes, reads from, and writes into message queues.
- Event: reads and writes events.
- Semaphore: creates, deletes, allocates, and releases semaphores.
- Mutex: creates, deletes, allocates, and releases mutexes.

Software timer

Creates, deletes, starts, and stops timers.

Spinlock

Initializes, allocates, and releases spin locks in SMP scenarios.

Dynamic loading

Loads and links the object files of a program only when the program is about to run. That is, the object files of programs do not need to be loaded all at once.

Huawei LiteOS supports dynamic loading of OBJ and SO files.

Scatter loading

Preferentially loads key services by means of image reordering to accelerate system startup.

Low power consumption

- Run-stop: also known as hibernation and wakeup. This mechanism saves onsite system images so that the system can be recovered from these images when necessary.
- Tickless: This mechanism calculates the time of the next meaningful clock interrupt to reduce unnecessary clock interrupts, thereby lowering power consumption. If the tickless function is enabled, the system will start the tickless mechanism when CPUs are idle.

Maintenance and test

- CPU usage: provides the CPU usage of the system or any specified task.
- Tracing: obtains the context of events in real time and writes the context into the buffer. You can customize the trace buffer, including designating the modules to trace, starting or stopping tracing, and clearing or outputting data in the buffer.
- Lite memory sanitizer (LMS): checks the validity of memory operations in real time. LMS can detect memory problems such as buffer overflow, use after free, double free, and wild pointer free.
- Perf: analyzes performance to identify performance bottlenecks and locate hotspots in code. Huawei LiteOS allows users to configure the sampling data buffer and sampling events, start or stop the Perf tool, and output sampling data.
- Shell: receives users' commands through a serial port or Telnet, and calls and executes applications through commands. Huawei LiteOS Shell supports common debugging commands, such as system, file, network, and dynamic loading commands. In addition, users can add customized commands.

C++ support

Huawei LiteOS supports some standard template library (STL) features, exceptions, and runtime type identification (RTTI) features. Other features are supported by the compiler.

2.2 Supported Cores

Table 2-1 Cores and chips supported by Huawei LiteOS

Core	Chip
ARM9, Cortex-A7, Cortex-A53, Cortex-A55	Hi3516A, Hi3518EV200, Hi3556, Hi3559A, Hi3516CV300 series, Hi3516EV200 series
Cortex-M7	STM32F769
Cortex-R7, Cortex-R8	Kirin series
Tensilica Xtensa Vision Q6 DSP, HiFi 3z DSP	Kirin series, Hi3751V900
LingLong	Kirin Audio DSP
RISC-V, LinxCore131, LinxCore170	Kirin series, Hi3886, Hi3931, Hi3862, Hi3892, Hi1162, Hi3321

2.3 Constraints

- In addition to its own APIs, Huawei LiteOS supports POSIX and CMSIS APIs, but hybrid use of them may lead to unpredictable results. (For example, a POSIX API is used for allocating semaphores while a Huawei LiteOS API is used for releasing semaphores.)
- Use only Huawei LiteOS APIs for driver development. POSIX APIs are recommended for app development.

3 Basic Kernel

- [3.1 Task](#)
- [3.2 Memory](#)
- [3.3 Interrupt](#)
- [3.4 Exception Management](#)
- [3.5 Error Handling](#)
- [3.6 Queue](#)
- [3.7 Event](#)
- [3.8 Semaphore](#)
- [3.9 Read/Write Semaphore](#)
- [3.10 Mutex](#)
- [3.11 Software Timer](#)
- [3.12 Spinlock](#)
- [3.13 Levels of Initialization](#)
- [3.14 Others](#)

3.1 Task

3.1.1 Overview

Basic Concepts

From the perspective of the operating system, tasks are the minimum running unit that competes for system resources. Tasks can use or wait for system resources such as CPU and memory, and run independently.

The task module in Huawei LiteOS provides multiple tasks for users. Users can switch among these tasks to manage different business processes. The task module in Huawei LiteOS has the following features:

- Multiple tasks are supported.
- Each task represents a thread.
- The preemptive scheduling mechanism is adopted for tasks with different priorities. Specifically, high-priority tasks can interrupt low-priority tasks, and low-priority task are scheduled only high-priority tasks are blocked or completed.
- Tasks with the same priority are scheduled in a round-robin way using time slices.
- A total of 32 priorities are defined, with priority 0 being the highest and 31 being the lowest.
- The attribute of a task can be either detached or joinable. By default, the attribute of a created task is detached.

Related Concepts

Task states

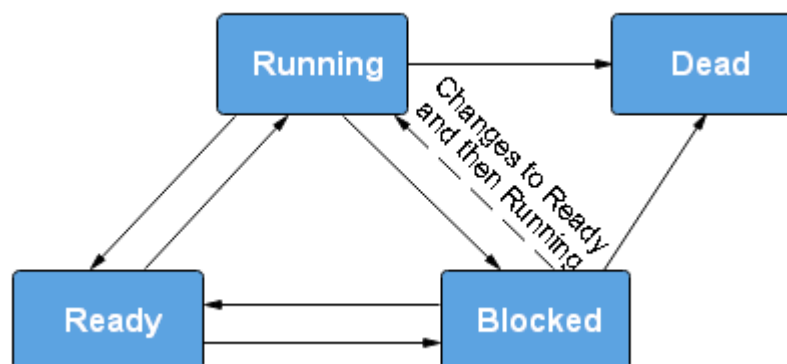
A task in Huawei LiteOS switches between different states. After the operating system is initialized, a created task is allowed to contend for system resources according to the scheduling procedure regulated by Huawei LiteOS Kernel.

There are usually four task states:

- Ready: The task is in the ready queue, waiting for execution by a CPU.
- Running: The task is being executed.
- Blocked: The task is not in the ready queue. The task may be suspended, delayed, waiting for a semaphore, waiting to read from or write to a queue, or reading from or writing to an event.
- Dead: The task execution is complete, and resources are waiting to be reclaimed. Such tasks may be in the zombie or unused state.

Task state transition

Figure 3-1 Task state schematic diagram



The state transition process is as follows:

- **Ready → Running**
A task enters the Ready state once created. When a task switch occurs, the task with the highest priority is selected from the ready queue and enters the Running state to be executed, and is removed from the ready queue.
- **Running → Blocked**
When a running task is blocked (suspended, delayed, or semaphore reading), it is deleted from the ready queue and enter the Blocked state. Then, the task with the highest priority in the ready queue is executed.
- **Blocked → Ready (Blocked → Running)**
After a blocked task is recovered (for example, if the task is resumed, the task successfully reads a semaphore, or if the delay period or semaphore read period expires), the task will be added to the ready queue and enters the Ready state. If the recovered task takes precedence over the running task, a task switch will occur to send the resumed task into the Running state.
- **Ready → Blocked**
When a task in the Ready state is blocked (suspended), the task changes to the Blocked state and is deleted from the ready queue. The blocked task will not be scheduled until it is resumed.
- **Running → Ready**
When a task with a higher priority than the running task is created or resumed, task scheduling occurs. The task with the highest priority in the ready queue enters the Running state. Then, the original running task enters the Ready state and is inserted into the ready queue.
- **Running → Dead**
When a running task is complete, it changes to the Dead state. The Dead state includes normal exit state as the task is complete and the zombie state. For example, if a joinable task is complete but resources are not reclaimed, the task is in zombie state, that is, the Dead state.
- **Blocked → Dead**
If an API is called to delete a blocked task, the task state change from Blocked to Dead.

Task ID

You will receive a task ID after successfully creating a task. All task IDs on the operating system are unique. You may suspend, resume, or query a task using its ID.

Task priority

Tasks are executed based on their priority. In the event of a task switch, the task with the highest priority will be selected from the ready queue.

Task entrypoint function

It refers to the function that will be executed after a new task is scheduled. This function is implemented by users and set in the task creation request.

Task stack

Each task has a separate task stack. The task stack stores information such as local variables, registers, function parameters, and function return addresses.

Task context

All the resources (such as registers) used by a running task are collectively known as the task context. After a task is suspended, other running tasks might modify the context of the suspended task. If the original context of the suspended task is not saved, the suspended task uses the modified context once resumed, incurring unpredictable errors.

Therefore, Huawei LiteOS saves the context information of switched-out tasks into its own task stack so that the context information can be resumed along with tasks and the system can start from the interrupted code after the tasks are resumed.

Task control block

Each task has a task control block (TCB). The TCB contains information such as the task context stack pointer (SP), task status, task priority, task ID, task name, and task stack size, which can reflect the running condition of each task.

Task switching

Task switching may involve obtaining the highest-priority task in the ready queue, saving the context of the switched-out task, and restoring the context of the switched-in task.

Task attributes

The attribute of a task can be either detached or joinable, which determines the termination mode of the task. The joinable attribute indicates a non-separated state. After a task in this state exits, resources are not automatically released. Resources can be released only after other tasks call the `LOS_TaskJoin` API to obtain the return value. The detached attribute indicates a detached state. After a task in this state exits, resources are automatically reclaimed. In addition, waiting for such task is meaningless.

Scheduler

Scheduling Mode

Huawei LiteOS provides global list scheduling mode and multi-list scheduling mode for task scheduling. Two modes have their own advantages in different scenarios. The global list scheduling mode is used by default.

- Global list scheduling mode: Only one priority list is used to store tasks to be executed. All cores adopt the same priority list and all tasks are scheduled in this priority list. Therefore, the global list scheduling mode has high real-time performance.
- Multi-list scheduling mode: This mode is effective only in SMP mode. The system creates a priority list for each core based on the number of cores. Each core schedules their tasks in its own priority list. The system migrates tasks in different priority lists by using the load balancing algorithm. Compared to the global list scheduling mode, this mode effectively reduces frequent inter-core migration.

Task Migration Algorithm

Task migration algorithm can balance priority lists in different cores. The global list scheduling mode has only one priority list, therefore, task migration algorithm

is not required for this mode. However, the task migration algorithm is required for multi-list scheduling mode to migrate tasks in different priority list based on certain rules. Currently, there are two task migration algorithms:

- Simple balancing algorithm: migrates tasks in different priority list to control the difference of waiting tasks number in different cores is within a certain range. This algorithm performs balancing only depend on the tasks number. It features simple load calculation and does not depend on other variables. Therefore, basic load balancing can be implemented.
- CPUP-based balancing algorithm: calculates the CPUP of each core through CPU components, and then migrates tasks based on the calculation result. In this way, the CPUP difference among different cores is within a certain range. This algorithm calculates the CPUP by consuming a few CPU resources and performs balancing based on the calculation result. Therefore, CPUP-based balancing algorithm is better than simple balancing algorithm in terms of the load balancing effect because it is based on the actual CPUP.

Operation Mechanism

When a user creates a task, the system initializes the task stack and presets the context. In addition, the system places the task entrypoint function in the corresponding position so that the function will be executed when the task enters the Running state for the first time.

3.1.2 Development Guidelines

Application Scenario

After a task is created, Huawei LiteOS Kernel can perform operations such as locking/unlocking task scheduling, suspending/resuming/delaying the task, and setting/obtaining the task priority.

Functions

The task management module of Huawei LiteOS provides the following functions. For details about the APIs, see the API reference.

Function Category	API	Description
Task creation and deletion	LOS_TaskCreateOnly	Creates a task and suspends the task without scheduling it. If scheduling is required, call LOS_TaskResume to set the task as Ready.
	LOS_TaskCreate	Creates a task and sets it as Ready. If the ready queue contains no higher-priority tasks, this task is executed.

Function Category	API	Description
	LOS_TaskCreateOnly Static	Creates a task, sets the task stack, and suspends the task without scheduling it. If scheduling is required, call LOS_TaskResume to set the task as Ready.
	LOS_TaskCreateStatic	Creates a task, sets the task stack, and sets the task as Ready. If the ready queue contains no higher-priority tasks, this task is executed.
	LOS_TaskDelete	Deletes a task.
	LOS_TaskJoin	Waits for a specified task in block mode and reclaims task resources.
Task state control	LOS_TaskResume	Resumes a task from suspended to ready.
	LOS_TaskSuspend	Suspends the specified task and performs task switching.
	LOS_TaskDelay	Delays a task and releases CPU resources. After the delay duration elapses, the task will become Ready again.
	LOS_TaskYield	Releases CPU resources and appends a task to a ready queue with the same priority as that task.
	LOS_TaskDetach	Sets the task attribute to detached .
Task scheduling control	LOS_TaskLock	Locks task scheduling, after which tasks can still be interrupted.
	LOS_TaskUnlock	Unlocks task scheduling.
	LOS_TaskUnlockNoSched	Unlocks the task scheduling and continuously executes tasks. (Use this function in special scenarios by referring to the comments to avoid incorrect scheduling time sequence.)
Task priority control	LOS_CurTaskPriSet	Sets the priority of the current task.
	LOS_TaskPriSet	Sets the priority of a particular task.
	LOS_TaskPriGet	Obtains the priority of a particular task.
Task affinity setting	LOS_TaskCpuAffiSet	Sets the CPU collection of a particular task. (This function is available only in SMP mode.)

Function Category	API	Description
Task stack resource reclamation	LOS_TaskResRecycle	Recycles all possible task stack resources.
Task information obtainment	LOS_CurTaskIDGet	Obtains the ID of the current task.
	LOS_TaskInfoGet	Obtains information about a particular task, including its state, priority, total task stack size, stack pointer (SP), entrypoint function, and used task stack size.
	LOS_TaskCpuAffiGet	Obtains the CPU collection of a particular task. (This function is available only in SMP mode.)
	LOS_TaskIsScheduled	Queries whether the current core has started task scheduling.
Task information maintenance and test	LOS_TaskSwitchHookReg	Registers a hook function for switching task context. This hook function is called upon task context switching only when the LOSCFG_BASE_CORE_TSK_MONITOR macro has been enabled.
Idle task callback	LOS_IdleHandlerHookReg	Registers a hook function for an idle task. This function is called when the system is idle.
Task security stack allocation	LOS_TaskAllocSecureContext	Allocates security stack of a task. This function is available only when the LOSCFG_TRUSTZONE macro has been enabled.

NOTICE

- Open the menu (by referring to [8.4 menuconfig](#)) and choose **Kernel > Enable Kernel SMP** to configure **LOSCFG_KERNEL_SMP** to enable the SMP mode. In the SMP submenu, you can set the number of cores, enable inter-core synchronization for tasks, and enable cross-core function calling.
- In SMP mode, **usCpuAffiMask** can be set during task creation to specify the CPU affinity of the task. This flag bit uses the "1 bit, 1 core" paradigm. For details, see the TSK_INIT_PARAM_S structure.
- The task stack size can be set during task creation. If it is set to **0**, the default size (**LOSCFG_BASE_CORE_TSK_DEFAULT_STACK_SIZE**) is used.

Task State

In Huawei LiteOS, most task states are defined by the kernel. Only Detached state can be defined by users. Users need to define Detached state during task creation.

Definition	Error Code	Description
LOS_TASK_STATUS_JOINABLE	0x0080	The task does not automatically release resources. Resources can be released only after other tasks obtain the return value.
LOS_TASK_STATUS_DETACHED	0x0100	The task is detached.

When calling the API for creating a task, you can set the **uwResved** field of the **TSK_INIT_PARAM_S** parameter to **LOS_TASK_STATUS_DETACHED** (detached) or **LOS_TASK_STATUS_JOINABLE** (joinable).

NOTICE

LOSCFG_TASK_JOINABLE determines the joinable attribute of a task.

- Open the menu (by referring to [8.4 menuconfig](#)) and choose **Kernel > Basic Config > Task > Enable Join/Detach mechanism** to configure **LOSCFG_TASK_JOINABLE**.
- If **LOSCFG_TASK_JOINABLE** is enabled, the joinable attribute of a task can be set only when the task state is set to **LOS_TASK_STATUS_JOINABLE**. The joinable task cannot be automatically deleted. You need to call **LOS_TaskJoin** to reclaim resources of the joinable task. You can also call the **LOS_TaskDetach** API to set the task attribute to **detached**. After the detached task is complete, resources are automatically reclaimed.
- If **LOSCFG_TASK_JOINABLE** is disabled, the task has only the detached attribute no matter whether the **uwResved** field of the **TSK_INIT_PARAM_S** parameter is set to **LOS_TASK_STATUS_JOINABLE**.
- Instead of the **LOS_TaskDelete** API, only the **LOS_TaskJoin** API can reclaim resources of a joinable task.

Task Priority

The priority range is (**LOS_TASK_PRIORITY_HIGHEST** - **LOS_TASK_PRIORITY_LOWEST**). A smaller value indicates a higher priority.

Definition	Error Code	Description
LOS_TASK_PRIORITY_HIGHEST	0	Highest priority of a task

Definition	Error Code	Description
LOS_TASK_PRIORITY_LOWEST	31	Lowest priority of a task

Task Error Codes

An error code is returned when a task fails to be created, deleted, suspended, resumed, or delayed. The error code usually suggests the possible cause of the failure.

N o.	Definition	Error Code	Description	Reference Solution
1	LOS_ERRNO_TASK_NO_MEMORY	0x03000200	Memory space is insufficient.	Expand the dynamic memory space in either of the following ways: - Set a larger system dynamic memory pool. The configuration item is OS_SYS_MEM_SIZE. - Release a part of the dynamic memory. If the error occurs in task initialization during LiteOS startup, reduce the maximum number of tasks supported by the system to solve the problem. If the error occurs during task creation, reduce the size of the task stack to solve the problem.
2	LOS_ERRNO_TASK_PTR_NULL	0x02000201	The initParam parameter transferred to the task creation API or a parameter transferred to the task information obtainment API is a null pointer.	Ensure that the input parameter is not a null pointer.
3	LOS_ERRNO_TASK_STKSZ_NOT_ALIGN	0x02000202	This error code is not used currently.	-

N o.	Definition	Error Code	Description	Reference Solution
4	LOS_ERRNO_TASK_PRIOR_ERROR	0x02000203	An incorrect priority is used during task creation or priority setting.	Ensure that the task priority ranges from 0 to 31.
5	LOS_ERRNO_TASK_ENTRY_NULL	0x02000204	The task entrypoint function set during task creation is a null pointer.	Specify the task entrypoint function.
6	LOS_ERRNO_TASK_NAME_EMPTY	0x02000205	The task name set during task creation is a null pointer.	Specify the task name.
7	LOS_ERRNO_TASK_STKSZ_TOO_SMALL	0x02000206	A small stack size is set during task creation.	Increase the size of the task stack to the minimum limit (LOS_TASK_MIN_STACK_SIZE) or above.
8	LOS_ERRNO_TASK_ID_INVALID	0x02000207	The task ID is not supported by the OS.	Check the task ID.
9	LOS_ERRNO_TASK_ALREADY_SUSPENDED	0x02000208	The task to be suspended is already suspended.	Suspend the task after it is resumed.
10	LOS_ERRNO_TASK_NOT_SUSPENDED	0x02000209	The task to be resumed is not suspended.	Suspend the task and then resume it.
11	LOS_ERRNO_TASK_NOT_CREATED	0x0200020a	No task is created.	Create the task. This error may occur during the following operations: • Deleting a task • Resuming/ Suspending a task • Setting the priority of a particular task • Obtaining information about a particular task • Setting the CPU collection of a particular task
12	LOS_ERRNO_TASK_DELETE_LOCKED	0x0300020b	The task to be deleted is locked.	Unlock the task and then delete it.

N o.	Definition	Error Code	Description	Reference Solution
13	LOS_ERRNO_TASK_DELAY_INTERRUPT	0x0300020d	The task to be delayed is under an interrupt.	Delay the task after it exits the interrupt.
14	LOS_ERRNO_TASK_DELAY_IN_LOCK	0x0200020e	The task to be delayed is locked.	Unlock the task and then delay it.
15	LOS_ERRNO_TASK_YIELD_IN_LOCK	0x0200020f	Failed to yield the task when the task is locked.	Unlock the task and then yield it.
16	LOS_ERRNO_TASK_YIELD_NOT_ENOUGH_TASK	0x02000210	No task is found in the ready queue with the same priority as the task to be yielded.	Add tasks with the same priority as the current task.
17	LOS_ERRNO_TASK_TCB_UNAVAILABLE	0x02000211	No idle TCB is available during task creation.	Call the LOS_TaskResRecyle API to recycle idle TCBs. If the failure persists, add TCBs in the system.
18	LOS_ERRNO_TASK_OPERATE_SYSTEM_TASK	0x02000214	It is forbidden to delete, suspend, or delay system-level tasks (such as idle tasks and software timer tasks), or modify the priorities of system-level tasks.	Check the task ID and do not operate system-level tasks.
19	LOS_ERRNO_TASK_SUSPEND_LOCKED	0x03000215	It is forbidden to suspend a locked task.	Unlock the task and then suspend it.
20	LOS_ERRNO_TASK_STKSZ_TOO_LARGE	0x02000220	A large stack size is set during task creation.	Reduce the size of the task stack.
21	LOS_ERRNO_TASK_CPU_AFFINITY_MASK_ERR	0x03000223	An incorrect CPU collection is set for the task.	Check the CPU mask.
22	LOS_ERRNO_TASK_YIELD_IN_INTERRUPT	0x02000224	It is forbidden to yield a task in an interrupt.	Yield the task after it exits the interrupt.

N o.	Definition	Error Code	Description	Reference Solution
23	LOS_ERRNO_TS K_MP_SYNC_RE SOURCE	0x0200 0225	Resource allocation fails during cross-core task deletion.	Set LOSCFG_BASE_IPC_SEM_LIMIT to a larger value to add semaphores.
24	LOS_ERRNO_TS K_MP_SYNC_FAI LED	0x0200 0226	A task is not promptly deleted during cross-core task deletion.	Check whether the target task frequently switches among different states. Such tasks cannot be deleted within the specified time.
25	LOS_ERRNO_TS K_ALLOC_SECU RE_INT	0x0200 0227	It is forbidden to allocate a task security stack in an interrupt.	Do not allocate a task security stack in an interrupt.
26	LOS_ERRNO_TS K_SECURE_ALR EADY_ALLOC	0x0200 0228	The security stack of the task has been allocated.	Do not repeatedly allocate a security stack for the same task.
27	LOS_ERRNO_TS K_ALLOC_SECU RE_FAILED	0x0200 0229	Security stack allocation is failed.	Release some memory of the TEE. The memory in the TEE is insufficient.
28	LOS_ERRNO_TS K_FREE_SECU RE_FAILED	0x0200 022a	Failed to release the security stack.	This error code is not in use.
29	LOS_ERRNO_TS K_NOT_ALLOW _IN_INT	0x0200 022b	This API is not allowed in the interrupt context.	Do not call the corresponding API in the interrupt.
30	LOS_ERRNO_TS K_SCHED_LOCK ED	0x0200 022c	This API may be blocked and scheduling cannot be disabled.	Check the scheduling lock logic.
31	LOS_ERRNO_TS K_ALREADY_JOI N	0x0200 022d	A joinable task cannot be waited by multiple tasks at the same time.	Check whether other tasks are waiting.
32	LOS_ERRNO_TS K_IS_DETACHE D	0x0200 022e	The attribute of the current task is set to detached .	Check whether a task with the detached attribute is joined or whether a task is repeatedly set to the detached attribute.

N o.	Definition	Error Code	Description	Reference Solution
33	LOS_ERRNO_TASK_NOT_JOIN_SELF	0x0200022f	The task cannot join itself.	Check whether the value of taskid is correct.
34	LOS_ERRNO_TASK_IS_ZOMBIE	0x02000230	The task is in the zombie state.	Check whether a task in the zombie state is deleted, whether its priority, attribute, and CPU affinity are set, or whether it is suspended or resumed.

NOTICE

- For the definition of error codes, see [Error Codes](#). Bits 8–15 belong to the task module, and the value is **0x02**.
- The error codes 0x16 and 0x1c in the task module are not defined and cannot be used.

Development Procedure

Task creation is used as an example to explain the development process.

- Open the menu and choose **Kernel > Basic Config > Task** to configure the task module.

Configuration Item	Description	Value Range	Default Value	Dependency
LOSCFG_BASE_CONFIG_TSK_LIMIT	Maximum number of tasks supported by the system.	(0,256]	The default value varies depending on the platform.	No.

Configuration Item	Description	Value Range	Default Value	Dependency
LOSCFG_BASE_CORE_TSK_MIN_STACK_SIZE	Minimum size of the task stack. In most cases, retain the default value.	(0, OS_SYS_MEM_SIZE)	The default value varies depending on the platform.	No ne.
LOSCFG_BASE_CORE_TSK_DEFAULT_STACK_SIZE	Default size of the task stack.	(0, OS_SYS_MEM_SIZE)	The default value varies depending on the platform.	No ne.
LOSCFG_BASE_CORE_TSK_IDLE_STACK_SIZE	Size of the idle task stack. In most cases, retain the default value.	(0, OS_SYS_MEM_SIZE)	The default value varies depending on the platform.	No ne.
LOSCFG_BASE_CORE_TSK_DEFAULT_PRIO	Default task priority. In most cases, retain the default value.	[0,31]	10	No ne.
LOSCFG_BASE_CORE_TIMESLICE	Whether to schedule the task based on time slices.	YES/NO	YES	No ne.

Configuration Item	Description	Value Range	Default Value	Dependency
LOSCFG_BASE_CORE_TIMESLICE_TIMEOUT	Maximum execution duration of tasks with the same priority. (unit: tick)	(0,65535]	The default value varies depending on the platform.	None.
LOSCFG_OBSOLETE_API	Whether to use the old task parameter UINTPTR auwArgs[4] instead of the new VOID *pArgs . You are advised to disable this function and use the new task parameter.	YES/NO	The default value varies depending on the platform.	None.
LOSCFG_BASE_CORE_TSK_MONITOR	Whether to monitor task stack overflows and tracks.	YES/NO	YES	None.
LOSCFG_TASK_STACK_STATIC_ALLOCATION	Whether to allow users to input the task stack during task creation.	YES/NO	NO	None.
LOSCFG_TASK_JOINABLE	Enables the kernel join/detach mechanism.	YES/NO	NO	None.

- Open the menu and choose **Kernel > Basic Config > Enable Kernel SMP** to enable and configure the SMP mode.

Configuration Item	Description	Value Range	Default Value	Dependency
LOSCFG_KERNEL_SMP_CORE_NUM	Number of cores used by the system.	[2,4]	2	LOSCFG_KERNEL_SMP

Configuration Item	Description	Value Range	Default Value	Dependency
LOSCFG_KERNEL_SMP_TASK_SYNC	Whether to enable inter-core synchronization for multiple tasks.	YES/NO	NO	LOSCFG_KERNEL_SMP
LOSCFG_KERNEL_SMP_CALL	Whether to enable cross-core function calling.	YES/NO	NO	LOSCFG_KERNEL_SMP
LOSCFG_SMP_CALL_STATIC_ALLOCATION	Whether to create function call objects using static allocation instead of dynamic allocation.	YES/NO	NO	LOSCFG_KERNEL_SMP_CALL
LOSCFG_SMP_CALL_OBJ_NUM	Number of function call objects created by using static allocation.	[1,655 35]	16	LOSCFG_SMP_CALL_STATIC_ALLOCATION
LOSCFG_KERNEL_SMP_GC_IDLE	Whether to use the idle task to recycle the task to be deleted.	YES/NO	NO	LOSCFG_KERNEL_SMP
LOSCFG_KERNEL_SMP_GC_SWTMR	Whether to use SWTMR to recycle tasks to be deleted.	YES/NO	YES	LOSCFG_KERNEL_SMP

- Open the menu and choose **Kernel > Basic Config > Enable Scheduler** to configure task scheduling.

Configuration Item	Description	Value Range	Default Value	Dependency
LOSCFG_SCHED_SQ	Whether to enable the global list scheduling mode.	YES/NO	YES	None.
LOSCFG_SCHED_MQ	Whether to enable the multi-list scheduling mode.	YES/NO	NO	LOSCFG_KERNEL_SMP
LOSCFG_SCHED_LOAD_BALANCE_SIMPLE	Whether to use simple balancing algorithm.	YES/NO	NO	LOSCFG_SCHED_MQ

Configuration Item	Description	Value Range	Default Value	Dependency
LOSCFG_SCHED_LOAD_BALANCE_CPUP	Whether to use CPUP-based balancing algorithm.	YES/NO	NO	LOSCFG_SCHE D_MQ LOSCFG_KERNEL_CPUP

4. Call the `LOS_TaskLock` API to lock task scheduling so that high-priority tasks will not be scheduled first.
5. Call the `LOS_TaskCreate` or `LOS_TaskCreateStatic` API to create a task. (The **LOSCFG_TASK_STACK_STATIC_ALLOCATION** macro must be enabled first.)
6. Call the `LOS_TaskUnlock` API to unlock task scheduling so that tasks will be scheduled by priority.
7. Call the `LOS_TaskDelay` API to delay the task.
8. Call the `LOS_TaskSuspend` API to suspend the task.
9. Call the `LOS_TaskResume` API to resume the suspended task.

Platform Differences

None.

3.1.3 Precautions

Task creation

- A task name is a pointer without memory space. Do not set a task name to the address of a local variable.
- When a task is created, the memory is allocated to the task stack based on the 16-byte size or `sizeof(UINTPTR) * 2` alignment mode. Follow the "nothing more and nothing less" principle while determining the task stack size.

Lock task

- Lock task scheduling does not disable interrupts. Therefore, tasks can still be interrupted.
- Lock task scheduling must be used together with unlock task scheduling.

Task priority

- Task scheduling may occur when you set the task priority.
- The `LOS_CurTaskPriSet` and `LOS_TaskPriSet` APIs cannot be used in interrupts and cannot be used to modify the priorities of software timer tasks.
- If the task corresponding to the task ID sent by the `LOS_TaskPriGet` API has not been created or exceeds the maximum number of tasks, 0xffff will be returned.

Forbidden operations

- If a task is locked, it cannot be suspended.
- Idle and software timer tasks must not be suspended or deleted.
- LOS_TaskDelay fails to be executed in the interrupt processing function or in the case of a lock task.

Task resources

- Running idle tasks reclaims the TCBs and stacks of previously deleted tasks.
- The maximum number of tasks that can be configured in the system refers to the total number of tasks in the entire system instead of the number of tasks that can be used by users. For example, if the system software timer occupies one more task resource, the number of available task resources decreases by one.
- When deleting a task, ensure that the resources (such as mutexes and semaphores) allocated by the task have been released.

SMP mode

- In SMP mode, the lock task scheduling can only be used to lock the scheduler of the current core. Tasks on other cores can still be scheduled properly.
- In SMP mode, tasks are deleted or suspended in asynchronous mode. Therefore, the return value of an operation does not indicate the final result but only indicates that the request has been sent. Additionally, there is a delay in completing the preceding operation.
- In SMP mode, if the cross-core deletion synchronization function (LOSCFG_KERNEL_SMP_TASK_SYNC option) is enabled, the result is returned only after the target task is deleted when the cross-core deletion task is executed. If the task is not deleted within the specified time, the error LOS_ERRNO_TSK_MP_SYNC_FAILED is returned. After this function is enabled, each task increases the overhead of one semaphore.
- In SMP mode, cross-core tasks are deleted in asynchronous mode, and some tasks are reclaimed by the reclamation mechanism. Set the LOSCFG_KERNEL_SMP_GC_IDLE and LOSCFG_KERNEL_SMP_GC_SWTMR to choose execution mode of the recycling mechanism. However, as the idle task has a low priority, if the CPU is always busy, the task may fail to be recycled. Therefore, exercise caution when choosing the idle task recycling mode.

3.1.4 Programming Example

Example Description

There are some examples giving some basic insight into priority-based task scheduling and use cases of APIs, including creating, delaying, suspending, and resuming two tasks with different priorities, and locking/unlocking task scheduling.

Example Code

Prerequisite: The task module has been configured in the **menuconfig** menu.

```
UINT32 g_taskHild;  
UINT32 g_taskLold;  
#define TSK_PRIOR_HI 4
```



```
#define TSK_PRIOR_LO 5

UINT32 Example_TaskHi(VOID)
{
    UINT32 ret;

    printf("Enter TaskHi Handler.\r\n");

    /* Delay TaskHi for 2 ticks. TaskHi is then suspended and the task (g_taskLoid) with the highest priority
    among the rest is executed first. */
    ret = LOS_TaskDelay(2);
    if (ret != LOS_OK) {
        printf("Delay Task Failed.\r\n");
        return LOS_NOK;
    }

    /* After 2 ticks elapse, TaskHi is resumed. */
    printf("TaskHi LOS_TaskDelay Done.\r\n");

    /* Suspend the task. */
    ret = LOS_TaskSuspend(g_taskHild);
    if (ret != LOS_OK) {
        printf("Suspend TaskHi Failed.\r\n");
        return LOS_NOK;
    }
    printf("TaskHi LOS_TaskResume Success.\r\n");

    return ret;
}

/* Task entrypoint function for the TaskLo. */
UINT32 Example_TaskLo(VOID)
{
    UINT32 ret;

    printf("Enter TaskLo Handler.\r\n");

    /* Delay TaskLo for 2 ticks. TaskLo is then suspended and the task (background task) with the highest
    priority among the rest is executed first. */
    ret = LOS_TaskDelay(2);
    if (ret != LOS_OK) {
        printf("Delay TaskLo Failed.\r\n");
        return LOS_NOK;
    }

    printf("TaskHi LOS_TaskSuspend Success.\r\n");

    /* Resume the suspended g_taskHild task. */
    ret = LOS_TaskResume(g_taskHild);
    if (ret != LOS_OK) {
        printf("Resume TaskHi Failed.\r\n");
        return LOS_NOK;
    }

    printf("TaskHi LOS_TaskDelete Success.\r\n");

    return ret;
}

/* Task test entrypoint function. Two tasks with different priorities are created. */
UINT32 Example_TskCaseEntry(VOID)
{
    UINT32 ret;
    TSK_INIT_PARAM_S initParam;

    /* Lock task scheduling to prevent higher-priority tasks from being scheduled prior to this task. */
    LOS_TaskLock();

    printf("LOS_TaskLock() Success!\r\n");
```

```

    initParam.pfnTaskEntry = (TSK_ENTRY_FUNC)Example_TaskHi;
    initParam.usTaskPrio = TSK_PRIOR_HI;
    initParam.pcName = "TaskHi";
    initParam.uwStackSize = LOSCFG_BASE_CORE_TSK_MIN_STACK_SIZE;
    initParam.uwResved = LOS_TASK_STATUS_DETACHED;
    /* Create a high-priority task. The task will not be executed immediately after being created, because
task scheduling is locked. */
    ret = LOS_TaskCreate(&g_taskHild, &initParam);
    if (ret != LOS_OK) {
        LOS_TaskUnlock();

        printf("Example_TaskHi create Failed!\r\n");
        return LOS_NOK;
    }

    printf("Example_TaskHi create Success!\r\n");

    initParam.pfnTaskEntry = (TSK_ENTRY_FUNC)Example_TaskLo;
    initParam.usTaskPrio = TSK_PRIOR_LO;
    initParam.pcName = "TaskLo";
    initParam.uwStackSize = LOSCFG_BASE_CORE_TSK_MIN_STACK_SIZE;
    initParam.uwResved = LOS_TASK_STATUS_DETACHED;

    /* Create a low-priority task. The task will not be executed immediately after being created, because task
scheduling is locked. */
    ret = LOS_TaskCreate(&g_taskLold, &initParam);
    if (ret != LOS_OK) {
        LOS_TaskUnlock();

        printf("Example_TaskLo create Failed!\r\n");
        return LOS_NOK;
    }

    printf("Example_TaskLo create Success!\r\n");

    /* Unlock task scheduling. The task with the highest priority in the Ready list is executed. */
    LOS_TaskUnlock();

    return LOS_OK;
}

```

Verification

The verification result is as follows:

```

LOS_TaskLock() Success!
Example_TaskHi create Success!
Example_TaskLo create Success!
Enter TaskHi Handler.
Enter TaskLo Handler.
TaskHi LOS_TaskDelay Done.
TaskHi LOS_TaskSuspend Success.
TaskHi LOS_TaskResume Success.
TaskHi LOS_TaskDelete Success.

```

Complete Code

sample_task.zip

3.1.5 Programming Example (SMP)

Example Description

There are some examples giving some basic insight into priority-based task scheduling and use cases of APIs, including creating, delaying, suspending, and resuming tasks, and locking/unlocking task scheduling.

1. Two tasks will be created: TaskHi and TaskLo.
2. TaskHi has a high priority and is bound to the CPU of the current test task.
3. TaskLo has a low priority and its CPU affinity is not configured.

NOTICE

Because affinity is not configured for TaskLo, the operation that calls `LOS_TaskLock` will fail, and the "Example_TaskLo create Success!" and "Enter TaskLo Handler." statements are not printed in a strict sequence.

Example Code

Prerequisite: The task module has been configured and SMP mode has been enabled in the **menuconfig** menu.

```
UINT32 g_taskLoid;
UINT32 g_taskHild;
#define TSK_PRIOR_HI 4
#define TSK_PRIOR_LO 5

UINT32 Example_TaskHi(VOID)
{
    UINT32 ret;

    printf("[cpu%d] Enter TaskHi Handler.\r\n", ArchCurrCpuid());

    /* Delay TaskHi for 2 ticks. TaskHi is then suspended and the task (g_taskLoid) with the highest priority
among the rest is executed first. */
    ret = LOS_TaskDelay(2);
    if (ret != LOS_OK) {
        printf("Delay Task Failed.\r\n");
        return LOS_NOK;
    }

    /* After 2 ticks elapse, TaskHi is resumed. */
    printf("TaskHi LOS_TaskDelay Done.\r\n");

    /* Suspend the task. */
    ret = LOS_TaskSuspend(g_taskHild);
    if (ret != LOS_OK) {
        printf("Suspend TaskHi Failed.\r\n");
        return LOS_NOK;
    }
    printf("TaskHi LOS_TaskResume Success.\r\n");
    return ret;
}

/* Task entrypoint function for the TaskLo */
UINT32 Example_TaskLo(VOID)
{
    UINT32 ret;
```

```

printf("[cpu%d] Enter TaskLo Handler.\r\n", ArchCurrCpuId());

/* Delay TaskLo for 2 ticks. TaskLo is then suspended and the task (background task) with the highest
priority among the rest is executed first. */
ret = LOS_TaskDelay(2);
if (ret != LOS_OK) {
    printf("Delay TaskLo Failed.\r\n");
    return LOS_NOK;
}

printf("TaskHi LOS_TaskDelete Success.\r\n");
return ret;
}

/* Task test entryptpoint function. Two tasks with different priorities are created. */
UINT32 Example_TskCaseEntry(VOID)
{
    UINT32 ret;
    TSK_INIT_PARAM_S initParam = {0};

    /* Lock task scheduling. */
    LOS_TaskLock();

    printf("LOS_TaskLock() Success on cpu%d!\r\n", ArchCurrCpuId());

    initParam.pfnTaskEntry = (TSK_ENTRY_FUNC)Example_TaskHi;
    initParam.usTaskPrio = TSK_PRIOR_HI;
    initParam.pcName = "TaskHi";
    initParam.uwStackSize = LOSCFG_BASE_CORE_TSK_MIN_STACK_SIZE;
    initParam.uwResved = LOS_TASK_STATUS_DETACHED;
#ifdef LOSCFG_KERNEL_SMP
    /* Bind high-priority tasks to CPU1 for execution. */
    initParam.usCpuAffiMask = CPUID_TO_AFFI_MASK(ArchCurrCpuId());
#endif
    /* Create a high-priority task. The task will not be executed immediately after being created, because the
scheduler of CPU1 is locked. */
    ret = LOS_TaskCreate(&g_taskHild, &initParam);
    if (ret != LOS_OK) {
        LOS_TaskUnlock();

        printf("Example_TaskHi create Failed!\r\n");
        return LOS_NOK;
    }

    printf("Example_TaskHi create Success!\r\n");

    initParam.pfnTaskEntry = (TSK_ENTRY_FUNC)Example_TaskLo;
    initParam.usTaskPrio = TSK_PRIOR_LO;
    initParam.pcName = "TaskLo";
    initParam.uwStackSize = LOSCFG_BASE_CORE_TSK_MIN_STACK_SIZE;
    initParam.uwResved = LOS_TASK_STATUS_DETACHED;
#ifdef LOSCFG_KERNEL_SMP
    /* Do not set CPU affinity for low-priority tasks. */
    initParam.usCpuAffiMask = 0;
#endif
    /* Create a low-priority task. The task is executed immediately on other CPUs after being created,
because the task is not bound to the current CPU though task scheduling is locked. */
    ret = LOS_TaskCreate(&g_taskLold, &initParam);
    if (ret != LOS_OK) {
        LOS_TaskUnlock();

        printf("Example_TaskLo create Failed!\r\n");
        return LOS_NOK;
    }

    printf("Example_TaskLo create Success!\r\n");

    /* Unlock task scheduling. The task with the highest priority in the Ready list is executed. */
    LOS_TaskUnlock();
}

```

```
    return LOS_OK;  
}
```

Verification

The verification result is as follows:

```
LOS_TaskLock() success on cpu1!  
Example_TaskHi create Success!  
Example_TaskLo create Success!  
[cpu2] Enter TaskLo Handler.  
[cpu1] Enter TaskHi Handler.  
TaskHi LOS_TaskDelete Success.  
TaskHi LOS_TaskDelay Done.
```

Complete Code

sample_task_smp.zip

3.2 Memory

3.2.1 Overview

Basic Concepts

Memory management, one of the core modules of the OS, manages the memory resources of the system. Memory management primarily involves initializing, allocating, and releasing memory.

While the operating system is running, the memory management module manages memory usage of users and the operating system by allocating and releasing memory. This helps reduce memory fragments as much as possible.

In Huawei LiteOS, memory management is divided into static and dynamic memory management, and provides functions such as memory initialization, allocation, and release.

- Dynamic memory: allocates memory blocks of a specified size in the dynamic memory pool. The bestfit (also called dlink) and bestfit_little memory management algorithms are supported.
 - Advantage: on-demand memory allocation
 - Disadvantage: risk of memory fragments
- Static memory: a memory block whose size is predefined at the time of initialization
 - Advantages: no memory fragments; efficient memory allocation and release
 - Disadvantage: memory cannot be allocated on demand

Dynamic Memory Operation Mechanism — bestfit

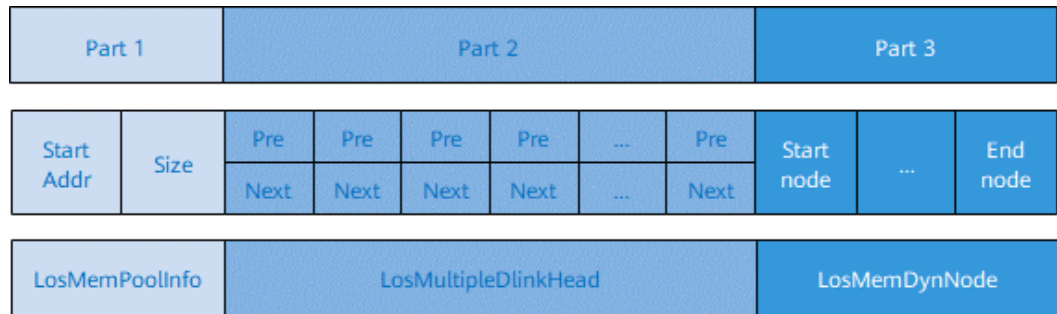
When memory resources are sufficient, memory blocks of any size are allocated from a large contiguous memory (memory pool or heap memory) configured in

the system based on user requirements. When users do not need a memory block, the memory block can be released back to the system for next use.

Comparing with static memory, the advantage of dynamic memory is on-demand allocation and its disadvantage is fragmentation.

The following figure shows the structure of bestfit.

Figure 3-2 Structure of bestfit

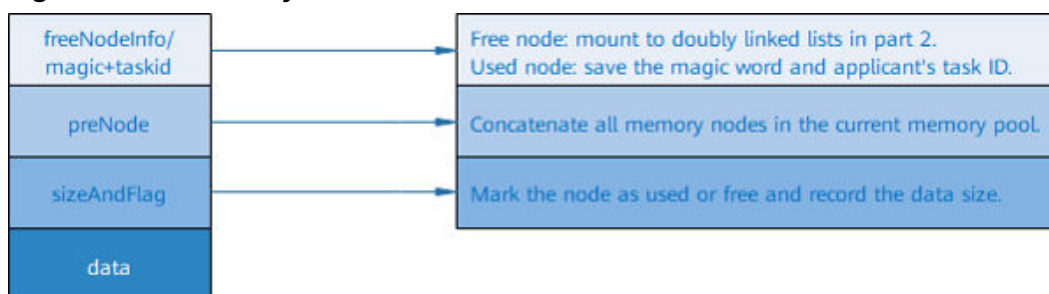
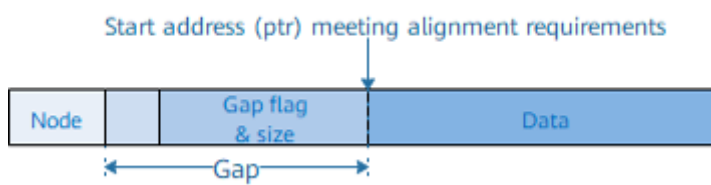


- Part 1**
 Indicates the start address and size of the heap memory (memory pool).
- Part 2**
 Indicates an array of which each element is a doubly linked list. Control headers of all free nodes are categorized and mounted to the doubly linked lists in this array.

 If the smallest node allowed by the memory is $2^{\min}$ bytes, the first doubly linked list in the array stores free nodes with $2^{\min} \leq \text{size} < 2^{\min+1}$. The second doubly linked list in the array stores free nodes with $2^{\min+1} \leq \text{size} < 2^{\min+2}$. The nth doubly linked list in the array stores free nodes with $2^{\min+n-1} \leq \text{size} < 2^{\min+n}$. When memory is allocated, a free node of appropriate size (the size of the node being created) is located and memory is allocated to the free node. When memory is released, the memory is stored to the array as free nodes for later use.
- Part 3**
 Uses most space in the memory pool and is the actual area that stores nodes. The LosMemDynNode structure is described as follows:

```
typedef struct {
    union {
        LOS_DL_LIST freeNodeInfo;    /* Free memory node */
        struct {
            UINT32 magic;
            UINT32 taskId : 16;
        };
    };
    struct tagLosMemDynNode *preNode;
    UINT32 sizeAndFlag;
} LosMemCtlNode;

typedef struct tagLosMemDynNode {
    LosMemCtlNode selfNode;
} LosMemDynNode;
```

Figure 3-3 LosMemDynNode structure**Figure 3-4** Memory allocation in alignment mode

If the start address of the data space contained in the allocated node does not meet alignment requirements, increase the gap to ensure that the returned pointer meets alignment requirements.

Dynamic Memory Operation Mechanism — `bestfit_little`

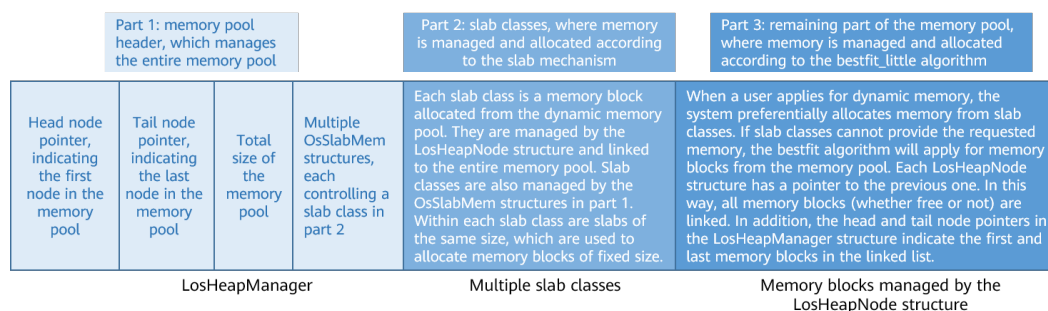
The `bestfit_little` algorithm is the result of integrating the slab mechanism into the `bestfit` algorithm. The `bestfit` algorithm is used to allocate the smallest but most appropriate memory block in the memory pool each time. The slab mechanism is used to allocate memory blocks of fixed size to reduce the possibility of fragmentation.

For the slab mechanism in Huawei LiteOS, users can configure the number of slab classes and the maximum space of each class.

Assume that there are four slab classes in the memory pool and the maximum space of each class is 512 bytes. These slab classes are allocated by the `bestfit` algorithm from the memory pool. The first slab class is divided into thirty-two 16-byte slabs, the second sixteen 32-byte slabs, the third eight 64-byte slabs, and the fourth four 128-byte slabs.

During initialization, the memory pool is initialized first. Then the `bestfit` algorithm allocates four slab classes from the initialized memory pool, and the slab mechanism initializes the four classes one by one.

During memory allocation, memory is allocated from the slab class of the most appropriate size. For example, if a user allocates 20-byte memory, the entire 32-byte slab will be allocated to the user. When the user no longer needs the memory, the entire slab will be reclaimed. However, if no memory block is available in that slab class, instead of allocating memory blocks from bigger slab classes, the `bestfit` algorithm will allocate memory blocks from the memory pool. Before releasing the memory, the system checks whether the released memory block belongs to the slab class. If yes, the memory block is returned to the slab class. If no, the memory block is returned to the memory pool.

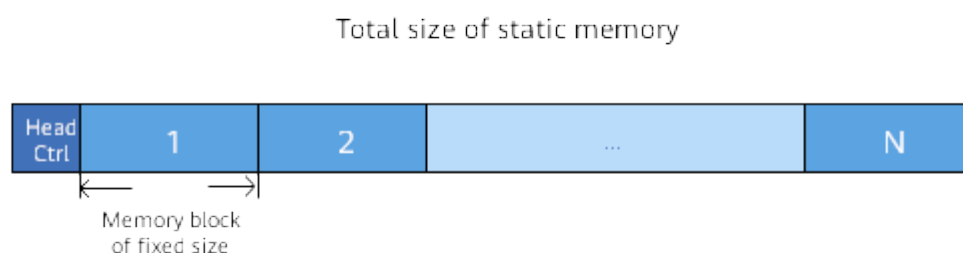


Static Memory Operation Mechanism

Static memory is in essence a static array. The size of a static memory block is defined at the time of initialization and cannot be changed since then.

A static memory pool consists of a control block and several memory blocks of same size. The control block is placed at the head of the static memory pool to manage memory blocks. The allocation and release of memory block according to the size of the blocks.

Figure 3-5 Static memory



3.2.2 Dynamic Memory

3.2.2.1 Development Guidelines

Application Scenario

The main task of memory management is to dynamically partition and manage user allocated memory intervals.

Dynamic memory management is used when users have different demands on memory blocks. When a user needs memory, the system calls `LOS_MemAlloc` to allocate the required amount of memory. When the user no longer needs the memory, the system calls `LOS_MemFree` to release it.

Functions

The dynamic memory management module of Huawei LiteOS provides the following functions. For details about the APIs, see the API reference.

Function Category	API	Description
Memory pool initialization and deletion	LOS_MemInit	Initializes a dynamic memory pool of the required size.
	LOS_MemDelnit	Deletes a memory pool. This function takes effect only when LOSCFG_MEM_MUL_POOL is enabled.
Dynamic memory allocation and release	LOS_MemAlloc	Allocates a memory block of the required size from a dynamic memory pool.
	LOS_MemFree	Releases the allocated memory.
	LOS_MemRealloc	Re-allocates a memory block of the required size and copies content from the original block. If the new memory block is successfully allocated, the original one is released.
	LOS_MemAllocAlign	Allocates a memory block of the required size whose address is aligned based on boundary , from a dynamic memory pool.
Memory pool information obtainment	LOS_MemPoolSizeGet	Obtains the total size of a dynamic memory pool.
	LOS_MemTotalUsedGet	Obtains the total usage of a dynamic memory pool.
	LOS_MemInfoGet	Obtains the structure of a memory pool, including the size of idle memory, size of used memory, number of idle memory blocks, number of used memory blocks, and maximum size of idle memory blocks.
	LOS_MemPoolList	Prints information about all initialized memory pools in the system, including the start addresses of the memory pools, size of the memory pools, total size of idle memory, total size of used memory, maximum size of idle memory blocks, number of idle memory blocks, and number of used memory blocks. This function takes effect only when LOSCFG_MEM_MUL_POOL is enabled.
Memory block information obtainment	LOS_MemFreeBlocksGet	Obtains the number of idle memory blocks in a memory pool.

Function Category	API	Description
	LOS_MemUsedBlocksGet	Obtains the number of used memory blocks in a memory pool.
	LOS_MemTaskIdGet	Obtains the ID of the task that allocates a specified memory block. This function takes effect only when LOSCFG_MEM_DEBUG or LOSCFG_MEM_TASK_STAT is enabled.
	LOS_MemLastUsedGet	Obtains the address of the first byte after the last byte used by the end node of the memory pool. If the end node is a used node, the last byte used by the end node is the end of the node. If the end node is an empty node, the last byte used by the end node is the node header structure.
	LOS_MemNodeSizeCheck	Obtains the total size of a memory block and the size of idle memory in it. This function takes effect only when LOSCFG_BASE_MEM_NODE_SIZE_CHECK is enabled.
	LOS_MemFreeNodeShow	Prints the size and number of idle memory blocks in a memory pool.
Memory pool integrity check	LOS_MemIntegrityCheck	Checks the integrity of a memory pool. This function takes effect only when LOSCFG_BASE_MEM_NODE_INTEGRITY_CHECK is enabled.
Sets and obtains the memory check level. This function takes effect only when LOSCFG_BASE_MEM_NODE_SIZE_CHECK is enabled.	LOS_MemCheckLevelSet	Sets the memory check level.
	LOS_MemCheckLevelGet	Obtains the memory check level.
Dynamic memory allocation and release for a module (effective only when	LOS_MemMalloc	Takes memory out of a specified dynamic memory pool and allocates the memory to a specified module, and records the amount of memory allocated to the module.

Function Category	API	Description
LOSCFG_MEM_MUL_MODULE is enabled)	LOS_MemMfree	Releases a memory block that has been allocated to a specified module and records the amount of memory that is released.
	LOS_MemMalloc Align	Allocates a memory block of the required size whose address is aligned based on boundary , from a dynamic memory pool, and records the amount of the allocated memory.
	LOS_MemMrealloc	Re-allocates a memory block of the required size and copies content from the original block, and records the amount of the re-allocated memory. If the new memory block is successfully allocated, the original one is released.
Obtains the memory usage of a specified module. This function takes effect only when LOSCFG_MEM_MUL_MODULE is enabled.	LOS_MemMused Get	Obtains the memory usage of a specified module. The unit is byte.

NOTICE

- Debugging functions are provided for dynamic memory. For details, see [Memory Debugging](#).
- For the bestfit_little algorithm, only [Multi-Memory Pool Mechanism](#) and [Memory Validity Check](#) are supported. Other memory debugging functions are not supported.
- Among the preceding APIs, the ones controlled through macro are all related to memory debugging.
- After the LOS_MemRealloc or LOS_MemMrealloc operation is performed on the memory allocated using LOS_MemAllocAlign or LOS_MemMallocAlign, the new start address of memory may become unaligned.
- For the bestfit_little algorithm, the LOS_MemRealloc operation cannot be performed on the memory allocated using LOS_MemAllocAlign. Otherwise, failure is returned.

 NOTE

Huawei LiteOS provides the multi-module memory statistics function. This function provides encapsulation APIs based on common memory APIs, adding the module ID as input parameter. When performing memory operations in different modules and calling related encapsulation APIs, users can collect statistics about memory usage in all modules and memory usage of a specified module by module ID.

Application scenario: The system has clear service modules and users need to collect statistics on the memory usage of each module.

Dependency: Currently, only the bestfit memory management algorithm supports this function. You need to enable **LOSCFG_KERNEL_MEM_BESTFIT**.

How to Use:

1. The configuration item of this function is **LOSCFG_MEM_MUL_MODULE**, which is stored in **Kernel > Memory Management > Dynamic Memory Management > Enable Memory module statistics**.
2. Each service module is configured with a unique module ID. In the module code, the memory operation calls the related API and transfers the related module ID.
3. The **LOS_MemMusedGet** API can obtain the memory usage of a specified module, which can be used for optimization analysis of module memory usage.

Precautions:

1. The value of module ID is specified by the macro **MEM_MODULE_MAX**. When the number of system modules exceeds the value specified, the **MEM_MODULE_MAX** needs to be modified accordingly.
2. The APIs starting with **LOS_MemM** is required for all memory operations in the module. If these APIs are not called for memory operations, the statistics may be inaccurate.

Development Procedure

This section describes the development process of dynamic memory in typical scenarios.

1. Configure the start address and size of a dynamic memory pool in the **los_config.h** file.

Configuration Item	Description	Value Range	Default Value	Dependency
OS_SYS_MEM_ADDR	Start address of the system dynamic memory pool.	[0, n)	&m_aucSysMem1[0]	None.
OS_SYS_MEM_SIZE	Size (in bytes) of the system dynamic memory pool (DDR adaptive).	[0, n)	From the end of the BSS segment to the end of the system DDR	None.

- **OS_SYS_MEM_ADDR**: In most cases, retain the default value.
 - **OS_SYS_MEM_SIZE**: In most cases, retain the default value.
2. Open the menu and choose **Kernel > Memory Management** to configure the dynamic memory management module.

Configuration Item	Description	Value Range	Default Value	Dependency
LOSCFG_KERNEL_MEM_BESTFIT	Whether to use the bestfit algorithm for memory management.	YES/NO	YES	None.
LOSCFG_KERNEL_MEM_BESTFIT_LITTLE	Whether to use the bestfit_little algorithm for memory management.	YES/NO	NO	None.
LOSCFG_KERNEL_MEM_SLAB_EXTENSION	Whether to enable the slab function to reduce memory fragments at runtime.	YES/NO	NO	None.
LOSCFG_KERNEL_MEM_SLAB_AUTO_EXPANSION_MODE	Whether to automatically expand slabs by allocating new space from the system memory pool, when their memory space is insufficient.	YES/NO	NO	LOSCFG_KERNEL_MEM_SLAB_EXTENSION
LOSCFG_MEMORY_TASK_STAT	Whether to enable task memory statistics.	YES/NO	YES	LOSCFG_KERNEL_MEM_BESTFIT or LOSCFG_KERNEL_MEM_BESTFIT_LITTLE
LOSCFG_MEMORY_MODULE	Whether to enable multi-module memory statistics	YES/NO	NO	LOSCFG_KERNEL_MEM_BESTFIT

3. Call `LOS_MemInit` to initialize a memory pool.

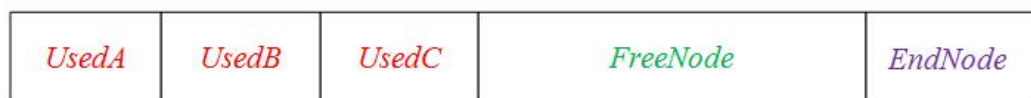
As shown in the figure below, an `EndNode` is generated after the initialization, and all the remaining memory is marked as a `FreeNode`. `EndNode` is the last node in the memory pool, and its size is 0.



4. Call `LOS_MemAlloc` to allocate dynamic memory of any size.

The system determines whether the dynamic memory pool has sufficient memory. If yes, it takes the required size out of the memory pool and returns a pointer. If no, it returns "NULL" instead.

The LOS_MemAlloc function can be called three times to create three nodes, namely, UsedA, UsedB, and UsedC with sizeA, sizeB, and sizeC, respectively. There is only one large FreeNode when the memory pool is initialized, so these memory blocks are divided from the FreeNode.



If there are multiple FreeNodes in the memory pool, the most appropriate size will be used to create a memory block during malloc, which can reduce memory fragments. If the created memory block is not the size of the selected FreeNode, the remaining memory will be marked as a new FreeNode.

5. Call LOS_MemFree to release dynamic memory.

The reclaimed memory block can be used next time.

Assume that LOS_MemFree is called to release UsedB. The reclaimed UsedB block will be marked as a FreeNode. When memory blocks are reclaimed, adjacent FreeNodes are automatically merged.



Platform Differences

None.

3.2.2.2 Precautions

Memory allocation

- The LOS_MemAllocAlign and LOS_MemMallocAlign APIs may consume extra memory for memory alignment, leading to memory loss. When the memory used for alignment is released, the lost memory will be reclaimed.
- If the LOS_MemRealloc or LOS_MemMrealloc API successfully re-allocates memory, the system determines whether it is necessary to release the previously allocated memory and returns the re-allocated memory address. If the reallocation fails, the previously allocated memory remains unchanged, and NULL will be returned. The use of pPtr = LOS_MemRealloc(pool, pPtr, uwSize) is not allowed, indicating that using the original pPtr to receive the returned value is forbidden.

Memory release

- If LOS_MemFree or LOS_MemMfree is called multiple times for the same memory block, only the first attempt succeeds. Subsequent attempts will be regarded as invalid pointers and may cause unpredictable results.

Others

- The dynamic memory module manages memory through control block structures that consume extra memory. Therefore, the memory space

available to users is usually less than the value of the configuration item **OS_SYS_MEM_SIZE**.

- In the memory node control block structure `LosMemDynNode` managed by the dynamic memory module, **sizeAndFlag** is of the `UINT32` type, wherein the most significant two bits indicate the flag and the remaining 30 indicate the size of the memory node. Therefore, the size of the initial memory pool cannot exceed 1 GB. Otherwise, unpredictable results may occur.

3.2.2.3 Programming Example

3.2.2.3.1 Common Memory Usage Example

Example Description

Prerequisite: Dynamic memory has been configured in the **menuconfig** menu.

In the programming example, the following steps will be performed:

1. Initialize a dynamic memory pool.
2. Allocate a memory block from the dynamic memory pool.
3. Store data in the memory block.
4. Print the data in the memory block.
5. Release the memory block.

Example Code

```
#define TEST_POOL_SIZE (2*1024*1024)
UINT8 g_testPool[TEST_POOL_SIZE];

VOID Example_DynMem(VOID)
{
    UINT32 *mem = NULL;
    UINT32 ret;

    ret = LOS_MemInit(g_testPool, TEST_POOL_SIZE);
    if (LOS_OK == ret) {
        dprintf("The memory pool is successfully initialized.\n");
    } else {
        dprintf("Failed to initialize the memory pool.\n");
        return;
    }

    /* Allocate memory. */
    mem = (UINT32 *)LOS_MemAlloc(g_testPool, 4);
    if (NULL == mem) {
        dprintf("Failed to allocate memory.\n");
        return;
    }
    dprintf("Memory is successfully allocated.\n");

    /* Assign a value. */
    *mem = 828;
    dprintf("***mem = %d\n", *mem);

    /* Release memory.*/
    ret = LOS_MemFree(g_testPool, mem);
    if (LOS_OK == ret) {
        dprintf("Memory is successfully released.\n");
    } else {
        dprintf("Failed to release memory.\n");
    }
}
```

```
}  
    return;  
}
```

Verification

The verification result is as follows:

```
The memory pool is successfully initialized.  
Memory is successfully allocated.  
*mem = 828  
Memory is successfully released.
```

Complete Code

sample_mem.c

3.2.2.3.2 Multi-Module Memory Usage Example

Example Description

Prerequisites: Multi-module memory statistics has been configured in the **menuconfig** menu.

In the programming example, the following steps will be performed:

1. Allocate a memory block for module 0 from the dynamic memory pool.
2. Obtain the memory usage of module 0.
3. Allocate a memory block for module 1 from the dynamic memory pool.
4. Obtain the memory usage of module 1.
5. Release the memory of modules 0 and module 1.

Example Code

```
void test(void)  
{  
    void *ptr = NULL;  
    void *ptrTmp = NULL;  
    UINT32 size = 0x10;  
    UINT32 module = 0;  
    UINT32 memUsed = 0;  
  
    ptr = LOS_MemMalloc(OS_SYS_MEM_ADDR, size, module);  
    if (ptr == NULL) {  
        PRINTK("module %d malloc failed\n", module);  
    } else {  
        PRINTK("module %d malloc succeeded\n", module);  
    }  
  
    memUsed = LOS_MemUsedGet(module);  
    PRINTK("module %d mem used size %d\n", module, memUsed);  
  
    module = 1;  
    ptrTmp = LOS_MemMalloc(OS_SYS_MEM_ADDR, size, module);  
    if (ptrTmp == NULL) {  
        PRINTK("module %d malloc failed\n", module);  
    } else {  
        PRINTK("module %d malloc succeeded\n", module);  
    }  
}
```



```
memUsed = LOS_MemMusedGet(module);
PRINTK("module %d mem used size %d\n", module, memUsed);

module = 0;
(VOID)LOS_MemMfree(OS_SYS_MEM_ADDR, ptr, module);
module = 1;
(VOID)LOS_MemMfree(OS_SYS_MEM_ADDR, ptrTmp, module);
}
```

Verification

The verification result is as follows:

```
module 0 malloc succeeded
module 0 mem used size 32
module 1 malloc succeeded
module 1 mem used size 32
```

Complete Code

sample_module_mem.rar

3.2.3 Static Memory

3.2.3.1 Development Guidelines

Application Scenario

When a user needs memory of fixed size, the system calls `LOS_MemboxAlloc` to allocate memory. When the user no longer needs the memory, the system calls `LOS_MemboxFree` to release it.

Functions

The static memory management module of Huawei LiteOS provides the following functions. For details about the APIs, see the API reference.

Function Category	API	Description
Static memory initialization	<code>LOS_MemboxInit</code>	Initializes a static memory pool; defines the start address and total size of the static memory pool, as well as the size of each memory block.
Static memory clearing	<code>LOS_MemboxClr</code>	Clears data in a memory block of fixed size.
Static memory allocation and free	<code>LOS_MemboxAlloc</code>	Allocates a memory block from a static memory pool.
	<code>LOS_MemboxFree</code>	Releases a static memory block.

Function Category	API	Description
Static memory pool information obtainment and printing	LOS_MemboxStatisticsGet	Obtains information about a static memory pool, including the total number of memory blocks, number of allocated memory blocks, and size of each memory block.
	LOS_ShowBox	Prints all node information for a static memory pool (the print level is LOS_INFO_LEVEL), including the start address of the memory pool, size of each memory block, total number of memory blocks, start address of each idle memory block, and start addresses of all memory blocks.

Development Procedure

This section describes the development process of static memory in typical scenarios.

1. Open the menu and choose **Kernel > Memory Management** to configure the static memory management module.

Configuration Item	Description	Value Range	Default Value	Dependency
LOSCFG_KERNEL_MEMBOX	Whether to enable membox management.	YES/NO	YES	None.
LOSCFG_KERNEL_MEMBOX_STATIC	Whether to implement membox in static memory mode.	YES/NO	YES	LOSCFG_KERNEL_MEMBOX
LOSCFG_KERNEL_MEMBOX_DYNAMIC	Whether to implement membox in dynamic memory mode.	YES/NO	NO	LOSCFG_KERNEL_MEMBOX

2. Plan a memory region as the static memory pool.
3. Call the LOS_MemboxInit API to initialize the static memory pool.
Initialize the static memory pool; divide the memory pool that matches the input parameters into N memory blocks, where N depends on the total size of static memory pool and the size of each static memory block; add all static memory blocks into a linked list of idle memory blocks; place a control header at the beginning of static memory pool.

4. Call the LOS_MemboxAlloc API to allocate static memory.
Take an idle memory block out of the linked list and return the user space address of the memory block.
5. Call the LOS_MemboxClr API.
Clear data in the static memory block that matches the input parameters.
6. Call the LOS_MemboxFree API.
Add the memory block to the free linked list.

Platform Differences

None.

3.2.3.2 Precautions

If a static memory pool region is obtained in dynamic memory allocation mode but it is not required, release the memory to avoid memory leakage.

3.2.3.3 Programming Example

Example Description

Prerequisite: Static memory has been configured in the **menuconfig** menu.

In the programming example, the following steps will be performed:

1. Initialize a static memory pool.
2. Allocate a memory block from the static memory pool.
3. Store data in the memory block.
4. Print the data in the memory block.
5. Clear the data in the memory block.
6. Release the memory block.

Example Code

```
VOID Example_StaticMem(VOID)
{
    UINT32 *mem = NULL;
    UINT32 blkSize = 10;
    UINT32 boxSize = 100;
    UINT32 boxMem[1000];
    UINT32 ret;

    ret = LOS_MemboxInit(&boxMem[0], boxSize, blkSize);
    if(ret != LOS_OK) {
        printf("Failed to initialize the memory pool.\n");
        return;
    } else {
        printf("The memory pool is successfully initialized.\n");
    }

    /* Allocate a memory block. */
    mem = (UINT32 *)LOS_MemboxAlloc(boxMem);
    if (NULL == mem) {
        printf("Failed to allocate memory.\n");
        return;
    }
}
```

```
printf("Memory is successfully allocated.\n");

/* Assign a value. */
*mem = 828;
printf("*mem = %d\n", *mem);

/* Clear content in the memory. */
LOS_MemboxClr(boxMem, mem);
printf("Content is successfully cleared from memory.\n *mem = %d\n", *mem);

/* Release memory.*/
ret = LOS_MemboxFree(boxMem, mem);
if (LOS_OK == ret) {
    printf("Memory is successfully released.\n");
} else {
    printf("Failed to release memory.\n");
}

return;
}
```

Verification

The verification result is as follows:

```
The memory pool is successfully initialized.
Memory is successfully allocated.
*mem = 828
Content is successfully cleared from memory.
*mem = 0
Memory is successfully released.
```

Complete Code

sample_membox.c

3.3 Interrupt

3.3.1 Overview

Basic Concepts

An interrupt refers to the process in which the CPU stops to execute a new program when necessary. That is, during the running of the program, a transaction that must be immediately processed by the CPU occurs. In this case, the CPU temporarily stops the execution of the ongoing program and processes the transaction. This process is called interruption.

Peripherals can complete certain work without the CPU. In some cases, however, the CPU needs to perform certain work for the peripherals. With the interrupt mechanism, the CPU can execute other tasks when the peripherals do not require the CPU. When the peripherals require the CPU, the CPU can interrupt the ongoing task to respond to the interrupt request by generating the interrupt signal. In this way, the CPU does not need to spend a lot of time in waiting and querying the peripheral status, which greatly improves the real-time performance and execution efficiency of the system.

The interrupt mechanism of Huawei LiteOS supports:

- Interrupt sharing, which can be configured.
- Interrupt nesting, that is, a high-priority interrupt can preempt a low-priority interrupt, which can be configured.
- Independent interrupt stack, which can be configured.
- Number of supported interrupt priorities, which can be configured.
- Number of supported interrupts, which can be configured.
- Interrupt bottom half, which can be configured.

Interrupt-related Hardware

Interrupt-related hardware can be classified into three types: device, interrupt controller, and CPU.

- **Device**
The device is the source that initiates an interrupt. When the device needs to request the CPU, an interrupt signal is generated and connected to the interrupt controller.
- **Interrupt controller**
The interrupt controller is one of the peripherals of the CPU. On the one hand, the interrupt controller receives the input of the interrupt pins of other peripherals. On the other hand, the interrupt controller sends interrupt signals to the CPU. The interrupt controller can be programmed to set the startup/stop status, priority, and trigger mode of the interrupt source. Common interrupt controllers include vector interrupt controller (VIC), general interrupt controller (GIC), platform-level interrupt controller (PLIC), and core-local interrupt controller (CLIC). The nested vector interrupt controller (NVIC) is used in the ARM Cortex-M series. The GIC is used in ARM Cortex-A7. The PLIC and CLIC are used in RISC-V.
- **CPU**
The CPU responds to the request of the interrupt source, interrupts the ongoing task, and executes the interrupt handler.

Interrupt-related Concepts

Interrupt number

Interrupt request signals are numbered so the computer can determine interrupt requests sent by devices.

Interrupt request

An emergency event needs to apply to the CPU (by sending an electrical pulse signal) for interruption, and requires the CPU to suspend the ongoing task and process the emergency event. This application process is called interrupt request.

Interrupt priority

To enable the system to respond to and handle all interrupts in time, the system classifies interrupt sources into several levels based on the importance and urgency of interrupt events.

Interrupt handler

After a peripheral generates an interrupt request, the CPU suspends the ongoing task and responds to the interrupt request. Each device that generates an interrupt has a corresponding interrupt handler.

Interrupt nesting

Interrupt nesting is also called interrupt preemption. When an interrupt handler is being executed, if another interrupt source with a higher priority raises an interrupt request, the interrupt handler of the interrupt source with a lower priority that is being executed is temporarily stopped, and the interrupt request with a higher priority is processed, after the processing is complete, the stopped interrupt handler continues to be executed.

Interrupt triggering

The interrupt source sends an interrupt signal to the interrupt controller. The interrupt controller arbitrates the interrupt, determines the priority, and sends the interrupt signal to the CPU. When the interrupt source generates an interrupt signal, the interrupt trigger is set to **1**, indicating that the interrupt source generates an interrupt and requires the CPU to respond to the interrupt.

Interrupt trigger type

An external interrupt request is sent to the NVIC or GIC through a physical signal. The interrupt can be level-triggered or edge-triggered.

Interrupt vector

Entry address of the interrupt service routine (ISR).

Interrupt vector table

Region for storing interrupt vectors. Interrupt vectors map to interrupt numbers. The interrupt vectors are stored in the interrupt vector table in sequence based on the interrupt numbers.

Interrupt sharing

When the number of peripherals is small, one peripheral corresponds to one interrupt number. To support more hardware devices, multiple devices can share one interrupt number. The interrupt handlers that share the same interrupt number form a linked list. When an external device generates an interrupt request, the system traverses the linked list until the interrupt handler of the device is found. During the traversal, each interrupt handler may determine, by detecting the device ID, whether an interrupt is generated by the corresponding device.

Inter-core interrupt

For the SMP system, the interrupt controller allows a hardware thread of a CPU to interrupt the hardware threads of other CPUs. This mode is called inter-core interrupt. Inter-core interrupts are implemented based on memory sharing among multiple CPUs. Inter-core interrupts can reduce the load of a CPU and improve system efficiency. Currently, only the GIC and LingLong interrupt controllers support inter-core interrupt.

Interrupt bottom half

The interrupt bottom half runs in the task context, which is different from interrupt handler. User can add non-urgent services that have low real-time requirements to the interrupt bottom half in the interrupt handler.

Operation Mechanism

- The interrupt mechanism of Huawei LiteOS supports interrupt sharing.
The implementation of interrupt sharing depends on the linked list. A linked list is created for each interrupt number. The linked list node contains the registered interrupt handler function and input parameters of the function. When interrupts are created for the same interrupt number for multiple times, the interrupt handler function and input parameters of the function are added to the linked list corresponding to the interrupt number. Therefore, when the hardware generates an interrupt, the corresponding linked list is found based on the interrupt number, and the linked list is traversed until the interrupt handler function of the corresponding device is found.
- The interrupt mechanism of Huawei LiteOS supports interrupt nesting.
The interrupt nesting of the GIC and NVIC is implemented by hardware.
The interrupt nesting mechanism of the RISC-V is as follows: In interrupt nesting mode, after interrupt A is triggered, the current operation is pushed to the stack. Before the interrupt handler is called, the MIE is set to **1** to allow new interrupts to be responded. During the execution of interrupt handler by interrupt A, if interrupt B with a higher priority is triggered, interrupt B pushes the current operation, that is, the operation related to interrupt A, into the stack, and then executes the interrupt handler of interrupt B. After the interrupt handler of interrupt B is executed, the MIE field in the mstatus register is temporarily set to **0**, the interrupt response is disabled, the operations related to interrupt A are popped from the stack, and the MIE field is set to **1** to allow the processor to respond to the interrupt again. After interrupt B ends, interrupt A continues to be executed.
- The interrupt mechanism of Huawei LiteOS supports interrupt bottom half.
The interrupt bottom half depends on the linked list, task, and event mechanism.
In the interrupt handler, user adds the processing function of bottom half to the list. After the interrupt exits, the processing function of bottom half added by user will be executed one by one in the interrupt bottom half tasks.

3.3.2 Development Guidelines

Application Scenario

When an interrupt request is generated, the CPU suspends the current task and responds to the peripheral request. You can allocate an interrupt, register an interrupt handler, and specify the operation to be performed when the CPU responds to an interrupt request.

Functions

The interrupt module of Huawei LiteOS provides the following functions. For details about the APIs, see the API reference.

Function Category	API	Description
Creating and deleting an interrupt	LOS_HwiCreate	Creates an interrupt, and registers the interrupt number, interrupt trigger mode, interrupt priority, and interrupt handler. When an interrupt is triggered, the handleirq calls the interrupt handler.
	LOS_HwiDelete	Deletes an interrupt. Do not call this API in the registered interrupt handler. Otherwise, invalid memory may be accessed under shared mode.
Enabling and disabling all interrupts	LOS_IntUnlock	Enables all interrupt responses of the current processor.
	LOS_IntLock	Disables all interrupt responses of the current processor.
	LOS_IntRestore	Restores to the status before all interrupts are disabled by using LOS_IntLock.
Enabling and disabling a specified interrupt	LOS_HwiDisable	Disables an interrupt by configuring the register to prevent the CPU from responding to the interrupt.
	LOS_HwiEnable	Enables an interrupt by configuring the register to allow the CPU to respond to the interrupt.
Setting interrupt priorities	LOS_HwiSetPriority	Sets the interrupt priority.
Triggering interrupts	LOS_HwiTrigger	Triggers an interrupt by writing data into the related register of the interrupt controller to simulate an external interrupt.
Clearing the status of the interrupt register	LOS_HwiClear	Clears the status bit of the interrupt register corresponding to the interrupt number. This API depends on the interrupt controller version and is optional.
Inter-core interrupt	LOS_HwiSendIpi	Sends inter-core interrupts to a specified core. This API depends on the interrupt controller version and CPU architecture, and is supported only in SMP mode.
Setting the interrupt affinity	LOS_HwiSetAffinity	Sets the interrupt affinity, that is, sets the interrupt response to a fixed core. The function is supported only in SMP mode.
Setting the hook function processing before interrupt	LOS_HwiPreHookReg	Sets the hook function processing before interrupt.

Function Category	API	Description
Setting the hook function processing after interrupt	LOS_HwiPos tHookReg	Sets the hook function processing after interrupt.
Obtaining the interrupt response times	LOS_HwiRes pCntGet	Obtains the interrupt response times.
Adding the function of interrupt bottom half	LOS_HwiBh workAdd	Adds the function of interrupt bottom half.

HWI Error Codes

Error codes are returned for operations that may fail to be performed to quickly locate error causes.

N o.	Definition	Error Code	Description	Reference Solution
1	LOS_ERRNO_HWI_NUM_INVALID (OS_ERRNO_HWI_NUM_INVALID)	0x02000900	An invalid interrupt number is input when an interrupt is created or deleted. The error code OS_ERRNO_HWI_NUM_INVALID of the earlier version is not supported in later versions.	Check the interrupt number and provide a valid interrupt number.
2	LOS_ERRNO_HWI_PROC_FUNC_NULL (OS_ERRNO_HWI_PROC_FUNC_NULL)	0x02000901	When an interrupt is created, the input interrupt handler pointer is null. If this error code is returned when an API is called, the API function is not supported. The error code OS_ERRNO_HWI_PROC_FUNC_NULL of the earlier version is not supported in later versions.	Input a non-empty interrupt handler pointer.

N o.	Definition	Error Code	Description	Reference Solution
3	LOS_ERRNO_HWI_NO_MEMORY (OS_ERRNO_HWI_NO_MEMORY)	0x02000903	The memory is insufficient during interrupt creation. The error code OS_ERRNO_HWI_NO_MEMORY of the earlier version is not supported in later versions.	Expand the dynamic memory space in either of the following ways: - Set a larger system dynamic memory pool. The configuration item is OS_SYS_MEM_SIZE. - Release a part of the dynamic memory.
4	LOS_ERRNO_HWI_ALREADY_CREATED (OS_ERRNO_HWI_ALREADY_CREATED)	0x02000904	The interrupt number to be registered has been created during interrupt creation. The error code OS_ERRNO_HWI_ALREADY_CREATED of the earlier version is not supported in later versions.	For a non-shared interrupt number, check whether the input interrupt number has been created. For a shared interrupt number, check whether the device ID that matches the function parameter exists in the linked list of the interrupt number.
5	LOS_ERRNO_HWI_PRIORITY_INVALID (OS_ERRNO_HWI_PRIORITY_INVALID)	0x02000905	The input interrupt priority is invalid during interrupt creation. The error code OS_ERRNO_HWI_PRIORITY_INVALID of the earlier version is not supported in later versions.	Input a valid interrupt priority. The priority range depends on the hardware and can be configured externally.
6	LOS_ERRNO_HWI_INTERRUPT (OS_ERRNO_HWI_INTERRUPT)	0x02000908	Call the request_irq or bus_request_intr APIs in the interrupt. The error code OS_ERRNO_HWI_INTERRUPT of the earlier version is not supported in later versions. Note that the open-source version does not contain the two APIs.	Check whether the request_irq and bus_request_intr APIs are correctly used.

N o.	Definition	Error Code	Description	Reference Solution
7	LOS_ERRNO_HWI_SHARED_ERROR (OS_ERRNO_HWI_SHARED_ERROR)	0x02000909	Creating an interrupt: hwiMode specifies that a shared interrupt is created, but the device ID is not set. hwiMode specifies that a non-shared interrupt is created, but the interrupt number has been used for a shared interrupt before. Or LOSCFG_SHARED_IRQ is set to No, but the input parameter specifies that the shared interrupt is created during interrupt creation. Deleting an interrupt: If a shared interrupt is created based on the setting during the creation of device ID, and the device ID is not set during interrupt deletion, the interrupt cannot be deleted. The error code OS_ERRNO_HWI_SHARED_ERROR of the earlier version is not supported in later versions.	Check the input parameters. During creation, the value of hwiMode must be the same as that of irqParam . If the value of hwiMode is 0 , the interrupt is not shared. In this case, the value of irqParam must be NULL . When hwiMode is set to IRQF_SHARED , the interrupt is shared. In this case, you need to set irqParam to the device ID. When LOSCFG_SHARED_IRQ is set to No , that is, in non-shared interrupt mode, only non-shared interrupts can be created. During interrupt deletion, the value of irqParam must be the same as that set during interrupt creation.
8	LOS_ERRNO_HWI_HWINUM_UNCREATE (OS_ERRNO_HWI_HWINUM_UNCREATE)	0x0200090b	In interrupt sharing mode, when an interrupt is deleted, the corresponding device ID cannot be matched in the linked list of the interrupt number. The error code OS_ERRNO_HWI_HWINUM_UNCREATE of the earlier version is not supported in later versions.	For a shared interrupt number, check whether the device ID that matches the function parameter exists in the linked list of the interrupt number.

N o.	Definition	Error Code	Description	Reference Solution
9	LOS_ERRNO_HWI_NO_CPUP_MEMORY	0x0200090c	After LOSCFG_CPUP_CB_NUM_CONFIGURABLE is enabled, this error code is returned if the number of interrupt-related control blocks of the CPUP is insufficient when an interrupt is created.	Configure LOSCFG_CPUP_IRQ_CB_NUM macro based on the actual numbers of interrupt instead of the maximum interrupt number. The value cannot be greater than the value of LOSCFG_PLATFORM_HWI_LIMIT.
10	LOS_ERRNO_HWI_NOT_INTERRUPT_CONTEXT	0x0200090d	The LOS_HwiBhworkAdd is not called in the interrupt context.	Call LOS_HwiBhworkAdd in the interrupt context.
11	LOS_ERRNO_HWI_PTR_NULL	0x0200090e	The input parameter pointer of the function is null.	Input a valid non-null pointer as the input parameter.
12	LOS_ERRNO_HWI_ARG_NOT_ENABLED	0x0200090f	When LOSCFG_HWI_WITH_ARG is not enabled, a non-null pointer is transferred to the last parameter of LOS_HwiCreate.	Transfer NULL to the last parameter of LOS_HwiCreate or enable LOSCFG_HWI_WITH_ARG.

NOTICE

For details about the error code definition, see [Error Codes](#). Bits 8–15 belong to the interrupt module, and the value is 0x09.

Development Procedure

1. Open the menu and choose **Kernel > Interrupt Management** to configure the interrupt module.

Configuration Item	Description	Value Range	Default Value	Dependency
LOSCFG_ARCH_INTERRUPT_PREENEMPTION	Whether to enable interrupt nesting.	YES/NO	NO	GICv3, RISC-V

Configuration Item	Description	Value Range	Default Value	Dependency
LOSCFG_IRQ_USE_STANDALONE_STACK	Whether to use independent interrupt stacks.	YES/NO	YES	Depend on the CPU architecture. Currently, the Cortex-A, Cortex-R, Arm64 and LingLong support this function. The RISC-V uses an independent interrupt stack by default.
LOSCFG_SHARED_IRQ	Whether to enable interrupt sharing.	YES/NO	YES	None.
LOSCFG_PLATFORM_HWLIMIT	Maximum number of used interrupts.	Adapted based on the chip manual.	Adapted based on the chip manual.	None.
LOSCFG_HWIPRIO_LIMIT	Number of configurable interrupt priorities.	Adapted based on the chip manual.	Adapted based on the chip manual.	None.
LOSCFG_IRQ_STACK_SIZE	Interrupt stack size	Auto-adaptation based on the board	0x2000	Currently, only RISC-V can be configured.
LOSCFG_NMI_STACK_SIZE	Size of the non-maskable interrupt stack	Auto-adaptation based on the board	0x800	Currently, only RISC-V can be configured.
LOSCFG_HWIPRE_POST_PROCESS	Whether to set hook function processing before and after an interrupt.	YES/NO	NO	None.

Configuration Item	Description	Value Range	Default Value	Dependency
LOSCFG_HWI_BOTTOM_HALF	Whether to enable the interrupt bottom half.	YES/NO	NO	LOSCFG_BASE_IPC_EVENT
LOSCFG_HWI_BOTTOM_HALF_WORK_LIMIT	Maximum number of buffers in the list of interrupt bottom half	Auto-adaptation based on the board	16	LOSCFG_HWI_BOTTOM_HALF

2. Create an interrupt by calling the LOS_HwiCreate API.
3. If the SMP mode is used, call the LOS_HwiSetAffinity API to set the affinity of the interrupt. Otherwise, go to 4.
4. Enable a specified interrupt by calling the LOS_HwiEnable API.
5. Trigger a specified interrupt by calling the LOS_HwiTrigger API. (This API simulates an external interrupt by writing data to the related register of the interrupt controller. This step is not required for common peripheral devices.)
6. In the interrupt processing function, call LOS_HwiBhworkAdd API to add the processing function of interrupt bottom half. If there is no processing function of interrupt bottom half, you can skip this step.
7. Disable a specified interrupt by calling the LOS_HwiDisable API. This API is used as required and determines whether to disable the interrupt.
8. Delete a specified interrupt by calling the LOS_HwiDelete API. This API is used as required to determine whether to delete the interrupt.

3.3.3 Precautions

Interrupt handling

- The interrupt handler cannot take a long time. Otherwise, the CPU cannot respond to interrupts in time.
- The function that causes scheduling cannot be executed during the response to interrupts.

Interrupt sharing

- The interrupt sharing mechanism allows different devices to use the same interrupt number to register the same interrupt handler. However, the input parameter **pDevId** (device ID) of the interrupt handler must be unique to represent different devices. That is, interrupt handlers can be mounted only once with the same interrupt numbers and device IDs. However, the interrupt handlers can be mounted repeatedly with the same interrupt numbers and different device IDs.

Interrupt triggering

- RISC-V himideerV200 does not support the LOS_HwiTrigger API because the RISC-V himideerV200 interrupt controller pending register cannot be written.

You need to simulate an interrupt trigger to perform the test. Use a soft interrupt or peripheral interrupt to verify the interrupt trigger process.

Interrupt register status clearing

- When interrupt nesting is enabled for RISC-V himideerV200, for the level-triggered mode, the LOS_HwiClear API needs to be called to proactively clear the interrupt register status when the interrupt handler ends. That is, for RISC-V himideerV200, the LOS_HwiClear API takes effect only in the interrupt nesting scenario. In the non-nested scenario, you do not need to clear the interrupt controller status. The LOS_HwiClear API does not have the interrupt status clear effect.

Interrupt bottom half

- Do not call the delay API in the processing function of interrupt bottom half to avoid calling the blocking API. The running time of the task in the interrupt bottom half is uncertain. The task in the interrupt bottom half runs only when the task has the highest priority.

Compatible APIs

- Huawei LiteOS has interrupt APIs compatible with Linux, including request_irq, disable_irq, enable_irq and free_irq. For details, see [Interrupt](#). Developers can select proper APIs as required.

Others

- Configure the maximum number of supported interrupts and the number of configurable interrupt priorities based on the specific hardware.
- The input parameter of LOS_IntRestore() must be the return value of the LOS_IntLock() API, that is, the CPSR value before the interrupt is disabled.
- Interrupts 0–15 in the Cortex-M series processors and interrupts 0–31 in the Cortex-A7 are for internal use. Therefore, you are not advised to allocate and create them.

3.3.4 Programming Example

Example Description

The following functions are implemented:

1. Creating interrupts
2. Setting the interrupt affinity
3. Enabling interrupts
4. Triggering interrupts
5. Disabling interrupts
6. Deleting interrupts

Example Code

Prerequisite: The maximum number of interrupts that can be used and the number of interrupt priorities that can be set have been configured in the **menuconfig** menu.

The code is as follows:

```
#include "los_hwi.h"
#include "los_typedef.h"
#include "los_task.h"

STATIC UINT32 g_BhworkCnt = 0;

STATIC VOID HwiBhfunc(VOID *data)
{
    UINT32 *count = (UINT32 *data);
    (*count)++;
}

STATIC VOID HwiUsrIrq(VOID)
{
    UINT32 ret;
    ret = LOS_HwiBhworkAdd(HwiBhfunc, &g_BhworkCnt);
    if (ret != LOS_OK) {
        printf("\n hwi bhwork add failed, %x \n", ret);
    }
    printf("\n in the func HwiUsrIrq \n");
}

/* cpu0 trigger, cpu0 response */
UINT32 It_Hwi_001(VOID)
{
    UINT32 ret;
    HWI_HANDLE_T irqNum = 26;
    HWI_PRIOR_T irqPri = 0x3;

    ret = LOS_HwiCreate(irqNum, irqPri, 0, (HWI_PROC_FUNC)HwiUsrIrq, 0);
    if (ret != LOS_OK) {
        return LOS_NOK;
    }

#ifdef LOSCFG_KERNEL_SMP
    ret = LOS_HwiSetAffinity(irqNum, CPUID_TO_AFFI_MASK(ArchCurrCpuId()));
    if (ret != LOS_OK) {
        return LOS_NOK;
    }
#endif
    ret = LOS_HwiEnable(irqNum);
    if (ret != LOS_OK) {
        return LOS_NOK;
    }

    ret = LOS_HwiTrigger(irqNum);
    if (ret != LOS_OK) {
        return LOS_NOK;
    }

    LOS_TaskDelay(1);

    ret = LOS_HwiDisable(irqNum);
    if (ret != LOS_OK) {
        return LOS_NOK;
    }

    ret = LOS_HwiDelete(irqNum, NULL);
    if (ret != LOS_OK) {
        return LOS_NOK;
    }

    return LOS_OK;
}
```


Complete Code

sample_hwi.rar

3.4 Exception Management

3.4.1 Overview

Basic Concepts

Exception management refers to a series of actions taken by the OS to handle exceptions (chip hardware exceptions) that occur during the running of the OS, for example, printing the call stack information of the current function, CPU information, and task stack information when an exception occurs.

As a commissioning method, exception management provides useful exception information such as the exception type and system status when an exception occurs, to help you locate and analyze faults.

When an exception occurs, Huawei LiteOS displays the information about the running task (including the task name, task ID, and stack size) and CPU information. For some chips with the RISC-V architecture that have high requirements on the memory size, Huawei LiteOS provides the small feature macro `LOSCFG_EXC_SIMPLE_INFO` (location in **menuconfig: Kernel > Exception Management > Enable Exception Simple Info**) to tailor redundant exception prompt character strings. However, all information about the CPU operating environment when an exception occurs is retained.

Operation Mechanism

Each function has its own stack space, which is called stack frame. When the function is called, the stack frame of the subfunction is created, and the input parameters, local variables, and registers of the function are put into the stack. The stack frame grows from a high address to a low address.

The ARM32 CPU architecture is used as an example. Each stack frame stores the historical values of the program counter (PC), link register (LR), stack pointer (SP), and frame point (FP) registers.

Stack Analysis

- The LR points to the return address of the function.
- R11 can be used as a general register. When specific compilation options are enabled, it can be used as an FP to implement the stack backtracking function.

By default, the GNU compiler collection (GCC) uses R11 as the general register for storing variables. Therefore, the stack backtracking function of the FP cannot be used by default. To support the call stack parsing function, the `-fno-omit-frame-pointer` option needs to be added to the compilation parameters to ask the compiler to use R11 as the FP.

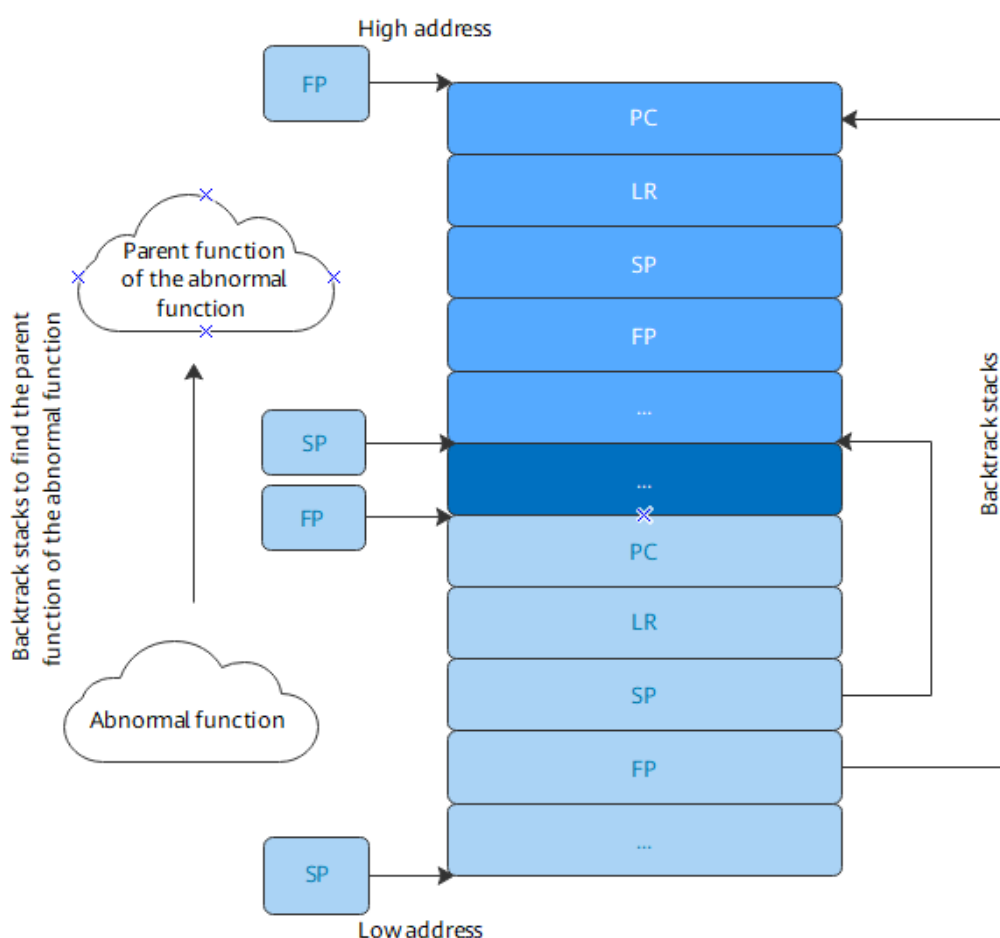
- The FP points to the stack frame start address of the parent function of the current function. The stack frame of the parent function can be obtained by

using the FP. The stack frame of the grandparent function can be obtained by obtaining the FP of the parent function from the stack frame. By analogy, the call stack of the program can be traced to obtain the call relationship between functions.

When an exception occurs in the system, the system prints the register content saved in the stack frame of the abnormal function and the LR and FP register content in the stack frame of the parent function and grandfather function. You can trace the call relationship between functions and locate the exception cause based on the content.

The actual stack information varies depending on the CPU architecture. The following figure shows the stack analysis principle, which is for reference only.

Figure 3-6 Stack analysis principle



In the figure, the registers in different colors indicate different functions. The register is saved when the function is called. The FP register is used to backtrack the stacks to the parent function of the abnormal function, continue to parse the stacks based on the rule, and determine the function call relationship to facilitate problem locating.

3.4.2 User Guide

Functions

The exception management function processes chip hardware exceptions that occur during system running. The exception types vary according to chips. For details about the exception types, see the chip manual.

Problem Locating Procedure

The procedure for exception management is as follows:

1. Open the disassembling image (ASM) file generated after compilation.
2. Search for the position of the PC (pointing to the command that is being executed) in the ASM file to find the abnormal function.
3. Find the parent function of the abnormal function based on the LR value.
4. Repeat 3 to obtain the call relationship between functions and locate the exception cause.

For details about the locating method, see the example.

Configuration Item

Open the menu and choose the **Kernel > Exception Management** to configure the exception management module.

Configuration Item	Description	Value Range	Default Value	Dependency
LOSCFG_EXC_SIMPLE_INFO	Reduces the number of abnormal output information.	YES/NO	NO	None.
LOSCFG_EXC_STACK_SIZE	Size of the exception stack	Auto-adaptation based on the board	0x800	Currently, only RISC-V can be configured.

3.4.3 Precautions

To view the call stack information, you must add the compilation option macro -fno-omit-frame-pointer to support stack frames. Otherwise, the FP is disabled during compilation.

3.4.4 Problem Locating Example

On an ARM32 platform, the memory is released through an error, which triggers a system exception. After the system is suspended due to the exception, the information about the exception call stack and key registers can be viewed through the serial port, shown as below. **excType** indicates the exception type. The

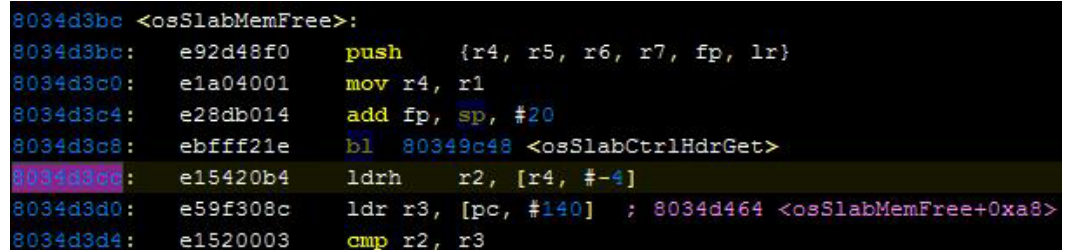
value **4** indicates that the data is terminated abnormally. For other values, see the chip manual. The information will help you locate the function where the exception occurs and call stack relations of the function.

```
excType: 4
taskName = MNT_send
taskId = 6
taskStackSize = 12288
excBuffAddr pc = 0x8034d3cc
excBuffAddr lr = 0x8034d3cc
excBuffAddr sp = 0x809ca358
excBuffAddr fp = 0x809ca36c
*****backtrace begin*****
traceback 0 -- lr = 0x803482fc
traceback 0 -- fp = 0x809ca38c
traceback 1 -- lr = 0x80393e34
traceback 1 -- fp = 0x809ca3a4
traceback 2 -- lr = 0x8039e0d0
traceback 2 -- fp = 0x809ca3b4
traceback 3 -- lr = 0x80386bec
traceback 3 -- fp = 0x809ca424
traceback 4 -- lr = 0x800a6210
traceback 4 -- fp = 0x805da164
```

To locate the exception, perform the following steps:

1. Open the .asm disassembly file generated after compilation. By default, the file is generated in the **Huawei_LiteOS/out/<Platform name>** directory.
2. Search for the position of the PC **8034d3cc**, in the ASM file. (**0x** needs to be removed.)

The PC address points to the command that the program is executing when the exception occurs. In the ASM file of the currently executed binary file, search for the PC value **8034d3cc** and find the command line that is being executed by the CPU. The result is shown in the following figure.



```
8034d3bc <osSlabMemFree>:
8034d3bc: e92d48f0    push    {r4, r5, r6, r7, fp, lr}
8034d3c0: e1a04001    mov     r4, r1
8034d3c4: e28db014    add     fp, sp, #20
8034d3c8: ebfff21e    bl     80349c48 <osSlabCtrlHdrGet>
8034d3cc: e15420b4    ldrh    r2, [r4, #-4]
8034d3d0: e59f308c    ldr     r3, [pc, #140] ; 8034d464 <osSlabMemFree+0xa8>
8034d3d4: e1520003    cmp     r2, r3
```

You can obtain the following information from the figure above:

- a. The CPU is executing the **ldrh r2, [r4, #-4]** command when the exception occurs.
- b. The exception occurs in the **osSlabMemFree** function.

According to the analysis, the **ldrh** command is used to read a value from the memory address (r4-4) and load the value to the register r2. Then, check the value of r4 based on the register information printed when the exception occurs. The following figure shows the register information printed when the exception occurs. The value of r4 is **0xffffffff**.

```

R0      = 0x807243c8
R1      = 0xffffffff
R2      = 0x80000013
R3      = 0x80623de0
R4      = 0xffffffff
R5      = 0x80000013
R6      = 0x807243c0
R7      = 0x1
R8      = 0x809ca490
R9      = 0x4e4
R10     = 0x11
R11     = 0x809ca36c
R12     = 0x800000d3
SP      = 0x809ca358
LR      = 0x8034d3cc
PC      = 0x8034d3cc
CPSR    = 0xa00000d3

```

The r4 value exceeds the memory range, and therefore a data termination exception occurs while the CPU is executing the command. According to the ASM file, r4 is moved from r1, and r1 is the second input parameter of the function. Therefore, the incorrect input parameter **0xffffffff** (or **-1**) is input when the osSlabMemFree function is called.

Next, you need to find out the caller of the osSlabMemFree function.

3. Search for the call stack based on the LR value.

The call stack information starts from backtrace begin of the printed exception information. Search for the LR corresponding to **backtrace 0** in the ASM file, as shown in the following figure.

```

803482c4 <LOS_MemFree>:
803482c4: e92d4bf0    push    {r4, r5, r6, r7, r8, r9, fp, lr}
803482c8: e28db01c    add fp, sp, #28
803482cc: e10f5000    mrs r5, CPSR
803482d0: e385c0c0    orr ip, r5, #192 ; 0xc0
803482d4: e121f00c    msr CPSR_c, ip
803482d8: e3510000    cmp r1, #0
803482dc: 13500000    cmpne r0, #0
803482e0: 1a000002    bne 803482f0 <LOS_MemFree+0x2c>
803482e4: e3a00001    mov r0, #1
803482e8: e121f005    msr CPSR_c, r5
803482ec: e8bd8bf0    pop {r4, r5, r6, r7, r8, r9, fp, pc}
803482f0: e1a04001    mov r4, r1
803482f4: e1a06000    mov r6, r0
803482f8: eb00142f    bl 8034d3bc <osSlabMemFree>
803482fc: e3500001    cmp r0, #1
80348300: 1afffff8    bne 803482e8 <LOS_MemFree+0x24>

```

The figure shows that the LOS_MemFree function calls the osSlabMemFree function. By using this method, you can find that the call relationships between functions are MNT_buf_send (service function) -> free -> LOS_MemFree -> osSlabMemFree.

By checking the implementation of MNT_buf_send, the problem of incorrectly using a pointer is found. This problem causes an incorrect address to be freed. As a result, the exception occurs.

3.5 Error Handling

3.5.1 Overview

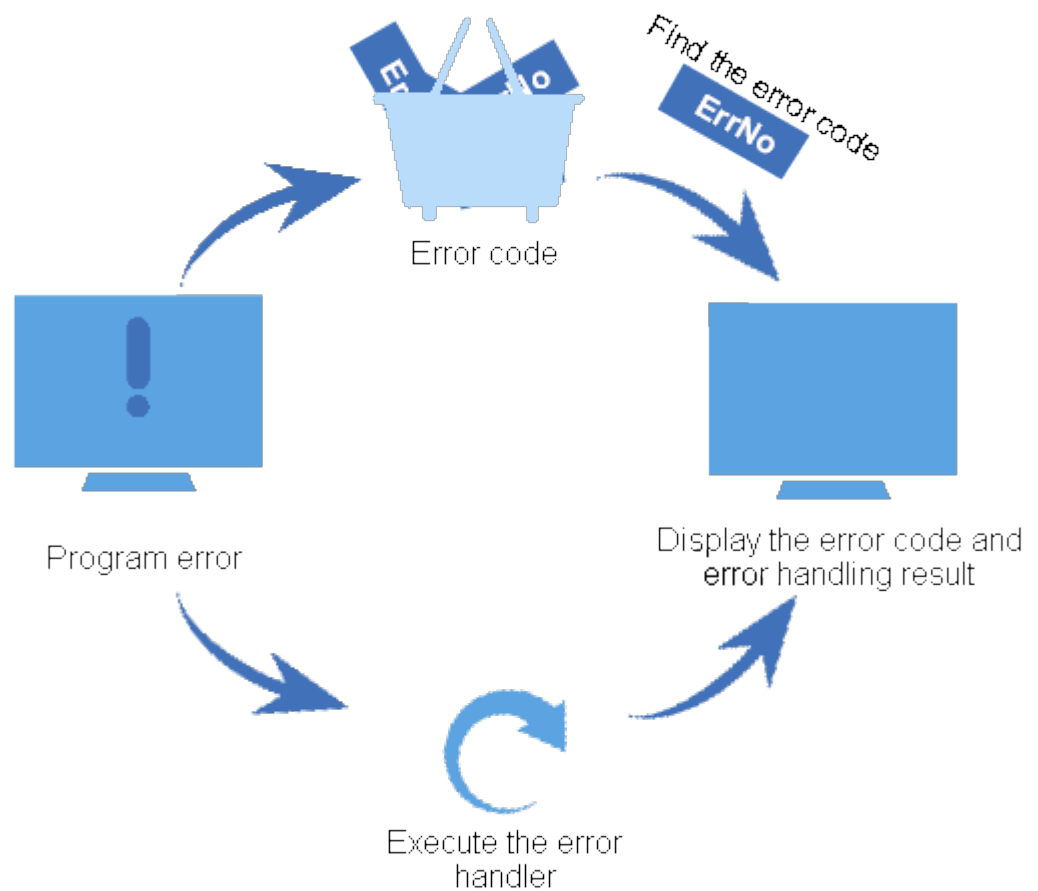
Basic Concepts

When an error occurs during the program running, the API function of the error handling module is called to report the error information, and the registered hook function is called to perform specific processing. The field is saved for locating the problem.

With error handling, illegal inputs in the program can be controlled and prompted to prevent the program from breaking down.

Operation Mechanism

Error handling is a mechanism used to handle exceptions. When an error occurs in the program, the corresponding error code is displayed. In addition, if the corresponding error handling function is registered, this function is executed.

Figure 3-7 Error handling

3.5.2 Development Guidelines

Error Codes

An error may occur when an API is called. In this case, the API returns the corresponding error code to help quickly locate the error cause.

An error code is a 32-bit unsigned integer. Bits 31 to 24 indicate the error level. Bits 23 to 16 indicate the error code flag (the current value is **0**). Digits 15 to 8 indicate the module to which the error code belongs. Bits 7 to 0 indicate the sequence number of the error code.

For example, error code `LOS_ERRNO_TSK_NO_MEMORY` in the task module is defined as a fatal-level error, the module ID is `LOS_MOD_TSK`, and the error code sequence number is 0. The definition is as follows:

```
#define LOS_ERRNO_TSK_NO_MEMORY LOS_ERRNO_OS_FATAL(LOS_MOD_TSK, 0x00)
```

```
#define LOS_ERRNO_OS_FATAL(MID, ERRNO) \
    (LOS_ERRTYPE_FATAL | LOS_ERRNO_OS_ID | ((UINT32)(MID) << 8) | ((UINT32)(ERRNO)))
```

NOTE

- **LOS_ERRTYPE_FATAL**: error level is fatal. The value is **0x03000000**.
- **LOS_ERRNO_OS_ID**: error code flag. The value is **0x000000**.
- **MID**: module. The value of **LOS_MOD_TSK** is **0x2**.
- **ERRNO**: sequence number of an error code.

Therefore, the value of **LOS_ERRNO_TSK_NO_MEMORY** is **0x03000200**.

Error Code Management

Sometimes, faults cannot be quickly and accurately located -using only error codes. To facilitate error analysis, the error handling module supports the registration of the hook function for error handling. When an error occurs, you can call the **LOS_ErrHandle** API to execute the function for error handling.

The error handling module of Huawei LiteOS provides the following APIs. For details about the APIs, see the API reference.

API	Description	Parameter	Remarks
LOS_RegErrHandle	Registers the hook function for error handling.	func : hook function for error handling.	-
LOS_ErrHandle	Calls the hook function to handle the error.	fileName : name of the file that stores error logs.	During internal calling, the input parameter is os_unspecific_file .
		lineNo : line number of the code where an error occurs.	If the value is 0xa1b2c3f8 during internal calling, the line number is not transferred.
		errnoNo : error code.	-
		paraLen : length of the input parameter para .	During internal calling, the input parameter is 0 .
		para : error label.	During internal calling, the input parameter is NULL .

NOTICE

The system actively calls the registered hook function at some error points that are difficult to locate. (Currently, the hook function is actively called only in the mutex lock module and semaphore module.)

3.5.3 Precautions

There is only one hook function for error handling in the system. When the hook function is registered for multiple times, the latest registered hook function overwrites the previous registered hook function.

3.5.4 Programming Example

Example Description

The following functions are implemented:

1. Registering the hook function for error handling
2. Executing the function for error handling

Example Code

The code is as follows:

```
#include "los_err.h"
#include "los_typedef.h"
#include <stdio.h>

void Test_ErrHandle(CHAR *fileName, UINT32 lineNo, UINT32 errorNo, UINT32 paraLen, VOID *para)
{
    printf("err handle ok\n");
}

static UINT32 TestCase(VOID)
{
    UINT32 errNo = 0;
    UINT32 ret;
    UINT32 errLine = 16;

    LOS_RegErrHandle(Test_ErrHandle);

    ret = LOS_ErrHandle("os_unspecific_file", errLine, errNo, 0, NULL);
    if (ret != LOS_OK) {
        return LOS_NOK;
    }

    return LOS_OK;
}
```

Verification

The verification result is as follows:

```
Huawei LiteOS # err handle ok
```

Complete Code

sample_err.c

3.6 Queue

3.6.1 Overview

Basic Concepts

A queue, also called a message queue, is a data structure used for communication between tasks. The queue receives messages of unfixed length from tasks or interrupts, and determines whether to store the transferred messages in the queue space based on different APIs.

Tasks can read messages from a queue. When the queue has no messages, the tasks are suspended. When a new message arrives in the queue, the suspended tasks are woken up and process the new message. Tasks can also write messages to the queue. When the queue is full, the tasks are suspended. When there is an idle message node in the queue, the suspended write tasks are woken up and write a message. If the timeout intervals of the read queue and write queue are set to **0**, the tasks are not suspended and the APIs directly return messages. This is the non-block mode.

Message queue provides an asynchronous processing mechanism that allows a message to be put into a queue, but not processed immediately. In addition, the queue can buffer messages.

Huawei LiteOS uses queues to implement asynchronous task communication. It has the following features:

- Messages are queued in first-in-first-out (FIFO) mode and can be read and written asynchronously.
- Both the read queue and write queue support the timeout mechanism.
- Each time a message is read, the message node becomes idle.
- The types of messages to be sent are determined by both communication parties. Messages of different lengths (not exceeding the message node size of the queue) are allowed.
- A task can receive messages from and send messages to any message queue.
- Multiple tasks can receive messages from and send messages to the same message queue.
- The queue space required for creating a queue is dynamically allocated by the system by default or is allocated by the user and transferred to the API.

Operation Mechanism

Queue Control Block

```
typedef enum {
    OS_QUEUE_READ = 0,
    OS_QUEUE_WRITE = 1,
    OS_QUEUE_N_RW = 2
} QueueReadWrite;

/**
 * Queue information block structure
 */
typedef struct
{
    UINT8    *queueHandle;           /* Queue pointer. */
    UINT8    queueState;             /* Queue status. */
    UINT8    queueMemType;           /* Create a queue in memory allocation mode. */
}
```

```

    UINT16    queueLen;           /* Number of message nodes in the queue, that is, the queue
length. */
    UINT16    queueSize;         /* Size of a message node. */
    UINT32    queueID;           /* Queue ID. */
    UINT16    queueHead;         /* Position of the message head node (array subscript). */
    UINT16    queueTail;         /* Position of the message tail node (array subscript). */
    UINT16    readWriteableCnt[OS_QUEUE_N_RW]; /* Array subscript of the number of readable or
writable messages in the queue. 0: Number of readable messages.
                                1: Number of writable messages. */
    LOS_DL_LIST readWriteList[OS_QUEUE_N_RW]; /* Task wait linked list of read or write messages.
Subscript 0: Wait linked list of read messages; Subscript 1: Wait linked
list of write messages. */
    LOS_DL_LIST memList;         /* Memory block linked list used by the MailBox module in the
CMSIS-RTOS. */
} LosQueueCB;

```

Each queue control block contains the queue status, indicating the usage of the queue.

- **OS_QUEUE_UNUSED**: The queue is not used.
- **OS_QUEUE_INUSED**: The queue is in use.

Each queue control block contains the memory allocation mode during queue creation.

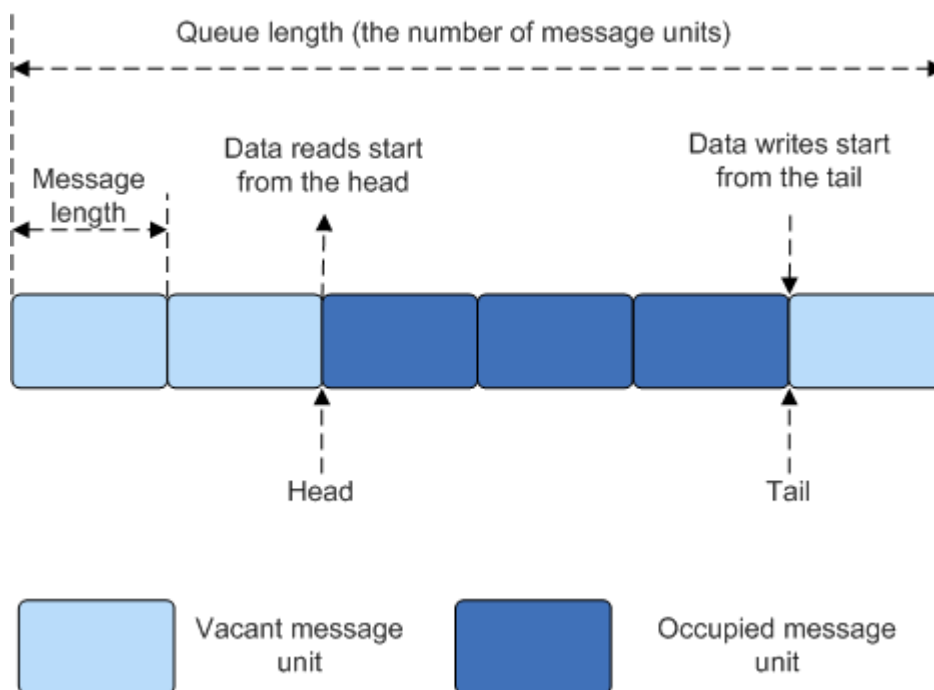
- **OS_QUEUE_ALLOC_DYNAMIC**: The queue space required for creating a queue is dynamically allocated by the system.
- **OS_QUEUE_ALLOC_STATIC**: The queue space required for creating a queue is allocated by the API caller and transferred to the API.

Queue Working Principles

Table 3-1

Function Category	Description
Creating a queue	The queue ID is returned if the queue is created successfully. In the queue control block, a message head node position Head and a message tail node position Tail are maintained to indicate a storage status of a message in a current queue. • Head indicates the start position of an occupied message node in the queue. • Tail indicates the end position of the occupied message node and the start position of an idle message node. (Note: When a queue is created, Head and Tail point to the start position of the queue.)

Function Category	Description
Writing data to a queue	When data is to be written to a queue, readWriteableCnt[1] is used to determine whether data can be written to the queue. Data cannot be written to a full queue (that is, readWriteableCnt[1] is 0). The write queue supports two write modes. • writing to the tail node of the queue: When data is written to the tail node, find the start idle message node based on the Tail as the data writing object. If the Tail has pointed to the tail of the queue, the rewind mode is used. • writing to the head node of the queue: When the head node is written, the previous node of the Head functions as the data writing object. If the Head points to the start position of the queue, rewinding starts.
Reading a queue	When a queue is read, readWriteableCnt[0] is used to determine whether messages in the queue need to be read. If all idle queues (that is, readWriteableCnt[0] is 0) are read, the tasks are suspended. • If the queue can read messages, read the message on the message node that is first written to the queue, which is found based on the Head. • If the Head points to the tail of the queue, rewinding starts.
Deleting a queue	To delete a queue, find the queue based on the queue ID, set the queue status to unused, and set the queue control block to the initial state. If the queue created using the memory is dynamically allocated by the system is deleted, the memory occupied by the queue is released.

Figure 3-8 Reading and writing data in a queue

The preceding figure shows the read and write queues. Only the mode of writing data to the tail node is shown in the figure. The mode of writing data to the head node is similar.

3.6.2 Development Guidelines

Application Scenario

Queues are used for communication between tasks to implement asynchronous message processing. In addition, the message sender and recipient do not need to communicate with each other because they are decoupled.

Functions

The queue module of Huawei LiteOS provides the following functions. For details about the APIs, see the API reference.

Function Category	API	Description
Creating or deleting a message queue	LOS_QueueCreate	Creates a message queue whose the queue space is dynamically allocated by the system.
	LOS_QueueCreateStatic	Creates a message queue whose queue space is allocated by the user and transferred to the API.
	LOS_QueueDelete	Deletes a specified queue based on the queue ID.

Function Category	API	Description
Reading or writing data in a queue (without copy)	LOS_QueueRead	Reads the specified head node data in the queue. The node data in a queue is an address.
	LOS_QueueWrite	Writes the value of the input parameter bufferAddr (that is, the buffer address) to the tail node of a specified queue.
	LOS_QueueWriteHead	Write the value of the input parameter bufferAddr (that is, the buffer address) to the head node of a specified queue.
Reading or writing data in a queue (with copy)	LOS_QueueReadCopy	Reads data from the head node of a specified queue.
	LOS_QueueWriteCopy	Writes the data saved in the input parameter bufferAddr to the tail node of a specified queue.
	LOS_QueueWriteHeadCopy	Writes the data saved in the input parameter bufferAddr to the head node of a specified queue.
Obtaining queue information	LOS_QueueInfoGet	Obtains information about a specified queue, including the queue ID, queue length, message node size, head node, tail node, number of readable nodes, number of writable nodes, tasks waiting for read operations, tasks waiting for write operations, and tasks waiting for mail operations.

Queue Error Codes

Error codes are returned for operations that may fail to be performed to quickly locate error causes.

N o.	Definition	Error Code	Description	Reference Solution
1	LOS_ERRNO_QUEUE_NO_MEMORY	0x02000601	Failed to allocate memory from the dynamic memory pool during queue initialization.	Set the configuration item OS_SYS_MEM_SIZE to a larger system dynamic memory pool, or reduce the maximum number of queues supported by the system.

N o.	Definition	Error Code	Description	Reference Solution
2	LOS_ERRNO_QUEUE_CREATE_NO_MEMORY	0x02000602	Failed to allocate memory from the dynamic memory pool during queue creation.	Set OS_SYS_MEM_SIZE to a larger system dynamic memory pool, or reduce the length and message node size of the queue to be created.
3	LOS_ERRNO_QUEUE_SIZE_TOO_BIG	0x02000603	The message node size exceeds the upper limit during queue creation.	Change the input parameter for message node size to ensure that the size does not exceed the upper limit.
4	LOS_ERRNO_QUEUE_CB_UNAVAILABLE	0x02000604	No idle queue exists in the system during queue creation.	Increase the maximum number of queues supported by the system.
5	LOS_ERRNO_QUEUE_NOT_FOUND	0x02000605	The queue ID transferred to the API for deleting a queue is greater than or equal to the maximum number of queues supported by the system.	Ensure that the queue ID is valid.
6	LOS_ERRNO_QUEUE_PENDING_LOCK	0x02000606	When a task is locked, the queue used by the task cannot block the wait for message writing or reading.	Unlock the locked task before using a queue.
7	LOS_ERRNO_QUEUE_TIMEOUT	0x02000607	Waiting for processing the queue timed out.	Check that the configured timeout interval is proper.
8	LOS_ERRNO_QUEUE_INTERRUPTED	0x02000608	The queue is involved in a blocked task and cannot be deleted.	Enables tasks to obtain resources instead of being blocked in the queue.
9	LOS_ERRNO_QUEUE_WRITE_INTERRUPT	0x02000609	The queue cannot be written in block mode in the interrupt handler.	Set the write queue to non-block mode, that is, set the timeout interval of the write queue to 0 .

N o.	Definition	Error Code	Description	Reference Solution
10	LOS_ERRNO_QUEUE_NOT_CREATE	0x0200060a	The queue has not been created.	Create the queue or replace it with an existing queue.
11	LOS_ERRNO_QUEUE_IN_TS_KWRITE	0x0200060b	The read and write operations in the queue are not synchronized.	Synchronize the read and write operations in the queue. Specifically, multiple tasks are not allowed to concurrently read data from or write data into the same queue.
12	LOS_ERRNO_QUEUE_CREAT_PTR_NULL	0x0200060c	For the API for creating a queue, the input parameter for saving the queue ID is a null pointer.	Ensure that the input parameter is not a null pointer.
13	LOS_ERRNO_QUEUE_PARAMETER_ISZERO	0x0200060d	For the API for creating a queue, the queue length or message node size is 0.	Input correct queue length and message node size.
14	LOS_ERRNO_QUEUE_INVALID	0x0200060e	The queue ID transferred to the API for reading data from a queue, writing data to queue, or obtaining the queue information is greater than or equal to the maximum number of queues supported by the system.	Ensure that the queue ID is valid.
15	LOS_ERRNO_QUEUE_READ_PTR_NULL	0x0200060f	The pointer transferred to the read queue API is null.	Ensure that the input parameter is not a null pointer.

N o.	Definition	Error Code	Description	Reference Solution
1 6	LOS_ERRNO_QUEUE_READ_SIZE_IS_INVALID	0x02000610	The size of the buffer transferred to the read queue API is 0 or greater than 0xFFFFB.	The input buffer size must be greater than 0 and less than 0xFFFFC.
1 7	LOS_ERRNO_QUEUE_WRITE_PTR_NULL	0x02000612	The buffer pointer transferred to the write queue API is null.	Ensure that the input parameter is not a null pointer.
1 8	LOS_ERRNO_QUEUE_WRITE_SIZE_ISZERO	0x02000613	The size of the buffer transferred to the write queue API is 0.	Input a correct buffer size.
1 9	LOS_ERRNO_QUEUE_WRITE_SIZE_TOO_BIG	0x02000615	The size of the buffer transferred to the write queue API is greater than the message node size of the queue.	Reduce the buffer size or increase the message node size of the queue.
2 0	LOS_ERRNO_QUEUE_ISFULL	0x02000616	No idle node is available when data is written to the queue.	Before writing data to a queue, ensure that there is an available idle node in the queue or use the block mode to write data to the queue. That is, set the timeout interval of the write queue to a value greater than 0.
2 1	LOS_ERRNO_QUEUE_PTR_NULL	0x02000617	The pointer transferred to the API for obtaining queue information is null.	Ensure that the input parameter is not a null pointer.
2 2	LOS_ERRNO_QUEUE_READ_IN_INTERRUPT	0x02000618	The queue cannot be read in block mode in an interrupt handler.	Set the read queue to non-block mode, that is, set the timeout interval of the read queue to 0 .

N o.	Definition	Error Code	Description	Reference Solution
2 3	LOS_ERRNO_QUEUE_MAIL_HANDLE_INVALID	0x02000619	When the memory block of the mail queue in CMSIS-RTOS 1.0 is released, the input mail queue ID is invalid.	Ensure that the input mail queue ID is correct.
2 4	LOS_ERRNO_QUEUE_MAIL_PTR_INVALID	0x0200061a	When the memory block of the mail queue in CMSIS-RTOS 1.0 is released, the pointer of the input mail memory pool is null.	Input non-null mail memory pool pointer.
2 5	LOS_ERRNO_QUEUE_MAIL_FREE_ERROR	0x0200061b	The memory block of the mail queue in CMSIS-RTOS 1.0 failed to be released.	Input non-null memory block pointer of the mail queue.
2 6	LOS_ERRNO_QUEUE_ISEMPTY	0x0200061d	The queue is empty.	Before reading a queue, ensure that there are unread messages in the queue or use the block mode to read the queue, that is, set the read queue timeout interval to a value greater than 0.
2 7	LOS_ERRNO_QUEUE_READ_SIZE_TOO_SMALL	0x0200061f	The size of the read buffer transferred to the read queue API is less than the message node size of the queue.	Increase the buffer size or reduce the message node size of the queue.

NOTICE

- For details about the error code definition, see [Error Codes](#). Bits 8–15 belong to the queue module, and the value is 0x06.
- The error codes 0x11 and 0x14 in the queue module are not defined and cannot be used.

Development Procedure

The typical process of using the queue module is as follows:

1. Open the menu and choose **Kernel > Enable Queue** to configure the queue module.

Configuration Item	Description	Value Range	Default Value	Dependency
LOSCFG_BASE_IPC_QUEUE	Whether to tailor the queue module.	YES/NO	YES	None.
LOSCFG_QUEUE_STATIC_ALLOCATION	Whether to create a queue whose memory is allocated by users.	YES/NO	NO	LOSCFG_BASE_IPC_QUEUE
LOSCFG_BASE_IPC_QUEUE_LIMIT	Maximum number of queues supported by the system.	<65535 >	1024	LOSCFG_BASE_IPC_QUEUE

2. Create a queue. After the queue is created, you can obtain the queue ID.
3. Write data to the queue.
4. Read the queue.
5. Obtain the queue information.
6. Delete the queue.

Platform Differences

Different platforms may have different data alignment requirements. For example, on the 3559a platform, the data written to the queue needs to be aligned with eight bytes.

3.6.3 Precautions

Queue creation

- The maximum number of queues supported by the system refers to the total number of queue resources in the entire system instead of the number of queue resources that can be used by users. For example, if the system software timer occupies one more queue resource, the number of available queue resources decreases by one.
- The queue name and flags input during queue creation are not used currently and are reserved for future use.

Queue creation

- LOS_QueueReadCopy, LOS_QueueWriteCopy, and LOS_QueueWriteHeadCopy are a group of APIs, and LOS_QueueRead, LOS_QueueWrite, and LOS_QueueWriteHead are another group of APIs. The two groups of APIs must be used together.
- As the LOS_QueueWrite, LOS_QueueWriteHead, and LOS_QueueRead APIs are called to perform operations on data addresses, you must ensure that the

memory area to which the pointer obtained by calling `LOS_QueueRead` points is not modified or released abnormally when the queue is being read. Otherwise, unpredictable results may occur.

- The `LOS_QueueWrite`, `LOS_QueueWriteHead`, and `LOS_QueueRead` APIs operate on data addresses, that is, the length of the message that is written or read is only a pointer. Therefore, before using these APIs, you need to ensure the size of the message node when the queue is created, length of a pointer to avoid unnecessary waste and read failures.

Others

- The input parameter `timeout` in the queue API function is a relative time.
- After the queue is used, if the memory that is dynamically allocated exists, the memory needs to be released in a timely manner.

3.6.4 Programming Example

Example Description

Create a queue and two tasks. Task 1 calls the queue write API to send messages, and task 2 receives messages through the queue read API.

1. Create task 1 and task 2 by calling `LOS_TaskCreate`.
2. Create a message queue by calling `LOS_QueueCreate`.
3. Send a message in task 1 the `send_Entry`.
4. Receive messages in task 2 `recv_Entry`.
5. Delete the queue by calling `LOS_QueueDelete`.

Example Code

Prerequisite: The queue module has been configured in the **menuconfig** menu.

```
#include "los_task.h"
#include "los_queue.h"

static UINT32 g_queue;
#define BUFFER_LEN 50

VOID send_Entry(VOID)
{
    UINT32 i = 0;
    UINT32 ret = 0;
    CHAR abuf[] = "test is message x";
    UINT32 len = sizeof(abuf);

    while (i < 5) {
        abuf[len - 2] = '0' + i;
        i++;

        ret = LOS_QueueWriteCopy(g_queue, abuf, len, 0);
        if (ret != LOS_OK) {
            dprintf("send message failure, error: %x\n", ret);
        }

        LOS_TaskDelay(5);
    }
}

VOID recv_Entry(VOID)
```

```

{
    UINT32 ret = 0;
    CHAR readBuf[BUFFER_LEN] = {0};
    UINT32 readLen = BUFFER_LEN;

    while (1) {
        ret = LOS_QueueReadCopy(g_queue, readBuf, &readLen, 0);
        if (ret != LOS_OK) {
            dprintf("recv message failure, error: %x\n", ret);
            break;
        }

        dprintf("recv message: %s\n", readBuf);
        LOS_TaskDelay(5);
    }

    while (LOS_OK != LOS_QueueDelete(g_queue)) {
        LOS_TaskDelay(1);
    }

    dprintf("delete the queue success!\n");
}

UINT32 Example_CreateTask(VOID)
{
    UINT32 ret = 0;
    UINT32 task1, task2;
    TSK_INIT_PARAM_S initParam;

    initParam.pfnTaskEntry = (TSK_ENTRY_FUNC)send_Entry;
    initParam.usTaskPrio = 9;
    initParam.uwStackSize = LOS_TASK_MIN_STACK_SIZE;
    initParam.pcName = "sendQueue";
#ifdef LOSCFG_KERNEL_SMP
    initParam.usCpuAffiMask = CPUID_TO_AFFI_MASK(ArchCurrCpuId());
#endif
    initParam.uwResved = LOS_TASK_STATUS_DETACHED;

    LOS_TaskLock();
    ret = LOS_TaskCreate(&task1, &initParam);
    if (ret != LOS_OK) {
        dprintf("create task1 failed, error: %x\n", ret);
        return ret;
    }

    initParam.pcName = "recvQueue";
    initParam.pfnTaskEntry = (TSK_ENTRY_FUNC)recv_Entry;
    ret = LOS_TaskCreate(&task2, &initParam);
    if (ret != LOS_OK) {
        dprintf("create task2 failed, error: %x\n", ret);
        return ret;
    }

    ret = LOS_QueueCreate("queue", 5, &g_queue, 0, BUFFER_LEN);
    if (ret != LOS_OK) {
        dprintf("create queue failure, error: %x\n", ret);
    }

    dprintf("create the queue success!\n");
    LOS_TaskUnlock();
    return ret;
}

```

Verification

```

create the queue success!
recv message: test is message 0
recv message: test is message 1
recv message: test is message 2

```

```
recv message: test is message 3
recv message: test is message 4
recv message failure, error: 200061d
delete the queue success!
```

Complete Code

sample_queue.c

3.7 Event

3.7.1 Overview

Basic Concepts

An event is a mechanism for communication between tasks. It can be used to synchronize tasks.

In multi-task environment, synchronization is required between tasks. A waiting task is a synchronization task. Events can be synchronized in one-to-many or many-to-many mode.

- In a multi-task environment, tasks must be synchronized. In the one-to-many synchronization mode, a task waits for multiple events. In the many-to-many synchronization mode, multiple tasks wait for multiple events.
- Tasks trigger or wait for events through event control blocks.

Characteristics of events in Huawei LiteOS are as follows:

- A task can trigger and wait for an event by creating an event control block.
- Events are not associated with tasks and are independent from each other. A 32-bit unsigned integer variable is used to indicate the type of the event in which a task is triggered. Each bit indicates one event type. Bit 25 is unavailable. Therefore, a maximum of 31 event types are supported.
- Events are used only for task synchronization, and not for data transmission.
- Writing the same event type to the event control book for multiple times is equivalent to writing for only once before clearing.
- Multiple tasks are allowed to read or write the same event.
- Huawei LiteOS supports event reading and writing timeout.

Event control block

```
/**
 * Event control structure
 */
typedef struct tagEvent {
    UINT32 uwEventID; /* bit that indicates an event type */
    LOS_DL_LIST stEventList; /* linked list of event reading tasks */
} EVENT_CB_S, *PEVENT_CB_S;
```

uwEventID indicates the type of the event in which a task is triggered. Each bit indicates one event type (**0** indicates that the event does not occur and **1** indicates that the event occurs). There are 31 bits that indicate event types (bit 25 is reserved).

Event reading mode

An event reading mode can be configured during event reading. Event reading modes are as follows:

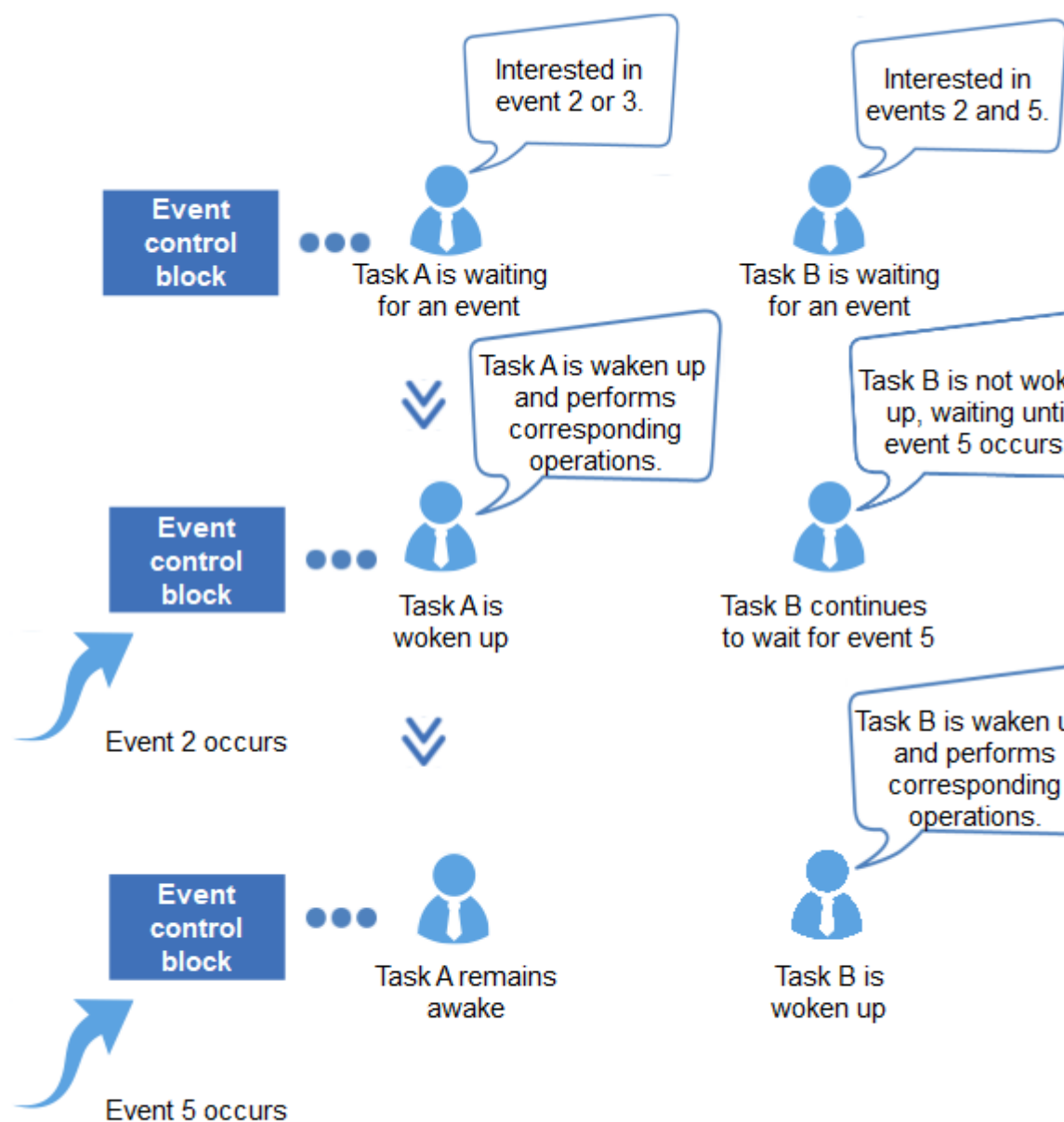
- All events (LOS_WAITMODE_AND): indicate that events of all event types specified by a mask need to be read. Event reading succeeds only when all events that are read occur. Otherwise, the task is blocked and waits, or an error code is returned.
- Any event (LOS_WAITMODE_OR): indicates that an event of an event type specified by a mask needs to be read. Event reading succeeds when the event that is read occurs. Otherwise, the task is blocked and waits, or an error code is returned.
- Clear event (LOS_WAITMODE_CLR): indicates an additional read mode that needs to be used with all events mode or any event mode (LOS_WAITMODE_AND | LOS_WAITMODE_CLR or LOS_WAITMODE_OR | LOS_WAITMODE_CLR). In this mode, after all events mode or any event mode that has been set is successfully read, the corresponding event type bit in the electronic code book is automatically cleared.

Operation Mechanism

When the LOS_EventRead API is called to read an event, task can read one or more event types based on the input parameter **eventMask**. After the event is read successfully, if LOS_WAITMODE_CLR is set, the read event type is cleared. Otherwise, the read event type is not cleared and needs to be cleared explicitly. You can select a read mode (reading all events or reading any event in the event mask type) based on the input parameter.

When the LOS_EventWrite API is called to write an event, a specified event type is written into a specified electronic code book. Multiple event types can be written at a time. Event writing may trigger task scheduling.

When the LOS_EventClear API is called to clear an event, the corresponding bit of the event is cleared based on the input parameter event type and the type of the event to be cleared.

Figure 3-9 Tasks woken up by events

3.7.2 Development Guidelines

Application Scenario

Events are applicable in a variety of task synchronization scenarios and are partially similar to semaphore in some scenarios.

Functions

The event module of Huawei LiteOS provides the following functions. For details about the APIs, see the API reference.

Function Category	API	Description
Event initialization	LOS_EventInit	Initializes an event control block.
Event reading/ writing	LOS_EventRead	Reads events of a specified type. The timeout period is a relative time. The unit is Tick.
	LOS_EventWrite	Writes events of a specified type.
	LOS_EventCondRead	Reads the events that meet the specified conditions. The timeout period is a relative time. The unit is Tick. This API is used with LOS_EventCondWrite.
	LOS_EventCondWrite	Writes a specified event. This API is used with LOS_EventCondRead.
Event clearance	LOS_EventClear	Clears events of a specified type.
Event mask verification	LOS_EventPoll	Determines whether an event meets the expectations based on the passed-in event ID, event mask, and read mode.
Event destroying	LOS_EventDestroy	Destroys a specified event control block.

Event Error Code

Error codes are returned for operations that may fail to be performed to quickly locate error causes.

No.	Definition	Error Code	Description	Reference Solution
1	LOS_ERRNO_EVENT_SETBIT_INVALID	0x02001c00	During event writing, bit 25 of the event ID is set to 1 . This bit is reserved in the OS and cannot be set to 1 .	Set bit 25 of the event ID to 0 .
2	LOS_ERRNO_EVENT_READ_TIMEOUT	0x02001c01	Event reading times out.	Increase the permitted wait time. Alternatively, re-read the event.

N o.	Definition	Error Code	Description	Reference Solution
3	LOS_ERRNO_EVENT_EVENTMA SK_INVALID	0x02001c02	The passed-in event ID is invalid.	Pass in a valid event ID.
4	LOS_ERRNO_EVENT_READ_IN_I NTERRUPT	0x02001c03	The event is being read when an interrupt is being processed.	Start a new task to obtain events.
5	LOS_ERRNO_EVENT_FLAGS_IN VALID	0x02001c04	The mode of event reading is invalid.	Pass in a valid mode.
6	LOS_ERRNO_EVENT_READ_IN_ LOCK	0x02001c05	The task is locked and fails to read the event.	Unlock the task, and then let the task read the event.
7	LOS_ERRNO_EVENT_PTR_NULL	0x02001c06	The passed-in pointer is null.	Pass in a non-null pointer.
8	LOS_ERRNO_EVENT_SHOULD_ NOT_DESTORY	0x02001c08	The linked list of events still contains tasks and cannot be destroyed.	Check whether the linked list of events is empty.

NOTICE

For details about the error code definition, see [Error Codes](#). Bits 8–15 belong to the event module, and the value is **0x1c**.

Development Procedure

The typical process of using the event module is as follows:

1. Open the menu and choose **Kernel > Enable Event** to configure the event module.

Configuration Item	Description	Value Range	Default Value	Dependency
LOSCFG_BASE_IPC_EVENT	Whether to enable the event tailoring function.	YES/NO	YES	None.

2. Call the `LOS_EventInit` API to initialize an event control block.
3. Call the `LOS_EventWrite` or `LOS_EventCondWrite` API to write an event or a specified event type.
4. Call the `LOS_EventRead` or `LOS_EventCondRead` API to read a specified event type or a conditional event.

5. Call the `LOS_EventClear` to clear a specified event type. This API applies only to events of a specified event type.
6. Call the `LOS_EventDestroy` API to destroy event.

Platform Differences

None.

3.7.3 Precautions

- Do not make calls to the `LOS_EventRead` and `LOS_EventWrite` APIs prior to the operating system being initialized. If this API is called, the system will not run properly.
- While an interrupt is underway, events can be written into an event control block but event reads are not allowed.
- Task blocking and event reading are not allowed while task scheduling is locked.
- Bit 25 of the event mask is merely used to distinguish whether the `LOS_EventRead` API returns an event or error code.
- The input parameter of `LOS_EventClear` is `~events` (reverse code of the type of events to be cleared).
- `LOS_EventWrite` and `LOS_EventRead` are used together, and `LOS_EventCondWrite` and `LOS_EventCondRead` are used together. The difference between the two pairs is that `LOS_EventWrite` and `LOS_EventRead` APIs are used to read or write events of a specified event type, while `LOS_EventCondWrite` and `LOS_EventCondRead` are used to read or write events with specified event conditions. The event conditions are implemented by users. Event writing task does not need to care about the event conditions. Instead, the event writing task broadcasts the event reading task, that is, wakes up the task reading. The reading task determines whether the reading event conditions are successful. This may incur some performance overhead, but it can simplify the event writing task.

3.7.4 Programming Example

3.7.4.1 Read and Write Based on the Event Type

Example Description

In the programming example, the `Example_TaskEntry` task is executed to create the `Example_Event` task. The `Example_Event` task is blocked from reading events. The `Example_TaskEntry` task writes events to the task. You can understand the task switching during event operations based on the sequence in which logs are recorded.

1. The `Example_TaskEntry` task is executed to create the `Example_Event` task. The `Example_Event` task takes a higher priority than the `Example_TaskEntry` task.
2. The `Example_Event` task is blocked from reading the event `0x00000001`. After the `Example_Event` task is blocked, a task switching occurs to execute the task with a lower priority, namely, the `Example_TaskEntry` task.

3. The Example_TaskEntry task writes the event 0x00000001 toward the Example_Event task. Task switching occurs, and the Example_Event task is executed.
4. The Example_Event task is executed until the task is complete.
5. The Example_TaskEntry task is executed.

Example Code

Prerequisite: The event module has been configured in the **menuconfig** menu.

The code is as follows:

```
#include "los_event.h"
#include "los_task.h"
#include "securec.h"

/* Task ID */
UINT32 g_testTaskId;

/* Event control structure */
EVENT_CB_S g_exampleEvent;

/* Event that the Example_Event task is waiting for */
#define EVENT_WAIT 0x00000001

/* Example task entrypoint function */
VOID Example_Event(VOID)
{
    UINT32 ret;
    UINT32 event;

    /* Wait for a completion in timeout mode, and the timeout interval is 100 ticks. If the event is not read
    within 100 ticks, the read operation times out and the task is woken up. */
    printf("Example_Event wait event 0x%x\n", EVENT_WAIT);

    event = LOS_EventRead(&g_exampleEvent, EVENT_WAIT, LOS_WAITMODE_AND, 100);
    if (event == EVENT_WAIT) {
        printf("Example_Event,read event :0x%x\n", event);
    } else {
        printf("Example_Event,read event timeout\n");
    }
}

UINT32 Example_TaskEntry(VOID)
{
    UINT32 ret;
    TSK_INIT_PARAM_S task1;

    /* Initialize the event. */
    ret = LOS_EventInit(&g_exampleEvent);
    if (ret != LOS_OK) {
        printf("init event failed .\n");
        return -1;
    }

    /* Create the task. */
    (VOID)memset_s(&task1, sizeof(TSK_INIT_PARAM_S), 0, sizeof(TSK_INIT_PARAM_S));
    task1.pfnTaskEntry = (TSK_ENTRY_FUNC)Example_Event;
    task1.pcName = "EventTsk1";
    task1.uwStackSize = OS_TSK_DEFAULT_STACK_SIZE;
    task1.usTaskPrio = 5;
    ret = LOS_TaskCreate(&g_testTaskId, &task1);
    if (ret != LOS_OK) {
        printf("task create failed .\n");
        return LOS_NOK;
    }
}
```

```
/* Write the g_testTaskId wait event. */
printf("Example_TaskEntry write event .\n");

ret = LOS_EventWrite(&g_exampleEvent, EVENT_WAIT);
if (ret != LOS_OK) {
    printf("event write failed .\n");
    return LOS_NOK;
}

/* Clear the flag. */
printf("EventMask:%d\n", g_exampleEvent.uwEventID);
LOS_EventClear(&g_exampleEvent, ~g_exampleEvent.uwEventID);
printf("EventMask:%d\n", g_exampleEvent.uwEventID);

/* Delete the task. */
ret = LOS_TaskDelete(g_testTaskId);
if (ret != LOS_OK) {
    printf("task delete failed .\n");
    return LOS_NOK;
}

return LOS_OK;
}
```

Verification

The verification result is as follows:

```
Example_Event wait event 0x1
Example_TaskEntry write event .
Example_Event,read event :0x1
EventMask:1
EventMask:0
```

Complete Code

sample_event.c

3.7.4.2 Read and Write Based on Event Conditions

Example Description

In the programming example, the ExampleTaskEntry task is executed to create readTask1, readTask2, and writeTask. The writeTask sets the waiting conditions of readTask1 and readTask2 to true in sequence and writes events. If the events read by readTask1 and readTask2 meet the expectation, the blocking is ended. You can understand the task switching during event operations based on the sequence in which logs are recorded.

1. Create two reading event tasks readTask1 and readTask2 in ExampleTaskEntry task.
2. The waiting condition EventCondition1 of readTask1 is false and the target event is not read and does not time out, thus the task is blocked.
3. The waiting condition EventCondition2 of readTask2 is false and the target event is not read and does not time out, thus the task is blocked.
4. Create an event writing task writeTask in ExampleTaskEntry task.
5. writeTask sets the waiting condition of readTask1 to true and writes the event.

6. Both readTask1 and readTask2 are woken up. Only the condition of readTask1 is true. Therefore, readTask1 successfully reads and executes the event. readTask2 fails to read the event and is blocked.
7. writeTask sets the waiting condition of readTask2 to true and writes the event.
8. readTask2 is woken up and its waiting condition is true. Therefore, readTask2 successfully reads and executes the event.

Example Code

Prerequisite: The event module has been configured in the **menuconfig** menu.

The code is as follows:

```
#include "los_event.h"
#include "los_task.h"
#include "securec.h"

/* Task ID */
static UINT32 g_readTskId1;
static UINT32 g_readTskId2;
static UINT32 g_writeTskId;

/* Waiting conditions for g_readTskId1 and g_readTskId2 */
static int g_testData1 = 0;
static int g_testData2 = 0;

/* Event control structure */
static EVENT_CB_S g_event;

/* Entry function of the event writing task */
static VOID WriteTaskFunc(VOID)
{
    UINT32 ret;

    g_testData1 = 1;
    printf("\nWriteTaskFunc: write for ReadTaskFunc1\n\n");
    ret = LOS_EventCondWrite(&g_event);
    if (ret != LOS_OK) {
        printf("WriteTaskFunc failed: ret = %u\n", ret);
        return;
    }
    LOS_TaskDelay(2); /* 2: delay 2 ticks */

    printf("\nWriteTaskFunc: write for ReadTaskFunc2\n\n");
    g_testData2 = 1;
    ret = LOS_EventCondWrite(&g_event);
    if (ret != LOS_OK) {
        printf("WriteTaskFunc failed: ret = %u\n", ret);
        return;
    }
    LOS_TaskDelay(2); /* 2: delay 2 ticks */

    return;
}

/* Event condition expected to be read by event reading task 1 */
static BOOL EventCondition1(VOID)
{
    BOOL ret;

    ret = (g_testData1 == 1) ? TRUE : FALSE;
    printf("EventCondition1: ret = %s\n", (ret ? "TRUE" : "FALSE"));
    return ret;
}
```

```

/* Entry function of event reading task 1 */
static VOID ReadTaskFunc1(VOID)
{
    UINT32 ret;

    ret = LOS_EventCondRead(&g_event, EventCondition1(), LOS_WAIT_FOREVER);
    if (ret != LOS_OK) {
        printf("ReadTaskFunc1 read failed!\n");
        return;
    }

    printf("ReadTaskFunc1 read succeeded!\n");
    return;
}

/* Event condition expected to be read by event reading task 2 */
static BOOL EventCondition2(VOID)
{
    BOOL ret;

    ret = (g_testData2 == 1) ? TRUE : FALSE;
    printf("EventCondition2: ret = %s\n", (ret ? "TRUE" : "FALSE"));
    return ret;
}

/* Entry function of event reading task 2 */
static VOID ReadTaskFunc2(VOID)
{
    UINT32 ret;

    ret = LOS_EventCondRead(&g_event, EventCondition2(), LOS_WAIT_FOREVER);
    if (ret != LOS_OK) {
        printf("ReadTaskFunc2 read failed!\n");
        return;
    }

    printf("ReadTaskFunc2 read succeeded!\n");
    return;
}

static VOID InitTaskParam(TSK_INIT_PARAM_S *task, char *taskName, TSK_ENTRY_FUNC entry, UINT16
prio, UINT16 affi)
{
    task->pcName      = taskName;
    task->pfnTaskEntry = entry;
    task->usTaskPrio   = prio;
    task->uwStackSize  = LOSCFG_BASE_CORE_TSK_DEFAULT_STACK_SIZE;
    task->uwResved     = LOS_TASK_STATUS_DETACHED;

#ifdef LOSCFG_KERNEL_SMP
    task->usCpuAffiMask = affi;
#endif
}

static UINT32 ExampleTaskEntry(VOID)
{
    UINT32 ret;
    TSK_INIT_PARAM_S readTask1 = {0};
    TSK_INIT_PARAM_S readTask2 = {0};
    TSK_INIT_PARAM_S writeTask = {0};

    ret = LOS_EventInit(&g_event);
    if (ret != LOS_OK) {
        return ret;
    }

    InitTaskParam(&readTask1, "EventReadTsk1", (TSK_ENTRY_FUNC)ReadTaskFunc1, 5,
CPUID_TO_AFFI_MASK(ArchCurrCpuId()));
    ret = LOS_TaskCreate(&g_readTskId1, &readTask1);

```

```
    if (ret != LOS_OK) {  
        return ret;  
    }  
  
    InitTaskParam(&readTask2, "EventReadTsk2", (TSK_ENTRY_FUNC)ReadTaskFunc2, 5,  
CPUID_TO_AFFI_MASK(ArchCurrCpuId()));  
    ret = LOS_TaskCreate(&g_readTskId2, &readTask2);  
    if (ret != LOS_OK) {  
        return ret;  
    }  
  
    InitTaskParam(&writeTask, "EventWriteTsk45", (TSK_ENTRY_FUNC)WriteTaskFunc, 5,  
CPUID_TO_AFFI_MASK(ArchCurrCpuId()));  
    ret = LOS_TaskCreate(&g_writeTskId, &writeTask);  
    if (ret != LOS_OK) {  
        return ret;  
    }  
  
    (VOID)LOS_TaskDelay(6); /* 6: delay 6 ticks */  
  
    (VOID)LOS_EventDestroy(&g_event);  
    return LOS_OK;  
}
```

Verification

The verification result is as follows:

```
EventCondition1: ret = FALSE  
EventCondition2: ret = FALSE  
  
WriteTaskFunc: write for ReadTaskFunc1  
  
EventCondition1: ret = TRUE  
ReadTaskFunc1 read succeeded!  
EventCondition2: ret = FALSE  
  
WriteTaskFunc: write for ReadTaskFunc2  
  
EventCondition2: ret = TRUE  
EventCondition2: ret = TRUE  
ReadTaskFunc2 read succeeded!
```

Complete Code

sample_event_cond.rar

3.8 Semaphore

3.8.1 Overview

Basic Concepts

A semaphore is a mechanism used for inter-task communication, to achieve synchronization or mutual exclusion (mutex) of access to shared resources between tasks.

Typically, the counter value of a semaphore corresponds to the number of valid resources, indicating the number of remaining shared resources that can be occupied. There are two types of counter values:

- 0: It means a semaphore that cannot be obtained. Therefore, tasks waiting for the semaphore may exist.
- Positive number: It means a semaphore that can be obtained.

A semaphore for synchronization is different from that for mutex in usage:

- When semaphores are used for mutex, the initial semaphore counter value is not 0, indicating the number of available shared resources. For using a shared resource, you need to obtain the semaphore, use the shared resource, and then release the semaphore after using it. In this way, when all shared resources are obtained, that is, the semaphore counter value is reduced to 0, other tasks that need to obtain semaphores are blocked, thereby ensuring exclusive access to the shared resources. In addition, when the number of shared resources is 1, you are advised to use binary semaphores. The maximum count value of a binary semaphore is 1, which is similar to the mutex mechanism.
- If semaphores are used for synchronization, the initial semaphore counter value is 0. When task1 attempts to get the semaphore, it is blocked because the counter value has reached the maximum. Task1 will enter Ready or Running state after task2 releases the semaphore, thereby achieving task synchronization.

Operation Mechanism

Semaphore control block

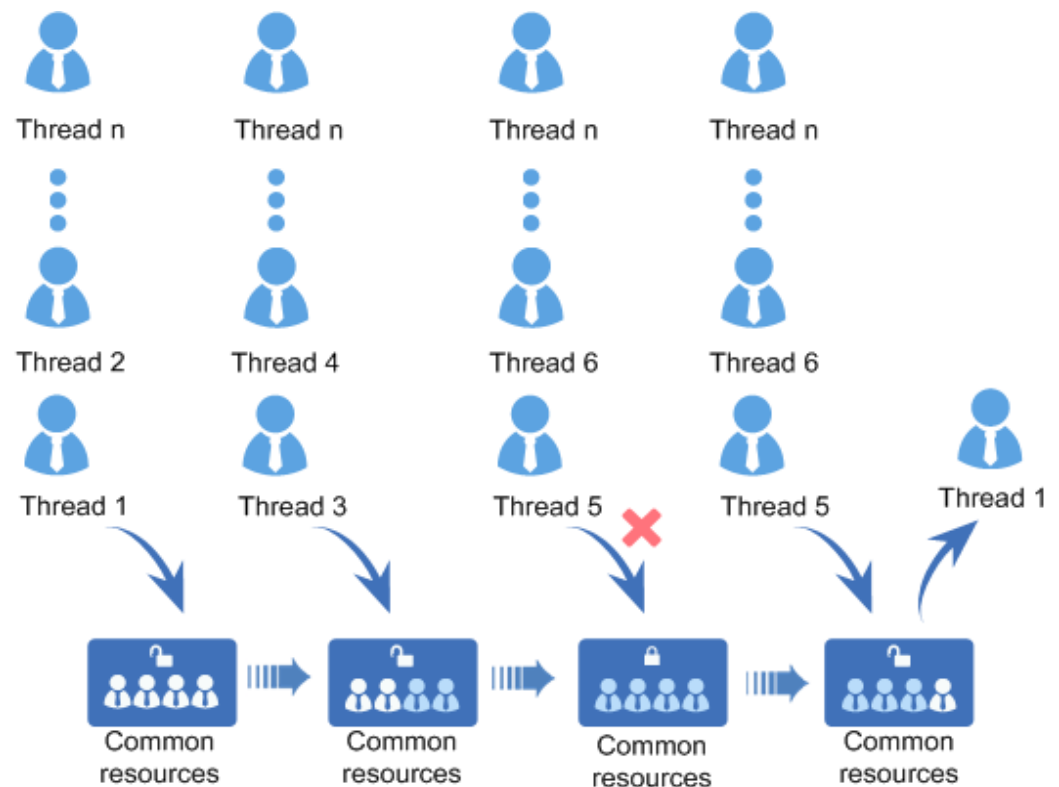
```
/**
 * Semaphore control structure.
 */
typedef struct {
    UINT8    semStat;      /* Whether to use flag bit */
    UINT8    semType;      /* Semaphore type */
    UINT16    semCount;     /* Semaphore count */
    UINT32    semID;       /* Semaphore quantity index */
    LOS_DL_LIST semList;    /* Suspend the task blocked on the semaphore */
} LosSemCB;
```

Semaphore operation principle

- Semaphore initialization: During semaphore initialization, memory is allocated to N semaphores (N is configurable and can be implemented using the `LOSCFG_BASE_IPC_SEM_LIMIT` macro.) All semaphores are initialized and added to the linked list of semaphores that are not in use.
- Semaphore creation: During semaphore creation, a semaphore is obtained from the linked list of semaphores that are not in use and the initial value of the semaphore is set.
- Semaphore allocation: During semaphore allocation, if the counter value of a semaphore is greater than 0, the counter value is decreased by 1 and then returned. Otherwise, tasks are blocked from getting the semaphore and wait for other tasks to post the semaphore. The timeout interval of waiting on the semaphore can be configured. If a task is blocked from getting the semaphore, suspend the task to the tail of the queue of tasks waiting on the semaphore.
- Semaphore release: If no tasks are waiting on a semaphore, the counter value is increased by 1 and the semaphore is posted. Otherwise, wake up the first task in the queue of tasks waiting on the semaphore.

- Semaphore deletion: During semaphore deletion, the semaphore that is in use is set to be not in use and is added to the linked list of semaphores that are not in use.
- A semaphore allows multiple tasks to access the same resource at the same time but sets a limit on the number of the tasks. When the number of tasks accessing the same resource reaches the maximum, other tasks that attempt to obtain the resource are blocked until the semaphore is released.

Figure 3-10 Working principle of semaphore



3.8.2 Development Guidelines

Application Scenario

In the multi-task system, semaphore is a flexible synchronization mode. It can be used in multiple scenarios to implement functions such as lock, synchronization, and resource counting. It can also be used in synchronization between tasks and between an interrupt and a task. Typically, a semaphore is used to coordinate a group of competing tasks to access to shared resources.

Functions

The semaphore module of Huawei LiteOS provides the following functions. For details about the APIs, see the API reference.

Function Category	API	Description
Semaphore creation or deletion	LOS_SemCreate	Creates a semaphore and returns a semaphore ID.
	LOS_BinarySemCreate	Creates a binary semaphore. The maximum counter value is 1.
	LOS_SemDelete	Deletes a semaphore.
Semaphore allocation or release	LOS_SemPend	Allocates a specified semaphore and sets the timeout interval.
	LOS_SemPost	Releases a semaphore.

NOTE

There are three semaphore allocation modes: non-block mode, permanent block mode, and scheduled block mode.

- Non-block mode: When a task allocates a semaphore, the input parameter timeout is 0. If the current semaphore counter value is not 0, the allocation is successful. Otherwise, an allocation failure message is returned immediately.
- Permanent mode: When a task allocates a semaphore, the input parameter timeout is 0xFFFFFFFF. If the current semaphore counter value is not 0, the allocation is successful. Otherwise, the task enters the blocked state, and the system switches to the ready task with the highest priority and continues task execution. After a task enters the blocked state, the blocked task can be executed again only after another task releases the semaphore.
- Scheduled block mode: When a task allocates a semaphore, the input parameter timeout is greater than 0 and smaller than 0xFFFFFFFF. If the current semaphore counter value is not 0, the allocation is successful. Otherwise, the task enters the blocked state, and the system switches to the ready task with the highest priority and continues task execution. After a task enters the blocked state, if another task releases the semaphore before the timeout occurs, the task can successfully obtain the semaphore and continue to be executed. If the semaphore is not obtained before the timeout occurs, the API returns the timeout error code.

Semaphore Error Codes

Error codes are returned for operations that may fail to be performed to quickly locate error causes.

N o.	Definition	Error Code	Description	Reference Solution
1	LOS_ERRNO_SEM_NO_MEMORY	0x02000700	The memory space is insufficient when the semaphore is initialized.	Adjust OS_SYS_MEM_SIZE to ensure that sufficient memory is available for semaphores, or reduce the maximum number (LOSCFG_BASE_IPC_SEM_LIMIT) of semaphores supported by the system.
2	LOS_ERRNO_SEM_INVALID	0x02000701	The semaphore ID is incorrect or the semaphore is not created.	Pass in a correct semaphore ID or create a semaphore before using it.
3	LOS_ERRNO_SEM_PTR_NULL	0x02000702	The passed-in pointer is null.	Pass in a non-null pointer.
4	LOS_ERRNO_SEM_ALL_BUSY	0x02000703	When a semaphore is created, no unused semaphore exists in the system.	Delete useless semaphores or increase the maximum number of semaphores (LOSCFG_BASE_IPC_SEM_LIMIT) supported by the system.
5	LOS_ERRNO_SEM_UNAVAILABLE	0x02000704	Failed to obtain the semaphore in non-block mode.	Block the task and wait, or handle the problem based on the error code.
6	LOS_ERRNO_SEM PEND_INTERRUPT	0x02000705	During the interrupt, LOS_SemPend is called to allocate a semaphore.	Do not call LOS_SemPend during the interrupt.
7	LOS_ERRNO_SEM PEND_IN_LOCK	0x02000706	The task is locked and fails to obtain the semaphore.	Do not call LOS_SemPend when the task is locked.
8	LOS_ERRNO_SEM_TIMEOUT	0x02000707	Obtaining the semaphore timed out.	Set the time to a proper range.
9	LOS_ERRNO_SEM_OVERFLOW	0x02000708	The semaphore counter value has reached the maximum, and the semaphore cannot be released.	Handle the problem based on the error code.

N o.	Definition	Error Code	Description	Reference Solution
1 0	LOS_ERRNO_SEM_PENDE	0x02000709	The task queue waiting for the semaphore is not empty.	Wake up all tasks that are waiting for the semaphore, and then delete the semaphore.

NOTICE

For details about the error code definition, see [Error Codes](#). Bits 8–15 belong to the semaphore module, and the value is **0x07**.

Development Procedure

The typical semaphore development process is as follows:

1. Open the menu and choose **Kernel > Enable Sem** to configure the semaphore module.

Configuration Item	Description	Value Range	Default Value	Dependency
LOSCFG_BASE_IPC_SEM	Whether to enable the tailoring function of the semaphore module.	YES/NO	YES	None.
LOSCFG_BASE_IPC_SEM_LIMIT	Maximum number of semaphores supported by the system	[0, 65535]	1024	None.

2. Call the LOS_SemCreate API to create a semaphore. To create a binary semaphore, call the LOS_BinarySemCreate API.
3. Call the LOS_SemPend API to allocate a semaphore.
4. Call the LOS_SemPost API to release a semaphore.
5. Call the LOS_SemDelete API to delete a semaphore.

Platform Differences

None.

3.8.3 Precautions

As interrupts cannot be blocked, the blocked mode cannot be used to allocate semaphores during interrupts.

3.8.4 Programming Example

Example Description

The following functions are implemented:

1. The Example_TaskEntry task is executed to create a semaphore. Task scheduling is locked. Two tasks Example_SemTask1 and Example_SemTask2 are created, where Example_SemTask2 takes a higher priority than Example_SemTask1. The two tasks allocate the same semaphore. After task scheduling is unlocked, the two tasks are blocked, the test task Example_TaskEntry releases the semaphore.
2. Example_SemTask2 is granted the semaphore, scheduled, and sent to sleep mode for 20 ticks. While Example_SemTask2 is delayed, Example_SemTask1 is woken up.
3. Example_SemTask1 allocates the semaphore and is willing to wait the semaphore for 10 ticks to become free. At the time when Example_SemTask1 allocates the semaphore, the semaphore is held by Example_SemTask2 and consequently Example_SemTask1 is suspended. After the 10-tick wait period elapses, the semaphore is still out of the reach of Example_SemTask1, and Example_SemTask1 is woken up, attempting to wait permanently for the semaphore. The wait for semaphore switches Example_SemTask1 to suspended state.
4. After 20 ticks, Example_SemTask2 is woken up and releases the semaphore. Example_SemTask1 is scheduled, granted the semaphore, and finally releases it.
5. 40 ticks after Example_SemTask1 is finished, Example_TaskEntry is woken up, deletes the semaphore and then the two tasks.

Example Code

Prerequisite: The semaphore module has been configured in the **menuconfig** menu.

The code is as follows:

```
#include "los_sem.h"
#include "securec.h"

/* Task ID */
static UINT32 g_testTaskId01;
static UINT32 g_testTaskId02;
/* Task priority */
#define TASK_PRIO_TEST 5
/* Semaphore structure ID */
static UINT32 g_semId;

VOID Example_SemTask1(VOID)
{
    UINT32 ret;

    printf("Example_SemTask1 try get sem g_semId ,timeout 10 ticks.\n");
    /* The task allocates the semaphore in scheduled blocking mode, with the wait period being 10 ticks. */
    ret = LOS_SemPend(g_semId, 10);

    /* The task is granted the semaphore. */
    if (ret == LOS_OK) {
        LOS_SemPost(g_semId);
    }
}
```

```

        return;
    }
    /* The task does not get the semaphore within the scheduled time. */
    if (ret == LOS_ERRNO_SEM_TIMEOUT) {
        printf("Example_SemTask1 timeout and try get sem g_semId wait forever.\n");
        /* The task allocates the semaphore in permanent blocking mode. */
        ret = LOS_SemPend(g_semId, LOS_WAIT_FOREVER);
        printf("Example_SemTask1 wait_forever and get sem g_semId .\n");
        if (ret == LOS_OK) {
            LOS_SemPost(g_semId);
            return;
        }
    }
}

VOID Example_SemTask2(VOID)
{
    UINT32 ret;
    printf("Example_SemTask2 try get sem g_semId wait forever.\n");
    /* The task allocates the semaphore in permanent blocking mode. */
    ret = LOS_SemPend(g_semId, LOS_WAIT_FOREVER);

    if (ret == LOS_OK) {
        printf("Example_SemTask2 get sem g_semId and then delay 20ticks .\n");
    }

    /* Enable the task to enter sleep mode for 20 ticks. */
    LOS_TaskDelay(20);

    printf("Example_SemTask2 post sem g_semId .\n");
    /* The task releases the semaphore. */
    LOS_SemPost(g_semId);
    return;
}

UINT32 ExampleTaskEntry(VOID)
{
    UINT32 ret;
    TSK_INIT_PARAM_S task1;
    TSK_INIT_PARAM_S task2;

    /* Create a semaphore. */
    LOS_SemCreate(0,&g_semId);

    /* Lock task scheduling. */
    LOS_TaskLock();

    /* Create task 1. */
    (VOID)memset_s(&task1, sizeof(TSK_INIT_PARAM_S), 0, sizeof(TSK_INIT_PARAM_S));
    task1.pfnTaskEntry = (TSK_ENTRY_FUNC)Example_SemTask1;
    task1.pcName = "TestTsk1";
    task1.uwStackSize = OS_TSK_DEFAULT_STACK_SIZE;
    task1.usTaskPrio = TASK_PRIO_TEST;
    ret = LOS_TaskCreate(&g_testTaskId01, &task1);
    if (ret != LOS_OK) {
        printf("task1 create failed .\n");
        return LOS_NOK;
    }

    /* Create task 2. */
    (VOID)memset_s(&task2, sizeof(TSK_INIT_PARAM_S), 0, sizeof(TSK_INIT_PARAM_S));
    task2.pfnTaskEntry = (TSK_ENTRY_FUNC)Example_SemTask2;
    task2.pcName = "TestTsk2";
    task2.uwStackSize = OS_TSK_DEFAULT_STACK_SIZE;
    task2.usTaskPrio = (TASK_PRIO_TEST - 1);
    ret = LOS_TaskCreate(&g_testTaskId02, &task2);
    if (ret != LOS_OK) {
        printf("task2 create failed .\n");
        return LOS_NOK;
    }
}

```

```
}

/* Unlock task scheduling. */
LOS_TaskUnlock();

ret = LOS_SemPost(g_semId);

/* Enable the task to enter sleep mode for 40 ticks. */
LOS_TaskDelay(40);

/* Delete the semaphore. */
LOS_SemDelete(g_semId);

/* Delete task 1. */
ret = LOS_TaskDelete(g_testTaskId01);
if (ret != LOS_OK) {
    printf("task1 delete failed .\n");
    return LOS_NOK;
}
/* Delete task 2. */
ret = LOS_TaskDelete(g_testTaskId02);
if (ret != LOS_OK) {
    printf("task2 delete failed .\n");
    return LOS_NOK;
}

return LOS_OK;
}
```

Verification

The verification result is as follows:

```
Example_SemTask2 try get sem g_semId wait forever.
Example_SemTask1 try get sem g_semId ,timeout 10 ticks.
Example_SemTask2 get sem g_semId and then delay 20ticks .
Example_SemTask1 timeout and try get sem g_semId wait forever.
Example_SemTask2 post sem g_semId .
Example_SemTask1 wait_forever and get sem g_semId .
```

Complete Code

sample_sem.c

3.9 Read/Write Semaphore

3.9.1 Overview

Basic Concepts

The read/write semaphore (rwsem) has similar functions as a common semaphore and is usually used for mutually exclusive access to shared resources and task synchronization. The read/write semaphore classifies owners into readers and writers. If an owner needs to write a shared resource protected by the semaphore, the owner is defined as a writer. If an owner only accesses the shared resource without modification, the owner is defined as a reader.

The read/write semaphore supports one semaphore and unlimited number of readers at the same time. That is, the read semaphore is not exclusive and supports concurrent reading. However, the read/write semaphore can have only

one writer at the same time. That is, the write semaphore is exclusive and prevents concurrent write/write or concurrent read/write.

Operation Mechanism

Read/Write semaphore control block

```
typedef struct {
    UINT8    rwsemState;    /* Whether to use flag bit. */
    INT16    rwsemCount;    /* Number of read or write semaphore owners. */
    UINT32    rwsemId;      /* Index of the read or write semaphore. */
    LOS_DL_LIST waitList;    /*Waiting list, which is used to record the tasks blocked in the read or write semaphore. */
} OsRwsemCB;
```

Operation Principle of Read/Write Semaphore

- Read/write semaphore resources initialization: Memory is allocated to N read/write semaphore. (N is configurable and can be implemented using the `LOSCFG_BASE_IPC_RWSEM_LIMIT` macro.) All read/write semaphores are initialized and added to the linked list of semaphores that are not in use.
- Read/write semaphore creation: A read/write semaphore is obtained from the linked list that is not in use.
- Read semaphore allocation: If read semaphore is not owned or is being owned by other readers, the read semaphore will be returned. Otherwise, tasks are blocked from getting the read semaphore and wait for other tasks to post a read semaphore. The timeout interval of waiting on the read semaphore can be configured. If a task is blocked from getting the read/write semaphore, suspend the task to the end of the queue of tasks waiting on the read/write semaphore.
- Write semaphore allocation: If write semaphore is not owned by writer, the write semaphore will be returned. Otherwise, tasks are blocked from getting the write semaphore and wait for other tasks to post a read semaphore. The timeout interval of waiting on the write semaphore can be configured. If a task is blocked from getting the read/write semaphore, suspend the task to the end of the queue of tasks waiting on the read/write semaphore.
- Read semaphore release: If no task is waiting on a read semaphore, the read semaphore will be returned directly. If the semaphore is in read state, the first task in the waiting queue can only be the writer. Wake up the writer and return the read semaphore.
- Write semaphore release: If no task is waiting on a write semaphore, the semaphore will be returned directly. If the first task in the waiting queue is a writer, the write semaphore will be returned after waking up the writer. If the first task in the waiting queue is a reader, wake up the reader and subsequent consecutive readers until the waiting queue is empty or encountering a writer and stop.
- Write semaphore degradation: The write semaphore is degraded to the read semaphore. If no task waits on the read semaphore or the first task in the waiting queue is the writer, the semaphore will be returned. Otherwise, wake up the reader and subsequent consecutive readers until the waiting queue is empty or encountering a writer and stop.
- Read/write semaphore deletion: The read/write semaphore that is in use is set to be not in use and is added to the linked list of semaphores that are not in use.

 NOTE

The read/write semaphore allows multiple tasks to access the resources at the same time, but only one task can modify the shared resources at the same time. In this way, the overall system performance can be effectively improved in scenarios where the number of read tasks is greater than that of write tasks.

3.9.2 Development Guidelines

Application Scenario

The read/write semaphore is an optimized mechanism used for multi-task synchronization and mutual exclusion based on the common semaphore. It can be used in multiple scenarios to implement functions such as lock, synchronization, and resource counting. It can also be used in synchronization between tasks and is applicable to scenarios where the number of read tasks is greater than that of write tasks.

Functions

The read/write semaphore module of Huawei LiteOS provides the following functions. For details about the APIs, see the API reference.

Function Category	API	Description
Creating or deleting a read/write semaphore	LOS_RwsemCreate	Creates a read/write semaphore and returns a read/write semaphore ID.
	LOS_RwsemDelete	Deletes a specified read/write semaphore.
Allocating or releasing a read/write semaphore	LOS_RwsemPendRead	Allocates a read semaphore as a specified semaphore and supports obtaining the timeout interval.
	LOS_RwsemPostRead	Releases the specified read semaphore.
	LOS_RwsemPendWrite	Allocates a write semaphore as a specified semaphore and supports obtaining the timeout interval.
	LOS_RwsemPostWrite	Releases the specified write semaphore.
	LOS_RwsemDowngradeWrite	Degrades a specified write semaphore to a read semaphore.

NOTE

There are three read/write semaphore allocation modes: non-block mode, permanent block mode, and scheduled block mode.

- Non-block mode: When a task allocates a read/write semaphore, the input parameter timeout is 0. If the current read/write semaphore is available, the allocation is successful. Otherwise, an allocation failure message is returned immediately.
- Permanent block mode: When a task allocates a read/write semaphore, the input parameter timeout is 0xFFFFFFFF. If the current read/write semaphore is available, the allocation is successful. Otherwise, the task enters the blocked state, and the system switches to the ready task with the highest priority and continues task execution. After a task enters the blocked state, the blocked task can be executed again only after another task releases the read/write semaphore.
- Scheduled block mode: When a task allocates a read/write semaphore, the input parameter timeout is greater than 0 and smaller than 0xFFFFFFFF. If the current read/write semaphore is available, the allocation is successful. Otherwise, the task enters the blocked state, and the system switches to the ready task with the highest priority and continues task execution. After a task enters the blocked state, if another task releases the semaphore before the timeout occurs, the task can successfully obtain the read/write semaphore and continue to be executed. If the read/write semaphore is not obtained before the timeout occurs, the API returns the timeout error code.

Read/Write Semaphore Error Codes

Error codes are returned for operations that may fail to be performed to quickly locate error causes.

N o.	Definition	Error Code	Description	Reference Solution
1	LOS_ERRNO_RWSEM_INVALID	0x02002200	The read/write semaphore ID is incorrect or the read/write semaphore is not created.	Pass in a correct read/write semaphore ID or create a read/write semaphore before using it.
2	LOS_ERRNO_RWSEM_PTR_NULL	0x02002201	The passed-in pointer is null.	Pass in a non-null pointer.
3	LOS_ERRNO_RWSEM_ALL_BUSY	0x02002202	When a read/write semaphore is created, no unused read/write semaphore exists in the system.	Delete useless read/write semaphores or increase the maximum number of read/write semaphores (LOSCFG_BASE_IPC_RWSEM_LIMIT) supported by the system.
4	LOS_ERRNO_RWSEM_PENDING_INTERRUPT	0x02002203	The API calling is unauthorized during an interrupt, for example, allocating read and write semaphores in block mode.	Call the API properly. Do not obtain the read or write semaphore in block mode during the interrupt.

N o.	Definition	Error Code	Description	Reference Solution
5	LOS_ERRNO_RWSEM_PENDING_LOCK	0x02002204	The task is locked and fails to obtain the read/write semaphore.	Do not call the API to request read/write semaphore when the task is locked.
6	LOS_ERRNO_RWSEM_TIMEOUT	0x02002205	Obtaining the read/write semaphore timed out.	Set the time to a proper range.
7	LOS_ERRNO_RWSEM_PENDING	0x02002206	The task queue waiting for the read/write semaphore is not empty.	Wake up all tasks that are waiting for the read/write semaphore, and then delete the read/write semaphore.
8	LOS_ERRNO_RWSEM_INVALID_STATUS	0x02002207	The read/write semaphore status is incorrect when the semaphore is released or degraded.	Release and degrade the read/write semaphore correctly.
9	LOS_ERRNO_RWSEM_UNAVAILABLE	0x02002208	Obtaining the read/write semaphore in non-block mode is failed.	Block the task and wait, or handle the problem based on the error code.

NOTICE

For details about the error code definition, see [Development Guidelines](#). Bits 8–15 belong to the semaphore module, and the value is **0x22**.

Development Procedure

The typical read/write semaphore development process is as follows:

1. Open the menu and choose **Kernel > Enable Rwsem** to configure the read/write semaphore module.

Configuration Item	Description	Value Range	Default Value	Dependency
LOSCFG_BASE_IPC_RWSEM	Whether to enable the tailoring function of the read/write semaphore module.	YES/NO	YES	None.
LOSCFG_BASE_IPC_RWSEM_LIMIT	Maximum number of read/write semaphores supported by the system	[0, 65535]	1024	LOSCFG_BASE_IPC_RWSEM

2. Call the LOS_RwsemCreate API to create a read/write semaphore.
3. Call the LOS_RwsemPendRead API to allocate a read/write semaphore.
4. Call the LOS_RwsemPostRead API to release a read/write semaphore.
5. Call the LOS_RwsemDelete API to delete a read/write semaphore.

Platform Differences

None.

3.9.3 Precautions

As interrupts cannot be blocked, the blocked mode cannot be used to allocate read/write semaphores during interrupts.

3.9.4 Programming Example

Example Description

The following functions are implemented:

1. The ExampleTaskEntry task creates a read/write semaphore and allocates a write semaphore. Task scheduling is locked. Three tasks ExampleReadTask1, ExampleReadTask2, and ExampleWriteTask2 are created, where ExampleReadTask1 and ExampleReadTask2 take a higher priority than ExampleWriteTask2 and they allocate read semaphores. ExampleWriteTask2 allocates write semaphores.
2. After ExampleTaskEntry unlocks task scheduling, ExampleReadTask1 and ExampleReadTask2 fail to obtain the read semaphore and wait permanently. ExampleWriteTask2 fails to obtain the write semaphore after waiting for 1 tick and is blocked permanently to obtain the write semaphore.

3. ExampleTaskEntry degrades the write semaphore to the read semaphore. ExampleReadTask1 and ExampleReadTask2 obtain the read semaphore successfully. ExampleWriteTask2 continues to be blocked.
4. ExampleReadTask1, ExampleReadTask2, and ExampleTaskEntry release the read semaphore, and then ExampleWriteTask2 successfully obtains the write semaphore.
5. ExampleWriteTask2 releases the write semaphore.

Example Code

Prerequisites: The read/write semaphores have been configured in the **menuconfig** menu.

The code is as follows:

```
#include "los_rwsem.h"
#include "securec.h"

static UINT32 g_rwsemId; /* Read or write semaphore structure ID */

VOID ExampleReadTask1(VOID)
{
    UINT32 ret;

    /* The task does not get the read semaphore within the scheduled time. */
    printf("[ExampleReadTask1] try get rwsem as read, wait forever.\n");
    /* The task allocates the read semaphore in permanent blocking mode. */
    ret = LOS_RwsemPendRead(g_rwsemId, LOS_WAIT_FOREVER);
    if (ret == LOS_OK) {
        printf("[ExampleReadTask1] get rwsem success.\n");
        LOS_TaskDelay(3);
        printf("[ExampleReadTask1] post rwsem.\n");
        LOS_RwsemPostRead(g_rwsemId);
        return;
    }
}

VOID ExampleReadTask2(VOID)
{
    UINT32 ret;

    /* The task does not get the read semaphore within the scheduled time. */
    printf("[ExampleReadTask2] try get rwsem as read, wait forever.\n");
    /* The task allocates the read semaphore in permanent blocking mode. */
    ret = LOS_RwsemPendRead(g_rwsemId, LOS_WAIT_FOREVER);
    if (ret == LOS_OK) {
        printf("[ExampleReadTask2] get rwsem success.\n");
        LOS_TaskDelay(3);
        printf("[ExampleReadTask2] post rwsem.\n");
        LOS_RwsemPostRead(g_rwsemId);
        return;
    }
}

VOID ExampleWriteTask2(VOID)
{
    UINT32 ret;

    printf("[ExampleWriteTask2] try get rwsem as write, timeout 1 tick.\n");
    /* The task allocates the write semaphore in scheduled blocking mode, with the wait period being 1 tick. */
    ret = LOS_RwsemPendWrite(g_rwsemId, 1);

    /* The task is granted the write semaphore. */
    if (ret == LOS_OK) {
```

```

        LOS_RwsemPostWrite(g_rwsemId);
        return;
    }
    /* The task does not get the write semaphore within the scheduled time. */
    if (ret == LOS_ERRNO_RWSEM_TIMEOUT) {
        printf("[ExampleWriteTask2] timeout and try get rwsem as write, wait forever.\n");
        /* The task allocates the write semaphore in permanent blocking mode. */
        ret = LOS_RwsemPendWrite(g_rwsemId, LOS_WAIT_FOREVER);
        if (ret == LOS_OK) {
            printf("[ExampleWriteTask2] get rwsem success.\n");
            LOS_TaskDelay(3);
            printf("[ExampleWriteTask2] post rwsem.\n");
            LOS_RwsemPostWrite(g_rwsemId);
            return;
        }
    }
    return;
}

static VOID InitTaskParam(TSK_INIT_PARAM_S *task, char *taskName, TSK_ENTRY_FUNC entry, UINIT16
prio, UINIT16 affi)
{
    task->pcName      = taskName;
    task->pfnTaskEntry = entry;
    task->usTaskPrio   = prio;
    task->uwStackSize  = LOSCFG_BASE_CORE_TSK_DEFAULT_STACK_SIZE;
    task->uwResved     = LOS_TASK_STATUS_DETACHED;

#ifdef LOSCFG_KERNEL_SMP
    task->usCpuAffiMask = affi;
#endif
}

UINIT32 ExampleTaskEntry(VOID)
{
    UINIT32 ret;
    UINIT32 readTsk1Id, readTsk2Id, writeTsk2Id;
    TSK_INIT_PARAM_S readTask1 = {0};
    TSK_INIT_PARAM_S readTask2 = {0};
    TSK_INIT_PARAM_S writeTask2 = {0};

    /* Create a read or write semaphore. */
    ret = LOS_RwsemCreate(&g_rwsemId);
    if (ret != LOS_OK) {
        printf("[ExampleTaskEntry] create rwsem failed.\n");
        return LOS_NOK;
    }

    ret = LOS_RwsemPendWrite(g_rwsemId, 0);
    if (ret != LOS_OK) {
        printf("[ExampleTaskEntry] get rwsem failed.\n");
        return LOS_NOK;
    }
    printf("[ExampleTaskEntry] get rwsem success.\n");

    /* Lock task scheduling. */
    LOS_TaskLock();

    /* Create read task 1. */
    InitTaskParam(&readTask1, "resemReadTsk1", (TSK_ENTRY_FUNC)ExampleReadTask1, 5,
CPUID_TO_AFFI_MASK(ArchCurrCpuId()));
    ret = LOS_TaskCreate(&readTsk1Id, &readTask1);
    if (ret != LOS_OK) {
        printf("[ExampleTaskEntry] readTask1 create failed.\n");
        return LOS_NOK;
    }

    /* Create read task 2. */
    InitTaskParam(&readTask2, "resemReadTsk2", (TSK_ENTRY_FUNC)ExampleReadTask2, 5,

```

```

CPUID_TO_AFFI_MASK(ArchCurrCpuId()));
ret = LOS_TaskCreate(&readTsk2Id, &readTask2);
if (ret != LOS_OK) {
    printf("[ExampleTaskEntry] readTask2 create failed.\n");
    return LOS_NOK;
}

/* Create write task 2. */
InitTaskParam(&writeTask2, "resemWriteTsk2", (TSK_ENTRY_FUNC)ExampleWriteTask2, 9,
CPUID_TO_AFFI_MASK(ArchCurrCpuId()));
ret = LOS_TaskCreate(&writeTsk2Id, &writeTask2);
if (ret != LOS_OK) {
    printf("[ExampleTaskEntry] writeTask2 create failed.\n");
    return LOS_NOK;
}

/* Unlock task scheduling. */
LOS_TaskUnlock();

/* Release the CPU resources for 5 ticks and wait for readTask1, readTask2, and writeTask2 to be
executed and blocked.*/
LOS_TaskDelay(5);

/* Degrade write semaphore to read semaphore. */
printf("[ExampleTaskEntry] downgrade rwsem.\n");
LOS_RwsemDowngradeWrite(g_rwsemId);

/* Enable the task to enter sleep mode for 5 ticks. */
LOS_TaskDelay(5);

LOS_RwsemPostRead(g_rwsemId);

/* Enable the task to enter sleep mode for 10 ticks. */
LOS_TaskDelay(10);

/*Delete the read or write semaphore.*/
LOS_RwsemDelete(g_rwsemId);

return LOS_OK;
}

```

Verification

The verification result is as follows:

```

[ExampleTaskEntry] get rwsem success.
[ExampleReadTask1] try get rwsem as read, wait forever.
[ExampleReadTask2] try get rwsem as read, wait forever.
[ExampleWriteTask2] try get rwsem as write, timeout 1 tick.
[ExampleWriteTask2] timeout and try get rwsem as write, wait forever.
[ExampleTaskEntry] downgrade rwsem.
[ExampleReadTask1] get rwsem success.
[ExampleReadTask2] get rwsem success.
[ExampleReadTask1] post rwsem.
[ExampleReadTask2] post rwsem.
[ExampleWriteTask2] get rwsem success.
[ExampleWriteTask2] post rwsem.

```

Complete Code

sample_rwsem_cond.rar

3.10 Mutex

3.10.1 Overview

Basic Concepts

A mutual exclusion (mutex) is a special binary semaphore designed to grant a task exclusive use of common resources. In addition, the mutex can resolve the priority inversion problem of semaphores.

At a given point in time, a mutex is either locked or unlocked. When a task holds a mutex, the mutex is locked and the task has exclusive ownership of the mutex. When the task releases the mutex, the mutex is unlocked and the task loses exclusive ownership of the mutex. While a task has exclusive ownership of a mutex, other tasks are unable to acquire or release the mutex.

The mutex provided by Huawei LiteOS has the following features:

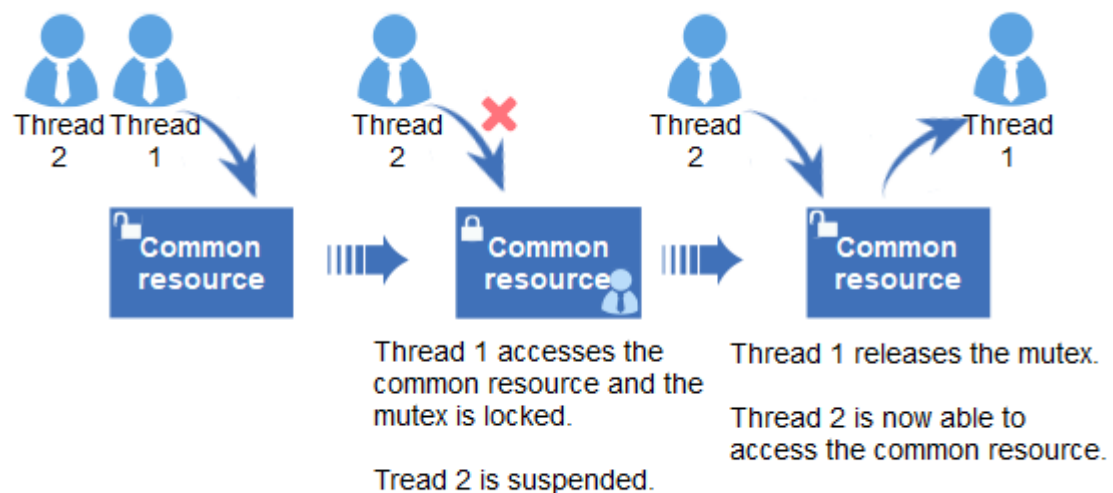
- Solve the priority inversion problem using the priority inheritance algorithm.
- In the scenario where multiple tasks are blocked and wait for the same lock, the task priority-based waiting mode and FIFO mode are supported.

Operation Mechanism

In a multi-task environment, multiple tasks may battle for the same common resource. If a common resource is not shareable, it must be used exclusively by a particular task. How does the mutex work to avoid the conflict?

When a task accesses a non-shareable common resource, the mutex is locked. Other tasks are blocked from accessing the resource until the task releases the mutex. In this way, only one task accesses the non-shareable common resource at a given point in time, which ensures the integrity of the non-shareable common resources.

Figure 3-11 Working principle of a mutex



3.10.2 Development Guidelines

Application Scenario

In a multi-task environment, multiple tasks often compete for the same critical resource. The mutex provides a mutex mechanism between tasks to prevent two tasks from accessing the same critical resource at the same time. In this way, exclusive access is implemented.

Functions

The mutex module of Huawei LiteOS provides the following functions. For details about the APIs, see the API reference.

Function Category	API	Description
Mutex creation and deletion	LOS_MuxCreate	Creates a mutex.
	LOS_MuxDelete	Deletes a mutex.
Mutex allocation and release	LOS_MuxPend	Allocates a mutex.
	LOS_MuxPost	Releases a mutex.

NOTE

There are three mutex allocation modes: non-block mode, permanent block mode, and scheduled block mode.

- Non-block mode: When a task allocates a mutex, the input parameter timeout is 0. If no task has acquired the mutex or the task that has acquired the mutex is the same as the allocation task, the operating system grants the mutex to the allocation task.
- Permanent mode: When a task allocates a mutex, the input parameter timeout is 0xFFFFFFFF. If the mutex has not been acquired by any task, the operating system grants the mutex to the allocation task. Otherwise, the task enters the blocked state, and the system switches to the ready task with the highest priority and continues task execution. After a task enters the blocked state, the blocked task can be executed again only after another task releases the mutex.
- Scheduled block mode: When a task allocates a mutex, the input parameter timeout is greater than 0 and smaller than 0xFFFFFFFF. If the mutex has not been acquired by any task, the operating system grants the mutex to the allocation task. Otherwise, the task enters the blocked state, and the system switches to the ready task with the highest priority and continues task execution. After a task enters the blocked state, if another task releases the mutex before the timeout occurs, the task can successfully obtain the mutex and continue to be executed. If the mutex is not obtained before the timeout occurs, the API returns the timeout error code.

Releasing a mutex.

- If some tasks are blocked due to the mutex, the operating system wakes up the blocked task with highest priority. The woken-up task then enters Ready state and is scheduled.
- If no tasks are blocked due to the mutex, the operating system released the mutex successfully.

Mutex Error Code

Error codes are returned for operations that may fail to be performed to quickly locate error causes.

N o.	Definition	Error Code	Description	Reference Solution
1	LOS_ERRNO_MUX_NO_MEMORY	0x02001d00	The memory is insufficient during initialization of the mutex module.	Set the configuration item OS_SYS_MEM_SIZE to a larger system dynamic memory pool, or reduce the maximum number of mutexes supported by the system.
2	LOS_ERRNO_MUX_INVALID	0x02001d01	The mutex is unavailable.	Pass in a valid mutex ID.
3	LOS_ERRNO_MUX_PTR_NULL	0x02001d02	The input parameter is null during mutex creation.	Pass in a non-null parameter.
4	LOS_ERRNO_MUX_ALL_BUSY	0x02001d03	No mutexes are available during mutex creation.	Increase the maximum number of mutexes supported by the system.
5	LOS_ERRNO_MUX_UNAVAILABLE	0x02001d04	The mutex fails to be allocated, because it has been acquired by another thread.	Wait for another thread to release the mutex. Alternatively, set a timeout period.
6	LOS_ERRNO_MUX_PEND_INTERRUPT	0x02001d05	Use mutexes in interrupts.	Do not allocate or release mutexes during interrupts.
7	LOS_ERRNO_MUX_PEND_IN_LOCK	0x02001d06	When task scheduling is disabled, tasks are not allowed to allocate a mutex in block mode.	Allocates a mutex in non-block mode. Alternatively, enable task scheduling before allocating a mutex in block mode.
8	LOS_ERRNO_MUX_TIMEOUT	0x02001d07	Mutex allocation times out.	Increase the wait time. Alternatively, set the mutex to the permanent block mode.
9	LOS_ERRNO_MUX_PENDEDED	0x02001d09	Delete a mutex that is being used.	Delete the mutex after it is released.

NOTICE

For details about the error code definition, see [Error Codes](#). Bits 8–15 belong to the mutex module, and the value is 0x1d.

Development Procedure

The development process of mutexes in typical scenarios is as follows:

1. Open the menu and choose **Kernel > Enable Mutex** to configure the mutex module.

Configuration Item	Description	Value Range	Default Value	Dependency
LOSCFG_BASE_IPC_MUX	Whether to enable the tailoring function of the mutex module.	YES/NO	YES	None.
LOSCFG_MUTEX_WAITMODE_PRIO	Task priority-based waiting mode for mutexes.	YES/NO	YES	LOSCFG_BASE_IPC_MUX
LOSCFG_MUTEX_WAITMODE_FIFO	FIFO-based waiting mode for mutexes.	YES/NO	NO	LOSCFG_BASE_IPC_MUX
LOSCFG_BASE_IPC_MUX_LIMIT	Maximum number of mutexes supported by the system.	< 65535	1024	LOSCFG_BASE_IPC_MUX

2. Call the `LOS_MuxCreate` API to create a mutex.
3. Call the `LOS_MuxPend` API to allocate a mutex.
4. Call the `LOS_MuxPost` API to release a mutex.
5. Call the `LOS_MuxDelete` API to delete a mutex.

Platform Differences

None.

3.10.3 Precautions

- Mutexes cannot be used in the ISR.
- As a real-time OS, Huawei LiteOS needs to ensure real-time task scheduling and avoid long-time task blocking. Therefore, a mutex needs to be released as soon as possible after being obtained.
- In Huawei LiteOS, do not call the `LOS_TaskPriSet` API to change the priority of a task that holds a mutex.
- Mutexes do not support the scenario where multiple tasks with the same priority are reversed.

3.10.4 Programming Example

Example Description

The following functions are implemented:

1. The Example_TaskEntry task is executed to create a mutex. Task scheduling is locked. Two tasks Example_MutexTask1 and Example_MutexTask2 are created, where Example_MutexTask2 takes a higher priority than Example_MutexTask1. Then, task scheduling is unlocked, and the Example_TaskEntry task stays in sleep mode for 300 ticks.
2. Example_MutexTask2 is scheduled, granted a mutex, and then sent to sleep mode for 100 ticks. While Example_MutexTask2 is suspended, Example_MutexTask1 is woken up.
3. Example_MutexTask1 allocates a mutex and is willing to wait the mutex for 10 ticks to become free. At the time when Example_MutexTask1 allocates the mutex, the mutex is held by Example_MutexTask2 and consequently Example_MutexTask1 is suspended. After the 10-tick wait period elapses, the mutex is still out of the reach of Example_MutexTask1, and Example_MutexTask1 is woken up, attempting to wait permanently for the mutex. The wait for the mutex switches Example_MutexTask1 to suspended state.
4. After 100 ticks, Example_MutexTask2 is woken up and releases the mutex, and then Example_MutexTask1 is woken up. Example_MutexTask1 is scheduled, granted the mutex, and finally releases it.
5. After the 300 ticks in sleep mode, Example_TaskEntry task is scheduled and executed. The mutex is deleted, and two tasks are deleted.

Example Code

Prerequisite: Open the menu and configure the mutex module.

The code is as follows:

```
/* Mutex handler ID */
UINT32 g_testMux;
/* Task ID */
UINT32 g_testTaskId01;
UINT32 g_testTaskId02;

VOID Example_MutexTask1(VOID)
{
    UINT32 ret;

    printf("task1 try to get  mutex, wait 10 ticks.\n");
    /* Allocate a mutex. */
    ret = LOS_MuxPend(g_testMux, 10);

    if (ret == LOS_OK) {
        printf("task1 get mutex g_testMux.\n");
        /* Release the mutex. */
        LOS_MuxPost(g_testMux);
        return;
    } else if (ret == LOS_ERRNO_MUX_TIMEOUT ) {
        printf("task1 timeout and try to get mutex, wait forever.\n");
        /* Allocate a mutex. */
        ret = LOS_MuxPend(g_testMux, LOS_WAIT_FOREVER);
        if (ret == LOS_OK) {
```

```

        printf("task1 wait forever, get mutex g_testMux.\n");
        /* Release the mutex. */
        LOS_MuxPost(g_testMux);
        return;
    }
}
return;
}

VOID Example_MutexTask2(VOID)
{
    printf("task2 try to get mutex, wait forever.\n");
    /* Allocate a mutex. */
    (VOID)LOS_MuxPend(g_testMux, LOS_WAIT_FOREVER);

    printf("task2 get mutex g_testMux and suspend 100 ticks.\n");

    /* Enable the task to enter sleep mode for 100 ticks. */
    LOS_TaskDelay(100);

    printf("task2 resumed and post the g_testMux\n");
    /* Release the mutex. */
    LOS_MuxPost(g_testMux);
    return;
}

UINT32 Example_TaskEntry(VOID)
{
    UINT32 ret;
    TSK_INIT_PARAM_S task1;
    TSK_INIT_PARAM_S task2;

    /* Create a mutex. */
    LOS_MuxCreate(&g_testMux);

    /* Lock task scheduling. */
    LOS_TaskLock();

    /* Create task 1. */
    memset(&task1, 0, sizeof(TSK_INIT_PARAM_S));
    task1.pfnTaskEntry = (TSK_ENTRY_FUNC)Example_MutexTask1;
    task1.pcName = "MutexTsk1";
    task1.uwStackSize = LOSCFG_BASE_CORE_TSK_DEFAULT_STACK_SIZE;
    task1.usTaskPrio = 5;
    ret = LOS_TaskCreate(&g_testTaskId01, &task1);
    if (ret != LOS_OK) {
        printf("task1 create failed.\n");
        return LOS_NOK;
    }

    /* Create task 2. */
    memset(&task2, 0, sizeof(TSK_INIT_PARAM_S));
    task2.pfnTaskEntry = (TSK_ENTRY_FUNC)Example_MutexTask2;
    task2.pcName = "MutexTsk2";
    task2.uwStackSize = LOSCFG_BASE_CORE_TSK_DEFAULT_STACK_SIZE;
    task2.usTaskPrio = 4;
    ret = LOS_TaskCreate(&g_testTaskId02, &task2);
    if (ret != LOS_OK) {
        printf("task2 create failed.\n");
        return LOS_NOK;
    }

    /* Unlock task scheduling. */
    LOS_TaskUnlock();
    /* Enable the task to enter sleep mode for 300 ticks. */
    LOS_TaskDelay(300);

    /* Delete the mutex. */
    LOS_MuxDelete(g_testMux);
}

```

```
/* Delete task 1. */  
ret = LOS_TaskDelete(g_testTaskId01);  
if (ret != LOS_OK) {  
    printf("task1 delete failed .\n");  
    return LOS_NOK;  
}  
/* Delete task 2. */  
ret = LOS_TaskDelete(g_testTaskId02);  
if (ret != LOS_OK) {  
    printf("task2 delete failed .\n");  
    return LOS_NOK;  
}  
  
return LOS_OK;  
}
```

Verification

The verification result is as follows:

```
task2 try to get  mutex, wait forever.  
task2 get mutex g_testMux and suspend 100 ticks.  
task1 try to get  mutex, wait 10 ticks.  
task1 timeout and try to get mutex, wait forever.  
task2 resumed and post the g_testMux  
task1 wait forever,get mutex g_testMux.
```

Complete Code

sample_mutex.c

3.11 Software Timer

3.11.1 Overview

Basic Concepts

A software timer is a timer simulated by software, and works based on system tick interrupts. When a predefined number of ticks elapse, a software timer triggers a user-defined callback function. The timer length is an integral number of ticks.

Only a limited number of hardware timers can be used due to hardware constraints. To meet user requirements of more timers, Huawei LiteOS provides the software timer function and supports the following features:

- Creates a software timer.
- Starts a software timer.
- Stops a software timer.
- Deletes a software timer.
- Obtains the number of remaining ticks of a software timer.
- Configures the number of supported software timers.

Operation Mechanism

Software timers are system resources and are allocated continuous memory at the initialization of the timer module.

Software timers are placed in a queue and are triggered in the first in first out order. The software timers with a short life cycle are placed at the beginning of queue so that they will be triggered earlier than those with a longer life cycle. The timer with a shorter time interval is always closer to the queue head than the timer with a longer time interval, and is therefore preferentially triggered.

The software timer uses ticks as the basic timing unit. When a software timer is created and started, Huawei LiteOS determines the timer expiry time based on the current system time (in ticks) and the specific time interval (in ticks) and adds the timer control structure to the global timing list.

When a tick interrupt occurs, the tick interrupt processing function scans the global timing list for expired timers. If such a timer is found, the timer is recorded. After the tick interrupt processing function finishes processing, the software timer task (with the highest priority) is woken up. In this task, the recorded timer callback function is called.

Software timer states

- **OS_SWTMR_STATUS_UNUSED** (The timer is not used.)
While the timer module is being initialized, the operating system initializes all timer resources in the system to OS_SWTMR_STATUS_UNUSED state.
- **OS_SWTMR_STATUS_TICKING** (The timer is timing.)
If the LOS_SwtmrStart API is called after the timer is created, the timer switches to OS_SWTMR_STATUS_TICKING state.
- **OS_SWTMR_STATUS_CREATED** (The timer is not started or had stopped.)
After a timer is created, it enters this state if it is not working.
- **OS_SWTMR_STATUS_DELETING** (The timer is deleted.)
If the timer is one-shot timer. After the timer times out, the timer enters this state.

Software timer modes

Depending on timer mode, software timers are classified into three types:

- **One-shot timer:** The timer triggers the timer event only once after it is started. Then, the timer is automatically deleted.
- **Periodic timer:** The timer triggers the timer event periodically until the timer is manually stopped.
- **One-shot timer:** The timer will not be automatically deleted after it expires. You need to call the LOS_SwtmrDelete API to delete this timer.

3.11.2 Development Guidelines

Application Scenario

- If you want to trigger a timer event only once, create a one-shot timer. When the timer expires, the user-defined callback function is executed.

- If you want to trigger a timer event periodically, create a periodic timer. When the timer expires, the user-defined callback function is executed.

Functions

The software timer module of Huawei LiteOS provides the following functions. For details about the APIs, see the API reference.

Table 3-2

Function Category	API	Description
Timer creation and deletion	LOS_SwtmrCreate	Creates a software timer, sets the time interval, timer mode, and callback function, and returns the timer ID.
	LOS_SwtmrDelete	Deletes software timer asynchronously. The API returns immediately after checking the input parameter, while the release of software timer resources is not sure. The software timer resources are released after all callbacks of the target timer are complete.
	LOS_SwtmrSyncDelete	Deletes timer synchronously. After checking the input parameters, the timer is deleted and software timer resources is released if the target timer completes all callbacks execution.
Timer start and stop	LOS_SwtmrStart	Starts a software timer.
	LOS_SwtmrStop	Stops a software timer.
Measurement of remaining ticks prior to timer expiry	LOS_SwtmrTimeGet	Measures the number of remaining ticks prior to expiry of a software timer.

Software Timer Error Codes

Error codes are returned for operations that may fail to be performed to quickly locate error causes.

N o.	Definition	Error Code	Description	Reference Solution
1	LOS_ERRNO_SWT_MR_PTR_NULL	0x02000300	The callback function of the software timer is null.	Define the callback function of the software timer.
2	LOS_ERRNO_SWT_MR_INTERVAL_NOT_SUITED	0x02000301	The time interval of the software timer is 0.	Redefine the time interval of the software timer.
3	LOS_ERRNO_SWT_MR_MODE_INVALID	0x02000302	The mode of the software timer is incorrect.	Change the mode of the software timer. Value range: [0, 2]
4	LOS_ERRNO_SWT_MR_RET_PTR_NULL	0x02000303	The passed-in pointer to the software timer ID is null.	Define the ID variable and pass in a valid pointer.
5	LOS_ERRNO_SWT_MR_MAXSIZE	0x02000304	The number of software timers exceeds the maximum value.	Redefine the maximum number of software timers, or wait until a software timer releases resources.
6	LOS_ERRNO_SWT_MR_ID_INVALID	0x02000305	The passed-in software timer ID is incorrect.	Pass in a valid software timer ID.
7	LOS_ERRNO_SWT_MR_NOT_CREATED	0x02000306	No software timer is created.	Create a software timer.
8	LOS_ERRNO_SWT_MR_NO_MEMORY	0x02000307	The memory is insufficient when the software timer module is initialized.	Adjust OS_SYS_MEM_SIZE to ensure that sufficient memory is available for the software timer.
9	LOS_ERRNO_SWT_MR_HWI_ACTIVE	0x02000309	A timer is used when the CPU is processing interrupts.	Modify the source code to ensure that no timer is used when the CPU is processing interrupts.
10	LOS_ERRNO_SWT_MR_QUEUE_CREATE_FAILED	0x0200030b	The software timer queue fails to be created during the initialization of the software timer.	Adjust OS_SYS_MEM_SIZE to ensure that sufficient memory is available for the software timer queue creation.

N o.	Definition	Error Code	Description	Reference Solution
1 1	LOS_ERRNO_SWT_MR_TASK_CREATE_FAILED	0x02000 30c	The software timer task fails to be created during the initialization of the software timer.	Adjust OS_SYS_MEM_SIZE to ensure that sufficient memory is available for the software timer task creation.
1 2	LOS_ERRNO_SWT_MR_NOT_STARTED	0x02000 30d	The software timer is not started.	Start a software timer.
1 3	LOS_ERRNO_SWT_MR_STATUS_INVALID	0x02000 30e	The software timer status is incorrect.	Check the software timer status.
1 4	LOS_ERRNO_SWT_MR_TICK_PTR_NULL	0x02000 310	The passed-in pointer used for obtaining the number of remaining software timer ticks is null.	Define a valid variable and pass in a valid pointer.
1 5	LOS_ERRNO_SWT_MR_SORTLINK_CREATE_FAILED	0x02000 311	The software timer linked list fails to be created during the initialization of the software timer.	Adjust OS_SYS_MEM_SIZE to ensure that sufficient memory is available for the software timer linked list creation.
1 6	LOS_ERRNO_SWT_MR_INVALID_SYNCDEL	0x02000 312	The software timer fails to be synchronously deleted during an interrupt or software timer callback.	Optimize the code logic to prevent the software timer from being deleted synchronously during an interrupt or software timer callback.

NOTICE

For details about the error code definition, see [Error Codes](#). Bits 8–15 belong to the software timer module, and the value is **0x03**.

Development Procedure

The typical software timer development process is as follows:

1. Open the menu and choose **Kernel > Enable Software Timer** to configure the software timer.

Configurati on Item	Description	Value Range	Defaul t Value	Dependency
LOSCFG_BASE_CORE_SWTMR	Whether to enable the tailoring function of the software timer module.	YES/NO	YES	LOSCFG_BASE_IPC_QUEUE
LOSCFG_BASE_CORE_SWTMR_LIMIT	Maximum number of supported software timers.	< 65535	1024	LOSCFG_BASE_CORE_SWTMR
LOSCFG_BASE_CORE_SWTMR_IN_ISR	Whether to directly execute the callback function in interrupts.	YES/NO	NO	LOSCFG_BASE_CORE_SWTMR
LOSCFG_BASE_CORE_TASK_SWTMR_STACK_SIZE (Location: Kernel > Basic Config > Task)	Size of the software timer task stack.	[LOSCFG_BASE_CORE_TSK_MIN_STACK_SIZE, OS_SYS_MEM_SIZE)	24576	LOSCFG_BASE_CORE_SWTMR

2. Call the LOS_SwtmrCreate API to create a timer, and set the time interval, timer mode, and callback function after timeout.
3. Call the LOS_SwtmrStart API to start a software timer.
4. Call the LOS_SwtmrTimeGet API to obtain the remaining number of ticks of a software timer.
5. Call the LOS_SwtmrStop API to stop a software timer.
6. Call the LOS_SwtmrDelete API to delete a software timer.

3.11.3 Precautions

- Do not perform too many operations in the callback function of the software timer. Do not use the APIs or operations that may cause task suspension or blocking. Otherwise, the software timer may not respond in a timely manner and may affect normal service functions.
- If the LOSCFG_BASE_CORE_SWTMR_IN_ISR macro is not enabled, the software timer uses a task resource of the system. The priority of a task in a software timer is set to 0, which cannot be modified.
- The number of software timers that can be configured in the system refers to the total number of software timers that can be used in the entire system rather than that can be used by users. For example, if the system software

timer occupies one more software timer, the software timers that can be used by users decreases by 1.

- A one-shot software timer that will not be automatically deleted after expiration needs to be deleted by calling the `LOS_SwtmrDelete` API. Resources allocated to the timer are reclaimed to avoid resource leaks.
- The timing precision of software timers is related to the tick clock period of the system.

3.11.4 Programming Example

Example Description

In the programming example, the following functions are demonstrated:

1. Create, delete, start, stop or restart a software timer.
2. Use a one-shot software timer and a periodical software timer.

Example Code

Prerequisite: The software timer module has been configured in the **menuconfig** menu.

The code is as follows:

```
UINT32 g_timerCount1 = 0;
UINT32 g_timerCount2 = 0;

VOID Timer1_CallBack(UINT32 arg)
{
    UINT64 lastTick;

    g_timerCount1++;
    lastTick=(UINT32)LOS_TickCountGet();
    dprintf("g_timerCount1=%d\n", g_timerCount1);
    dprintf("tick_last1=%d\n", lastTick);
}

VOID Timer2_CallBack(UINT32 arg)
{
    UINT64 lastTick;

    lastTick=(UINT32)LOS_TickCountGet();
    g_timerCount2++;
    dprintf("g_timerCount2=%d\n", g_timerCount2);
    dprintf("tick_last2=%d\n", lastTick);
}

VOID Timer_example(VOID)
{
    UINT16 id1;    // Timer1 id
    UINT16 id2;    // Timer2 id
    UINT32 tick;

    LOS_SwtmrCreate(1000, LOS_SWTMR_MODE_ONCE, Timer1_CallBack, &id1, 1);
    LOS_SwtmrCreate(100, LOS_SWTMR_MODE_PERIOD, Timer2_CallBack, &id2, 1);
    dprintf("create Timer1 success\n");

    LOS_SwtmrStart(id1);
    dprintf("start Timer1 sucess\n");
    LOS_TaskDelay(200);
    LOS_SwtmrTimeGet(id1, &tick);
    dprintf("tick =%d\n", tick);
}
```

```
    LOS_SwtmrStop(id1);  
    dprintf("stop Timer1 sucess\n");  
  
    LOS_SwtmrStart(id1);  
    LOS_TaskDelay(1000);  
    LOS_SwtmrDelete(id1);  
    dprintf("delete Timer1 sucess\n");  
  
    LOS_SwtmrStart(id2);  
    dprintf("start Timer2\n");  
    LOS_TaskDelay(1000);  
    LOS_SwtmrStop(id2);  
    LOS_SwtmrDelete(id2);  
}
```

Verification

The verification result is as follows:

```
Create Timer1 success  
start Timer1 sucess  
tick =800  
Stop Timer1 sucess  
g_timerCount1=1201  
tick_last1=1201  
delete Timer1 success  
Start Timer2  
g_timerCount2=1  
tick_last2=1301  
g_timerCount2=2  
tick_last2=1401  
g_timerCount2=3  
tick_last2=1501  
g_timerCount2=4  
tick_last2=1601  
g_timerCount2=5  
tick_last2=1701  
g_timerCount2=6  
tick_last2=1801  
g_timerCount2=7  
tick_last2=1901  
g_timerCount2=8  
tick_last2=2001  
g_timerCount2=9  
tick_last2=2101  
g_timerCount2=10  
tick_last2=2201
```

Complete Code

sample_Timer.c

3.12 Spinlock

3.12.1 Overview

In the SMP environment, multiple tasks may access the same resource due to the shared memory. An exclusive access mechanism is required to ensure that only one core performs operations at a time. Spinlocks are used as such a mechanism.

A spinlock is a lock which causes a thread trying to acquire it to simply wait in a loop ("spin") while repeatedly checking if the lock is available. The thread exits

the loop only until the lock is acquired. Therefore, spinlocks are recommended for operations that take a short time, preventing adverse impact on system performance.

Similar to mutexes, spinlocks are designed to grant a task exclusive use of shared resources. Either a mutex or a spinlock can have only one holder at a given point in time. However, the scheduling mechanisms of the two modes are slightly different. For a mutex, if a common resource is occupied, other allocators will be blocked. A spinlock, however, does not cause caller blocking, and allocators keep checking whether the holder releases the spinlock.

3.12.2 Development Guidelines

Application Scenario

A spinlock provides a mutual exclusion mechanism to prevent two tasks from accessing the same shared resource at the same time.

Functions

The spinlock module of Huawei LiteOS provides the following functions. For details about the APIs, see the API reference.

Function Category	API	Description	Precautions
Spinlock initialization	LOS_SpinInit	Dynamically initializes a spinlock.	Ensure that the input parameter is valid.
	SPIN_LOCK_INIT	Statically initializes a spinlock.	N/A.
Spinlock allocation and release	LOS_SpinLock	Allocates a specified spinlock. If the spinlock cannot be obtained, the system waits in a loop.	Ensure that the input parameter is valid.
	LOS_SpinTryLock	Attempts to allocate a specified spinlock. If the spinlock cannot be obtained, a failure message is returned, and the system does not wait in a loop.	Ensure that the input parameter is valid.
	LOS_SpinUnlock	Releases a spinlock.	Ensure ensure that the input parameter is valid.

Function Category	API	Description	Precautions
	LOS_SpinUnl ockNoSched	Releases a specified spinlock and continuously executes the task. (Use this function in special scenarios by referring to the comments to avoid incorrect scheduling time sequence.)	Ensure that the input parameter is valid.
Spinlock allocation and release (with disabled interrupt)	LOS_SpinLoc kSave	Allocates a spinlock after the interrupt is disabled.	Ensure that the input parameter is valid.
	LOS_SpinUnl ockRestore	Releases a spinlock and restores the interrupt.	Ensure that the input parameter is valid.
Spinlock holding status	LOS_SpinHel d	Checks whether a spinlock is held.	Ensure that the input parameter is valid.

Development Procedure

The typical spinlock development process is as follows:

1. Spinlocks depend on the SMP. Open the menu and choose **Kernel > Enable Kernel SMP** to configure the spinlock.

Configuration Item	Description	Value Range	Default Value	Dependency
LOSCFG_KERNEL_SMP	Whether to enable the SMP.	YES/NO	YES	The hardware supports SMP mode.
LOSCFG_KERNEL_SMP_CORE_NUM	Number of cores in the SMP scenario.	Depends on the architecture.	2	None.

2. To create a spinlock, call the LOS_SpinInit API to initialize the spinlock, or call SPIN_LOCK_INIT API to initialize the spinlock of the static memory.
3. Call the LOS_SpinLock, LOS_SpinTrylock, or LOS_SpinLockSave API to allocate a spinlock. If the allocation is successful, the code for lock protection is executed. If the allocation fails, the system enters a busy loop till a spinlock is obtained.

4. Call the `LOS_SpinUnlock` or `LOS_SpinUnlockRestore` API to release a spinlock. After the lock protection code is executed, the corresponding spinlock is released, so that other cores can allocate the spinlock.

3.12.3 Precautions

- Ensure that the input parameter is valid.
- Tasks are unable to lock the same spinlock for multiple times, which causes a deadlock.
- Spinlocks are used to lock tasks. Therefore, task scheduling can be performed only after the outermost layer is unlocked.
- `LOS_SpinLock` and `LOS_SpinUnlock` can be used independently. That is, the interrupt can be left enabled, but you need to ensure that the called APIs are used either in tasks or interrupts. If the called APIs are used in both tasks and interrupts, use `LOS_SpinLockSave` and `LOS_SpinUnlockRestore`. In this case, using `LOS_SpinLock` may cause deadlock when the interrupt is enabled.
- Exercise caution when using spinlocks for time-consuming operations. You can use mutexes for protection.
- In the UP scenario with SMP disabled, the spinlock function is invalid. Only the `LOS_SpinLockSave` and `LOS_SpinUnlockRestore` APIs can disable the interrupt recovery function.
- It is recommended that `LOS_SpinLock` and `LOS_SpinUnlock` be used together, and `LOS_SpinLockSave` and `LOS_SpinUnlockRestore` be used together to avoid errors.

3.12.4 Programming Example

Example Description

The following functions are implemented:

1. The `Example_TaskEntry` task is executed to initialize a spinlock. Two tasks `Example_SpinTask1` and `Example_SpinTask2` are created, which run on two cores respectively.
2. Both `Example_SpinTask1` and `Example_SpinTask2` allocate the spinlock, and a certain delay is performed after the spinlock is held, to simulate an actual operation.
3. After 300 ticks, `Example_TaskEntry` is scheduled and executed to delete `Example_SpinTask1` and `Example_SpinTask2`.

NOTICE

The running sequence in SMP is not fixed. Therefore, the task execution sequence may be different.

Example Code

Prerequisite: The configuration item **LOSCFG_KERNEL_SMP** has been enabled in the **menuconfig** menu and the number of cores has been set.

The code is as follows:

```
#include "los_spinlock.h"
#include "los_task.h"

/* Spinlock handler ID */
SPIN_LOCK_S g_testSpinlock;
/* Task ID */
UINT32 g_testTaskId01;
UINT32 g_testTaskId02;

VOID Example_SpinTask1(VOID)
{
    UINT32 i;
    UINTPTR intSave;

    /* Allocate a spinlock. */
    dprintf("task1 try to get spinlock\n");
    LOS_SpinLockSave(&g_testSpinlock, &intSave);
    dprintf("task1 got spinlock\n");
    for(i = 0; i < 5000; i++) {
        asm volatile("nop");
    }

    /* Release the spinlock. */
    dprintf("task1 release spinlock\n");
    LOS_SpinUnlockRestore(&g_testSpinlock, intSave);

    return;
}

VOID Example_SpinTask2(VOID)
{
    UINT32 i;
    UINTPTR intSave;

    /* Allocate a spinlock. */
    dprintf("task2 try to get spinlock\n");
    LOS_SpinLockSave(&g_testSpinlock, &intSave);
    dprintf("task2 got spinlock\n");
    for(i = 0; i < 5000; i++) {
        asm volatile("nop");
    }

    /* Release the spinlock. */
    dprintf("task2 release spinlock\n");
    LOS_SpinUnlockRestore(&g_testSpinlock, intSave);

    return;
}

UINT32 Example_TaskEntry(VOID)
{
    UINT32 ret;
    TSK_INIT_PARAM_S stTask1;
    TSK_INIT_PARAM_S stTask2;

    /* Initialize the spinlock. */
    LOS_SpinInit(&g_testSpinlock);

    /* Create task1. */
    memset(&stTask1, 0, sizeof(TSK_INIT_PARAM_S));
    stTask1.pfnTaskEntry = (TSK_ENTRY_FUNC)Example_SpinTask1;
    stTask1.pcName = "SpinTsk1";
    stTask1.uwStackSize = LOSCFG_BASE_CORE_TSK_MIN_STACK_SIZE;
    stTask1.usTaskPrio = 5;
#ifdef LOSCFG_KERNEL_SMP
    /* Bind tasks to CPU0 for running. */
    stTask1.usCpuAffiMask = CPUID_TO_AFFI_MASK(0);
#endif
    return ret;
}
```

```
ret = LOS_TaskCreate(&g_testTaskId01, &stTask1);
if(ret != LOS_OK) {
    dprintf("task1 create failed .\n");
    return LOS_NOK;
}

/* Create task2. */
memset(&stTask2, 0, sizeof(TSK_INIT_PARAM_S));
stTask2.pfnTaskEntry = (TSK_ENTRY_FUNC)Example_SpinTask2;
stTask2.pcName       = "SpinTsk2";
stTask2.uwStackSize  = LOSCFG_BASE_CORE_TSK_MIN_STACK_SIZE;
stTask2.usTaskPrio   = 5;
#ifdef LOSCFG_KERNEL_SMP
/* Bind tasks to CPU1 for running. */
stTask1.usCpuAffiMask = CPUID_TO_AFFI_MASK(1);
#endif
ret = LOS_TaskCreate(&g_testTaskId02, &stTask2);
if(ret != LOS_OK) {
    dprintf("task2 create failed .\n");
    return LOS_NOK;
}

/* Enable the task to enter sleep mode for 300 ticks. */
LOS_TaskDelay(300);

/* Delete task1. */
ret = LOS_TaskDelete(g_testTaskId01);
if(ret != LOS_OK) {
    dprintf("task1 delete failed .\n");
    return LOS_NOK;
}
/* Delete task2. */
ret = LOS_TaskDelete(g_testTaskId02);
if(ret != LOS_OK) {
    dprintf("task2 delete failed .\n");
    return LOS_NOK;
}

return LOS_OK;
}
```

Verification

The verification result is as follows:

```
task2 try to get spinlock
task2 got spinlock
task1 try to get spinlock
task2 release spinlock
task1 got spinlock
task1 release spinlock
```

Complete Code

sample_spinlock.zip

3.13 Levels of Initialization

3.13.1 Overview

Basic Concept

The kernel startup process is divided into multiple phases. The later initialization phase depends on the APIs in the previous initialization phase. In case of new calls during kernel startup, you do not need to modify the startup code to add function calls. Instead, you can use different levels of initialization macros to register the APIs. When the kernel is started, all registered initialization APIs are automatically executed in sequence.

Operation Mechanism

- For each function that needs to be executed during kernel initialization, a function pointer variable is generated by using the initialization macro to point to the function.
- When an image is generated during compilation, the link script prefixes the preceding function pointer variables.
- During kernel initialization, all function pointers are extracted by prefix to execute all initialization functions in sequence.

3.13.2 Development Guidelines

Application Scenario

By dividing initialization to levels, the kernel is decoupled from other modules. The kernel is not aware of specific modules to be initialized and modules do not care how they are initialized.

Function

Huawei LiteOS provides the following function to divide initialization to levels. For details about the API, see the API reference.

API	Description
LOS_SYS_INIT	Registers an initialization function, which is run in the kernel.

The initialization is divided into the following levels, among which the former one is executed first.

Level of Initialization Macro	Description
SYS_INIT_LEVEL_EARLY	The early phase of kernel startup is used only by users. The initialization functions registered in this phase cannot depend on kernel resources.

Level of Initialization Macro	Description
SYS_INIT_LEVEL_AHEAD	Kernel startup preparation phase is used only in system modules to prepare for kernel initialization.
SYS_INIT_LEVEL_ARCH	This phase is used only in system modules to initialize architectures.
SYS_INIT_LEVEL_KERNEL	This phase is used only in system modules to initialize basic kernel submodules.
SYS_INIT_LEVEL_KERNEL_ADDITION	This phase is used only in system modules to initialize extended kernel modules.
SYS_INIT_LEVEL_COMPONENT	This phase is used only in system modules to initialize components.
SYS_INIT_LEVEL_APP	This phase is used to initialize applications.
SYS_INIT_LEVEL_RESERVE	The last kernel startup phase is used by users. The initialization functions registered in this phase can depend on kernel resources.

At the same level, one initialization function may depend on another. You can set the priority for a function. The **sync** value can only be used to prioritize functions at the same level. A smaller **sync** value indicates a higher priority and earlier execution.

Priority of Initialization Macro	Description
SYS_INIT_SYNC_0	Sync 0, which is executed first at the same level.
SYS_INIT_SYNC_1	Sync 1
SYS_INIT_SYNC_2	Sync 2
SYS_INIT_SYNC_3	Sync 3
SYS_INIT_SYNC_4	Sync 4

Development Procedure

1. Determine the dependency among initialization APIs, and then determine their initialization levels and priorities.
2. Register initialization APIs by calling LOS_SYS_INIT.

3.13.3 Precautions

- Only functions with typedef unsigned int (*SysinitcallFunc)(void) can be used for initialization at different levels.

- No initialization function is directly called in the code. Some compilers may optimize the initialization function, causing registration failures. You are advised to use the **-u** link option.
- Multiple initialization APIs can be registered at the same level and with the same **sync** value, but these APIs cannot depend on each other.

3.13.4 Programming Example

Description

The following example describes how to divide initialization to different levels of registration APIs:

1. Determine the dependency among initialization APIs, and their levels and priorities.
2. Register the initialization APIs.

Example Code

```
#include "stdio.h"
#include "los_init.h"
#include "los_list.h"
#include "los_hwi.h"

#ifdef __cplusplus
#if __cplusplus
extern "C" {
#endif /* __cplusplus */
#endif /* __cplusplus */

UINT32 g_count;
LOS_DL_LIST g_DLlist;

UINT32 InitLevelTest1(VOID)
{
    dprintf("InitLevelTest1 init\n");
    g_count = 10;
    LOS_ListInit(&g_DLlist);

    return LOS_OK;
}
/* InitLevelTest1 has no dependence, set SYS_INIT_LEVEL_EARLY level */
LOS_SYS_INIT(InitLevelTest1, SYS_INIT_LEVEL_EARLY, SYS_INIT_SYNC_0);

STATIC VOID HwiUsrIrq(VOID)
{
    g_count++;
}

UINT32 InitLevelTest2(VOID)
{
    UINT32 ret;
    HWI_HANDLE_T irqNum = 26;
    HWI_PRIOR_T irqPri = 0x3;

    dprintf("InitLevelTest2 init\n");
    ret = LOS_HwiCreate(irqNum, irqPri, 0, (HWI_PROC_FUNC)HwiUsrIrq, 0);
    if (ret != LOS_OK) {
        return LOS_NOK;
    }
    return ret;
}
/* InitLevelTest2 rely on hwi module, set SYS_INIT_LEVEL_RESERVE level */
LOS_SYS_INIT(InitLevelTest2, SYS_INIT_LEVEL_RESERVE, SYS_INIT_SYNC_0);
```

```
#ifdef __cplusplus
#if __cplusplus
}
#endif /* __cplusplus */
#endif /* __cplusplus */
```

Result Verification

The verification result is as follows:

```
InitLevelTest1 init
InitLevelTest2 init
```

Complete Code

sample_initlevel.rar

3.14 Others

3.14.1 Time Management

3.14.1.1 Overview

Basic Concepts

Time management provides time services to applications and uses system time as the reference time.

System time is generated when an output pulse of a timer/counter triggers an interrupt, it is an integral number or long integral number of ticks. The period of the output pulse is called a "clock tick". The system clock is also called time scale or tick.

User time is measured in seconds or milliseconds, whereas the OS clock measures time in ticks. When a user initiates an operation to the operating system, such as suspending or delaying a task, the time management module converts user time between seconds/milliseconds and ticks.

The time management module of Huawei LiteOS provides time conversion, measurement, and deferral to satisfy what users need.

Related Concepts

- Cycle
Cycle is the minimal time unit of the operating system. The system clock speed is represented in the form of cycles per second.
- Tick
A tick is the basic time unit used in OS. The tick length is user configurable. Typically, it is determined by the system clock speed and represented in the form of ticks per second.

3.14.1.2 Development Guidelines

Application Scenario

Read the topic when you want to learn more about system time and conversion between ticks and seconds/milliseconds.

Functions

The time management module of Huawei LiteOS provides the following functions. For details about the APIs, see the API reference.

Function Category	API	Description
Time conversion	LOS_MS2Tick	Uses the rounding-up method to convert milliseconds to ticks.
	LOS_Tick2MS	Converts ticks into milliseconds
Time statistics	LOS_CyclePerTickGet	Obtains the number of cycles for each tick.
	LOS_TickCountGet	Obtains the number of ticks since the system starts.
	LOS_GetCpuCycle	Obtains the number of cycles since the system starts.
	LOS_CurrNanosec	Obtains the number of nanoseconds since the system starts.
Delay management	LOS_Udelay	Obtains the busy waiting in μ s. Resources can be preempted by another task with a higher priority.
	LOS_Mdelay	Obtains the busy waiting in ms. Resources can be preempted by another task with a higher priority.

Time Management Error Codes

Error codes are returned if errors occur during time conversion. The error codes can facilitate fault locating.

N o.	Definition	Error Code	Description	Reference Solution
1	LOS_ERRNO_TICK_CFG_INVALID	0x02000400	The system tick configuration is invalid.	Configure a valid system main clock frequency OS_SYS_CLOCK when configuring board-level adaption, and open the menu to configure LOSCFG_BASE_CORE_TICK_PER_SECOND .

Development Procedure

The typical time management development process is as follows:

1. Determine whether to enable the **LOSCFG_BASE_CORE_TICK_HW_TIME** macro based on site requirements to select an external timer and configure the system main clock frequency **OS_SYS_CLOCK** (unit: Hz) when configuring board-level adaption. The default value of **OS_SYS_CLOCK** is configured based on the hardware platform.
2. Open the menu and choose **Kernel > Basic Config > Task** to configure **LOSCFG_BASE_CORE_TICK_PER_SECOND**.

Configuration Item	Description	Value Range	Default Value	Dependency
LOSCFG_BASE_CORE_TICK_PER_SECOND	Number of ticks per second.	(0, 1000]	100	None.

3. Call the clock conversion/statistics API.

3.14.1.3 Precautions

- The time management module depends on **OS_SYS_CLOCK** and **LOSCFG_BASE_CORE_TICK_PER_SECOND**.
- When measured in ticks, system runtime is not accurate, because it is not measured while interrupts are disabled.

3.14.1.4 Programming Example

Example Description

The programming example will cover the following functions:

1. Time conversion: from milliseconds to ticks, or conversely
2. Time measurement and deferral: measures the number of cycles per second, the number of ticks for which the system is running, and the number of ticks for which the deferral lasts

Example Code

Prerequisites

- The default value **100** of **LOSCFG_BASE_CORE_TICK_PER_SECOND** is used. This configuration item specifies the number of ticks in a second.
- The system main clock frequency **OS_SYS_CLOCK** (in Hz) is set.

Time conversion:

```
VOID Example_TransformTime(VOID)
{
    UINT32 ms;
    UINT32 tick;

    tick = LOS_MS2Tick(10000); //Convert 10000 ms into ticks
    dprintf("tick = %d \n", tick);
    ms = LOS_Tick2MS(100); //Convert 100 ticks into ms
    dprintf("ms = %d \n", ms);
}
```

Time measurement and delay:

```
VOID Example_GetTime(VOID)
{
    UINT32 cyclePerTick;
    UINT64 tickCount;

    cyclePerTick = LOS_CyclePerTickGet();
    if(0 != cyclePerTick) {
        dprintf("LOS_CyclePerTickGet = %d \n", cyclePerTick);
    }

    tickCount = LOS_TickCountGet();
    if(0 != tickCount) {
        dprintf("LOS_TickCountGet = %d \n", (UINT32)tickCount);
    }

    LOS_TaskDelay(200);
    tickCount = LOS_TickCountGet();
    if(0 != tickCount) {
        dprintf("LOS_TickCountGet after delay = %d \n", (UINT32)tickCount);
    }
}
```

Verification

The verification result is as follows:

Time conversion:

```
tick = 1000
ms = 1000
```

Time measurement and delay:

```
LOS_CyclePerTickGet = 495000
LOS_TickCountGet = 1
LOS_TickCountGet after delay = 201
```

Complete Code

sample_time.c

3.14.2 Doubly Linked List

3.14.2.1 Overview

Basic Concepts

A doubly linked list is a linked data structure that consists of a set of sequentially linked records called nodes. Each node in a doubly linked list contains two pointers that reference to the previous and to the next node in the sequence of nodes. The head of the doubly linked list is deterministic and immediately accessible.

Any node of a doubly linked list, once obtained, can be used to initiate a new traversal of the list in either direction (towards the beginning or end) from the given node. This allows a lot of data to be quickly traversed. Because of the symmetric nature of a doubly linked list, nodes can easily be inserted into or removed from the list.

3.14.2.2 Development Guidelines

Functions

The doubly linked list module of Huawei LiteOS provides the following functions. For details about the APIs, see the API reference.

Function Category	API	Description	Precautions
List initialization	LOS_ListInit	Initializes a specified node to a doubly linked list node.	Ensure that the input parameter is valid.
	LOS_DL_LIST_HEAD	Defines a node and initializes it to a doubly linked list node.	N/A.
Node insertion	LOS_ListAdd	Inserts a specified node to the head of a doubly linked list.	Ensure that the input parameter is valid.
	LOS_ListHeadInsert	Inserts a specified node to the head of a doubly linked list, which is the same as LOS_ListAdd.	Ensure that the input parameter is valid.
	LOS_ListTailInsert	Inserts a specified node to the tail of a doubly linked list.	Ensure that the input parameter is valid.
Node deletion	LOS_ListDelete	Deletes a specified node from a doubly linked list.	Ensure that the input parameter is valid.

Function Category	API	Description	Precautions
	LOS_ListDelInit	Deletes a specified node from the linked list and uses the node to initialize the linked list.	Ensure that the input parameter is valid.
Checking whether a doubly linked list is empty	LOS_ListEmpty	Checks whether a doubly linked list is empty.	Ensure that the input parameter is valid.
Obtaining a node	LOS_DL_LIST_LAST	Obtains the previous node of a specified node.	N/A.
	LOS_DL_LIST_FIRST	Obtains the next node of a specified node.	N/A.
Obtaining structure information	LOS_DL_LIST_ENTRY	Obtains the address of the structure that contains the linked list. The first input parameter of the API indicates a node in the linked list, the second input parameter indicates the name of the structure to be obtained, and the third input parameter indicates the name of the linked list in the structure.	N/A.
	LOS_OFF_SET_OF	Obtains the offset of a member in a specified structure relative to the start address of the structure.	N/A.
Traversing the doubly linked list	LOS_DL_LIST_FOR_EACH	Traverses a doubly linked list.	N/A.
	LOS_DL_LIST_FOR_EACH_SAFE	Traverses a doubly linked list, and stores the next node of the current node for security verification.	N/A.
Traversing the structure that contains the doubly linked list	LOS_DL_LIST_FOR_EACH_ENTRY	Traverses a specified doubly linked list and obtain the address of the structure that contains the linked list node.	N/A.

Function Category	API	Description	Precautions
	LOS_DL_LIST_FOR_EACH_ENTRY_SAFE	Traverses a specified doubly linked list, obtains the structure address of the node that contains the linked list, and stores the structure address that contains the next node of the current node.	N/A.
	LOS_DL_LIST_FOR_EACH_ENTRY_HOOK	Traverses a specified doubly linked list, obtain the structure address of the link list node, and call the hook function in each loop.	N/A.

Development Procedure

The doubly linked list development process is as follows:

1. Call the LOS_ListInit or LOS_DL_LIST_HEAD API to initialize the doubly linked list.
2. Call the LOS_ListAdd or LOS_ListHeadInsert API to insert a node to the head of the linked list.
3. Call the LOS_ListTailInsert API to insert a node into the tail of the list.
4. Call the LOS_ListDelete API to delete a node from the list.
5. Call the LOS_ListEmpty API to determine whether the doubly linked list is empty.
6. Call the LOS_ListDelInit API to delete a node and use the node to initialize the doubly linked list.

3.14.2.3 Precautions

- Ensure that the input parameter is valid.
- Pay attention to the operations of the front and back pointer of the node.
- The linked list operation APIs are underlying APIs and do not check whether the input parameters are empty. You need to ensure that the input parameters are valid.
- If the memory of a linked list node is dynamically allocated, release the memory when deleting the node.

3.14.2.4 Programming Example

Example Description

The following functions are implemented:

1. Initialize a doubly linked list.
2. Insert a node into the list.
3. Delete a node from the list.
4. Check whether the insertion and deletion was successful.

Example Code

The code is as follows:

```
#include "stdio.h"
#include "los_list.h"

#ifdef __cplusplus
#if __cplusplus
extern "C" {
#endif /* __cplusplus */
#endif /* __cplusplus */

static UINT32 DList_sample(VOID)
{
    LOS_DL_LIST DList = {NULL,NULL};
    LOS_DL_LIST DListNode01 = {NULL,NULL};
    LOS_DL_LIST DListNode02 = {NULL,NULL};
    LOS_DL_LIST DListNode03 = {NULL,NULL};

    dprintf("Initial head\n");
    LOS_ListInit(&DList);

    LOS_ListAdd(&DList, &DListNode01);
    if (DListNode01.pstNext == &DList && DListNode01.pstPrev == &DList) {
        dprintf("Add DListNode01 success \n");
    }

    LOS_ListTailInsert(&DList, &DListNode02);
    if (DListNode02.pstNext == &DList && DListNode02.pstPrev == &DListNode01) {
        dprintf("Tail insert DListNode02 success \n");
    }

    LOS_ListHeadInsert(&DListNode02, &DListNode03);
    if (DListNode03.pstNext == &DList && DListNode03.pstPrev == &DListNode02) {
        dprintf("Head insert DListNode03 success \n");
    }

    LOS_ListDelInit(&DListNode03);
    LOS_ListDelete(&DListNode01);
    LOS_ListDelete(&DListNode02);

    if (LOS_ListEmpty(&DList)) {
        dprintf("Delete success \n");
    }

    return LOS_OK;
}

#ifdef __cplusplus
#if __cplusplus
}
#endif /* __cplusplus */
#endif /* __cplusplus */
```

Verification

The verification result is as follows:

```
Initial head
Add DListNode01 success
```

```
Tail insert DListNode02 success  
Head insert DListNode03 success  
Delete success
```

3.14.3 Atomic Operation

3.14.3.1 Overview

Basic Concepts

In a multi-tasking operating system (OS), data reading, modification, and writing are three essential steps of modifying the data in a specified memory. However, the data may be concurrently accessed by multiple tasks. If the data modification is interrupted by other tasks, unexpected modification result will occur.

Multiple tasks can be successfully executed by enabling and disabling interrupts, but this method affects OS performance.

The ARMv6 architecture introduces the LDREX and STREX instructions to support more discreet non-blocking synchronization of the shared memory. The atomic operation ensures that a "read-modify-write" operation on same data is not interrupted during execution of the operation, that is, atomicity of the operation.

Operation Mechanism

Huawei LiteOS provides atomic operation APIs by encapsulating LDREX and STREX of the ARMv6 architecture.

- **LDREX Rx, [Ry]**

The following is the method of reading data in the memory and mark the exclusive access to the memory:

- Read the 4-byte memory data pointed to by the Ry register and store the read memory data to the Rx register.
- Add an exclusive access flag to the memory segment pointed by Ry.

- **STREX Rf, Rx, [Ry]**

The following describes the method of checking the memory has the exclusive access flag, and updating the memory and clearing the flag if the flag exists:

- If the memory has an exclusive access flag, the memory data will be updated.
 - i. Update the memory data pointed to the Ry register to the value in the Rx register.
 - ii. Set the Rf flag register to 0.
- If the memory does not have an exclusive access flag, the memory data should be updated.
 - i. The memory data pointed to by Ry will not be updated.
 - ii. Set the Rf flag register to 1.

- **Flag register**

- If the flag register is 0, the loop exits and the atomic operation ends.

- If the flag register is 1, the atomic operation cycle proceeds and the atomic operation starts again.

3.14.3.2 Development Guidelines

Application Scenario

When multiple tasks are performing increasing, decreasing, and exchanging operations on the same memory data, the use of atomic operations will ensure that operation results are predictable.

Functions

Atomic data of Huawei LiteOS includes two types: Atomic (32-bit signed) and Atomic64 (64-bit signed). The atomic operation module of Huawei LiteOS provides the following functions:

Table 3-3 Atomic operation functions

Function Category	API	Description	Precautions
Read	LOS_AtomicRead	Reads memory data.	Ensure that the input parameter is valid.
Write	LOS_AtomicSet	Writes memory data.	Ensure that the input parameter is valid.
Add	LOS_AtomicAdd	Adds data to the memory data.	Ensure that the input parameter is valid.
	LOS_AtomicSub	Subtracts the memory data.	Ensure that the input parameter is valid.
	LOS_AtomicInc	Adds one to the memory data.	Ensure that the input parameter is valid.
	LOS_AtomicIncRet	Adds one to the memory data and returns the result.	Ensure that the input parameter is valid.
Subtract	LOS_AtomicDec	Subtracts one from the memory data.	Ensure that the input parameter is valid.
	LOS_AtomicDecRet	Subtracts one from the memory data and return the result.	Ensure that the input parameter is valid.
Swap	LOS_AtomicXchg32bits	Swaps the memory data. The original value in the memory is returned as a return value.	Ensure that the input parameter is valid.

Function Category	API	Description	Precautions
	LOS_AtomicCmpXchg32bits	Compares and swaps memory data, and returns the comparison result.	Ensure that the input parameter is valid.

Table 3-4 Atomic64 atomic operations

Function Category	API	Description	Precautions
Read	LOS_Atomic64Read	Reads the 64-bit memory data.	Ensure that the input parameter is valid.
Write	LOS_Atomic64Set	Writes the 64-bit memory data.	Ensure that the input parameter is valid.
Add	LOS_Atomic64Add	Adds data to the 64-bit memory data.	Ensure that the input parameter is valid.
	LOS_Atomic64Sub	Subtracts the 64-bit memory data.	Ensure that the input parameter is valid.
	LOS_Atomic64Inc	Adds one to the 64-bit memory data.	Ensure that the input parameter is valid.
	LOS_Atomic64IncRet	Adds one to the 64-bit memory data and returns the operation result.	Ensure that the input parameter is valid.
Subtract	LOS_Atomic64Dec	Subtracts one from the 64-bit memory data.	Ensure that the input parameter is valid.
	LOS_Atomic64DecRet	Subtracts one to the 64-bit memory data and returns the operation result.	Ensure that the input parameter is valid.
Swap	LOS_AtomicXchg64bits	Swaps the 64-bit memory data. The original value in the memory is returned as a return value.	Ensure that the input parameter is valid.
	LOS_AtomicCmpXchg64bits	Compares and swaps the 64-bit memory data and returns the comparison result.	Ensure that the input parameter is valid.

NOTICE

In an atomic operation, the number of operands and their results cannot exceed the maximum number of bits supported by the function.

Platform Differences

None.

3.14.3.3 Precautions

- Ensure that the input parameter is valid.
- Currently, atomic operation APIs support only integers.
- The atomic operation can only ensure the atomicity of the operation, but cannot ensure that the operation does not cause reversal overflow.

3.14.3.4 Programming Example

Example Description

Perform the following two steps and view the result:

Step 1 Create two tasks.

1. Task 1: Call `LOS_AtomicInc` to increase the global variable by one for 100 times.
2. Task 2: Call `LOS_AtomicDec` to decrease the global variable by one for 100 times.

Step 2 After the subtasks are complete, print the global variable value in the main task.

----End

Example Code

```
#include "los_hwi.h"
#include "los_atomic.h"
#include "los_task.h"

UINT32 g_testTaskId01;
UINT32 g_testTaskId02;
Atomic g_sum;
Atomic g_count;

UINT32 Example_Atomic01(VOID)
{
    int i = 0;
    for(i = 0; i < 100; ++i) {
        LOS_AtomicInc(&g_sum);
    }

    LOS_AtomicInc(&g_count);
    return LOS_OK;
}

UINT32 Example_Atomic02(VOID)
{
    int i = 0;
```

```
    for(i = 0; i < 100; ++i) {
        LOS_AtomicDec(&g_sum);
    }

    LOS_AtomicInc(&g_count);
    return LOS_OK;
}

UINT32 Example_TaskEntry(VOID)
{
    TSK_INIT_PARAM_S stTask1={0};
    stTask1.pfnTaskEntry = (TSK_ENTRY_FUNC)Example_Atomic01;
    stTask1.pcName       = "TestAtomicTsk1";
    stTask1.uwStackSize  = LOSCFG_BASE_CORE_TSK_DEFAULT_STACK_SIZE;
    stTask1.usTaskPrio   = 4;
    stTask1.uwResved     = LOS_TASK_STATUS_DETACHED;

    TSK_INIT_PARAM_S stTask2={0};
    stTask2.pfnTaskEntry = (TSK_ENTRY_FUNC)Example_Atomic02;
    stTask2.pcName       = "TestAtomicTsk2";
    stTask2.uwStackSize  = LOSCFG_BASE_CORE_TSK_DEFAULT_STACK_SIZE;
    stTask2.usTaskPrio   = 4;
    stTask2.uwResved     = LOS_TASK_STATUS_DETACHED;

    LOS_TaskLock();
    LOS_TaskCreate(&g_testTaskId01, &stTask1);
    LOS_TaskCreate(&g_testTaskId02, &stTask2);
    LOS_TaskUnlock();

    while(LOS_AtomicRead(&g_count) != 2);
    dprintf("g_sum = %d\n", g_sum);

    return LOS_OK;
}
```

Verification

```
g_sum = 0
```

Complete Code

```
sample_atomic.c
```

3.14.4 Bit Operation

3.14.4.1 Overview

Basic Concepts

A bitwise operation refers to the bit operation on binary numerals. A program may set a variable as a status word, and each bit (flag bit) in the status word may have a self-defined meaning.

3.14.4.2 Development Guidelines

Application Scenario

Huawei LiteOS can set a flag bit to 1 or clear the bit, change the flag bit content, and obtain the most significant bit (MSB) and the least significant bit (LSB) with a

flag bit of 1 from the SW. Developers can also perform bit operations on system registers.

Functions

The bit operation module of Huawei LiteOS provides the following functions. For details about the APIs, see the API reference.

Function Category	API	Description
Setting the flag bit to 1 or 0	LOS_BitmapSet	Sets a flag bit of an SW to 1.
	LOS_BitmapClr	Clears a flag bit of an SW to 0.
Obtaining the MSB with a flag bit of 1	LOS_HighBitGet	Obtains the MSB with a flag bit of 1 in the SW.
	LOS_LowBitGet	Obtains the LSB with a flag bit of 1 in the SW.

3.14.4.3 Precautions

None.

3.14.4.4 Programming Example

Example Description

In this example, the following functions will be implemented by performing bit operations on data:

1. Set a flag bit to 1.
2. Obtain the MSB with a flag bit of 1.
3. Set a flag bit to 0.
4. Obtain the LSB with a flag bit of 1.

Example Code

```
#include "los_bitmap.h"
#include "los_printf.h"

static UINT32 Bit_Sample(VOID)
{
    UINT32 flag = 0x10101010;
    UINT16 pos;

    dprintf("\nBitmap Sample!\n");
    dprintf("The flag is 0x%8x\n", flag);

    pos = 8;
    LOS_BitmapSet(&flag, pos);
    dprintf("LOS_BitmapSet:\t pos : %d, the flag is 0x%0+8x\n", pos, flag);

    pos = LOS_HighBitGet(flag);
    dprintf("LOS_HighBitGet:\t The highest one bit is %d, the flag is 0x%0+8x\n", pos, flag);
}
```

```

    LOS_BitmapClr(&flag, pos);
    dprintf("LOS_BitmapClr:\t pos : %d, the flag is 0x%0+8x\n", pos, flag);

    pos = LOS_LowBitGet(flag);
    dprintf("LOS_LowBitGet:\t The lowest one bit is %d, the flag is 0x%0+8x\n\n", pos, flag);

    return LOS_OK;
}

```

Verification

The verification result is as follows:

```

Bitmap Sample!
The flag is 0x10101010
LOS_BitmapSet: pos : 8, the flag is 0x10101110
LOS_HighBitGet:The highest one bit is 28, the flag is 0x10101110
LOS_BitmapClr: pos : 28, the flag is 0x00101110
LOS_LowBitGet: The lowest one bit is 4, the flag is 0x00101110

```

3.14.5 ringbuf

3.14.5.1 Overview

Basic Concepts

The ringbuf is a ring buffer mechanism of data, and can implement buffering of a data stream in a first in first out manner.

ringbuf control block

```

typedef struct {
    UINT32 startIdx;           /* ringbuf read subscript */
    UINT32 endIdx;             /* ringbuf write subscript */
    UINT32 size;                /* Total ringbuf size, in bytes */
    UINT32 remain;             /* Remaining available size of ringbuf, in bytes */
    SPIN_LOCK_S lock;          /* ringbuf spin lock */
    RingbufStatus status;      /* ringbuf status */
    RingbufFlag flag;          /* Ringbuf flag */
    CHAR *fifo;                /* Start address of the ringbuf buffer */
} Ringbuf;

```

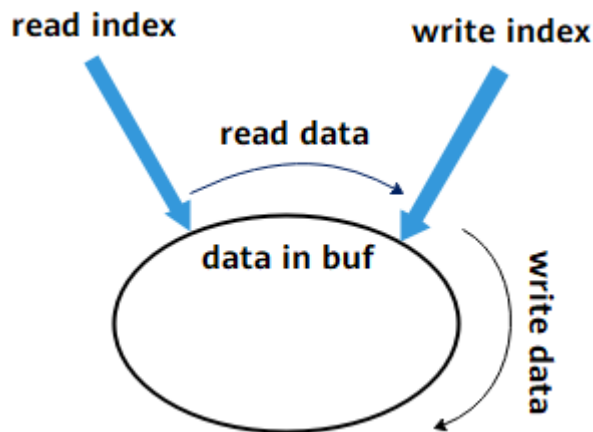
Each ringbuf can be in either of the following states:

- RBUF_UNINIT: The ringbuf is not initialized.
- RBUF_INITED: The ringbuf has been initialized.

Each ringbuf has two modes:

- RBUF_NORMAL: The normal mode of ringbuf.
- RBUF_OVERWRITE: The overwrite mode of ringbuf.

Operation Principle



The ringbuf control block maintains the start address of the buffer and the relative offset of the read/write pointer. The `startIdx` is the offset of the read pointer relative to the start address of the ringbuf, and `endIdx` is the relative offset of the write pointer. The data between `startIdx` and `endIdx` is buffered data. When ringbuf is just created, the offsets of the read and write pointers are both 0.

When data is written to the ringbuf, the ringbuf flag is checked first. When the flag is `RBUF_NORMAL`, the system is in normal mode. When the flag is `RBUF_OVERWRITE`, the system is in overwrite mode.

Normal mode: Before writing data to ringbuf, check whether ringbuf has available memory. Data is written only when there is available memory. As the value of `endIdx` increases, the available memory of ringbuf decreases. Otherwise, the written data is discarded. You can determine the amount of written data based on the return value of `LOS_RingbufWrite()`. If ringbuf is full, data cannot be written because old data will not be overwritten.

Overwrite mode: When data is written to the ringbuf, the data to be written to the ringbuf is checked first. If the length of the data to be written is greater than the size, the ringbuf is cleared and only the length of the size is written. If the length of the data to be written is less than the size and greater than the remaining memory, the earliest cached data is cleared and then the data is written. If the length of the data to be written is less than the remaining memory, the data is directly written. Finally, the actual written length is returned.

When data is read, the earliest cached data is read in first in first out (FIFO) mode, the value of `startIdx` increases, and the available memory of ringbuf increases. After the data is read, the read pointer moves backward. Therefore, the same data cannot be read again.

When the value of `endIdx` increases to the value of `startIdx`, the ring buffer is full. Therefore, data cannot be written to the ring buffer until the data is read.

3.14.5.2 Development Guidelines

Application Scenario

The ringbuf is used for data buffering and complies with the first in first out (FIFO) principle.

Functions

The ringbuf module of Huawei LiteOS provides the following functions. For details about the APIs, see the API reference.

Function Category	API	Description
Initializing/ Resetting the ring buffer	LOS_RingbufInit	Initializes the ring buffer. The buffer memory, size and flag are transferred through input parameters.
	LOS_RingbufReset	Resets the buffer status, resets the read/write offset to 0, and clears the data in the buffer.
Read/Write ring buffer	LOS_RingbufRead	Reads data of a specified size from the ring buffer to a specified address space, and returns the read data volume.
	LOS_RingbufPeek	Pre-reads data of a specified size from the ring buffer to a specified address space, and returns the read data volume.
	LOS_RingbufWrite	Writes data of a specified size to the ring buffer and returns the amount of data that is successfully written to the buffer.
Obtains the buffered data volume.	LOS_RingbufUsedSize	Obtains the cached data volume in the ring buffer.

Development Procedure

The typical development process using the ringbuf module is as follows:

1. Open the menu and choose **Kernel > Enable Ringbuf** to configure the ringbuf module.

Configuration Item	Description	Value Range	Default Value	Dependency
LOSCFG_KERNEL_RINGBUF	Whether to tailor ringbuf	YES/NO	NO	None.

2. Allocate a memory block for ringbuf.
3. Initialize ringbuf by using LOS_RingbufInit.
4. Write data to the ringbuf.
5. Read ringbuf at a proper time.

3.14.5.3 Precautions

- The API writing behavior of ringbuf is determined by the LOS_RingbufInit API during initialization.
- After the data in the buffer is read through the LOS_RingbufRead API, the data cannot be read again.
- If the LOS_RingbufPeek API only pre-reads the data in the buffer and does not consume the data, the data can still be read.
- When the buffer is full, no more data can be written under the normal mode. Check the return value of LOS_RingbufWrite to confirm the amount of data that is actually written.
- After ringbuf is used, you need to release the allocated memory.

3.14.5.4 Programming Example

Example Description

Create a ringbuf and two tasks. Task 1 periodically writes data to ringbuf. Task 2 reads data when the ringbuf data reaches a certain amount.

1. Lock task scheduling and create task 1 and task 2 by calling LOS_TaskCreate.
2. Allocate a memory block as the ring buffer.
3. Initialize a buffer by calling LOS_RingbufInit.
4. Unlock task scheduling.
5. Task 1 writes data to the ring buffer.
6. Task 2 reads data after determining that the data reaches the waterline.

Example Code

```
static Ringbuf g_Ringbuf = {0};
#define READ_TRIGGER_THRESHOLD 32
#define RINGBUF_SIZE 128

VOID WriteTask(VOID)
{
    UINT32 i = 0;
    UINT32 ret = 0;
    CHAR data[READ_TRIGGER_THRESHOLD * 2 + 1] = "LiteOS test data, send in WriteTask, receive & check in ReadTask";
    for (;;) {
        ret = LOS_RingbufWrite(&g_Ringbuf, data + i, READ_TRIGGER_THRESHOLD / 2);
        if (ret != READ_TRIGGER_THRESHOLD / 2) {
            PRINT_ERR("real write count is %u\n", ret);
        }
        i += ret;
        if (i >= strlen(data)) {
            break;
        } else {
            LOS_Msleep(1000); /* 1000: sleep 1000ms */
        }
    }
}

VOID ReadTask(VOID)
{
    UINT32 i = 0;
    UINT32 ret = 0;
    UINT32 count = 0;
```



```

CHAR data[READ_TRIGGER_THRESHOLD * 2 + 1] = {0};
for (;;) {
    count = LOS_RingbufUsedSize(&g_Ringbuf);
    if (count >= READ_TRIGGER_THRESHOLD) {
        ret = LOS_RingbufRead(&g_Ringbuf, data + i, count);
        i += ret;
        PRINTK("read data count %u \n", i);
        if (i == READ_TRIGGER_THRESHOLD * 2) {
            PRINTK("data receive done: %s\n", data);
            break;
        }
    } else {
        PRINTK("info: data count %u < %u\n", count, READ_TRIGGER_THRESHOLD);
    }
    LOS_Msleep(1000); /* 1000: sleep 1000ms */
}

UINT32 SampleTaskCreat(VOID)
{
    UINT32 ret = 0;
    UINT32 task1, task2;
    CHAR *buff = NULL;
    TSK_INIT_PARAM_S initParam;

    initParam.pfnTaskEntry = (TSK_ENTRY_FUNC)WriteTask;
    initParam.usTaskPrio = LOSCFG_BASE_CORE_TSK_DEFAULT_PRIO;
    initParam.uwStackSize = LOSCFG_BASE_CORE_TSK_DEFAULT_STACK_SIZE;
    initParam.pcName = "WriteTask";
#ifdef LOSCFG_KERNEL_SMP
    initParam.usCpuAffiMask = CPUID_TO_AFFI_MASK(ArchCurrCpuId());
#endif
    initParam.uwResved = LOS_TASK_STATUS_DETACHED;

    LOS_TaskLock();
    ret = LOS_TaskCreate(&task1, &initParam);
    if (ret != LOS_OK) {
        PRINT_ERR("create WriteTask failed, error: %x\n", ret);
        return ret;
    }

    initParam.pcName = "ReadTask";
    initParam.pfnTaskEntry = (TSK_ENTRY_FUNC)ReadTask;
    ret = LOS_TaskCreate(&task2, &initParam);
    if (ret != LOS_OK) {
        PRINT_ERR("create ReadTask failed, error: %x\n", ret);
        return ret;
    }

    buff = (CHAR *)LOS_MemAlloc(m_aucSysMem0, RINGBUF_SIZE);
    if (buff == NULL) {
        PRINT_ERR("malloc failed");
        return LOS_NOK;
    }
    ret = LOS_RingbufInit(&g_Ringbuf, buff, RINGBUF_SIZE, RBUF_NORMAL);
    if (ret != LOS_OK) {
        PRINT_ERR("ringbuf init failed");
        return LOS_NOK;
    }

    PRINTK("create the ringbuf success!\n");
    LOS_TaskUnlock();
    return ret;
}

```

Verification

```

create the ringbuf success!
info: data count 16 < 32

```

```
read data count 32
info: data count 16 < 32
read data count 64
data receive done: LiteOS test data, send in WriteTask, receive & check in ReadTask
```

Complete Code

sample_ringbuf.c

3.14.6 Universal Memory Access

3.14.6.1 Overview

Basic Concept

When a register or memory is located in a memory space, the register or memory is referred to as an I/O memory. Huawei LiteOS provides a group of common APIs to operate the I/O memory.

3.14.6.2 Development Guidelines

Application Scenario

The system provides the operation of reading and writing 1-, 2-, 4-, and 8-byte registers to facilitate the writing of driver code.

Function

Huawei LiteOS provides the following functions. For details about the APIs, see the API reference.

Function Category	API	Description
Reading and writing 64-bit data	WRITE64(addr, value)	Writes 64-bit data (value) to a 64-bit register (addr).
	READ64(addr, value)	Reads data from a 64-bit register (addr).
	WRITE64_MB(addr, value)	Writes 64-bit data (value) to a 64-bit register (addr). Before writing, the data synchronization barrier is used for synchronization.
	READ64_MB(addr, value)	Reads data from a 64-bit register (addr). Before reading, the data synchronization barrier is used for synchronization.

Function Category	API	Description
Reading and writing 32-bit data	WRITE32(addr, value)	Writes 32-bit data (value) to a 32-bit register (addr).
	READ32(addr, value)	Reads data from a 32-bit register (addr).
	WRITE32_MB(addr, value)	Writes 32-bit data (value) to a 32-bit register (addr). Before writing, the data synchronization barrier is used for synchronization.
	READ32_MB(addr, value)	Reads data from a 32-bit register (addr). Before reading, the data synchronization barrier is used for synchronization.
Reading and writing 16-bit data	WRITE16(addr, value)	Writes 16-bit data (value) to a 16-bit register (addr).
	READ16(addr, value)	Reads data from a 16-bit register (addr).
	WRITE16_MB(addr, value)	Writes 16-bit data (value) to a 16-bit register (addr). Before writing, the data synchronization barrier is used for synchronization.
	READ16_MB(addr, value)	Reads data from a 16-bit register (addr). Before reading, the data synchronization barrier is used for synchronization.
Reading and writing 8-bit data	WRITE8(addr, value)	Writes 8-bit data (value) to an 8-bit register (addr).
	READ8(addr, value)	Reads data from an 8-bit register (addr).
	WRITE8_MB(addr, value)	Writes 8-bit data (value) to an 8-bit register (addr). Before writing, the data synchronization barrier is used for synchronization.
	READ8_MB(addr, value)	Reads data from an 8-bit register (addr). Before reading, the data synchronization barrier is used for synchronization.

- If data is written to a 64-bit register in the 32-bit Arm architecture and accessed by using the **str** command, the data in the upper 32-bit register address and lower 32-bit register address cannot take effect at the same time. Therefore, the **strd** command is used instead.

3.14.6.3 Precautions

None.

4 Extended Kernels

- [4.1 Dynamic Loading](#)
- [4.2 Scatter Loading](#)
- [4.3 Low Power Consumption Framework](#)
- [4.4 Access Permission Management](#)
- [4.5 CPUP](#)
- [4.6 Trace](#)
- [4.7 LMS](#)
- [4.8 Perf](#)
- [4.9 C++ Support](#)
- [4.10 Dynamic Shared Memory](#)
- [4.11 Inter-core Communication](#)

4.1 Dynamic Loading

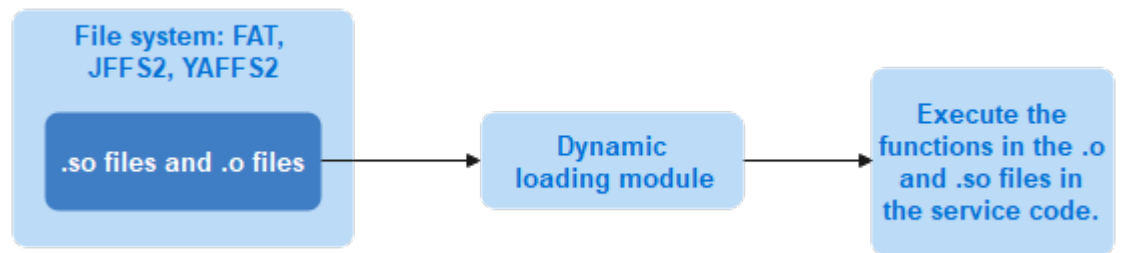
4.1.1 Overview

Basic Concepts

Dynamic loading is a program loading technology.

Static links link all module files of a program into an executable file, so that these files can be loaded into the memory as a whole. Dynamic loading enables developers to compile each module of a program into an independent .o file and .so file for dynamic loading into the memory, instead of linking all modules into an executable file.

Huawei LiteOS supports dynamic loading of OBJ and SO files, and the target files and binary image files are used together.



Related Concept

Symbol table

Symbol tables are arrays that record symbol names and the address information of the symbols in the memory. Symbol tables are loaded to the symbol management structure of the dynamic loading module when the dynamic linker is initialized. When a symbol needs to be relocated during module loading, the dynamic loading module obtains the symbol address by searching the symbol management structure.

Operation Mechanism

SO files may be stored in the storage medium in ZIP (compressed) or NOZIP (non-compressed) format. There are two loading policies:

- NOZIP policy: The information about common SO files can be obtained through lseek and read for multiple times. Calculate the size of the loadable segment based on the SegmentHeader information in the SO file, allocate the corresponding space, load the loadable segment to the memory, and complete dynamic loading. Because only a small part of the file is read each time, there is no memory waste.
- ZIP policy: When a ZIP file is read, all files must be read to the memory (mem1) at a time. However, the size of the loadable segment in the corresponding SO file is not known when the file is read for the first time. Therefore, the memory (mem2) needs to be allocated for the second time. Because mem2 already has all the information required for the dynamic loading, if mem1 and mem2 exist at the same time, there is a waste of memory and the memory peak value is too high. In this case, the ZIP file is read twice. In the first reading, the size of the required memory is calculated (the corresponding memory is released immediately after the calculation is complete). In the second reading, the dynamic loading based on the corresponding information is complete. Therefore, the ZIP policy avoids an excessively high memory peak value at the cost of instructions.

4.1.2 Development Guidelines

Application Scenario

The static link links the files of each module into a whole, and loads them into the memory at a time when the program is running. The static link facilitates the code loading. However, when the program is large and modules are frequently changed

and upgraded, the memory and disk space may be wasted, and it is difficult to update modules.

The dynamic loading technology can resolve the issues of the static link. When a program needs to execute code in an external module, the external module is dynamically loaded to a memory, and is unloaded when the module is not required. In this way, functions such as sharing of public code and smooth upgrade of the module can be implemented.

Functions

The dynamic loading module of Huawei LiteOS provides the following functions. For details about the APIs, see the API reference.

Function Category	API	Description
Destroying dynamic loading	LOS_LdDestroy	Destroys a dynamic loading module.
Dynamically loading module files	LOS_SoLoad	Dynamically loads an SO module (depending on the file system).
	LOS_MemLoad	Dynamically loads an SO module (independent of the file system and loaded from the specified memory space).
	LOS_ObjLoad	Dynamically loads an OBJ module (depending on the file system).
Searching for a symbol address	LOS_FindSymByName	Searches for a symbol address in a module or symbol table.
Unloading a module	LOS_ModuleUnload	Unloads a module.
Setting the search path, parameters, and memory pool for dynamic loading	LOS_PathAdd	Adds a module search path.
	LOS_DynParamReg	Sets the dynamic loading parameters of the SO module. To use this API, you need to enable the LOSCFG_DYNLOAD_DYN_FROM_FS macro. That is, the dynamic loading parameters can be set only when the SO module is loaded from the file system.
	LOS_DynMemPoolSet	Sets the memory pool address for dynamic loading.

NOTE

The LOS_DynMemPoolSet interface entry must be the memory pool address initialized by LOS_MemInit, which is managed by the Huawei LiteOS memory management algorithm. And ensure that the memory pool does not coincide with the system memory pool. This interface needs to be used before loading .so or .o files.

Development Procedure

The implementation of dynamic loading involves the following steps:

1. [Preparing the Compilation Environment](#)
2. [Compiling the Module File to Be Loaded](#)
3. [Compiling the Service Code of the Dynamic Loading Module](#)
4. [Compiling the System Image](#)
5. [Preparing the System Environment](#)

Preparing the Compilation Environment

Step 1 Add compilation options of the .o module and the .so module.

- Add **-nostdlib -fno-PIC** to the compilation options of .o modules.
- Add **-nostdlib -fPIC -shared** to the compilation options of .so modules.

NOTE

Determine whether to use the following compilation option based on your needs.

-z max-page-size=value

This compilation option is used to set loadable program segments to be aligned to **value**. This compilation option helps reduce the blank areas required for the alignment of the virtual addresses of adjacent loadable segments. If this compilation option is not used, the default alignment is 0x10000.

NOTICE

- To implement the dynamic loading in an IP camera (IPC), the start addresses of all nodal regions of the LD_SHT_PROGBITS and LD_SHT_NOBITS types in the module files must be on the boundary of four bytes. Otherwise, the modules will not be loaded.
- To generate a dynamic library, do not use **-ldl**, **-lpthread**, and **-lc** to specify a standard library in the compiler. However, the **-nostdlib** compiler option must be used.

The following is an example of adding compilation options of .o and .so modules:

```
RM = -rm -rf

CC = arm-himix100-linux-gcc

SRCS = $(wildcard *.c)
OBS = $(patsubst %.c,%.o,$(SRCS))
SOS = $(patsubst %.c,%.so,$(SRCS))

all: $(SOS)

$(OBS): %.o : %.c
    @$(CC) -nostdlib -c $< -fno-PIC -o $@

$(SOS): %.so : %.c
    @$(CC) -nostdlib $< -fPIC -shared -o $@

clean:
    @$(RM) $(SOS) $(OBS)
```



```
.PHONY: all clean
```

Step 2 Compile the system makefile.

The makefile used to compile the system image file must include the **config.mk** file under the root directory. The **LITEOS_CFLAGS** or the **LITEOS_CXXFLAGS** compilation option in the makefile should be used. The code is similar to the following:

```
LITEOSTOPDIR ?= ../..

SAMPLE_OUT = .

include $(LITEOSTOPDIR)/config.mk
RM = -rm -rf

LITEOS_LIBDEPS := --start-group $(LITEOS_LIBDEP) --end-group

SRCS = $(wildcard sample.c)

OBJS = $(patsubst %.c,$(SAMPLE_OUT)/%.o,$(SRCS))

all: $(OBJS)

clean:
    @$ (RM) *.o sample *.bin *.map *.asm

$(OBJS): $(SAMPLE_OUT)/%.o : %.c
    $(CC) $(LITEOS_CFLAGS) -c $< -o $@

$(LD) $(LITEOS_LDFLAGS) -uinit_jffspar_param --gc-sections -Map=$(SAMPLE_OUT)/sample.map -o $
(SAMPLE_OUT)/sample ./$@ $(LITEOS_LIBDEPS) $(LITEOS_TABLES_LDFLAGS) $(LITEOS_DYNLDFLAGS)
$(OBJCOPY) -O binary $(SAMPLE_OUT)/sample $(SAMPLE_OUT)/sample.bin
$(OBJDUMP) -d $(SAMPLE_OUT)/sample >$(SAMPLE_OUT)/sample.asm
```

----End

Compiling the Module File to Be Loaded

Perform the following steps to compile the system image: The makefile is used as an example.

Step 1 Compile .o modules and .so modules and copy the .o and .so files required for system running to a directory.

NOTE

1. If **a.so** needs to call functions in **b.so**, or **a.so** uses data in **b.so**, **a.so** depends on **b.so**.
2. When **a.so** depends on **b.so**, and **b.so** needs to be automatically loaded when **a.so** is being loaded, **b.so** should be a compilation parameter during the compilation of **a.so**. If **b.so** is not a compilation parameter during the loading of **a.so**, load **b.so** before loading **a.so**.

Step 2 Access the **Huawei_LiteOS/build/scripts/dynload_tools** directory and run the **sym.sh** script as follows:

```
$ ./sym.sh /home/wmin/customer/out out/dynload_sym
```

 NOTE

- The input parameter **/home/wmin/customer/out** of the **sym.sh** is the absolute path of the .o and .so files. **out/dynload_sym** is the absolute path of the directory where **los_dynload_gsymbol.c** is generated. You are advised to set it to the directory where the project is generated after compilation.
- If the required .o and .so files are updated, run the script again. This script extracts all system symbols of the .o and .so files to be loaded. The compiler will calculate the addresses of those symbols when compiling the system image.
- Run this command under the **Huawei_LiteOS/build/scripts/dynload_tools** directory.
- After running the **sym.sh** script, ensure the **los_dynload_gsymbol.c** file has been generated in the **out/dynload_sym** project directory, the file has been compiled, and the generated target file has been added to the link script. Otherwise, symbols will not be readdressed in the process of loading .o and .so files.

----End

Compiling the Service Code of the Dynamic Loading Module

Step 1 Open the menu and choose **Kernel > Enable Extend Kernel > Enable Dynamic Load Feature** to configure the dynamic loading module.

Configuration Item	Description	Value Range	Default Value	Dependency
LOSCFG_KERNEL_DYNLOAD	Determines whether to enable the dynamic loading module.	YES/NO	YES	LOSCFG_KERNEL_EXTKERNEL
LOSCFG_KERNEL_DYNLOAD_DYN	Determines whether to enable the loading of SO files.	YES/NO	YES	LOSCFG_KERNEL_DYNLOAD
LOSCFG_DYNLOAD_DYN_FROM_FS	Determines whether to enable the SO files to be loaded from the file system.	YES/NO	YES	LOSCFG_KERNEL_DYNLOAD_DYN
LOSCFG_DYNLOAD_DYN_FROM_MEM	Determines whether to enable the SO files to be loaded from the specified memory.	YES/NO	YES	LOSCFG_KERNEL_DYNLOAD_DYN
LOSCFG_KERNEL_DYNLOAD_REL	Determines whether to enable the loading of OBJ files.	YES/NO	YES	LOSCFG_KERNEL_DYNLOAD
LOSCFG_DYNLOAD_REL_FROM_FS	Determines whether to enable the OBJ files to be loaded from the file system.	YES/NO	YES	LOSCFG_KERNEL_DYNLOAD_REL

Configuration Item	Description	Value Range	Default Value	Dependency
LOSCFG_KERNEL_DYN_HEAPSIZE	Configures the heap size for the dynamic loading module (unit: MB).	[0, OS_SYS_MEMORY_SIZE)	2	LOSCFG_KERNEL_NX && LOSCFG_KERNEL_DYNLOAD, and the Cortex-A chip

NOTICE

If both LOSCFG_KERNEL_NX (It depends on the Cortex-A chip, its data segment is not executable, and its location in **menuconfig** is **Kernel > Enable Data Sec NX Feature**) and the dynamic loading module are enabled, only SO files can be dynamically loaded. Other types of loading may be abnormal. In addition, you need to configure the heap size for the dynamic loading module (located at the end of the dynamic memory pool of the system).

Step 2 (Optional) Set the address of the memory pool for the dynamic loading module.

Call LOS_DynMemPoolSet to set the memory pool used by the dynamic loading module. If this parameter is not set, the system memory pool is used by default.

Step 3 Specify the loading policy for SO files.

In various application scenarios, SO files may be stored in storage media in ZIP or NOZIP format. Operations of reading and writing into ZIP files differ from those for NOZIP files. Therefore, the loading policy of SO files needs to be specified before the initialization of the dynamic loading module.

```
DYNLOAD_PARAM_S dynloadParam = {ZIP}; // Sets the ZIP or NOZIP Policy.
LOS_DynParamReg(&dynloadParam);      // Sets the specific loading policy.
```

NOTE

If the SO files are stored in ZIP format, use the ZIP policy. For common SO files, both policies can be used, and the NOZIP policy is recommended in this scenario. If no policy is specified, the NOZIP policy is used by default.

Step 4 (Optional) Specify whether to use a relative path.

If you want to use the relative path when using the dynamically loading module, call LOS_PathAdd to add the absolute paths of the .so and .o files.

```
ret = LOS_PathAdd("/yaffs/bin/dynload");
if (ret != LOS_OK) {
    printf("add relative path failed");
    return 1;
}
```

After adding the paths, enter the file name when calling LOS_SoLoad, LOS_ObjLoad, and LOS_MemLoad. During dynamic loading, the system searches for the specified file in the paths.

 NOTE

- The relative path can be used when the dynamic loading module APIs are called only after the paths are added through the LOS_PathAdd API.
- You can add multiple relative paths by calling the LOS_PathAdd API for multiple times.
- If the added modules have the same file name in the added paths, the system searches all the paths according to the sequence in which the modules are added and loads only the first file that is found.

Step 5 Load a module.

The dynamic loading module can load .o and .so modules.

- Call the LOS_ObjLoad API to dynamically load OBJ files, (OBJ modules can be loaded only from the file system.)

```
if ((handle = LOS_ObjLoad("/yaffs/bin/dynload/foo.o")) == NULL) {  
    printf("load module ERROR!!!!!!\n");  
    return 1;  
}
```

- Call the LOS_SoLoad API to dynamically load SO files.

```
if ((handle = LOS_SoLoad("/yaffs/bin/dynload/foo.so")) == NULL) {  
    printf("load module ERROR!!!!!!\n");  
    return 1;  
}
```

 NOTE

For the dynamic loading of SO files:

- When module A depends on module B, and the dependency is specified with **B.so** being a compilation parameter during the compilation of **A.so**, module B will be automatically loaded during the loading of module A. If the dependency is not specified, ensure that module B has been successfully loaded during the loading of module A.
- For the platforms that do not support the file system, the SO files can be dynamically loaded from the specified memory space by calling the LOS_MemLoad API.

Step 6 Obtain the symbol address in the user module.

- Search for a symbol in a specified user module.

When searching for a symbol in a specified user module, call the LOS_FindSymByName API and set the first argument value to the handle of the module in which the symbol is searched for.

```
if ((magic = LOS_FindSymByName(handle, "os_symbol_table")) == NULL) {  
    printf("symbol not found\n");  
    return 1;  
}
```

- Search for a symbol in the global symbol table.

When searching for a symbol in the global symbol table (OS modules including your modules and other user modules), call the LOS_FindSymByName API and set the first argument value to **NULL**.

```
if ((funTestCase0 = LOS_FindSymByName(NULL, "printf")) == NULL) {  
    printf("symbol not found\n");  
    return 1;  
}
```

Step 7 Use the obtained symbol address.

LOS_FindSymByName returns a symbol address (VOID *pointer). You can change the type of the symbol that is located at this address for different use. The following examples describe the use of two types of symbols:

- **Integer symbol (data symbol)**

test.c needs to be loaded, and the global variable **UINT32 g_test = 0** is available.

Run the following code to obtain the address of **g_test**:

```
const char *g_pscOsOSSymtblFilePath = "/yaffs/bin/dynload/test.so";
UINT32 *g_testPtr = NULL;
INT8 *ptr = (INT8 *)NULL;
if ((pOSSymtblHandler = LOS_SoLoad(g_pscOsOSSymtblFilePath)) == NULL) {
    return LOS_NOK;
}
if ((ptr = LOS_FindSymByName(pOSSymtblHandler, "g_test")) == NULL) {
    printf("g_uwTest not found\n");
    return LOS_NOK;
}
g_testPtr = (UINT32 *)ptr; /* Forcibly change the pointer type to a real pointer type. */
```

- **Function symbol**

The test_0 function expecting no arguments and the test_2 function expecting two arguments are defined in foo.c. foo.o can be generated by compiling foo.c.

```
foo.c:
int test_0(void)
{
    return 0;
}
int test_2(int i, int j)
{
    return 0;
}
```

The following code shows how to obtain and call the functions in the **foo.o** module in **demo.c**.

```
demo.c:
typedef unsigned int (* TST_CASE_FUNC)(); /* Declaration of the type of the pointer to a function
that expects no parameters */
typedef unsigned int (* TST_CASE_FUNC1)(UINT32); /* Declaration of the type of the pointer to a
function that expects one parameter */
typedef unsigned int (* TST_CASE_FUNC2)(UINT32, UINT32); /* Declaration of the type of the pointer
to a function that expects two parameters */

TST_CASE_FUNC funTestCase0 = NULL; /* Definition of a pointer to a function */
TST_CASE_FUNC2 funTestCase2 = NULL;
int ret;

handle = LOS_ObjLoad("/yaffs/bin/dynload/foo.o");
funTestCase0 = LOS_FindSymByName(handle, "test_0");
if (funTestCase0 == NULL) {
    printf("can not find the function name\n");
    return 1;
}
ret = funTestCase0();

funTestCase2 = LOS_FindSymByName(NULL, "test_2");
if (funTestCase2 == NULL){
    printf("can not find the function name\n");
    return 1;
}
ret = funTestCase2(42, 57);
```

Step 8 Unload the module.

Call the `LOS_ModuleUnload` API to unload a module and use the handle of the module to be unloaded as the input parameter. Call the `LOS_ModuleUnload` API to unload an .obj file handle or an SO file handle that has been loaded.

```
ret = LOS_ModuleUnload(handle);
if (ret != LOS_OK) {
    printf("unload module failed");
    return 1;
}
```

Step 9 Destroy the dynamic loading module.

Call the `LOS_LdDestroy` API to destroy the dynamic loading module when it is no longer in need.

NOTICE

- All modules that have been loaded will be automatically unloaded when the dynamic loading module is destroyed.
- Ensure that the modules are no longer needed before destroying the dynamic loading module.

----End

Compiling the System Image

Run the **make** command in the root directory of the Huawei LiteOS source code to compile the system image.

After the compilation is complete, the **vs_server.bin** file is generated in the **out/platform name** directory.

NOTE

If the OBJ and SO files to be loaded contain undefined external symbols (neither defined in the OBJ and SO files nor a valid global system symbol), an error message is displayed when the system image file is compiled. In this case, you need to check the error information and ensure that the system image file is compiled correctly.

Preparing the System Environment

SO files (or OBJ files) and the system image file are used together.

- If the module is loaded from the file system, the module files must be stored in the file system, such as the jffs2, yaffs, and fat file systems.
- If you choose to load the module file from the specified memory space, skip step 2 and burn the module file to the specified memory space.

Perform the following steps:

Step 1 Burn the system image file to the flash memory of the development board.

Step 2 Save the SO file (or OBJ file) to the flash memory of the development board. There are two scenarios:

- If the module file is stored on a hot-swappable SD card device, insert the SD card into the development board. To update the SO file (or OBJ file), insert the SD card to the computer.
- If the module file is stored on a storage device that does not support hot swap, use either of the following methods to update the file:
 - a. Compile the module file into the file system image, and then burn the file system image to the flash memory of the development board.
 - b. After the Huawei LiteOS system is started, run the tftp command to download the SO file (or OBJ file). The following is an example:

```
tftp -g -l /yaffs/bin/dynload/foo.so -r foo.so 10.67.211.235
```

Step 3 Start the system for verification.

----End

Shell Debugging

A series of commands related to dynamic loading are encapsulated in the shell to facilitate debugging. For details about the shell commands, see [Dynamic Loading Commands](#).

4.1.3 Precautions

Compilation option

- Add **-nostdlib -fno-PIC** to the compilation options of .o modules.
- Add **-nostdlib -fPIC -shared** to the compilation options of .so modules.

Security

- For the loaded module file, the provider needs to ensure that its source is reliable and secure. Do not load files from high-risk media such as SD cards or USB flash drives. If such file loading is necessary, ensure that the files are reliable. Huawei is not liable for any loss or problems caused thereby.
- Problematically loaded files will result in problems including but not limited to device damage, data leakage, and data tampering, for which Huawei assumes no responsibility.

Others

- Before the compilation of the system image, the required .o and .so files must be available to ensure that the external symbols used by the files have been integrated into the system image when the image file is being compiled.
- While loading a library file, once Huawei LiteOS detects that the external symbols referenced by this file are repeatedly defined in multiple modules that have no dependency relationships with this file, the relocation of the symbol references will be rejected. Then the library file fails to be loaded. Ensure that no symbols (variables or functions) that are repeatedly defined exist in the library file to be loaded.

4.1.4 Programming Example

Example Description

The test_0 function expecting no parameters and the test_2 function expecting two parameters are defined in **sample_foo.c**. **foo.o** can be generated by compiling **sample_foo.c**. The following code shows how to obtain and call the functions in the **foo.o** module in **sample_Dynamic_loading.c**.

Example Code

Prerequisite: The dynamic loading module has been configured in the **menuconfig** menu.

The code is as follows:

```
int test_0(void)
{
    printf("test_0\n");
    return 0;
}
int test_2(int i, int j)
{
    printf("test_2: %d %d\n", i, j);
    return 0;
}
```

The code of **sample_Dynamic_loading.c** is as follows:

```
typedefint (* TST_CASE_FUNC)(VOID);          /* Declaration of the type of the pointer to a function
that expects no parameters */
typedefint (* TST_CASE_FUNC2)(UINT32, UINT32); /* Declaration of the type of the pointer to a function
that expects two parameters */
TST_CASE_FUNC funTestCase0 = NULL;           /* Definition of a pointer to a function */
TST_CASE_FUNC2 funTestCase2 = NULL;
int ret;
void *handle = NULL;
handle = LOS_ObjLoad("/jffs0/bin/dynload/dynload_test.o");
funTestCase0 = LOS_FindSymByName(handle, "test_0");
if (funTestCase0 == NULL) {
    printf("can not find the function name\n");
    return;
}
ret = funTestCase0();                        /* Call the function pointer. */
funTestCase2 = LOS_FindSymByName(NULL, "test_2");
if (funTestCase2 == NULL){
    printf("can not find the function name\n");
    return;
}
ret = funTestCase2(42, 57); /* Call the function pointer.*/
ret = LOS_ModuleUnload(handle);
if (ret != LOS_OK) {
    printf("unload module failed");
    return;
}
LOS_LdDestroy();
```

Verification

The verification result is as follows:

```
Huawei LiteOS#
test_0
test_2: 42 57
```


Complete Code

```
sample_foo.c  
sample_Dynamic_loading.rar
```

4.2 Scatter Loading

4.2.1 Overview

Basic Concepts

Scatter loading is a technology that achieves fast boot of specified code. It shortens the time between the boot of an OS and the execution of specified code by preferentially loading specified code into memory. In this way, scatter loading can be used to fast boot key services.

An embedded OS loads image files on the flash memory into memory through U-Boot. Image files are probably large and the speed of flash reading is limited. Therefore, the requirements on the boot speed of time-sensitive services probably cannot be met if all images are executed after they are loaded.

Scatter loading enables the fast boot of key services by preferentially loading and executing some images containing time-sensitive services.

Operation Mechanism

The scatter loading module of Huawei LiteOS loads and executes some time-sensitive key services in advance. Specifically, it lays out data and code segments related to these services to the front end of the image file. The image loading process is divided into two phases:

- In the first phase, the frontend images that contain time-sensitive services are loaded to the memory and executed by using U-Boot.
- In the second phase, the remaining part of the image is loaded to the memory for execution. The specific implementation is as follows: In the code in the first phase, the scatter loading API is called to load the remaining part of the image and run the image.

In the first phase, the speed of loading some images is faster than that of loading the complete image. Therefore, the time from system startup to running of time-sensitive services is shortened.

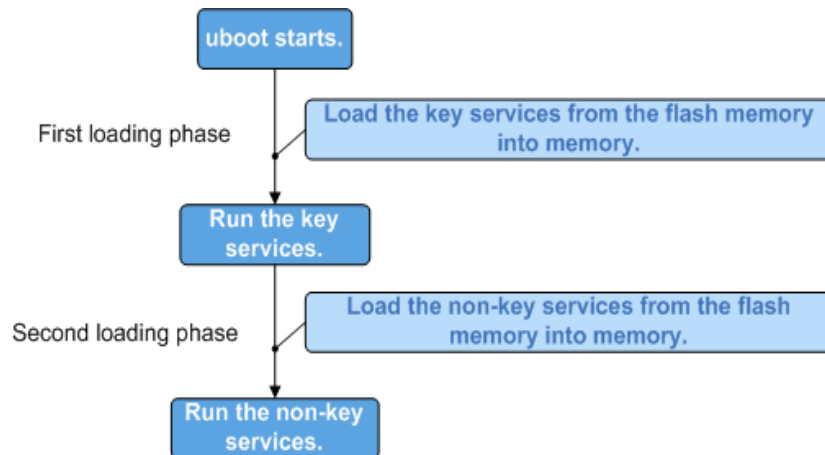
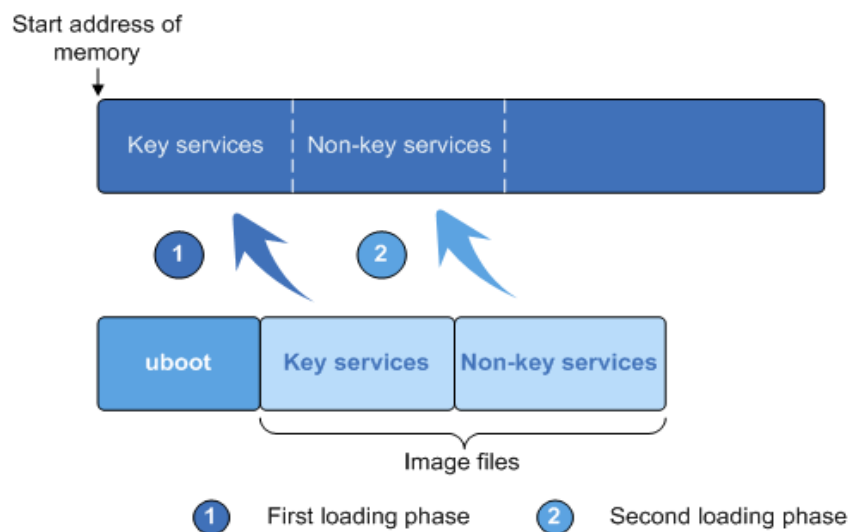
Figure 4-1 Scatter loading process

Figure 4-2 shows the internal principle of distributed loading. For details about the operation sequence, see **Figure 4-1**.

Figure 4-2 Internal conceptual diagram of scatter loading

4.2.2 Development Guidelines

Application Scenario

The scatter loading is applied in the scenario where fast boot of time-sensitive services is needed.

In the embedded system, some services may have high requirements on the startup time. For example, a project has high requirements on the time from startup to recording preview. The scatter loading technology can be used to implement quick startup of the recording preview service. The time from startup to recording is within 1 second, better than the average 3 seconds to 4.5 seconds of Linux.

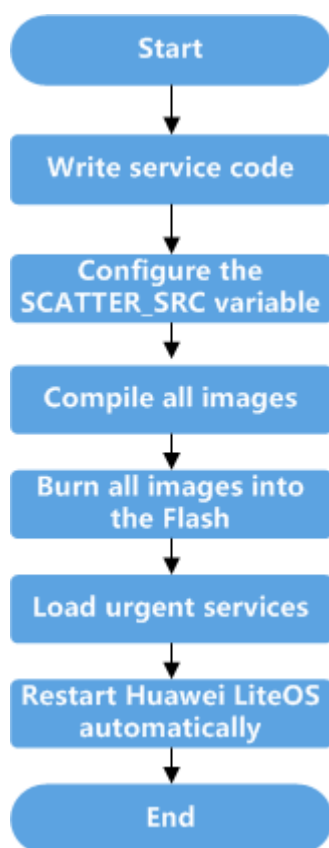
Functions

The scatter loading module of Huawei LiteOS provides the following functions. For details about the APIs, see the API reference.

Function Category	API	Description
Scatter loading API	LOS_ScatterLoad	Loads non-urgent services from the flash memory to the memory. This API is called at the end of the code in the phase of the scatter loading. The readAlignSize parameter indicates the block alignment size when the flash memory reads data and needs to be set based on the actual read/write features of the flash memory.

Development Procedure

The following figure shows the operation process of scatter loading:



Take the Hi3516A platform as an example. To use scatter loading, perform the following steps by using makefile as the compilation tool:

Step 1 Compile the service code for scatter loading and call LOS_ScatterLoad.

The entry function of the service code is `app_init`, which is stored in the **targets/\$ (LITEOS_PLATFORM)/os_adapt/os_adapt.c** file in the root directory of the

Huawei LiteOS source code. Call the `LOS_ScatterLoad` function after urgent service code for scatter loading and enable the **MAKE_SCATTER_IMAGE** macro to control the compilation of non-urgent service code. The example code is as follows:

```
void app_init() {
    proc_fs_init();
    hi_uartdev_init();
    system_console_init("/dev/uartdev-0");
    LOS_CppSystemInit((unsigned long)&__init_array_start, (unsigned long)&__init_array_end,
    BEFORE_SCATTER);
    LOS_ScatterLoad(0x100000, flash_read, NAND_READ_ALIGN_SIZE);

    /* The following services are non-urgent services. */
    #ifndef MAKE_SCATTER_IMAGE
        LOS_CppSystemInit((unsigned long)&__init_array_start, (unsigned long)&__init_array_end,
        AFTER_SCATTER);

        extern unsigned int osShellInit(void);
        osShellInit();
        rdk_fs_init();
        SDK_init();

        hi_product_driver_init();
        char *apszArgv[3]={"vs_server", "./higv.bin", "-i"};
        vs_server(3, apszArgv);
    #endif
}
```

Step 2 Configure the **SCATTER_SRC** variable.

Run the following command to set the **SCATTER_SRC** variable in **build/mk/wow_scatter.mk** under the root directory to the source file of the services that call the scatter loading function `LOS_ScatterLoad`. **LITEOSTOPDIR** indicates the root directory of Huawei LiteOS code.

```
SCATTER_SRC := $(LITEOSTOPDIR)/targets/$(LITEOS_PLATFORM)/os_adapt/os_adapt.c
```

Step 3 Open the menu and choose **Kernel > Enable Extend Kernel > Enable Scatter Feature** to enable the scatter loading module whose macro is **LOSCFG_KERNEL_SCATTER**.

Currently, the scatter loading module of Huawei LiteOS depends on the flash memory. You need to enable the flash memory in **Driver > Enable MTD** first. The NAND flash macro is **LOSCFG_DRIVERS_MTD_NAND**, and the SPI NOR flash macro is **LOSCFG_DRIVERS_MTD_SPI_NOR**.

Step 4 Run the **make** command to compile the system image.

- Run the following command in the root directory to compile all service code.
Huawei_LiteOS\$ make

After compilation, information about the size of urgent service images will be returned, which is similar to the following.

```
#####
#####Calculate the size of scatter#####
the size is 0x4E0000
#####end#####
```

- View the image segment allocation. If there are scatter loading segments in the images, the scatter loading is successfully started. In the directory where system images are generated, for example, **out/hi3516a** of the Hi3516A platform, run the **readelf -S vs_server** command to open the system image file **vs_server**.

[15]	.fast_rodta	PROGBITS	80159000	088868	0a6adc
[16]	.fast_text	PROGBITS	80200000	12f868	27ece0
[17]	.fast_data	PROGBITS	8047f000	3ae868	067b78
[18]	.got	PROGBITS	804e6b78	4163e0	000504
[19]	.text	PROGBITS	804e8000	417868	1b1b90
[20]	.rodata	PROGBITS	8069a000	5c9868	09f4d8
[21]	.data	PROGBITS	8073a000	669868	01cd7c
[22]	.bss	NOBITS	80757000	6865e4	6d74e0

Information similar to the following will be displayed, including segment name, start address, and offset. In the following figure, **.fast_rodta**, **.fast_text**, and **.fast_data** indicate the read-only segment, code segment, and data segment of scatter loading respectively.

- View the .text segment in the link script **Huawei_LiteOS/tools/scripts/ld/scatter.ld** of scatter loading. `scatter.o(*.text*)` is added, as shown in the following figure, indicating that symbols related to the fast booted code of scatter loading are placed in the same area.

```
.fast_text ALIGN (0x1000): { __fast_text_start = ABSOLUTE(.); . = .;
scatter.o(*.text*);
}

. = (ABSOLUTE (.) + (0x1000 - 1)) & ~ (0x1000 - 1);
__fast_text_end = ABSOLUTE (.);
```

Step 5 Burn all images to the flash memory.

Enter the serial port tool UI, and run the following command to burn all images to the 0x100000 address bit of the flash memory:

```
tftp 0x82000000 vs_server.bin;nand erase 0x100000 0x700000;nand write 0x82000000 0x100000 0x700000;
```

In the preceding command, **vs_server.bin** is the name of the system image file, which is burnt to the high address bit 0x82000000 in the memory. Then burn the image to the flash memory. The start address is 0x100000, and the length of the image to be burnt is 0x700000. That is, the size of the image to be burnt cannot exceed 7 MB. Adjust the length based on the actual image size.

NOTE

You can select the image burning tool and method as required.

Step 6 Load the urgent service.

Run the following command to read the image that contains the urgent service from the 0x100000 address of the flash memory and load the image to the 0x80008000 memory address: The image size is displayed on the CLI after the **make** command is executed in [Step 4](#). In this example, the image size is 0x4E0000.

```
nand read 0x80008000 0x100000 0x4E0000;
```

Run the following command to start the system from the memory address 0x80008000:

```
go 0x80008000;
```

Step 7 The system restarts automatically.

The system automatically restarts and loads the image at 0x80008000.

----End

4.2.3 Precautions

- The OS will be abnormal if the data copied in the first phase is not sufficient or the offset addresses are not aligned based on different storage media. Therefore, the size of images to be loaded by U-Boot needs to be the size returned when the compilation ends, as shown in [Step 4](#).
- The scatter loading function depends on the storage medium. Currently, Huawei LiteOS and configuration depend on the flash memory. If the flash memory is disabled, the scatter loading function is disabled. If you need to enable scatter loading for other storage medium, refer to current flash configuration for adaptation.

4.3 Low Power Consumption Framework

4.3.1 Overview

The low-power framework is a unified management framework for low-power modules. It supports various low-power capabilities, such as RunStop, system hibernation, and tickless.

The RunStop and tickless functions are enabled by default in the low power consumption framework. In addition, the low-power feature is closely related to specific services and hardware. To better adapt to hardware and extend functions, low-power framework opens related functional APIs to facilitate adaptation. In addition, you can customize specific policies in the default low-power framework.

Table 4-1 APIs

API	Description
LOS_PowerMgrInit	Initializes the low-power framework. Input parameters are the framework parameter pointers, which can be used to customize the low-power strategy and hibernation actions. The input parameter cannot be empty. Otherwise, the registration fails.
LOS_LowpowerHookReg	Registers the hibernation entry function of the low-power framework. The hibernation function is called when the system is idle.

4.3.2 RunStop

4.3.2.1 Overview

Basic Concepts

RunStop is a mechanism that saves onsite system images so that the system can be recovered quickly from these images when necessary.

When the system is running, you can call the RunStop API to save the system status snapshot (CPU and memory) to the flash memory. After the system is restarted, you can use the snapshot to restore the system so that the system continues to run from the state saved in the snapshot.

Operation Mechanism

After the service module is initialized and runs stably, the RunStop API is called to save the system snapshot. The snapshot contains all CPU and memory status required by the service module. After the system runs for a period of time, when the service module is idle, the primary core is automatically powered off and enters the low power mode. When a service needs to be processed, the MCU sends a specific data packet to power on the primary core. After the hardware status is restored, the system restores from the flash snapshot. The service module does not need to be initialized after the system is powered on again, implementing quick restoration of the service module.

4.3.2.2 Developer Guide

Application Scenario

If the system is expected to enter low-power mode after running for a period of time, the system can quickly restore to the state before the low-power mode when a service arrives, so as to quickly restore a service processing capability. In this application scenario, the RunStop function can be implemented.

In a project, the RunStop feature is used to implement the RunStop of the chip in the following way. After the system services run stably, the snapshot of the current system status is saved. When the system enters the idle state, the primary core is automatically powered off and enters the low-power mode. When a service message arrives, the MMU powers on the primary core. The system quickly starts from the snapshot and resumes service execution. Take the Wi-Fi chip as an example. The RunStop feature allows the primary core to continuously connect to the Wi-Fi network when the primary core is in low-power mode and to quickly restore the Wi-Fi state.

Functions

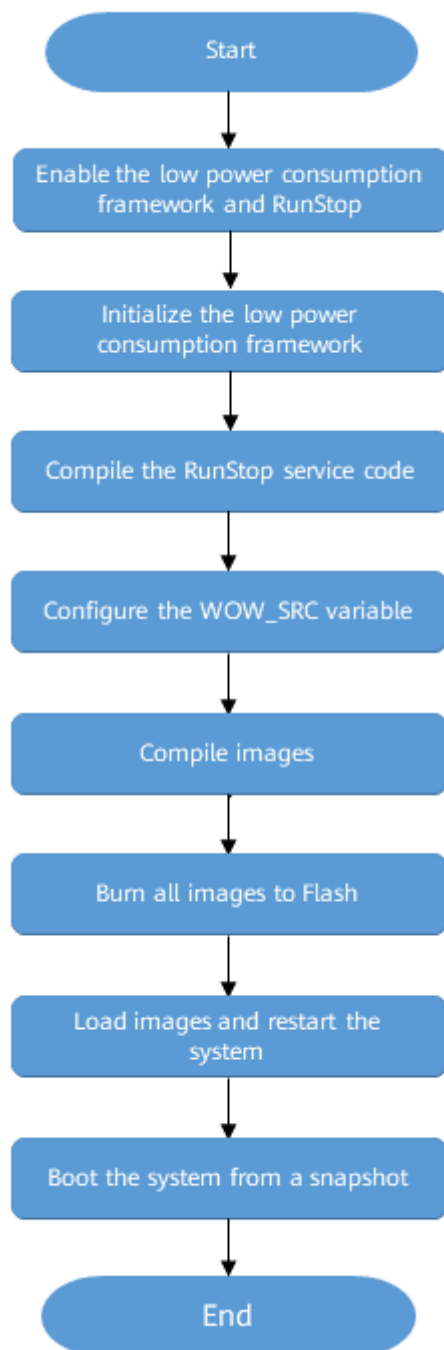
The RunStop module provides the following functions. For details about the APIs, see the API reference.

Table 4-2 Functions

API	Description
LOS_MakeImage	Saves the system status snapshot to the specified medium.

Development Procedure

Figure 4-3 RunStop process



Step 1 Enable the low-power framework and RunStop.

Open the menu and choose **Kernel > Enable Extend Kernel > Enable Low Power Management Framework**.

Configuration Item	Description	Value Range	Default Value	Dependency
LOSCFG_KERNEL_LOWPOWER	Enables/Disables the low-power framework tailoring.	YES/NO	YES	LOSCFG_KERNEL_EXTKERNEL
LOSCFG_KERNEL_RUNSTOP	Enables/Disables RunStop.	YES/NO	NO	The implementation of the RunStop feature depends on the flash, U-Boot, and bestfit memory management algorithms. LOSCFG_DRIVERS_MTD_NAND specifies the macro of NAND flash, and LOSCFG_DRIVERS_MTD_SPI_NOR specifies the marco of spi_nor flash.

Step 2 Initialize the low-power framework.

In the entry function main, call `LOS_PowerMgrInit(NULL)` before `OsStart` to initialize the low-power framework to the default system framework. The sample code is as follows:

```
#ifdef LOSCFG_KERNEL_LOWPOWER
#include "los_lowpower.h"
const PowerMgrRunOps runOps = {
    .getSleepMode = NULL,
};

UINT32 OsPreSystemConfig(VOID)
{
    PowerMgrParameter param;
    param.runOps = runOps;
    return LOS_PowerMgrInit(&param);
}

#endif
LITE_OS_SEC_TEXT_INIT int main(void)
{
    .....
#ifdef LOSCFG_KERNEL_LOWPOWER
    ret = OsPreSystemConfig();
    if (ret != LOS_OK) {
        return LOS_NOK;
    }
#endif
    OsStart();
    .....
}
```

Step 3 Call `LOS_MakeImage` to write the snapshot service code.

The service code entry function is `app_init`, which is stored in the **targets/board/\$ (LITEOS_PLATFORM)/os_adapt/os_adapt.c** file in the root directory of the source code.

Call `LOS_MakeImage` after the snapshot service code to save the status snapshot of the system to a specified medium, and use the `MAKE_WOW_IMAGE` macro to control the compilation of non-snapshot service code to compile the snapshot image and all other images. The sample code is as follows:

```
void wakeup_callback(void)
{
#ifdef LOSCFG_KERNEL_CPUP
    LOS_CpupReset();
#endif
    (VOID)LOS_HwiEnable(83);
}
#define NOR_ERASE_ALIGN_SIZE    (64 * 1024)
#define NOR_READ_ALIGN_SIZE    1
#define NOR_WRITE_ALIGN_SIZE    1
int flash_read(void *memaddr, size_t start, size_t size)
{
    return spiflash_read(memaddr, start, size);
}
int flash_write(void *memaddr, size_t start, size_t size)
{
    spiflash_erase(start, size);
    return spiflash_write(memaddr, start, size);
}
void app_init(void)
{
#ifdef LOSCFG_KERNEL_RUNSTOP
    RUNSTOP_PARAM_S stRunstopParam;    // Define a LOS_MakeImage parameter variable
#endif
#ifdef LOSCFG_DRIVERS_MTD_SPI_NOR
    (void)spinor_init();
    mtd = get_mtd("spinor");
#endif
#if defined(LOSCFG_KERNEL_RUNSTOP) && defined(LOSCFG_DRIVERS_MTD_SPI_NOR)
    memset(&stRunstopParam, 0, sizeof(RUNSTOP_PARAM_S));
    /* LOS_MakeImage parameter configuration */
    stRunstopParam.pfFlashReadFunc    = flash_read;        // Specify the media read function.
    stRunstopParam.pfFlashWriteFunc   = flash_write;       // Specify the media read function.
    stRunstopParam.pfIdleWakeupCallback = idle_wakeup_callback; // Specify the callback function to be
    executed in the idle task after the snapshot is restored.
    stRunstopParam.pfImageDoneCallback = NULL;             // Specify the callback function after the
    snapshot image is generated. The value is null here.
    stRunstopParam.pfWakeupCallback   = wakeup_callback;   // Specify the callback function to be
    executed after the snapshot is restored.
    stRunstopParam.uwFlashEraseAlignSize = NOR_ERASE_ALIGN_SIZE; // Alignment erasing parameter
    of the medium
    stRunstopParam.uwFlashWriteAlignSize = NOR_WRITE_ALIGN_SIZE; // Alignment writing parameter
    of the medium
    stRunstopParam.uwFlashReadAlignSize = NOR_READ_ALIGN_SIZE; // Alignment reading parameter
    of the medium
    stRunstopParam.uwImageFlashAddr     = 0x100000;        // Start address of the medium for
    burning the full image of the system
    stRunstopParam.uwWowFlashAddr       = 0x800000;        // Start address of the medium for
    storing snapshots
    LOS_MakeImage(&stRunstopParam);
#endif
#ifdef MAKE_WOW_IMAGE
#ifdef LOSCFG_KERNEL_RUNSTOP
    PRINTK("Image length 0x%x\n", OsWowImageSizeGet());
#endif
#endif /* MAKE_WOW_IMAGE */
}
```

Step 4 Configure the WOW_SRC variable.

Configure the WOW_SRC variable in the **build/mk/wow_scatter.mk** file in the root directory of the source code. Define the variable as the path of the service source file for calling the sleep and wakeup function, as shown in the following figure. **LITEOSTOPDIR** indicates the root directory of the source code.

```
WOW_SRC := $(LITEOSTOPDIR)/targets/board/$(LITEOS_PLATFORM)/os_adapt/os_adapt.c
```

Step 5 Compile the system image.

1. Run the following command in the root directory to compile all service code:
Huawei_LiteOS\$ make
2. After the compilation is complete, you can view the layout of image segments to check whether the snapshot image is successfully generated. In the directory where system images are generated, for example, **out/hi3556v200** of the Hi3556 V200 platform, run the **readelf -S vs_server** command to open the system image file **vs_server**.

Figure 4-4 System ELF file vs_server readelf result

[12]	.wow_rodata	PROGBITS	80008020	0000d8	00b550	00	A	0	0	8
[13]	.wow_text	PROGBITS	80013570	00b628	04bb44	00	AX	0	0	8
[14]	.wow_data	PROGBITS	8005f0b4	05716c	001c3c	00	WA	0	0	8
[15]	.wow_bss	NOBITS	80060cf0	058da8	008e1c	00	WA	0	0	8

The figure shows segment information (including the segment name, start address, and offset) related to RunStop. In the preceding figure, .wow_rodata is read-only data segments in the RunStop image, .wow_text is the code segment, .wow_data is the data segment, and .wow_bss is the BSS segment.

3. Check the .text segment in the RunStop link script **Huawei_LiteOS/tools/scripts/ld/wow.ld**. wow.O(*.text*) is added to integrate some code symbols of the RunStop snapshot into the same segment, as shown in the following figure.

Figure 4-5 .text segment in the link script

```
.wow_text ALIGN (0x4): {
    wow.O(*.text*);
}

.wow_data ALIGN (0x4): {
    . = ALIGN (4); KEEP(wow.O( SORT (.liteos.table.*.wow.*))) ; . = ALIGN (4); wow.O(*.data*);
}

_wow_bss_start = ABSOLUTE (.);
```

Step 6 Burn all images to the flash memory.

Enter the serial port tool UI, and run the following command to burn all images to the 0x100000 address bit of the flash memory:

```
tftp 0x81000000 vs_server.bin;sf probe 0;sf erase 0x100000 0x300000;sf write 0x81000000 0x100000 0x300000;
```

In the preceding command, **vs_server.bin** is the name of the system image file, which is burnt to the high address bit 0x81000000 in the memory. Then burn the image to the flash memory. The start address is 0x100000, and the length of the image to be burnt is 0x300000. That is, the size of the image to be burnt cannot exceed 3 MB. Adjust the length based on the actual image size.

NOTE

You can select the image burning tool and method as required.

Step 7 Load all images.

You can run the following command to read all images from the 0x100000 address of the flash memory and load the images to the 0x80200000 memory address:

```
sf probe 0;sf read 0x80200000 0x100000 0x300000;
```

Run the following command to start the system from the memory address 0x80200000:

```
go 0x80200000;
```

Step 8 Based on the snapshot image size provided by the system, restore the system from the snapshot.

Figure 4-6 File that records print information during system startup

```
Relocation Offset is: 0d6d7000
Relocating to 8fed7000, new gd at 8fe36ef0, sp at 8fe36ed0
SPI Nor: Check Flash Memory Controller v100 ... Found
SPI Nor ID Table Version 1.0
SPI Nor(cs 0) ID: 0xc2 0x20 0x18
Block:64KB Chip:16MB Name:"MX25L128XX"
SPI Nor total size: 16MB
NAND: 0 MiB
MMC:
In: serial
Out: serial
Err: serial
Net: eth0
Hit any key to stop autoboot: 2 □□□ 1 □□□ 0
device 0 offset 0x100000, size 0x300000

SF: 3145728 bytes @ 0x100000 Read: OK
## Starting application at 0x80200000 ...

*****Hello Huawei LiteOS*****

Processor : Cortex-A7 * 2
Run Mode : SMP
GIC Rev : GICv2
build time : Aug 1 2019 09:47:25

*****

main core booting up...
osAppInit
releasing 1 secondary cores
cpu 0 entering scheduler
cpu 1 entering scheduler
Spi Nor ID:0xC2 0x20 0x18 0xC2 0x20 0x18 0xC2 0x20
Spi Nor Flash Info:
Name:"MX25L128XX" Size:16MB Block:64KB
Image length 0x170000

Huawei LiteOS # VFP_V300R003C28SPC006 (lwIP) [Aug 1 2019][09:47:40]
###tcpip_init done####
Ethernet start.hisi_eth: User did not set phy mode, use default=rmi
hisi_eth: User did not set phy addr, auto scan...
Detected phy addr 1, phyid: 0x1cc816
Link is Up - 100Mbps/Full
```

Based on the information printed during system startup, you can obtain the size of the snapshot image for RunStop, as shown in the red box in the preceding figure.

In this sample, the snapshot size is 0x170000. The snapshot is written to the 0x800000 address of the medium (the start address of the medium for saving the snapshot is 0x800000 according to the snapshot service code written in [Step 3](#)).

Based on the information printed during system startup, you can obtain the size of the snapshot image for RunStop, as shown in the red box in the preceding figure. In this sample, the snapshot size is 0x170000.

```
sf probe 0;sf read 0x80200000 0x800000 0x170000;
go 0x80200000;
```

The following figure is an example of the system startup results.

Figure 4-7 Example of print information after commands are executed in U-Boot

```
Relocation Offset is: 0d6d7000
Relocating to 8fed7000, new gd at 8fe36ef0, sp at 8fe36ed0
SPI Nor: Check Flash Memory Controller v100 ... Found
SPI Nor ID Table Version 1.0
SPI Nor(cs 0) ID: 0xc2 0x20 0x18
Block:64KB Chip:16MB Name:"MX25L128XX"
SPI Nor total size: 16MB
NAND: 0 MiB
MMC:
In: serial
Out: serial
Err: serial
Net: eth0
Hit any key to stop autoboot: 2 □□□ 1 □□□ 0
device 0 offset 0x800000, size 0x180000
SF: 1572864 bytes @ 0x800000 Read: OK
## Starting application at 0x80200000 ...
Image length 0x170000

Huawei LiteOS # VPP_V300R003C28SPC006 (lwIP) [Aug 1 2019][09:47:40]
####tcpip_init done####
Ethernet start.hisi_eth: User did not set phy mode, use default=rmi
hisi_eth: User did not set phy addr, auto scan...
Detected phy addr 1, phyid: 0x1ec816
Link is Up - 100Mbps/Full
```

The system is successfully started from the snapshot.

----End

4.3.2.3 Precautions

- When compiling the sleep and wakeup service, ensure that all code and data of the snapshot are covered before `#ifndef MAKE_WOW_IMAGE`. Otherwise, the system restored from the snapshot does not contain any sleep and wakeup services.
- The sleep and wakeup function depends on the storage medium. LiteOS depends on the flash memory. Therefore, this function depends on the flash memory. If the flash memory is disabled, the sleep and wakeup function is also disabled. If you need to enable the sleep and wakeup function on other storage media, perform adaptation based on the current flash configuration.

4.3.3 System Sleep

4.3.3.1 Overview

Basic Concept

System sleep is a system low-power consumption mechanism that balances system performance and low-power consumption. You can configure the sleep policy to adjust the system clock frequency and peripheral working mode or power off the system in standby mode, thereby reducing power consumption and improving the battery life.

The low-power consumption management and control of the system sleep are as follows:

1. Sleep mode
2. Sleep voting mechanism
3. System frequency modulation

Low-Power Process Registration Structure

Currently, system sleep supports light sleep and deep sleep modes. By default, the system sleep contains the light sleep logic. If you need to customize the sleep logic or deep sleep, you can register the low-power consumption process structure **struct PowerMgrParameter**.

```
typedef struct {
    PowerMgrRunOps runOps;          /* power manager run operations */
} PowerMgrParameter;

typedef struct {
#ifdef LOSCFG_KERNEL_DYN_FREQ
    UINT32 (*changeFreq)(LosFreqMode freq); /**< Tune system frequency */
#endif
#ifdef (LOSCFG_KERNEL_LIGHTSLEEP) || defined (LOSCFG_KERNEL_DEEPSLEEP)
    LosSleepMode (*getSleepMode)(VOID); /**< GetSleepMode, provided for special needs before entering sleep */
#endif
#ifdef LOSCFG_KERNEL_LIGHTSLEEP
    VOID (*enterLightSleep)(VOID); /**< Enter light sleep */
#endif
#ifdef LOSCFG_KERNEL_DEEPSLEEP
#ifdef LOSCFG_LOWPOWER_SLEEP_USERCONFIG
    VOID (*userPreConfig)(VOID); /**< UserPreconfig, provided for special needs before entering sleep */
    VOID (*userPostConfig)(VOID); /**< UserPostconfig, provided for special needs after wakeup */
#endif
    VOID (*systemWakeup)(VOID); /**< System wakeup */
    VOID (*suspendConfig)(VOID); /**< Suspend device before entering deep sleep */
    VOID (*resumeConfig)(VOID); /**< Resume device after wakeup from deep sleep */
#ifdef LOSCFG_KERNEL_SMP
    VOID (*otherCoreResume)(VOID); /**< Other core Resume for multi-core scenes */
#endif
    VOID (*enterDeepSleep)(VOID); /**< Enter deep sleep */
    VOID (*setWakeUpTimer)(UINT32 timeout); /**< Set wakeup timer */
    VOID (*withdrawWakeUpTimer)(VOID); /**< Withdraw wakeup timer */
#endif
#ifdef LOSCFG_KERNEL_RAM_SAVE
    VOID (*contextSave)(VOID); /**< Context save */
    VOID (*contextRestore)(VOID); /**< Context restore */
#endif
#ifdef LOSCFG_LOWPOWER_SLEEP_WFI
    VOID (*enterWFI)(VOID); /**< Enter WFI mode */
#endif
} PowerMgrRunOps;
```

4.3.3.2 Development Guidelines

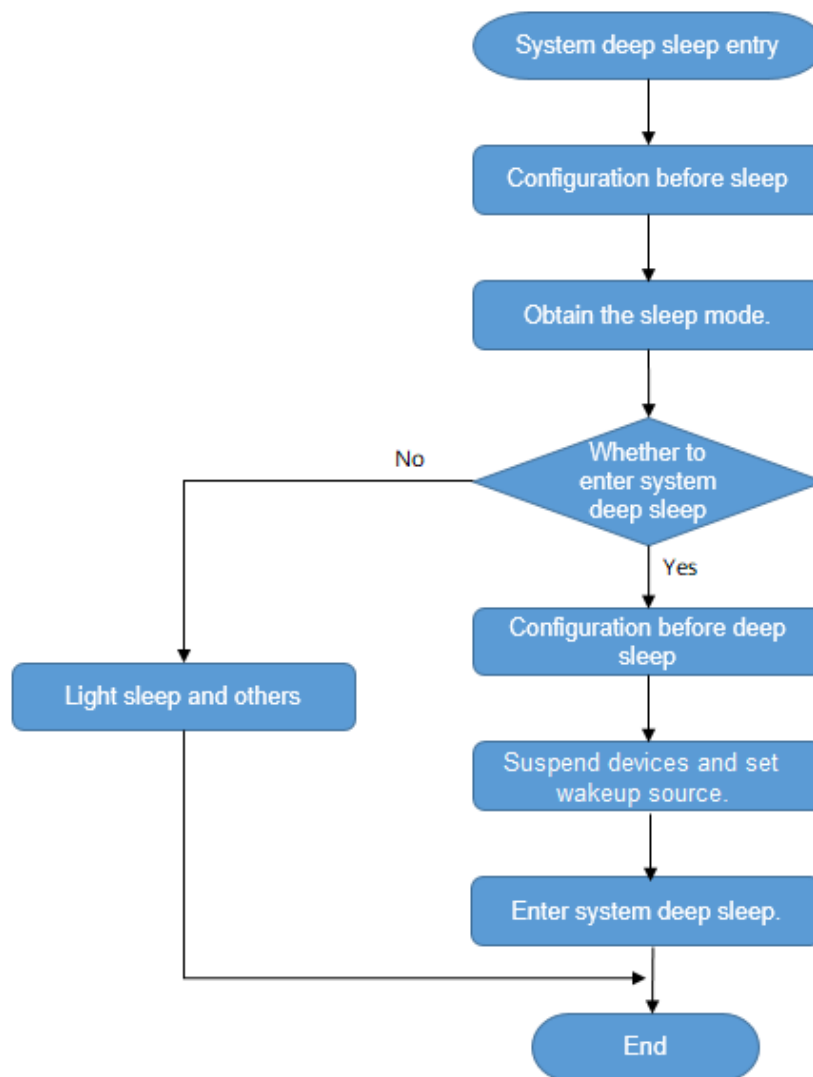
You can customize sleep policies based on service scenarios, define the sleep modes that can or cannot be entered, and select different sleep modes. Deep sleep mode may involve specific hardware and clock operations. Therefore, you can implement the corresponding functions to adapt to the hardware. In addition, the system sleep module has reserved APIs at the initial and end phases of the sleep process. You can extend the process for scenarios with special requirements.

The system sleep module of LiteOS provides the following functions. For details about the APIs, see the API reference.

Table 4-3 Functions

Function Category	API	Description
Enabling or disabling system sleep	LOS_EnableLowPower	Enables the system sleep function, which takes effect only in debug mode.
	LOS_DisableLowPower	Disables the system sleep function, which takes effect only in debug mode.
System sleep	LOS_PowerMgrSleepLock	Prohibits entering the specified sleep mode.
	LOS_PowerMgrSleepUnlock	Allows to enter the specified sleep mode.
	LOS_LowpowerHookReg	Registers the entry function for system sleep.
System frequency modulation	LOS_PowerMgrChangeFreq	Changes system frequencies.

System sleep provides a default system sleep process, as shown in the following figure.



When the system enters the sleep mode, the sleep mode is obtained based on the sleep time of the system and the sleep policy defined by the user.

- If the deep sleep mode is used, the configuration before the deep sleep is performed, for example, masking interrupts and saving the device status. Then the device is suspended and the wakeup source is set. The system enters the deep sleep mode.
- If light sleep mode or another mode is used, the corresponding process is executed. Generally, the tickless mode is used together with the light sleep mode to mask tick interrupts. You can also use the customized tick masking and recovery mechanism. By default, the light sleep mode is used only in Tickless + WFI mode.

Development Procedure

The typical development process of system sleep is as follows:

1. Open the menu and choose **Kernel > Enable Extend Kernel > Enable Low Power Management Framework > Low Power Management Configure**.

Configuration Item	Description	Value Range	Default Value	Dependency
config LOSCFG_KERNEL_LIGHTSLEEP	Enables/Disables light sleep mode tailoring.	YES/NO	NO	LOSCFG_KERNEL_LOWPOWER
LOSCFG_KERNEL_DEEPSLEEP	Enables/Disables deep sleep mode tailoring.	YES/NO	NO	LOSCFG_KERNEL_LOWPOWER, which requires the support of the hardware platform
LOSCFG_KERNEL_RAM_SAVE	Enables/Disables memory saving during system deep sleep.	YES/NO	NO	LOSCFG_KERNEL_DEEPSLEEP, which requires the support of the hardware platform
LOSCFG_LOWPOWER_SLEEP_USERCONFIG	Enables/Disables pre-/post-processing during system sleep.	YES/NO	NO	LOSCFG_KERNEL_DEEPSLEEP
LOSCFG_KERNEL_DYN_FREQ	Enables/Disables frequency modulation tailoring during system sleep.	YES/NO	NO	LOSCFG_KERNEL_LOWPOWER, which requires the support of the hardware platform
LOSCFG_LOWPOWER_SLEEP_VETO	Enables/Disables vote tailoring during system sleep.	YES/NO	NO	LOSCFG_KERNEL_LOWPOWER
LOSCFG_LOWPOWER_SLEEP_WFI	Enables/Disables WFI mode customization during system sleep.	YES/NO	NO	LOSCFG_KERNEL_LOWPOWER
LOSCFG_MAX_SLEEP_TIME	Configures the maximum sleep threshold during system sleep.	INT	10000	LOSCFG_KERNEL_LOWPOWER
LOSCFG_MIN_LIGHT_SLEEP_TIME	Configures the minimum light sleep threshold.	INT	50	LOSCFG_KERNEL_LIGHTSLEEP
LOSCFG_MIN_DEEP_SLEEP_TIME	Configures the minimum deep sleep threshold.	INT	2000	LOSCFG_KERNEL_DEEPSLEEP

Configuration Item	Description	Value Range	Default Value	Dependency
LOSCFG_DELTA_TICKS	Configures the tick compensation during system deep sleep.	INT	1	LOSCFG_KERNEL_DEEPSLEEP
LOSCFG_EXT_TIMER_FREQ	Configures the external clock frequency during system deep sleep.	INT	1	LOSCFG_KERNEL_DEEPSLEEP

2. In menuconfig, you can configure the system sleep and wakeup policy, including the minimum light/deep sleep thresholds and maximum sleep threshold. If the deep sleep mode is enabled, you can determine whether to enable the customized WFI, pre- and post-sleep processing, deep sleep tick compensation, and external clock frequency as required.
3. The sleep API corresponding to the sleep mode is adapted. For details, see [Low-Power Process Registration Structure](#).
4. When the system is busy, call the LOS_PowerMgrSleepLock API to forbid the system to enter the specified sleep mode (deep sleep or light sleep). When the system service is idle, call the LOS_PowerMgrSleepUnlock API to allow the system to enter the specified sleep mode. Note that the two APIs must be used together.
5. When the system is in standby state, the system automatically enters the corresponding sleep mode based on the service voting and the actual sleep duration.

4.3.3.3 Precautions

1. When deep sleep mode is enabled, you are advised to write the cache back to the memory in the enterDeepSleep hook function of the substructure member struct PowerMgrRunOps in [Low-Power Process Registration Structure](#).
2. The frequency modulation mechanism provided by low power consumption. If the system enters the idle state after startup and before the first service frequency adjustment, the system adjusts the frequency to the highest value by default.

4.3.3.4 Programming Example

Description

The following functions are implemented:

1. Register the low-power process framework and adapt to functional APIs such as the deep sleep API and RTC enable API.
2. Create a user task and delays 5000 ticks in the task to enable the system to enter the deep sleep mode.

Programming Example

Prerequisites: The system deep sleep mode has been configured in menuconfig, and the sleep thresholds **LOSCFG_MIN_SLEEP_TIME**, **LOSCFG_MAX_SLEEP_TIME**, and **LOSCFG_MIN_DEEP_SLEEP_TIME** have been configured.

The implementation of some codes is as follows:

1. Register the low-power framework.

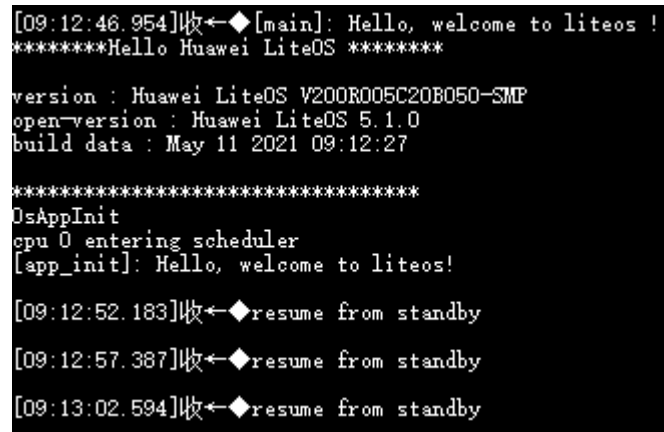
```
#ifndef LOSCFG_KERNEL_LOWPOWER
#include "los_lowpower.h"
#endif
const PowerMgrParameter gPowerMgrParameter = {    // Allows users to customize the system
sleep configuration. Member functions can be implemented as required.
    .runOps = {
#ifdef LOSCFG_KERNEL_LIGHTSLEEP
        .enterLightSleep = EnterLightSleep,        // Light sleep action. Set it to enter the WFI mode.
        .getSleepMode = GetSleepMode,              // Obtains the user sleep mode.
    #endif
#ifdef LOSCFG_KERNEL_DEEPSLEEP
        .suspendConfig = SuspendDevice,            // Disables peripheral action.
        .resumeConfig = ResumeDevice,              // Resumes peripherals.
        .enterDeepSleep = EnterDeepSleep,          // Deep sleep action. Set it to the STM32 standby
mode.
        .setWakeUpTimer = SetWakeUpTimer,          // Sets the wakeup source. In this example, the
external real-time clock (RTC) is enabled based on the input parameter (sleep time).
        .withdrawWakeUpTimer = WithdrawWakeUpTimer, // Disables wakeup source. Set this
parameter to stop RTC timing.
    #endif
    }
};
INT32 main(VOID)
{
    UINT32 ret;
#ifdef LOSCFG_KERNEL_LOWPOWER
    // Register the low-power consumption process framework. If only the default tickless and light
sleep modes are required, call LOS_PowerMgrInit(NULL) to register the default system framework.
    LOS_PowerMgrInit(gPowerMgrParameter);
#endif
    HardwareInit();
    printf("[main]: Hello, welcome to liteos !\n");
    PRINT_RELEASE("*****Hello Huawei LiteOS *****\n"
        "\nversion : %s\nopen-version : %s %u.%u.%u\n"
        "build data : %s %s\n\n"
        "*****\n",
        HW_LITEOS_VER, HW_LITEOS_SYSNAME, MAJ_V, MIN_V, REL_V, __DATE__,
        __TIME__);
    OsSetMainTask();
    OsCurrTaskSet(OsGetMainTask());
    ret = OsMain();
    if (ret != LOS_OK) {
        return LOS_NOK;
    }
    RTC_Start();
    g_kernelInit = 1;
    OsStart();
    return LOS_OK;
}
```

2. Create a task and perform the delay operation in the task entry function:

```
STATIC VOID task_demo1(VOID)
{
    while (1) {
        osDelay(5000); // The task delay is 5000 ticks.
    }
}
```

Result Verification

The deep sleep threshold is set to **2000** and the task delay is 5000 ticks. Therefore, the system enters the deep sleep mode:



```
[09:12:46.954]收←◆[main]: Hello, welcome to liteos !
*****Hello Huawei LiteOS *****

version : Huawei LiteOS V200R005C20B050-SMP
open-version : Huawei LiteOS 5.1.0
build data : May 11 2021 09:12:27

*****
OsAppInit
cpu 0 entering scheduler
[app_init]: Hello, welcome to liteos!

[09:12:52.183]收←◆resume from standby
[09:12:57.387]收←◆resume from standby
[09:13:02.594]收←◆resume from standby
```

As shown in the preceding figure, the system wakes up from the standby mode every 5s.

Complete Code

demo.rar

4.3.4 Tickless

4.3.4.1 Overview

Basic Concepts

Tickless mechanism is a new timing mechanism that uses dynamic clock interrupts to replace periodic clock timing.

Task scheduling is usually triggered by clock interrupts. When periodic timing is implemented, whether task scheduling requirements are met is checked when each timer interrupt occurs. However, only a few timer interrupts are able to trigger task scheduling. The software timer task can be implemented only when the clock interrupt is triggered. Each time the clock interrupt is triggered, the system checks whether the software timer expires. If the timer expires, the callback function of the timer is scheduled. When most clock interrupts occur, the software timer does not expire. In the tickless mechanism, unnecessary clock interrupts are omitted to determine the time when the next meaningful clock interrupt is generated, and no meaningless clock interrupt is generated. The system power consumption can be effectively reduced by eliminating meaningless clock interrupts when the CPU is idle.

Operation Mechanism

The tickless mechanism of LiteOS uses idle mode, that is, the tickless mechanism is enabled when the CPU is idle (in idle tasks). By calculating the time for the next task scheduling and the time for the next software timer to expire, the smaller

value is set as the time for the next clock interrupt. In this way, meaningless clock interrupts are reduced.

4.3.4.2 Development Guidelines

Usage Scenarios

The CPU is idle for a long time, and power saving is required.

Development Process

The typical development process of the tickless mode is as follows.

1. Open the menu and choose **Kernel > Enable Extend Kernel > Enable Low Power Management Framework** to enable tickless mode.

Configuration Item	Description	Value Range	Default Value	Dependency
LOSCFG_KERNEL_TICKLESS	Tailoring switch of the tickless mode	YES/NO	YES	LOSCFG_KERNEL_LOWPOWER

2. When the system does not run services for a long time, the system automatically enters the tickless mode based on the sleep time after entering the idle task state.

Platform Differences

None

4.3.4.3 Precautions

An interrupt timer is used in both the tickless implementation and calculation of system time. You are not advised to enable the tickless mode when high resolution of time is required.

The number of bits in the systick register varies according to platforms. To avoid register overflow, you need to calculate the maximum number of ticks that can run properly in tickless mode based on the maximum value allowed by the register, system clock frequency, and **LOSCFG_BASE_CORE_TICK_PER_SECOND** when configuring the tickless sleep time. For example, on the cortex-m4 platform, the SysTick->LOAD register is 24 bits, the system clock frequency is 120 MHz, and **LOSCFG_BASE_CORE_TICK_PER_SECOND** is set to **1000**. In this case, the tickless sleep time must be less than or equal to the threshold $(2^{24} - 1) / (120000000 / 1000) = 139$ ticks.

4.4 Access Permission Management

4.4.1 Overview

Access permission management: The memory protection/memory management unit of the hardware is configured to control the access permission on the destination address when the CPU is in different running states.

CAUTION

The memory protection/memory management unit design may vary according to the target platform. Before using the APIs provided in this section, read the related documents of the target platform to understand the hardware capabilities and usage restrictions of the platform.

Before using this function, ensure that the access permission control is enabled (you can open the menu and choose **Targets > Enable Access Permission Control** to enable the corresponding macro `LOSCFG_APC_ENABLE`). After the access permission control is enabled, the system enables the corresponding memory management units based on the architecture, including the PMP in RISC-V, the MPU in the Cortex-M architecture, and MMU in the Cortex-A architecture.

4.4.2 PMP

4.4.2.1 Overview

Basic Concepts

To improve system security and configure the access permission of physical addresses using software, RISC-V provides a standard control status register (CSR) to support memory protection. On a general RISC-V platform, a Physical Memory Protection (PMP) module supports 16 PMP entries, and the PMP entries are configured by using two types of registers: `pmpcfg0-3` (8-bit for each entry) and `pmpaddr0-15`. Huawei-developed chips may have an extra type of registers: `memattr/memattrh` (4-bit for each entry). Each entry has its own programmable start address, size (`pmpaddrX`), and setting (`pmpcfgX`). Some background memory area features (`memattr/memattrh`) are also supported.

The PMP provides the following functions:

- The SRAM or RAM space is defined as Non-Writable to prevent code from being tampered with.
- The SRAM or RAM space is defined as Non-Executable to prevent code injection attacks.
- The PMP `memattr/memattrh` is used to define the attributes of the memory.

4.4.2.2 Development Guidelines

Functions

The PMP of Huawei LiteOS provides the following functions:

Table 4-4 Supported APIs by the PMP

Function Category	API	Description
Disabling PMP regions	ArchRegionDisable	Disables address protection regions.
Setting PMP regions	ArchProtectionRegionSet	Sets address protection regions.
Enabling the cache	ArchICacheEnable	Enables iCache and sets the cache size.
	ArchDCacheEnable	Enables DCache and sets the cache size.
	ArchICachePrefetchEnable	Enables iCache prefetch.
	ArchDCachePrefetchEnable	Enables DCache prefetch.
Refreshing all caches	ArchICacheFlush	Refreshes all iCache lines.
	ArchDCacheFlush	Refreshes all DCache lines.
Refreshes the cache based on the specified space size.	ArchDCacheFlushByVa	Refreshes DCache based on the specified start address and space size.
	ArchICacheFlushByVa	Refreshes iCache based on the specified start address and space size.
Refreshes the cache within a specified address range.	ArchICacheFlushByAddr	Refreshes iCache within a specified address range.
	ArchDCacheFlushByAddr	Updates DCache within a specified address range.
	ArchDCacheCleanByAddr	After DCache within the specified address range is refreshed, DCache within the address range is invalid.
Invalid caches	ArchDCacheInvalidByVa	DCache is invalid according to the specified start address and space size.
	ArchDCacheInvalidByAddr	Invalid DCache within the specified address range.

PMP Error Codes

N o.	Definition	Error Code	Description	Reference Solution
1	LOS_ERRNO_P MP_PTR_NULL	0x0200 1200	Null pointer detection error	Ensure that parameters are set correctly.
2	LOS_ERRNO_P MP_INVALID_B ASE_ADDRESS	0x0200 1201	When a region is configured, the base address is not 4-byte- aligned.	Ensure that the base addresses are aligned.
3	LOS_ERRNO_P MP_INVALID_C APACITY	0x0200 1202	When a region in NAPOT mode is configured, the size is not an integer power of 2.	Ensure that the range protected is an integer power of 2.
4	LOS_ERRNO_P MP_ATTR_INVA LID	0x0200 1203	An invalid value is used when the memory attribute of the region is configured.	Ensure that the memory attributes set are supported by the current hardware platform.
5	LOS_ERRNO_P MP_INVALID_N UMBER	0x0200 1204	Invalid region ID	Select a valid region.
6	LOS_ERRNO_P MP_TOR_INVAL ID_MATCH_MO DE	0x0200 1205	When the TOR/OFF mode of a region is changed to a non- TOR/OFF mode, the system detects that the next region has been set to the TOR mode.	Do not adjust the mode.
7	LOS_ERRNO_P MP_NAPOT_IN VALID_MATCH_ MODE	0x0200 1206	When a region is changed from the NAPOT mode to the non-NAPOT mode, it is detected that the next region has been set to the TOR mode.	Do not adjust the mode.
8	LOS_ERRNO_P MP_REGION_IN VALID_ACCESS	0x0200 1207	Do not use this error code currently.	-

N o.	Definition	Error Code	Description	Reference Solution
9	LOS_ERRNO_PMP_BASE_ADDRESS_NOT_IN_RANGE	0x02001208	When a region is configured, the base address of the region is not within the range allowed by the system.	Adjust the base address to a proper value.
10	LOS_ERRNO_PMP_REGION_SIZE_IS_TOO_BIG	0x02001209	When a region is configured, the range of the region is too large.	Adjust the range to a proper value.
11	LOS_ERRNO_PMP_ENTRY_IS_LOCKED	0x0200120a	The region to be operated on has been locked.	Select and operate on another region.
12	LOS_ERRNO_PMP_INVALID_MATCH	0x0200120b	The configuration mode is NA4, which is not supported by the hardware.	Use other modes.
13	LOS_ERRNO_PMP_NAPOT_ENCODING_NON_MATCHED	0x0200120c	When a region in NAPOT mode is configured, the range set is improper.	Set a proper range.
14	LOS_ERRNO_PMP_TOR_LOWER_BOUND_LOCKED	0x0200120d	When a region in TOR mode is configured, the previous region has been locked.	Check whether the previous region has been locked. If yes, select a proper region.
15	LOS_ERRNO_PMP_TOR_LOWER_BOUND_NO_MATCHED	0x0200120e	When a region in TOR mode is configured, the low address of the region is invalid.	Check whether the low address of the region is properly set. If not, adjust it.
16	LOS_ERRNO_PMP_SECTL_INVALID	0x0200120f	An invalid sectl is set.	Set a valid sectl.

NOTICE

For details about the error code definition, see [Error Codes](#). The ID of the PMP module is the same as that of the MPU module. The value is 0x12.

PMP Entry Structure Definition (for LX131)

```
typedef struct {
    /* Index of PMP entry */
    UINT8 ucNumber;
```

```

/* whether lock, Works in any supported mode */
BOOL bLocked;

/*
 * Address matching mode of PMP configuration,
 * Currently supported modes: PMP_RGN_ADDR_MATCH_OFF,
 * PMP_RGN_ADDR_MATCH_TOR and PMP_RGN_ADDR_MATCH_NAPOT.
 */
UINT8 ucAddressMatch;

/*
 * access permission of read, write and instruction execution.
 * The changed member is not valid in PMP_RGN_ADDR_MATCH_OFF mode.
 */
PmpAccInfo accPermission;

/*
 * set the base address of the protected region,
 * the base address must be in the range of RAM.
 * It should be cache line aligned under the cache.
 */
UINT32 uwBaseAddress;

/*
 * PMP entry region size: It is valid only in PMP_RGN_ADDR_MATCH_NAPOT mode,
 * requiring >= 8byte and aligned power-of-2 byte.
 * It should be cache line aligned under the cache.
 */
UINT32 uwRegionSize;

/*
 * PMP memory attribute register information:
 * The changed member is not valid in PMP_RGN_ADDR_MATCH_OFF mode.
 */
PmpMemAttrInfo memAttr;

/*
 * Security control register information:
 * The changed member is not valid in PMP_RGN_ADDR_MATCH_OFF mode.
 */
PmpSeControlInfo seCtl;
} PMP_ENTRY_INFO;

```

The data structure consists of five parts:

- PMP entry number
- PMP configuration register: Lock bit + Address Matching + Access Permission(R/W/X)
- PMP address register: base address + region size (or end address)
- PMP memory attribute register: memory attribute, including cacheability and allocation policies
- PMP security control register: non-secure or secure

References for PMP

For details, see [Specifications](#) on the RISC-V official website.

For details about Huawei-developed chips, see related documents.

4.4.2.3 Precautions

- If PMP registers overlap and the same memory is located in the registers of two PMP entries, the memory access attribute and permission are based on the register with the smallest number.

- Only Machine-Mode (M-Mode) is available in Huawei LiteOS, and PMP is configured in M-Mode. Generally, the access permission of the PMP entry is set to be valid only in the User-Mode (U-Mode), and the access permission of the PMP entry is valid in the M-Mode only when the PMP entry is locked.

4.4.2.4 Programming Example

Example Description

In this example, PMP entry 3 is set. Set the memory area whose start address is 0x1000000 and size is 16 MB to readable, writable, and executable, and set the memory attribute to write-back write and read allocate. If the address mode is set to NAPOT, the setting of pmpaddr for the previous entry 2 is not required.

Example Code

Prerequisite: Access permission control has been enabled on the menu.

```
static UINT32 SetPmpEntry3(VOID)
{
    UINT32 ret;

    pstPmpInfo.ucNumber = 3;

    // set pmpXcfg register
    pstPmpInfo.bLocked = TRUE;
    pstPmpInfo.ucAddressMatch = PMP_RGN_ADDR_MATCH_NAPOT;
    pstPmpInfo.accPermission.readAcc = E_MEM_RD_ACC_RD;
    pstPmpInfo.accPermission.writeAcc = E_MEM_WR_ACC_WR;
    pstPmpInfo.accPermission.excuteAcc = E_MEM_EX_ACC_EX;

    // set pmpaddrX register
    pstPmpInfo.uwBaseAddress = 0x1000000;
    pstPmpInfo.uwRegionSize = PMP_REGION_SIZE_16MB;

    // set memattr register
    pstPmpInfo.memAttr = MEM_ATTR_WB_RW_ALLOC;
    pstPmpInfo.seCtl = SEC_CONTRLO_SECURE_NOMMU;

    ret = LOS_ProtectionRegionSet(&pstPmpInfo);
    if(ret != LOS_OK) {
        printf("set protection region failed.\n");
        return LOS_NOK;
    }

    return LOS_OK;
}
```

4.4.3 MPU

4.4.3.1 Overview

A memory protection unit (MPU) can be configured in the Cortex-M processor to protect the memory (mainly the memory and peripheral registers) and make the software more robust and reliable. If you intend to enable an MPU, you must program it as needed before using it. If the MPU is not enabled, it is equivalent to that no MPU is configured in the system. The MPU provides the following functions to improve system reliability:

- Preventing a user application from damaging the data used by the operating system
- Preventing a task from accessing the data area of another task
- Configuring key data area as read-only to eliminate data damage
- Detecting abnormal storage accesses, such as stack overflows and array access violations
- Configuring the access attributes of memory regions, for example, deciding whether an access is cacheable or bufferable.

4.4.3.2 Development Guidelines

Functions

The MPU of Huawei LiteOS provides the following functions:

Table 4-5 Supported APIs by the MPU

API	Description
ArchRegionDisable	Disables address protection regions.
ArchProtectionRegionSet	Sets address protection regions.

MPU Error Codes

No.	Definition	Error Code	Description	Reference Solution
1	LOS_ERRNO_MPU_PTR_NULL	0x02001200	Null pointer detection error	Ensure that parameters are set correctly.
2	LOS_ERRNO_MPU_INVALID_BASE_ADDRESS	0x02001201	An invalid base address is used during region configuration.	Ensure that a valid base address is used.
3	LOS_ERRNO_MPU_INVALID_CAPACITY	0x02001202	An invalid region size is used during region configuration.	Ensure that a valid region size is used.
4	LOS_ERRNO_MPU_NOT_CONFIGURED	0x02001203	The hardware platform does not support the MPU.	Ensure that the hardware platform supports the MPU.
5	LOS_ERRNO_MPU_INVALID_NUMBER	0x02001204	Invalid region ID	Ensure that you operate on a valid region.

No.	Definition	Error Code	Description	Reference Solution
6	LOS_ERRNO_MPU_REGION_IS_ENABLED	0x02001205	An enabled region is configured.	Ensure that the operation is performed on a proper region.
7	LOS_ERRNO_MPU_REGION_IS_DISABLED	0x02001206	Disable a region that is not enabled.	Ensure that the operation is performed on a proper region.
8	LOS_ERRNO_MPU_REGION_INVALID_ACCESS	0x02001207	The permission configured on the region is invalid.	Ensure that the configured permission is valid.
9	LOS_ERRNO_MPU_BASE_ADDRESS_NOT_IN_RAM	0x02001208	Do not use this error code currently.	-
10	LOS_ERRNO_MPU_REGION_SIZE_IS_TOO_BIG	0x02001209	Do not use this error code currently.	-

NOTICE

For details about the error code definition, see [Error Codes](#). Bits 8–15 belong to the MPU module, and the value is 0x12.

Parameters in MPU region

```
typedef struct {
    UINT8 number;           /* number of MPU register to be checked */
    UINT32 baseAddress;      /* set the base address of the protected region,
                             the base address must be in the range of RAM */
    UINT32 accessPermission; /* privilege level and user level access permission */
    BOOL sharable;          /* whether to share */
    BOOL cachable;          /* whether cache */
    BOOL buffable;          /* whether buffer */
    UINT32 regionSize;      /* region size */
    BOOL hfnmiena;          /* Whether in the NMI and hard fault service routine
                             is not mandatory in addition to MPU */
    BOOL xn;                /* To indicate whether instructions are fetchable in this region,
                             0 fetchable; 1: otherwise */
} MPU_REGION_INFO;
```

The data structure consists of four parts:

- MPU Region number
- MPU Access Permission(R/W/X)
- MPU address register: base address + region size (or end address)
- MPU memory attribute register: memory attribute, including cacheability and allocation policies

References for MPU

For details, see [documents on the Arm official website](#).

4.4.3.3 Precautions

- If the memory access permission conflicts with the access permission defined in the MPU, or the location of the accessed memory is not defined in the MPU register, the data transfer is blocked and an exception is triggered.
- If MPU registers overlap and the same memory is located in two MPU regions, the memory access attribute and permission are based on the register with the largest number.
- Only Privileged is available in Huawei LiteOS, and the MPU can be configured only under Privileged.

4.4.3.4 Programming Example

Example Description

In this example, region 0 and region 1 are configured. Configure the memory attributes and access permission of the 512 KB address space starting from 0x00000000 for region 0, and configure the memory attributes and access permission of the 512 KB address space starting from 0x20000000 for region 1.

Example Code

Prerequisite: Access permission control has been enabled on the menu.

```
VOID MPU_Config(VOID)
{
    MPU_REGION_INFO mpuInfo;

    mpuInfo.ucNumber = 0;
    mpuInfo.uwBaseAddress = 0x00000000; // start addr of text
    mpuInfo.uwAccessPermission = MPU_DEFS_RASR_AP_RO;
    mpuInfo.bSharable = 1;
    mpuInfo.bCachable = 1;
    mpuInfo.bBuffable = 1;
    mpuInfo.uwRegionSize = MPU_REGION_SIZE_512KB;
    mpuInfo.bHfnmiena = 1;
    mpuInfo.bXN = 0;
    LOS_SetProtectionRegion(&mpuInfo);

    mpuInfo.ucNumber = 1;
    mpuInfo.uwBaseAddress = 0x20000000; // start addr of data
    mpuInfo.uwAccessPermission = MPU_DEFS_RASR_AP_FULL_ACCESS;
    mpuInfo.bSharable = 1;
    mpuInfo.bCachable = 1;
    mpuInfo.bBuffable = 1;
    mpuInfo.uwRegionSize = MPU_REGION_SIZE_512KB;
    mpuInfo.bHfnmiena = 1;
    mpuInfo.bXN = 1;
    LOS_SetProtectionRegion(&mpuInfo);
}
```

4.4.4 MMU

4.4.4.1 Overview

Basic Concepts

A memory management unit (MMU) can be configured in the Cortex-A processor to map the virtual address to the physical address (virtual memory management) and provide protection for the memory based on the hardware.

Application Scenario

Some memory in the system cannot be modified. Otherwise, unpredictable results may occur. In this case, you can use the MMU to change the access permission of the memory to read-only. The access permission is checked when the segment of memory is accessed. If the permission is incorrect, an exception is triggered to protect the segment of memory data.

The cache and buffer can be used to control the memory cache states, including cache disabling, write-through, and write-back.

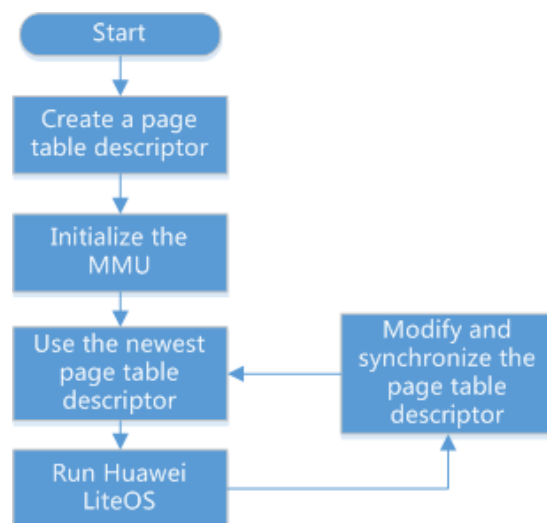
4.4.4.2 ARM32 MMU

Operation Mechanism

A table for page table descriptors is established to map a physical address to a virtual address, and an actual physical memory is operated and managed by using the virtual address as a medium.

The table for page table descriptors is created, modified, and managed based on the page table descriptor format of the primary chip. The memory performs mapping and permission control based on the table for page table descriptors. After the table for page table descriptors is created or modified, it needs to be written into the coprocessor CP15 to take effect. The coprocessor CP15 is the actual executor of memory management.

To sum up, the MMU operation is implemented by modifying the page table descriptor and controlling the CP15 coprocessor. The following figure shows the operation process.



The MMU of Huawei LiteOS has the following functions:

- Provides an API for controlling the memory cache/nocache attribute based on the hardware.
- Provides the API for controlling the memory access permission based on the hardware.

Functions

The MMU module of Huawei LiteOS provides the following functions.

API	Description
ArchSecPageEnable	Establishes the address relationship between the level-1 page table and the entered level-2 page table, and initializes the memory read and write permissions.
ArchMMUParamSet	Modifies the memory cache status, buffer status, and read/write permission status of a specified address segment.

The input parameter data type of the API is described as follows:

```
typedef struct
{
    UINT32 startAddr;    /* Start address */
    UINT32 endAddr;      /* End address */
    UINT32 uwFlag;        /* Configure the following attributes:
        * bit0: ACCESS_PERM_RW_RW/ACCESS_PERM_RO_RO(1/0)
        * bit1: CACHE_ENABLE/CACHE_DISABLE(1/0)
        * bit2: BUFFER_ENABLE/BUFFER_DISABLE(1/0)
        * bit3: EXEC_DISABLE/EXEC_ENABLE(1/0)
        * bit4: FIRST_SECTION/SECOND_PAGE(1/0)
        * bit5~7: ignore
        */
    SECONDD_PAGE *stPage; /* Specify the level-2 page table to be operated on. If uwFlag is set to
        FIRST_SECTION, stPage will be ignored */
} MMU_PARAM;
```

Structure of the level-2 page table is described as follows:

```
typedef struct
{
    UINT32 page_addr;        /* Use the start address of the memory described in the level-2 page table,
        which must be 1 MB alignment */
    UINT32 page_length;      /* Use the memory length described in the level-2 page table, which must
        be 1 MB alignment */
    UINT32 page_descriptor_addr; /* Address for storing level-2 page table entries, which must be 1 KB
        alignment */
    UINT32 page_type;        /* Level-2 page table attributes, including small pages (4 KB) and large
        pages (64 KB) */
}SECONDD_PAGE;
```

Example:

```
void code_protect(void)
{
    MMU_PARAM mPara;
    mPara.startAddr = (unsigned long)&__text_start;
    mPara.endAddr = (unsigned long)&__rodata1_end;
    mPara.uwFlag = BUFFER_ENABLE | CACHE_ENABLE | ACCESS_PERM_RO_RO;
```

```

    mPara.stPage = (SECONDD_PAGE *)&osPage; /* Flag A */
    ArchMMUParamSet(&mPara);
}

```

Description:

Set (write back) the memory between `__text_start` and `__rodata1_end` to cacheable, bufferable, and read-only. Multiple level-2 page tables may exist in the system. Mark A is used to specify the object page table operated on by the API.

4.4.4.2.1 Development Guidelines

Development Procedure

The typical development process of the MMU is as follows.

- Step 1** Define a level-2 page table structure variable, which is generally defined as a static or global variable.

```

SECONDD_PAGE osPage = {0};
__attribute__((aligned(MMU_1K))) UINT8 second_page_table_os[MMU_16K]; //Define a level-2 page
table with the size of 16 KB

```

- Step 2** Configure the attributes of the level-2 page table.

```

osPage.page_addr = SYS_MEM_BASE; // SYS_MEM_BASE is the start address of
the memory, which is generally the start address of the kernel text segment
osPage.page_length = (((UINTPTR)&__ram_data_start - SYS_MEM_BASE) + MMU_1M - 1) & ~(MMU_1M -
1)); // __ram_data_start is the start address of the data segment
osPage.page_descriptor_addr = (UINTPTR)second_page_table_os;
osPage.page_type = MMU_SECOND_LEVEL_SMALL_PAGE_TABLE_ID; //Specify
the page size to 4 KB instead of 64 KB

```

- Step 3** Enable the specified level-2 page table.

```

ArchSecPageEnable(&osPage, BUFFER_ENABLE | CACHE_ENABLE | ACCESS_PERM_RW_RW);

```

- Step 4** Change memory attributes.

```

void test_func(void)
{
    MMU_PARAM mPara;
    mPara.startAddr = (unsigned long)&__text_start;
    mPara.endAddr = (unsigned long)&__rodata1_end;
    mPara.uwFlag = CACHE_ENABLE | ACCESS_PERM_RO_RO;
    mPara.stPage = (SECONDD_PAGE *)&osPage;
    ArchMMUParamSet(&mPara);
}

```

----End

4.4.4.2.2 Precautions

Compared with the level-1 page table configuration, the memory address translation efficiency of the level-2 page table configuration is lower, and more TLB resources are occupied. Therefore, you are advised to use the level-1 page table for the memory that is frequently accessed.

4.4.4.2.3 Programming Example

Example Description

Modify the read/write permission of a memory region using the `ArchMMUParamSet` API, and perform write operations in the memory region to check whether the read/write permission is correctly modified.

Step 1 Change the read/write permission on the memory of a range to read-only.

Step 2 A simple write operation is performed on the memory in the segment.

If an exception occurs in the system, the memory is set to read-only successfully.

Step 3 Comment out the write operation in Step 2, call the API to change the read/write permission on the memory to read/write, and then perform the write operation again.

If no exception occurs in the system, the memory is set to the readable and writable state successfully.

----End

Example Code

```
extern char __text_start;
extern char __text_end;

UINT32 MMU_Sample()
{
    UINT32 *alignAddr;
    extern SECONDD_PAGE osPage;
    MMU_PARAM mPara;

    mPara.startAddr = (unsigned long)&__text_start;
    mPara.endAddr = (unsigned long)&__text_end;
    mPara.uwFlag = BUFFER_ENABLE | CACHE_ENABLE | ACCESS_PERM_RO_RO;
    mPara.stPage = (SECONDD_PAGE *)&osPage;

    PRINTK("---- TEST START ----\n");
    alignAddr = (UINT32 *)&__text_start;
    PRINTK(">>1\n");
    ArchMMUParamSet(&mPara);
    *alignAddr = 0xa;    //if write in the memory, an exception will be triggered.

    PRINTK(">>2\n");
    mPara.uwFlag = BUFFER_ENABLE | CACHE_ENABLE | ACCESS_PERM_RW_RW;
    ArchMMUParamSet(&mPara);
    *alignAddr = 0xb;

    PRINTK(">>3\n");
    PRINTK("---- TEST END ----\n");
    return LOS_OK;
}
```

Running Results

Do not comment out the `*alignAddr = 0xa;` code.

```

---- TEST START ----
>>1
uwExcType = 0x4
puwExcBuffAddr pc = 0x8000adc8
puwExcBuffAddr lr = 0x8000aa78
puwExcBuffAddr sp = 0x80093e30
puwExcBuffAddr fp = 0x80093e44
*****backtrace begin*****
traceback 0 -- lr = 0x8000b1e4
traceback 0 -- fp = 0x80093e84
traceback 1 -- lr = 0x8000b2c4
traceback 1 -- fp = 0x80093e9c
traceback 2 -- lr = 0x8000ae8c
traceback 2 -- fp = 0x80093eac
traceback 3 -- lr = 0x8000ac4c
traceback 3 -- fp = 0x80093ebc
traceback 4 -- lr = 0x800104fc
traceback 4 -- fp = 0x80093ecc
traceback 5 -- lr = 0x80026e1c
traceback 5 -- fp = 0x11111111

Name                      TID    Priority  Status
----                      -
Swt_Task                  0x0     0        QueuePend
IdleCore000               0x1    31        Ready
IT_TST_INI                0x2    25        Running

R0      = 0x800040c0
R1      = 0x800040a8
R2      = 0x800040a4
R3      = 0x0
R4      = 0x8000a000
R5      = 0x0
R6      = 0x8002a000
R7      = 0x800302e4
R8      = 0x80051df4
R9      = 0x8002c020
R10     = 0x80051df4
R11     = 0x80093e44
R12     = 0x2a
SP       = 0x80093e30
LR       = 0x8000aa78
PC       = 0x8000adc8
CPSR     = 0x200f0013
pcTaskName = IT_TST_INI
TaskID = 2
Task StackSize = 86016
system mem addr:0x800561c0

stack name      stack addr  total size  used size
-----
undef_stack_addr 0x8002c820   0x20       0x0
abt_stack_addr   0x8002c840   0x20       0xb
irq_stack_addr   0x8002c880   0x40      0x10
fiq_stack_addr   0x8002c8c0   0x40       0x0
svc_stack_addr   0x8002d8c0  0x1000     0x14f
startup_stack_addr 0x800563a0  0x200     0x138

```

Comment out the `*alignAddr = 0xa;` code.

```

---- TEST START ----
>>1
>>2
>>3
---- TEST END ----

```

Complete Code

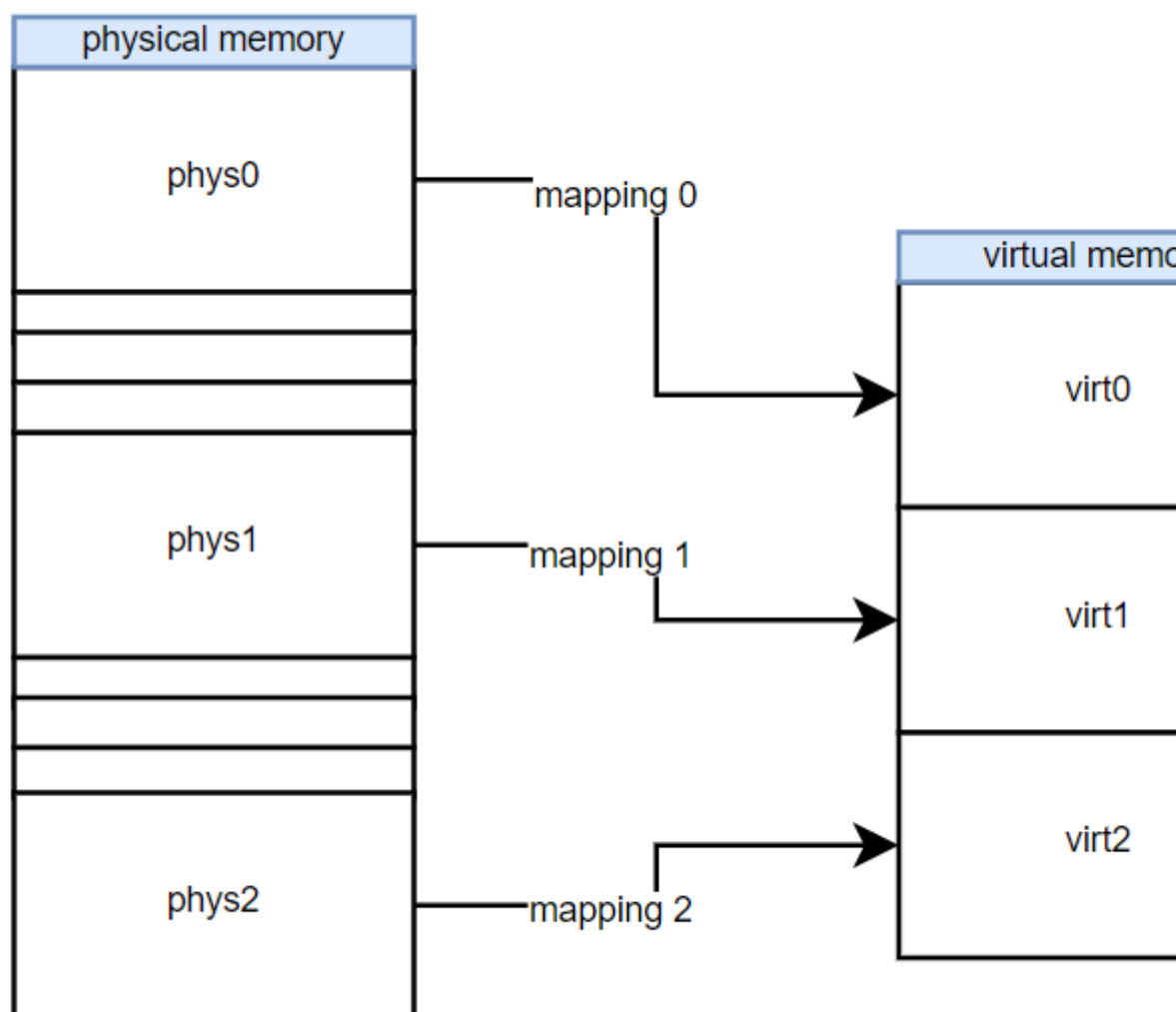
sample_MMU.c

4.4.4.3 ARM64 MMU

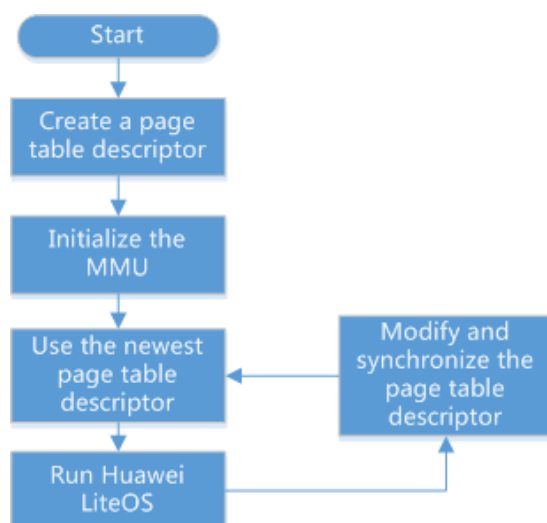
Operation Mechanism

A table for page table descriptors is established to map a physical address to a virtual address, and an actual physical memory is operated and managed by using the virtual address as a medium. The table for page table descriptors is created, modified, and managed based on the page table descriptor format of the primary chip. The memory performs mapping and permission control based on the table for page table descriptors.

Only one physical address can be mapped at a time by calling the ArchMmuMap API, for example, Phys0->virt0 in the following figure. If multiple inconsecutive physical addresses need to be mapped, you need to call the ArchMmuMap API for multiple times to establish the mapping and ensure that the mapped virtual addresses are consecutive, for example, Phys0->virt0, Phys1->virt1, and Phys2->virt2 in the following figure.



The operation is the same as that of the ARM32 MMU, which is implemented by maintaining the page table descriptor and controlling the system register. The following figure shows the detailed operation process.



4.4.4.3.1 Development Guidelines

Functions

The ARM64 MMU module of Huawei LiteOS provides the following functions.

API	Description
ArchMmuMap	Maps a physical page and a virtual address.
ArchMmuUnmap	Unmaps a physical page and a virtual address.
ArchMmuTableWalk	Traverses the virtual-physical mapping relationship in the virtual address space.
ArchMmuRegionInsert	Inserts a mapping relationship and creates a page table when the kernel starts (before the MMU is enabled).
ArchMmuTableStatistics	Collects statistics on page table usage.
ArchMmuLogLevelCfg	Sets the information display level.

MMU Error Codes

Error codes are returned for operations that may fail to be performed to quickly locate error causes.

N o.	Definition	Error Code	Description	Reference Solution
1	LOS_ERRNO_MMU_INP_UT_OVERFLOW	0x02000d00	The entered virtual address or physical address is too large.	Check the input.
2	LOS_ERRNO_MMU_INP_UT_ZERO	0x02000d01	The memory size is 0.	Check the input.
3	LOS_ERRNO_MMU_INP_UT_FLAG_INVALID	0x02000d02	Invalid page table attribute.	Check the page table attributes.
4	LOS_ERRNO_MMU_PAGETABLE_OOM	0x02000d03	The page table memory is exhausted.	Allocate more MMU page table memory.
5	LOS_ERRNO_MMU_PTE_BUSY	0x02000d04	The page table entry has been mapped.	Unmap the page table entry.
6	LOS_ERRNO_MMU_REGION_NOT_ENOUGH	0x02000d05	The number of regions is insufficient.	Increase the number of regions.
7	LOS_ERRNO_MMU_REGION_OVERLAPPED	0x02000d06	The entered region address range overlaps the previous region address range.	Check the input address.
8	LOS_ERRNO_MMU_INP_UT_NULL	0x02000d07	The address is empty.	Check the input of the ArchMmuTableStatistics function.
9	LOS_ERRNO_MMU_INP_UT_ADDR_INVALID	0x02000d08	The virtual address is different from the physical address.	- Check the virtual address and physical address. - Check whether the LOSCFG_ARCH_MMU_VA_MANAGE option needs to be enabled.

MMU Page Table Attributes

Page Table Attribute	Description
LOS_MMU_FLAGS_RW_NX_NORMAL_NC	Readable, writable, non-executable, non-cacheable
LOS_MMU_FLAGS_RO_X_NORMAL_WB	Read-only, executable, write-back cacheable
LOS_MMU_FLAGS_RW_NX_DEVICE_MAP	Readable, writable, non-executable, Device-nGnRE memory
LOS_MMU_FLAGS_RO_NX_NORMAL_WB	Read-only, non-executable, write-back cacheable

NOTE

Multiple log levels, such as debug, info, error, and emg, are provided to facilitate fault locating. The ArchMmuLogLevelCfg API can be used to change the log level of the MMU module. The default log level is error.

Development Procedure

1. Open the menu, enter the **Targets** menu, and enable the **Enable mmu** option to configure the MMU module.

Configuration Item	Description	Value Range	Default Value	Dependency
LOSCFG_ARCH_MMU_ENABLE	Determines whether to enable the MMU.	YES/NO	YES	LOSCFG_ARCH_ARM_AARCH64
LOSCFG_ARCH_MMU_VA_MANGLE	Determines whether the virtual address can be different from the physical address.	YES/NO	NO	LOSCFG_ARCH_MMU_ENABLE
LOSCFG_ARCH_MMU_PAGE_TBL_NUM	Specifies the number of MMU page tables.	[4, 512]	512	LOSCFG_ARCH_MMU_ENABLE
LOSCFG_ARCH_ARM64_PA_BITS	Specifies the bit width of the physical address.	{32, 36, 40, 42, 44, 48}	40	LOSCFG_ARCH_MMU_ENABLE

Configuration Item	Description	Value Range	Default Value	Dependency
LOSCFG_ARCH_ARM64_VA_BITS	Specifies the bit width of the virtual address.	{39, 48}	39	LOSCFG_ARCH_MMU_ENABLE

2. Map virtual and physical addresses.
3. Unmap virtual and physical addresses.

4.4.4.3.2 Precautions

In the current design, only 4 KB and 39-bit virtual addresses are supported. When the LOSCFG_ARCH_MMU_VA_MANAGE function is enabled, you need to properly divide the virtual address space layout by using the link script to ensure that the virtual addresses do not overlap.

4.4.4.3.3 Programming Example

Example Description

The following functions are implemented:

1. Collect statistics on page table usage.
2. Set the information display level.
3. Map virtual and physical addresses.
4. Traverse a memory mapping relationship.
5. Unmap virtual and physical addresses.

Example Code

The code is as follows:

```
#include "arch/mmu.h"
#define STRIDE_MEM_SIZE    0x100000
#define PHYS_MEM_START1    SYS_MEM_END
#define PHYS_MEM_START2    (SYS_MEM_END + 2 * STRIDE_MEM_SIZE)
#define VIRT_MEM_START1    0x0480000000
#define VIRT_MEM_START2    (VIRT_MEM_START1 + STRIDE_MEM_SIZE)
VOID SampleMmu(VOID)
{
    PageTablesStat ptr;
    ArchMmuLogLevelCfg(LOS_INFO_LEVEL); // Set the information display level.
    ArchMmuTableStatistics(&ptr);        // Statistics page table usage
    ArchMmuMap(VIRT_MEM_START1, PHYS_MEM_START1, STRIDE_MEM_SIZE,
MMU_RO_NX_NORMAL_WB);
    ArchMmuTableWalk(VIRT_MEM_START1, STRIDE_MEM_SIZE);
    ArchMmuMap(VIRT_MEM_START2, PHYS_MEM_START2, STRIDE_MEM_SIZE,
MMU_RO_NX_NORMAL_WB);
    ArchMmuUnmap(VIRT_MEM_START1, STRIDE_MEM_SIZE); // Unmap the memory.
}
```

Verification

The verification result is as follows:

```
Total page num:512 and pte num:262144, used page num:5 and pte num:652
Mapping [usr name]: vAddr 0x480000000 pAddr 0x520000000 size 0x100000 attr 0x600000000007a8
Allocating new page table index[5], table addr:0x42020000
Allocating new page table index[6], table addr:0x42021000
Unmapping: vAddr 0x480000000 size 0x200000
Recycling page table index[6], pte addr:0x420217f8, table addr:0x42021000
Recycling page table index[5], pte addr:0x42020000, table addr:0x42020000
```

Complete Code

sample_mmu.zip

4.5 CPUP

4.5.1 Overview

Basic Concepts

The central processing unit percent (CPUP) of Huawei LiteOS consists of the system CPUP and task CPUP.

System CPUP refers to the CPU usage of the system within a period of time. It reflects the CPU load and the system running status (idle or busy) in the given period of time.

Task CPUP refers to the CPU usage of a single task. It reflects the task status, busy or idle, in a period of time.

Interrupt CPUP refers to the interrupt usage within a period of time. It reflects the interrupt processing time of the system within a period of time.

Operation Mechanism

Huawei LiteOS records CPUP on a task basis. During task switching, the task switch-in time is recorded. Before the next task switching, the task switch-out time is recorded. The recorded time is added to the total CPU time of the task.

The CPUP is calculated as follows:

System CPUP = Total running time of all tasks except idle tasks/Total running time of the system

Task CPUP = Total running time of the task/Total running time of the system

Interrupt CPUP = Total running time of the interrupt/Total running time of the system

4.5.2 Development Guidelines

Application Scenario

Check whether the current system load exceeds the designed specifications based on the system CPUP.

Check whether the CPU usage of each task in the system meets the designed requirements based on each task CPUP.

Functions

The CPUP module of Huawei LiteOS provides the following functions. For details about the APIs, see the API reference.

Table 4-6 Functions

Function Category	API	Description
Obtaining the system CPUP	LOS_HistorySysCpuUsage	Obtains the system CPUP, excluding the CPUP of idle tasks.
Obtaining the task CPUP	LOS_HistoryTaskCpuUsage	Obtains the CPUP of a specified task.
	LOS_AllCpuUsage	Obtains the CPUP of all interrupts, when LOSCFG_CPUP_INCLUDE_IRQ is enabled and the input parameter flag is set to 0. The input parameter flag is set to a non-zero value or LOSCFG_CPUP_INCLUDE_IRQ is disabled to obtain the CPUP of all tasks, including that of idle tasks.
Resetting the CPUP	LOS_CpupReset	Resets the CPUP data, including the CPUP of the system and tasks.
Stopping CPUP statistics	LOS_CpupStop	Stops the CPUP statistics, including not updating the CPUP of interrupts and tasks.
Starting CPUP statistics	LOS_CpupStart	Starts the CPUP statistics, including clearing and re-collecting the CPUP of interrupts and tasks.

NOTE

1. The CPUP obtained using the preceding APIs is a permillage. Therefore, the valid range of the CPUP is 0 to 1000. If the system CPUP is 1000, it indicates that the system is fully loaded. If the task CPUP is 1000, it indicates that the task keeps running during a period of time.
2. The CPUP can be obtained in three modes, which are specified by the input parameter **mode**.
 - CPUP_LAST_MULTIRECORD (value: 0; original name: CPUP_LAST_TEN_SECONDS): indicates that the CPUP in the latest multiple sampling records is obtained. The default value is 10s.
 - CPUP_LAST_ONERECORD (value: 1; original name: CPUP_LAST_ONE_SECONDS): indicates that the CPUP of the latest sampling record is obtained. The default value is 1s.
 - CPUP_ALL_TIME: indicates the CPUP since the system starts. The value is 0xffff or any other value except 0 and 1.

CPUP Error Codes

No.	Definition	Error Code	Description	Reference Solution
1	LOS_ERRNO_CPUP_NO_MEMORY	0x02001e00	The memory is insufficient during initialization of the CPUP module.	Adjust OS_SYS_MEM_SIZE to ensure that sufficient memory is available for the CPUP module.
2	LOS_ERRNO_CPUP_TASK_PTR_NULL	0x02001e01	The input parameters contain at least one null pointer.	Set the pointers correctly.
3	LOS_ERRNO_CPUP_NO_INIT	0x02001e02	The CPUP module is not initialized.	Initialize the CPUP module before using it.
4	LOS_ERRNO_CPUP_MAXNUM_INVALID	0x02001e03	The input parameter (maximum number of threads or interrupts) set for the LOS_AllCpuUsage API is invalid.	Enter a correct maximum number of threads or interrupts.
5	LOS_ERRNO_CPUP_THREAD_NO_CREATED	0x02001e04	When the CPUP of a specified task is obtained, the task does not exist.	The system can only collect statistics on the CPUP of the tasks that have been created. Check whether the task corresponding to the entered task ID has been created.
6	LOS_ERRNO_CPUP_TSK_ID_INVALID	0x02001e05	An invalid task ID is set when the API for obtaining the CPUP of a specified task is called.	Check whether the set task ID is correct.

NOTICE

For details about the error code definition, see [Error Codes](#). Bits 8–15 belong to the CPUP module, and the value is 0x1e.

Development Procedure

The typical development process of the CPUP is as follows.

1. Open the menu and choose **Kernel > Enable Extend Kernel > Enable Cpu** to configure the CPUP module.

Configuration Item	Description	Value Range	Default Value	Dependency
LOSCFG_KERNEL_CPUP	Enables or disables the CPUP module tailoring.	YES/NO	YES	LOSCFG_KERNEL_EXTKERNEL
LOSCFG_CPU_P_START_STOP	Dynamically starts or stops CPUP.	YES/NO	NO	LOSCFG_KERNEL_CPUP && ! LOSCFG_SCHED_LOAD_BALANCE_CPUP
LOSCFG_CPU_P_INCLUDE_IRQ	After this configuration item is enabled, the CPUP of interrupts can be obtained from the LOS_HistoryTaskCpuUsage and LOS_AllCpuUsage APIs. If this configuration item is disabled, only the CPUP of tasks can be obtained.	YES/NO	YES	LOSCFG_KERNEL_CPUP && LOSCFG_ARCH_INTERRUPT_TAKEOVER
LOSCFG_CPU_P_CB_NUM_CONFIGURABLE	If this configuration item is enabled, the number of interrupt-related control blocks of the CPUP can be separately configured and does not depend on LOSCFG_PLATFORM_HWI_LIMIT. In this way, the CPUP size can be optimized.	YES/NO	NO	LOSCFG_CPUP_INCLUDE_IRQ
LOSCFG_CPU_P_IRQ_CB_NUM	Specifies the number of interrupt-related control blocks of the CPUP. Set this parameter based on the actual number of interrupts. The value cannot be greater than the value of LOSCFG_PLATFORM_HWI_LIMIT.	[1, LOSCFG_PLATFORM_HWI_LIMIT]	LOSCFG_PLATFORM_HWI_LIMIT	LOSCFG_CPUP_CB_NUM_CONFIGURABLE

Configuration Item	Description	Value Range	Default Value	Dependency
LOSCFG_CPU_P_SAMPLE_PERIOD	Specifies the CPUP sampling period. The unit is tick. It indicates the interval for running the CPUP software timer.	[10,1000]. The actual time range is [100ms, 10s].	LOSCFG_BASE_CORE_TICK_PER_SECOND	LOSCFG_KERNEL_CPUP && ! LOSCFG_SCHED_LOAD_BALANCE_CPUP
LOSCFG_CPU_P_HISTORY_RECORD_NUM	Specifies the maximum number of sampling records of the CPUP. It is actually a ring buffer, indicating that you can view the CPUP of the latest sampling records.	[1,10]	10	LOSCFG_KERNEL_CPUP && ! LOSCFG_SCHED_LOAD_BALANCE_CPUP

2. Obtain the system CPUP `LOS_HistorySysCpuUsage`
3. Obtain the CPUP of a specified task or interrupt `LOS_HistoryTaskCpuUsage`.
4. Obtain the CPUP of all tasks or interrupts `LOS_AllCpuUsage`.

Platform Differences

None.

4.5.3 Precautions

- The CPUP affects the performance to some extent. Generally, the CPUP of each task is required only during product development. Therefore, it is recommended that the CPUP be disabled during product release.
- After the configuration item `LOSCFG_CPUP_INCLUDE_IRQ` is disabled, the interrupt duration in the system is counted in the tasks where interrupts occur.

4.5.4 Programming Example

Example Description

The following functions are implemented:

1. Create a task for testing the CPUP.
2. Obtain the CPUP of all tasks or interrupts in the latest 1 second.
3. Obtain the total CPUP of the system (except idle tasks) in the latest 10 seconds.

4. Obtain the CPUP of a CPUP test task.

Example Code

Prerequisite: The CPUP module has been configured on the menu.

The code is as follows:

```
#include <unistd.h >
#include "los_task.h"
#include "los_cpup.h"

#define MAXTASKNUM 32

UINT32 cpupUse;
UINT32 g_cpuTestTaskId;

VOID Example_CPUP(VOID)
{
    printf("entry cpup test example\n");
    while(1) {
        usleep(100);
    }
}

UINT32 Example_CPUP_Test(VOID)
{
    UINT32 ret;
    TSK_INIT_PARAM_S cpupTestTask;
    CPUP_INFO_S cpupInfo[MAXTASKNUM];

    /* Create a task for testing the CPUP. */
    memset(&cpupTestTask, 0, sizeof(TSK_INIT_PARAM_S));
    cpupTestTask.pfnTaskEntry = (TSK_ENTRY_FUNC)Example_CPUP;
    cpupTestTask.pcName = "TestCpupTsk"; /* Test task name */
    cpupTestTask.uwStackSize = LOSCFG_BASE_CORE_TSK_DEFAULT_STACK_SIZE;
    cpupTestTask.usTaskPrio = 5;
    cpupTestTask.uwResved = LOS_TASK_STATUS_DETACHED;

    ret = LOS_TaskCreate(&g_cpuTestTaskId, &cpupTestTask);
    if(ret != LOS_OK) {
        printf("cpupTestTask create failed.\n");
        return LOS_NOK;
    }
    usleep(100);

    /* Number of running tasks or interrupts in the system */
    UINT16 maxNum = MAXTASKNUM;

    /* Obtain the CPUP of all tasks or interrupts in the latest 1 second */
    cpupUse = LOS_AllCpuUsage(maxNum, cpupInfo, CPUP_LAST_ONE_SECONDS, 0);
    printf("the system cpu usage in last 1s: %d\n", cpupUse);

    /* Obtain the total CPUP of the system (except idle tasks) in the latest 10 seconds. */
    cpupUse = LOS_HistorySysCpuUsage(CPUP_LAST_TEN_SECONDS);
    printf("the history system cpu usage in last 10s: %d\n", cpupUse);

    /* Obtain the CPUP of the specified task in the latest 1 second. The specified task in this test case is the
    CPUP test task created in the previous step. */
    cpupUse = LOS_HistoryTaskCpuUsage(g_cpuTestTaskId, CPUP_LAST_ONE_SECONDS);
    printf("cpu usage of the cpupTestTask in last 1s:\n TaskID: %d\n usage: %d\n", g_cpuTestTaskId,
    cpupUse);

    return LOS_OK;
}
```

Verification

The verification result is as follows:

```
the system cpu usage in last 1s: 15
the history system cpu usage in last 10s: 3
cpu usage of the cpupTestTask in last 1s:
TaskID: 10
usage: 0
```

Complete Code

sample_cpup.rar

4.6 Trace

4.6.1 Overview

Basic Concepts

The trace module records system behaviors in real time, which is similar to the video recording function. When an exception occurs in the system, you can use this function to view historical events to locate the fault.

Operation Mechanism

The trace module of Huawei LiteOS uses the static code stub in buffer recording mode. When the stub is executed, the context of the event is obtained and written to the buffer.

The input parameters of the instrumentation function need to provide the trace event type, event operation object, and event parameters. The information is written into the buffer. In addition, the event context information such as the event occurrence time and task information in the system is recorded in the buffer.

4.6.2 Development Guidelines

Application Scenario

- Use the trace function to understand the system running track and the running mechanism of the system.
- Use the trace function to analyze system operations before a system exception occurs and locate faults.

Functions

The trace module of Huawei LiteOS provides the following functions. For details about the APIs, see the API reference.

Table 4-7 Functions

Function Category	API	Description
Configuring the trace buffer	LOS_TraceInit	Configures the address and size of the trace buffer.
Starting/ Stopping trace event recording	LOS_TraceStart	Starts event recording.
	LOS_TraceStop	Stops event recording.
Recording data of trace operations	LOS_TraceRecordDump	Outputs trace buffer data.
	LOS_TraceRecordGet	Obtains the first address of the trace buffer.
	LOS_TraceReset	Clears events in the trace buffer.
Module that filters trace records	LOS_TraceEventMaskSet	Sets the event mask to record only events of some modules
Masking events with specific interrupt IDs	LOS_TraceHwiFilterHookReg	Registering the hook function for filtering events of a specific interrupt ID
Instrumentation function	LOS_TRACE_EASY	Simple instrumentation
	LOS_TRACE	Standard instrumentation

 NOTE

1. `LOS_TRACE_EASY(TYPE, IDENTITY, params...)` simple instrumentation.
 - Instrumentation in a sentence. You only need to insert the API into the target source code.
 - The value range of `TYPE` is `[0, 0xF]`, indicating different event types.
 - `IDENTITY` type `UINTPTR`, indicating the main object of the event operation.
 - The `Params` type is `UINTPTR`, indicating the parameter of the event.
 - Example:


```
LOS_TRACE_EASY(1, userId0, userParam1, userParam2);
LOS_TRACE_EASY(2, userId0);
LOS_TRACE_EASY(1, userId1, userParam1);
LOS_TRACE_EASY(2, userId1);
LOS_TRACE_EASY(3, userParam1, userParam2, userParam3); // params can reuse the
IDENTITY field (when the IDENTITY field is not required)
```
2. `LOS_TRACE(TYPE, IDENTITY, params...)` standard instrumentation.
 - Compared with simple instrumentation, dynamic event filtering and parameter tailoring are supported by standard instrumentation. You need to extend the functions based on rules.
 - `TYPE` is used to set the specific event type. You can customize the event type in enum `LOS_TRACE_TYPE` in the header file `los_trace.h`. For details about defining methods and rules, see other event types.
 - The types and meanings of `IDENTITY` and `Params` are the same as those of simple instrumentation.
 - Example:
 1. Define the event mask, that is, the module-level event type in enum `LOS_TRACE_MASK`. The definition specification is `TRACE_#MOD#_FLAG`, in which `#MOD#` indicates the module name. For example:


```
TRACE_FS_FLAG = 0x2000
```
 2. Define the event type in enum `LOS_TRACE_TYPE`. The definition specification is `#TYPE# = TRACE_#MOD#_FLAG | NUMBER`. For example:


```
FS_READ = TRACE_FS_FLAG | 0; // Read a file
FS_WRITE = TRACE_FS_FLAG | 1; // Write a file
```
 3. Define event parameters. The definition specification is `#TYPE#_PARAMS(IDENTITY, parma1...)` `IDENTITY, ...`, in which, `#TYPE#` is the same as the value of `#TYPE#` in the preceding specification in 2. For example:


```
#define FS_READ_PARAMS(fp, fd, flag, size) fp, fd, flag, size
```

The parameters defined by the macro correspond to the event parameters recorded in the trace buffer. You can tailor any parameter fields.

If this parameter is left empty, events of this type are not traced:

```
#define FS_READ_PARAMS(fp, fd, flag, size) // Do not trace file reading events
```
 4. Insert the code stub in a proper position. The definition specification is `LOS_TRACE(#TYPE#, #TYPE#_PARAMS(IDENTITY, parma1...))`.


```
LOS_TRACE(FS_READ, fp, fd, flag, size); // Code stub for reading files. The input parameter
following #TYPE# is the input parameter of the FS_READ_PARAMS function as described in 3.
```
3. The preset trace events and parameters of Huawei LiteOS can be tailored by using the preceding method. For details about the parameters, see `kernel\include\los_trace.h`.
4. For the trace mask event filtering API `LOS_TraceEventMaskSet(UINT32 mask)`, only the upper 28 bits of the input parameter mask (corresponding to the enable bit of a module in `LOS_TRACE_MASK`) take effect. This API is used only for module filtering and does not support fine-grained filtering of a specific event. For example, `LOS_TraceEventMaskSet(0x202)` indicates that the effective mask is `0x200` (`TRACE_QUE_FLAG`) and all events of the `QUE` module are collected. The recommended method is as follows:


```
LOS_TraceEventMaskSet(TRACE_EVENT_FLAG | TRACE_MUX_FLAG | TRACE_SEM_FLAG |
TRACE_QUE_FLAG);
```
5. If only simple instrumentation events need to be filtered, set **Trace Mask** to **TRACE_MAX_FLAG**.

6. The trace buffer size is limited. When the trace buffer is full, events are overwritten. You can identify the latest events based on CurEvtIndex printed in LOS_TraceRecordDump.

Trace Error Codes

To quickly locate the cause of the trace operation failure, you need to return the corresponding error code when you initialize or start the trace. For other APIs that do not return any value, such as the APIs for stopping trace tasks, clearing trace data, and dumping trace data, the error cause is that the trace is invalid, and the system directly displays the error cause.

No	Definition	Error Code	Description	Reference Solution
1	LOS_ERRNO_TRACE_ERROR_STATUS	0x02001400	The status is incorrect when trace is initialized or started.	• Delete duplicate trace initializations. • Initialize the trace before starting it.
2	LOS_ERRNO_TRACE_NO_MEMORY	0x02001401	The system fails to allocate a buffer when initializing trace.	Two solutions are available: • Set LOSCFG_TRACE_BUFFER_SIZE to reduce the size of the trace buffer. You can change the size by choosing the menuconfig menu items: Kernel > Enable Extend Kernel > Enable Trace Feature > Trace work mode (Offline mode) > Trace record buffer size. • Set LOSCFG_BASE_CORE_TSK_LIMIT to reduce the maximum number of tasks. You can change the maximum number of tasks by choosing the menuconfig menu items: Kernel > Basic Config > Task Max Task Number.

No.	Definition	Error Code	Description	Reference Solution
3	LOS_ERRNO_TRACE_BUF_TOO_SMALL	0x02001402	When trace is initialized, the buffer size is set to a small value.	Set LOSCFG_TRACE_BUFFER_SIZE to increase the size of the trace buffer by choosing Kernel > Enable Extend Kernel > Enable Trace Feature > Trace work mode (Offline mode) > Trace record buffer size .
4	LOS_ERRNO_TRACE_BUF_IS_NULL	0x02001403	The trace is initialized in offline mode, the dynamic memory is disabled, and the specified buffer is NULL.	The following solutions are available: • Input a valid buffer address when the trace is initialized. • Enable dynamic memory. • Use the online mode.

NOTICE

For details about the error code definition, see [Error Codes](#). Bits 8–15 belong to the trace module, and the value is 0x14.

Development Procedure

The typical development process of the trace module is as follows:

1. Open the menu and choose **Kernel > Enable Extend Kernel > Enable Trace Feature** to configure the trace module.

Configuration Item	Description	Value Range	Default Value	Dependency
LOSCFG_KERNEL_TRACE	Enables/Disables the trace module tailoring	YES/NO	NO	LOSCFG_KERNEL_EXTKERNEL
LOSCFG_RECORDER_MODE_OFFLINE	Trace works in offline mode.	YES/NO	YES	LOSCFG_KERNEL_TRACE
LOSCFG_RECORDER_MODE_ONLINE	Trace works in online mode.	YES/NO	NO	LOSCFG_KERNEL_TRACE

Configuration Item	Description	Value Range	Default Value	Dependency
LOSCFG_TRACE_CLIENT_INTERACT	Determines whether to enable interaction with the Trace IDE (Huawei LiteOS Studio), including data visualization and process control.	YES/NO	NO	LOSCFG_KERNEL_TRACE
LOSCFG_TRACE_PIPELINE_SERIAL	Selects a serial port as the IDE data visualization channel.	YES/NO	YES	LOSCFG_TRACE_CLIENT_INTERACT
LOSCFG_TRACE_CONTROL_VIA_SHELL	Selects shell as the IDE process control mode.	YES/NO	YES	LOSCFG_TRACE_CLIENT_INTERACT && LOSCFG_SHELL
LOSCFG_TRACE_CONTROL_AGENT	Selects agent as the IDE process control mode.	YES/NO	NO	LOSCFG_TRACE_CLIENT_INTERACT
LOSCFG_TRACE_NO_CONTROL	Disables the IDE process control.	YES/NO	NO	LOSCFG_TRACE_CLIENT_INTERACT
LOSCFG_TRACE_MSG_EXTEND	Records more extended information about the system.	YES/NO	NO	LOSCFG_KERNEL_TRACE
LOSCFG_TRACE_FRAME_CORE_MSG	Records the CPU ID, interruption status, and lock task status.	YES/NO	NO	LOSCFG_TRACE_MSG_EXTEND
LOSCFG_TRACE_FRAME_EVENT_COUNT	Sequence number of an event	YES/NO	NO	LOSCFG_TRACE_MSG_EXTEND
LOSCFG_TRACE_FRAME_MAX_PARAMS	Configuring the maximum number of parameters for recording events	INT	3	LOSCFG_KERNEL_TRACE
LOSCFG_TRACE_BUFFER_SIZE	Configuring the trace buffer size	INT	2048	LOSCFG_RECORDER_MODE_OFFLINE

2. (Optional) Preset event parameters and event stubs (or use the default event parameter settings and event stubs).

3. (Optional) Clear the `LOS_TraceReset` buffer after trace is stopped by calling the `LOS_TraceStop` API. (Trace is started by default.)
4. (Optional) Set the mask of the event to be traced by calling the `LOS_TraceEventMaskSet` API. (The default event mask enables only interrupt and task switching.)
5. Call the `LOS_TraceStart` API at the start point of the code where the event needs to be recorded.
6. Call the `LOS_TraceStop` API at the end point of the code where the event needs to be recorded.
7. Call the `LOS_TraceRecordDump` API to output the data in the buffer. (The input parameter of the function is of the Boolean type. `FALSE` indicates that the data is output in the formatted format, and `TRUE` indicates that the data is output to a Windows client.)

NOTE

The APIs in steps 4 to 7 are encapsulated with the corresponding shell commands. The mapping is as follows. For details about the commands, see [Trace Commands](#):

- `LOS_TraceEventMaskSet` — `trace_mask`
- `LOS_TraceStart` — `trace_start`
- `LOS_TraceStop` — `trace_stop`
- `LOS_TraceRecordDump` — `trace_dump`

Platform Differences

None.

4.6.3 Precautions

You are advised to disable the trace function before product release because the trace function affects the system performance and you need to know the events that occur in the system only during product development.

4.6.4 Programming Example

Example Description

The following functions are implemented:

1. Create a task for the trace test.
2. Set the event mask.
3. Start the trace.
4. Stop the trace.
5. Output trace data in a required format.

Example Code

Prerequisite: The trace module has been configured in the **menuconfig** menu.

The code is as follows:

```
#include "los_trace.h"
```

```

UINT32 g_traceTestTaskId;
VOID Example_Trace(VOID)
{
    UINT32 ret;

    LOS_TaskDelay(10);

    /* Start trace */
    ret = LOS_TraceStart();
    if (ret != LOS_OK) {
        dprintf("trace start error\n");
        return;
    }

    /* Event that triggers task switching*/
    LOS_TaskDelay(1);

    LOS_TaskDelay(1);

    LOS_TaskDelay(1);

    /* Stop trace */
    LOS_TraceStop();

    LOS_TraceRecordDump(FALSE);
}

UINT32 Example_Trace_test(VOID)
{
    UINT32 ret;
    TSK_INIT_PARAM_S traceTestTask;

    /* Create a task for testing trace. */
    memset(&traceTestTask, 0, sizeof(TSK_INIT_PARAM_S));
    traceTestTask.pfnTaskEntry = (TSK_ENTRY_FUNC)Example_Trace;
    traceTestTask.pcName = "TestTraceTsk"; /* Test task name */
    traceTestTask.uwStackSize = 0x800;
    traceTestTask.usTaskPrio = 5;
    traceTestTask.uwResved = LOS_TASK_STATUS_DETACHED;
    ret = LOS_TaskCreate(&g_traceTestTaskId, &traceTestTask);
    if(ret != LOS_OK){
        dprintf("TraceTestTask create failed .\n");
        return LOS_NOK;
    }

    /* By default, the trace function is enabled. Therefore, you can disable the trace function, clear the buffer,
    and then restart the trace function. */
    LOS_TraceStop();
    LOS_TraceReset();

    /* Enable the event recording function of the task module. */
    LOS_TraceEventMaskSet(TRACE_TASK_FLAG);

    return LOS_OK;
}

```

Verification

The verification result is as follows:

```

*****TraceInfo begin*****
clockFreq = 50000000
CurEvtIndex = 7

```

Index	Time(cycles)	EventType	CurTask	Identity	params
0	0x366d5e88	0x45	0x1	0x0	0x1f 0x4 0x0
1	0x366d74ae	0x45	0x0	0x1	0x0 0x8 0x1f
2	0x36940da6	0x45	0x1	0xc	0x1f 0x4 0x9
3	0x3694337c	0x45	0xc	0x1	0x9 0x8 0x1f
4	0x36eea56e	0x45	0x1	0xc	0x1f 0x4 0x9

```

5  0x36eec810  0x45  0xc  0x1  0x9  0x8  0x1f
6  0x3706f804  0x45  0x1  0x0  0x1f  0x4  0x0
7  0x37070e59  0x45  0x0  0x1  0x0  0x8  0x1f
*****TraceInfo end*****

```

The output event information includes the event time, event type, task where the event occurs, event operation object, and other parameters.

- **Time(cycles)**: indicates the time when an event occurs. The value is the number of cycles of the clock.
- **EventType**: For details about the event type, see enum `LOS_TRACE_TYPE` in the header file **los_trace.h**.
- **CurTask**: indicates the current task. The value is a task ID.
- **Identity**: indicates the main object of the event operation. You can view `#TYPE#_PARAMS` in the header file **los_trace.h**.
- **params**: event parameter. For details, see `#TYPE#_PARAMS` in the header file **los_trace.h**.

The following uses the output item whose sequence number is 0 as an example:

Index	Time(cycles)	EventType	CurTask	Identity	params
0	0x366d5e88	0x45	0x1	0x0	0x1f 0x4 0x0

- **Time(cycles)** can be converted into time. The conversion formula is Value of **cycles**/Value of **clockFreq**. The unit is s.
- 0x45 indicates the `TASK_SWITCH` event, which is the task switching event. The ID of the running task is 0x1.
- For the meanings of **Identity** and **params**, see the definition of the `TASK_SWITCH_PARAMS` macro:

```
#define TASK_SWITCH_PARAMS(taskId, oldPriority, oldTaskStatus, newPriority, newTaskStatus) \
taskId, oldPriority, oldTaskStatus, newPriority, newTaskStatus
```

Because of `#TYPE#_PARAMS(IDENTITY, parma1...)` **Identity**, ..., **Identity** is set to **taskId (0x0)**, and the first parameter is set to **oldPriority (0x1f)**....

NOTE

The number of params is specified by **Enable Trace Feature > Record max params** in menuconfig. The default value is **3**. The parameters that exceed the value are not recorded. You need to set the parameter properly.

In conclusion, the task is switched from 0x1 to 0x0. The priority of the 0x1 task is 0x1f, the status is 0x4, and the priority of the 0x0 task is 0x0.

Complete Code

sample_trace.zip

4.7 LMS

4.7.1 Overview

Basic Concepts

Lite Memory Sanitizer (LMS) is a commissioning tool used to check the validity of memory operations in real time. The LMS can check buffer overflow, use after

free, and wild pointer in real time, and can notify the operating system immediately when an exception occurs. With the help of locating methods such as Backtrace, the LMS can locate the code line where the memory problem occurs. This greatly improves the efficiency of locating memory problems.

Operation Mechanism

The LMS uses the shadow memory mapping to mark the status of the system memory. There are three states: readable and writable, unreadable and unwritable, and released. The shadow memory is stored in the tail of the memory pool.

- After the memory is applied for from the heap, the shadow memory in the data area is set to readable and writable, and the shadow memory in the head node area is set to the unreadable and unwritable.
- When the memory is released from the heap, the shadow memory of the released memory is set to the released state.
- During code compilation, a check function is inserted before the read/write instruction in the code to check the validity of the address. Generally, the status value of the shadow memory that accesses the memory is checked. If the shadow memory is unreadable and unwritable, an overflow error is reported. If the shadow memory is released, a Use-After-Free error is reported.
- When the memory is released, the status value of the shadow memory at the released address is checked. If the shadow memory is not readable or writable, a double free error is reported.

Functions

The LMS module of Huawei LiteOS provides the following functions. For details about the APIs, see the API reference.

1. Supports check of multiple memory pools.
2. Supports check of all LiteOS memory algorithms, including bestfit, bestfit-little, slab, and slab extend.
3. Supports check of the memory allocated by **LOS_MemAlloc**, **LOS_MemAllocAlign**, and **LOS_MemRealloc**.
4. Supports check of secure function access.
5. Supports the check of high-frequency functions accessed by libc, including **memset**, **memcpy**, **memmove**, **strcat**, **strcpy**, **strncat**, and **strncpy**.
6. Supports the full llibc check. It can be enabled by setting **LOSCFG_LMS_LIBC_FULL_CHECK** in menuconfig of the LMS.

The functions that can be dynamically set are as follows:

Table 4-8 API list

Function	API	Description
Adding a specified memory pool to be checked	LOS_LmsAddC heckPool	Adds the address range of a memory pool to the LMS check linked list. LMS performs a validity check when the accessed address is within the linked list. In addition, LOS_MemInit calls this API to add the initialized memory pool to the LMS check linked list by default.
Deleting a memory pool from the LMS check linked list	LOS_LmsDelC heckPool	Cancels the validity check on the specified memory pool.
Protecting a specified memory segment	LOS_LmsAddr Protect	Locks a memory segment to prevent it from being read or written. Once the locked memory segment is accessed, an error will be reported.
Disabling protection of a specified memory segment	LOS_LmsAddr DisableProtect	Unlocks a memory segment to make it readable and writable.

7. The following functions can be tailored:

Run the **make menuconfig** command, and choose **Debug ---> Enable Lite Memory Sanitizer** to configure and tailor the LMS.

Configuration Item	Description	Value Range	Default Value	Dependency
LOSCFG_KERNEL_LMS	Whether to enable LMS	YES/NO	NO	NA
LOSCFG_LMS_MAX_RECORD_POOL_NUM	Maximum number of memory pools that can be checked by LMS	INT	10	LOSCFG_KERNEL_LMS
LOSCFG_LMS_LOAD_CHECK	Whether to enable LMS read check	YES/NO	YES	LOSCFG_KERNEL_LMS
LOSCFG_LMS_STORE_CHECK	Whether to enable LMS write check	YES/NO	YES	LOSCFG_KERNEL_LMS
LOSCFG_LMS_CHECK_STRICT	Whether to enable LMS byte-by-byte check	YES/NO	NO	LOSCFG_KERNEL_LMS

Configuration Item	Description	Value Range	Default Value	Dependency
LOSCFG_LMS_LIBC_FULL_CHECK	Whether to enable LMS full libc check	YES/NO	NO	LOSCFG_KERNEL_LMS

NOTE

1. After full libc check is enabled, the size of the text segment in the image generated by libc increases by about 17%.

```
SizeTable sorted by text+data
liba_file
total
libc.a
```

```
SizeTable sorted by text+data
liba_file
total
libc.a
```

2. You can configure read-only check or write check to improve check efficiency.
3. You can enable the configuration macro **LOSCFG_LMS_CHECK_STRICT** to perform byte-by-byte validity check on the memory address. However, the check performance deteriorates. By default, the system checks the first and last bytes.

4.7.2 Development Guidelines

Application Scenario

- The system memory is suspected to be faulty and the fault needs to be located.
- During development, you need to check whether memory problems exist.

Development Procedure

Step 1 Add the LMS compilation check option `-fsanitize=kernel-address` to the **Makefile** file of the detected module.

Step 2 Use **menuconfig** to configure the compiler not to be optimized.

LOSCFG_COMPILER_OPTIMIZE_NONE=y [Compiler > Optimize Option > Optimize None]

Step 3 To output the Backtrace information, enable the following configuration item:

LOSCFG_BACKTRACE=y [Debug > Enable Backtrace]

Step 4 Enable the LMS configuration item.

LOSCFG_KERNEL_LMS=y [Debug > Enable Lite Memory Sanitizer]

Step 5 Recompile the code and check the serial port output. If a memory problem is detected, the detection result is displayed. The detection information displayed by the LMS is similar to that shown in the following figure.

```

[ERR] ***** Kernel Address Sanitizer Error Detected Start *****
[ERR] Heap buffer overflow error detected 1
[ERR] Illegal WRITE address at: [0x1206a8] 2
[ERR] Shadow memory address: [0xf20252 : 4] Shadow memory value: [2] 3
runTask->taskName = IT_TST_INI
runTask->taskId = 3 4
fp:0x0010cb3c
*****backtrace begin*****
5
traceback 0 -- lr = 0x218bc    fp = 0x10cb4c
traceback 1 -- lr = 0x2364c    fp = 0x10cb7c
traceback 2 -- lr = 0x237e8    fp = 0x10cb8c
traceback 3 -- lr = 0x21ea4    fp = 0x10cba4
traceback 4 -- lr = 0x139b4    fp = 0x10cbb4
traceback 5 -- lr = 0x12c1c    fp = 0x10cd1c
traceback 6 -- lr = 0x12b64    fp = 0x10cd2c
traceback 7 -- lr = 0x12a20    fp = 0x10cd4c
traceback 8 -- lr = 0x13a04    fp = 0x10cd64
traceback 9 -- lr = 0x1393c    fp = 0x10cd6c
traceback 10 -- lr = 0x11bf0    fp = 0x10cd74
traceback 11 -- lr = 0x11c30    fp = 0x10cd7c
traceback 12 -- lr = 0x11f04    fp = 0x10cd9c
traceback 13 -- lr = 0x1b200    fp = 0x10cdb4
traceback 14 -- lr = 0x39a30    fp = 0xb0b0b0b
*****backtrace end*****

[LMS] Dump info around address [0x 1206a8]:

```

[0x120628]:	0b 0b 0b 0b 30 9a 03 00	[0xf2024a 4]:	0
[0x120630]:	00 00 00 00 cb f9 ed ff	[0xf2024b 0]:	2
[0x120638]:	03 00 00 00 20 fe 11 00	[0xf2024b 4]:	2
[0x120640]:	74 00 00 80 00 00 00 00	[0xf2024c 0]:	2
[0x120648]:	00 00 00 00 00 00 00 00	[0xf2024c 4]:	0
[0x120650]:	00 00 00 00 00 00 00 00	[0xf2024d 0]:	0
[0x120658]:	00 00 00 00 00 00 00 00	[0xf2024d 4]:	0
[0x120660]:	00 00 00 00 00 00 00 00	[0xf2024e 0]:	0
[0x120668]:	00 00 00 00 00 00 00 00	[0xf2024e 4]:	0
[0x120670]:	00 00 00 00 00 00 00 00	[0xf2024f 0]:	0
[0x120678]:	00 00 00 00 00 00 00 00	[0xf2024f 4]:	0
[0x120680]:	00 00 00 00 00 00 00 00	[0xf20250 0]:	0
[0x120688]:	00 00 00 00 00 00 00 00	[0xf20250 4]:	0
[0x120690]:	00 00 00 00 00 00 00 00	[0xf20251 0]:	0
[0x120698]:	00 00 00 00 00 00 00 00	[0xf20251 4]:	0
[0x1206a0]:	00 00 00 00 00 00 00 00	[0xf20252 0]:	0
[0x1206a8]:	[30] d4 0f 00 30 d4 0f 00	[0xf20252 4]:	[2]
[0x1206b0]:	34 06 12 00 48 d8 df 00	[0xf20253 0]:	2
[0x1206b8]:	00 00 00 00 00 00 00 00	[0xf20253 4]:	3

LMS detection information includes the following types of information:

1. Detected memory problem. In the example, the problem is heap buffer overflow.
2. Address of the memory where the problem occurs. In the example, the address is 0x1206a8, and the operation type is invalid write.
3. Shadow memory corresponding to the faulty memory address and the value of the shadow memory. In the example, the shadow memory address is 0xf20252, the byte offset is 4, and the value of the shadow memory address is 2. The value **2** indicates that the shadow memory address cannot be read or written (**0** indicates that the shadow memory address can be read and written, and **3** indicates that the shadow memory is released).
4. Running task. In the example, the task name is IT_TST_INI and the task ID is 3.
5. Backtrace stack (**backtrace begin** to **backtrace end**) when the problem occurs. The calling sequence is tracetrack 14 -> tracetrack 13 ->...-> tracetrack 0. The address of the function called at each layer is the value of **lr**. You can locate the specific function by viewing the .asm file.
6. Memory values before and after the faulty memory address and the corresponding shadow memory value.

----End

Platform Differences

None.

4.7.3 Precautions

Unsupported items

- The LMS does not support overflow detection of the stack memory and global variables, which requires the support of the compiler.

Others

- The LMS is a commissioning feature. When the product is released, the tailoring switch of the LMS module LOSCFG_KERNEL_LMS needs to be disabled, the LMS compilation check option -fsanitize=kernel-address needs to be deleted, and other configuration items enabled for enabling the LMS feature need to be restored. For details, see [Development Procedure](#).
- If a large number of read and write operations are performed on the heap memory of the detected task, increase the size of the task stack.
- The LMS compilation check option -fsanitize=kernel-address does not need to be added to the memory and LMS modules.

4.8 Perf

4.8.1 Overview

Basic Concepts

Perf is a performance analysis tool. It uses the performance monitoring unit (PMU) to count sampling events and collect context information for the statistics on the hot spots and hot paths of the algorithm.

Operation Mechanism

According to the event sampling principles, when an event occurs, the corresponding event counter overflows and an interrupt occurs. The interrupt processing function records the event information, including the current PC, ID of the running task, and call stack.

4.8.2 Development Guidelines

Application Scenario

Use perf to locate performance bottlenecks and hot spot code to optimize system performance more efficiently.

Functions

The perf module of Huawei LiteOS provides the following functions. For details about the APIs, see the API reference.

Table 4-9 Functions

Function Category	API	Description
Initializing perf	LOS_PerfInit	Initializes the PMU and sampling buffer.
Enabling/ Disabling perf sampling	LOS_PerfStart	Starts the sampling.
	LOS_PerfStop	Stops the sampling.
Configuring a perf sampling event	LOS_PerfConfig	Configures the type and period of the sampling event.
Reading sampling data	LOS_PerfData Read	Reads the sampling data to a specified address.
Registering the hook function of the sampling data buffer	LOS_PerfNotifyHookReg	Registers the processing hook where the threshold of the registration buffer is reached
	LOS_PerfFlushHookReg	Registers the hook for flushing the cache in the buffer.
Deinitializing perf	LOS_PerfDeinit	Deinitializes the PMU.

 CAUTION

The sampling data buffer is a ring buffer, and only areas that have been read in the buffer can be overwritten.

Perf Error Codes

To quickly locate the cause of the perf operation failure, the perf initialization and sampling configuration APIs need to return the corresponding error codes. The following table describes the details.

No.	Definition	Error Code	Description	Reference Solution
1	LOS_ERRNO_PERF_STATUS_INVALID	0x02002000	The status is incorrect when the perf is initialized or the sampling event is configured.	Do not initialize the perf module repeatedly, and configure sampling events only when the perf module is stopped.
2	LOS_ERRNO_PERF_HW_INIT_ERROR	0x02002001	The hardware PMU fails to be initialized during the perf module initialization.	Check that the platform has the hardware PMU and that the interrupt number of the PMU is correct.
3	LOS_ERRNO_PERF_TIMED_INIT_ERROR	0x02002002	The high-precision Hrtimer PMU fails to be initialized during the perf module initialization.	Check that the platform has a high-precision timer unit.
4	LOS_ERRNO_PERF_SW_INIT_ERROR	0x02002003	The software PMU fails to be initialized during the perf module initialization.	Check that software PMU is enabled.
5	LOS_ERRNO_PERF_BUF_ERROR	0x02002004	The buffer fails to be initialized during the perf module initialization.	The configured perf buffer size is excessively large or small. Change the value of LOS_PERF_BUFFER_SIZE in los_config.h .

No	Definition	Error Code	Description	Reference Solution
6	LOS_ERRNO_PERF_INVALID_PMU	0x02002005	The PMU that is configured with the sampling event is not registered.	Each PMU can be configured by using menuconfig . Ensure that the corresponding PMU is enabled.
7	LOS_ERRNO_PERF_PMU_CONFIG_ERROR	0x02002006	The event parameter for configuring the sampling event is invalid.	Check that the event type, event ID, and event period in LOS_PerfConfig are valid. Event types are provided in enum PerfEventType, including: • Hardware events The event IDs are provided in enum PmuHwId. Period range: [0x101,0xFFFFFFFF] Note: In the ARMv8-M architecture, the cycle counter period range is [0x101,0xFFFFFFFF], and the event counter period range is [0x101,0xFFFF]. • Software events The event IDs are provided in enum PmuSwId. Period: >0 • Periodic event The event IDs are provided in enum PmuTimedId. Period: >0
8	LOS_ERRNO_PERF_CONFIG_NULL	0x02002007	The input parameter of the sampling event is set to a null pointer.	Do not set LOS_PerfConfig input parameters to a null pointer.

No	Definition	Error Code	Description	Reference Solution
9	LOS_ERRNO_PERF_CONFIG_TASK_FILTER_ERROR	0x02002008	Failed to configure a task whitelist.	Check whether the memcpy_s input parameter is correct when the whitelist is copied.
10	LOS_ERRNO_PERF_INIT_NO_MEMORY	0x02002009	The memory for initializing the perf is insufficient.	Check the remaining system memory or disable the per-task count function to reduce memory usage.

Development Procedure

The typical development process of the perf module is as follows:

1. Open the menu and choose **Kernel > Enable Extend Kernel > Enable Perf Feature** to configure the perf module.

Configuration Item	Description	Value Range	Default Value	Dependency
LOSCFG_KERNEL_PERF	Determines whether to enable the perf module.	YES/NO	NO	LOSCFG_KERNEL_EXTKERNEL
LOSCFG_PERF_CALC_TIME_BY_TICK	Determines whether the perf module uses tick as the timing unit.	YES/NO	YES	LOSCFG_KERNEL_PERF
LOSCFG_PERF_CALC_TIME_BY_CYCLE	Determines whether the perf module uses system cycle as the timing unit.	YES/NO	NO	LOSCFG_KERNEL_PERF
LOSCFG_PERF_HW_PMU	Determines whether to enable the hardware PMU sampling event.	YES/NO	NO	LOSCFG_KERNEL_PERF && LOSCFG_ARCH_PMU
LOSCFG_PERF_TIMER_PMU	Determines whether to enable the high-precision periodic PMU sampling event.	YES/NO	NO	LOSCFG_KERNEL_PERF && LOSCFG_COMPAT_LINUX_HRTIMER

Configuration Item	Description	Value Range	Default Value	Dependency
LOSCFG_PERF_SW_PMU	Determines whether to enable the software PMU sampling event.	YES/NO	YES	LOSCFG_KERNEL_PERF
LOSCFG_KERNEL_PERF_PER_TASK	Determines whether to enable the perf per-task count mode.	YES/NO	NO	LOSCFG_KERNEL_PERF
LOSCFG_KERNEL_PERF_SEPERATE_IRQ	Determines whether to enable the perf separated interrupt count mode.	YES/NO	NO	LOSCFG_KERNEL_PERF && LOSCFG_ARCH_INTERRUPT_TAKEOVER

2. Call the `LOS_PerfConfig` API to configure the events to be sampled.
Perf provides two modes and three types of event configurations:
 - Two modes: counting mode (count only the number of event occurrences) and sampling mode (collect context information such as task IDs, PC, and backtrace)
 - Three types: CPU hardware events (such as cycle, branch, icache, and dcache), OS software events (such as task switch, mux pend, and irq), and high-precision periodic events (cpu clock)
3. Call the `LOS_PerfStart` API at the start point of the code where sampling is required.
4. Call the `LOS_PerfStop` API at the end point of the code where sampling is required.
5. Call the `LOS_PerfDataRead` API to read the sampling data and use LiteOS Studio to parse the data. (Click the **Performance Analysis** tab page in the debug tool.)

Platform Differences

The sampling of hardware events depends on the hardware PMU and the platforms are different. Currently, Huawei LiteOS perf supports PMUs of the ARMv7, ARMv8, RISC-V, XEA2, and LingLong architectures.

4.8.3 Precautions

The perf module is used for performance commissioning, which may affect system performance. You are advised to disable `LOSCFG_KERNEL_PERF` of the perf module before product release.

4.8.4 Programming Example

Description

The following functions are implemented:

1. Configure a sampling event.
2. Start the perf.
3. Execute the algorithm to be counted.
4. Stop the perf.
5. Output the statistical result.

Example Code

Prerequisite: The perf module has been configured in the **menuconfig** menu.

The code is as follows:

```
#include "los_perf.h"
#include "los_memory.h"

#define TEST_PERF_BUFFER_SIZE 1024
#define OUTPUT_BUFFER_SIZE 0x200

#ifdef LOSCFG_KERNEL_MEM_ALLOC
STATIC CHAR g_buffer[TEST_PERF_BUFFER_SIZE] = {0};
#endif

STATIC VOID OsPrintBuff(const CHAR *buf, UINT32 num)
{
    UINT32 i;
    dprintf("num: ");
    for (i = 0; i < num; i++) {
        dprintf(" %02u", i);
    }
    dprintf("\n");
    dprintf("hex: ");
    for (i = 0; i < num; i++) {
        dprintf(" %02x", buf[i]);
    }
    dprintf("\n");
}

VOID PerfTestHWEvent(VOID)
{
    UINT32 ret;
    CHAR buf[OUTPUT_BUFFER_SIZE] = {0};
    UINT32 len;
    CHAR *buffer = NULL;

    PerfConfigAttr attr1 = {
        .eventsCfg = {
            .type = PERF_EVENT_TYPE_HW,
            .events = {
                [0] = {PERF_COUNT_HW_CPU_CYCLES, 0xFFFF},
                [1] = {PERF_COUNT_HW_BRANCH_INSTRUCTIONS, 0xFFFFFFFF0},
            },
            .eventsNr = 2,
            .predivided = 1, /* cycle counter increase every 64 cycles */
        },
        .taskIds = {0},
        .taskIdsNr = 0,
        .needSample = 0,
        .sampleType = PERF_RECORD_IP | PERF_RECORD_CALLCHAIN,
```

```

};
#ifdef LOSCFG_KERNEL_MEM_ALLOC
    buffer = (CHAR *)LOS_MemAlloc(m_aucSysMem1, TEST_PERF_BUFFER_SIZE);
#else
    buffer = g_buffer;
#endif

ret = LOS_PerfInit(buffer, TEST_PERF_BUFFER_SIZE);
if (ret != LOS_OK) {
    PRINT_ERR("perf init error %u\n", ret);
    return;
}

ret = LOS_PerfConfig(&attr1);
if (ret != LOS_OK) {
    PRINT_ERR("perf config error %u\n", ret);
    return;
}

PRINTK("-----count mode-----\n");
LOS_PerfStart(0);
test(); /* code snippets concerned by users */
LOS_PerfStop();

PRINTK("-----sample mode----- \n");
attr1.needSample = 1;
LOS_PerfConfig(&attr1);
LOS_PerfStart(2);
test(); /* code snippets concerned by users */
LOS_PerfStop();

len = LOS_PerfDataRead(buf, OUTPUT_BUFFER_SIZE); /* get sample data */
OsPrintBuff(buf, len); /* print data */
memset(buf, 0, sizeof(buf));
#ifdef LOSCFG_KERNEL_MEM_ALLOC
    (VOID)LOS_MemFree(m_aucSysMem1, buffer);
#endif
LOS_PerfDeinit();
}

```

Verification

The verification result is as follows:

```

-----count mode-----
[EMG] perf count [tid 0 / Swt_Task]:
[EMG] [cycles] eventType: 0xff count: 0
[EMG] [branches] eventType: 0xc count: 0
[EMG] perf count [tid 1 / IdleCore000]:
[EMG] [cycles] eventType: 0xff count: 0
[EMG] [branches] eventType: 0xc count: 0
[EMG] perf count [tid 2 / app_Task]:
[EMG] [cycles] eventType: 0xff count: 40043481344
[EMG] [branches] eventType: 0xc count: 10002153167
[EMG] perf count [tid 3 / Swt_Task]:
[EMG] [cycles] eventType: 0xff count: 0
[EMG] [branches] eventType: 0xc count: 0
[EMG] perf count [tid 4 / IdleCore000]:
[EMG] [cycles] eventType: 0xff count: 100718720
[EMG] [branches] eventType: 0xc count: 5182967
[EMG] perf count [tid 5 / SerialShellTask]:
[EMG] [cycles] eventType: 0xff count: 0
[EMG] [branches] eventType: 0xc count: 0
[EMG] perf count [tid 6 / SerialEntryTask]:
[EMG] [cycles] eventType: 0xff count: 0
[EMG] [branches] eventType: 0xc count: 0
[EMG] perf count [irq@core 0]:
[EMG] [cycles] eventType: 0xff count: 891017
[EMG] [branches] eventType: 0xc count: 2948096

```

```
[EMG] perf count [irq@core 1]:
[EMG] [cycles] eventType: 0xff count: 889686
[EMG] [branches] eventType: 0xc count: 2948154
[EMG] time used: 44668000(us)
-----sample mode-----
[EMG] time used: 44673000(us)
num: 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
32 33 34 35 36 37 38 39 40 ...
hex: 00 ef ef ef 00 00 00 00 14 00 00 00 60 00 00 00 02 00 00 00 90 2d 20 80 02 00 00 00 d4 5f 20 80 54
55 21 80 90 2d 20 80 02 ...
```

In count mode, the following information is displayed after perf is stopped:

- After the per-task count function is enabled, the header information of each task is printed first. The subsequent event count is the count of this task.
perf count [tid 2 / app_Task]
Task ID: **2**, and task name: **app_Task**
- After the per-task count function is disabled, the header information of each core is printed first. The subsequent event count is the count of this core.
perf count [core 0]
Core ID: **core 0**
- After the separated interrupt count function is disabled, the interrupt header information of each core is printed first. The subsequent event count is the interrupt count of this core.
perf count [irq@core 0]
Core ID: **core 0**
- Event count information:
[cycles] eventType: 0xff count: 40043481344
Event name: **cycles**, event type: **0xff**, and number of event occurrences: **40043481344**

NOTE

When the sampled event is a hardware PMU event, the displayed event type is the actual hardware event ID, instead of the abstract type defined in **enum PmuHwid**. For details about hardware event IDs, see the related architecture manuals.

- For the sample mode, the system records the sampled data to the specified memory, that is, the input parameter of `LOS_PerfInit`. You can print the address of the memory, export the memory through the JTAG API, and use the LiteOS Studio offline tool to parse the memory. You can also read data to a specified address for viewing or further processing by calling `LOS_PerfDataRead`. In the example, **OsPrintBuff** is the test API, which displays the read sampled data by byte, in which **num** indicates the byte number, and **hex** indicates the value of the byte.

CAUTION

1. In the RISC-V architecture, to use the sampling mode, you need to call the `OsPerfCheckEventCount()` function in the timing interrupt processing function. Generally, the tick interrupt processing function is used.
2. In perf separated interrupt mode, PMU interrupts are not counted separately. In addition, the count may be different from the expected value because the Timed PMU also depends on interrupts. If the Timed PMU is required, disable the perf separated interrupt mode.

Complete Code

sample_perf.zip

4.9 C++ Support

4.9.1 Overview

Basic Concepts

C++ is one of the most widely used programming languages. It is an object-oriented programming language with features including classes, encapsulation, and reloading.

Operation Mechanism

STL is a collection of some "containers", as well as a collection of algorithms and other components. The objective is to develop a standardized component that can be used without developing again, directly using an off-the-shelf component.

4.9.2 Development Guidelines

Functions

Function Category	API	Description
Initialize C++ constructors	LOS_CppSystemInit	Initializes C++ constructors.

NOTE

This function expects three parameters:

- The first parameter specifies the start address of the **init_array** segment.
- The first parameter specifies the end address of the **init_array** segment.
- The third parameter identifies the scenario where C++ is called, including **BEFORE_SCATTER** (calling C++ at fast startup of scatter loading), **AFTER_SCATTER** (calling C++ at non-fast startup of scatter loading), and **NO_SCATTER** (calling C++ in non-scatter loading or not calling C++ in scatter loading.)

Development Procedure

1. Open the menu and choose **Kernel > Enable Extend Kernel > C++ Support** to enable C++.

Configuration Item	Description	Value Range	Default Value	Dependency
LOSCFG_KERNEL_CPPSUPPORT	Determines whether to enable the C++ feature.	YES/NO	YES	LOSCFG_KERNEL_EXTKERNEL

- Before using the C++ feature, call the `LOS_CppSystemInit` function to initialize the C++ constructor.

In the scatter loading scenario, the related code and data segment loading involved in C++ initialization are different. Therefore, the calling of the C++ initialization function `LOS_CppSystemInit` varies depending on whether scatter loading is enabled. The initialization of the constructor is stored in the `init_array` segment. The segment range is transferred through the variables `__init_array_start` and `__init_array_end`.

- Scatter loading is disabled**

Before calling C++ code, call the `LOS_CppSystemInit` API with the `NO_SCATTER` parameter.

```
LOS_CppSystemInit((unsigned long)&__init_array_start, (unsigned long)&__init_array_end, NO_SCATTER);
```

- Scatter loading is enabled**

- To call C++ code at fast startup of scatter loading, perform the following operations:

- If C++ code needs to be called at the fast startup phase of scatter loading, first call `LOS_CppSystemInit` with the `BEFORE_SCATTER` parameter.

```
LOS_CppSystemInit((unsigned long)&__init_array_start, (unsigned long)&__init_array_end, BEFORE_SCATTER);
```

- Then, at the non-fast startup phase of scatter loading, call `LOS_CppSystemInit` with the `AFTER_SCATTER` parameter.

```
LOS_CppSystemInit((unsigned long)&__init_array_start, (unsigned long)&__init_array_end, AFTER_SCATTER);
```

- If you do not want to call C++ code at the fast startup phase of scatter loading, perform the following operations:

Call `LOS_CppSystemInit` with the `NO_SCATTER` parameter:

```
LOS_CppSystemInit((unsigned long)&__init_array_start, (unsigned long)&__init_array_end, NO_SCATTER);
```

- Call both C functions and C++ functions.

To call C functions in C++, add the macros contained in C++ to the code as follows:

```
#ifndef __cplusplus
#ifdef __cplusplus
extern "C" {
#endif /* __cplusplus */
#endif /* __cplusplus */
/* code */
...
#ifdef __cplusplus
#ifdef __cplusplus
```



```

}
#endif /* __cplusplus */
#endif /* __cplusplus */

```

4.9.3 Precautions

- The C++ function of Huawei LiteOS is supported only when the compiler is adapted. The C library provided by Huawei LiteOS needs to be used when the compiler compiles and links the C++ code.
- The cout function of C++ depends on the console function.

4.9.4 Programming Example

Example Description

This instance shows how to run C++ code on Huawei LiteOS.

1. Compile the C++ code.
2. Before running the C++ code, call the app_init function to initialize C++ constructors. The scatter loading feature is not enabled. Therefore, you only need to use to call LOS_CppSystemInit with the NO_SCATTER parameter once.
3. Call the compiled C++ code after LOS_CppSystemInit.

Example Code

Prerequisite: C++ support has been enabled on the menu.

The C++ code is as follows:

```

#ifdef __cplusplus
#if __cplusplus
extern "C" {
#endif /* __cplusplus */
#endif /* __cplusplus */

using namespace std;

class TestClass {
public:
    TestClass(int arg);
    ~TestClass(void);
    void PrintTest(void);
    void StringTest(void);
    void MapTest(void);
private:
    int intTest;
    string stringTest;
    map<string, int> mapTest;
};

TestClass::TestClass(int arg)
{
    cout << "TestClass is constructed here, arg = " << arg << endl;
    intTest = arg;
}

TestClass::~~TestClass(void)
{
    cout << "TestClass is destructed" << endl;
}

void TestClass::PrintTest(void)
{
    cout << __FUNCTION__ << " enter" << endl;
    cout << " intTest = " << this->intTest << endl;
}

```

```

}
void TestClass::StringTest(void)
{
    cout << __FUNCTION__ << " enter" << endl;
    string a("Lite");
    string b("OS");
    string c("LiteOS");
    if (a != b) {
        cout << " " << a << " != " << b << endl;
    }
    a += b;
    if (a == c) {
        cout << " " << a << " == " << c << endl;
    }
}
void TestClass::MapTest(void)
{
    cout << __FUNCTION__ << " enter" << endl;
    mapTest.insert(pair<string, int>("Huawei", 1));
    mapTest.insert(pair<string, int>("LiteOS", 2));
    mapTest.insert(pair<string, int>("Open", 3));
    mapTest.insert(pair<string, int>("Source", 4));
    cout << " show map key&value" << endl;
    for (auto &it : mapTest) {
        cout << " " << it.first << " " << it.second << endl;
    }
    mapTest["LiteOS"] = 8; /* 8: new value */
    cout << " change value of \"LiteOS\" key" << endl;
    for (auto &it : mapTest) {
        cout << " " << it.first << " " << it.second << endl;
    }
}

void CppTestEntry(void)
{
    cout << "LiteOS cpp sample start" << endl;
    TestClass test(123);
    test.PrintTest();
    test.StringTest();
    test.MapTest();
    cout << "LiteOS cpp sample stop" << endl;
}

#ifdef __cplusplus
#if __cplusplus
}
#endif /* __cplusplus */
#endif /* __cplusplus */

```

After the C++ constructor is initialized in **app_init**, call the C++ code. The code is as follows:

```

void app_init(void)
{
    .....
    /* Initialize the C++ constructor. */
    LOS_CppSystemInit((UINT32)&__init_array_start, (UINT32)&__init_array_end, NO_SCATTER);
    /* Call the C++ code. */
    CppTestEntry();
    .....
}

```

Verification

Run the CppTestEntry() function. The verification result is as follows:

```

LiteOS cpp sample start
TestClass is constructed here, arg = 123
PrintTest enter

```

```
intTest = 123
StringTest enter
Lite != OS
LiteOS == LiteOS
MapTest enter
show map key&value
Huawei 1
LiteOS 2
Open 3
Source 4
change value of "LiteOS" key
Huawei 1
LiteOS 8
Open 3
Source 4
LiteOS cpp sample stop
TestClass is destructed
```

Complete Code

TestClass.zip

4.10 Dynamic Shared Memory

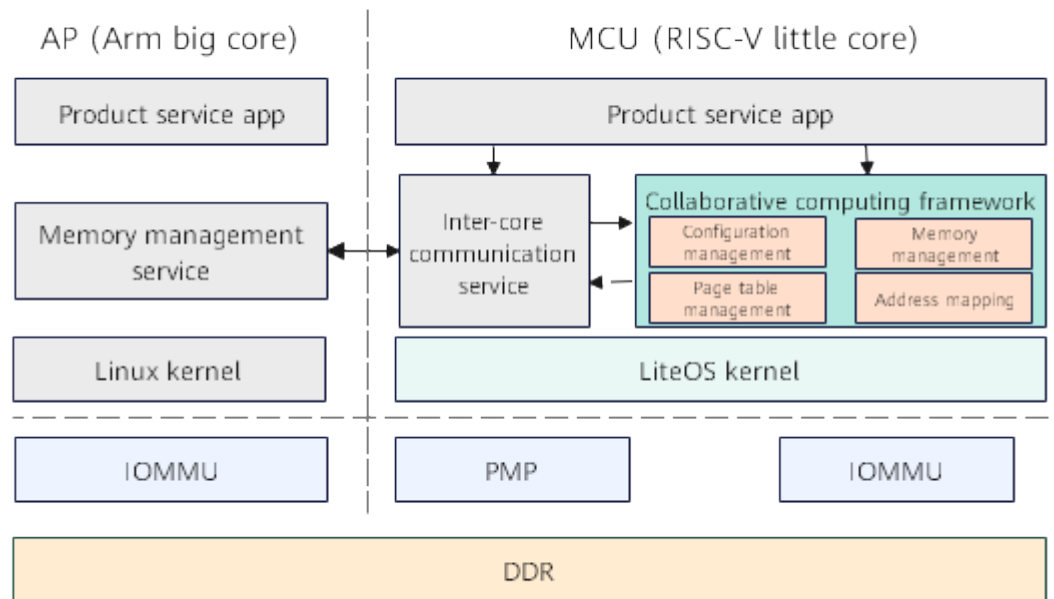
4.10.1 Overview

Basic Concepts

Based on the input-output memory management unit (IOMMU) and physical memory protection (PMP) unit, Huawei LiteOS designs a dynamic shared memory management framework between AP (Android/Linux, Arm big core) and MCU (LiteOS, RISC-V little core). Zero copy is implemented for data interaction between big and little cores, improving performance.

Operation Mechanism

The following figure shows the overall framework design.



The actual physical address space of the shared DDR is managed and allocated by the memory management service (MMS) of the AP big core in a unified manner. When services on the MCU require DDR shared memory, the MCU requests memory from MMS of the AP core through the inter-core communication service. The MMS allocates an `iommuAddr` virtual address and establishes the mapping between DDR physical pages in the page table. The inter-core communication service transfers the memory address and size to the dynamic shared memory management module of the collaborative computing framework. The dynamic shared memory management module allocates an idle virtual address (`virAddr`) in the virtual address space of the MCU to map the physical address allocated by the MMS. The dynamic shared memory includes configuration management, address mapping, memory management, and page table management. The functions are as follows:

- (a) Configuration management: manages configuration and initialization of the dynamic shared memory management module.
- (b) Address mapping: maps and unmaps from the IOMMU virtual address on the AP to the virtual address on the MCU.
- (c) Memory management: allocates and releases the virtual memory and DDR physical shared memory on the MCU.
- (d) Page table management: manages the translation of the IOMMU virtual address on the AP and the virtual address on the MCU, and reconstruction and clearing of the page table.

4.10.2 Development Guidelines

Application Scenario

- Perform shared memory interaction between the big and little cores.
- Perform the translation of the virtual and physical addresses.

Functions

The dynamic shared memory module of Huawei LiteOS provides the following functions. For details about the APIs, see the API reference.

Table 4-10 Functions

Function Category	API	Description
Initializing configuration management	LOS_DynShmInit	Initializes the dynamic shared memory module.
Mapping addresses	LOS_DynShmMap	Maps the virtual IP address on the AP to the virtual IP address on the MCU.
	LOS_DynShmUnmap	Unmaps the virtual address on the MCU.
Translating addresses	LOS_DynShmAddrTranslate	Translates between the AP virtual address, MCU virtual address, and memory physical address.
Managing memory and allocating and releasing shared physical memory	LOS_DynShmBulkMalloc	Allocates DDR physical shared memory from the AP in batches and maps the virtual and physical addresses on the MCU.
	LOS_DynShmBulkFree	Releases the DDR physical shared memory to the AP in batches and unmaps the virtual and physical addresses on the MCU.

NOTE

1. The LOS_DynShmInit API is used to initialize the dynamic shared memory module. If the initialization is repeated, an error is reported.
2. Other APIs can be called only after LOS_DynShmInit is called to initialize the module. Otherwise, the operation fails.
3. LOS_DynShmBulkMalloc and LOS_DynShmBulkFree involve inter-core communication between the AP and MCU, which takes a long time. It is recommended that you set size to a large value to allocate resources in batches at a time.

Error Codes of the Dynamic Shared Memory

No.	Definition	Error Code	Description	Reference Solution
1	LOS_ERRNO_DYN_SHM_NO_MEMORY	0x02002300	Insufficient management resources or virtual memory.	Change the value of LOSCFG_KERNEL_DYN_SHM_VM_BLOCK_NUM or increase the size of the virtual memory.
2	LOS_ERRNO_DYN_SHM_INPUT_INVALID	0x02002301	The input parameter is invalid.	Check whether the input parameters are correct.
3	LOS_ERRNO_DYN_SHM_INIT_FAILED	0x02002302	The initialization fails.	Check whether the internal management resource mutex is successfully created and whether the PMP is successfully configured.
4	LOS_ERRNO_DYN_SHM_ALREADY_MAPPED	0x02002303	Duplicate mapping.	Check whether the start address and size of the input parameter conflict with those before.
5	LOS_ERRNO_DYN_SHM_INIT_REPEATEDLY	0x02002304	The LOS_DynShmInit API is called to initialize the module repeatedly.	Check whether the API is called for multiple times.
6	LOS_ERRNO_DYN_SHM_INTERRUPT	0x02002305	The API is called in the upper part of the interrupt.	Check whether the API is called in the upper part of the interrupt.
7	LOS_ERRNO_DYN_SHM_NOT_READY	0x02002306	The address mapping, address translation, or memory allocation and release API is called before the module is initialized.	Check whether the API is called before initialization.

Development Procedure

The typical development process of the dynamic shared memory is as follows:

1. Open the menu and choose **Kernel > Enable Extend Kernel > Enable Dynamic Shared Memory** and then **Driver > Enable IOMMU** to configure the dynamic shared memory module.

Configuration Item	Description	Value Range	Default Value	Dependency
LOSCFG_KERNEL_DYNSHM	Enables or disables the dynamic shared memory module.	YES/NO	NO	None.
LOSCFG_KERNEL_DYNSHM_SECURE_ENBALE	Determines whether secure access is supported.	YES/NO	NO	The hardware must support the security mechanism, and the LOSCFG_KERNEL_DYNSHM configuration macro must be enabled.
LOSCFG_KERNEL_DYNSHM_VM_BLOCK_NUM	Specifies the number of virtual areas.	Positive integer	1000	LOSCFG_KERNEL_DYNSHM
LOSCFG_DRIVERS_IOMMU	Enables or disables the IOMMU drive module.	YES/NO	NO	The LOSCFG_KERNEL_DYNSHM configuration macro must be enabled.
LOSCFG_DRIVERS_HISI_CUSTOM_IOMMU	Determines whether to use the IOMMU driver customized for HiSilicon.	YES/NO	YES	The LOSCFG_DRIVERS_IOMMU configuration item must be enabled.
LOSCFG_DRIVERS_IOMMU_VA_BITS	Specifies the number of bits in the IOMMU virtual address.	36bit	36bit	The LOSCFG_DRIVERS_IOMMU configuration item must be enabled.

2. Call the LOS_DynShmInit API to initialize the module.
3. Call the LOS_DynShmAddrTranslate API to query the translation between virtual and physical addresses.
4. Call LOS_DynShmBulkMalloc or LOS_DynShmBulkFree to allocate or release the physical shared memory. When the input parameter **noNeedMap** is set to **FALSE**, the virtual-physical address mapping or demapping is performed. Otherwise, only the physical shared memory is allocated or released.

Platform Differences

The hardware must support the IOMMU and RISC-V PMP. Currently, the Hi3751 platform is supported.

4.10.3 Precautions

- Currently, secure access is not supported. The `LOSCFG_KERNEL_DYNshm_SECURE_ENBALE` macro is used for isolation. This function is disabled by default and will be supported in the future.
- The dynamic shared memory module is implemented based on the IOMMU and RISC-V PMP customized for HiSilicon. Therefore, there are requirements on specific hardware.

4.11 Inter-core Communication

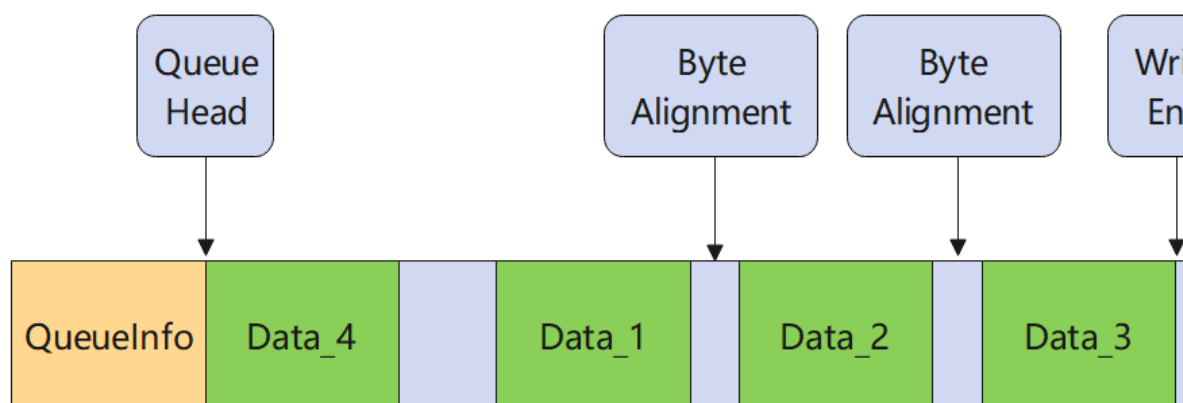
4.11.1 Overview

Basic Concept

Inter-core communication is used for communication between CPUs. It aims to make full use of hardware resources and mechanisms, implement efficient communication, and provide simple and easy-to-use APIs for specific applications.

This module uses circular queues to send and receive data, ensuring performance and reliability and maximizing memory usage. Data is consecutively written to the queues and are automatically aligned by 8 bytes. The receiver directly processes data at the original address without copying the data. In addition, read and write operations can be performed at the same time.

Operation Mechanism



1. In the preceding example, data block **Data_1** already exists in the queue.
2. Data blocks **Data_2** and **Data_3** are written to the queue after they are aligned by start address.
3. Then, data block **Data_4** is written to the queue. Because the space at the queue tail is insufficient, **Data_4** is written to the queue head. **WriteEnd** indicates the data wraparound position of the queue.
4. Queues are read in the write sequence.

4.11.2 Development Guidelines

Table 4-11 Functions

Function Category	API	Description
Initialization	LOS_RpqueueInit	Initializes an inter-core communication queue.
Sending and receiving data	LOS_RpqueueSend	Writes data to the inter-core communication queue. The entire data packet is written to the queue and the addresses are consecutive.
	LOS_RpqueueRecv	Reads data from the inter-core communication queue. The data addresses returned by the receiver are always 8-byte aligned.
Obtaining the length of data that can be sent	LOS_RpqueueMaxSendSizeGet	Obtains the maximum length of packets that can be sent in a queue.

NOTE

1. LOS_RpqueueInit is called by the sender to initialize the queue.
2. Only one thread is allowed to receive data in the same communication queue. If reentrant occurs when data is received in the same queue, the program may crash.
3. LOS_RpqueueSend does not support multiple threads. When multiple threads write data to the same queue, inter-task communication methods such as mutex locks are required to implement multi-thread operations.
4. The data packet whose length is 0 can be sent in the communication queue. Therefore, it is not an error that the LOS_RpqueueMaxSendSizeGet API obtains the data packet whose length is 0.

4.11.3 Precautions

- Do not use the cache for the memory used for inter-core communication to avoid data inconsistency.
- The inter-core communication module cannot verify the address validity. Use a valid shared memory address to perform initialization, read, and write operations.

4.11.4 Programming Example

Description

The following functions are implemented:

1. Core 1 receives data from channel 1, and then sends the received data from channel 2 after **cmd+1** is executed.

2. Core 2 actively sends data from channel 1, and then waits for data reception from channel 2.

Example Code

Prerequisite: Open the menu to enable the inter-core communication module.

The code is as follows:

Core 1 code:

```
#include "los_rpqueue.h"

STATIC UINT8 *g_rpqueueCh1 = (UINT8 *)0x38000000U;
STATIC UINT8 *g_rpqueueCh2 = (UINT8 *)0x38001000U;

STATIC VOID RpqueueSmapleRcv(UINT8 cmd, UINT8 *data, UINT16 len)
{
    LosRpqueueSendCfg queCfg;
    printf("Core 1 Rpqueue Smaple Receive cmd = 0x%02X, len = %d\n", cmd, len);
    queCfg.cmd = cmd + 1;
    queCfg.option = 0;
    queCfg.timeout = 0;
    LOS_RpqueueSend(g_rpqueueCh2, data, len, &queCfg);
}

UINT32 RpqueueSmapleCase(VOID)
{
    printf("Core 1 Rpqueue Smaple Start!\n");

    LOS_RpqueueInit(g_rpqueueCh2, 0x1000, 0);
    while (1) {
        LOS_RpqueueRcv(g_rpqueueCh1, RpqueueSmapleRcv);
    }

    return LOS_OK;
}
```

Core 2 code:

```
#include "los_rpqueue.h"

#define SAMPLE_SEND_LEN 3

STATIC UINT8 *g_rpqueueCh1 = (UINT8 *)0x38000000U;
STATIC UINT8 *g_rpqueueCh2 = (UINT8 *)0x38001000U;

STATIC VOID RpqueueSmapleRcv(UINT8 cmd, UINT8 *data, UINT16 len)
{
    printf("Core 2 Rpqueue Smaple Receive cmd = 0x%02X, len = %d, data : ", cmd, len);
    for (int i = 0; i < len; i++) {
        printf(" %02X", data[i]);
    }
    printf("\n");
}

UINT32 RpqueueSmapleCase(VOID)
{
    UINT32 ret;
    LosRpqueueSendCfg queCfg = {0};
    UINT8 data[SAMPLE_SEND_LEN] = {0x11, 0x22, 0x33};

    printf("Core 2 Rpqueue Smaple Start!\n");

    LOS_RpqueueInit(g_rpqueueCh1, 0x1000, 0);

    queCfg.cmd = 0x55;
    ret = LOS_RpqueueSend(g_rpqueueCh1, data, SAMPLE_SEND_LEN, &queCfg);
    if (ret != LOS_OK) {
```

```
    printf("Core 2 LOS_RpqueueSend Error!\n");
    return LOS_NOK;
}

do {
    if (LOS_RpqueueRecv(g_rpqueueCh2, RpqueueSmapleRecv) == LOS_OK) {
        break;
    }
} while(1);
printf("Core 2 Rpqueue Smaple END!\n");

return LOS_OK;
}
```

Result Verification

The verification result is as follows:

```
Core 1:
Core 1 Rpqueue Smaple Start!
Core 1 Rpqueue Smaple Receive cmd = 0x55, len = 3
Core 2:
Core 2 Rpqueue Smaple Start!
Core 2 Rpqueue Smaple Receive cmd = 0x56, len = 3, data : 11 22 33
Core 2 Rpqueue Smaple END!
```

Complete Code

Sample_Rpqueue_Core1.zip

Sample_Rpqueue_Core2.zip

5 File System

- 5.1 Functions Overview
- 5.2 VFS
- 5.3 NFS
- 5.4 JFFS2
- 5.5 FAT
- 5.6 YAFFS2
- 5.7 RAMFS
- 5.8 PROC
- 5.9 ROMFS
- 5.10 LITTLEFS

5.1 Functions Overview

This chapter uses Hi3556 V200 as an example to describe the file systems currently supported by Huawei LiteOS, including VFS, NFS, JFFS2, FAT, YAFFS2, RAMFS, PROC, ROMFS, and LITTLEFS.

Summary of File Systems

Table 5-1 File system functions

File System	Function
VFS	VFS is short for virtual file system. It uses the standard Unix system to call and read various file systems on different physical media, providing a unified operation mode for various file systems.
NFS	NFS is short for network file system. It can share files between different machines and operating systems over the network.

File System	Function
JFFS2	Journalling Flash File System version 2 (JFFS2) is a log file system mainly used to manage files in the NOR flash memory. In Huawei LiteOS, JFFS2 supports multiple partitions.
FAT	FAT is short for file allocation table, including FAT12, FAT16, and FAT32. FAT is typically used on removable media storage devices, including USB flash drives, secure digital memory cards (SD cards), and removable hard disks, to maintain good compatibility between devices and desktop systems such as Windows and Linux.
YAFFS2	Yet Another Flash File System (YAFFS) is an open-source embedded file system for the NAND flash. It applies to large-capacity storage devices and enables the NAND flash to be efficient and robust. YAFFS2 provides wear leveling and power failure protection for file systems to prevent data damage caused by accidents during file system modification. In Huawei LiteOS, YAFFS2 supports multiple partitions.
RAMFS	RAM File System (RAMFS) is a RAM-based cache mechanism of dynamic file systems. RAMFS places all files in a RAM. Files are read and written from and to the RAM, accelerating the read/write speed and avoiding frequent access to storage.
PROC	PROC is a kind of pseudo file system that exists only in memory and does not use external storage. It provides an interface for accessing the data in Huawei LiteOS Kernel.
ROMFS	ROM file system (ROMFS) is a read-only file system. It provides simple functions and occupies less system resources. ROMFS does not support file writing but only file reading. It is designed to provide a more space-saving file system than a common file system in the boot disk scenario.
LITTLEFS	Little File System (LittleFS) is a block-based fault-tolerant file system. It uses small logs to store metadata and uses the COW structure to store file data. LITTLEFS is mainly used for file management of NOR flash memories. LITTLEFS provides wear leveling and power failure protection for file systems to prevent data damage caused by accidents during file system modification.

5.2 VFS

5.2.1 Overview

Basic Concepts

VFS is an abstraction layer between lower-layer file systems and upper-layer applications. It provides a uniform Unix file operation interface.

Multiple types of file systems that require different accessing interfaces need to be accessed in different modes and through different non-standard interfaces. The VFS layer provides a uniform file system accessing interface to mask the differences between the lower-layer file systems, enabling applications to access the file systems regardless of the types of lower-layer storage media and file systems and improving the developing efficiency.

In Huawei LiteOS, the VFS framework is realized through the tree structure in memory. Each node of the tree is an inode structure. A node is generated in the tree according to the directory where a device is registered and mounted.

The VFS provides the following functions:

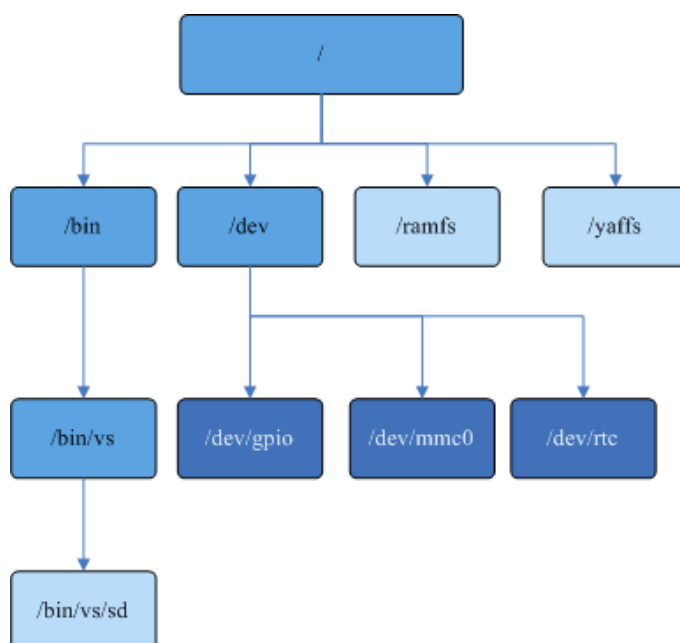
1. Searching for nodes
2. Unified invoking (standard)

Operation Mechanism

VFS enables access to different file systems on different media by calling standard Unix file operation functions (such as open, read, and write).

The key elements of an inode are the `u` field that indicates the pointer to the function structure and the `i_private` field that indicates the pointer to data. The types of inode tree nodes in VFS are as follows:

1. Virtual node: A virtual node serves as a virtual file of the VFS framework, which keeps the continuity of the tree. For example, `/bin` or `/bin/vs` or `/dev` is a virtual node.
2. Device node: A device node is under the `/dev` directory and corresponds to a device. For example, `/dev/mmc0` is a device node.
3. Mounting node: A mounting node, for example, `/bin/vs/sd`, `/ramfs`, and `/yaffs`, is generated after the mount function is called.

Figure 5-1 VFS frame structure

During system initialization, `los_vfs_init()` is called and the slash (/) is used as `root_inode`.

File Descriptors

In this design, file descriptors are classified into two types. Common file descriptors are managed in the form of dynamic memory arrays, and network descriptors are managed in the form of global arrays. The two arrays are discontinuous in the memory.

- File descriptor: A File descriptor is a normal file descriptor, in which 0, 1, and 2 are reserved to serve as system `stdin` (standard input), `stdout` (standard output), and `stderr` (standard error). File descriptors that can be allocated are from 3 to `CONFIG_NFILE_DESCRIPTOR` - 1.
- Socket descriptor: Socket descriptors that can be allocated are from `CONFIG_NFILE_DESCRIPTOR`.

5.2.2 Development Guidelines

API Description

Header File	API	Description
open_source/ incubator- nuttx/ liteos/fs/ include/fs/fs.h	register_driver	Registers the character driver to the file system.
	unregister_driver	Deregisters the character driver.
	register_blockdriver	Registers the block device driver to the file system.

Header File	API	Description
	unregister_blockdriver	Deregisters the block device driver.
	open_blockdriver	Opens the block device.
	close_blockdriver	Closes the block device.
	find_blockdriver	Finds the block device.
fs/include/ los_fs.h	chattr	Changes the attribute of a file. The API is only used in FAT. There are four types of attributes: F_RDO (read-only), F_HID (hidden), F_SYS (system file), and F_ARC (archive file).
	ls	Displays the directory content.
	rindex	Searches for the position where the specified character appears for the first time from the right of the specified character string.

NOTE

About the chattr API:

- Currently, the API is only used to change the file attribute in FAT. The file attributes in other file systems are configured in other ways.
- The F_RDO (read-only), F_HID (hidden), F_SYS (system file), and F_ARC (archive file) attributes do not conflict and can be modified randomly.
- In Huawei LiteOS, read-only files and directories must not be deleted.
- In Huawei LiteOS, read-only files and directories can be renamed.
- Read-only files must not be opened in the O_CREAT and O_TRUNC modes or with read permissions.
- In Huawei LiteOS, hidden files are visible. However, hidden files are invisible in Windows when hidden files are set not to be displayed.
- If system files are configured with the hidden property in Huawei LiteOS, it can only be found in Windows by running commands. These files are invisible regardless of whether hidden files are displayed or not.

Development Procedure

It is recommended that driver developers use the VFS framework to register and uninstall device and use open() and read() operation character devices to make device files invoke the driver on the application layer.

1. Open the menu and choose **FileSystem > Enable VFS** to enable VFS.

Configuration Item	Description	Value Range	Default Value	Dependency
LOSCFG_FS_VFS	Whether to enable the VFS	YES/NO	YES	None.

- The `register_driver()` and `register_blockdriver()` APIs are called to generate device nodes, and the `mount()` API is called to generate mounting nodes where devices are mounted. For details about the development process of the device generation node, see [Development Guidelines](#).

5.2.3 Precautions

- The maximum length of a directory name or a file name created in all file systems under VFS is 255 characters. If the length exceeds 255 characters, the creation fails.
- The maximum full path length supported by the VFS layer is 259 characters. If the full path length exceeds 259 characters, the creation fails.
- Only hyphens (-) and underscores (_) are allowed to exist in names of files and directories in all file systems in Huawei LiteOS. Results of containing other special characters in file and directory names are unpredictable, and it is recommended that not contain those characters.
- For all file systems under VFS, files are opened in creation mode or write-only mode. In this case, the file permission is the value of the input parameter **mode** of the `open` API. The default value is **0666**. Because the file systems under VFS do not fully support the file permission modes, 0666 is recommended.
- During VFS optimization and iteration, the return values of `errno` for abnormal error branches of some POSIX interfaces (such as `rename`, `dup`, and `opendir`) are changed to comply with the POSIX standard. Exercise caution when using the return values of `errno`.
- For a block device node (for example, `/dev/mmc0`), if the `bpos` of the block device driver implements the `unlink` callback function and the return value is OK, `unlink("/dev/mmc0")` is invoked by mistake. As a result, the block device node is deleted and the node disappears. In addition, normal service functions are affected. To keep block device nodes resident, do not implement `unlink` callback.
- The `ls` command displays the size of the block device node file. The size is the capacity of the block device. If you do not want to display the capacity of the block device, the `bpos` of the block device driver does not implement the `geometry` callback function.
- Multilevel directories can be created recursively in VFS.
- Devices are classified into character devices and block devices. To ensure data security on devices, you need to mount the corresponding file system and operate data through the file system API.
- The `los_vfs_init()` API can be only called once. File systems will be abnormal if this API is called for multiple times.

- Parameters **O_RDWR**, **O_WRONLY**, and **O_RDONLY** are mutually exclusive. Only one of them can be used to open a file. Do not use more than one of them to open a file. Otherwise, an unexpected error may occur.
- Huawei LiteOS does not allow the operation of directly obtaining directory data by calling `open()` on a directory. Instead, call the `opendir` API.
- `Open()`, `read()`, and `write()` are non-thread secure APIs. In multi-thread scenarios, you are advised to use the `fopen()`, `fread()`, and `fwrite()` APIs.
- Please mount the file system in strict accordance with the manual, a mount may damage the equipment and system.
- A mount point must be an empty directory and cannot be used for repeated mounting or mounted to another mount point.
- Do not mount a file system that does not exist during wakeup or the scatter loading phase.
- Before uninstalling the device using `unregister_driver`, ensure that the character device is correctly closed. That is, the number of open times must be the same as the number of close times. Otherwise, the uninstallation fails.
- All directories and files must be closed before `umount` operations for file systems in Huawei LiteOS. Otherwise, `umount` operations will fail. Forcible `umount` operations will cause problems including but not limited to damages to file systems and devices, for which Huawei assumes no responsibility.
- All directories and files must be closed before an SD card is removed. Forcibly removing an SD card will cause problems including but not limited to loss of data on the SD card and SD card damage, for which Huawei assumes no responsibility.

5.3 NFS

5.3.1 Overview

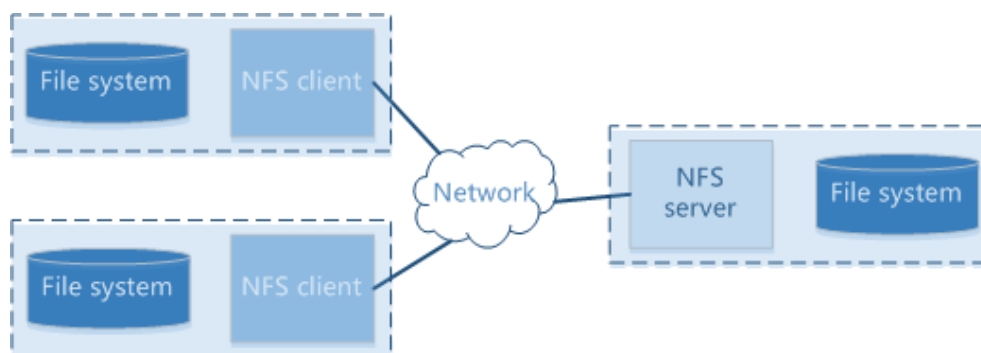
Basic Concepts

NFS is short for network file system. In the NFS, different machines and operating systems can share files through the network. For this reason, it is sometimes regarded as a file system service similar to a shared folder in Windows.

An NFS client user can mount the directory shared by the remote NFS server to the computer or device of the client, run programs, and share files. For the client, having a directory on the remote server is like adding a disk. In addition, the disk does not occupy the system resources of the client, which effectively saves the space of the local disk on the client. In addition, storage devices such as floppy drives, CD-ROMs, and Zips (a high-density disk drive and disk) can be used by other machines on the network, which reduces the number of removable media devices on the entire network.

Operation Mechanism

Figure 1 depicts typical NFS networking.

Figure 5-2 NFS networking

NFS maps the NFS directory (`\home\Huawei LiteOS\nfs`) on an NFS host to the `/nfs` directory on an NFS client, and synchronize one directory with another.

5.3.2 Development Guidelines

Development Procedure

To use NFS functions, perform the following steps:

1. Open the menu and choose **FileSystem** > **Enable NFS** to enable NFS.

Configuration Item	Description	Value Range	Default Value	Dependency
LOSCFG_FS_NFS	Whether to enable the NFS	YES/NO	NO	LOSCFG_FS_VFS=y && LOSCFG_NET_LWIP_SACK=n

2. [Setting a Board as an NFS Client](#)
3. [Setting a Board as an NFS Client](#)
4. [Using NFS to Share Files](#)

Setting Up the NFS Server

The procedure of setting up an NFS server will be described with Ubuntu OS as an example.

- Step 1** Install NFS server software.

Specify the Ubuntu download source and run the following command when network connectivity is good:

```
sudo apt-get install nfs-kernel-server
```

- Step 2** Set the NFS server.

Add the following line to the `/etc/exports` NFS configuration file:

```
/home/yourusername *(rw,no_root_squash,async)
```

/home/yourusername is the root directory for NFS sharing, please set this directory based on the actual situation.

Step 3 Start the NFS server.

Run the following command to start the NFS server:

```
sudo /etc/init.d/nfs-kernel-server start
```

Run the following command to restart the NFS server:

```
sudo /etc/init.d/nfs-kernel-server restart
```

----End

Setting a Board as an NFS Client

The NFS client in this guide refers to a device that runs Huawei LiteOS.

Step 1 Configure hardware connectivity.

Connect the NFS client to the network serving the NFS server. Allocate IP addresses to the NFS client and NFS server. Be sure that the two IP addresses fall within the same address range. For example, set the IP address of the NFS server to 10.67.212.178 and the IP address of the Huawei LiteOS device to 10.67.212.3. (Set the first several segments of the two IP addresses to the same based on the number of subnet mask bits. In this way, the two IP addresses are in the same network segment.).

You can run the **ifconfig** command to view the IP address of the NFS client.

Step 2 Activate the network to establish good network connectivity between the NFS client and the NFS server.

Activate the Ethernet or another type of network, and run the ping command to check whether the server is pingable.

```
Huawei LiteOS # ping 10.67.212.178
[0]Reply from 10.67.212.178: time=1ms TTL=63
[1]Reply from 10.67.212.178: time=0ms TTL=63
[2]Reply from 10.67.212.178: time=1ms TTL=63
[3]Reply from 10.67.212.178: time=1ms TTL=63
--- 10.67.212.178 ping statistics ---
4 packets transmitted, 4 received, 0 loss
```

Step 3 Initialize the NFS client.

Run the following command to mount the shared directory (for example, **/home/sqbin/nfs**) on the server to the local directory (for example, **/nfs**) on the client:

```
Huawei LiteOS# mount 10.67.212.178:/home/sqbin/nfs /nfs nfs 1011 1000
```

If information similar to the following is displayed, the NFS client is successfully initialized.

```
Huawei LiteOS# mount 10.67.212.178:/home/sqbin/nfs /nfs nfs 1011 1000
Mount nfs on 10.67.212.178:/home/sqbin/nfs
Mount nfs finished.
```

 NOTE

The format of the mount command is as follows:

```
mount [SERVER_IP:SERVER_PATH] [CLIENT_PATH] nfs <uid gid>
```

- **SERVER_IP** indicates the IP address of the NFS server.
- **SERVER_PATH** indicates the NFS shared directory on the server. Note that the drive letter of the directory is in lowercase and does not contain colons (:).
- **CLIENT_PATH** indicates the NFS path of the client, that is, the shared directory of the server is mounted to the client directory.
- *uid* indicates the user ID and user group ID of the server, which are used to obtain the permission on the shared directory on the NFS server. If NFS access control is not required, set the permission on the NFS root directory to 777.

```
chmod -R 777 /home/sqbin/nfs
```

The setting of the NFS client is completed, and the NFS is successfully mounted to the NFS client.

----End

Using NFS to Share Files

Create a **dir** directory and save it on the NFS server. On the NFS client, run the **ls** command to view the files in the shared directory:

```
Huawei LiteOS# ls /nfs
```

You may receive the following response from the serial port:

```
Huawei LiteOS#
```

```
Huawei LiteOS# ls /nfs
```

```
Directory /nfs:
```

```
dir          <DIR>dir          <DIR>dir          <DIR>dir          <DIR>
```

The **dir** directory created on the NFS server has been synchronized to the **/nfs** directory on the client. Likewise, files and directories on the NFS client can be accessed from the NFS server.

Platform Differences

Currently, NFS clients only meet parts of NFS v3 specifications and therefore cannot be fully compatible with NFS servers that also meet parts of NFS v3 specifications. During development and tests, you are advised to use the NFS servers for Linux that meet all specifications of NFS v3.

5.3.3 Precautions

- NFS is under debugging and disabled by default. Do not use it in formal products.
- To open a file, **o_flags** must contain one of the **O_RDONLY**, **O_WRONLY**, and **O_RDWR** attributes. The read and write permissions set by **o_flags** must match the mode set when the file is created.
- No tasks can be blocked from reading or writing NFS files.
- NFS files do not support signal functions.
- By default, the NFS uses TCP as the communication socket.

- The length of the mount path in NFS file system (excluding the length of the IP address) should not exceed 259 characters. Otherwise, an ENAMETOOLONG error is returned.
- NFS files support open, close, read, write, seek, dup, opendir, closedir, readdir, rewinddir, statfs, remove, mkdir, rmdir, rename, stat, stat64 and seek64 operations.
- The content of a file can be cleared only when you pass in the O_TRUNC parameter during the call to the open API and have the write permission on the file.
- After the name of file A is changed to the name of file B using rename() when both files are opened, the content of file A will be returned if you read the file descriptor of file B.
- Disclaimer: Huawei shares no responsibilities for any risks caused in the use of NFS in commercial products.

5.4 JFFS2

5.4.1 Overview

Basic Concepts

Journalling Flash File System Version (JFFS2) is mainly used for NOR flash and has the following features:

- Readable and writable.
- Supports data compression.
- Offers protection against breakdown and power failures.
- Be capable of write balancing.

Flash memory differs greatly from disks. Running a disk file system on flash memory might compromise system performance and security. With JFFS2, a file system for flash memory, this problem will no longer be seen.

Huawei LiteOS JFFS2 is mainly used for file management of the NOR flash memory. It supports multiple partitions and directory entry cache (dentry cache) to improve access performance. A maximum of 128 directory entries can be cached at the same time.

5.4.2 Development Guidelines

API Description

Header File	API	Description
fs/include/ mtd_partition. h	add_mtd_partition	Adds a JFFS2 partition. This function will automatically name the device node, the name rule is "/dev/spinorblk" plus the partition number.

Header File	API	Description
	delete_mtd_partition	Deletes an unmounted partition.

NOTE

- When a JFFS2 partition is added by calling `add_mtd_partition`, the system automatically aligns the start address and partition size based on the block size of the device. This function has four parameters:
 - The first parameter indicates the medium (nand or spinor). spinor is used for JFFS2 partitions, and nand is used for YAFFS2.
 - The second parameter indicates the start address of a partition, which is in hexadecimal format.
 - The second parameter indicates the partition size, which is in hexadecimal format.
 - The last parameter indicates the partition number. The value range is from 0 to 19.
- The `delete_mtd_partition` function has two parameters:
 - The first parameter indicates the partition number.
 - The second parameter indicates the media type. This function corresponds to the `add_mtd_partition()` function.

Development Procedure

To use the functions of JFFS2, perform the following operations:

- Open the menu and choose **FileSystem > Enable JFFS2** to enable JFFS2.

Configuration Item	Description	Value Range	Default Value	Dependency
LOSCFG_FS_JFFS	Indicates whether to enable JFFS2.	YES/NO	YES	LOSCFG_FS_VFS && LOSCFG_DRIVER_S_MTD_SPI_NO R && LOSCFG_BASE_IPC_MUX && LOSCFG_BASE_IPC_EVENT
LOSCFG_FS_JFFS_DCACHE	Determines whether to enable the JFFS2 directory entry cache function.	YES/NO	YES	LOSCFG_FS_JFFS

Configuration Item	Description	Value Range	Default Value	Dependency
LOSCFG_FS_JFFS_GC_LAZY	Determines whether to enable the lazy JFFS2 garbage collection.	YES/NO	NO	LOSCFG_FS_JFFS

2. [\(Optional\) Creating and Burning a JFFS2 Image](#)
3. [Adding JFFS2 Partitions](#)
4. [Mounting JFFS2 Partitions](#)
5. [Unmounting JFFS2 Partitions](#)
6. [Deleting JFFS2 Partitions](#)

(Optional) Creating and Burning a JFFS2 Image

Currently, the mkfs.jffs2 tool can be used to create a JFFS2 image. The following is a command example for creating a JFFS2 image:

```
./mkfs.jffs2 -s 0x1000 -e 0x10000 -p 0x100000 -d rootfs/ -o rootfs_64k.jffs2
```

Table 5-2 Parameters

Parameter	Description
-s	Sets the page size.
-e	Sets the eraseblock size.
-p	Sets the image size.
-d	Sets the source directory of the image to be created.
-o	Sets the image name.

NOTE

You can change the parameter values in the command based on site requirements. You can query mkfs.jffs2 to view other parameters.

After the image is created, you can burn the image.

Adding JFFS2 Partitions

Call the add_mtd_partition function to add JFFS2 partitions.

```
if (ret = add_mtd_partition("spinor", 0x100000, 0x800000, 0) != 0) {
    dprintf("add jffs partition 0 failed, return %d\n", ret);
} else {
    dprintf("add jffs partition 0 successfully.\n");
}
```



```

}
if (ret = add_mtd_partition("spinor", 0x900000, 0x200000, 1) != 0) {
    dprintf("add jffs partition 1 failed, return %d\n", ret);
} else {
    dprintf("add jffs partition 1 successfully.\n");
}

```

After the partitions are successfully added, you can run the partition spinor command in Shell to query information about the spinor flash partition.

```

Huawei LiteOS# partition spinor
spinor partition num:0, dev name:/dev/spinorblk0, mountpt:(null), startaddr:0x0100000,length:0x0800000
spinor partition num:1, dev name:/dev/spinorblk1, mountpt:(null), startaddr:0x0900000,length:0x0200000

```

Mounting JFFS2 Partitions

Call the mount() function to mount a device node to the mount point.

```

ret = mount("/dev/spinorblk0", "/jffs0", "jffs", 0, NULL);
if (ret) {
    dprintf("mount jffs err %d\n", ret);
    dprintf("Mount jffs2 on nor finished.\n");
}

```

NOTE

The mount() function has five parameters:

- The first parameter indicates the device node. This parameter needs to be associated with the add_mtd_partition() function.
- The second parameter indicates the mount point.
- The third parameter indicates the file system type.
- The fourth parameter indicates the mount flag. The default value is **0**.
- The last parameter indicates the private data. The default value is **NULL**.

To check whether the mount() function is successfully called, run the ls command in Shell to view the directories in the root directory. If the following information is displayed, JFFS2 partitions are successfully mounted:

```

Huawei LiteOS# ls
Directory /:
bin          <DIR>
dev          <DIR>
jffs0        <DIR>
ramfs        <DIR>
yaffs0       <DIR>

```

In addition to calling the mount() function in the source code, you can also use the mount command in Shell. The first three parameters are the same as those of the mount() function, and the last two parameters do not need to be provided.

```

Huawei LiteOS# mount /dev/spinorblk1 /jffs1 jffs

```

```

Huawei LiteOS# ls
Directory /:
bin          <DIR>
dev          <DIR>
jffs0        <DIR>
jffs1        <DIR>
ramfs        <DIR>
yaffs0       <DIR>

```

After JFFS2 partitions are successfully mounted, you can read from and write to NOR flash.

Unmounting JFFS2 Partitions

Call the `umount()` function to unmount JFFS2 partitions. Only the correct mount point needs to be specified. You can also run the following `umount` command in Shell:

```
Huawei LiteOS# umount /jffs1
```

If the following information is displayed, the uninstallation is successful:

```
Huawei LiteOS# umount /jffs1
umount ok
```

Deleting JFFS2 Partitions

Call the `delete_mtd_partition` function to delete the unmounted JFFS2 partitions.

```
ret =delete_mtd_partition(1, "spinor");
if (ret != 0) {
    printf("delte jffs error\n");
} else {
    printf("delete jffs ok\n");
}
```

You can run the **partition spinor** command in shell to check whether the partition is successfully deleted.

```
Huawei LiteOS# partition spinor
spinor partition num:0, dev name:/dev/spinorblk0, mountpt:/jffs0, startaddr:0x0100000,length:0x0800000
```

5.4.3 Precautions

- Currently, JFFS2 is used for NOR flash, and the NOR flash drive interface need to be called. Therefore, before using JFFS2, ensure that NOR flash is available on hardware and the driver is successfully initialized (the value returned by `spinor_init()` is 0).
- Currently, JFFS2 supports the `open`, `close`, `read`, `write`, `seek`, `seek64`, `opendir`, `closedir`, `readdir`, `rewinddir`, `statfs`, `unlink`, `mkdir`, `rmdir`, `rename`, `stat`, and `stat64` operations.
- When you use the `open` operation with the parameter **O_TRUNC** to open a file, the file content will be cleared.
- Currently, JFFS2 does not support the `open` operation with the **O_DIRECTORY** attribute. Use `opendir()` to open a directory.
- JFFS2 supports the read-only mounting attribute. If the input parameter **mountflags** of the `mount` function is set to **MS_RDONLY**, the read-only mounting attribute is enabled for JFFS2. All APIs used to write data, such as `write`, `mkdir`, and `unlink`, and files that are opened by using the attribute other than **O_RDONLY** are rejected and the error code `EACCESS` is returned.
- To open a file, `o_flags` must contain one of the `O_RDONLY`, `O_WRONLY`, and `O_RDWR` attributes. The read and write permissions set by `o_flags` must match the mode set when the file is created.

5.5 FAT

5.5.1 Overview

Basic Concepts

File allocation table (FAT) is classified into three types: FAT12, FAT16, and FAT32. FAT divides a disk into five sectors: master boot record (MBR), dos boot record (DBR), FAT, DIR, and DATA.

FAT can be implemented on diverse media, especially on removable media storage devices including USB flash drives, secure digital memory cards (SD cards), and removable hard disks. In addition, it maintains good compatibility between embedded devices and desktop systems such as Windows and Linux, which is convenient for managing operation files.

FAT in Huawei LiteOS has the following features:

- Has small amount of code and uses less resources; be tailorable and supports multiple types of physical media.
- Be compatible with operating systems including Windows and Linux.
- Supports multiple devices and the identification of multiple partitions.
- Supports disk partitioning; file operations can be performed on the primary partition and logical partitions.
- Identifies other types of file systems on hard disks, such as NTFS.
- Supports the FAT32-based virtual partitioning feature.

If the virtual partitioning feature is enabled, you can configure the number of virtual partitions, space percentage, and entry name for a specified physical partition by calling `los_set_virpartparam()`. In addition, a corresponding folder is created in the root directory as the entry of the virtual partition based on the configured entry name.

- Performing operations in the entry of a virtual partition means performing operations in this virtual partition.
- If the total space percentage of all virtual partitions does not reach 100%, you can perform write operations outside the entry directory of the virtual partition. In this case, the total writable space outside the virtual partition entry is the total space minus the space of all virtual partitions. If the space percentage of all virtual partitions reaches 100%, write operations will be rejected outside the entry directory of the virtual partition due to insufficient space.

5.5.2 Development Guidelines

Description of Basic FAT APIs

Header File	API	Description
fs/include/ los_fs.h	getlabel	Obtains a drive letter.
	set_label	Sets a drive letter.
	format	Formats disks.

Header File	API	Description
	los_set_systime_status	Validates or invalidates the current system time of FAT.
	LOS_SetSyncThreadPrio	Sets the priority of the asynchronous cache flushing task of FAT.
	LOS_SetSyncThreadInterval	Sets the interval for asynchronous cache flushing of FAT.
	LOS_GetSyncThreadInterval	Obtains the interval for asynchronous cache flushing of FAT.
	LOS_SetDirtyRatioThreshold	Sets the dirty data block percentage thresholds for asynchronous cache flushing of FAT.
	LOS_GetDirtyRatioThreshold	Obtains the dirty data block percentage thresholds for asynchronous cache flushing of FAT.
	LOS_SetBlockExpireInterval	Sets the maximum expiration interval of FAT blocks.
	LOS_GetBlockExpireInterval	Obtains the maximum expiration interval of FAT blocks.

NOTE

- After asynchronous cache flushing is enabled for FAT, the system creates a task to flush the cache. A thread is woken up to flush dirty data when the following conditions are met: 1. The thread wakes up every 5s. 2. Dirty data stays for more than 3s. 3. When the amount of dirty data exceeds 50% and any of the preceding conditions is met, the thread immediately flushes the dirty data to the disk.
- You can configure FAT in the **open_source/FatFs/source/ffconf.h** file. **FF_FS_LOCK** indicates the maximum number of files (folders) that can be opened at the same time.

Description of Virtual Partition APIs

Header File	API	Function Description
fs/include/los_fs.h	los_set_virpartparam	Sets virtual partition parameters.
	virstatfs	Obtains the status of a specified virtual partition.

The definition of the allocation structure of a virtual partition is as follows:

```
#define _MAX_ENTRYLENGTH 16
#define _MAX_VIRVOLUMES 5

typedef struct virtual_partition_info
{
```

```
char *devpartpath;  
int virpartnum;  
double virpartpercent[_MAX_VIRVOLUMES];  
char virpartname[_MAX_VIRVOLUMES][_MAX_ENTRYLENGTH + 1];  
} virpartinfo;
```

Item	Description	Precautions
devpartpath	Path of a device node	Set this configuration item to the name of the selected device node. The path must be a device node path under /dev . The upper limit is DISK_NAME .
virpartnum	Number of virtual partitions	The minimum value is 1 , and the maximum value is _MAX_VIRVOLUMES , that is, 5 .
virpartpercent	Space percentage of the virtual partitions	The sum must be greater than 0.00 and less than or equal to 1.00. The value of each member must be greater than 0.00 and less than 1.00.
virpartname	Directory entry of the virtual partitions	This parameter can contain a maximum of 16 characters (defined by _MAX_ENTRYLENGTH). The value cannot be null. The input set cannot contain duplicate items. The entry name cannot contain invalid characters.

NOTICE

- In Huawei LiteOS, the virtual partitioning feature is enabled for a specific physical partition. You can set the device node by allocating the virtual partition parameter API.
- After you configure the specified physical device node, the configured parameters are locked and cannot be modified. During the mounting, the configured parameters are used. After the unmounting, the settings are unlocked.
- After the uninstallation, you need to reconfigure virtual partition parameters by calling the corresponding API.
- If the specified physical partition has a virtual partition, and the original parameter configuration of the virtual partition is inconsistent with that configured through the allocation API, the system manages the virtual partition based on the original parameter configuration.

API Behavior Change Related to the Virtual Partition

After the virtual partitioning feature is enabled, the behavior of some APIs changes in the virtual partition. The following API behavior changes are only for physical partitions that have successfully applied the virtual partitioning feature. For basic FAT physical partitions that do not apply the virtual partitioning feature and other file systems, the API behavior does not change.

Only the operation behavior of the following APIs changes in the virtual partition. The function calling methods and parameters do not change.

No.	API	Behavior Change	Return Value and Error Code Description
1	mount	After the basic FAT partition is mounted, this API checks whether the current physical partition has a virtual partition, loads the configured virtual partition information, and checks and repairs the virtual partition entry. If the physical partition does not have a virtual partition and meets the criteria for creating a virtual partition, this API automatically creates a virtual partition in the physical partition.	If the return value is -1, the basic FAT partition fails to be mounted. In this case, the libc error code is triggered to indicate the error. If the returned value is 0, the basic FAT partition is successfully mounted. In this case, if the error code is 0, the virtual partition is successfully mounted. If the error code is VIRERR_MODIFIED, VIRERR_CHAIN_ERR, VIRERR_OCCUPIED, VIRERR_NOTCLEAR, VIRERR_NOTFIT, or VIRERR_NOPARAM, the virtual partition fails to be mounted.
2	format	After the FAT partition is mounted, if the file system is formatted into the FAT32 file system, the virtual partition is applied to the physical partition after the formatting is completed. If the file system is formatted into a system other than FAT32, the virtual partition is removed from the physical partition after the formatting is completed. If the FAT partition is not mounted and no file system is available, the virtual partition is removed from the physical partition after the formatting is completed.	If the return value is -1, the basic FAT partition fails to be formatted. In this case, the libc error code is triggered to indicate the error. If the return value is 0, the basic FAT partition is successfully formatted. If the partition is not mounted, the error code is VIRERR_NOTMOUNT. If the formatted partition is not the device node path specified by the allocation API or the allocation API does not set parameters, VIRERR_NOPARAM is returned. If the partition is mounted and the error code is 0, the application or re-application of the virtual partition is successful, or the error code may be VIRERR_NOTFIT.
3	open	See the description in Compatibility of Virtual Partitions .	None.

No.	API	Behavior Change	Return Value and Error Code Description
4	mkdir	See the description in Compatibility of Virtual Partitions .	None.
5	rmdir	After the virtual partition is successfully applied in the current physical partition, this API cannot be used to delete the virtual partition entry directory.	None.
6	remove	After the virtual partition is successfully applied in the current physical partition, this API cannot be used to delete the virtual partition entry directory.	None.
7	rename	After the virtual partition is successfully applied in the current physical partition, this API cannot be used to rename the virtual partition entry directory.	None.

Virtual Partition Error Codes

An error occurs in the application of the virtual partition only when the virtual partition is mounted or formatted. After operations on the basic FAT are completed, configure and apply the virtual partition. In this case, if a problem occurs or a condition is triggered, the error code of the virtual partition is transmitted to the upper layer, so that the situation in the application of the virtual partition is known. At the same time, the partition is managed as a basic FAT instead of a virtual partition.

NOTE

Returned values upon mounting and formatting indicate the success or failure of the basic FAT.

- If the return value is **0**, the basic FAT operation is successful. In this case, you can obtain the error code in **errno** to check whether any exception occurs during the application of the virtual partition.
- If the return value is **-1**, the basic FAT operation fails. Therefore, **errno** is the libc error code, which indicates the cause of the basic FAT operation failure and does not provide virtual partition error information.

No.	Definition	Error Code	Description	Triggered By
1	VIRERR_MODIFIED	0x10000001	The reserved sector information of the virtual partition is damaged.	The partition is repartitioned, the partition size is adjusted, or the SD card is damaged.
2	VIRERR_CHAIN_ERR	0x10000002	The entry directory of the virtual partition has been removed from the virtual partition.	The entry directory of the virtual partition is deleted and then recreated on other platforms. This problem will be triggered in a certain probability.
3	VIRERR_OCCUPIED	0x10000003	The virtual partition entry is occupied by the file with the same name.	A root directory of the physical partition has a file with the same name, which hinders creation of the virtual partition entry.
4	VIRERR_NOT CLEAR	0x10000004	The physical partition is not empty.	A non-null virtual partition is applied to a new physical partition.
5	VIRERR_NOT FIT	0x10000005	The physical partition is not a FAT32 file system.	The physical partition is not a FAT32 file system.
6	VIRERR_NOT MOUNT	0x10000006	The device is not mounted.	The device is not mounted during formatting.
7	VIRERR_INTERRUPT_ERR	0x10000007	An unexpected error occurs.	An error occurred during the application of the virtual partition.
8	VIRERR_NOPARAM	0x10000008	The parameters are not configured for the virtual partition.	The allocation API is not called before performing the mounting or formatting operation.
9	VIRERR_PARAMETER_LOCKED	0x10000009	The virtual partition configuration is locked.	You attempt to modify virtual partition parameters after the configuration is locked.
10	VIRERR_PARAMETER_NUMERR	0x1000000A	The VirPartNum field of the allocation API is incorrect.	The VirPartNum field is incorrectly configured.

No.	Definition	Error Code	Description	Triggered By
11	VIRERR_PARMPERCENTERR	0x1000000B	The VirPartPercent field of the allocation API is incorrect.	The VirPartPercent field is incorrectly configured.
12	VIRERR_PARMNAMEERR	0x1000000C	The VirPartName field of the allocation API is incorrect.	The VirPartName field is incorrectly configured.
13	VIRERR_PARMDEVERR	0x1000000D	The DevPartPath field of the allocation API is incorrect.	The DevPartPath field is incorrectly configured.

Development Procedure

To use the functions of FAT, perform the following operations. The operations on the virtual partition are optional and are required only when FAT uses virtual partitioning. Select the operations based on the actual service situation.

1. Open the menu and choose **FileSystem > Enable FAT** to configure FAT.

Configuration Item	Description	Value Range	Default Value	Dependency
LOSCFG_FS_FAT	Determines whether to enable FAT.	YES/NO	YES	LOSCFG_FS_VFS && LOSCFG_DRIVE_R_DISK
LOSCFG_FS_FAT_CACHE	Determines whether to enable the FAT cache function.	YES/NO	YES	LOSCFG_FS_FAT
LOSCFG_FS_FAT_CACHE_SYNC_THRESHOLD	Determines whether to enable asynchronous cache flushing of FAT.	YES/NO	YES	LOSCFG_FS_FAT_CACHE
LOSCFG_FS_FAT_CACHE_PROC	Determines whether to enable the FAT cache statistics.	YES/NO	YES	LOSCFG_FS_FAT_CACHE

Configuration Item	Description	Value Range	Default Value	Dependency
LOSCFG_FS_FAT_C HINESE	Determines whether to enable FAT to support Chinese.	YES/NO	YES	LOSCFG_FS_FAT
LOSCFG_FS_FAT_D CACHE	Determines whether to enable the FAT directory entry cache function.	YES/NO	YES	LOSCFG_FS_FAT
LOSCFG_FS_FAT_T RIM	Determines whether to enable the FAT media TRIM function.	YES/NO	YES	LOSCFG_FS_FAT
LOSCFG_FS_FAT_U NI_TRIM	Determines whether to enable the FAT media UNION-TRIM function.	YES/NO	YES	LOSCFG_FS_FAT
LOSCFG_FS_FAT_D IRECT_TRIM	Determines whether to enable the FAT media DIRECT-TRIM function.	YES/NO	NO	LOSCFG_FS_FAT
LOSCFG_FS_FAT_V IRTUAL_PARTITIO N	Determines whether to enable FAT to support virtual partitioning.	YES/NO	NO	LOSCFG_FS_FAT

NOTE

- After the virtual partitioning feature is enabled, Huawei LiteOS FAT is preferentially managed by virtual partitioning. Virtual partitioning does not support FAT directory entry cache.
- After the FAT directory entry cache is enabled, some memory is consumed to cache the directory entries of the FAT file system, reducing the time required for accessing some interfaces again.
- After the FAT TRIM function is enabled, the disk write amplification can be effectively slowed down, improving the stability of the data write rate.

2. [Identifying Devices](#)
3. [Mounting FAT](#)
4. [Unmounting FAT](#)

5. (Optional) Checking the Conditions for Operating Virtual Partitions
6. (Optional) Allocating Virtual Partitions
7. (Optional) Mounting Virtual Partitions
8. (Optional) Unmounting Virtual Partitions
9. (Optional) Creating Virtual Partitions
10. (Optional) Deleting Virtual Partitions

Identifying Devices

You can enable multiple partitions and devices in the `open_source/FatFs/source/ffconf.h` file. After the multi-partition function is enabled, multiple partitions on a device can be operated. Otherwise, only the first partition can be operated. After the multi-device function is enabled, multiple devices can be mounted to FAT. The configuration items are as follows:

- When `FF_MULTI_PARTITION` is set to `1`, the multi-partition function is enabled.
- When `FF_VOLUMES` is set to a value greater than `1`, the multi-device function is enabled.

The system automatically identifies the inserted SD cards and registers the following device nodes. `mmcblk0` (card 0) and `mmcblk1` (card 1) are independent primary devices. `mmcblk0p0` and `mmcblk0p1` are two partitions of card 0, and `mmcblk1p0` is a partition of card 1. These partitions can be used as partition devices. **Do not use the primary device if a partition device exists.**

```
Huawei LiteOS# ls
Directory /dev:
acodec          0
adec            0
aenc           0
ai             0
ao            0
console        0
fb0            0
hi_gpio        0
hi_mipi        0
hi_rtc         0
hi_tde         0
i2c-0          0
i2c-1          0
i2c-2          0
isp_dev        0
lcd            0
logmpp         0
mem            0
mmcblk0        0
mmcblk0p0      0
mmcblk0p1      0
mmcblk1        0
mmcblk1p0      0
```

You can run the `partinfo` command in Shell to view the device partition information.

```
Huawei LiteOS # partinfo /dev/sdap0
part info :
disk id    : 3
part_id in system: 0
```

```
part no in disk : 0
part no in mbr  : 1
part filesystem : 0C
part dev name   : sdap0
part sec start  : 2048
part sec count  : 167794688
```

Mounting FAT

Call the mount() function to mount a device node to the mount point.

```
ret = mount("/dev/mmcbk0p0", "/bin/vs/sd", "vfat", 0, NULL);
if (ret) {
    dprintf("mount fat filesystem err %d\n", ret);
}
```

NOTE

The mount() function has five parameters:

- The first parameter indicates the device node.
- The second parameter indicates the mount point.
- The third parameter indicates the file system type.
- The fourth parameter indicates the mount flag. The default value is **0**.
- The last parameter indicates the private data. The default value is **NULL**.

In addition to calling the mount() function in the source code, you can also use the mount command in Shell for mounting. The first three parameters are the same as those of the mount() function, and the last two parameters do not need to be configured. If the following information is displayed, FAT is successfully mounted:

```
Huawei LiteOS# mount /dev/mmcbk0p0 /bin/vs/sd vfat
mount ok
```

Unmounting FAT

Call the umount() function to unmount FAT partitions. Only the correct mount point needs to be specified. You can also run the umount command in Shell. If the following information is displayed, the unmounting is successful:

```
Huawei LiteOS# umount /bin/vs/sd
umount ok
```

(Optional) Checking the Conditions for Operating Virtual Partitions

- If the virtual partitioning feature is not used for a physical partition in a storage device (such as an SD card, USB flash drive, and portable hard disk) in Huawei LiteOS, ensure that:
 - The physical partition is a FAT32 partition.
 - No file exists in the physical partition.
- If the virtual partitioning feature is used for a physical partition in a storage device in Huawei LiteOS, ensure that:
 - The virtual partition entry directory in the physical partition is not deleted, renamed, or damaged in any other forms.
 - Data in the reserved sectors of the physical partition is complete.

If any of the preceding conditions is not met, the physical partition should use a regular FAT instead of the virtual partitioning feature.

(Optional) Allocating Virtual Partitions

The virtual partitioning feature is used for physical partitions in a specific scenario. You need to specify a partition device and set configuration parameters for the partition by using the allocation API `los_set_virpartparam()`. By allocating the virtual partition parameter API, you can create virtual partitions that meet the scenario requirements.

To enable the virtual partitioning feature after system startup, you need to call `los_set_virpartparam()` to configure the number of virtual partitions and set key global parameters of the virtual partitions. If these parameters are not set, subsequent application of virtual partitions will fail.

- If the virtual partitioning feature is not used for a physical partition in a storage device of Huawei LiteOS, use the allocation API to set the parameters required by the physical partition before the application of the virtual partition. In normal cases, after the allocation is successful, perform the mount operation on the physical partition to complete the application of the virtual partition.
- If the virtual partitioning feature has been applied to the physical partition, ignore the parameters set by the allocation API as long as the configured virtual partition is verified correctly. The parameters are managed unconditionally according to the internal parameters of the physical partition.

(Optional) Mounting Virtual Partitions

After FAT is mounted, the virtual partition is automatically applied to the mounted partition. If the virtual partition is successfully applied, the virtual partition starts managing the physical partition. If the virtual partition fails to be applied, the physical partition is managed as a regular FAT, and a specified error code is returned to indicate the error that occurred during the application of the virtual partition.

After a virtual partition is successfully mounted, the corresponding number of directories with the corresponding names are created in the root directory as directory entries of virtual partitions. Creating a file inside the directory entry of a virtual partition means creating a file in this virtual partition. The maximum available space in a virtual partition entry is configured as a percentage

(Optional) Unmounting Virtual Partitions

The procedure for unmounting virtual partitions is the reverse of the procedure of mounting and therefore is not described again.

(Optional) Creating Virtual Partitions

In Huawei LiteOS, after the virtual partitioning feature is used for a new physical partition, you can create virtual partitions by performing the mount operation. This virtual partition can be repeatedly used in Huawei LiteOS. The basic parameters of the created virtual partition are subject to the parameters set in the allocation API.

To re-apply a virtual partition to a physical partition, format the disk as a FAT32 file system on the mounted partition. After the formatting, the virtual partition is re-applied to the physical partition.

NOTICE

The formatting operation will cause a loss of user data. Exercise caution when performing this operation.

(Optional) Deleting Virtual Partitions

To cancel the virtual partitioning feature for a physical partition, format the physical partition. Virtual partition information is then deleted from the physical partition.

NOTICE

The formatting operation may cause data loss. Therefore, exercise caution when performing this operation.

5.5.3 Compatibility of Virtual Partitions

The virtual partitioning feature is the FAT-based file management mode specific for Huawei LiteOS. When performing operations in physical partitions to which the virtual partitioning feature has been applied on other platforms, ensure that you understand the following compatibility issues to avoid unexpected errors:

- Operations such as re-partitioning the storage medium and adjusting the size of the storage medium may cause the virtual partitioning feature applied to physical partitions to become invalid.
- When creating folders in the physical partitions to which the virtual partitioning feature has been applied on other platforms, pay attention to the following items:
 - If the created folder is outside the virtual partition entry and the total percentage of all virtual partitions is 100%, new files or folders cannot be created in the folder in Huawei LiteOS.
- When writing files into the physical partition to which virtual partitioning feature has been applied on other platforms, pay attention to the following items:
 - If the files are written outside the virtual partition entry and the total percentage of all virtual partitions is 100%, these files can be accessed only in read-only mode in Huawei LiteOS.
 - If the files are written within the virtual partition entry and the cluster chains of the files are within the management scope of the virtual partition, these files can be normally read and written in Huawei LiteOS.
 - If the files are written within the virtual partition entry but part or all of the cluster chains of the files are not within the management scope of the virtual partition, these files can only be read in Huawei LiteOS.

- If you delete the virtual partition entry directory on other platforms, the virtual partition will be lost. The system will attempt to rebuild the lost virtual partition when you mount the directory to Huawei LiteOS.
 - If a folder with the same name is created after the deletion, and the entire cluster chain of the folder falls within the management scope of the corresponding virtual partition, the folder can be identified when it is mounted in Huawei LiteOS.
 - If a folder with the same name is created after the deletion, and part or all of the entire cluster chain falls outside the management scope of the corresponding virtual partition, the virtual partition entry verification fails and the virtual partitioning request is rejected.
- If you rename the virtual partition entry directory on other platforms, the virtual partition will be lost. The system will attempt to rebuild the lost virtual partition when you mount the directory to Huawei LiteOS. The original files are not lost, but the virtual partitioning feature no longer takes effect on these files, and these file can only be read.

5.5.4 Precautions

- In a FAT, the size of a file cannot be greater than 4 GB.
- For a file name with an absolute path, the total length of the file name excluding the mount point cannot exceed 252 bytes.
- Only the FAT32 format is supported. To use other file formats, format the medium to FAT32 and then connect the medium to Huawei LiteOS.
- FAT supports the open, close, read, write, seek, sync, opendir, closedir, readdir, rewinddir, statfs, vstatfs, unlink, mkdir, rmdir, rename, stat, lstat, stat64, utime, chattr, seek64, getlabel, set_label, format, fallocation, fallocation64, truncate, truncate64, and fscheck operations.
- After a file is opened in writable mode, the file cannot be opened again before it is closed. If you want to open a file again before it is closed, you must open the file in read-only mode. If a file is opened for a long time and is not closed, data loss will occur. The file can be saved only after being closed.
- When you use **O_TRUNC** to open a file, the file content will be cleared.
- If you do not specify the value of the input parameter **mode** for the open API, you cannot use the **mode** input parameter to control the file permission.
- Currently, FAT does not support the open API with the **O_DIRECTORY** attribute. Use opendir() to open a directory.
- The read and write pointers are not separated. Therefore, after a file is opened in **O_APPEND** mode, the read pointer is also at the end of the file. You must manually set the position of the read pointer before reading the file.
- The file modification time is obtained by the stat and lstat functions. The file creation time and last access time cannot be obtained. The Microsoft FAT protocol supports only time after A.D. 1980.
- If a FAT has been mounted before the format operation, ensure that all directories and files are closed. Otherwise, the format operation will fail.
- FAT supports the read-only mounting attribute:
 - If the input parameter of the mount function is set to **MS_RDONLY**, the read-only mounting attribute is enabled for FAT. All APIs (except format) used to write data, such as write, mkdir, and unlink, and files that are

opened by using the attribute other than **O_RDONLY** are rejected and the error code EACCESS is returned.

- When **MS_NOSYNC** is carried in the mount function, FAT does not proactively write the content in the cache back to the storage device. The FAT APIs, such as open, close, unlink, rename, mkdir, rmdir, chattr and truncate, will not automatically perform the sync operation. The speed can be improved, but they need the upper layer to actively call sync to synchronize the data. Otherwise, the data may be lost after a power outage.
- The default cache size is 16 blocks. Each block contains 256 sectors. You can change the number of cache blocks **CONFIG_FS_FAT_BLOCK_NUMS** and the number of sectors **CONFIG_FS_FAT_SECTOR_PER_BLOCK** in the **fs/include/vfs_config.h** file. Note that the value of **CONFIG_FS_FAT_SECTOR_PER_BLOCK** must be a multiple of 32 and cannot be greater than 2048.
- If two SD card slots are available, the SD card that is inserted first is card 0; the one inserted later is card 1.
- To prevent SD card exceptions and memory leakage, opened files and directories must be closed and mounted nodes must be unmounted before an SD card is removed.

5.6 YAFFS2

5.6.1 Overview

Basic Concepts

Yet another flash file system (YAFFS) is an open-source embedded file system designed for NAND flash. In YAFFS, the minimum storage unit is page.

Two versions of YAFFS are available in current YAFFS file system: YAFFS and YAFFS2. The major difference between them is that YAFFS2 supports NAND flash chips with 2 KB pages, far larger than the chips with 512-byte pages supported by YAFFS. Another difference lies in the fact that YAFFS2 has a 64-byte spare area to store bad block information and carry out error checking and correction (ECC).

- YAFFS2 is suitable for large-capacity storage devices. It is specially designed for NAND flash to keep high efficiency and robustness.
- YAFFS2 supports chips with 2 KB pages. Moreover, it uses less memory, and allows faster read/write and junk data reclamation.
- Wear leveling and power failure protection ensures that data will not be corrupted when a power outage occurs amid file system modification.
- YAFFS2 in Huawei LiteOS supports multiple partitions and uses the bidirectional linked list structure.

5.6.2 Development Guidelines

API Description

Header File	API	Description
fs/include/ mtd_partition. h	add_mtd_partition	Adds a YAFFS2 partition. This function will automatically name the device node, the name rule is "/dev/nandblk" plus the partition number.
	delete_mtd_partition	Deletes an unmounted partition.

NOTE

- When a YAFFS2 partition is added by calling add_mtd_partition, the system automatically aligns the start address and partition size based on the block size of the device. This function has four parameters:
 - The first parameter indicates the medium (nand or spinor). nand is used for YAFFS2 partitions, and spinor is used for JFFS2.
 - The second parameter indicates the start address of a partition, which is in hexadecimal format.
 - The second parameter indicates the partition size, which is in hexadecimal format.
 - The last parameter indicates the partition number. The value range is from 0 to 19.
- The delete_mtd_partition function has two parameters:
 - The first parameter indicates the partition number.
 - The second parameter indicates the media type. This function corresponds to the add_mtd_partition() function.

Development Procedure

To use the functions of YAFFS2 in Huawei LiteOS, perform the following steps:

- Open the menu and choose **FileSystem** > **Enable YAFFS2** to enable YAFFS2.

Configuration Item	Description	Value Range	Default Value	Dependency
LOSCFG_FS_YAF FS	Whether to enable YAFFS2	YES/NO	YES	LOSCFG_FS_VFS && LOSCFG_DRIVER S_MTD_NAND

- [\(Optional\) Creating and Burning a YAFFS2 Image](#)
- [Calling add_mtd_partition to Create Partitions](#)
- [Calling mount to Mount Partitions](#)
- [Calling umount to Unmount Partitions](#)
- [Calling delete_mtd_partition to Delete Partitions](#)

(Optional) Creating and Burning a YAFFS2 Image

Currently, the mkyaffs2image tool can be used to create a JFFS2 image. The following is a command example for creating a JFFS2 image:

```
./mkyaffs2image rootfs rootfs.yaffs2 1 2
```

Table 5-3 Parameter description

Parameter	Description
rootfs	Source directory of the file system image to be created
rootfs.yaffs2	Image name.
1	Page size of the Nand flash device. 1 indicates that the page size is 2K bytes.
2	ECC type of the Nand flash device. 2 indicates 4 bits/512.

NOTE

You can change the values of the parameters in the command based on the actual situation.

After the image is created, you can burn the image.

Calling add_mtd_partition to Create Partitions

Command for creating partitions:

```
if (ret = add_mtd_partition("nand", 0x900000, 0x200000, 0) < 0) {  
    dprintf("add yaffs partition 0 failed, return %d\n", ret);  
}  
  
if (ret = add_mtd_partition("nand", 0xb00000, 0x200000, 1) < 0) {  
    dprintf("add yaffs partition 1 failed, return %d\n", ret);  
}
```

After the partitions are successfully created, you can run the partition nand command in Shell to query information of Nand flash partitions.

```
Huawei LiteOS# partition nand  
nand partition num:0, dev name:/dev/nandblk0, mountpt:(null), startaddr:0x0900000,length:0x0200000  
nand partition num:1, dev name:/dev/nandblk1, mountpt:(null), startaddr:0xb00000,length:0x0200000
```

Calling mount to Mount Partitions

Call the mount() function to mount a device node to the mount point.

```
ret = mount("/dev/nandblk0", "/yaffs0", "yaffs", 0, NULL);  
if (ret) {  
    dprintf("mount yaffs err %d\n", ret);  
}
```

For details about the input parameters of the mount() function, see [Mounting JFFS2 Partitions](#).

NOTE

The mount() function has five parameters:

- The first parameter indicates the device node. This parameter needs to be associated with the add_mtd_partition() function.
- The second parameter indicates the mount point.
- The third parameter indicates the file system type.
- The fourth parameter indicates the mount flag. The default value is **0**.
- The last parameter indicates the private data. The default value is **NULL**.

In addition to calling the mount() function in the source code, you can also use the mount command in Shell for mounting. The first three parameters are the same as those of the mount() function, and the last two parameters do not need to be provided. If **mount ok** is displayed, the mounting is successful.

```
Huawei LiteOS# mount /dev/nandblk1 /yaffs1 yaffs
mount ok
```

After the mounting is successful, you can run the partition nand command in Shell to view the mount point information of the device node.

```
Huawei LiteOS# partition nand
nand partition num:0, dev name:/dev/nandblk0, mountpt:/yaffs0, startaddr:0x0900000,length:0x0200000
nand partition num:1, dev name:/dev/nandblk1, mountpt:/yaffs1, startaddr:0x0b00000,length:0x0200000
```

Calling umount to Unmount Partitions

Call the umount() function to unmount YAFFS2 partitions. Only the correct mount point needs to be specified. Alternatively, partitions can be unmounted by running the umount command in Shell. If **umount ok** is displayed, the command execution is successful.

```
Huawei LiteOS# umount /yaffs1
umount ok
```

```
Huawei LiteOS# umount /yaffs0
umount ok
```

After the unmounting is successful, you can run the partition nand command in Shell and view that the mount point of the device node becomes empty.

```
Huawei LiteOS# partition nand
nand partition num:0, dev name:/dev/nandblk0, mountpt:(null), startaddr:0x0900000,length:0x0200000
nand partition num:1, dev name:/dev/nandblk1, mountpt:(null), startaddr:0x0b00000,length:0x0200000
```

Calling delete_mtd_partition to Delete Partitions

Before calling delete_mtd_partition to delete a partition, you need to unmount the partition.

```
ret = delete_mtd_partition(1, "nand");
if (ret != 0) {
    printf("delete yaffs error\n");
} else {
    printf("delete yaffs ok\n");
}
```

5.6.3 Precautions

- YAFFS2 supports multiple partitions. The start address of partitions can be flexibly configured, but the addresses of partitions cannot overlap. You need to properly configure the space based on the flash usage to prevent conflict with the space of other file systems or space that is not suitable for mounting YFFAS2.
- The minimum partition that can be added is of the size of one block, but the minimum partition that can be mounted is of the size of 9 blocks (determined by the YAFFS2 features), which are two distinguished concepts.
- Before using a partition, you are advised to erase data in the partition.
- YAFFS2 in Huawei LiteOS only allows you to continuously open 20 directories.
- YAFFS2 supports the open, close, read, write, seek, sync, dup, opendir, closedir, readdir, rewinddir, statfs, unlink, mkdir, rmdir, rename, stat, stat64, and seek64 operations.
- When you use the open operation with the parameter **O_TRUNC** to open a file, the file content will be cleared only when you have the write permission on the file.
- Currently, YAFFS2 does not support the function of obtaining atime, mtime, and ctime of a file. The value obtained by using the stat API is 0.
- Currently, YAFFS2 does not support the open operation with the **O_DIRECTORY** attribute. Use opendir() to open a directory.
- YAFFS2 supports the read-only mounting attribute. If the input parameter of the mount function is set to **MS_RDONLY**, the read-only mounting attribute is enabled for YAFFS2. All APIs used to write data, such as write, mkdir, and unlink, and files that are opened by using the attribute other than **O_RDONLY** are rejected and the error code EACCES is returned.
- To open a file, o_flags must contain one of the O_RDONLY, O_WRONLY, and O_RDWR attributes. The read and write permissions set by o_flags must match the mode set when the file is created.

5.7 RAMFS

5.7.1 Overview

Basic Concepts

RAMFS is a RAM-based file system that can be dynamically resized. RAMFS does not have backup storage resources. When files are written into RAMFS, directory entries and page caches will be allocated, but data is not written to any other storage media. Data will be lost when the power is off.

RAMFS stores all files in RAM, and read or write operations are performed in RAM. RAMFS is generally used to store temporary data or data that needs to be frequently modified, such as the **/tmp** and **/var** directories. Using RAMFS reduces the read or write loss of the memory and improves the data read/write speed.

RAMFS in Huawei LiteOS is a simple file system:

- It provides a buffer for RAM-based dynamic file systems.
- It is based on VFS and cannot be formatted.

5.7.2 Development Guidelines

Development Procedure

To use the functions of RAMFS, perform the following operations:

1. Open the menu and choose **FileSystem > Enable RAMFS** to enable RAMFS.

Configuration Item	Description	Value Range	Default Value	Dependency
LOSCFG_FS_RAMFS	Whether to enable RAMFS	YES/NO	YES	LOSCFG_FS_VFS

2. [Mounting RAMFS](#)
3. [Unmounting RAMFS](#)

Mounting RAMFS

Call the mount() function to mount RAMFS to the mount point.

```
ret = mount((const char *)NULL, "/ramfs", "ramfs", 0, NULL);
if (ret) {
    dprintf("mount ramfs err %d\n", ret);
}
```

For details about the input parameters of the mount() function, see [Mounting FAT](#).

In addition to calling the mount() function in the source code, you can also use the mount command in Shell. The first three parameters are the same as those of the mount() function, and the last two parameters do not need to be provided. If the following information is displayed, the mounting is successful:

```
Huawei LiteOS# mount 0 /ramfs ramfs
mount ok
```

Unmounting RAMFS

Call the umount() function to unmount the partition. You only need to provide the correct mount point. You can also run the **umount** command in the shell to unmount the partition. If the following information is displayed, the partition is successfully unmounted:

```
Huawei LiteOS# umount /ramfs
umount ok
```

5.7.3 Precautions

- RAMFS is under debugging and enabled by default. Do not use it in formal products.
- RAMFS can be mounted only once. After it is mounted to a directory, it cannot be mounted to other directories.

- The number of RAMFS files is restricted by semaphore resources and cannot exceed the value of **LOSCFG_BASE_IPC_SEM_LIMIT**.
- RAMFS supports the open, close, read, write, seek, opendir, closedir, readdir, rewinddir, statfs, unlink, mkdir, rmdir, rename, stat, stat64, and seek64 operations.
- When you use **O_TRUNC** to open a file, the file content will be cleared.
- If you do not specify the value of the input parameter **mode** for the open API, you cannot use the **mode** input parameter to control the file permission.
- The read and write pointers are not separated. Therefore, after a file is opened in **O_APPEND** mode, the read pointer is also at the end of the file. You must manually set the position of the read pointer before reading the file.
- Disclaimer: Huawei shares no responsibilities for any risks caused in the use of RAMFS in commercial products.

5.8 PROC

5.8.1 Overview

Basic Concepts

PROC is a pseudo file system that exists only in memory and does not use external storage. It provides an interface for accessing the data in Huawei LiteOS Kernel.

PROC in Huawei LiteOS is a virtual file system and does not support multi-threaded operations.

5.8.2 Development Guidelines

API Description

Header File	API	Description
fs/include/ proc_fs.h	proc_fs_init	Initializes PROC.
	create_proc_entry	Creates a PROC file node.
	remove_proc_entry	Deletes a PROC node.
	proc_mkdir	Creates a PROC directory node.
	proc_create	Create a PROC node and assign a value to the operation function.

Development Procedure

To use the functions of PROC, perform the following operations:

1. Open the menu and choose **FileSystem > Enable PROC** to enable PROC.

Configuration Item	Description	Value Range	Default Value	Dependency
LOSCFG_FS_PRO C	Whether to enable PROC	YES/NO	YES	LOSCFG_FS_VFS

2. [Initializing PROC](#)
3. [Creating a PROC Node](#)
4. [Querying PROC Node Information](#)
5. [Modifying the PROC Node Information](#)
6. [Deleting a PROC Node](#)

Initializing PROC

Call the `proc_fs_init()` API to initialize PROC. If the following information is displayed when Huawei LiteOS is started, PROC is successfully initialized:

```
Mount procfs finished
```

Creating a PROC Node

- Call the `proc_mkdir` function to create a directory node. The first parameter specifies the name of the node to be created. The second parameter specifies the parent node of the node to be created. If **NULL** is passed in as the value of the second parameter, the parent node is `/proc` by default. This function will return the created PROC directory node.

```
struct proc_dir_entry *handle;

handle = proc_mkdir("test", NULL);
if (handle == NULL) {
    dprintf("create test error!\n");
    return;
}
```

- Call the `create_proc_entry` function to create a file node. The first parameter specifies the name of the node to be created. The second parameter specifies the file mode, including the file type and permission. The third parameter specifies the parent node of the node to be created. If **NULL** is passed in as the value of the third parameter, the parent node is `/proc` by default. If the parent node does not exist, call the `proc_mkdir` function to create a parent node directory. This function will return the created PROC file node.

```
struct proc_dir_entry *handle;

handle = create_proc_entry("mounts", 0, NULL);
if (handle == NULL) {
    dprintf("create mounts error!\n");
    return;
}
```

- After a node is created, you can assign an operation function to it. For example:

```
handle->proc_fops = &mounts_proc_fops;
```

The default operation function is called if no operation function is assigned.

Querying PROC Node Information

Run the **cat** command in shell to view the node information. The following is an example:

```
Huawei LiteOS # cat /proc/test
Huawei LiteOS # -----LOG BUFFER STATE-----
MaxLen ReadPos WritePos ButtPos
64(KB)    0    113  65536

-----CURRENT LOG LEVEL-----
vb : 3
sys : 3
region : 3
chnl : 3
vpss : 3
venc : 3
vda : 3
h264e : 3
jpege : 3
vou : 3
viu : 3
rc : 3
aio : 3
ai : 3
ao : 3
aenc : 3
adec : 3
isp : 3
ive : 3
tde : 3
vgs : 3
h265e : 3
```

Modifying the PROC Node Information

Run the **writeproc** command in shell to view the node information. The following is an example:

```
Huawei LiteOS # writeproc 'sys=2' >> /proc/test
```

After the sys level is changed, information similar to following is displayed:

```
sys=2 >> /proc/test
```

Run the **cat** command again to query node information. From the displayed node information, you will see that the sys level has been changed.

```
Huawei LiteOS # cat /proc/test
Huawei LiteOS # -----LOG BUFFER STATE-----
MaxLen ReadPos WritePos ButtPos
64(KB)    0    0  65536

-----CURRENT LOG LEVEL-----
vb : 3
sys : 2
region : 3
chnl : 3
vpss : 3
venc : 3
vda : 3
h264e : 3
jpege : 3
vou : 3
viu : 3
rc : 3
aio : 3
```



```
ai : 3
ao : 3
aenc : 3
adec : 3
isp : 3
ive : 3
tde : 3
vgs : 3
h265e : 3
```

Deleting a PROC Node

Call the `remove_proc_entry()` function to delete a node. The first parameter specifies the name of the node to be deleted, and the second parameter specifies the parent node of the node to be deleted. If **NULL** is passed in as the value of the second parameter, the parent node is `/proc` by default.

5.8.3 Precautions

- PROC is under debugging and enabled by default. Do not use it in formal products.
- In PROC, a file name or a directory name contains a maximum of 32 characters.
- `proc_fs_init()` can be called only once. If it is called for multiple times, PROC will be abnormal.
- If the `ls` command is run to query files in PROC after PROC is initialized, the size of each file is displayed as zero. The file size can be correctly displayed only after the `cat` command is run to retrieve real-time kernel information.
- PROC in Huawei LiteOS cannot be unmounted and does not support creating and deleting files or directories.
- PROC in Huawei LiteOS supports the `open`, `close`, `read`, `write`, `seek`, `opendir`, `closedir`, `readdir`, `rewinddir`, `statfs`, `unlink`, `stat`, `stat64`, and `seek64` operations.
- If you do not specify the value of the input parameter **mode** for the `open` API, you cannot use the **mode** input parameter to control the file permission.
- In the `open` command that is used to open files in PROC, only `read` and `write` properties take effect.
- In the `fseek` function, `SEEK_END` does not take effect, indicating that the offset started from the end of a file is not supported.
- Disclaimer: Huawei shares no responsibilities for any risks caused in the use of PROC in commercial products.

5.8.4 Programming Example

Example Description

Create the **proc_example** file in the `/proc` directory. You can perform the `open`, `read`, `write`, or `seek` operation on the file.

1. Call `proc_example_init()` to initialize the PROC, create a **proc_example** file in the `/proc` directory, and define the operation functions of the **proc_example** file.

- When the file is opened, execute the `example_proc_open()` function and register `example_proc_show()` in the function. The `example_proc_show()` function defines the content that can be read when the file is read.
 - Execute the `seq_read()` function when the file is read.
2. Call `proc_test()` to open and read the **/proc/proc_example** file. In this case, `seq_read()` executes `example_proc_show()`, copies the content in the **seq_file** file to buffer, and returns the contents through buffer.

Example Code

```
#include "proc_fs.h"
#include "linux/seq_file.h"

#include "stdio.h"
#include "stdlib.h"
#include "fcntl.h"

static int example_proc_show(struct seq_file *m, void *v)
{
    PRINTK("this is example_proc_show\n");
    seq_printf(m, "%s%s\n", "my test proc file1!", "f");
    return 0;
}

static int example_proc_open(struct inode *inode_ptr, struct proc_file *pf)
{
    PRINTK("this is example_proc_open\n");
    return single_open(pf, example_proc_show, NULL);
}

static const struct proc_file_operations example_proc_fops = {
    .open      = example_proc_open,
    .read      = seq_read,
    .llseek    = seq_lseek,
    .release   = seq_release,
    .write     = NULL,
};

void proc_example_init(void)
{
    struct proc_dir_entry *handler;

    PRINTK("proc fs init ...\n");
    proc_fs_init();
    handler = create_proc_entry("proc_example", 0, (struct proc_dir_entry *)NULL);
    if (handler == NULL) {
        PRINT_ERR("creat mounts error!\n");
        return;
    }
    PRINTK("proc_example_init end\n");

    handler->proc_fops = &example_proc_fops;
}

void proc_test(void)
{
    char buf[50] = {0};
    int fd = open("/proc/proc_example", O_NONBLOCK | O_CREAT | O_RDWR, 0777);
    if (fd < 0) {
        PRINTK("fd open err\n");
        return;
    }

    int len = read(fd, buf, 50);
    PRINTK("read data:%s\n", buf);
}
```

```
close(fd);  
}
```

Verification

```
proc fs init ...  
Mount procfs finished.  
proc_example_init end  
this is example_proc_open  
this is example_proc_show  
read data:my test proc file1!f
```

Complete Code

proc_test.c

5.9 ROMFS

5.9.1 Overview

ROM file system (ROMFS) is a read-only file system. It provides simple functions and occupies less system resources. It does not support file writing but supports only file reading. ROMFS is designed to provide a more space-saving file system than a common file system in the boot disk scenario.

Huawei LiteOS ROMFS supports the SD card or NOR flash. When creating a ROMFS, you need to use genromfs. In the Linux OS, you can use this tool to create a ROMFS image from a directory and save the image to the SD card or USB flash drive, or burn the image to the NOR flash, and then mount the image to Huawei LiteOS to read the content.

5.9.2 Development Guidelines

Development Procedure

To use the ROMFS, perform the following steps:

1. Open the menu and choose **FileSystem > Enable ROMFS** to enable ROMFS.

Configuration Item	Description	Value Range	Default Value	Dependency
LOSCFG_FS_ROMFS	Whether to enable ROMFS	YES/NO	YES	LOSCFG_FS_VFS

2. [Creating a ROMFS Image](#)
3. [Burning the ROMFS Image](#)
4. [Mounting ROMFS](#)
5. [Unmounting ROMFS](#)

Creating a ROMFS Image

- SD card or USB flash drive

Insert an SD card or a USB flash drive into a computer with the Linux OS. A device (for example, **/dev/sda1**) is added in the **/dev/** directory after the Linux OS identifies the SD card or the USB flash drive. Create a directory in Linux system and copy a file that needs to be made into ROMFS to the directory. Use the `genromfs` tool to create a ROMFS image from the directory. Assume that the directory name is **romfs_dir** and the device name is **/dev/sda1**.

```
sudo genromfs -d romfs_dir -f /dev/sda1
```

- NOR Flash

Create a directory on a computer with the Linux OS and copy a file that needs to be made into ROMFS to the directory. Use `genromfs` to make the directory into the ROMFS image. Assume that the directory name is **romfs_dir** and the image name is **romfs.img**.

```
sudo genromfs -d romfs_dir -f romfs.img
```

Burning the ROMFS Image

- SD card or USB flash drive

Insert the SD card or USB flash drive into the development board.

- NOR Flash

Burn the created ROMFS image to the NOR flash.

Mounting ROMFS

- SD card or USB flash drive

After the SD card or USB flash drive is inserted, Huawei LiteOS automatically identifies the device, for example, **/dev/mmcbk0p0**. Run the following command in Shell to mount the SD card or USB flash drive as ROMFS:

```
Huawei LiteOS# mount /dev/mmcbk0p0 /romfs romfs
```

If the following information is displayed, the mounting is successful:

```
mount ok
```

- NOR Flash

Call the `add_mtd_partition` function to add a partition. This function automatically names the device node. The naming rule is **/dev/spinorblk** plus the partition number. Note that the start address and length of the partition must be the same as those when the ROMFS image is burnt. Details are as follows:

```
if (ret = add_mtd_partition("spinor", 0x100000, 0x500000, 0) != 0) {  
    dprintf("add partition failed, return %d\n", ret);  
}
```

After adding a partition, you can identify the NOR Flash partition on Huawei LiteOS, such as **/dev/spinorblk0**, and run the mount command to mount ROMFS:

```
Huawei LiteOS# mount /dev/spinorblk0 /romfs romfs
```

If the following information is displayed, the mounting is successful:

```
mount ok
```

Unmounting ROMFS

Run the following command:

```
Huawei LiteOS# umount /romfs
```

If the following information is displayed, the unmounting is successful.

```
umount ok
```

5.9.3 Precautions

- The maximum size of a ROMFS is 2 GB.
- The maximum size of a single file in a ROMFS is 256 MB.
- The length of each directory name or file name in a ROMFS cannot exceed 16 bytes.
- A file can be opened only when the parameter is **O_RDONLY**.
- If you do not specify the value of the input parameter **mode** for the open API, you cannot use the **mode** input parameter to control the file permission.
- ROMFS supports the open, close, read, seek, dup, opendir, closedir, readdir, rewinddir, statfs, stat, stat64, and seek64 operations.
- A ROMFS supports hard links instead of soft links.
- If the fread API is used for the read operation, the buffer will be turned on by default and 512 bytes are read each time, which may cause a slow read rate. The setvbuf API can be used to set the no buffer mode, and the third parameter should be **_IONBF**.
- A ROMFS can be used in SD cards, USB flash drives, or NOR Flash. By default, a ROMFS is mounted to NOR Flash. Currently, only source code configuration is supported.

5.10 LITTLEFS

5.10.1 Overview

Basic Concepts

LITTLE file system (LITTLEFS) is a fault-tolerant file system specially designed for microcontrollers. It is mainly used in NOR flash memory and has the following features:

- Readable and writable.
- Provides loss balance.
- Provides power failure protection. When data in the file system is modified, data is not damaged even if an accident occurs.
- The memory and code are simplified, occupying a small amount of RAM and flash memory.

LITTLEFS is a block-based file system. Disks are divided into block arrays of even sizes as logical storage units. These blocks are allocated by the public block

allocator when being used. LITTLEFS uses small logs to store metadata and a larger COW structure to store file data.

The LITTLEFS of Huawei LiteOS is mainly used for file management of the NOR flash memory and supports multiple partitions.

5.10.2 Development Guidelines

API Description

Header File	API	Description
fs/include/ mtd_partition. h	add_mtd_partition	Adds a file system partition. This function automatically names the device node. For LittleFs, the naming rule is /dev/spinorblk0 .
	delete_mtd_partition	Deletes the partition that has been unmounted.
open_source/ musl/ include/sys/ mount.h	mount	Mounts the file system.
	umount	Unmounts the file system.

Development Procedure

1. [Enabling LITTLEFS](#)
2. [Adding a LittleFs Partition](#)
3. [Mounting a LittleFs Partition](#)
4. [Unmounting the LittleFs Partition](#)
5. [Deleting a LittleFs Partition](#)

Enabling LITTLEFS

Open the menu and choose **FileSystem** > **Enable LITTLEFS** to enable LITTLEFS.

Configuration Item	Description	Value Range	Default Value	Dependency
LOSCFG_FS_LITTLEFS	Determines whether to enable LittleFs.	YES/NO	YES	LOSCFG_FS_VFS && LOSCFG_DRIVERS_MTD_SPI_NOR

Adding a LittleFs Partition

Call the **add_mtd_partition()** function to add a LittleFs partition.

```
if (add_mtd_partition("spinor", 0x100000, 0x500000, 0) != 0) {
    dprintf("add spinor partition failed\n");
}
```

NOTE

When the LittleFs partition is added by running **add_mtd_partition()**, the system automatically aligns the start address and size of the partition based on the block size of the device.

This function has four parameters:

- The first parameter indicates the media (nand or spinor). The LittleFs partition is used on the SPI NOR flash.
- The second parameter indicates the start address of a partition, in hexadecimal format.
- The third parameter indicates the partition size in hexadecimal format.
- The last parameter indicates the partition number. The value ranges from 0 to 19.

After the partitions are successfully added, you can run the **partition spinor** command in shell to query information about the SPI NOR flash partition. For details, see [partition](#).

```
Huawei LiteOS# partition spinor
spinor partition num:0, dev name:/dev/spinorblk0, mountpt:(null), startaddr:0x0100000, length:0x0500000
```

Mounting a LittleFs Partition

Call the **mount()** function to mount the device node.

```
ret = mount("/dev/spinorblk0", "/littlefs0", "littlefs", 0, NULL);
if (ret != 0) {
    dprintf("ERROR:mount littlefs failed\n");
}
```

NOTE

The **mount()** function has five parameters:

- The first parameter indicates the device node. This parameter must correspond to the **add_mtd_partition()** function.
- The second parameter indicates the mount point.
- The third parameter indicates the file system type.
- The fourth parameter indicates the mount flag. The default value is **0**.
- The last parameter indicates the private data. The default value is **NULL**.

To check whether the **mount()** function is successfully called, run the **ls** command in Shell to view the directories in the root directory.

```
Huawei LiteOS# ls
Directory /:
bin          <DIR>
dev          <DIR>
littlefs0    <DIR>
```

After the mounting is successful, you can read and write the NOR flash.

Unmounting the LittleFs Partition

Call the **umount()** function to unmount the partition. You only need to provide the correct mount point.

```
ret = umount("/littlefs0");
if (ret != 0) {
    dprintf("umount littlefs err\n");
}
```

NOTE

The **umount()** function has only one parameter indicating the mount point.

Deleting a LittleFs Partition

Call the `delete_mtd_partition` function to delete the unmounted JFFS2 partitions.

```
delete_mtd_partition(0, "spinor");
```

NOTE

The **delete_mtd_partition()** function has two parameters:

- The first parameter indicates the partition number.
- The second parameter is the media type. This parameter corresponds to the **add_mtd_partition()** function.

5.10.3 Precautions

- If a file is opened in **O_TRUNC** mode, the content in the file is cleared.
- If you do not specify the value of the input parameter **mode** for the open API, you cannot use the **mode** input parameter to control the file permission.
- The read and write pointers of the LittleFs file system are separated. After a file is opened in **O_APPEND** (additional write) mode, the read pointer is at the beginning of the file and the write pointer is at the end of the file.
- The LittleFs file system is used for the NOR flash and finally calls the NOR flash driver API. Therefore, before using the LittleFs file system, ensure that the NOR flash exists and the driver has been initialized successfully (the return value of `spinor_init()` is 0).
- Currently, LittleFs supports the open, close, read, write, seek, opendir, closedir, readdir, rewinddir, unlink, mkdir, rmdir, rename, stat, stat64, and seek64 operations.
- The default cache size is 1024 bytes. You can change the value of **LFS_CACHE_SIZE** in the `fs/littlefs/include/vfs_littlefs.h` file. The value must be a multiple of **LFS_MIN_CACHE_SIZE** and cannot be greater than 65536 bytes.
- When a file is opened and data is written to the file, if the file is renamed before it is closed, the data will be lost.
- Currently, the access operation can only determine whether a file exists but cannot determine whether the file has the specified permission.
- Currently, the stat operation can only obtain the file name, size, and file type (file or directory), but cannot obtain the file permission.
- Currently, LittleFs does not support the open operation with the **O_DIRECTORY** attribute. To open a directory, use **opendir()**.
- If you want to open the same file at the same time, you must open the file in read-only mode. If a file is opened in writable mode and data is written to the file, the file can be saved only after being closed. Only the data that is closed last time is saved.

- LittleFs supports the mounting of read-only attributes. When the input parameter **mountflags** of the mount function is set to **MS_RDONLY**, the read-only attribute is enabled for LittleFs. All APIs with write operations, such as write, mkdir, and unlink, and files opened using non-**O_RDONLY** attributes, all requests will be rejected and the EACCESS error code will be sent.

6 Compatible APIs

6.1 Overview

6.2 POSIX APIs

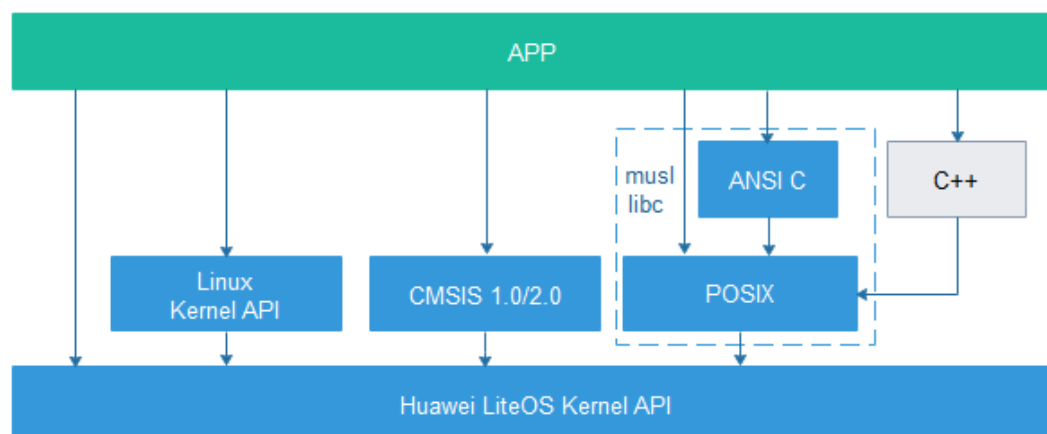
6.3 Libc/Libm APIs

6.4 C++ Compatibility Specifications

6.5 CMSIS APIs

6.6 Adaption to Linux

6.1 Overview



Huawei LiteOS provides a set of OS APIs and supports the musl libc library, C++ standard library, CMSIS APIs, and Linux APIs. It is recommended that upper-layer applications call compatible APIs such as the musl libc library. It is not recommended that upper-layer applications directly call Huawei LiteOS's own APIs. The following sections list the compatible APIs supported by Huawei LiteOS.

CAUTION

The libc library function adapted to LiteOS complies with the open-source musl implementation of libc. The musl open-source code complies with the C programming language standard ISO/IEC 9899:1999 and POSIX standard 1003.1™-2008. This simplest bottom-layer library function does not check whether the pointer type parameter is null. The user ensures the validity of the input parameter.

The time obtaining APIs implemented by LiteOS start timing since the system scheduling, which is inconsistent with the interface standard definition. Pay special attention to the differences.

6.2 POSIX APIs

6.2.1 Supported POSIX APIs

Huawei LiteOS supports a set of POSIX APIs. The following table lists their specifications.

Header File	API	Type	Description
arpa/inet.h	inet_addr	Network function	Converts an IP address from a character string to a number.
arpa/inet.h	inet_ntoa	Network function	Converts a binary network number into a network address.
arpa/inet.h	inet_ntop	Network function	Converts a binary integer to a dotted decimal notation.
arpa/inet.h	inet_pton	Network function	Converts a dotted decimal notation to a binary integer.
ctype.h	isalnum_l	Data identification	Determines whether the input parameter is a letter or digit.
ctype.h	isalpha_l	Data identification	Determines whether the input parameter is a letter.

Header File	API	Type	Description
ctype.h	isascii	Data identification	Determines whether the input parameter is the ASCII code.
ctype.h	isblank	Data identification	Determines whether the input parameter is a space or tab.
ctype.h	isblank_l	Data identification	Determines whether the input parameter is a space or tab.
ctype.h	iscntrl_l	Data identification	Determines whether the input parameter is a control character.
ctype.h	isdigit_l	Data identification	Determines whether the input parameter is a decimal number.
ctype.h	isgraph_l	Data identification	Determines whether the input parameter is a graphic character.
ctype.h	islower_l	Data identification	Determines whether the input parameter contains lowercase letters.
ctype.h	isprint_l	Data identification	Determines whether the input parameter can be printed.
ctype.h	ispunct_l	Data identification	Determines whether the input parameter is a punctuation mark.
ctype.h	isspace_l	Data identification	Determines whether the input parameter is a space.
ctype.h	isupper_l	Data identification	Determines whether the input parameter contains uppercase letters.
ctype.h	isxdigit_l	Data identification	Determines whether the input parameter is a hexadecimal number.
ctype.h	toascii	Data conversion function	Converts characters to ASCII codes.

Header File	API	Type	Description
ctype.h	tolower_l	Data conversion function	Converts uppercase letters to lowercase letters.
ctype.h	toupper_l	Data conversion function	Converts lowercase letters to uppercase letters.
dirent.h	alphasort	Directory operation	Sorts the directory structure in alphabetical order.
dirent.h	closedir	Directory operation	Closes a directory.
dirent.h	opendir	Directory operation	Opens a directory.
dirent.h	readdir	Directory operation	Reads a directory.
dirent.h	rewinddir	Directory operation	Resets the position of a directory to be read to the beginning position.
dirent.h	scandir	Directory operation	Scans the contents of the selected directory.
dirent.h	seekdir	Directory operation	Sets the position of the directory to be read next time.
dirent.h	telldir	Directory operation	Obtains the read position on a directory stream.

Header File	API	Type	Description
dlfcn.h	dlclose	Dynam ic loadin g	Unloads a module.
dlfcn.h	dlopen	Dynam ic loadin g	Dynamically loads a SO module.
dlfcn.h	dlsym	Dynam ic loadin g	Searches for symbols in the module or system symbol table.
fcntl.h	creat	File operati on functio n	Creates a file.
fcntl.h	fcntl	File operati on functio n	Performs operations on the file descriptor.
fcntl.h	open	File operati on functio n	Opens a file.
langinfo.h	nl_langinfo	System functio n	Returns the specified local information.
langinfo.h	nl_langinfo_l	System functio n	Returns the specified local information.
libgen.h	basename	File path	Returns the file name.
mqueue.h	mq_close	Messa ge queue	Closes a message queue.
mqueue.h	mq_getattr	Messa ge queue	Obtains the message queue attribute.

Header File	API	Type	Description
mqqueue.h	mq_open	Message queue	Opens a message queue.
mqqueue.h	mq_receive	Message queue	Receives messages from a message queue.
mqqueue.h	mq_send	Message queue	Sends a message to a message queue.
mqqueue.h	mq_setattr	Message queue	Sets the message queue attribute.
mqqueue.h	mq_timedreceive	Message queue	Receives messages at a specified time.
mqqueue.h	mq_timedsend	Message queue	Sends messages at a specified time.
mqqueue.h	mq_unlink	Message queue	Removes a message queue.
net/if.h	if_freenameindex	Network function	Releases the dynamically allocated memory area after obtaining the API name and index through if_nameindex() .
net/if.h	if_indextoname	Data conversion function	Converts the network adapter serial number to the network adapter name.
net/if.h	if_nameindex	Network function	Returns a dynamically allocated struct if_nameindex structure array. Each element in the array corresponds to a local network interface.
net/if.h	if_nametoindex	Data conversion function	Converts the network adapter name to the network adapter serial number.

Header File	API	Type	Description
netdb.h	freeaddrinfo	Network function	Releases the memory space allocated by getaddrinfo.
netdb.h	getaddrinfo	Network function	Converts the host name and service name to the socket address.
netdb.h	getnameinfo	Network function	Converts the socket address to the host name and service name.
poll.h	poll	File operation function	Monitors file descriptors.
pthread.h	pthread_attr_destroy	pthread_d	Deletes a thread attribute.
pthread.h	pthread_attr_get_detachstate	pthread_d	Obtains the detached state.
pthread.h	pthread_attr_get_inheritsched	pthread_d	Obtains the task scheduling mode.
pthread.h	pthread_attr_get_schedparam	pthread_d	Obtains task scheduling parameters.
pthread.h	pthread_attr_get_schedpolicy	pthread_d	Obtains the task scheduling policy. Currently, Huawei LiteOS supports only the SCHED_RR policy and does not support SCHED_OTHER and SCHED_FIFO SCHED_RR.
pthread.h	pthread_attr_get_scope	pthread_d	Obtains the task scope. Currently, Huawei LiteOS supports only PTHREAD_SCOPE_SYSTEM and does not support PTHREAD_SCOPE_PROCESS.
pthread.h	pthread_attr_get_stackaddr	pthread_d	Obtains the start address of a task stack.
pthread.h	pthread_attr_get_stacksize	pthread_d	Obtains the size of a task stack.
pthread.h	pthread_attr_init	pthread_d	Initializes task attributes.

Header File	API	Type	Description
pthread.h	pthread_attr_setdetachstate	pthread_t	Sets the detach state of a task.
pthread.h	pthread_attr_setinheritsched	pthread_t	Sets the task scheduling mode.
pthread.h	pthread_attr_setschedparam	pthread_t	Sets task scheduling parameters. In Huawei LiteOS, a smaller value indicates a higher priority, which is opposite to implementation of the standard library function. In addition, the priority ranges from 0 to 31. The inheritsched field in pthread_attr_t must be set to PTHREAD_EXPLICIT_SCHED . Otherwise, the configured task scheduling priority will not take effect. The default value is PTHREAD_INHERIT_SCHED .
pthread.h	pthread_attr_setschedpolicy	pthread_t	Sets the task scheduling policy. Currently, Huawei LiteOS supports only the SCHED_RR policy and does not support SCHED_OTHER and SCHED_FIFO/SCHED_RR.
pthread.h	pthread_attr_setscope	pthread_t	Sets the task scope. Currently, Huawei LiteOS supports only PTHREAD_SCOPE_SYSTEM and does not support PTHREAD_SCOPE_PROCESS.
pthread.h	pthread_attr_setstackaddr	pthread_t	Sets the start address of a task stack.
pthread.h	pthread_attr_setstacksize	pthread_t	Sets the size of a task stack.
pthread.h	pthread_cancel	pthread_t	Cancels a task. Sets the PTHREAD_CANCEL_ASYNCHRONOUS status before calling pthread_cancel to cancel a task.
pthread.h	pthread_cond_broadcast	pthread_t	Wakes up all the threads blocked on a condition variable.
pthread.h	pthread_cond_destroy	pthread_t	Releases a condition variable.
pthread.h	pthread_cond_init	pthread_t	Initializes a condition variable.

Header File	API	Type	Description
pthread.h	pthread_cond_signal	pthread_t	Releases a thread that is blocked on a condition variable.
pthread.h	pthread_cond_timedwait	pthread_t	Waits for a condition variable within the timeout period. If the timeout period is relative time, LiteOS cannot process the timeout situation.
pthread.h	pthread_cond_wait	pthread_t	Waits for a condition variable.
pthread.h	pthread_condattr_destroy	pthread_t	Deletes a storage and invalidate attributes.
pthread.h	pthread_condattr_getclock	pthread_t	Obtains a task clock.
pthread.h	pthread_condattr_getpshared	pthread_t	Obtains the shared attribute of a condition variable. Huawei LiteOS supports only PTHREAD_PROCESS_PRIVATE.
pthread.h	pthread_condattr_init	pthread_t	Initializes the attributes of a condition variable.
pthread.h	pthread_condattr_setclock	pthread_t	Sets the task clock. Only the CLOCK_MONOTONIC or CLOCK_REALTIME mode is supported.
pthread.h	pthread_condattr_setpshared	pthread_t	Sets the condition variable attribute. Only the PTHREAD_PROCESS_PRIVATE attribute is supported.
pthread.h	pthread_create	pthread_t	Creates a task.
pthread.h	pthread_detach	pthread_t	Detaches a task.
pthread.h	pthread_equal	pthread_t	Determines whether tasks are the same.
pthread.h	pthread_exit	pthread_t	Exits a task.
pthread.h	pthread_getschedparam	pthread_t	Obtains the task priority and scheduling policy. Currently, Huawei LiteOS supports only the SCHED_RR policy and does not support SCHED_OTHER and SCHED_FIFO SCHED_RR.

Header File	API	Type	Description
pthread.h	pthread_getspecific	pthread_t	Gets the key binding of the calling thread.
pthread.h	pthread_join	pthread_t	Blocks a task.
pthread.h	pthread_key_create	pthread_t	Allocates keys that identify thread-specific data in a process.
pthread.h	pthread_key_delete	pthread_t	Destroys an existing thread-specific data key.
pthread.h	pthread_mutex_destroy	pthread_t	Deletes a mutex.
pthread.h	pthread_mutex_getprioceiling	pthread_t	Obtains the upper limit of mutex priorities.
pthread.h	pthread_mutex_init	pthread_t	Initializes a mutex.
pthread.h	pthread_mutex_lock	pthread_t	Allocates a mutex (blocked).
pthread.h	pthread_mutex_setprioceiling	pthread_t	Sets the upper limit of mutex priorities.
pthread.h	pthread_mutex_timedlock	pthread_t	Allocates a mutex (blocked only within the specified period).
pthread.h	pthread_mutex_trylock	pthread_t	Attempts to allocate a mutex (unblocked).
pthread.h	pthread_mutex_unlock	pthread_t	Releases a mutex.
pthread.h	pthread_mutexattr_destroy	pthread_t	Destroys the mutex attribute object.
pthread.h	pthread_mutexattr_getprioceiling	pthread_t	Obtains the upper limit of mutex priorities.
pthread.h	pthread_mutexattr_getprotocol	pthread_t	Obtains the protocol of a mutex.
pthread.h	pthread_mutexattr_gettype	pthread_t	Obtains the type of a mutex.
pthread.h	pthread_mutexattr_init	pthread_t	Initializes mutex attributes.
pthread.h	pthread_mutexattr_setprioceiling	pthread_t	Sets the upper limit of mutex priorities.

Header File	API	Type	Description
pthread.h	pthread_mutexattr_setprotocol	pthread_t	Sets the protocol of a mutex.
pthread.h	pthread_mutexattr_settype	pthread_t	Sets the type of a mutex.
pthread.h	pthread_once	pthread_t	Operates a task once.
pthread.h	pthread_self	pthread_t	Obtains a task ID.
pthread.h	pthread_setcancelstate	pthread_t	Enables task cancellation.
pthread.h	pthread_setcanceltype	pthread_t	Sets the cancellation type.
pthread.h	pthread_setschedparam	pthread_t	Sets the task priority and scheduling policy. Currently, Huawei LiteOS supports only the SCHED_RR policy and does not support SCHED_OTHER and SCHED_FIFO. The task priority ranges from 0 to 31.
pthread.h	pthread_setschedprio	pthread_t	Sets the task priority. The task priority ranges from 0 to 31.
pthread.h	pthread_setspecific	pthread_t	Configures thread data.
pthread.h	pthread_testcancel	pthread_t	Cancels a task.
sched.h	sched_get_priority_max	Scheduling function	Obtains the highest priority supported by the system.
sched.h	sched_get_priority_min	Scheduling function	Obtains the lowest priority supported by the system.
sched.h	sched_yield	Scheduling function	Makes the current thread relinquish the CPU.
semaphore.h	sem_destroy	Semaphore	Destroys an unnamed semaphore.
semaphore.h	sem_getvalue	Semaphore	Obtains the value of a specified semaphore.

Header File	API	Type	Description
semaphore.h	sem_init	Semaphore	Initializes an unnamed semaphore.
semaphore.h	sem_post	Semaphore	Releases an unnamed semaphore.
semaphore.h	sem_timedwait	Semaphore	Allocates an unnamed semaphore for timeout waiting. If the timeout period is relative time, LiteOS cannot process the timeout situation.
semaphore.h	sem_trywait	Semaphore	Attempts to allocate an unnamed semaphore.
semaphore.h	sem_wait	Semaphore	Allocates an unnamed semaphore.
stdio.h	dprintf	Standard I/O	Prints information.
stdio.h	fdopen	File operation function	Associates a standard I/O stream with an existing file descriptor.
stdio.h	fileno	File operation function	Obtains the file descriptor used by the file stream specified by stream . If the file stream is closed, -1 is returned and an error code is set.
stdio.h	fseeko	File operation function	Changes the read/write position on a file stream.
stdio.h	ftello	File operation function	Obtains the read position on a file stream.
stdlib.h	initstate	Random number function	Initializes the random number generator state.

Header File	API	Type	Description
stdlib.h	lrand48	Random number function	Obtains a positive random number of the long integer type.
stdlib.h	mkstemp	File processing function	Generates a unique temporary file.
stdlib.h	rand48	Random number function	Returns a positive random number.
stdlib.h	posix_memalign	Memory operation	Allocates and aligns memory, and returns a memory pointer.
stdlib.h	random	Random number function	Returns a random number in the range of 0 to RAND_MAX .
stdlib.h	realpath	File processing function	When a relative path is converted to an absolute path, the length of the value of resolved_path must be greater than or equal to the value of PATH_MAX .
stdlib.h	seed48	Random number function	Sets a 48-bit random number seed.
stdlib.h	setstate	Random number function	Sets the random number state.

Header File	API	Type	Description
stdlib.h	srand48	Random number function	Sets a 48-bit random number seed.
stdlib.h	srandom	Random number function	Generates a random number seed.
string.h	strcpy	String processing function	Assigns a string to an array.
string.h	strncpy	String processing function	Assigns a string to an array.
string.h	strcoll_l	String processing function	Compares two strings based on LC_COLLATE .
string.h	strdup	String processing function	Copies a string into a newly allocated string.
string.h	strerror_l	String processing function	Returns a pointer to an error message string.
string.h	strerror_r	String processing function	Returns a pointer to an error message string. It is reentrant.

Header File	API	Type	Description
string.h	strlen	String processing function	Returns the length of a string in bytes.
string.h	strtok_r	String processing function	Splits a string.
string.h	strxfrm_l	String processing function	Converts the character string src based on LC_COLLATE. If the length of n is less than that of src , the length of src is returned without conversion.
strings.h	ffs	Bitwise operation function	Searches for the first bit value in an integer.
strings.h	strcasecmp	String processing function	Compares two strings.
strings.h	strcasecmp_l	String processing function	Compares two strings.
strings.h	strncasecmp	String processing function	Compares two strings.
strings.h	strncasecmp_l	String processing function	Compares two strings.

Header File	API	Type	Description
sys/ioctl.h	ioctl	File operation function	Performs the specified operation.
sys/select.h	select	File operation function	Monitors file descriptors. Ensure that the number of file descriptors to be monitored is less than the positive value of FD_SETSIZE . Otherwise, undefined behavior may occur.
sys/socket.h	accept	Network function	Accepts a connection request on a socket.
sys/socket.h	bind	Network function	Binds a socket to an address.
sys/socket.h	connect	Network function	Initiates a connection between two sockets.
sys/socket.h	getpeername	Network function	Gets the foreign protocol address associated with a socket.
sys/socket.h	getsockname	Network function	Gets the name of a socket.
sys/socket.h	getsockopt	Network function	Obtains the state of a socket.
sys/socket.h	listen	Network function	Changes the socket to the listening state and waits for the connection.
sys/socket.h	recv	Network function	Receives messages from a connected socket.

Header File	API	Type	Description
sys/socket.h	recvfrom	Network function	Receives messages from a socket (connected or not).
sys/socket.h	recvmsg	Network function	Receives messages through the specified socket.
sys/socket.h	send	Network function	Sends messages through a socket.
sys/socket.h	sendmsg	Network function	Sends messages through the specified socket.
sys/socket.h	sendto	Network function	Sends messages through the specified socket. If the destination is connected, messages are directly sent to the destination. If the destination is not connected, messages are sent to the specified socket.
sys/socket.h	setsockopt	Network function	Sets the state of a socket.
sys/socket.h	shutdown	Network function	Shuts down the connection of a socket.
sys/socket.h	socket	Network function	Creates a socket.
sys/stat.h	fstat	File operation API	Obtains file attributes.
sys/stat.h	lstat	File operation API	Obtains file attributes.

Header File	API	Type	Description
sys/stat.h	mkdir	Directory operation API	Creates a directory.
sys/stat.h	stat	File operation API	Obtains file attributes.
sys/time.h	gettimeofday	Time function	Obtains the current calendar time, storing it as seconds and microseconds.
sys/times.h	times	Time function	Obtains CPU ticks to a buff.
sys/uio.h	readv	File operation function	Reads a file.
sys/uio.h	writew	File operation function	Writes a file.
sys/utsname.h	uname	System function	Obtains system information. The LOSCFG_LIB_VENDORNAME option can be configured to change the nodename member of the returned value. The hardware identifier and domain name information are not returned.
syslog.h	syslog	System function	Outputs system logs. The first input parameter cannot output logs by level.
time.h	asctime_r	Time function	Represents the time and date as a character string.
time.h	clock_getres	Time function	Obtains the time accuracy provided by the corresponding clock type.
time.h	clock_gettime	Time function	Retrieves the time of a specified clock.

Header File	API	Type	Description
time.h	clock_gettime	Time function	Sets the time of a specified clock.
time.h	ctime_r	Time function	Represents the time and date as a character string.
time.h	gmtime_r	Time function	Obtains the current date and time.
time.h	localtime_r	Time function	Obtains the current local date and time.
time.h	nanosleep	Time function	Sets hibernation time for processes, which is usually measured by nanoseconds. However, in Huawei LiteOS, the time unit is tick (10 ms by default). The second argument in this API is not supported, and the first argument (number of seconds) cannot be greater than 4292 seconds.
time.h	strptime_l	Time function	Formats the date and time. The format character does not support the format conversion of %z or %Z.
time.h	strptime	Time function	Parses the input string according to the format string.
time.h	timer_create	Time function	Creates a timer. SIGEV_THREAD must be processed within threads. (Huawei LiteOS has two implementation modes: processing the timer callback function in a thread or in an interrupt. This mode is set in LOSCFG_BASE_CORE_SWTMR_IN_ISR.) The value of clock_id is fixed at CLOCK_REALTIME .
time.h	timer_delete	Time function	Deletes a timer.
time.h	timer_getoverrun	Time function	Obtains the number of times that a timer has timed out.

Header File	API	Type	Description
time.h	timer_gettime	Time function	Obtains the time left in a timer.
time.h	timer_settime	Time function	Initializes or cancels the timer. When <i>it_interval</i> is set to 0 , the timer needs to be manually deleted after being used.
unistd.h	access	File operation function	Checks file permissions.
unistd.h	chdir	Directory operation API	Changes the current working directory.
unistd.h	close	File operation API	Closes a file.
unistd.h	dup	File operation function	Copies a file.
unistd.h	dup2	File operation function	Copies a file.
unistd.h	fsync	File operation function	Synchronizes a file.
unistd.h	ftruncate	File operation function	Changes a file size.
unistd.h	getcwd	Directory operation API	Obtains the current working directory.

Header File	API	Type	Description
unistd.h	getopt	System function	Analyzes command line arguments.
unistd.h	getpid	System function	Obtains a task ID.
unistd.h	isatty	File operation function	Determines whether the file is a terminal device.
unistd.h	lseek	File operation function	Changes a file read/write pointer.
unistd.h	pread	File operation function	Reads a file.
unistd.h	pwrite	File operation function	Writes a file.
unistd.h	read	File operation function	Reads a file.
unistd.h	rmdir	Directory operation API	Deletes a directory.
unistd.h	sleep	Time function	Blocks processes in seconds.
unistd.h	sync	File operation function	Synchronizes a file.

Header File	API	Type	Description
unistd.h	sysconf	System function	Obtains system parameters.
unistd.h	unlink	File operation function	Deletes a file.
unistd.h	write	File operation function	Writes a file.
utime.h	utime	File operation function	Changes file time.
wchar.h	mbsnrtowcs	Wide character processing function	Converts a multibyte sequence into a wide character (with a length limit).
wchar.h	wscoll_l	Wide character processing function	Compares two wide strings, using the collating sequence of the current locale for collation.
wchar.h	wcsnlen	Wide character processing function	Returns the length of a multibyte character (with a length limit).
wchar.h	wcsnrtombs	Conversion function	Converts wide character strings to multibyte character strings (with a length limit).

Header File	API	Type	Description
wchar.h	wcsxfrm_l	Wide character processing function	Converts the first <i>n</i> characters in a wide string using the LC_COLLATE determined by the current locale.
wctype.h	iswalnum_l	Wide character processing function	Determines whether a wide character is a letter or digit.
wctype.h	iswalpha_l	Wide character processing function	Determines whether a wide character is an English letter.
wctype.h	iswblank	Wide character processing function	Determines whether a wide character is a tab or space.
wctype.h	iswblank_l	Wide character processing function	Determines whether a wide character is a tab or space.
wctype.h	iswcntrl_l	Wide character processing function	Determines whether a wide character is a control character.

Header File	API	Type	Description
wctype.h	iswctype_l	Wide character processing function	Determines whether the character type is a wide character.
wctype.h	iswdigit_l	Wide character processing function	Checks whether a wide character is a digit (0 through 9).
wctype.h	iswgraph_l	Wide character processing function	Determines whether a wide character is a printable character except a space.
wctype.h	iswlower_l	Wide character processing function	Determines whether a wide character is a lowercase letter.
wctype.h	iswprint_l	Wide character processing function	Determines whether a wide character is printable.
wctype.h	iswpunct_l	Wide character processing function	Determines whether a wide character is a punctuation or special character.

Header File	API	Type	Description
wctype.h	iswspace_l	Wide character processing function	Determines whether a wide character is a space character.
wctype.h	iswupper_l	Wide character processing function	Determines whether a wide character is an uppercase letter.
wctype.h	iswxdigit_l	Wide character processing function	Determines whether a wide character is a hexadecimal number.
wctype.h	towlower_l	Wide character processing function	Converts wide characters to lowercase letters.
wctype.h	towupper_l	Wide character processing function	Converts wide characters to uppercase letters.
wctype.h	wctype_l	Wide character processing function	Determines whether the character type is a wide character.

6.2.2 Supported POSIX NP APIs

Huawei LiteOS provides a set of POSIX NP (nonportable) APIs to support certain SMP NP APIs. The NP APIs are non-standard POSIX APIs and are supplement to the POSIX APIs. The following APIs can operate affinity only when the SMP mode is enabled. In UP mode, "ENOERR" will be returned.

File	API	Description
pthread.h	pthread_attr_setaffinity_np	Set the affinity attribute.
pthread.h	pthread_attr_getaffinity_np	Obtain the affinity attribute.
pthread.h	pthread_setaffinity_np	Set the affinity of a pthread task.
pthread.h	pthread_getaffinity_np	Obtain the affinity of a pthread task.

6.2.3 Unsupported POSIX APIs

Huawei LiteOS does not support the POSIX APIs listed below.

Header File	API
aio.h	aio_cancel
aio.h	aio_error
aio.h	aio_fsync
aio.h	aio_read
aio.h	aio_return
aio.h	aio_suspend
aio.h	aio_write
aio.h	lio_listio
dirent.h	dirfd
dirent.h	fdopendir
dirent.h	readdir_r
dlfcn.h	dlopen
fcntl.h	openat
fcntl.h	posix_fadvise
fcntl.h	posix_fallocate

Header File	API
fntmsg.h	fntmsg
fnmatch.h	fnmatch
ftw.h	ftw
ftw.h	nftw
glob.h	glob
glob.h	globfree
grp.h	endgrent
grp.h	getgrent
grp.h	getgrgid
grp.h	getgrgid_r
grp.h	getgrnam
grp.h	getgrnam_r
grp.h	setgrent
iconv.h	iconv
iconv.h	iconv_close
iconv.h	iconv_open
inttypes.h	imaxabs
inttypes.h	imaxdiv
libgen.h	dirname
locale.h	duplocale
locale.h	freelocale
locale.h	newlocale
locale.h	uselocale
monetary.h	strfmon
monetary.h	strfmon_l
mqueue.h	mq_notify
netdb.h	endhostent
netdb.h	endnetent
netdb.h	endprotoent
netdb.h	endservent

Header File	API
netdb.h	gai_strerror
netdb.h	gethostent
netdb.h	getnetbyaddr
netdb.h	getnetbyname
netdb.h	getnetent
netdb.h	getprotobyname
netdb.h	getprotobynumber
netdb.h	getprotoent
netdb.h	getservbyname
netdb.h	getservbyport
netdb.h	getservent
netdb.h	sethostent
netdb.h	setnetent
netdb.h	setprotoent
netdb.h	setservent
nl_types.h	catclose
nl_types.h	catgets
nl_types.h	catopen
pthread.h	pthread_atfork
pthread.h	pthread_attr_getguardsize
pthread.h	pthread_attr_getstack
pthread.h	pthread_attr_setguardsize
pthread.h	pthread_attr_setstack
pthread.h	pthread_barrier_destroy
pthread.h	pthread_barrier_init
pthread.h	pthread_barrier_wait
pthread.h	pthread_barrierattr_destroy
pthread.h	pthread_barrierattr_getpshared
pthread.h	pthread_barrierattr_init
pthread.h	pthread_barrierattr_setpshared

Header File	API
pthread.h	pthread_getattr_default_np
pthread.h	pthread_getattr_np
pthread.h	pthread_getconcurrency
pthread.h	pthread_getcpuclockid
pthread.h	pthread_getname_np
pthread.h	pthread_mutex_consistent
pthread.h	pthread_mutexattr_getpshared
pthread.h	pthread_mutexattr_getrobust
pthread.h	pthread_mutexattr_setpshared
pthread.h	pthread_mutexattr_setrobust
pthread.h	pthread_rwlock_destroy
pthread.h	pthread_rwlock_init
pthread.h	pthread_rwlock_rdlock
pthread.h	pthread_rwlock_timedrdlock
pthread.h	pthread_rwlock_timedwrlock
pthread.h	pthread_rwlock_tryrdlock
pthread.h	pthread_rwlock_trywrlock
pthread.h	pthread_rwlock_unlock
pthread.h	pthread_rwlock_wrlock
pthread.h	pthread_rwlockattr_destroy
pthread.h	pthread_rwlockattr_getpshared
pthread.h	pthread_rwlockattr_init
pthread.h	pthread_rwlockattr_setpshared
pthread.h	pthread_setattr_default_np
pthread.h	pthread_setconcurrency
pthread.h	pthread_setname_np
pthread.h	pthread_spin_destroy
pthread.h	pthread_spin_init
pthread.h	pthread_spin_lock
pthread.h	pthread_spin_trylock

Header File	API
pthread.h	pthread_spin_unlock
pthread.h	pthread_timedjoin_np
pthread.h	pthread_tryjoin_np
pwd.h	endpwent
pwd.h	getpwent
pwd.h	getpwnam
pwd.h	getpwnam_r
pwd.h	getpwuid
pwd.h	getpwuid_r
pwd.h	setpwent
regex.h	regcomp
regex.h	regerror
regex.h	regexec
regex.h	regfree
setjmp.h	siglongjmp
setjmp.h	sigsetjmp
sched.h	sched_getparam
sched.h	sched_getscheduler
sched.h	sched_rr_get_interval
sched.h	sched_setparam
sched.h	sched_setscheduler
search.h	hcreate
search.h	hdestroy
search.h	hsearch
search.h	insque
search.h	lfind
search.h	lsearch
search.h	remque
search.h	tdelete
search.h	tfind

Header File	API
search.h	tsearch
search.h	twalk
semaphore.h	sem_close
semaphore.h	sem_open
semaphore.h	sem_unlink
signal.h	kill
signal.h	killpg
signal.h	psiginfo
signal.h	psignal
signal.h	pthread_kill
signal.h	pthread_sigmask
signal.h	sigaction
signal.h	sigaddset
signal.h	sigaltstack
signal.h	sigdelset
signal.h	sigemptyset
signal.h	sigfillset
signal.h	sighold
signal.h	sigignore
signal.h	siginterrupt
signal.h	sigismember
signal.h	sigpause
signal.h	sigpending
signal.h	sigprocmask
signal.h	sigqueue
signal.h	sigrelse
signal.h	sigsuspend
signal.h	sigtimedwait
signal.h	sigwait
signal.h	sigwaitinfo

Header File	API
spawn.h	posix_spawn
spawn.h	posix_spawn_file_actions_addclose
spawn.h	posix_spawn_file_actions_adddup2
spawn.h	posix_spawn_file_actions_addopen
spawn.h	posix_spawn_file_actions_destroy
spawn.h	posix_spawn_file_actions_init
spawn.h	posix_spawnattr_destroy
spawn.h	posix_spawnattr_getflags
spawn.h	posix_spawnattr_getpgroup
spawn.h	posix_spawnattr_getschedparam
spawn.h	posix_spawnattr_getschedpolicy
spawn.h	posix_spawnattr_getsigdefault
spawn.h	posix_spawnattr_getsigmask
spawn.h	posix_spawnattr_init
spawn.h	posix_spawnattr_setflags
spawn.h	posix_spawnattr_setpgroup
spawn.h	posix_spawnattr_setschedparam
spawn.h	posix_spawnattr_setschedpolicy
spawn.h	posix_spawnattr_setsigdefault
spawn.h	posix_spawnattr_setsigmask
spawn.h	posix_spawnnp
stdio.h	ctermid
stdio.h	flockfile
stdio.h	fmemopen
stdio.h	ftrylockfile
stdio.h	funlockfile
stdio.h	getc_unlocked
stdio.h	getchar_unlocked
stdio.h	getdelim
stdio.h	getline

Header File	API
stdio.h	open_memstream
stdio.h	pclose
stdio.h	popen
stdio.h	putc_unlocked
stdio.h	putchar_unlocked
stdio.h	renameat
stdio.h	tempnam
stdlib.h	_Exit
stdlib.h	a64l
stdlib.h	drand48
stdlib.h	erand48
stdlib.h	exit
stdlib.h	getsubopt
stdlib.h	grantpt
stdlib.h	jrand48
stdlib.h	l64a
stdlib.h	lcong48
stdlib.h	mkdtemp
stdlib.h	mrnd48
stdlib.h	posix_openpt
stdlib.h	ptsname
stdlib.h	putenv
stdlib.h	rand_r
stdlib.h	setenv
stdlib.h	setkey
stdlib.h	unlockpt
stdlib.h	unsetenv
string.h	memccpy
string.h	strndup
string.h	strsignal

Header File	API
stropts.h	isastream
sys/ipc.h	ftok
sys/mman.h	mlock
sys/mman.h	mlockall
sys/mman.h	mmap
sys/mman.h	mprotect
sys/mman.h	msync
sys/mman.h	munlock
sys/mman.h	munlockall
sys/mman.h	munmap
sys/mman.h	posix_madvise
sys/mman.h	shm_open
sys/mman.h	shm_unlink
sys/msg.h	msgctl
sys/msg.h	msgget
sys/msg.h	msgrcv
sys/msg.h	msgsnd
sys/resource.h	getpriority
sys/resource.h	getrlimit
sys/resource.h	getrusage
sys/resource.h	setpriority
sys/resource.h	setrlimit
sys/select.h	pselect
sys/sem.h	semctl
sys/sem.h	semget
sys/sem.h	semop
sys/shm.h	shmat
sys/shm.h	shmctl
sys/shm.h	shmdt
sys/shm.h	shmget

Header File	API
sys/socket.h	socketatmark
sys/socket.h	socketpair
sys/stat.h	chmod
sys/stat.h	fchmod
sys/stat.h	fchmodat
sys/stat.h	fstatat
sys/stat.h	futimens
sys/stat.h	mkdirat
sys/stat.h	mkfifo
sys/stat.h	mkfifoat
sys/stat.h	mknod
sys/stat.h	mknodat
sys/stat.h	umask
sys/stat.h	utimensat
sys/time.h	getitimer
sys/time.h	setitimer
sys/time.h	utimes
sys/wait.h	wait
sys/wait.h	waitid
sys/wait.h	waitpid
syslog.h	closelog
syslog.h	openlog
syslog.h	setlogmask
termios.h	cfgetispeed
termios.h	cfgetospeed
termios.h	cfsetispeed
termios.h	cfsetospeed
termios.h	tcdrain
termios.h	tcflow
termios.h	tcflush

Header File	API
termios.h	tcgetattr
termios.h	tcgetsid
termios.h	tcsendbreak
termios.h	tcsetattr
time.h	clock_getcpuclockid
time.h	clock_nanosleep
time.h	getdate
time.h	tzset
ulimit.h	ulimit
unistd.h	_exit
unistd.h	alarm
unistd.h	chown
unistd.h	confstr
unistd.h	crypt
unistd.h	ctermid
unistd.h	encrypt
unistd.h	execl
unistd.h	execle
unistd.h	execlp
unistd.h	execv
unistd.h	execve
unistd.h	execvp
unistd.h	faccessat
unistd.h	fchdir
unistd.h	fchown
unistd.h	fchownat
unistd.h	fdatasync
unistd.h	fexecve
unistd.h	fork
unistd.h	fpathconf

Header File	API
unistd.h	getegid
unistd.h	geteuid
unistd.h	getgid
unistd.h	getgroups
unistd.h	gethostid
unistd.h	gethostname
unistd.h	getlogin
unistd.h	getlogin_r
unistd.h	getpgid
unistd.h	getpgrp
unistd.h	getppid
unistd.h	getsid
unistd.h	getuid
unistd.h	lchown
unistd.h	link
unistd.h	linkat
unistd.h	nice
unistd.h	pathconf
unistd.h	pause
unistd.h	pipe
unistd.h	readlink
unistd.h	readlinkat
unistd.h	setegid
unistd.h	seteuid
unistd.h	setgid
unistd.h	setpgid
unistd.h	setpgrp
unistd.h	setregid
unistd.h	setreuid
unistd.h	setsid

Header File	API
unistd.h	setuid
unistd.h	swab
unistd.h	symlink
unistd.h	symlinkat
unistd.h	tcgetpgrp
unistd.h	tcsetpgrp
unistd.h	truncate
unistd.h	ttyname
unistd.h	ttyname_r
unistd.h	unlinkat
utmpx.h	endutxent
utmpx.h	getutxent
utmpx.h	getutxid
utmpx.h	getutxline
utmpx.h	pututxline
utmpx.h	setutxent
wchar.h	open_wmemstream
wchar.h	wcpcpy
wchar.h	wcpncpy
wchar.h	wscasecmp
wchar.h	wscasecmp_l
wchar.h	wcsdup
wchar.h	wcsncasecmp
wchar.h	wcsncasecmp_l
wchar.h	wcswidth
wchar.h	wcwidth
wctype.h	towctrans_l
wctype.h	wctrans_l
wordexp.h	wordexp
wordexp.h	wordfree

6.3 Libc/Libm APIs

6.3.1 Supported Libc APIs

Huawei LiteOS supports a set of Libc APIs. The following table lists their specifications.

Header File	API	Type	Description
arpa/inet.h	inet_aton	Network function	Converts the character string IP address to a number and store the number in the input parameter address.
arpa/inet.h	htonl	Network function	Converts the number of hosts to the network byte order of an unsigned long integer.
arpa/inet.h	htons	Network function	Converts integer variables from host byte order to network byte order.
arpa/inet.h	ntohl	Network function	Converts an unsigned long integer from network byte order to host byte order.
arpa/inet.h	ntohs	Network function	Converts a 16-digit number from network byte order to host byte order.
assert.h	assert	Assertion macro	Terminates the program if its condition returns an error. This API is used for debugging.
ctype.h	isalnum	Data identification	Determines whether the input parameter is a letter or digit.
ctype.h	isalpha	Data identification	Determines whether the input parameter is a letter.
ctype.h	iscntrl	Data identification	Determines whether the input parameter is a control character.
ctype.h	isdigit	Data identification	Determines whether the input parameter is a decimal number.

Header File	API	Type	Description
ctype.h	isgraph	Data identification	Determines whether the input parameter is a graphic character.
ctype.h	islower	Data identification	Determines whether the input parameter contains lowercase letters.
ctype.h	isprint	Data identification	Determines whether the input parameter can be printed.
ctype.h	ispunct	Data identification	Determines whether the input parameter is a punctuation mark.
ctype.h	isspace	Data identification	Determines whether the input parameter is a space.
ctype.h	isupper	Data identification	Determines whether the input parameter contains uppercase letters.
ctype.h	isxdigit	Data identification	Determines whether the input parameter is a hexadecimal number.
ctype.h	tolower	Data conversion function	Converts uppercase letters to lowercase letters.
ctype.h	toupper	Data conversion function	Converts lowercase letters to uppercase letters.
err.h	err	Error message	Outputs error information.
err.h	errx	Error message	Outputs error information.
err.h	verr	Error message	Outputs error information.
err.h	verrx	Error message	Outputs error information.
err.h	vwarn	Error message	Outputs warning information.
err.h	vwarnx	Error message	Outputs warning information.

Header File	API	Type	Description
err.h	warn	Error message	Outputs warning information.
err.h	warnx	Error message	Outputs warning information.
errno.h	errno	Error information	Specifies an error type.
errno.h	get_errno	Error information	Obtains the error type.
errno.h	set_errno	Error information	Sets the error type.
fcntl.h	creat64	File operation function	Creates a file (64-bit).
fcntl.h	fallcoate	File operation function	Pre-allocates physical space to a file.
fcntl.h	fallcoate64	File operation function	Pre-allocates physical space to a file (64-bit).
fcntl.h	open64	File operation function	Opens a file (64-bit).
ifaddrs.h	freeifaddrs	Network function	Releases the local IP address.
ifaddrs.h	getifaddrs	Network function	Obtains the local IP address.
inttypes.h	strtoimax	Data conversion function	Parses integer values in a character string.
inttypes.h	strtoumax	Data conversion function	Parses integer values in a character string.
inttypes.h	wcstoimax	Data conversion function	Parses integer values in a character string.
inttypes.h	wcstoumax	Data conversion function	Parses integer values in a character string.

Header File	API	Type	Description
locale.h	setlocale	System function	Sets or reads a locale. When the locale is read, C.UTF-8 is returned. The locale setting is invalid because LiteOS does not support internationalization.
malloc.h	calloc	Memory operation	Allocates memory and returns a memory pointer.
malloc.h	free	Memory operation	Frees the allocated dynamic memory.
malloc.h	malloc	Memory operation	Allocates memory and returns a memory pointer.
malloc.h	memalign	Memory operation	Allocates and aligns memory, and returns a memory pointer.
malloc.h	realloc	Memory operation	Adjusts the allocated memory and returns a memory pointer.
malloc.h	zalloc	Memory operation	Allocates memory, clears the memory area, and returns the memory pointer.
netdb.h	gethostbyname	Network function	Obtains the host information corresponding to the given host name.
netdb.h	gethostbyname_r	Network function	Obtains the host information corresponding to the given host name. It is reentrant.
poll.h	notify_poll	File operation function	Monitors file descriptors.
poll.h	notify_poll_with_key	File operation function	Monitors file descriptors.
resolv.h	dn_comp	Network function	Compresses a domain name.
sched.h	sched_getaffinity	Scheduling function	Obtains the CPU affinity of a specified task pair.
sched.h	sched_setaffinity	Scheduling function	Sets the CPU affinity of a specified task pair.
setjmp.h	longjmp	System function	Specifies a long jump.

Header File	API	Type	Description
setjmp.h	setjmp	System function	Identifies return points for non-local exits and provides a facility for goto-like jumps.
stdio.h	asprintf	Formatting function	Copies a formatted string.
stdio.h	clearerr	Error handling function	Clears the error flag (locked).
stdio.h	clearerr_unlocked	Error handling function	Clears the error flag (not locked).
stdio.h	fclose	File operation function	Closes the file. If you perform an operation after the file is closed, the operation fails and an error code is set.
stdio.h	feof	File operation function	Checks whether the file stream reads the file tail.
stdio.h	feof_unlocked	File operation function	Checks whether the file stream reads the file tail (not locked).
stdio.h	ferror	File operation function	Checks whether the file stream is correct.
stdio.h	ferror_unlocked	File operation function	Checks whether the file stream is correct (not locked).
stdio.h	fflush	File operation function	Updates the buffer.
stdio.h	fflush_unlocked	File operation function	Updates the buffer (not locked).
stdio.h	fgetc	File operation function	Reads a character from a file.
stdio.h	fgetpos	File operation function	Obtains the read position on a file stream.

Header File	API	Type	Description
stdio.h	fgets	File operation function	Reads a character string from a file.
stdio.h	fgets_unlocked	File operation function	Reads a character string from a file (not locked).
stdio.h	fileno_unlocked	File operation function	Obtains the file descriptor used by the file stream specified by stream . If the file stream is closed, -1 is returned and the error code is set.
stdio.h	fopen	File operation function	Opens a file. A file cannot be opened in RX mode. Otherwise, NULL is returned.
stdio.h	fopen64	File operation function	Opens a file (64-bit).
stdio.h	fprintf	File operation function	Formats output data to a file.
stdio.h	fputc	File operation function	Writes characters to a file stream.
stdio.h	fputs	File operation function	Writes a character string to a file stream.
stdio.h	fputs_unlocked	File operation function	Writes a character string to a file stream (not locked).
stdio.h	fread	File operation function	When data is read from the file stream, the size of the read file is not aligned. You need to ensure that the size is aligned.
stdio.h	fread_unlocked	File operation function	Reads data from a file stream (not locked).
stdio.h	freopen	File operation function	Closes an opened file, opens a new file based on the input mode, and associates the new file stream with the old file stream.

Header File	API	Type	Description
stdio.h	freopen64	File operation function	Closes an opened file, opens a new file based on the input mode, and associates the new file stream with the old file (64-bit).
stdio.h	fscanf	File operation function	Reads formatted input.
stdio.h	fseek	File operation function	Changes the read position on a file stream.
stdio.h	fseeko64	File operation function	Changes the read/write position on a file stream (64-bit).
stdio.h	fsetpos	File operation function	Changes the read position on a file stream.
stdio.h	fsetpos64	File operation function	Changes the write position on a file stream (64-bit).
stdio.h	ftell	File operation function	Obtains the read position on a file stream.
stdio.h	ftello64	File operation function	Obtains the read position on a file stream (64-bit).
stdio.h	fwrite	File operation function	When data is written to the file stream, the size of the written file is not aligned. You need to ensure that the size is aligned and the validity of input parameters is verified.
stdio.h	fwrite_unlocked	File operation function	Writes data to a file stream (not locked).
stdio.h	getc	Standard I/O	Reads a character from a file.
stdio.h	getchar	Standard I/O	Reads a character from a file.
stdio.h	gets	Standard I/O	Reads a character string from a file.

Header File	API	Type	Description
stdio.h	perror	Standard I/O	Outputs an error message to a standard error stream.
stdio.h	printf	Standard I/O	Formats output data.
stdio.h	putc	Standard I/O	Writes a specified character to a file.
stdio.h	putchar	Standard I/O	Writes a specified character to the standard output stream.
stdio.h	puts	Standard I/O	Writes a specified string to the standard output stream.
stdio.h	putw	Standard I/O	Writes an integer to a file.
stdio.h	remove	File operation function	Deletes a file.
stdio.h	rename	File operation function	Renames a file.
stdio.h	rewind	File operation function	Moves the read position of a file stream to the start position.
stdio.h	scanf	Formatted I/O	Formats the input string.
stdio.h	setbuf	File operation function	Sets the buffer of the file stream.
stdio.h	setbuffer	File operation function	Sets the buffer of the file stream.
stdio.h	setvbuf	File operation function	Sets the buffer of the file stream. If the input parameter size is less than 0, -1 is returned and an error code is set. When the input parameter buf is NULL, the default size space is dynamically allocated as the file buffer.
stdio.h	snprintf	Formatted I/O	Formats a string.
stdio.h	sprintf	Formatted I/O	Formats a string.

Header File	API	Type	Description
stdio.h	sscanf	Formatted I/O	Formats the input string.
stdio.h	tmpnam	File operation function	Returns a pointer to a unique file name.
stdio.h	ungetc	File operation function	Writes a specified character back to a file stream.
stdio.h	vasprintf	Formatted I/O	Formats a string.
stdio.h	vfprintf	Formatted I/O	Formats output data to a file.
stdio.h	vscanf	Formatted I/O	Reads a string from a stream and converts it based on format .
stdio.h	vprintf	Formatted I/O	Formats output.
stdio.h	vscanf	Formatted I/O	Formats the input string.
stdio.h	vsnprintf	Formatted I/O	Formats a string.
stdio.h	vsprintf	Formatted I/O	Formats a string.
stdio.h	vsscanf	Formatted I/O	Formats the input string.
stdlib.h	abort	System function	Aborts the system.
stdlib.h	abs	Mathematical function	Returns the absolute value of an integer.
stdlib.h	atof	Data conversion function	Converts a string to a floating point number.
stdlib.h	atoi	Data conversion function	Converts a string to an integer.
stdlib.h	atol	Data conversion function	Converts a string to a long integer.

Header File	API	Type	Description
stdlib.h	atoll	Data conversion function	Converts a character string to a long integer.
stdlib.h	bsearch	Data structure function	Performs binary search.
stdlib.h	labs	Mathematical function	Returns the absolute value of a long integer.
stdlib.h	llabs	Mathematical function	Returns the absolute value of a long integer.
stdlib.h	mblen	Multi-byte character function	Returns the length of a multibyte character string.
stdlib.h	mbstowcs	Multi-byte character function	Converts a multibyte character string to a wide string.
stdlib.h	mbtowc	Multi-byte character function	Converts a multibyte character to a wide character.
stdlib.h	mkostemps	File processing function	Generates a unique temporary file name.
stdlib.h	mkstemps	File processing function	Generates a unique temporary file.
stdlib.h	qsort	Sorting function	Sorts an array by using a quick sorting method.
stdlib.h	qsort_r	Sorting function	Sorts an array by using a quick sorting method.
stdlib.h	rand	Random number function	Generates a random number.
stdlib.h	srand	Random number function	Sets a random number seed.
stdlib.h	strtod	Data conversion function	Converts a string to a floating point number.

Header File	API	Type	Description
stdlib.h	strtof	String processing function	Converts a string to a single-precision floating-point number.
stdlib.h	strtol	Data conversion function	Converts a string to a long integer.
stdlib.h	strtold	String processing function	Converts a string to a long double-precision floating-point number.
stdlib.h	strtoll	String processing function	Converts a string to a long long integer.
stdlib.h	strtoul	Data conversion function	Converts a string to an unsigned long integer.
stdlib.h	strtoull	String processing function	Converts a string to an unsigned long long integer.
stdlib.h	wctomb	Multi-byte character function	Converts a wide character to a multibyte character.
string.h	memchr	String processing function	Searches for a specific character in a memory range.
string.h	memcmp	String processing function	Compares memory contents.
string.h	memcpy	String processing function	Copies memory contents.
string.h	memmove	String processing function	Copies memory contents.
string.h	memrchr	String processing function	Searches for the position where a character appears for the first time.
string.h	memset	String processing function	Fills a memory space with a value.

Header File	API	Type	Description
string.h	strcasestr	String processing function	Searches for a specified character string in a character string, case insensitive.
string.h	strcat	String processing function	Concatenates two character strings.
string.h	strchr	String processing function	Searches for the position where a character appears for the first time.
string.h	strchrnul	String processing function	Searches for the position where a character appears for the first time.
string.h	strcmp	String processing function	Compares strings.
string.h	strcoll	String processing function	Compares two strings based on LC_COLLATE .
string.h	strcpy	String processing function	Copies a character string.
string.h	strcspn	String processing function	The number of consecutive characters in the character string of input parameter 1 does not belong to the character string of input parameter 2.
string.h	strerror	String processing function	Returns a pointer to an error message string.
string.h	strncpy	String processing function	Copies a character string.
string.h	strlen	String processing function	Returns the length of a string in bytes.
string.h	strncat	String processing function	Concatenates two character strings.
string.h	strncmp	String processing function	Compares strings.

Header File	API	Type	Description
string.h	strncpy	String processing function	Copies a character string.
string.h	strpbrk	String processing function	Searches for the first specified character in a character string.
string.h	strrchr	String processing function	Searches for the last specified character in a character string.
string.h	strsep	String processing function	Splits a string into several strings.
string.h	strspn	String processing function	Returns the number of consecutive characters that do not contain the specified string.
string.h	strstr	String processing function	Searches for a specified character string in a character string.
string.h	strtok	String processing function	Splits a string.
string.h	strxfrm	String processing function	Converts the character string src based on LC_COLLATE. If the length of n is less than that of src , the length of src is returned without conversion.
strings.h	bcmp	Memory operation	Same as memcmp.
strings.h	bcopy	Memory operation	Same as memmove.
strings.h	bzero	Memory operation	Same as memset.
strings.h	rindex	Character processing function	Processes a character.
sys/mount.h	mount	File system operation	Mounts a file system.
sys/mount.h	umount	File system operation	Unmounts a file system.

Header File	API	Type	Description
sys/prctl.h	prctl	pthread	Performs thread operations. Currently, only PR_SET_NAME (change thread names) is supported.
sys/stat.h	fstat64	File operation API	Obtains file attributes (64-bit).
sys/statfs.h	statfs	File operation API	Obtains file system attributes.
sys/time.h	adjtime	Time function	Fine-tunes the system time based on the time value delta.
sys/time.h	settimeofday	Time function	Sets the system time.
threads.h	tss_get	pthread	Gets the key binding of the calling thread, which is the same as pthread_getspecific.
time.h	asctime	Time function	Represents the time and date as a character string.
time.h	clock	Time function	Obtains the time used by the processor clock.
time.h	ctime	Time function	Represents the time and date as a character string.
time.h	difftime	Time function	Returns the difference (in seconds) between two time points.
time.h	gmtime	Time function	Obtains the current date and time.
time.h	localtime	Time function	Obtains the current local date and time.
time.h	mktime	Time function	Converts the time structure data into the number of elapsed seconds.
time.h	stime	Time function	Sets time.
time.h	strftime	Time function	Formats the date and time. The format character does not support the format conversion of %z or %Z.
time.h	time	Time function	Obtains the current system time.

Header File	API	Type	Description
time64.h	asctime64	Time function	Represents the time and date as a character string.
time64.h	asctime64_r	Time function	Represents the time and date as a character string.
time64.h	ctime64	Time function	Represents the time and date as a character string.
time64.h	ctime64_r	Time function	Represents the time and date as a character string.
time64.h	gmtime64	Time function	Obtains the current date and time.
time64.h	gmtime64_r	Time function	Obtains the current date and time.
time64.h	gettimeofday64	Time function	Obtains the current system time. Huawei LiteOS re-implements this API to obtain time later than 2038 on 32-bit platforms.
time64.h	localtime64	Time function	Obtains the current local date and time.
time64.h	localtime64_r	Time function	Obtains the current local date and time.
time64.h	mktime64	Time function	Converts the time structure data into the number of elapsed seconds.
time64.h	settimeofday64	Time function	Sets the current time. Huawei LiteOS re-implements this API to set time later than 2038 on 32-bit platforms.
tzdst.h	settimezone	Time function	Sets the time zone.
tzdst.h	dst_disable	Time function	Disables DST.
tzdst.h	dst_enable	Time function	Enables DST.
tzdst.h	dst_inquire	Time function	Queries DST.
uchar.h	c32rtomb	Wide character processing function	Converts a 32-bit wide character into a multibyte character.

Header File	API	Type	Description
uchar.h	mbrtoc32	Wide character processing function	Converts a multibyte character to a 32-bit wide character.
unistd.h	ftruncate64	File operation function	Changes a file size.
unistd.h	getentropy	Random number function	Obtains a random number of a specified length. If the hardware cannot provide a random number, a failure message is returned.
unistd.h	getpagesize	System function	Obtains the page size.
unistd.h	lseek64	File operation function	Changes a file read/write pointer.
unistd.h	pread64	File operation function	Reads a file.
unistd.h	pwrite64	File operation function	Writes a file.
unistd.h	usleep	Time function	Blocks processes in microseconds.
wchar.h	btowc	Wide character processing function	Converts a single byte to a wide character.
wchar.h	fgetwc	Wide character processing function	Converts a single byte to a wide character.
wchar.h	fgetwc_unlocked	Wide character processing function	Converts a single byte to a wide character (not locked).
wchar.h	fputwc	Wide character processing function	Writes a wide character to a file stream.

Header File	API	Type	Description
wchar.h	fputwc_unlocked	Wide character processing function	Writes a wide character to a file stream (not locked).
wchar.h	fwide	File operation function	Sets the direction of the file stream to be queried.
wchar.h	getwc	Wide character processing function	Reads a wide character from a file.
wchar.h	getwc_unlocked	Wide character processing function	Reads a wide character from a file (not locked).
wchar.h	mbrlen	Wide character processing function	Returns the length of a multibyte character.
wchar.h	mbrtowc	Wide character processing function	Converts a multibyte sequence into a wide character.
wchar.h	mbsinit	Wide character processing function	Tests the initial conversion status.
wchar.h	mbsrtowcs	Wide character processing function	Converts a multibyte sequence into a wide character.
wchar.h	putwc	Wide character processing function	Writes a specified wide character to a file.
wchar.h	putwc_unlocked	Wide character processing function	Writes a specified wide character to a file (not locked).
wchar.h	swprintf	Formatted I/O	Formats a wide string.

Header File	API	Type	Description
wchar.h	ungetwc	Wide character processing function	Writes wide characters back to the specified file stream.
wchar.h	vfwprintf	Formatted I/O	Formats wide character data to a file.
wchar.h	vfwscanf	Formatted I/O	Formats a wide string from an input stream.
wchar.h	vswprintf	Formatted I/O	Formats a wide string.
wchar.h	wcrtomb	Wide character processing function	Converts the wide character code into a character.
wchar.h	wcschr	Wide character processing function	Searches for the first specified wide character in a wide string.
wchar.h	wcscmp	Wide character processing function	Compares two wide strings.
wchar.h	wcscoll	Wide character processing function	Compares two wide strings, using the collating sequence of the current locale for collation.
wchar.h	wcsftime	Wide character processing function	Formats time.
wchar.h	wcsftime_l	Wide character processing function	Formats time.
wchar.h	wcslen	Wide character processing function	Returns the length of a wide string.
wchar.h	wcsncmp	Wide character processing function	Compares a specified number of characters in two wide strings.

Header File	API	Type	Description
wchar.h	wcsncpy	Wide character processing function	Copies a wide string.
wchar.h	wcsrtombs	Conversion function	Converts wide strings to multibyte character strings.
wchar.h	wcsstr	Wide character processing function	Searches for another wide string in a wide string.
wchar.h	wcstol	Wide character processing function	Converts a wide string to a long integer.
wchar.h	wcstoll	Wide character processing function	Converts a wide string to a long long integer.
wchar.h	wcstoul	Wide character processing function	Converts a wide string to an unsigned long integer.
wchar.h	wcstoull	Wide character processing function	Converts a wide string to an unsigned long long integer.
wchar.h	wcsxfrm	Wide character processing function	Converts the first <i>n</i> characters in a wide string using the LC_COLLATE determined by the current locale.
wchar.h	wctob	Wide character processing function	Converts a wide character to a single-byte character.
wchar.h	wmemchr	Wide character processing function	Searches in a wide character array.
wchar.h	wmemcmp	Wide character processing function	Compares two wide character arrays.

Header File	API	Type	Description
wchar.h	wmemcpy	Wide character processing function	Copies a wide character array.
wchar.h	wmemmove	Wide character processing function	Copies a wide character array.
wchar.h	wmemset	Wide character processing function	Fills a wide character array.
wctype.h	iswalnum	Wide character processing function	Determines whether a wide character is a letter or digit.
wctype.h	iswalpha	Wide character processing function	Determines whether a wide character is an English letter.
wctype.h	iswcntrl	Wide character processing function	Determines whether a wide character is a control character.
wctype.h	iswctype	Wide character processing function	Determines whether the character type is a wide character.
wctype.h	iswdigit	Wide character processing function	Checks whether a wide character is a digit (0 through 9).
wctype.h	iswgraph	Wide character processing function	Determines whether a wide character is a printable character except a space.
wctype.h	iswlower	Wide character processing function	Determines whether a wide character is a lowercase letter.

Header File	API	Type	Description
wctype.h	iswprint	Wide character processing function	Determines whether a wide character is printable.
wctype.h	iswpunct	Wide character processing function	Determines whether a wide character is a punctuation or special character.
wctype.h	iswspace	Wide character processing function	Determines whether a wide character is a space character.
wctype.h	iswupper	Wide character processing function	Determines whether a wide character is an uppercase letter.
wctype.h	iswxdigit	Wide character processing function	Determines whether a wide character is a hexadecimal number.
wctype.h	towlower	Wide character processing function	Converts wide characters to lowercase letters.
wctype.h	towupper	Wide character processing function	Converts wide characters to uppercase letters.
wctype.h	wctype	Wide character processing function	Determines whether the character type is a wide character.

6.3.2 Supported Libm APIs

Huawei LiteOS supports a set of open-source Libm APIs. The following table lists their specifications.

NOTICE

The returned error codes cannot be configured for Libm APIs.

Header File	API	Type	Description
complex.h	cabs	Function for computing complex numbers	Return the absolute value of the complex number.
math.h	acos	Mathematical function	Return the arccosine of x .
math.h	acosf	Mathematical function	Return the arccosine of x .
math.h	acosh	Mathematical function	Return the inverse hyperbolic cosine of x .
math.h	acoshf	Mathematical function	Return the inverse hyperbolic cosine of x .
math.h	acoshl	Mathematical function	Return the inverse hyperbolic cosine of x .
math.h	acosl	Mathematical function	Return the arccosine of x .
math.h	asin	Mathematical function	Return the arcsine of x .
math.h	asinf	Mathematical function	Return the arcsine of x .

Header File	API	Type	Description
math.h	asinh	Mathematical function	Return the inverse hyperbolic sine of x .
math.h	asinhf	Mathematical function	Return the inverse hyperbolic sine of x .
math.h	asinhll	Mathematical function	Return the inverse hyperbolic sine of x .
math.h	asinl	Mathematical function	Return the arcsine of x .
math.h	atan	Mathematical function	Return the arctangent of x .
math.h	atan2	Mathematical function	Return the arctangent of x .
math.h	atan2f	Mathematical function	Return the arctangent of x . The value is in units of radians.
math.h	atan2l	Mathematical function	Return the arctangent of x . The value is in units of radians.
math.h	atanf	Mathematical function	Return the arctangent of x .
math.h	atanh	Mathematical function	Return the inverse hyperbolic tangent of x .

Header File	API	Type	Description
math.h	atanhf	Mathematical function	Return the inverse hyperbolic tangent of x .
math.h	atanhl	Mathematical function	Return the inverse hyperbolic tangent of x .
math.h	atanl	Mathematical function	Return the arctangent of x .
math.h	cbrt	Mathematical function	Return the cube root of x .
math.h	cbrtf	Mathematical function	Return the cube root of x .
math.h	cbrtl	Mathematical function	Return the cube root of x .
math.h	ceil	Mathematical function	Round x upwards to the nearest integer.
math.h	ceilf	Mathematical function	Round x upwards to the nearest integer.
math.h	ceil	Mathematical function	Round x upwards to the nearest integer.
math.h	copysign	Mathematical function	Return a value whose absolute value matches that of x , but whose sign bit matches that of y .

Header File	API	Type	Description
math.h	copysignf	Mathematical function	Return a value whose absolute value matches that of x , but whose sign bit matches that of y .
math.h	copysignl	Mathematical function	Return a value whose absolute value matches that of x , but whose sign bit matches that of y .
math.h	cos	Mathematical function	Return the cosine of x .
math.h	cosf	Mathematical function	Return the cosine of x .
math.h	cosh	Mathematical function	Return the hyperbolic cosine of x .
math.h	coshf	Mathematical function	Return the hyperbolic cosine of x .
math.h	coshl	Mathematical function	Return the hyperbolic cosine of x .
math.h	cosl	Mathematical function	Return the cosine of x .
math.h	drem	Mathematical function	Compute the remainder (double).
math.h	dremf	Mathematical function	Compute the remainder (floating).

Header File	API	Type	Description
math.h	erf	Mathematical function	Return the error function of x .
math.h	erfc	Mathematical function	Return the complementary error function of x .
math.h	erfcf	Mathematical function	Return the complementary error function of x .
math.h	erfcl	Mathematical function	Return the complementary error function of x .
math.h	erff	Mathematical function	Return the error function of x .
math.h	erfl	Mathematical function	Return the error function of x .
math.h	exp	Mathematical function	Compute e raised to the power x .
math.h	exp10	Mathematical function	Compute 10 raised to the power x .
math.h	exp10l	Mathematical function	Compute 10 raised to the power x .
math.h	exp2	Mathematical function	Compute 2 raised to the power x .

Header File	API	Type	Description
math.h	exp2f	Mathematical function	Compute 2 raised to the power x .
math.h	exp2l	Mathematical function	Compute 2 raised to the power x .
math.h	expf	Mathematical function	Compute e raised to the power x .
math.h	expl	Mathematical function	Compute e raised to the power x .
math.h	expm1	Mathematical function	$\exp(x) - 1.0$
math.h	expm1f	Mathematical function	$\exp(x) - 1$
math.h	expm1l	Mathematical function	$\exp(x) - 1$
math.h	fabs	Mathematical function	Return the absolute value of a floating-point number.
math.h	fabsf	Mathematical function	Return the absolute value of a floating-point number.
math.h	fabsl	Mathematical function	Return the absolute value of a floating-point number.

Header File	API	Type	Description
math.h	fdim	Mathematical function	Return the positive difference between x and y .
math.h	fdimf	Mathematical function	Return the positive difference between x and y .
math.h	fdiml	Mathematical function	Return the positive difference between x and y .
math.h	finite	Mathematical function	Return a nonzero value if x is neither infinite nor a "not a number" value, and zero otherwise.
math.h	finitef	Mathematical function	Return a nonzero value if x is neither infinite nor a "not a number" value, and zero otherwise.
math.h	floor	Mathematical function	Round x downwards to the nearest integer
math.h	floorf	Mathematical function	Round x downwards to the nearest integer
math.h	floorl	Mathematical function	Round x downwards to the nearest integer
math.h	fma	Mathematical function	Perform floating-point multiply-add. This is the operation $(x \cdot y) + z$.
math.h	fmaf	Mathematical function	Perform floating-point multiply-add. This is the operation $(x \cdot y) + z$.

Header File	API	Type	Description
math.h	fmal	Mathematical function	Perform floating-point multiply-add. This is the operation $(x \cdot y) + z$.
math.h	fmax	Mathematical function	Return the greater of the two values x and y .
math.h	fmaxf	Mathematical function	Return the greater of the two values x and y .
math.h	fmaxl	Mathematical function	Return the greater of the two values x and y .
math.h	fmin	Mathematical function	Return the lesser of the two values x and y .
math.h	fminf	Mathematical function	Return the lesser of the two values x and y .
math.h	fminl	Mathematical function	Return the lesser of the two values x and y .
math.h	fmod	Mathematical function	Compute the remainder.
math.h	fmodf	Mathematical function	Compute the remainder.
math.h	fmodl	Mathematical function	Compute the remainder.

Header File	API	Type	Description
math.h	frexp	Mathematical function	Split a floating-point number into a normalized fraction and an exponent.
math.h	frexpf	Mathematical function	Split a floating-point number into a normalized fraction and an exponent.
math.h	frexpl	Mathematical function	Split a floating-point number into a normalized fraction and an exponent.
math.h	hypot	Mathematical function	Return the length of the hypotenuse of a right triangle.
math.h	hypotf	Mathematical function	Return the length of the hypotenuse of a right triangle.
math.h	hypotl	Mathematical function	Return the length of the hypotenuse of a right triangle.
math.h	ilogb	Mathematical function	This function is equivalent to the corresponding logb function except that it returns a signed integer value.
math.h	ilogbf	Mathematical function	This function is equivalent to the corresponding logb function except that it returns a signed integer value.
math.h	ilogbl	Mathematical function	This function is equivalent to the corresponding logb function except that it returns a signed integer value.
math.h	isnan	Classification of characters	Return a nonzero value if <i>x</i> is a "not a number" value, and zero otherwise.

Header File	API	Type	Description
math.h	j0	Mathematical function	Return the Bessel function of the first kind of order 0 of x .
math.h	j0f	Mathematical function	Return the Bessel function of the first kind of order 0 of x .
math.h	j1	Mathematical function	Return the Bessel function of the first kind of order 1 of x .
math.h	j1f	Mathematical function	Return the Bessel function of the first kind of order 1 of x .
math.h	jn	Mathematical function	Return the Bessel function of the first kind of order n of x .
math.h	jnf	Mathematical function	Return the Bessel function of the first kind of order n of x .
math.h	ldexp	Mathematical function	Return the result of multiplying x by 2 raised to the power <i>exponent</i> .
math.h	ldexpf	Mathematical function	Return the result of multiplying x by 2 raised to the power <i>exponent</i> .
math.h	ldexpl	Mathematical function	Return the result of multiplying x by 2 raised to the power <i>exponent</i> .
math.h	lgamma	Mathematical function	Return the natural logarithm of the absolute value of the gamma function: $\gamma(x) = \int_0^{\infty} t^{x-1} e^{-t} dt$

Header File	API	Type	Description
math.h	lgamma_r	Mathematical function	lgamma_r is just like lgamma, but it stores the sign of the intermediate result in the variable pointed to by the second input parameter instead of in the global variable. This means it is reentrant.
math.h	lgammaf	Mathematical function	Return the natural logarithm of the absolute value of the gamma function: $\gamma(x) = \int_0^{\infty} t^{x-1} e^{-t} dt$
math.h	lgammaf_r	Mathematical function	lgamma_r is just like lgamma, but it stores the sign of the intermediate result in the variable pointed to by the second input parameter instead of in the global variable. This means it is reentrant.
math.h	lgammal	Mathematical function	Return the natural logarithm of the absolute value of the gamma function: $\gamma(x) = \int_0^{\infty} t^{x-1} e^{-t} dt$
math.h	lgammal_r	Mathematical function	lgamma_r is just like lgamma, but it stores the sign of the intermediate result in the variable pointed to by the second input parameter instead of in the global variable. This means it is reentrant.
math.h	llrint	Mathematical function	Round x to an integer according to the current rounding mode.
math.h	llrintf	Mathematical function	Round x to an integer according to the current rounding mode.
math.h	llrintl	Mathematical function	Round x to an integer according to the current rounding mode.
math.h	llround	Mathematical function	Round to the nearest integer.
math.h	llroundf	Mathematical function	Round to the nearest integer.

Header File	API	Type	Description
math.h	llroundl	Mathematical function	Round to the nearest integer.
math.h	log	Mathematical function	Return the base- e logarithm of x .
math.h	log10	Mathematical function	Return the base-10 logarithm of x .
math.h	log10f	Mathematical function	Return the base-10 logarithm of x .
math.h	log10l	Mathematical function	Return the base-10 logarithm of x .
math.h	log1p	Mathematical function	$\log(1+x)$
math.h	log1pf	Mathematical function	$\log(1+x)$
math.h	log1pl	Mathematical function	$\log(1+x)$
math.h	log2	Mathematical function	Return the base-2 logarithm of x .
math.h	log2f	Mathematical function	Return the base-2 logarithm of x .

Header File	API	Type	Description
math.h	log2l	Mathematical function	Return the base-2 logarithm of x .
math.h	logb	Mathematical function	Return the logarithm of the absolute value of x , using FLT_RADIX as base for the logarithm.
math.h	logbf	Mathematical function	Return the logarithm of the absolute value of x , using FLT_RADIX as base for the logarithm.
math.h	logbl	Mathematical function	Return the logarithm of the absolute value of x , using FLT_RADIX as base for the logarithm.
math.h	logf	Mathematical function	Return the base- e logarithm of x .
math.h	logl	Mathematical function	Return the base- e logarithm of x .
math.h	lrint	Mathematical function	Round x to an integer according to the current rounding mode and return a long int.
math.h	lrintf	Mathematical function	Round x to an integer according to the current rounding mode and return a long int.
math.h	lrintl	Mathematical function	Round x to an integer according to the current rounding mode and return a long int.
math.h	lround	Mathematical function	Round x to an integer according to the current rounding mode and return a long int.

Header File	API	Type	Description
math.h	lroundf	Mathematical function	Round x to an integer according to the current rounding mode and return a long int.
math.h	lroundl	Mathematical function	Round x to an integer according to the current rounding mode and return a long int.
math.h	modf	Mathematical function	Break a floating-point number into an integer part and a fractional part, and return the fractional part.
math.h	modff	Mathematical function	Break a floating-point number into an integer part and a fractional part.
math.h	modfl	Mathematical function	Break a floating-point number into an integer part and a fractional part.
math.h	nan	Classification of characters	Return a representation of NaN.
math.h	nanf	Classification of characters	Return a representation of NaN.
math.h	nanl	Classification of characters	Return a representation of NaN.
math.h	nearbyint	Mathematical function	Round x to an integer according to the current rounding mode.
math.h	nearbyintf	Mathematical function	Round x to an integer according to the current rounding mode.

Header File	API	Type	Description
math.h	nearbyintl	Mathematical function	Round x to an integer according to the current rounding mode.
math.h	nextafter	Mathematical function	Return the next representable neighbor of x in the direction towards y . If $x = y$, the function simply returns y . x and y are both of the double type.
math.h	nextafterf	Mathematical function	Return the next representable neighbor of x in the direction towards y . If $x = y$, the function simply returns y .
math.h	nextafterl	Mathematical function	Return the next representable neighbor of x in the direction towards y .
math.h	nexttoward	Mathematical function	Return the previous representable neighbor of x in the direction towards y .
math.h	nexttowardf	Mathematical function	Return the previous representable neighbor of x in the direction towards y .
math.h	nexttowardl	Mathematical function	Return the previous representable neighbor of x in the direction towards y .
math.h	pow	Mathematical function	Return the value of x raised to the power y .
math.h	pow10	Mathematical function	Return the value of x raised to the power 10.
math.h	pow10f	Mathematical function	Return the value of x raised to the power 10.

Header File	API	Type	Description
math.h	pow10l	Mathematical function	Return the value of x raised to the power 10.
math.h	powf	Mathematical function	Return the value of x raised to the power y .
math.h	powl	Mathematical function	Return the value of x raised to the power y .
math.h	remainder	Mathematical function	Compute the remainder and round it to the nearest integer.
math.h	remainderf	Mathematical function	Compute the remainder and round it to the nearest integer.
math.h	remainderl	Mathematical function	Compute the remainder and round it to the nearest integer.
math.h	remquo	Mathematical function	Compute the remainder.
math.h	remquof	Mathematical function	Compute the remainder.
math.h	remquol	Mathematical function	Compute the remainder.
math.h	rint	Mathematical function	Round a floating-point number to the nearest integer.

Header File	API	Type	Description
math.h	rintf	Mathematical function	Round a floating-point number to the nearest integer.
math.h	rintl	Mathematical function	Round a floating-point number to the nearest integer.
math.h	round	Mathematical function	Round x to the nearest integer.
math.h	roundf	Mathematical function	Round x to the nearest integer.
math.h	roundl	Mathematical function	Round x to the nearest integer.
math.h	scalb	Mathematical function	Return the result of multiplying x by 2 raised to the power <i>exponent</i> .
math.h	scalbf	Mathematical function	Return the result of multiplying x by 2 raised to the power <i>exponent</i> .
math.h	scalbln	Mathematical function	Return the result of multiplying x by 2 raised to the power <i>exponent</i> .
math.h	scalblnf	Mathematical function	Return the result of multiplying x by 2 raised to the power <i>exponent</i> .
math.h	scalblnl	Mathematical function	Return the result of multiplying x by 2 raised to the power <i>exponent</i> .

Header File	API	Type	Description
math.h	scalbn	Mathematical function	Return the result of multiplying x by FLT_RADIX raised to the power n .
math.h	scalbnf	Mathematical function	Return the result of multiplying x by FLT_RADIX raised to the power n .
math.h	scalbnl	Mathematical function	Return the result of multiplying x by FLT_RADIX raised to the power n .
math.h	significand	Mathematical function	Return the mantissa of x scaled to the range $[1, 2)$. It is equivalent to <i>scalb(x, (double) -ilogb(x))</i> .
math.h	significandf	Mathematical function	Return the mantissa of x scaled to the range $[1, 2)$. It is equivalent to <i>scalb(x, (double) -ilogb(x))</i> .
math.h	sin	Mathematical function	Return the sine of x .
math.h	sincos	Mathematical function	Return the sine of x in <i>*sinx</i> and the cosine of x in <i>*cosx</i> .
math.h	sincosf	Mathematical function	Return the sine of x in <i>*sinx</i> and the cosine of x in <i>*cosx</i> .
math.h	sincosl	Mathematical function	Return the sine of x in <i>*sinx</i> and the cosine of x in <i>*cosx</i> .
math.h	sinf	Mathematical function	Return the sine of x .

Header File	API	Type	Description
math.h	sinh	Mathematical function	Return the hyperbolic sine of x .
math.h	sinhf	Mathematical function	Return the hyperbolic sine of x .
math.h	sinhl	Mathematical function	Return the hyperbolic sine of x .
math.h	sinl	Mathematical function	Return the sine of x .
math.h	sqrt	Mathematical function	Return the nonnegative square root of x .
math.h	sqrtf	Mathematical function	Return the nonnegative square root of x .
math.h	sqrtl	Mathematical function	Return the nonnegative square root of x .
math.h	tan	Mathematical function	Return the tangent of x .
math.h	tanf	Mathematical function	Return the tangent of x .
math.h	tanh	Mathematical function	Return the hyperbolic tangent of x .

Header File	API	Type	Description
math.h	tanhf	Mathematical function	Return the hyperbolic tangent of x .
math.h	tanh	Mathematical function	Return the hyperbolic tangent of x .
math.h	tanl	Mathematical function	Return the tangent of x .
math.h	tgamma	Mathematical function	Apply the gamma function to x .
math.h	tgammaf	Mathematical function	Apply the gamma function to x .
math.h	tgammal	Mathematical function	Apply the gamma function to x .
math.h	trunc	Time function	Truncate a date or number.
math.h	truncf	Time function	Truncate a date or number.
math.h	truncl	Time function	Truncate a date or number.
math.h	y0	Mathematical function	Return the Bessel function of the second kind of order 0 of x .
math.h	y0f	Mathematical function	Return the Bessel function of the second kind of order 0 of x .

Header File	API	Type	Description
math.h	y1	Mathematical function	Return the Bessel function of the second kind of order 1 of x .
math.h	y1f	Mathematical function	Return the Bessel function of the second kind of order 1 of x .
math.h	yn	Mathematical function	Return the Bessel function of the second kind of order n of x .
math.h	ynf	Mathematical function	Return the Bessel function of the second kind of order n of x .

6.3.3 Unsupported Libc/Libm APIs

Huawei LiteOS does not support the Libc/Libm APIs listed below.

Header File	API
alloca.h	alloca
arpa/inet.h	inet_lnaof
arpa/inet.h	inet_makeaddr
arpa/inet.h	inet_netof
arpa/inet.h	inet_network
arpa/nameser.h	ns_get16
arpa/nameser.h	ns_get32
arpa/nameser.h	ns_initparse
arpa/nameser.h	ns_name_uncompress
arpa/nameser.h	ns_parserr
arpa/nameser.h	ns_put16
arpa/nameser.h	ns_put32
arpa/nameser.h	ns_skiprr

Header File	API
byteswap.h	bswap_16
byteswap.h	bswap_32
byteswap.h	bswap_64
complex.h	cabsf
complex.h	cabsl
complex.h	cacos
complex.h	cacosf
complex.h	cacosh
complex.h	cacoshf
complex.h	cacoshl
complex.h	cacosl
complex.h	carg
complex.h	cargf
complex.h	cargl
complex.h	casin
complex.h	casinf
complex.h	casinh
complex.h	casinhf
complex.h	casinhl
complex.h	casinl
complex.h	catan
complex.h	catanf
complex.h	catanh
complex.h	catanhf
complex.h	catanhl
complex.h	catanl
complex.h	ccos
complex.h	ccosf
complex.h	ccosh
complex.h	ccoshf

Header File	API
complex.h	ccoshl
complex.h	ccosl
complex.h	cexp
complex.h	cexpf
complex.h	cexpl
complex.h	cimag
complex.h	cimagf
complex.h	cimagl
complex.h	clog
complex.h	clogf
complex.h	clogl
complex.h	conj
complex.h	conjf
complex.h	conjl
complex.h	cpow
complex.h	cpowf
complex.h	cpowl
complex.h	cproj
complex.h	cprojf
complex.h	cprojl
complex.h	creal
complex.h	crealf
complex.h	creall
complex.h	csin
complex.h	csinf
complex.h	csinh
complex.h	csinhf
complex.h	csinhl
complex.h	csinl
complex.h	csqrt

Header File	API
complex.h	csqrtf
complex.h	csqrtl
complex.h	ctan
complex.h	ctanf
complex.h	ctanh
complex.h	ctanhf
complex.h	ctanhl
complex.h	ctanl
crypt.h	crypt
crypt.h	crypt_r
dirent.h	getdents
dirent.h	versionsort
dlfcn.h	dladdr
dlfcn.h	dlopen
fcntl.h	lockf
fcntl.h	lockf64
fcntl.h	name_to_handle_at
fcntl.h	open_by_handle_at
fcntl.h	openat64
fcntl.h	posix_fadvise64
fcntl.h	posix_fallocate64
fcntl.h	readahead
fcntl.h	splice
fcntl.h	sync_file_range
fcntl.h	tee
fcntl.h	vmsplice
fenv.h	feclearexcept
fenv.h	fegetenv
fenv.h	fegetexceptflag
fenv.h	fegetround

Header File	API
fenv.h	feholdexcept
fenv.h	feraiseexcept
fenv.h	fesetenv
fenv.h	fesetexceptflag
fenv.h	fesetround
fenv.h	fetestexcept
fenv.h	feupdateenv
getopt.h	getopt_long
getopt.h	getopt_long_only
libintl.h	bind_textdomain_codeset
libintl.h	bindtextdomain
libintl.h	dcgettext
libintl.h	dcngettext
libintl.h	dgettext
libintl.h	dngettext
libintl.h	gettext
libintl.h	ngettext
libintl.h	textdomain
link.h	dl_iterate_phdr
locale.h	localeconv
malloc.h	malloc_usable_size
malloc.h	valloc
mntent.h	addmntent
mntent.h	endmntent
mntent.h	getmntent
mntent.h	getmntent_r
mntent.h	hasmntopt
mntent.h	setmntent
netdb.h	gethostbyaddr
netdb.h	gethostbyaddr_r

Header File	API
netdb.h	gethostbyname2
netdb.h	gethostbyname2_r
netdb.h	getservbyname_r
netdb.h	getservbyport_r
netdb.h	herror
netdb.h	hstrerror
netinet/ether.h	ether_aton
netinet/ether.h	ether_aton_r
netinet/ether.h	ether_hostton
netinet/ether.h	ether_line
netinet/ether.h	ether_ntoa
netinet/ether.h	ether_ntoa_r
netinet/ether.h	ether_ntohost
poll.h	ppoll
pty.h	forkpty
pty.h	openpty
pwd.h	fgetpwent
pwd.h	putpwent
resolv.h	dn_expand
resolv.h	dn_skipname
resolv.h	res_init
resolv.h	res_mkquery
resolv.h	res_query
resolv.h	res_querydomain
resolv.h	res_search
resolv.h	res_send
sched.h	clone
sched.h	sched_getcpu
sched.h	setns
sched.h	unshare

Header File	API
search.h	hcreate_r
search.h	hdestroy_r
search.h	hsearch_r
search.h	tdestroy
shadow.h	endspent
shadow.h	fgetspent
shadow.h	getspent
shadow.h	getspnam
shadow.h	getspnam_r
shadow.h	lckpword
shadow.h	putspent
shadow.h	setspent
shadow.h	sgetspent
shadow.h	ulckpword
signal.h	raise
signal.h	sigandset
signal.h	sigisemptyset
signal.h	sigorset
spawn.h	posix_spawn
spawn.h	posix_spawn_file_actions_addchdir_np
spawn.h	posix_spawn_file_actions_addclose
spawn.h	posix_spawn_file_actions_adddup2
spawn.h	posix_spawn_file_actions_addfchdir_np
spawn.h	posix_spawn_file_actions_addopen
spawn.h	posix_spawn_file_actions_destroy
spawn.h	posix_spawn_file_actions_init
spawn.h	posix_spawnattr_destroy
spawn.h	posix_spawnattr_getflags
spawn.h	posix_spawnattr_getpgroup
spawn.h	posix_spawnattr_getschedparam

Header File	API
spawn.h	posix_spawnattr_getschedpolicy
spawn.h	posix_spawnattr_getsigdefault
spawn.h	posix_spawnattr_getsigmask
spawn.h	posix_spawnattr_init
spawn.h	posix_spawnattr_setflags
spawn.h	posix_spawnattr_setpgroup
spawn.h	posix_spawnattr_setschedparam
spawn.h	posix_spawnattr_setschedpolicy
spawn.h	posix_spawnattr_setsigdefault
spawn.h	posix_spawnattr_setsigmask
spawn.h	posix_spawnnp
stdio.h	cuserid
stdio.h	fgetc_unlocked
stdio.h	fgetln
stdio.h	fopencookie
stdio.h	fputc_unlocked
stdio.h	getw
stdio.h	setlinebuf
stdio.h	tmpfile
stdio.h	tmpfile64
stdlib.h	aligned_alloc
stdlib.h	at_quick_exit
stdlib.h	div
stdlib.h	ecvt
stdlib.h	fcvt
stdlib.h	gcvt
stdlib.h	getenv
stdlib.h	getloadavg
stdlib.h	ldiv
stdlib.h	lldiv

Header File	API
stdlib.h	mkdtemp
stdlib.h	mkostemp
stdlib.h	ptsname_r
stdlib.h	quick_exit
stdlib.h	reallocarray
stdlib.h	system
stdlib.h	tcgetwinsize
stdlib.h	tcsetwinsize
stdlib.h	valloc
stdlib.h	wcstombs
string.h	explicit_bzero
string.h	memmem
string.h	mempcpy
string.h	strlcat
string.h	strverscmp
strings.h	ffsl
strings.h	ffsll
strings.h	index
sys/acct.h	acct
sys/auxv.h	getauxval
sys/cachectl.h	cachectl
sys/cachectl.h	cacheflush
sys/epoll.h	epoll_create
sys/epoll.h	epoll_create1
sys/epoll.h	epoll_ctl
sys/epoll.h	epoll_pwait
sys/epoll.h	epoll_wait
sys/eventfd.h	eventfd
sys/eventfd.h	eventfd_read
sys/eventfd.h	eventfd_write

Header File	API
sys/fanotify.h	fanotify_init
sys/fanotify.h	fanotify_mark
sys/file.h	flock
sys/fsuid.h	setfsuid
sys/fsuid.h	setfsuid
sys/inotify.h	inotify_add_watch
sys/inotify.h	inotify_init
sys/inotify.h	inotify_init1
sys/inotify.h	inotify_rm_watch
sys/io.h	ioperm
sys/io.h	iopl
sys/klog.h	klogctl
sys/membarrier.h	membarrier
sys/mman.h	madvise
sys/mman.h	memfd_create
sys/mman.h	mincore
sys/mman.h	mlock2
sys/mman.h	mremap
sys/mman.h	remap_file_pages
sys/mount.h	umount2
sys/personality.h	personality
sys/ptrace.h	ptrace
sys/quota.h	quotactl
sys/random.h	getrandom
sys/reboot.h	reboot
sys/resource.h	prlimit
sys/sem.h	semtimedop
sys/sendfile.h	sendfile
sys/signalfd.h	signalfd
sys/socket.h	accept4

Header File	API
sys/socket.h	recvmsg
sys/socket.h	sendmsg
sys/stat.h	fstat64
sys/stat.h	lchmod
sys/stat.h	lstat64
sys/statfs.h	fstatfs
sys/statvfs.h	fstatvfs
sys/statvfs.h	statvfs
sys/swap.h	swapoff
sys/swap.h	swapon
sys/sysinfo.h	get_avphys_pages
sys/sysinfo.h	get_nprocs
sys/sysinfo.h	get_nprocs_conf
sys/sysinfo.h	get_phys_pages
sys/sysinfo.h	sysinfo
sys/time.h	futimes
sys/time.h	futimesat
sys/time.h	lutimes
sys/timeb.h	ftime
sys/timerfd.h	timerfd_create
sys/timerfd.h	timerfd_gettime
sys/timerfd.h	timerfd_settime
sys/timex.h	adjtimex
sys/timex.h	clock_adjtime
sys/uio.h	preadv
sys/uio.h	process_vm_readv
sys/uio.h	process_vm_writev
sys/uio.h	pwritev
sys/wait.h	wait3
sys/wait.h	wait4

Header File	API
sys/xattr.h	fgetxattr
sys/xattr.h	flistxattr
sys/xattr.h	fremovexattr
sys/xattr.h	fsetxattr
sys/xattr.h	getxattr
sys/xattr.h	lgetxattr
sys/xattr.h	listxattr
sys/xattr.h	llistxattr
sys/xattr.h	lremovexattr
sys/xattr.h	lsetxattr
sys/xattr.h	removexattr
sys/xattr.h	setxattr
syslog.h	vsyslog
termios.h	cfmakeraw
termios.h	cfsetspeed
threads.h	call_once
threads.h	cnd_broadcast
threads.h	cnd_destroy
threads.h	cnd_init
threads.h	cnd_signal
threads.h	cnd_timedwait
threads.h	cnd_wait
threads.h	mtx_destroy
threads.h	mtx_init
threads.h	mtx_lock
threads.h	mtx_timedlock
threads.h	mtx_trylock
threads.h	mtx_unlock
threads.h	thrd_create
threads.h	thrd_current

Header File	API
threads.h	thrd_detach
threads.h	thrd_equal
threads.h	thrd_exit
threads.h	thrd_join
threads.h	thrd_sleep
threads.h	thrd_yield
threads.h	tss_create
threads.h	tss_delete
threads.h	tss_set
time.h	timegm
time.h	timespec_get
time64.h	timegm64
time64.h	timelocal64
uchar.h	c16rtomb
uchar.h	mbrtoc16
ucontext.h	getcontext
ucontext.h	makecontext
ucontext.h	setcontext
ucontext.h	swapcontext
unistd.h	_Fork
unistd.h	acct
unistd.h	brk
unistd.h	chroot
unistd.h	copy_file_range
unistd.h	daemon
unistd.h	dup3
unistd.h	eaccess
unistd.h	endusershell
unistd.h	execvpe
unistd.h	get_current_dir_name

Header File	API
unistd.h	getdomainname
unistd.h	getdtablesize
unistd.h	gettid
unistd.h	getusershell
unistd.h	issetugid
unistd.h	pipe2
unistd.h	posix_close
unistd.h	sbrk
unistd.h	setdomainname
unistd.h	setresgid
unistd.h	setresuid
unistd.h	setusershell
unistd.h	syncfs
unistd.h	syscall
unistd.h	truncate64
unistd.h	ualarm
unistd.h	vfork
unistd.h	vhangup
utmp.h	endutent
utmp.h	getutent
utmp.h	getutid
utmp.h	getutline
utmp.h	login_tty
utmp.h	pututline
utmp.h	setutent
utmp.h	updwtmp
utmp.h	utmpname
utmpx.h	updwtmpx
utmpx.h	utmpxname
wchar.h	fgetws

Header File	API
wchar.h	fgetws_unlocked
wchar.h	fputws
wchar.h	fputws_unlocked
wchar.h	fwprintf
wchar.h	fwscanf
wchar.h	getwchar
wchar.h	getwchar_unlocked
wchar.h	putwchar
wchar.h	putwchar_unlocked
wchar.h	swscanf
wchar.h	vswscanf
wchar.h	vwprintf
wchar.h	vwscanf
wchar.h	wcscat
wchar.h	wcscpy
wchar.h	wcscspn
wchar.h	wcsncat
wchar.h	wcspbrk
wchar.h	wcsrchr
wchar.h	wcsspn
wchar.h	wcstod
wchar.h	wcstof
wchar.h	wcstok
wchar.h	wcstold
wchar.h	wcswcs
wchar.h	wprintf
wchar.h	wscanf
wctype.h	towctrans
wctype.h	wctrans

NOTICE

From V200R005C00, Huawei LiteOS no longer supports the **atexit()** provided in the header file **stdlib.h**.

6.4 C++ Compatibility Specifications

The following tables list the compatibility specifications of the C++ standard library and standard template library (STL).

 NOTE

The C++ standard library supports the exception processing and the run-time type information (RTTI) features. Other features are supported by the compiler. The following tables describe the features supported by the STL. Other features are currently not supported.

- Language support

Header File	Description
<limits>	Provides definitions related to basic data types. For example, defines the maximum and minimum values and the number of binary digits for each numeric data type in this file.
<new>	Supports dynamic memory allocation.

- Tool functions

Header File	Description
<utility>	Defines the overloaded rational operator, which simplifies the write of the rational operator; defines the pair type, which is a template type and can be used to store value pairs. These functions are used elsewhere in the library.
<functional.>	Defines the types of function objects and supports the utilities of function objects. Function objects are any objects that support the function call operator.
<memory>	Defines the standard memory allocator for container functions, memory management functions, and the auto_ptr template class.

- String processing

Header File	Description
<string>	Provides supports and definitions for character string types, including single-character strings (consisting of char types) and multi-character strings (consisting of wchar_t types).

- Templates for container classes

Header File	Description
<vector>	Defines the vector sequence template, which is resizable array type and is safer and more flexible than plain arrays.
<list>	Defines the list sequence template, which is a linked list for sequences that often have elements inserted or deleted from arbitrary positions.
<deque>	Defines the deque sequence template, which supports efficient insertion and deletion at each beginning and end.
<queue>	Defines sequence adapters "queue" and "priority_queue" for queue (first in, first out) data structures.
<stack>	Define sequence adapter "stack" for stack (last in, first out) data structures.
<map>	An associative container type that allows values to be searched by a key value. The key values are unique and are stored in ascending order.
<set>	An associative container type that stores unique values in ascending order.
<bitset>	Defines the bitset template for fixed-length bit sequences. A bitset template can be considered as a fixed-length packed bool array.

- Iterators

Header File	Description
<iterator>	Provides definitions and support for iterators.

- Algorithms

Header File	Description
<algorithm>	Provides a set of algorithm-based functions, including substitution, sequencing, merge, and search functions.

- Numerical operations

Header File	Description
<complex>	Support complex numerical definitions and operations.
<valarray>	Supports numerical vector operations.
<numeric>	Defines a group of common mathematics operations, such as "accumulate" and "inner_product" for a numerical sequence.

- Thread processing

Header File	Description
<mutex>	Support Mutex operations.
<thread>	Support thread function.
<condition_variable>	Support condition variables.

6.5 CMSIS APIs

6.5.1 CMSIS v1.0

This section describes CMSIS v1.0 APIs that adapt to Huawei LiteOS. For details about the API declaration, input parameters, return value types, and API usage conditions, see the [CMSIS v1.0 Documentation](#).

6.5.1.1 Supported CMSIS v1.0 APIs

Huawei LiteOS supports most CMSIS v1.0 APIs. API declamation is available in **compat/cmsis/1.0/cmsis_os1.h**. The following table describes the APIs in detail.

API	Type	Description
osThreadCreate	Task/thread	Create a task.
osThreadGetId	Task/thread	Obtains the current task handle.
osThreadTerminate	Task/thread	Stop a task.
osThreadYield	Task/thread	Switch to a Ready task with the same priority.
osThreadSetPriority	Task/thread	Set the task priority.

API	Type	Description
osThreadGetPriority	Task/thread	Obtain the task priority.
osDelay	Delay	Delay a task.
osTimerCreate	Timer	Create a timer.
osTimerStart	Timer	Start a timer. (If the timer is timing, it will be stopped.)
osTimerStop	Timer	Stop a timer.
osTimerDelete	Timer	Delete a timer.
osSignalSet	Signal	Set a signal.
osSignalClear	Signal	Clear a signal.
osSignalWait	Signal	Wait for a signal.
osMutexCreate	Mutex	Create a mutex.
osMutexWait	Mutex	Obtain a mutex (blocked).
osMutexRelease	Mutex	Release a mutex.
osMutexDelete	Mutex	Delete a mutex.
osSemaphoreCreate	Semaphore	Create a semaphore.
osSemaphoreWait	Semaphore	Obtain a semaphore (blocked).
osSemaphoreRelease	Semaphore	Release a semaphore.
osSemaphoreDelete	Semaphore	Delete a semaphore.
osPoolCreate	Memory block	Create a memory block.
osPoolAlloc	Memory block	Apply for memory.
osPoolCAlloc	Memory block	Apply for and reset memory.
osPoolFree	Memory block	Free up memory.
osMessageCreate	Pointer message	Creates a message queue without content, which is usually a data pointer.
osMessagePut	Pointer message	Put messages into a message queue.
osMessageGet	Pointer message	Obtain messages from a message queue.
osMailCreate	Content message	Create a message queue with content. It can be considered as a combination of osPool and osMessage.

API	Type	Description
osMailAlloc	Content message	Apply for memory for storing message content.
osMailCAlloc	Content message	Apply for and reset memory for storing message content.
osMailPut	Content message	Put messages into a message queue.
osMailGet	Content message	Obtain messages from a message queue.
osMailFree	Content message	Free up the applied memory.

6.5.1.2 Unsupported CMSIS v1.0 APIs

The following table describes the unsupported CMSIS v1.0 APIs.

API	Type	Description
osWait	Delay	Waits for any signal or message.

6.5.1.3 Differences from Standard CMSIS v1.0 APIs

In consideration of API usability and implementation inside Huawei LiteOS, Huawei LiteOS makes adaptation to some CMSIS v1.0 APIs. For details, see the following table.

API	Type	Description
osTimerStart	Timer	Adds an osTimerDelay timer. This timer and the osTimerOnce timer are both one-shot timers. The difference is that the osTimerOnce timer will be deleted after it times out, but the osTimerDelay timer will not be deleted and can be started repeatedly by calling the osTimerStart API.
osBinarySemaphoreCreate	Semaphore	New API for creating binary semaphores.

6.5.2 CMSIS v2.0

This section describes CMSIS v2.0 APIs that adapt to Huawei LiteOS. For details about the API declaration, input parameters, return value types, and API usage conditions, see the [CCMSIS v2.0 Documentation](#).

6.5.2.1 Supported CMSIS v2.0 APIs

Huawei LiteOS supports most CMSIS v2.0 APIs. The API declamation is available in `open_source/CMSIS/CMSIS/RTOS2/Include/cmsis_os2.h`. The following table describes the APIs in detail.

API	Type	Description
osKernelInitialize	Kernel	Initializes the operating system.
osKernelGetInfo	Kernel	Obtains the system version.
osKernelGetState	Kernel	Obtains the system status. For details, see <code>osThreadState_t</code> .
osKernelStart	Kernel	Starts the operating system.
osKernelLock	Kernel	Locks the kernel (lock scheduling).
osKernelUnlock	Kernel	Unlocks the kernel (unlock scheduling).
osKernelRestore-Lock	Kernel	Restores the kernel locking status.
osKernelGetTick-Count	Kernel	Obtains the time (unit: tick) after the system starts.
osKernelGetTick-Freq	Kernel	Obtains the number of ticks in a second.
osKernelGetSysTi-merCount	Kernel	Obtains the time (unit: cycle) after the system starts.
osKernelGetSysTi-merFreq	Kernel	Obtains the number of CPU cycles in a second.
osThreadNew	Task/Thread	Creates a task.
osThreadJoin	Task/Thread	Waits for a specified task and reclaims task resources. (The kernel LOSCFG_TASK_JOINABLE configuration must be enabled.)
osThreadDetach	Task/Thread	Sets the task attribute to detached . (The kernel LOSCFG_TASK_JOINABLE configuration must be enabled.)
osThreadGetNam-e	Task/Thread	Obtains the task name.
osThreadGetId	Task/Thread	Obtains the task handle.

API	Type	Description
osThreadGetStat e	Task/Thread	Obtains the task status.
osThreadGetStac kSize	Task/Thread	Obtains the size of a task stack.
osThreadGetStac kSpace	Task/Thread	Obtains the space of an unused task stack.
osThreadSetPrior ity	Task/Thread	Sets the task priority.
osThreadGetPrior ity	Task/Thread	Obtains the task priority.
osThreadYield	Task/Thread	Switches to a Ready task with the same priority.
osThreadSuspend	Task/Thread	Suspends a task. (The task cannot be scheduled before being resumed.)
osThreadResume	Task/Thread	Resumes a task.
osThreadTermina te	Task/Thread	Stops a task (not recommended).
osThreadGetCou nt	Task/Thread	Obtains the number of created tasks.
osThreadFlagsSet	Task event	Writes an event.
osThreadFlagsCle ar	Task event	Clears an event.
osThreadFlagsGe t	Task event	Obtains the event of the current task.
osThreadFlagsWa it	Task event	Waits for an event.
osDelay	Delay	Delays a task (unit: tick).
osDelayUntil	Pointer message	Delays to a specific time point (unit: tick).
osTimerNew	Timer	Creates a timer.
osTimerGetName	Timer	Obtains the timer name. (Currently, the value is fixed to NULL .)
osTimerStart	Timer	Starts a timer. (If the timer is timing, it will be stopped.)
osTimerStop	Timer	Stops a timer.
osTimerIsRunnin g	Timer	Checks whether a timer is timing.

API	Type	Description
osTimerDelete	Timer	Deletes a timer.
osEventFlagsNew	Event	Creates an event. (Unlike the task event ThreadFlags, this event has an independent handle and control block.)
osEventFlagsGet-Name	Event	Obtains the event name. (Currently, the value is fixed to NULL .)
osEventFlagsSet	Event	Writes an event.
osEventFlagsClear	Event	Clears an event.
osEventFlagsGet	Event	Obtains the current event value.
osEventFlagsWait	Event	Waits for an event.
osEventFlagsDelete	Event	Deletes an event.
osMutexNew	Mutex	Creates a mutex.
osMutexGetName	Mutex	Obtains the mutex name. (Currently, the value is fixed to NULL .)
osMutexAcquire	Mutex	Obtains a mutex (blocked).
osMutexRelease	Mutex	Releases a mutex.
osMutexGetOwner	Mutex	Obtains the handle of the task that holds the current mutex.
osMutexDelete	Mutex	Deletes a mutex.
osSemaphoreNew	Semaphore	Creates a semaphore.
osSemaphoreGetName	Semaphore	Obtains the semaphore name. (Currently, the value is fixed to NULL .)
osSemaphoreAcquire	Semaphore	Obtains a semaphore (blocked).
osSemaphoreRelease	Semaphore	Releases a semaphore.
osSemaphoreGetCount	Semaphore	Obtains the count of a semaphore.
osSemaphoreDelete	Semaphore	Deletes a semaphore.
osMessageQueueNew	Message queue	Creates a message queue.

API	Type	Description
osMessageQueueGetName	Message queue	Obtains the name of a message queue. (Currently, the value is fixed to NULL .)
osMessageQueuePut	Message queue	Puts messages into a message queue.
osMessageQueueGet	Message queue	Obtains messages from a message queue.
osMessageQueueGetCapacity	Message queue	Obtains the number of nodes in a message queue.
osMessageQueueGetMsgSize	Message queue	Obtains the size of nodes in a message queue.
osMessageQueueGetCount	Message queue	Obtains the number of messages in a message queue.
osMessageQueueGetSpace	Message queue	Obtains the number of remaining messages in a message queue.
osMessageQueueDelete	Message queue	Deletes a message queue.

6.5.2.2 Unsupported CMSIS v2.0 APIs

API	Type	Description
osKernelSuspend	Task/thread	Suspend a kernel to prevent scheduling, which is generally used for low-power processing. Currently, Huawei LiteOS provides low-power mechanisms such as Tickless and RunStop, and therefore does not support this API.
osKernelResume	Task/thread	Same as above.
osThreadExit	Task/thread	Huawei LiteOS supports automatic deletion of tasks. There is no need to call osThreadExit before exiting tasks.
osThreadEnumerate	Task/thread	This API is used to obtain the list of created tasks. Currently, this API is not supported. Users can call maintenance and test APIs such as LOS_TaskInfoGet to obtain task status.
osMemoryPoolAlloc	Memory block	This API involves memory block application within a timeout period. Currently, Huawei LiteOS does not provide this mechanism.

API	Type	Description
osMemoryPoolFree	Memory block	Because osMemoryPoolAlloc is not implemented, the remaining osMemPool interfaces are not implemented either.
osMemoryPoolGetCapacity	Memory block	Same as above.
osMemoryPoolGetBlockSize	Memory block	Same as above.
osMemoryPoolGetCount	Memory block	Same as above.
osMemoryPoolGetSpace	Memory block	Same as above.
osMemoryPoolDelete	Memory block	Same as above.
osMessageQueueReset	Message queue	Huawei LiteOS does not provide this queue clearance API. Users can perform operations on queue content to prevent resource leakage or other exceptions.

6.5.2.3 Differences from Standard CMSIS v2.0 APIs

In consideration of API usability and implementation inside Huawei LiteOS, Huawei LiteOS makes adaptation to some CMSIS v2.0 APIs. For details, see the following table.

API	Type	Description
osKernelGetTick2ms	Kernel	New API for obtaining the time (unit: ms) after the system starts. The API declaration is in compat/cmsis/cmsis_os.h .
osMs2Tick	Kernel	New API for converting milliseconds to ticks. The API declaration is in compat/cmsis/cmsis_os.h .
osThreadNew	Task/Thread	The priority range is [osPriorityLow3,osPriorityHigh].
osThreadSetPriority	Task/Thread	The priority range is [osPriorityLow3,osPriorityHigh].

6.6 Adaption to Linux

Currently, Huawei LiteOS is compatible with linux-4.19.90. The adaptation to Linux is tailorable. To use this function, choose **Compat > Enable Linux** from the navigation menu to enable the adaptation to Linux module.

Configuration Item	Description	Value Range	Default Value	Dependency
LOSCFG_COMPAT_LINUX	Specifies whether to enable the adaptation to Linux module.	YES/NO	YES	None.

6.6.1 Completion

6.6.1.1 Overview

Basic Concepts

Completion is a supplement to semaphore. It is a lightweight mechanism for task synchronization. In Linux, **up()** and **down()** functions can be executed concurrently in the same semaphore. In the multi-CPU environment, **up()** may mistakenly access a semaphore that does not exist. Completion is designed to prevent this error.

In the scenario in which task A can be executed only after task B has completed a particular operation, the completion enables task B to wake up task A at completion of the particular operation, thereby achieving task synchronization.

6.6.1.2 Development Guidelines

Application Scenario

Use the completion mechanism in the multi-task environment to synchronize one task with another.

Functions

Function Category	API	Description	Compatibility
Initializing a completion	init_completion	Initializes a completion.	Not fully compatible. If the input parameter is a null pointer, the API exits and does not perform initialization.

Function Category	API	Description	Compatibility
Waiting for a completion	wait_for_completion	Waits for a completion until the completion occurs.	Not fully compatible. If the input parameter is a null pointer or the API usage scenario is incorrect, the API exits.
	wait_for_completion_timeout	Waits for a completion for a specified number of ticks.	Not fully compatible. The invalid value judgment is added to the API implementation. For details, see the API reference.
Waking up a task waiting for a completion	complete	Wakes up the first task in the queue waiting for a completion.	Not fully compatible. If the input parameter is null, the API exits.
	complete_all	Wakes up all tasks waiting for a completion.	Not fully compatible. If the input parameter is null, the API exits.

Development Procedure

A typical development process of the completion mechanism is as follows:

1. Open the menu and choose **Compat > Enable Linux > Enable completion** to enable the completion.

Configuration Item	Description	Value Range	Default Value	Dependency
LOSCFG_COMPAT_LINUX_COMPLETION	Determines whether to enable the Linux completion.	YES/NO	NO	None.

2. Call the `init_completion` API to initialize a completion.
3. Call the `wait_for_completion_timeout/wait_for_completion` API to wait for a completion.
4. Call the `complete/complete_all` API to wake up a task waiting for the completion.

6.6.1.3 Precautions

- The completion mechanism is similar to the semaphore mechanism. For details, see [Precautions](#). Permanent blocking and timed blocking are not allowed while an interrupt is underway, since interrupts cannot be blocked.
- The input parameter to the completion APIs must be a valid completion pointer. For example, a task is not allowed to wait for a completion while an interrupt is underway.

6.6.1.4 Programming Example

Example Description

In the programming example, the Example_Completion task is created in the Example_TaskEntry task. The Example_Completion task is blocked while waiting for a completion. Then, the Example_TaskEntry task wakes up the Example_Completion task waiting for the completion. Based on the information printed on the screen, you can learn the task switching that occurs along with the completion operation.

1. Create the Example_Completion task in the Example_TaskEntry task. The Example_Completion task takes a higher priority over the Example_TaskEntry task.
2. The Example_Completion task waits for a completion and is blocked. In this case, the Example_TaskEntry task is executed.
3. The Example_TaskEntry task completes and wakes up the Example_Completion for execution.
4. The Example_Completion task is executed.
5. The Example_TaskEntry task is executed.

Example Code

Prerequisite: The adaptation to Linux module has been enabled through the **menuconfig** menu.

The code is as follows:

```
#include "linux/completion.h"
#include "los_task.h"

/* Task ID */
UINT32 g_testTaskId;

/* Completion */
struct completion exampleCompletion;

/* Example task entry function */
VOID Example_Completion(VOID)
{
    UINT32 ret;

    /* Wait for the completion in timeout mode. The timeout interval is 100 ticks. */
    printf("Example_Completion wait completion\n");
    ret = wait_for_completion_timeout(&exampleCompletion, 100);

    if(ret == 0) {
        printf("Example_Completion, wait completion timeout\n");
    } else {
```

```

        printf("Example_Completion, wait completion success\n");
    }
    return;
}

UINT32 Example_TaskEntry(VOID)
{
    UINT32 ret;
    TSK_INIT_PARAM_S task;

    /* Initialize the completion. */
    init_completion(&exampleCompletion);

    /* Create a task. */
    memset(&task, 0, sizeof(TSK_INIT_PARAM_S));
    task.pfnTaskEntry = (TSK_ENTRY_FUNC)Example_Completion;
    task.pcName       = "EventTsk1";
    task.uwStackSize  = LOSCFG_BASE_CORE_TSK_DEFAULT_STACK_SIZE;
    task.usTaskPrio   = 8;
    ret = LOS_TaskCreate(&g_testTaskId, &task);
    if(ret != LOS_OK) {
        printf("task create failed \n");
        return LOS_NOK;
    }

    /* Wake up the task waiting for the completion. */
    printf("Example_TaskEntry complete\n");
    complete(&exampleCompletion);

    printf("Delete Task.\n");
    /* Delete the task. */
    ret = LOS_TaskDelete(g_testTaskId);
    if(ret != LOS_OK) {
        printf("task delete failed\n");
        return LOS_NOK;
    }

    return LOS_OK;
}

```

Verification

The verification result is as follows:

```

Example_Completion wait completion
Example_TaskEntry complete
Example_Completion, wait completion success
Delete Task.

```

Complete Code

sample_completion.c

6.6.2 Work Queue

6.6.2.1 Overview

Basic Concepts

Any system modules can place works into a work queue. Works are processed by the work queue (a type of tasks) in the first in first out (FIFO) order.

The work queue processing task can be re-scheduled or sent to sleep mode to save system resources.

The work queue mechanism of Huawei LiteOS provides rich external APIs to manage workqueues. When a work is added to a work queue, the work queue processing task is woken up to process the work. After all works in the work queue are processed, the work queue processing task is sent to sleep mode.

6.6.2.2 Development Guidelines

Application Scenario

A work queue is applicable only in delay-tolerant scenarios because works in the work queue can be processed only after the tasks in the work queue are executed. Many factors are considered while determining whether the tasks in the work queue can be woken up or scheduled. For example, a task may be blocked because another task with a higher priority is waiting to be executed or because an interrupt is underway.

Functions

Function Category	API	Description	Compatibility
Creating or destroying a work queue	create_singlethread_workqueue	Creates a work queue on the current CPU.	Fully compatible
	create_workqueue	Huawei LiteOS creates a work queue only on the current CPU. In the multi-CPU scenario, Linux creates a work queue on each CPU.	Not fully compatible.
	destroy_workqueue	Destroys a work queue.	Fully compatible
Initializing a work	INIT_WORK	Binds a work to the processing function, which will be executed when the work is processed.	Fully compatible
	INIT_DELAYED_WORK	Binds a delayed work to the processing function, which will be executed when the work is processed.	Fully compatible

Function Category	API	Description	Compatibility
Placing a work in a work queue	queue_work	Places a work in a work queue.	Not fully compatible. Parameter invalidity check is involved. For details, see the API reference.
	queue_delayed_work	Places a delayed work in a work queue.	Not fully compatible. Parameter invalidity check is involved. For details, see the API reference.
	schedule_work	Places a work in the default work queue.	Fully compatible
	schedule_delayed_work	Places a delayed work in the default work queue.	Not fully compatible. Parameter invalidity check is involved. For details, see the API reference.
Querying the work status	work_busy	Queries the status of a work.	Not fully compatible. The return values are different. For details, see the API reference.
Executing a work	flush_work	Blocks waiting for a specified and non-delayed work item to be executed and returns after the execution is complete.	Fully compatible
	flush_delayed_work	Cancels the delay and waits for the work item to be executed. After the work item is executed, return to the previous page.	Fully compatible
Cancelling a work	cancel_delayed_work	Cancels a delayed work item. If another task is canceling the work, a failure message is returned.	Not fully compatible. The return values are different.

Function Category	API	Description	Compatibility
	cancel_delayed_work_sync	Cancels a delayed work item after the work item is complete. If another task is canceling the work item, it waits until the task cancels the work item.	Not fully compatible. The return values are different.
	cancel_work_sync	Cancels a work item after the work item is complete. If another task is canceling the work item, it waits until the task cancels the work item.	Not fully compatible. The return values are different.

NOTE

- When a work queue is created, the semaphore is initialized for the work queue to process the task sleep and wakeup, initialize important structures, and set up a work queue linked list.
- When a work queue is destroyed, the system locks the resource, deletes the work queue tasks, releases the semaphore resource and memory space, and unlocks the resource.
- Works and delayed works are provided. Works are placed in a work queue immediately, while delayed works are placed in a work queue only after a specified time.

Development Procedure

The typical work queue development process is as follows:

1. Open the menu and choose **Compat > Enable Linux > Enable workqueue** to enable the work queue.

Configuration Item	Description	Value Range	Default Value	Dependency
LOSCFG_COMPAT_LINUX_WORKQUEUE	Determines whether to enable the Linux work queue.	YES/NO	NO	LOSCFG_COMPAT_LINUX_TIMER

2. Call the create_workqueue API to create a work queue.

3. Call the INIT_WORK/INIT_DELAYED_WORK API to initialize a work.
4. Call the queue_work/queue_delayed_work API to place the work to the work queue.
5. Call the flush_work/flush_delayed_work API to immediately execute the work.
6. Call the cancel_delayed_work_sync/cancel_work_sync API to cancel the work.
7. Call the destroy_workqueue API to delete the work queue.

6.6.2.3 Precautions

- Do not use a work queue in delay-sensitive scenarios because there is much uncertainty in the time when a task can be scheduled to process a work in a work queue.
- Work queues are identified by their names. Therefore, each work queue must have a unique name.
- The schedule_work or schedule_delayed_work API can be used to place a work in the default work queue without first creating a work queue.

6.6.2.4 Programming Example

Description

The following functions are implemented:

1. Creating a work queue named wq_test
2. Allocating work memory and initializing a work
3. Placing a work to the work queue
4. Executing the work immediately
5. Destroying the work queue

Example Code

Prerequisite: The adaptation to Linux module has been enabled through the **menuconfig** menu.

The code is as follows:

```
#include "los_printf.h"
#include "linux/workqueue.h"

static void Work_Func(struct work_struct *work)
{
    int i;
    for (i = 0; i < 2; i++) {
        dprintf("workqueue function is been called!..%d..%d.\n", i, work->work_status);
    }
}

UINT32 Example_Workqueue(VOID)
{
    struct workqueue_struct *wq;
    struct work_struct *work;
    UINT32 ret = LOS_OK;

    wq = create_workqueue("test1008");
    dprintf("create the workqueue successfully!\n\n");
```

```
work =(struct work_struct *)malloc(sizeof(struct work_struct));
if (!work) {
    ret = LOS_NOK;
}
dprintf("create work ok!\n\n");

INIT_WORK(work, Work_Func);
dprintf("init the work ok!\n\n");

ret = queue_work(wq, work);
dprintf("mount the work into workqueue successfully!\n\n");

ret = flush_work(work);
dprintf("flush the work ok!\n\n");

destroy_workqueue(wq);
dprintf("destroy the work ok!\n\n");

return ret;
}
```

Result Verification

```
create the workqueue successfully!

create work ok!

init the work ok!

mount the work into workqueue successfully!

workqueue function is been called!..0..3..
workqueue function is been called!..1..3..
flush the work ok!

destroy the work ok!
```

Complete Code

sample_workqueue.zip

6.6.3 Wait Queue

6.6.3.1 Overview

Basic Concepts

A wait queue is usually used by threads to wait for resources.

When a thread must obtain some resources before performing the next operation and does not occupy CPU resources, the thread can be placed in the wait queue. Then the system switches to another process and wakes up the wait queue after the resources are ready. The wait queues are classified to two types: timeout and cyclic wait queues.

The wait queue mechanism of Huawei LiteOS provides abundant external APIs to manage the wait queue, which can meet most of the functions of the wait queue on the Linux system.

6.6.3.2 Development Guidelines

Application Scenario

Compared with the work queue, the wait queue is used in scenarios that have high requirements on time, especially the timeout mechanism. Typical scenarios are as follows: synchronous system resource access, interrupt bottom half, and asynchronous event notification.

Functions

Function Category	API	Description	Compatibility	Precautions
Initializing the head of the wait queue	DECLARE_WAIT_QUEUE_HEAD	Defines and initializes the head of the wait queue.	Fully compatible	N/A.
	init_waitqueue_head	Initializes the head of the wait queue.	Fully compatible	N/A.
Obtaining the validity of the wait queue	waitqueue_active	Checks whether the wait queue is available.	Fully compatible	Ensure that the input parameter is a valid pointer and cannot be null.
Waiting for an event	wait_event	Waits for an event.	Fully compatible	N/A.
	wait_event_interruptible	Waits for an event.	Not fully compatible. This event cannot be interrupted by signals.	N/A.

Function Category	API	Description	Compatibility	Precautions
	wait_event_interruptible_timeout	Waits for an event within a specified period of time.	Not fully compatible. This event cannot be interrupted by signals. The return values are different. The input parameter range is limited. For details, see the API reference.	N/A.
Waking up an event	wake_up_interruptible	Wakes up an event.	Not fully compatible. The value is returned directly when the input parameter is valid.	N/A.
	wake_up_interruptible_poll	Wakes up an event.	Fully compatible	N/A.
	wake_up	Wakes up an event.	Fully compatible	N/A.

NOTE

There are two types of wait queues:

- Cyclical waiting. The system exits the waiting state only after resources are obtained.
- Timed waiting. Even if the resource is not obtained within the specified period, the wait exits.

Development Procedure

The typical development process of the wait queue is as follows:

1. Open the menu and choose **Compat > Enable Linux > Enable waitqueue** to enable the wait queue.

Configuration Item	Description	Value Range	Default Value	Dependency
LOSCFG_COMPAT_LINUX_WAITQUEUE	Determines whether to enable the Linux wait queue.	YES/NO	NO	LOSCFG_BASE_IPC_EVENT

2. Call **DECLARE_WAIT_QUEUE_HEAD/init_waitqueue_head** to initialize the head of the wait queue.
3. Call **wait_event**, **wait_event_interruptible**, or **wait_event_interruptible_timeout** to wait for resources.
4. Call **wake_up**, **wake_up_interruptible**, or **wake_up_interruptible_poll** to wake up the wait queue.

6.6.3.3 Precautions

1. The wait queue of Huawei LiteOS does not support the signal interruption mechanism of the Linux kernel.
2. Huawei LiteOS does not support the addition of multiple wait items to a wait queue. A wait queue can contain only one wait item.

6.6.3.4 Programming Example

Description

The following functions are implemented:

1. Defining and initializing a wait queue
2. Creating a task, suspending an event in the task, and waiting for the event to be woken up
3. Waking up an event

Example Code

Prerequisite: The adaptation to Linux module has been enabled through the **menuconfig** menu.

The code is as follows:

```
#include "los_printf.h"
#include "los_task.h"
#include "linux/wait.h.h"

static UINT32 g_TestTaskID;
static DECLARE_WAIT_QUEUE_HEAD(g_waitQueue);
static BOOL g_condition = FALSE;

static VOID Example_Task(VOID)
{
    UINT32 timeout = 10; // 10 ticks timeout
```

```
UINT32 ret;

ret = wait_event_interruptible_timeout(g_waitQueue, g_condition, timeout);
if (ret > 0) {
    g_TestCount++;
}
}
UINT32 Example_Waitqueue(VOID)
{
    UINT32 ret;
    TSK_INIT_PARAM_S stTask = {0};

    stTask.pfnTaskEntry = (TSK_ENTRY_FUNC)Example_Task;
    stTask.usTaskPrio   = TSK_PRIOR_HI;
    stTask.pcName       = "wait_task";
    stTask.uwStackSize  = LOSCFG_BASE_CORE_TSK_MIN_STACK_SIZE;
    stTask.uwResved     = LOS_TASK_STATUS_DETACHED;

    g_TestCount = 0;

    ret = LOS_TaskCreate(&g_TestTaskID, &stTask);
    if (ret != LOS_OK) {
        printf("Example_Task create Failed!\r\n");
        return LOS_NOK;
    }

    LOS_TaskDelay(2);
    g_condition = TRUE;
    wake_up_interruptible(&g_waitQueue);
    LOS_TaskDelay(1);

    if (g_TestCount == 1) {
        printf("Wakeup succeeded!\r\n");
        return LOS_OK;
    } else {
        printf("Wakeup failed!\r\n");
        return LOS_NOK;
    }
}
```

Result Verification

The verification result is as follows:

```
Wakeup succeeded!
```

Complete Code

sample_waitqueue.rar

6.6.4 Interrupt

6.6.4.1 Overview

Basic Concepts

Linux kernel has dedicated interrupt functions. The interrupt mechanism of Huawei LiteOS adapts to the interrupt-related Linux APIs, making Huawei LiteOS more user-friendly.

Huawei LiteOS has the following interrupt-related functions:

- Requesting an interrupt
- Freeing an interrupt
- Enabling an interrupt
- Disabling an interrupt
- Interrupt bottom half (based on work queues)

Interrupt top half and bottom half

The interrupt handler must consume a short time to meet the requirements for quick response to interrupts. To balance the performance and workload, the interrupt handler is divided into two parts: the top half and the bottom half.

The top half completes very few urgent tasks. It simply registers information, for example, reads the interrupt status in the register and clears the interrupt flag bit, and sends time-consuming works of the bottom half handler to the bottom half work queue.

6.6.4.2 Development Guidelines

Functions

The following table lists Linux APIs to which the interrupt module in Huawei LiteOS adapts. For details about the APIs, see the API reference.

API	Description	Compatibility
request_irq	Allocates an interrupt and registers the interrupt handler with the linked list corresponding to the specified interrupt ID. Multiple interrupt handlers can be registered with one interrupt ID.	Not fully compatible. For details, see the API reference.
free_irq	Frees an interrupt.	Fully compatible
enable_irq	Enables an interrupt.	Fully compatible
disable_irq	Disables an interrupt.	Fully compatible

NOTE

The bottom half of the interrupt can be implemented by using the work queue. Initialize a work item in the top half and add it to the work queue. The system executes the bottom half of the work queue in idle hours. For details, see the [Work Queue](#).

Development Procedure

1. Open the menu and choose **Compat > Enable Linux > Enable irq interrupt** to enable the interrupt.

Configuration Item	Description	Value Range	Default Value	Dependency
LOSCFG_COMPAT_LINUX_IRQ	Determines whether to enable a Linux interrupt.	YES/NO	NO	LOSCFG_ARCH_INTERRUPT_TAKEOVER

2. Call the `request_irq` API to request an interrupt.
3. Call the `disable_irq` API to disable an interrupt.
4. Call the `enable_irq` API to enable an interrupt.
5. Call the `free_irq` API to free an interrupt.

6.6.4.3 Precautions

- While calling `request_irq()`, ensure that the input parameter of the interrupt handler is in the `(int, void*)` format. While calling `LOS_HwiCreate()`, note that the input parameter of the interrupt handler can be `NULL`.
- An interrupt handler is not allowed to call the `request_irq()` or `free_irq()` API.
- If an interrupt ID is shared among external peripherals, interrupts with the interrupt ID cannot be created using `LOS_HwiCreate()` and the input parameter `dev` pointer of the `request_irq()` API must match the interrupt handler instead of being `NULL`. Interrupts created using `LOS_HwiCreate()` cannot be deleted using `free_irq()`.

6.6.4.4 Programming Example

Example Description

The following functions are implemented:

1. Requesting an interrupt
2. Freeing an interrupt

Example Code

Prerequisites:

- The adaptation to Linux module has been enabled through the **menuconfig** menu.
- The maximum number of interrupts and the number of configurable interrupt priorities have been configured through the **menuconfig** menu. For details, see [1](#).

The code is as follows:

```
#include "los_hwi.h"
#include "linux/interrupt.h"

#define HWI_NUM_INT50 50
void Uart_Irqhandle(int irq, void *dev)
```



```
{
    printf("\nuart0:the function1 \n");
}
void Hwi_Test(void)
{
    int a = 1;
    void *dev = &a;
    unsigned long flags = 0;
    const char * name = "hwiTest";

    request_irq(HWI_NUM_INT50, Uart_Irqhandle, flags, name, dev); // Create the interrupt.
    free_irq(HWI_NUM_INT50, dev);                               // Free the interrupt.
}
```

Complete Code

sample_irq.c

6.6.5 High Resolution Timer

6.6.5.1 Overview

Basic Concepts

Using hardware timer resources and algorithms, high resolution timers satisfy the requirement for precise time.

A high resolution timer in Huawei LiteOS is designed with a software architecture and provides microsecond timing resolution, satisfying applications and kernel drivers' urgent need for precise time.

6.6.5.2 Development Guidelines

Application Scenario

The kernel's software timers provide millisecond-level timing, which is not high and does not meet the high resolution time requirements in some scenarios. High resolution timers provide microsecond-level timing APIs to satisfy aforementioned scenarios.

Functions

API	Description	Compatibility
hrtimer_init	Initializes the resources for a high resolution timer. This function has been implemented during kernel initialization. This API is only an adapted Linux API and is not implemented on Huawei LiteOS.	Fully compatible
hrtimer_create	Sets the timing period and creates the callback function.	Huawei LiteOS customized API

API	Description	Compatibility
hrtimer_start	Starts a high resolution timer.	Not fully compatible. The return values are different. For details, see the API reference.
hrtimer_cancel	Cancels the high resolution timers that have not expired.	Not fully compatible. The return values are different. For details, see the API reference.
hrtimer_forward	Changes the timing period of a high resolution timer that has not expired.	Not fully compatible. The input parameters are different. For details, see the API reference.
hrtimer_is_queued	Checks whether a high resolution timer is created. If the high resolution timer is created, 0 is returned. Otherwise, 1 is returned.	Not fully compatible. The return values are different. For details, see the API reference.

Development Procedure

The typical process of using a high resolution timer is as follows:

1. Open the menu and choose **Compat > Enable Linux > Enable Linux Hrtimer** to enable the high resolution timer.

Configuration Item	Description	Value Range	Default Value	Dependency
LOSCFG_COMPAT_LINUX_HRTIMER	Determines whether to enable a Linux high resolution timer.	YES/NO	YES	LOSCFG_COMPAT_LINUX

2. Declare an **hrtimer** variable and a **ktimer** variable.
3. Set the values of the **sec** and **usec** fields for the **ktimer** variable according to the timing period.
4. Call the **hrtimer_create** API to set the timing period and create the callback function.
5. Call the **hrtimer_start** API to start a high resolution timer.
6. Call the **hrtimer_cancel** and **hrtimer_forward** APIs to cancel or modify the timeout interval of the timer.
7. Call the **hrtimer_is_queued** API to find out whether a high resolution timer is created or expires.

6.6.5.3 Precautions

- The **hrtimer** variable required by high resolution timer APIs should be provided by users. The APIs do not allocate any resources for the variable.
- The **hrtimer_init** API is not implemented and is only an adapted Linux API. Therefore, this API does not need to be called when using high resolution timers.
- Currently, high resolution timers in Huawei LiteOS support relative timing mode rather than absolute timing mode.
- The high-precision timer is hardware dependent. Check that the hardware is supported before using it.
- The high-precision timer consumes instruction time. Therefore, the actual triggering time is greater than the configured time.

6.6.5.4 Programming Example

Example Description

The following functions are implemented:

1. Declare a **swtmr** variable and a time variable.
2. Call the **hrtimer_create** API to set the timing period and create the callback function.
3. Call the **hrtimer_start** API to start a high resolution timer.
4. Call the **hrtimer_forward** API to change the timing period from 80 ms to 40 ms.
5. Call the **LOS_Msleep** function to delay a task for 50 ms.
6. Call the **hrtimer_cancel** to cancel a timer. Then it is found that the timer has expired and does not exist.

Example Code

Prerequisite: The high resolution timer has been enabled.

The code is as follows:

```
#include "los_base.h"
#include "linux/hrtimer.h"

static enum hrtimer_restart Hrtimer_Func(struct hrtimer *arg)
{
    dprintf("The hrtimer is timeout!!!\n");
    return HRTIMER_NORESTART;
}

static UINT32 TestCase(VOID)
{
    struct hrtimer swtmr;
    union ktime time;
    union ktime interval = { 0 };
    interval.tv.sec = 0;
    interval.tv.usec = 40000;
    int ret;

    dprintf("----->test start<-----\n");

    time.tv.sec = 0;
```

```

time.tv.usec = 80000;
ret = hrtimer_create(&swtmr, time, Hrtimer_Func);
if (ret == 0) {
    dprintf("Hrtimer create successfully!\n");
}

ret = hrtimer_start(&swtmr, time, HRTIMER_MODE_REL);
if (ret == 0) {
    dprintf("Hrtimer start successfully!\n");
}

ret = hrtimer_forward(&swtmr, interval);

LOS_Msleep(50);

ret = hrtimer_cancel(&swtmr);
if (ret == 0) {
    dprintf("Hrtimer already timeout!\n");
}

dprintf("----->test end<-----\n");

return LOS_OK;
}

```

Verification

The verification result is as follows:

```

----->test start<-----
Hrtimer create successfully!
Hrtimer start successfully!
The hrtimer is timeout!!!
Hrtimer already timeout!
----->test end<-----

```

Complete Code

sample_hrtimer.c

6.6.6 Supported Linux APIs

Huawei LiteOS supports the following Linux APIs.

N o.	File Name	API	Description	Compatibility	Precautions
1	atomic.h	atomic_add	Add an integer to an atomic variable.	Fully compatible	N/A.
2	atomic.h	atomic_add_return	Add an integer to an atomic variable and return the new variable value.	Fully compatible	N/A.
3	atomic.h	atomic_dec	Decrement an atomic variable by one.	Fully compatible	N/A.

N o.	File Name	API	Description	Compatibility	Precautions
4	atomic.h	atomic_dec_and_test	Decrement an atomic variable by one and test whether the result is zero.	Fully compatible	N/A.
5	atomic.h	atomic_dec_return	Decrement an atomic variable by one and return the new variable value.	Fully compatible	N/A.
6	atomic.h	atomic_inc	Increment an atomic variable by one.	Fully compatible	N/A.
7	atomic.h	atomic_inc_return	Increment an atomic variable by one and return the new variable value.	Fully compatible	N/A.
8	atomic.h	atomic_read	Read the value of an atomic variable.	Fully compatible	N/A.
9	atomic.h	atomic_set	Set the value of an atomic variable.	Fully compatible	N/A.
10	atomic.h	atomic_sub	Subtract an integer from an atomic variable.	Fully compatible	N/A.
11	checksum.h	csum_partial	Verify the data.	Fully compatible	N/A.
12	checksum.h	csum_partial_copy_nocheck	Copy and verify the data.	Fully compatible	N/A.
13	checksum.h	in_cksum	Calculate the checksum.	Huawei LiteOS customized API	N/A.
14	checksum.h	in_cksum_copy	Calculate the checksum and copy it.	Huawei LiteOS customized API	N/A.
15	clock.h	sched_clock	Obtain the system time.	Fully compatible	N/A.

N o.	File Name	API	Description	Compatibility	Precautions
16	completion.h	complete	Wake up a thread waiting on a completion.	Not fully compatible. If the input parameter is null, the API exits.	N/A.
17	completion.h	complete_all	Wake up all threads waiting on a completion.	Not fully compatible. If the input parameter is null, the API exits.	N/A.
18	completion.h	init_completion	Initialize a completion.	Not fully compatible. If the input parameter is null, the API exits and is not initialized.	N/A.
19	completion.h	wait_for_completion	Wait for a completion.	Fully compatible	N/A.
20	completion.h	wait_for_completion_timeout	Wait for completion for a specified period of time. If the time expires, the waiting ends.	Not fully compatible. The invalid value judgment is added to the API implementation. For details, see the API reference.	N/A.
21	crc32.h	crc32	Calculates the CRC.	Fully compatible	N/A.
22	delay.h	msleep	Cause a handler sleep for some milliseconds.	Fully compatible	N/A.
23	delay.h	udelay	Cause a handler busy delay for some microseconds.	Fully compatible	N/A.

N o.	File Name	API	Description	Compatibility	Precautions
24	delay.h	mdelay	Cause a handler busy delay for some milliseconds.	Fully compatible	N/A.
25	hrtimer.h	hrtimer_cancel	Cancel the timer and wait for the handler to complete.	Not fully compatible. The return values are different and depend on the platform. The implementation requires adaptation. For details, see the API reference.	N/A.
26	hrtimer.h	hrtimer_create	Set the timing period and create a callback function.	Huawei LiteOS customized API, which depends on the platform and can be implemented only after adaptation.	N/A.
27	hrtimer.h	hrtimer_forward	The forwarding timer expires.	Not fully compatible. The input parameters are different and depend on the platform. The implementation requires adaptation. For details, see the API reference.	N/A.
28	hrtimer.h	hrtimer_init	Initialize the timer to a given clock.	Platform dependent, implemented only after adaptation	N/A.

N o.	File Name	API	Description	Compatibility	Precautions
29	hrtimer.h	hrtimer_is_queued	Help the handler check whether the timer is on a queue.	Not fully compatible. The return values are different and depend on the platform. The implementation requires adaptation. For details, see the API reference.	N/A.
30	hrtimer.h	hrtimer_start	Start or restart hrtimer on the current CPU.	Not fully compatible. The return values are different and depend on the platform. The implementation requires adaptation. For details, see the API reference.	N/A.
31	interrupt.h	disable_irq	Disable an interrupt.	Fully compatible	N/A.
32	interrupt.h	enable_irq	Enable an interrupt.	Fully compatible	N/A.
33	interrupt.h	free_irq	Free an interrupt.	Fully compatible	N/A.
34	interrupt.h	request_irq	Allocate an interrupt.	Fully compatible	N/A.
35	io.h	readb	Read one byte from I/O.	Fully compatible	N/A.
36	io.h	readl	Read four bytes from I/O.	Fully compatible	N/A.
37	io.h	readw	Read two bytes from I/O.	Fully compatible	N/A.

N o.	File Name	API	Description	Compatibility	Precautions
38	io.h	writéb	Write one byte to I/O.	Fully compatible	N/A.
39	io.h	writel	Write four bytes to I/O.	Fully compatible	N/A.
40	io.h	writew	Write two bytes to I/O.	Fully compatible	N/A.
41	jiffies.h	get_jiffies_64	Obtain the number of ticks since the system starts.	Fully compatible	N/A.
42	jiffies.h	jiffies_to_msecs	Convert ticks to milliseconds.	Fully compatible. The parameter value range is (0, 0xFFFFFFFF x LOSCFG_BASE_CORE_TICK_PER_SECOND / OS_SYS_MS_PER_SECOND].	N/A.
43	jiffies.h	msecs_to_jiffies	Convert milliseconds to ticks.	Not fully compatible. Currently, LOSCFG_BASE_CORE_TICK_PER_SECOND must be less than or equal to 1000 and can be exactly divided by 1000.	N/A.
44	kernel.h	div_s64	Do a division.	Fully compatible	N/A.

N o.	File Name	API	Description	Compatibility	Precautions
45	kernel.h	div_s64_rem	Do a division.	Fully compatible	Ensure that the input parameter is a valid pointer and cannot be a null pointer.
46	kernel.h	div_u64	Do a division.	Fully compatible	N/A.
47	kernel.h	div_u64_rem	Do a division.	Fully compatible	Ensure that the input parameter is a valid pointer and cannot be a null pointer.
48	kernel.h	div64_s64	Do a division.	Fully compatible	N/A.
49	kernel.h	div64_u64	Do a division.	Fully compatible	N/A.
50	kernel.h	div64_u64_rem	Do a division.	Fully compatible	Ensure that the input parameter is a valid pointer and cannot be a null pointer.
51	kernel.h	do_div_imp	Do a division.	Fully compatible	N/A.
52	kernel.h	do_div_s64_imp	Do a division.	Fully compatible	N/A.
53	kernel.h	ERR_PTR	Return an error code.	Fully compatible	N/A.
54	kernel.h	IS_ERR	Test whether a returned pointer is an error code.	Fully compatible	N/A.
55	kernel.h	PTR_ERR	Return an error code.	Fully compatible	N/A.

N o.	File Name	API	Description	Compatibility	Precautions
56	list.h	LIST_HEAD_INIT	Initialize the doubly linked table (the input parameter is not of the pointer type).	Fully compatible	N/A.
57	list.h	LIST_HEAD	Declare a variable and initialize a linked list header.	Fully compatible	N/A.
58	list.h	INIT_LIST_HEAD	Initialize the doubly linked table (the input parameter is of the pointer type).	Not fully compatible. Huawei LiteOS is a macro function. In earlier Linux versions, the macro function is used. In versions later than 4.9.37, the macro function is changed to the inline function.	Ensure that the input parameter is a valid pointer and cannot be a null pointer.
59	list.h	list_add	Add a new node next to the node.	Fully compatible	Ensure that the input parameter is a valid pointer and cannot be a null pointer.
60	list.h	list_add_tail	Add a new node before the node.	Fully compatible	Ensure that the input parameter is a valid pointer and cannot be a null pointer.

N o.	File Name	API	Description	Compatibility	Precautions
6 1	list.h	list_is_last	Determine whether a node is the tail node of a doubly linked table.	Fully compatible	Ensure that the input parameter is a valid pointer and cannot be a null pointer.
6 2	list.h	list_empty	Determine whether the linked list is empty.	Fully compatible	Ensure that the input parameter is a valid pointer and cannot be a null pointer.
6 3	list.h	list_entry	Obtain the address of the struct that contains the linked list based on the linked list address.	Fully compatible	Ensure that the input parameter is a valid pointer and cannot be a null pointer.
6 4	list.h	list_first_entry	Obtain the struct pointer that contains the first node of the linked list.	Fully compatible	Ensure that the input parameter is a valid pointer and cannot be a null pointer.
6 5	list.h	list_for_each	Traverse a linked list.	Fully compatible	Ensure that the input parameter is a valid pointer and cannot be a null pointer.
6 6	list.h	list_for_each_entry	Traverse the doubly linked table to obtain the struct address of the linked list node.	Fully compatible	Ensure that the input parameter is a valid pointer and cannot be a null pointer.

N o.	File Name	API	Description	Compatibility	Precautions
67	list.h	list_for_each_entry_reverse	Traverse the double-chain table to obtain the structure address of the linked list node.	Fully compatible	Ensure that the input parameter is a valid pointer and cannot be a null pointer.
68	list.h	list_for_each_entry_safe	Traverse the doubly linked table, obtain the struct address that contains the linked list node, and store the successor node of the current node for the next cycle. When the current node is deleted during the traversal, the successor node can be found.	Fully compatible	Ensure that the input parameter is a valid pointer and cannot be a null pointer.
69	list.h	list_for_each_safe	Traverse the bidirectional linked list and store the subsequent nodes of the current node for the next cycle. When the current node is deleted during the traversal, the subsequent nodes can be found.	Fully compatible	Ensure that the input parameter is a valid pointer and cannot be a null pointer.
70	list.h	list_del	Delete a node from a doubly linked table.	Fully compatible	Ensure that the input parameter is a valid pointer and cannot be a null pointer.

N o.	File Name	API	Description	Compatibility	Precautions
7 1	list.h	list_move	Delete the node from the original linked list and insert the node to another linked list as the head node.	Fully compatible	Ensure that the input parameter is a valid pointer and cannot be a null pointer.
7 2	list.h	list_del_init	Delete the node and re-initialize the node.	Fully compatible	Ensure that the input parameter is a valid pointer and cannot be a null pointer.
7 3	platform_device.h	platform_device_register	Register a device.	Fully compatible	N/A.
7 4	platform_device.h	platform_device_unregister	Deregister a device.	Fully compatible	N/A.
7 5	platform_device.h	platform_driver_register	Register a driver.	Fully compatible	N/A.
7 6	platform_device.h	platform_driver_unregister	Deregister a driver.	Not fully compatible. The return values are different. For details, see the API reference.	N/A.
7 7	platform_device.h	platform_get_drvdata	Obtain private data on a driver.	Fully compatible	N/A.
7 8	platform_device.h	platform_get_irq	Obtain the device interrupt ID.	Fully compatible	N/A.
7 9	platform_device.h	platform_get_resource	Obtain device resources.	Fully compatible	N/A.

N o.	File Name	API	Description	Compatibility	Precautions
80	platform_device.h	platform_ioremap_resource	Obtain the device memory address.	Fully compatible	N/A.
81	platform_device.h	platform_set_drvdata	Set private data on a driver.	Fully compatible	N/A.
82	rbtree.h	rb_erase	Remove a node.	Fully compatible	N/A.
83	rbtree.h	rb_first	Return the first node.	Fully compatible	N/A.
84	rbtree.h	rb_insert_color	Color the inserted node.	Fully compatible	N/A.
85	rbtree.h	rb_next	Return the next node.	Fully compatible	N/A.
86	rbtree.h	rb_prev	Return the previous node.	Fully compatible	N/A.
87	rbtree.h	rb_replace_node	Replace a node.	Fully compatible	N/A.
88	rtc.h	is_leap_year	Determine whether a specified year is a leap year.	Fully compatible	N/A.
89	rtc.h	rtc_time_to_tm	Convert absolute time to year, month, day, hour, minute, and second.	Fully compatible	N/A.
90	rtc.h	rtc_tm_to_time	Convert year, month, day, hour, minute, and second to absolute time.	Fully compatible	N/A.
91	rwsem.h	DECLARE_RWSEM	Declare and initialize a read/write semaphore.	Fully compatible	N/A.
92	rwsem.h	init_rwsem	Initialize a read/write semaphore.	Fully compatible	N/A.

N o.	File Name	API	Description	Compatibilit y	Precautions
9 3	rwsem. h	down_read	Obtain a read semaphore. If the semaphore cannot be obtained, the system enters the sleep state.	Fully compatible	N/A.
9 4	rwsem. h	down_read_trylock	Try to obtain the read semaphore. The system will not sleep.	Fully compatible	N/A.
9 5	rwsem. h	down_write	Obtain a read semaphore. If the semaphore cannot be obtained, the system enters the sleep state.	Fully compatible	N/A.
9 6	rwsem. h	down_write_trylock	Try to obtain the read semaphore. The system will not sleep.	Fully compatible	N/A.
9 7	rwsem. h	up_read	Release a read semaphore.	Fully compatible	N/A.
9 8	rwsem. h	up_write	Release a write semaphore.	Fully compatible	N/A.
9 9	rwsem. h	downgrade_write	Degrade the write semaphore to the read semaphore.	Fully compatible	N/A.
1 0 0	scatterlist.h	sg_init_one	Initialize a scatter list.	Fully compatible	Ensure that the input parameter is a valid pointer and cannot be a null pointer.

N o.	File Name	API	Description	Compatibilit y	Precautions
1 0 1	scatterlist.h	sg_init_table	Initialize a scatter list.	Fully compatible	Ensure that the input parameter is a valid pointer and cannot be a null pointer.
1 0 2	scatterlist.h	sg_mark_end	Mark the end of a scatter list.	Fully compatible	Ensure that the input parameter is a valid pointer and cannot be a null pointer.
1 0 3	scatterlist.h	sg_set_buf	Set a scatter list.	Fully compatible	Ensure that the input parameter is a valid pointer and cannot be a null pointer.
1 0 4	sched.h	schedule_timeout	Delay a task by #timeout and wakes up the task when the time arrives. When the value of timeout is 0, the actual delay is 1 tick due to the time difference of the timer.	Fully compatible	N/A.
1 0 5	semaphore.h	DEFINE_SEMAPHORE	Declare a variable of the struct semaphore type as a semaphore based on the input parameter and initializes the semaphore.	Fully compatible	N/A.

N o.	File Name	API	Description	Compatibility	Precautions
106	semaphore.h	sema_init	Initialize a semaphore dynamically .	Not fully compatible. If the count value is within the range of #val<0 or #val>1, the system does not set the count value to 0 and does not initialize the count value, and an error message is displayed.	N/A.
107	semaphore.h	down	Obtain the semaphore. If the semaphore cannot be obtained, the system enters the sleep state and cannot be woken up.	Note fully compatible. The count value > 1 is not implemented.	N/A.
108	semaphore.h	down_interruptible	Obtain the semaphore. Otherwise, the system enters the sleep state. After the semaphore is released, activate the process.	Note fully compatible. The count value > 1 is not implemented. There is no single wakeup mode and the return values are different. For details, see the API Reference.	N/A.

N o.	File Name	API	Description	Compatibilit y	Precautions
1 0 9	semaph ore.h	down_trylock	Attempts to acquire a semaphore. If the semaphore is not available, down_trylock immediately returns with a nonzero return value, and the caller will not wait on the semaphore.	Note fully compatible. The count value > 1 is not implemented.	N/A.
1 1 0	semaph ore.h	up	Release a semaphore.	Note fully compatible. The count value > 1 is not implemented.	N/A.
1 1 1	semaph ore.h	sema_destory	Destroy semaphore and reclaim kernel memory resources.	Non-standard interface.	N/A.
1 1 2	seq_file .h	seq_lseek	Deviate the pointer to a sequential file.	Fully compatible	N/A.
1 1 3	seq_file .h	seq_open	Open a sequential file.	Fully compatible	N/A.
1 1 4	seq_file .h	seq_printf	Write formatted information to a sequential file.	Fully compatible	N/A.
1 1 5	seq_file .h	seq_read	Read a sequential file.	Fully compatible	N/A.
1 1 6	seq_file .h	seq_release	Free the dynamic memory allocated by sequential files.	Fully compatible	N/A.
1 1 7	seq_file .h	single_open	Open a sequential file.	Fully compatible	N/A.

N o.	File Name	API	Description	Compatibilit y	Precautions
1 1 8	seq_file .h	single_release	Free the dynamic memory allocated by sequential files.	Fully compatible	N/A.
1 1 9	slab.h	kfree	Free memory.	Fully compatible	N/A.
1 2 0	slab.h	kmalloc	Allocate memory.	Fully compatible	N/A.
1 2 1	slab.h	kzalloc	Allocate memory and initialize the memory.	Fully compatible	N/A.
1 2 2	slab.h	vfree	Free memory.	Fully compatible	N/A.
1 2 3	slab.h	vmalloc	Allocate memory.	Fully compatible	N/A.
1 2 4	spinlock_types .h	DEFINE_SPINLOCK	Declare a variable of the spinlock_t type as a spinlock based on the input parameter and initialize the spinlock.	Fully compatible	N/A.
1 2 5	spinlock.h	spin_lock_init	Initialize the specified spinlock dynamically.	Fully compatible	N/A.
1 2 6	spinlock.h	spin_lock	Obtain the specified spinlock.	Fully compatible	N/A.
1 2 7	spinlock.h	spin_unlock	Release the specified spinlock.	Fully compatible	N/A.

N o.	File Name	API	Description	Compatibilit y	Precautions
1 2 8	spinlock.h	spin_lock_irqsave	Save the current status of the local interrupt, disable the local interrupt, and obtain the specified spinlock.	Fully compatible	N/A.
1 2 9	spinlock.h	spin_unlock_irqrestore	Release the specified spinlock and restore the local interrupt to the previous state.	Fully compatible	N/A.
1 3 0	string.h	strncpy	Copy a string.	Fully compatible	N/A.
1 3 1	timer.h	add_timer	Add a timer.	Fully compatible. The parameters are as follows: The value range of <i>timer->expires</i> is [0, 0xFFFFFFFF].	N/A.
1 3 2	timer.h	del_timer	Delete a timer.	Fully compatible	N/A.
1 3 3	timer.h	del_timer_sync	Delete a timer.	Fully compatible	N/A.
1 3 4	timer.h	init_timer	Initialize a timer.	Fully compatible	N/A.
1 3 5	timer.h	mod_timer	Add a timer.	Fully compatible	N/A.
1 3 6	wait.h	init_waitqueue_head	Initialize an existing wait queue head.	Fully compatible	N/A.

N o.	File Name	API	Description	Compatibility	Precautions
137	wait.h	wait_event	Wait event	Fully compatible	N/A.
138	wait.h	wait_event_interruptible	Wait event	Not fully compatible. This event cannot be interrupted by signals.	N/A.
139	wait.h	wait_event_interruptible_timeout	Timeout wait event	Not fully compatible. The input parameter range is fixed, and the return values are different. For details, see the API reference.	N/A.
140	wait.h	waitqueue_active	Test whether a wait queue is empty.	Fully compatible	N/A.
141	wait.h	wake_up	Wake up a task.	Fully compatible	N/A.
142	wait.h	wake_up_interruptible	Wake up a task that has been put to an interruptible sleep.	Not fully compatible. There is no signal wakeup mode	N/A.
143	wait.h	wake_up_interruptible_poll	Wake up a task.	Fully compatible	N/A.
144	wait.h	DECLARE_WAIT_QUEUE_HEAD	Define and initialize a wait queue.	Fully compatible	N/A.
145	workqueue.h	cancel_delayed_work	Cancel a pending delayed work.	Fully compatible	N/A.

N o.	File Name	API	Description	Compatibilit y	Precautions
1 4 6	workqu eue.h	cancel_delayed _work_sync	Cancel a pending delayed work and wait for its execution to finish.	Not fully compatible. The work item is not valid, the work item has already been executed. Returns an error when a work item is executing or has already been executed.	N/A.
1 4 7	workqu eue.h	cancel_work_sy nc	Cancel a work and wait for its execution to finish.	Not fully compatible. The work item is not valid, the work item has already been executed.	N/A.
1 4 8	workqu eue.h	create_singleth read_workqueu e	Create a workqueue.	Fully compatible	N/A.
1 4 9	workqu eue.h	create_workqu eue	Create a workqueue.	Not fully compatible. Huawei LiteOS creates only one work queue on the current CPU. In the multi- CPU scenario, Linux creates a work queue on each CPU.	N/A.
1 5 0	workqu eue.h	destroy_workq ueue	Destroy a workqueue.	Fully compatible	N/A.

N o.	File Name	API	Description	Compatibility	Precautions
151	workqueue.h	flush_delayed_work	Wait for a delayed work to finish executing.	Fully compatible	N/A.
152	workqueue.h	flush_work	Wait for a work to finish executing.	Fully compatible	N/A.
153	workqueue.h	INIT_DELAYED_WORK	Initialize a delayed work.	Fully compatible	N/A.
154	workqueue.h	INIT_WORK	Initialize a work.	Fully compatible	N/A.
155	workqueue.h	queue_delayed_work	Add a delayed work to a specified workqueue.	Fully compatible	N/A.
156	workqueue.h	queue_work	Add a work to a specified workqueue.	Not fully compatible. Validity of parameters is checked.	N/A.
157	workqueue.h	schedule_delayed_work	Add a delayed work to the default workqueue.	Not fully compatible. Validity of parameters is checked.	N/A.
158	workqueue.h	schedule_work	Add a work to the default workqueue.	Not fully compatible. Validity of parameters is checked.	N/A.
159	workqueue.h	work_busy	Test the status of a work.	Not fully compatible. Validity of parameters is checked.	N/A.

NOTE

Fully compatible indicates that the functions are the same as those of Linux. The returned error codes are subject to the actual codes. **Partially compatible** indicates that some functions are the same as those of Linux.

6.6.7 Overview of APIs Not Supported by Linux

Some Linux APIs are not supported by Huawei LiteOS, including but not limited to those listed below.

Header File	API	Description	Compatibility
bug.h	BUG	Provide assertions and dumps information.	Not supported
compiler.h	likely	Conditional statement that indicates that the value is likely to be true.	Not supported
compiler.h	unlikely	Conditional statement that indicates that the value is likely to be false.	Not supported
fb.h	fb_alloc_cmap	Allocates memory to a color map.	Not supported
fb.h	fb_cmap_to_user	Copies a color map.	Not supported
fb.h	fb_copy_cmap	Copies a color map.	Not supported
fb.h	fb_dealloc_cmap	Deallocates a color map that was previously allocated.	Not supported
fb.h	fb_default_cmap	Sets the default color map.	Not supported
fb.h	fb_pan_display	Refreshes the operation screen.	Not supported
fb.h	fb_set_cmap	Sets a color map.	Not supported
fb.h	fb_set_user_cmap	Sets a color map.	Not supported
fb.h	fb_set_var	Sets the display mode of fbinfo and the variadic arguments.	Not supported

Header File	API	Description	Compatibility
fb.h	framebuffer_alloc	Allocates memory from the kernel space.	Not supported
fb.h	framebuffer_release	Releases space to the kernel.	Not supported
fb.h	register_framebuffer	Registers fbinfo into the kernel.	Not supported
fb.h	unregister_framebuffer	Releases fbinfo.	Not supported
kernel.h	copy_from_user	Copies data from user space to kernel space.	Not supported
kernel.h	copy_to_user	Copies data from kernel space to user space.	Not supported
kernel.h	ioremap_cached	Maps physical memory to kernel virtual address space.	Not supported
kernel.h	ioremap_nocache	Maps physical memory to kernel virtual address space.	Not supported
kernel.h	iounmap	Cancels the mapping made by ioremap.	Not supported
module.h	module_put	Decreases the number of times the module is referenced.	Not supported
module.h	try_module_get	Increases the number of times the module is referenced.	Not supported
pm.h	dpm_resume_end	Resumes all non-system devices, that is, executes the resume callback function of all registered devices.	Not supported
pm.h	dpm_suspend_start	Suspends all non-system devices, that is, executes the suspend callback function of all registered devices.	Not supported

Header File	API	Description	Compatibility
sched.h	cond_resched	Schedules a new process for running.	Not supported
sched.h	signal_pending	Checks whether the current process is processes by a signal.	Not supported
semaphore.h	init_MUTEX	Initializes a semaphore and set the semaphore value to one.	Not supported
semaphore.h	init_MUTEX_LOCKED	Initialize a semaphore and sets the semaphore value to zero.	Not supported
string.h	simple_strtol	Converts a string to an unsigned long long number.	Not supported
suspend.h	pm_resume	Resumes the system.	Not supported
suspend.h	pm_suspend	Suspends the system.	Not supported
timer.h	timer_pending	Checks whether a timer in the timer management queue has been scheduled and executed.	Not supported
wait.h	add_wait_queue	Adds a process to the tail of a queue.	Not supported
wait.h	DECLARE_WAITQUEUE	Defines and initializes a wait queue.	Not supported
wait.h	remove_wait_queue	Removes a wait queue from the wait queue linked list pointed to by the wait queue head associated with the wait queue to be removed.	Not supported
wakelock.h	wake_lock	Activates a lock.	Not supported
wakelock.h	wake_lock_active	Checks whether a lock is active.	Not supported

Header File	API	Description	Compatibility
wakelock.h	wake_lock_init	Initializes a lock.	Not supported
wakelock.h	wake_unlock	Unlocks and deactivates a lock.	Not supported

7 Driver Development

This section takes Hi3556 V200 as an example to describe how to operate and control Huawei LiteOS peripheral driver devices connected to driver modules such as the ETH, USB 2.0 DRD, and SD/MMC/eMMC card.

NOTE

Hi3556 V200/Hi3559 V200 does not have the ETH module.

[7.1 Driver Framework](#)

[7.2 File Adaptation](#)

[7.3 MMC Driver Development](#)

[7.4 USB Driver Development](#)

[7.5 UART Driver Development](#)

7.1 Driver Framework

7.1.1 Overview

Basic Concepts

As shown in Figure 1, driver development indicates hardware implementation and abstraction according to certain specifications, so as to provide a set of unified APIs for developers to develop upper-layer services.

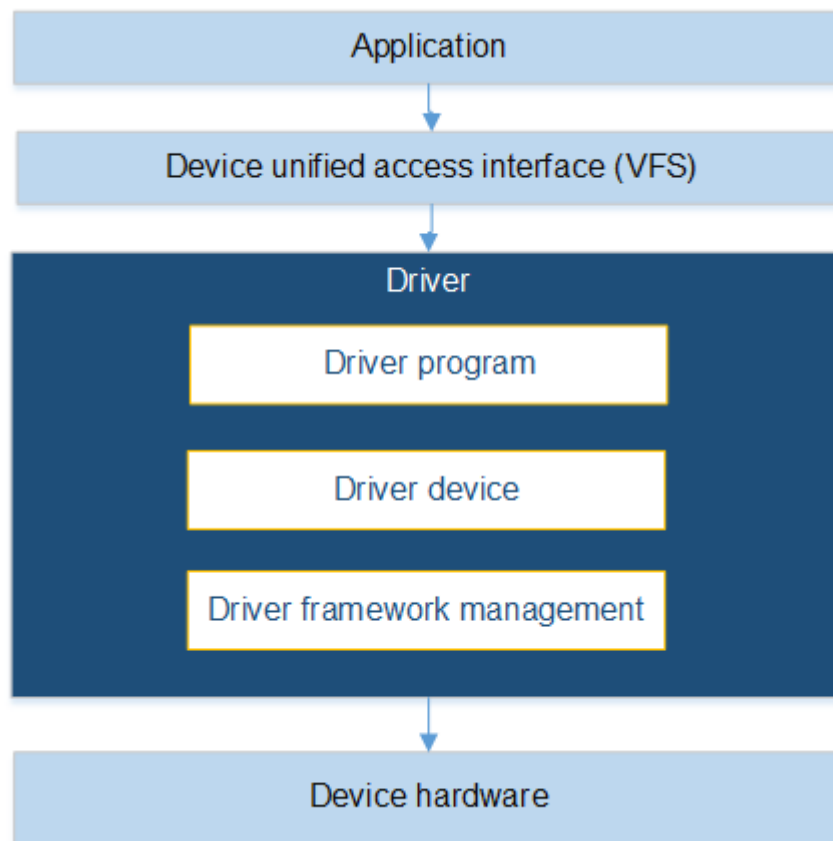
1. Driver devices

The system considers all external devices as a type of special files, which are called device files. External devices are classified into the following types:

- Character devices: send and receive data in the form of characters. Random access is not supported.
- Block devices: A block of system memory is used as the buffer. All data is transmitted through the buffer. Block devices are design for slow devices such as flash memory, preventing CPU from wasting too much time in waiting.

- Network devices: All visits to network hardware are through APIs, which are a set of operations used to process the sending and receiving of basic data for all types of network hardware.
2. **Driver programs**
APIs between kernel and external devices. Driver programs shield details of hardware implementation and enables applications to operate external devices as common files. Applications can perform operations such as open (), close (), read (), and write () on hardware devices.
 3. **Driver framework management**
Driver framework management is a module used to manage driver devices and driver programs.

Figure 7-1 Basic device driver framework



Operation Mechanism

The driver framework involves two important data structures: **LosDevice** (used to describe driver devices) and **LosDriver** (used to describe drivers). There are two global bidirectional linked lists in the system, which respectively maintain all mounted driver devices and driver programs in the system. One is a global device linked list, which manages all mounted driver devices, and the other is a global driver linked list, which manages all mounted driver programs.

The following describes the two data structures:

1. **LosDevice**

LosDevice represents a driver device. The important members in the structure are as follows:

- **const CHAR *name**: device name, which is used to match the driver.
- **struct LosDriver *driver**: driver program of the driver device.
- **struct LosDeviceConfig cfg**: configures the start address, size, and interrupt ID of the device I/O.
- **LOS_DL_LIST driverNode**: corresponding driver node, which is mounted to deviceList in the driver structure.
- **LOS_DL_LIST deviceItem**: driver device node, which is mounted to the global device linked list.

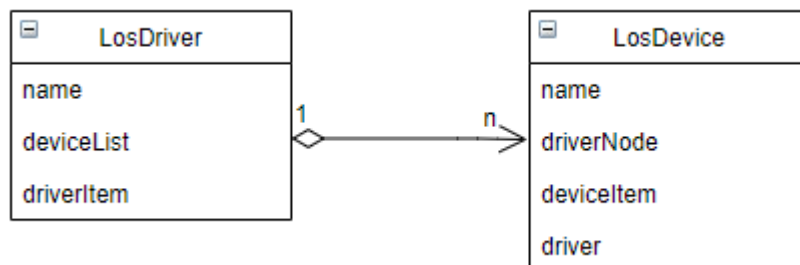
2. LosDriver

LosDriver represents a device driver program. The important members in the structure are as follows:

- **const CHAR *name**: driver name, which is used to match the device.
- **LOS_DL_LIST deviceList**: bidirectional linked list of the driver device that uses the current driver program.
- **LOS_DL_LIST driverItem**: driver program node, which is mounted to the global driver linked list.
- **struct LosDriverOps ops**: operation APIs implemented by the driver, such as probe and remove.

The following figure shows the relationship between **LosDevice** and **LosDriver**.

Figure 7-2 Relationship between **LosDevice** and **LosDriver**



LosDriver is mounted to the global driver linked list and contains a deviceList linked list, indicating all devices operated (or controlled) by the driver program.

LosDevice is mounted to the global device linked list and contains a driver pointer, indicating the device driver program corresponding to the device.

One driver device corresponds to only one driver program, but one driver program may support multiple driver devices. When the corresponding device/driver is mounted, the corresponding registered driver/device is searched in the global driver/device linked list according to the name, and the registration process is executed.

7.1.2 Development Guidelines

Functions

The driver framework of Huawei LiteOS provides the following functions. For details about the APIs, see the API reference.

Function Category	API	Description
Driver device operation	LOS_DeviceRegister	Registers a device and adds it to the global device linked list.
	LOS_DeviceUnregister	Deregisters a device and removes it from the global device linked list.
	LOS_DeviceDataGet	Obtains the private data of a specified device.
	LOS_DeviceRegBaseGet	Obtains the I/O start address of a specified device.
	LOS_DeviceRegSizeGet	Obtains the I/O size of a specified device.
	LOS_DeviceIrqNumGet	Obtains the interrupt ID of a specified device.
Driver program registration and deregistration	LOS_DriverRegister	Registers the driver program and bind it to the corresponding device.
	LOS_DriverUnregister	Deregisters the driver program.
Low power consumption	LOS_PmSuspended	Suspends all devices.
	LOS_PmResume	Resumes all devices.

Development Procedure

The following uses the USB driver registration as an example to describe the driver development process based on the driver framework.

- Step 1** Open the menu and choose **Driver > Enable Driver Base** to enable the driver framework.

Configuration Item	Description	Value Range	Default Value	Dependency
LOSCFG_DRIVERS_BASE	Determines whether to enable the driver framework.	YES/NO	YES	None.

Step 2 Register a driver device.

1. Define the USB driver device instance.

```
struct LosDeviceRegs hiudc3_regs[] = {
    {
        .base = DWC_USB3_PORT1_BASE_ADDR,
        .size = DWC_USB3_PORT1_ADDR_OFFSET,
    },
};
static struct LosDevice hiudc3_device = {
    .name      = "hi_udc3",
    .id        = -1,
    .cfg       = {

        .irqNum = NUM_HAL_INTERRUPT_USB_DEV,

        .numRegs = ARRAY_SIZE(hiudc3_regs),
        .regs = hiudc3_regs,
    },
};
```

2. Register the driver device with the global device linked list.

```
LOS_DeviceRegister(&hiudc3_device);
```

3. Perform preloading operations.

Call the pre-loading mounting function in **linux/module.h** (in the **compat/linux/include** directory under the root directory) to mount the driver device registration function to the system. When the system is started, preloading nodes are traversed according to the priority of mounting functions, in order to register driver devices. Following pre-loading mounting functions with the descending priority are provided by the system:

```
pure_initcall(f)    //0
core_initcall(f)    //1
postcore_initcall(f) //2
arch_initcall(f)    //3
subsys_initcall(f)  //4
fs_initcall(f)      //5
device_initcall(f)  //6
late_initcall(f)    //7
```

For details, see [Programming Example](#).

NOTICE

The execution of preloading may affect some system functions. For example, if driver initialization takes a long time, the fast startup phase of scatter loading may be affected. Currently, only pre-loading functions with the priority level less than or equal to 3 are supported.

Step 3 Register a driver program.

1. Define USB driver program instance.

```
static struct LosDriver hiudc3_driver = {
    .name = "hi_udc3",
    .ops = {
        .probe = hiudc3_probe,
        .remove = hiudc3_remove,
    },
    .pmOps = {
        .suspend = hiudc3_suspend,
        .resume = hiudc3_resume,
    },
};
```

 NOTE

probe is used to initialize the driver program.

remove is used to deinitialize the driver program.

The **name** attribute of driver program must be consistent with the **name** of the corresponding driver device.

suspend/resume: suspends or resumes a driver device.

2. Register a USB driver program.

```
LOS_DriverRegister(&hiudc3_driver);
```

Step 4 Register driven instances to the file system.

Driver instances are registered with the file system, so that users are allowed to perform operations to drivers as to files. For details, see [File Adaptation](#).

----End

7.1.3 Precautions

If you want to invoke the preloading driver function, add the **-u** to the **LITEOS_TABLES_DRIVER_LDFLAGS** of **build/mk/liteos_tables_ldflags.mk**. Otherwise, linker will optimize driver initialization module, causing unpredictable errors.

```
LITEOS_TABLES_DRIVER_LDFLAGS := \
-ui2c_init \
-ugpio_init \
-uregulator_init \
-umtd_init_list \
```

7.1.4 Programming Example

Example Description

The following describes how to add a USB driver.

Example Code

Prerequisites: Enable the driver framework in **menuconfig**.

- Step 1 Define the driver device in the **board.c** directory of the specified development board.

```
/* Define the driver device instance. */
#ifdef LOSCFG_DRIVERS_USB3_DEVICE_CONTROLLER
struct LosDeviceRegs hiudc3_regs[] = {
{
.base = DWC_USB3_PORT1_BASE_ADDR,
.size = DWC_USB3_PORT1_ADDR_OFFSET,
},
};

static struct LosDevice hiudc3_device = {
.name = "hi_udc3",
.id = -1,
.cfg = {
.irqNum = NUM_HAL_INTERRUPT_USB_DEV,
.numRegs = ARRAY_SIZE(hiudc3_regs),
.reg = hiudc3_regs,
},
};
```

```

#endif

int machine_init(void)
{
#ifdef LOSCFG_DRIVERS_USB3_DEVICE_CONTROLLER
    (void)LOS_DeviceRegister(&hiudc3_device);
#endif
}
arch_initcall(machine_init); // Register preloading driver device. When the kernel is initialized, the
preloading driver device is registered.

```

Step 2 Define the USB device driver in the **drivers\usb\adapt_liteos\controller\usb_device** directory.

```

/* Operation API implemented by the driver program */
static int hiudc3_probe(struct LosDevice *pdev)
{
    .....
}

static int hiudc3_remove(struct LosDevice *pdev)
{
    .....
}

static int hiudc3_suspend(struct LosDevice *pst_dev)
{
    .....
}

static int hiudc3_resume(struct LosDevice *pst_dev)
{
    .....
}

/* Define the driver program example. */
static struct LosDriver hiudc3_driver = {
    .name = "hi_udc3",
    .ops = {
        .probe = hiudc3_probe,
        .remove = hiudc3_remove,
    },
    .pmOps = {
        .suspend = hiudc3_suspend,
        .resume = hiudc3_resume,
    },
};

/* Initialize the driver program, that is, register the driver program. */
int hiudc3_init(void)
{
    return (int)LOS_DriverRegister(&hiudc3_driver);
}

/* Deregister the driver program. */
int hiudc3_exit(void)
{
    return (int)LOS_DriverUnregister(&hiudc3_driver);
}

```

----End

7.2 File Adaptation

7.2.1 Overview

Basic Concepts

Driver development means implementing and abstracting functions of hardware based on OS specifications and then provides them to application developers to call.

While transplanting system on a new chip, driver development must be carried out based on peripheral equipment supported by the chip specifications.

The driver initialization function for Huawei LiteOS is mainly used to create a driver structure of devices and generate the control node that registers drivers for the upper platform.

7.2.2 Development Guidelines

Development Procedure

It is recommended that driver developers use the VFS framework to register and uninstall device. Driver development is related to following steps:

1. Open the menu and choose **FileSystem > Enable VFS** to enable VFS.

Configurati on Item	Description	Value Range	Default Value	Depend ency
LOSCFG_FS_VFS	Specifies whether to enable the VFS.	YES/NO	YES	None.

2. [Driver Initialization.](#)
3. [Driver Node Operation.](#)

Driver Initialization

The first step of driver development is to compile a driver initialization function. In Huawei LiteOS, the driver initialization function is used to initialize a driver structure of devices and generate the driver control node.

The driver initialization function must be used together with the device driver registration function that is used to register and generate a driver node.

```
register_driver(FAR const char *path, FAR const struct file_operations_vfs *fops, mode_t mode, FAR void *priv)
register_blockdriver(FAR const char *path, FAR const struct block_operations *bops, mode_t mode, FAR void *priv)
```

Table 1 describes the parameters expected by the device driver registration function.

Table 7-1 Parameter description

Parameter	Description
path	Path of the driver node. Application programs will access driver node through this path, and then access operation API supported by device driver.
fops/bops	Driver operation structure that provides application programs with an operation function set. fops indicates a character device, and bops indicates a block device.
mode	Application's permission configuration to read from or write into the driver node. This parameter is not supported now.
priv	Parameter to be passed in during driver node registration.

After the driver program initialization function is compiled, you need to boot this initialization function in a right place. The codes in **sample.c** file under the **/sample/sample_osdrv** directory are used as a simple boot. You can call the compiled device initialization function in the **app_init** function to boot device initialization.

Driver Node Operation

The device driver node generated after driver initialization provides an interface for applications to access and operate devices. The call relationship between application programs and driver operation functions is described with the i2c device driver as an example.

Operation function set

The driver operation function set is essential to the call relationship between application programs and driver operation functions. During driver compilation, the operation function set needs to implement various mechanisms of a hardware device and register the mechanisms during device registration. The operation function set meets all requirements on application functions.

Table 2 shows the operation function set provided by the i2c device driver.

Table 7-2 Operation function set for the I2C device driver

Operation Function Set	Application-Layer API
i2cdev_open	open
i2cdev_release	close
i2cdev_read	read
i2cdev_write	write
i2cdev_ioctl	ioctl

1. open Operation

When an application program opens a node file, Huawei LiteOS will call the open function in the driver operation function set used when the driver node is initialized.

The open function instantiates and initializes the driver structure. After an application program opens a node file, the file descriptor of the driver node will be got. The program can then access the driver that has the file descriptor.

2. read/write Operation

The read/write operation is a common way for applications to access the driver. The functions of the read/write operation vary depending on the type of devices and drivers. For an i2c device, the read/write operation is used to read from or write into i2c peripherals.

```
i2cdev_read(struct file * filep, char __user *buf, size_t count)
i2cdev_write(struct file * filep , const char __user *buf, size_t count)
```

Table 3 describes the parameters expected by the read/write function.

Table 7-3 Parameter description

Parameter	Description
filep	Pointer to the file description structure.
buf	Buffer that stores the read or written data.
count	Length of the read or written data.

3. ioctl Operation

An ioctl operation provides driver configuration management functions, particularly, defines or accesses device attributes in the device driver by running specific commands. You can run commands to configure or access device attributes.

For an i2c device, the I2C_16BIT_REG command is used to define the bit width of a transmission register, the I2C_16BIT_DATA command is used to define the bit width of transmitted data, and the I2C_TIMEOUT command is used to define the command expiry time.

```
i2cdev_ioctl(struct file *filep, int cmd, unsigned long arg)
```

Table 4 describes the parameters expected by the ioctl function.

Table 7-4 Parameter description

Parameter	Description
filep	Pointer to the file description structure.
cmd	Command for an operation
arg	Additional parameter

4. close Operation

The close operation corresponds to the release function in the driver operation function set. The release function is used to release driver resources.

7.2.3 Precautions

None.

7.3 MMC Driver Development

7.3.1 Overview

Multimedia Card (MMC) is an embedded storage standard formulated by the MultiMedia Card Association (MMCA) for mobile phones and tablets. The MMC driver is used to identify, read, and write the secure digital memory card (SD card) and embedded multimedia card (eMMC), and supports extended peripherals such as Bluetooth, Wi-Fi, and GPS through the SDIO protocol.

Huawei LiteOS MMC driver supports the TF card, eMMC memory, and Wi-Fi extended peripherals.

- T-Flash (TF) card, also called micro SD card.
The TF card is mounted as the FAT file system to implement the read and write operations on the device. Before mounting the TF card, you need to identify the device. You can also perform the read and write operations on the raw TF card through the raw read and write APIs.

NOTICE

Currently, Huawei LiteOS does not support user-defined TF card partitions. If necessary, you need to partition the TF card on the PC in advance.

- eMMC memory
The eMMC is mounted as the FAT file system to implement the read and write operations on the device. Before mounting the eMMC, you need to call the API to customize partitions and then identify the device. You can also perform the read and write operations on the raw eMMC memory through the raw read and write API.
- SDIO extended peripherals
Currently, only the Wi-Fi module is supported. The device uses the Wi-Fi module for communication and data transmission. Before device identification, pin multiplexing must be configured based on hardware requirements.

Huawei LiteOS MMC driver module consists of the protocol layer and control layer:

- The protocol layer supports the SD, eMMC, and SDIO protocols.
- The control layer supports the Himci and Sdhci controllers. The corresponding controller driver is selected based on the hardware IP address for various chip platforms.

7.3.2 Development Guidelines

MMC Device Identification

The MMC devices in Huawei LiteOS consist of embedded and non-embedded devices:

- Embedded devices include eMMCs and SDIO extended peripherals, which are not hot-swappable. Currently, the eMMC device can be identified only for the first time. The extended SDIO peripheral can be unloaded and loaded by calling **hisi_sdio_rescan** (depending on the software power-on and power-off of the extended peripheral).
- The non-embedded device is a TF card. The TF card supports hot swap and can be dynamically loaded and unloaded through thread polling.

NOTICE

Except that the extended SDIO peripherals can be identified by calling **sdio_rescan** API, the identification of other MMC devices is implemented in the same thread. When there are multiple MMC devices, the devices are identified in serial mode based on the controller ID sequence.

Driver Initialization APIs

API	Description
SD_MMC_Host_init	Traverses and initializes all controllers on the platform.
MMC_HostInitById	Initializes the specified controller of the platform. The controller is determined by the input parameter ID, which reduces the overall time for initializing the MMC driver.

NOTICE

After the MMC driver is initialized successfully, the **proc/mci/mci_info** node is generated if the system supports the **proc** file system. You can view the device information by running **cat proc/mci/mci_info**. The two initialization APIs cannot be used together. Besides, either a single thread or mutual exclusive time is required to call the **MMC_HostInitById** API.

APIs for Adding eMMC Partitions

API	API Parameter	Description
add_mmc_partition	struct disk_divide_info *info	Input parameter, which is a device information structure, indicating the eMMC partition information. Currently, only the global variable <i>struct disk_divide_info</i> eMMC is supported.
	size_t sectorStart	Input parameter, which indicates the first sector.
	size_t sectorCount	Input parameter, which indicates the number of sectors in a partition.

The device information structure is defined as follows:

```
#define MAX_DIVIDE_PART_PER_DISK 16    // Maximum number of logical partitions supported by each device
#define MAX_PRIMARY_PART_PER_DISK 4    // Maximum number of primary partitions supported by each device

struct disk_divide_info{
    UINT64  sector_count;           // Number of partition blocks
    UINT32  part_count;             // Partition ID
    UINT32  sector_size;           // Partition size
    struct partition_info part[MAX_DIVIDE_PART_PER_DISK + MAX_PRIMARY_PART_PER_DISK]; // Partition information structure
};
```

Raw Read/Write APIs

- Raw read/write APIs of the TF card

API	API Parameter	Description
mmc_direct_write mmc_direct_read	uint32_t host_idx	Input parameter, which indicates the ID of the controller where the TF card is located.
	char * buffer	Input parameter, which indicates the buffer address for storing read and write data (aligned by CACHE_ALIGNED_SIZE).
	uint32_t start_sector	Input parameter, which indicates the start block address for read and write.
	uint32_t nsectors	Input parameter, which indicates the number of read and write blocks.

- eMMC raw read/write APIs

API	API Parameter	Description
emmc_raw_write emmc_raw_read	char * buffer	Input parameter, which indicates the buffer address for storing read and write data (aligned by CACHE_ALIGNED_SIZE).
	uint32_t start_sector	Input parameter, which indicates the start block to be read or written.
	uint32_t nsectors	Input parameter, which indicates the number of read and write blocks.

- Read/Write APIs of MMC exception (interrupt) context

API	API Parameter	Description
mmc_write_in_exception mmc_read_in_exception	uint32_t host_idx	Input parameter, which indicates the ID of the controller where the MMC device is located. The TF card and eMMC are supported.
	char * buffer	Input parameter, which indicates the buffer address for storing read and write data (aligned by CACHE_ALIGNED_SIZE).
	uint32_t start_sector	Input parameter, which indicates the start block address for read and write.
	uint32_t nsectors	Input parameter, which indicates the number of read and write blocks.

NOTICE

The read/write APIs of MMC exception (interrupt) context support the TF card and eMMC but does not support the SDIO device. You are not advised to call this API in non-exception (interrupt) scenarios. Before calling this API, ensure that the system is not in the MMC logic execution process. Otherwise, the read/write operation may fail.

SDIO APIs

Huawei LiteOS provides a complete set of SDIO external APIs, including those for sending commands and receiving and transmitting data. The SDIO extended peripherals implement the operations on the extended peripherals by calling these APIs.

API	Description
sdio_get_func	Obtains sdio_func , which is generated after the SDIO device is identified. The sdio_func is uniquely identified by the func ID, vendor ID, and device ID (obtained based on the SDIO peripheral).
sdio_en_func	Enables sdio_func , which is implemented by writing the 0x2 address through the CMD52 command.
sdio_dis_func	Disables the specified sdio_func . This API is implemented by writing the 0x2 address using the CMD52 command.
sdio_require_irq	Registers the SDIO interrupt handling function for the specified sdio_func . The interrupt handling function needs to be implemented by the user. The function type is defined as void (sdio_irq_handler_t)(struct sdio_func *).
sdio_release_irq	Releases the SDIO interrupt processing function registered in the specified sdio_func .
sdio_read_byte	Reads a byte of data from the specified address addr of the specified sdio_func and returns the read data.
sdio_read_byte_ext	Reads one byte of data from the specified address addr of the specified sdio_func , adds the input parameter <i>in</i> to the CMD parameter (not supported currently), and returns the read data.
sdio_read_incr_block	Reads data of a specified length from the specified address addr of the specified sdio_func to the specified address dst of the memory. The device addresses (for example, the device memory address) increase in sequence during reading. This API is implemented by running the CMD53 command.
sdio_read_fifo_block	Reads data of a specified length from the specified address addr of the specified sdio_func to the specified address dst of the memory. The device address (for example, the device FIFO) is fixed during reading. This API is implemented by running the CMD53 command.
sdio_write_incr_block	Writes data of the size length in the specified address src to the specified address addr of the specified sdio_func . The device addresses (for example, the device memory address) increase in sequence during writing. This API is implemented by running the CMD53 command.

API	Description
sdio_write_fifo_block	Writes data of the size length in the specified address src to the specified address addr of the specified sdio_func . The device address (for example, FIFO of the device) is fixed during writing. This API is implemented by running the CMD53 command.
sdio_write_byte	Writes one-byte data to the specified address addr of the specified sdio_func . This API is implemented by running the CMD52 command.
sdio_write_byte_raw	Writes one-byte data to the specified address addr of the specified sdio_func , and then reads the data back (by setting the RAW flag). This API is implemented by running the CMD52 command.
sdio_func0_read_byte	Reads a byte of data from a specified address of a device and returns the read data. This API is implemented by running the CMD52 command to read func0 .
sdio_func0_write_byte	Writes a byte of data to the specified address addr of the device. This API is implemented by running the CMD52 command to write to func0 .
sdio_set_cur_blk_size	Configures size of the current SDIO block. The size should not be larger than 512.
sdio_rescan	Identifies the SDIO device again. Actually, the operation is to uninstall and reinstall the device.
sdio_reset_comm	Resets the SDIO device.

Based on the controller capability, Huawei LiteOS provides data transmission in discrete mode and a set of dedicated read/write APIs. The SDIO extended peripherals implement discrete transfer by calling this API. That is, the data in multiple buffers can be transmitted or read to multiple buffers at a time to reduce data copies in the multi-block data transfer scenario and improve the data transfer performance.

API	Description
<code>sdio_readv_incr_block</code>	Reads data from the specified address addr of the specified sdio_func. The length of the read data and the storage address are specified by the input parameter struct mmc_sg *sg. The number of data records in sg is determined by the input parameter sg_nums. The device addresses (for example, the device memory address) increase in sequence during reading. This API is implemented by running the CMD53 command.
<code>sdio_readv_fifo_block</code>	Reads data from the specified address addr of the specified sdio_func. The length of the read data and the storage address are specified by the input parameter struct mmc_sg *sg. The number of data records in sg is determined by the input parameter sg_nums. The device address (for example, FIFO of the device) is fixed during reading. This API is implemented by running the CMD53 command.
<code>sdio_writev_incr_block</code>	Writes the data in the input parameter struct mmc_sg *sg to the specified address addr of the specified sdio_func. The data address and length are provided in sg. The number of data in sg is determined by the input parameter sg_nums. The device addresses (for example, the memory address of the device) increase in sequence during writing. This API is implemented by running the CMD53 command.
<code>sdio_writev_fifo_block</code>	Writes the data in the input parameter struct mmc_sg *sg to the specified address addr of the specified sdio_func. The data address and length are provided in sg. The number of data records in sg is determined by the input parameter sg_nums. The device address (for example, FIFO of the device) is fixed during writing. This API is implemented by running the CMD53 command.

The **mmc_sg** structure is as follows:

```
struct mmc_sg {
    size_t length; // Data length, which cannot be greater than the size of the buffer memory space.
    void *data;    // Buffer address, which must be cache line-aligned. The size of the buffer memory space
                  // to which the buffer address points must also be cache line-aligned.
};
```

eMMC Development Process

1. Add eMMC partitions.

```
size_t part0_start_sector = 16 * (0x100000/512);
size_t part0_count_sector = 1024 * (0x100000/512);
extern struct disk_divide_info emmc;
add_mmc_partition(&emmc, part0_start_sector, part0_count_sector);
```

2. Initialize the driver.

```
MMC_HostInitById(emmc_host_id);
```

NOTE

emmc_host_id is the ID of the controller corresponding to the eMMC memory.

After the eMMC medium is identified, the **/dev/mmcblkxPy** node is generated for mounting the file system. (**x** indicates the device ID, which is generally 0 or 1. **y** indicates the partition ID, which is determined by the number of device partitions and starts from 0.)

3. After the device is identified, the device is read and written.

a. Raw read/write.

Call the eMMC raw read/write API to perform the read/write operation.

b. Mount as the FAT file system for read and write.

```
mount("/dev/mmcblk0p0", "/sd", "vfat", 0, NULL);
```

After the device is mounted, you can read and write the device through the standard API of the file system.

NOTE

/dev/mmcblk0p0 is the device name registered after the eMMC memory is identified. Transfer the device name based on the actual situation.

TF Card Development Process

1. Initialize the driver.

```
MMC_HostInitById(sd_host_id);
```

NOTE

sd_host_id indicates the ID of the controller corresponding to the TF card. After inserting and identifying the SD card, **/dev/mmcblkxPy** node is generated for mounting the file system. **x** is the device ID, which is usually 0 or 1. **y** indicates the partition ID, which is determined by the number of device partitions and starts from 0.

2. After the device is identified, the device is read and written.

a. Perform raw read/write.

Call the raw read/write API of the TF card to perform the read/write operation.

b. Mount as the FAT file system for read and write.

```
mount("/dev/mmcblk0p0", "/sd", "vfat", 0, NULL);
```

After the device is mounted, you can read and write the medium through the standard API of the file system.

NOTE

/dev/mmcblk0p0 indicates the name of the device registered after the TF card is identified. Set this parameter based on the actual requirements.

Development Process of SDIO Extended Peripherals

1. Perform pin multiplexing.

Configure the I/O registers, and configure the registers of the CLK, CMD, DATA0, DATA1, DATA2 and DATA3 pin registers corresponding to the SDIO peripherals.

2. Initialize the driver.

```
MMC_HostInitById(sdio_host_id);
```

 NOTE

sdio_host_id is the ID of the controller corresponding to the SDIO extended peripheral. After the SDIO Wi-Fi device is successfully identified, the **/dev/sdio0** node is generated.

3. Obtain and enable the **func** after device identification.

```
struct sdio_func* func;
func = sdio_get_func(func_num, manf_id, device_id);
sdio_en_func(func);
```

4. Set the current block size.

```
sdio_set_cur_blk_size(func, blk_size);
```

5. Register the interrupt processing function.

```
void handler_sample(struct sdio_func *func)
{
    dprintf("%s,%d func:0x%x\n",__FUNCTION__,__LINE__,func);
    return;
}
sdio_require_irq(func, handler_sample);
```

Then, you can perform the read and write operations through the external read and write APIs of the SDIO card.

7.3.3 Precautions

- Observe the following principles:
 - Ensure that the metal sheet of the card is in good contact with the card slot (otherwise, a detection error or a data read/write error may occur). When testing a thin MMC, you can press the communication end of the card slot if necessary.
 - If the card is removed abnormally during the read/write operations, the driver directly uninstalls the device node. In this scenario, you must release the resources (such as opened files and directories) that are being operated and unmount the mount point. Otherwise, resource leakage may occur or the mount point cannot be mounted again.
 - After the SDIO peripheral service registers an interrupt by calling **sdio_require_irq**, ensure that the **sdio_release_irq** API is called to release the interrupt and related resources before the device is uninstalled. Otherwise, the system may be abnormal.
 - Partitions must be added before the eMMC driver is initialized. Otherwise, partitions cannot be obtained during device identification. After the device is identified, partitions cannot be dynamically added or deleted.
 - The **dev** device must be formatted before the eMMC is mounted as the FAT file system. Otherwise, the mounting may fail.
 - The pin multiplexing must be configured before the driver of the SDIO extended peripheral is initialized. Otherwise, the device cannot be identified.
- Forbidden operations:

- Do not remove an SD card during read/write operations. Otherwise, exception information is displayed, and files in the SD card or the file system may be damaged.
- Do not remove the SD card during formatting. Otherwise, the SD card may be permanently damaged.
- Do not remove the SDIO extended peripheral when the board is powered on. Otherwise, the SDIO extended peripheral or the board may be damaged.
- The medium varies according to the platform controller. For details about the controller ID and supported medium type, see the "eMMC/SD/SDIO controller" in the "User Guide" of the corresponding chip platform.

7.4 USB Driver Development

7.4.1 Overview

USB is short for Universal Serial Bus. It is an external bus standard used to regulate the connection and communication between a computer and external devices. The Huawei LiteOS USB driver consists of the protocol layer, composite layer, and controller driver.

1. Protocol layer

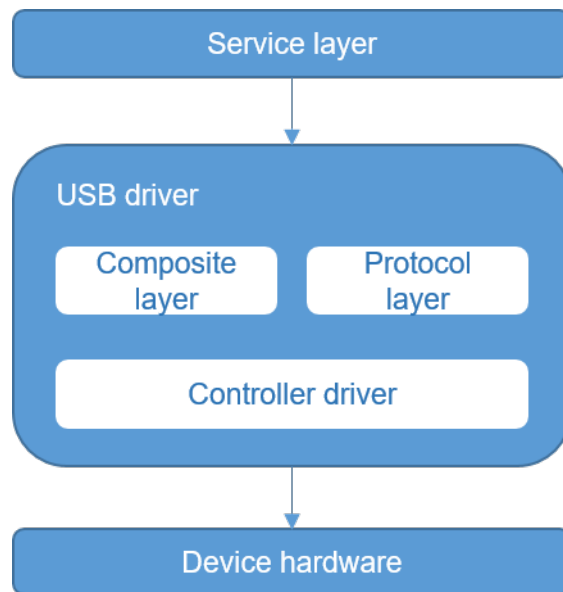
The protocol layer is the implementation layer of the device and supports transmitting and receiving data through the USB port for upper-layer services.

2. Composite layer

The composite (composite device) layer connects the protocol layer and the controller driver, and is responsible for the loading and unloading of specific protocols and the process of device enumeration. The composite device is a virtual device. Other USB devices must be mounted to the composite device. The composite layer encapsulates the process of interacting with the control port. The USB device only needs to register the corresponding processing function with the composite layer. The composite layer loads the specific protocol and enumerates the devices.

3. Controller driver

The controller driver interacts with the hardware to transmit and receive data and control ports.

Figure 7-3 Basic framework of the USB driver

Operation Mechanism

The process of loading the USB protocol driver and transmitting and receiving data is as follows:

1. USB driver loading

The loading of the USB device driver is encapsulated at the Composite layer. The Composite layer invokes the loading function of a specific protocol to load the driver. After the device is connected to the host through the USB cable, the composite layer uniformly processes the enumeration process between the device and the host, and allocates ports and initializes the private data of the protocol by using the processing function registered at the protocol layer.

2. USB data transmission and receiving

After the USB device is loaded successfully, the service layer transmits and receives data through the protocol layer, and the protocol layer directly calls the port control API provided by the controller driver to transmit and receive port data.

7.4.2 Development Guidelines

7.4.2.1 Functions

Functions

The USB function of the Hi35xx supports the host and device modes. For details about the protocols and functions, see [Table 7-5](#) and [Table 7-6](#).

Table 7-5 Functions supported in host mode

Function	Description	Instruction
USB flash drive in host mode	Connects to a USB flash drive to perform read and write operations on the USB flash drive.	• Supports CD-ROM USB flash drive. • Supports multiple USB flash drives. • Supports hot swapping of the card reader and SD card. • 2 TB and larger hard disk • Supports the hub and a maximum of five USB devices by default.
USB-to-Ethernet port	Connects to the USB-to-Ethernet port module to transmit and receive network data through the USB port.	• Supported USB to network port models: Ax88178a, Ax88178, Ax88772, Ax88772A, Ax88772B, and Ax88772C • Does not support the setting of the MAC address of the network adapter. • Does not support the setting of layer 2 sockets.
Host-RNDIS	Network data can be transmitted and received through the USB cable.	• Does not support the setting of the MAC address of the network adapter. • Does not support the setting of layer 2 sockets.
4G-Modem	Connects to a USB-4G module.	• The following 4G module models are supported: ME909s-120, ME909s-821 4G, and Quectel EC20/EC200T • Supports AT commands. • Supports the ECM NIC mode only. • Does not support the setting of the MAC address of the network adapter. • Does not support the setting of layer 2 sockets. • During compilation, you need to select the following at the same time: USB to serial port and 4G_Modem.
USB-WIFI	Connects to a USB-Wi-Fi module.	Supports the following models: Rtl8188eus and mt7601u

Function	Description	Instruction
HOST-UVC	Connects the UVC device.	• Supports the BULK and ISOC transmission. • Supports the YUYV and MJPEG streams. • Does not support connecting to two or more UVC devices at the same time. • Does not support UVC extended commands.
HOST-HID	Connects to HID game controllers.	• Supports the Xbox 360 game controller. • Connects to multiple game controllers at the same time.

Table 7-6 Functions supported by the device mode

Function	Description	Instruction
USB flash drive in device mode	Supports the use of the SD card/MMC medium mounted to the system as a USB flash drive for the USB host to perform read and write operations.	• Supports the virtualization of the SD card into a USB flash drive. • Supports the virtualization of the eMMC into a USB flash drive. • Supports mounting identification of the specified partition of the eMMC medium. • Supports formatting and removal of the USB flash drive by right-clicking the drive letter on the PC.
USB Video Class (UVC)	Video data can be transmitted through the USB port.	• Supports the UVC extended commands of Android STBs. • Supports the following video specifications: Format: H.265/H.264/MJPEG/YUYV/NV12/NV21 Resolution@frame rate: 1920x1080@30fps, 1280x720@30fps, 640x480@30fps, 640x360@30fps, 352x288@30fps, 320x240@30fps, 176 x 144@30 fps

Function	Description	Instruction
USB Audio Class (UAC)	Audio data can be transmitted through the USB port.	• Does not support USB 3.0. • Supports the following PCM specifications: 48K, 32K, 16K, and 8K • Supports microphone and speaker functions. • Supports UAC1.0 and UAC2.0.
UVC&UAC(UAV)	Audio data can be transmitted through the USB port.	• Does not support USB 3.0. • For details about the supported specifications, see the preceding UVC and UAC specifications. • Do not support the UAC1.0 speaker function and UAC2.0.
DFU(Device Firmware Upgrade)	Supports firmware upgrade through the USB port and interconnection with the DFU protocol on the host.	• Does not support USB 3.0. • Supports PC upgrade. • Supports Android STB upgrade. • Supports of the querying of the upgrade status.
HID(Human Interface Device)	Receives and transmits data of the HID device through the USB port.	• Does not support USB 3.0. • Does not support enabling interfaces and protocols. • Supports HID devices such as USB mouse, keyboard, and game joystick. • Supports dynamic configuration of report descriptors.
Virtual port	Supports the transmission and receiving of network data through the USB port to simulate network ports.	Does not support the setting of layer 2 sockets. Supports the RNDIS protocol.
Virtual serial port	Supports serial communication through the USB port to simulate serial ports.	Commissioning function

Function	Description	Instruction
Composite device (virtual network port and virtual serial port)	Supports simultaneous simulation of the network port and serial port based on the USB port.	- Commissioning function - Does not support the setting of layer 2 sockets.

NOTE

- USB 1.1 supports OHCI, USB 2.0 supports EHCI in host mode, and USB 3.0 supports XHCI in host mode, which are used to identify full-speed USB devices, high-speed USB devices, and ultra-high-speed USB devices, respectively.
- When a USB 3.0 device is connected to the USB 2.0 host, the rate cannot be reduced due to hardware limitations of the device. As a result, the device cannot be identified.

The specifications supported by the Huawei LiteOS USB driver vary with the limited chip hardware resources. For details, see [Table 7-7](#) and [Table 7-8](#).

Table 7-7 Specifications for the host mode

Chipset	USB Flash Drive	4G mode m	USB-to-Ethernet Port	Host-RNDIS	USB-WIFI	HOST-UVC	HOST-HID
Hi3516DV300 (USB2.0)	√	√	√	√	√	√	√

Table 7-8 Specifications for the device mode

Chipset	USB Flash Drive	UVC	UAC	UAV	DFU	HID	Virtual Serial Port	Virtual Port	Composite Device
Hi3516DV300 (USB2.0)	√	√	√	√	√	√	√	√	√

NOTE

The USB 2.0 port of the Hi3516DV300/Hi3556V200/Hi3559V200 is operated based on the USB 3.0 controller. Therefore, you need to select the USB 3.0 controller during compilation.

7.4.2.2 Configuring the Memory of the USB Module

The normal running of the USB module requires an uncached memory segment. Therefore, you need to configure a separate memory for the USB module.

You can define the following functions to implement this function:

```
void board_config(void)
{
    g_sys_mem_addr_end = 0x88000000;
    g_usb_mem_addr_start = g_sys_mem_addr_end;
    g_usb_mem_size = 0x100000;
}
```

NOTICE

- Do not modify the statement of **void board_config(void)**. If the **void board_config(void)** function is not defined at the application layer, Huawei LiteOS allocates the OS visible memory size based on the default value. You are advised to set this parameter based on the actual requirements.
- **g_sys_mem_addr_end** specifies the end position of the OS visible memory.
- **g_usb_mem_addr_start** specifies the start position of the USB memory.
- **g_usb_mem_size** specifies the size of the USB memory.
- If the system does not support USB-related functions, set **g_usb_mem_addr_start** and **g_usb_mem_size** to 0.

7.4.2.3 Dynamically Switching the USB Protocol

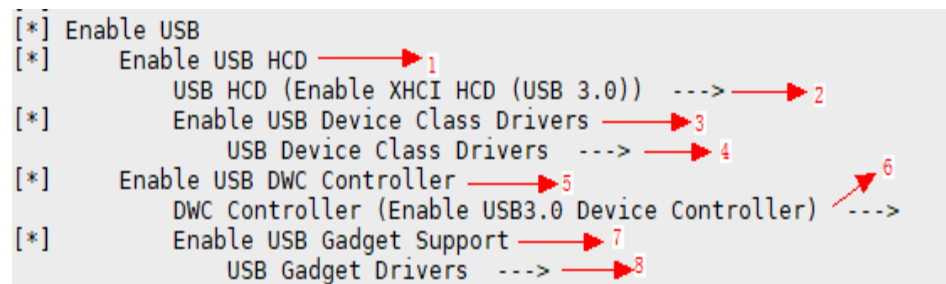
The USB driver supports dynamic switching between the device and host protocols. During dynamic switching, call **usb_deinit** to uninstall the current protocol, and then use **usb_init** to reload the new protocol.

- Supports switching between the host mode and device mode.
- Some protocols (such as DFU, UVC, camera, and HID) cannot be uninstalled when they are used. You need to stop the current service and then call **usb_deinit** to uninstall the protocols.

7.4.2.4 Compiling the Module

The source code path of the USB driver is **drivers/usb**. Run **make** in the root directory of Huawei LiteOS to compile the corresponding USB module library. After the compilation is successful, the **libusb_base.a** and **libusb_device.a** library files are generated in **out/<platform name>/lib**.

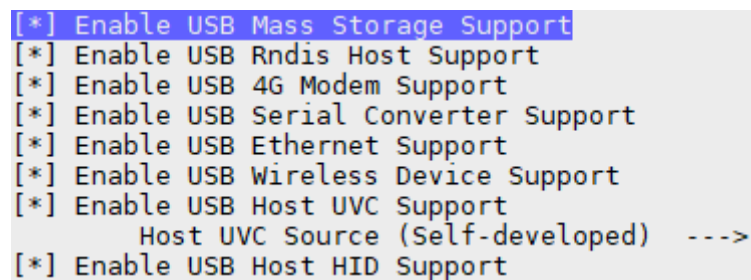
The following figure shows the **menuconfig** configuration item of the USB driver on Huawei LiteOS. In the **Driver** configuration item, you can enable or disable the USB driver function by selecting or deselecting **Enable USB**.

Figure 7-4 menuconfig configuration of the USB driver

Host

Figure 7-4 shows the configuration items 1 to 4:

- 1 specifies whether to enable the USB drive host function.
- 2 indicates the host controller type, for example, USB 2.0 or USB 3.0 (depending on the USB port type of the specific board).
- 3 specifies whether to enable the USB protocol in host mode.
- 4 indicates the USB protocol function in host mode. The supported USB protocols are as follows, and the configuration items of menuconfig are shown in **Figure 7-5**:
 - USB flash drive
 - Host RNDIS
 - 4G Modem
 - USB to serial port
 - USB-to-Ethernet port
 - USB-WIFI
 - HOST-UVC
 - HOST-HID

Figure 7-5 Configuring the USB driver on the host

NOTICE

To initialize the host mode, you can call **usb_init(HOST, 0)**. You can select a protocol or all protocols based on service requirements. The protocol stack automatically loads the corresponding driver based on the type of the inserted device. However, the dependent peripheral components need to be initialized in advance. For details, see the programming example in **7.4.3.1 Host**.

Device

The configuration items are shown in 5 to 8 in [Figure 7-4](#).

- 5 specifies whether to enable the USB drive device function.
- 6 specifies the device controller type, for example, DWC 2.0 or DWC 3.0 (depending on the USB port type of the specific board).
- 7 specifies whether to enable the USB protocol in device mode.
- 8 indicates the USB protocol function in device mode. The supported USB protocols are as follows, and the configuration items of menuconfig are shown in [Figure 7-6](#):
 - USB flash drive in device mode
 - UVC
 - UAC
 - UVC&UAC(UAV)
 - DFU
 - Virtual port
 - Virtual serial port
 - Composite device (virtual serial port and virtual network port)
 - HID

Figure 7-6 Configuring the USB driver on the device

```
[*] Enable USB Mass Storage Gadget
[*] Enable USB Video Class Gadget
[ ] Enable USB Video Class Ioctl Mode
    UVC Version (UVC 1.1) --->
    Transter Mode (Isochronous Mode) --->
    YUV Format (YUYV Format) --->
[*] Enable USB Audio Class Gadget
    UAC Version (UAC1.0) --->
[*] Enable USB Camera Class Gadget
[*] Enable USB Dfu Gadget
[*] Enable USB Serial Gadget
    Serial Mode (Default Serial) --->
[*] Enable USB Ethernet Class Gadget
[*] Enable USB Ethernet & Serial Gadget
[*] Enable USB HID Class Gadget
```

NOTICE

To initialize the device mode, call **usb_init(DEVICE, xxx)**. Enable the required protocols based on service requirements. You can also enable all protocols. The difference between this mode and the host mode is that the protocol to load needs to be specified during initialization. For details about the macro definition of the corresponding protocol, see [usb_init](#). In addition, the dependent peripheral components must be loaded in advance. For details, see the programming examples in [Device](#).

7.4.3 Programming Examples

7.4.3.1 Host

7.4.3.1.1 USB Flash Drive in Host Mode

Example Description

Step 1 Call the following initialization API to register the USB driver:

```
usb_init(HOST, 0); // Initialize the USB host module. You do not need to specify dtype.
```

Step 2 Insert the USB flash drive. After the USB host enumerates the USB flash drive, the **/dev/sda** node is generated in the **/dev** directory.

Step 3 Mount the USB device node to the correct file system to read and write the USB flash drive.

----End

Example Code

```
#define WRITE_BUFFER_SIZE 40
#define READ_BUFFER_SIZE 40

INT32 Udisk_fs_ctrl(void)
{
    int fd = 0;
    int ret = 0;
    char w_buf[WRITE_BUFFER_SIZE] = "Hello,this is a test! Monday,Tuesday";
    char r_buf[READ_BUFFER_SIZE];
    char * charp = NULL;
    unsigned int file_length = 0;
    int i = 0;

    memset(r_buf, 0, READ_BUFFER_SIZE);

    dprintf("1.open file:\n");
    fd = open("/sd0/testfile", O_RDWR|O_CREAT, S_IRUSR|S_IWUSR);
    if (fd == -1) {
        dprintf("open/create Testfile fail ! \n");
        return -1;
    }
    dprintf("open file success ! fd = %d \n\n", fd);

    dprintf("2.write file:");
    ret = write(fd, w_buf, WRITE_BUFFER_SIZE);
    if (ret == -1) {
        dprintf("file write fail! \n");
        close(fd);
        return -1;
    }
    dprintf("write file success!\n\n");

    ret = lseek(fd, 0, SEEK_SET);
    if (ret == -1) {
        dprintf("lseek fail!\n");
        close(fd);
        return -1;
    }
    dprintf("lseek file success!\n\n");

    dprintf("4.read file:");
```

```

    if (file_length > READ_BUFFER_SIZE) {
        file_length = READ_BUFFER_SIZE;
    }
    ret = read(fd, r_buf, file_length);
    if (ret == -1) {
        dprintf("file read fail! \n");
        close(fd);
        return -1;
    }
    charp = r_buf;
    for ( i = 0; i < ret; i++) {
        dprintf("%c",*charp);
        charp++;
    }
    dprintf("\n");
    charp = NULL;
    close(fd);
    return 0;
}
void app_init()
{
    g_usb_mem_addr_start= g_sys_mem_addr_end;
    g_usb_mem_size= 0x40000;    //recommend 256K nonCache for USB
    usb_init(HOST,0);          // Initialize the USB host module. You do not need to specify dtype.

    /* Wait for the USB flash drive to be inserted. The driver registers the /dev/sdap0 node. */
    mount("/dev/sdap0", "/sd0", "vfat", 0, 0);
    Udisk_fs_ctrl();
}

```

7.4.3.1.2 Host USB-to-Ethernet Port and RNDIS

Call the following APIs in the initialization function to register the TCP/IP protocol stack and USB driver:

```

tcpip_init(NULL, NULL);    // Initialize the TCP/IP protocol stack.
usb_init(HOST,0);          // Initialize the USB host module. You do not need to specify dtype.

```

The USB module performs the **netif_set_up** operation on the virtual network port. For details about how to operate the Ethernet port, see [Shell](#)

7.4.3.1.3 Host 4G-Modem

Call the following APIs in the initialization function to register the TCP/IP protocol stack and USB driver:

```

tcpip_init(NULL, NULL);    // Initialize the TCP/IP protocol stack.
usb_init(HOST,0);          // Initialize the USB host module. You do not need to specify dtype.

```

The USB module registers the **/dev/ttyUSB0** node, opens the device node through the **u3g_write_from_shell** API, and writes the corresponding AT command to the node to communicate with the 4G module.

7.4.3.1.4 Host USB-WIFI

Call the following APIs in the initialization function to register the TCP/IP protocol stack and USB driver:

```

tcpip_init(NULL, NULL);    // Initialize the TCP/IP protocol stack.
usb_init(HOST,0);          // Initialize the USB host module. You do not need to specify dtype.

```

 NOTE

- If the Wi-Fi driver is developed by using the API provided by **linux_usb.c**, the **usb_linux_register** API must be called to register the hook function with the USB driver after the **usb_init** API is called. Otherwise, the Wi-Fi device cannot be identified.
- Before operating the USB Wi-Fi device, call **usb_wait_device_ready** to check whether the current Wi-Fi device is successfully identified. If the Wi-Fi device is successfully identified, perform other Wi-Fi operations.

7.4.3.1.5 Host UVC

Call the following initialization API to register the USB driver:

```
usb_init(HOST,0); // Initialize the USB host module. You do not need to specify dtype.
```

 NOTE

- After a UVC device is connected to a USB port and the USB host enumerates the UVC device, the **/dev/video** standby node is generated in the **/dev** directory. The open, close, poll, and ioctl operations are supported. Table 1 lists the command parameters supported by ioctl.
- ISOC transmission has high requirements on the real-time performance, that is, the CPU performance; therefore CPU processing cannot be interrupted by other services for a long time. Otherwise, the USB controller restarts.

Table 7-9 Command parameters supported by ioctl

Command Parameter	Description
VIDIOC_QUERYCAP	Queries UVC device attributes (including the driver name, device name, and driver version of the current device).
VIDIOC_ENUM_FMT	Obtains the stream format supported by the UVC device.
VIDIOC_ENUM_FRAME SIZES	Obtains the resolution supported by the UVC device.
VIDIOC_ENUM_FRAME INTERVALS	Obtains the frame interval supported by the UVC device.
VIDIOC_S_FMT	Sets the stream format and resolution for the UVC device.
VIDIOC_G_FMT	Obtains the current stream format and resolution of the UVC device.
VIDIOC_TRY_FMT	Queries whether the UVC device supports a specified stream format and resolution.
VIDIOC_S_PARM	Sets the frame interval for the UVC device.
VIDIOC_G_PARM	Obtains the current frame rate of the UVC device.
VIDIOC_REQBUFS	Allocates a buffer based on the number of buffer frame and data types.

Command Parameter	Description
VIDIOC_QUERYBUF	Queries the buffer frame v4l2_buffer information based on the index number of buffer frame input by the application layer.
VIDIOC_QBUF	Adds the processed buffer at the application layer to the idle queue that can capture videos.
VIDIOC_DQBUF	The application layer obtains the buffer frames that have stored video data from the driver.
VIDIOC_STREAMON	Enables the data stream transmission.
VIDIOC_STREAMOFF	Disables the data stream transmission.

7.4.3.1.6 Host HID

Call the following initialization API to register the USB driver:

```
usb_init(HOST,0); // Initialize the USB host module. You do not need to specify dtype.
```

NOTE

- Insert the USB game controller. After the USB host enumerates the game handle, the **/dev/uhidX-Y** device node is generated in the **/dev** directory. **X** indicates the serial number of the handle, and **Y** indicates the number of the USB physical port connected to the handle. Then perform the open, close, read, and write operations.
- The game controller continuously sends data to the host. Therefore, data needs to be read in a timely manner. Otherwise, packet loss occurs when the DMA buffer is full.

7.4.3.2 Device

7.4.3.2.1 Virtual USB Flash Drive in Device Mode

Initialization

Call the following initialization APIs to initialize the SD card driver and USB driver:

```
SD_MMC_Host_init();  
usbd_set_device_info(DEV_MASS, &str_manufacturer, &str_product, &str_serial_number, dev_id);  
usb_init(DEVICE, DEV_MASS); // Set dtype to DEV_MASS.
```

 NOTE

- When two or more SD/MMC cards are mounted to the system, the USB 2.0 device is bound to the first SD/MMC card device node by default. For example, if **/dev/mmcblk0p0** and **/dev/mmcblk1p0** are available, the USB 2.0 device is bound to **/dev/mmcblk0p0**. Therefore, the host can access only one SD/MMC card over the USB port.
- After the USB 2.0 device is connected to the host, the host can access the files of the system storage medium over the USB port. When the host is accessing the storage medium, do not read, write, or format the SD/MMC card. Otherwise, exceptions may occur.
- A USB flash drive in device mode can be connected to a Windows PC for quick formatting only. Do not use common formatting operations on a Windows PC or advanced formatting commands on a Linux PC. Otherwise, the partition information may be modified, and the device may fail to be mounted after the virtual USB flash drive is uninstalled. If such operation is performed, you can format the storage medium again after the mounting fails.

Registering the Callback Function of the Device USB Flash Drive

The application layer registers the notify callback function by using the **fmass_register_notify(void(*notify)(void* context, int status), void* context)** function. The driver invokes this callback function when identifying the access and removal of the host, thereby implementing the removal and insertion of the USB flash drive in device mode.

After the USB module is initialized, call **fmass_register_notify()** to register the device callback function.

NOTICE

In the Notify callback function, when the USB device is connected, the SD card partition node identified by the PC needs to be started by using **fmass_partition_startup()**, and ensure that the standby node transferred by the API exists. If the standby node does not exist, by default, **/dev/mmcblk0p0** (the device node must exist) is used as the virtual USB flash drive. Otherwise, the operation fails.

Precautions

- Observe the following principles:
 - The USB flash drive in device mode hands over the SD card to the USB host for processing. Therefore, after the USB flash drive in device mode is identified by the host, a callback function is used to notify the app software module. The app must stop all the services that involve the SD card, such as recording and snapshots, and umount the SD card partition. After the USB host is offline, mount and restore related services.
 - Do not remove the SD/MMC card of the board when the USB host is reading or writing data. Otherwise, some exception information may be printed, and the files or the file system may be damaged.
- Perform the following operations to remove an exception:

After the USB flash drive in device mode is removed and quickly inserted again, the host may fail to detect the USB flash drive. This is because the

registration/deregistration process of the USB flash drive in device mode takes a period of time.

Example Code

```
extern int fmass_register_notify(void(*notify)(void* context, int status), void* context);
extern int fmass_partition_startup(char* path);
void fmass_app_notify(void* conext, int status)
{
    char *path = "/dev/mmcbk0p0";
    if(status == 1)/*usb device connect*/
    {
        /* Stop the read and write operations on the /dev/mmcbk0p0 partition, such as recording and
        snapshot, and unmount the partition. */
        fmass_partition_startup(path); // startup fmass access partition
    }
    else
    {
        /* Mount the partition and notify other modules that the partition can be read and written. */
    }
}
void app_init()
{
    uint32_t ret;
    int handle;

    const char manufacturer[38] = {
        'H', 0, 'i', 0, 'S', 0, 'i', 0, 'l', 0, 'i', 0, 'c', 0, 'o', 0, 'n', 0,
        '(', 0, 'S', 0, 'h', 0, 'a', 0, 'n', 0, 'g', 0, 'h', 0, 'a', 0, 'i', 0, ')', 0
    };
    struct device_string str_manufacturer = {
        .str = manufacturer,
        .len = 38
    };

    const char product[18] = {
        'U', 0, 'S', 0, 'B', 0, 'D', 0, 'E', 0, 'V', 0, 'I', 0,
        'C', 0, 'E', 0
    };
    struct device_string str_product = {
        .str = product,
        .len = 18
    };

    const char serial[16] = {
        '2', 0, '0', 0, '2', 0, '0', 0, '0', 0, '6', 0, '2', 0,
        '4', 0
    };
    struct device_string str_serial_number = {
        .str = serial,
        .len = 16
    };

    struct device_id dev_id = {
        .vendor_id = 0x3361,
        .product_id = 0x0108,
        .release_num = 0x0318
    };

    SD_MMC_Host_init();

    g_usb_mem_addr_start= g_sys_mem_addr_end;
    g_usb_mem_size= 0x40000; //recommend 256K nonCache for usb

    ret = usbd_set_device_info(DEV_MASS, &str_manufacturer, &str_product, &str_serial_number, dev_id);
    if (ret != 0) {
        dprintf("set fail!\n");
        return;
    }
}
```

```

    handle = fmass_register_notify(fmass_app_notify,NULL);
    if (handle == -1) {
        return;
    }
    ret = usb_init(DEVICE, DEV_MASS); // usb_init must after SD_MMC_Host_init
    if(ret != 0) {
        fmass_unregister_notify(handle);
        dprintf("usb init failed!\n");
        return;
    }
}

```

FAT32 File System in the Memory Virtualized as a USB Flash Drive

Step 1 Create a file system image in FAT format (Ubuntu is used as an example).

1. Create a disk image in FAT format.

```
mkfs.fat -s 64 -S 512 -f 2 -n "Internal SD" -F 32 -C usbdisk.tmp 2048
```

In the preceding information, **2048** indicates the size of the disk space, in KB.

2. Mount the FAT disk image.

```
mount -o loop -t vfat usbdisk.tmp test/
```

In the preceding command, **test** indicates the mount directory.

3. Store the required files in the **test** directory and synchronize the files to the disk.

```
sync test/
```

4. Unmount the disk mirror.

```
umount test/
```

5. Create a file system image.

```
dd if=usbdisk.tmp of=udisk.img bs=1024 count=2048
```

Create the file system image **udisk.img** by using the first 2 MB content of **usbdisk.tmp**.

Step 2 Download the created file system image to the flash memory.

Store the **udisk.img** file to the specified path **FAT_PART_IMAGE_PATH** ("/jffs0/**udisk.img**").

Step 3 Call the **ram_mass_storage_init** API to register the **RAM_DEV_NAME** (/dev/**uram**) device node.

Step 4 Call the **fmass_register_notify** API to set the device node **/dev/uram** as a virtual USB flash drive.

Step 5 Load the virtual USB flash drive protocol by calling **usbd_set_device_info** and **usb_init**.

NOTICE

The capacity of the file system image to be created is 2 MB. You need to customize the file system image with a specific capacity and change the value of **FAT_PART_SIZE** based on the actual service requirements.

----End

7.4.3.2.2 Device UVC

You can run **make menuconfig** to select UVC 1.0 or UVC 1.1, which are supported by the Device UVC. The UVC 1.1 protocol is used by default. The UVC 1.0 protocol is compatible with the Windows XP operating system.

Operation Guide

1. The upper-layer UVC application sets the UVC transmission capability set before the USB initialization, enables the video input (VI) function, and switches the UVC device on.
2. After the connection is ready, wait for the UVC connection at the bottom layer, start the video processing subsystem (VPSS) and video encoding (VENC), and transmit the stream data until the UVC channel is disconnected.
3. To switch to a new stream format, the application must disable the current encoding task, clear the cache, and then restart encoding.

Operation Example

- Method 1: Directly call the device UVC APIs to control the data transmission.

Step 1 Set the UVC transmission capability set (including the encoding format, resolution, and frame rate) using the **fuvv_frame_descriptors_get** API.

Step 2 Call the **usbd_set_device_info** and **usb_init** APIs to register the USB driver:

```
usbd_set_device_info(DEV_UVC, &str_manufacturer, &str_product, &str_serial_number, dev_id);
usb_init(DEVICE, DEV_UVC); // Set dtype to DEV_UVC.
```

Step 3 After the VI module is enabled, open the UVC device by calling **uvc_open_device**. You need to specify the resolution and frame rate.

Step 4 Wait until the board is successfully connected to the USB host by calling **uvc_wait_host**.

Step 5 After the VPSS and VENC are enabled, start the transmission of video data by calling **uvc_rcv_pack**.

Step 6 Stop the transmission of video data by calling **uvc_video_stop**, and disable the USB device by calling **uvc_close_device**.

----End

- Method 2: Open the device node registered by the **device uvc** and control data transmission through the system call function.

Step 1 Call the **usbd_set_device_info** and **usb_init** APIs to register the USB driver:

```
usbd_set_device_info(DEV_UVC, &str_manufacturer, &str_product, &str_serial_number, dev_id);
usb_init(DEVICE, DEV_UVC); // Set dtype to DEV_UVC.
```

Step 2 After the VI module is enabled, open the UVC device node **/dev/usb_uvc-0** by calling **open** function, and call the **select** function to monitor the device connection.

Step 3 When the board is successfully connected to the USB host and data can be transmitted, the message can be obtained by calling **select** function. In this case, the **ioctl** API is called to send the **VIDIOC_DQEVENT** command and provide the **struct usbd_uvc_video_event** structure for data receiving. Send the information

about the video resolution and frame rate to the VPSS and VENC based on the received data.

Step 4 After the VPSS and VENC are enabled, start the transmission of video data by calling **uvc_recv_pack**. At the same time, call the ioctl API by system to send the **VIDIOC_STREAMON** command that instructs the USB device to start data transmission.

Step 5 Stop video transmission by calling **VIDIOC_STREAMOFF**, and then close the USB device by calling **close** function.

----End

NOTICE

Both method 1 and method 2 can open the device by directly calling the **uvc_open_device** API and calling the **open** function. When the two APIs are called at the same time, the device can be opened only once. The later API cannot be used to open the device and its all operations become invalid.

7.4.3.2.3 Device UAC

The data processing flow of the UAC is similar to that of the UVC. The procedure is as follows:

Step 1 Call the **usbd_set_device_info** and **usb_init** APIs to register the USB driver:

```
usbd_set_device_info(DEV_UAC, str_manufacturer, str_product, str_serial_number, dev_id);  
usb_init(DEVICE, DEV_UAC); // Set dtype to DEV_UAC.
```

Step 2 After the VI module is enabled, wait for the board and USB device by calling **uac_wait_host**.

Step 3 After the VPSS and VENC are enabled, you can transmit audio data by calling **fuac_send_message**.

Step 4 Stop calling the **fuac_send_message** API. That is, stop transmitting audio data.

----End

7.4.3.2.4 Device UAV (Camera)

The UAV supports the UAC and UVC. For details, see the UVC and UAC examples. The difference lies in the protocol type transferred during initialization:

```
usbd_set_device_info(DEV_CAMERA, &str_manufacturer, &str_product, &str_serial_number, dev_id);  
usb_init(DEVICE, DEV_CAMERA); // dtype is set to DEV_CAMERA.
```

7.4.3.2.5 Device Virtual Serial Port

Call the following initialization API to register the USB driver:

```
usbd_set_device_info(DEV_SERIAL, &str_manufacturer, &str_product, &str_serial_number, dev_id);  
usb_init(DEVICE, DEV_SERIAL);
```

If shell communication interaction and debugging information printing need to be performed based on the USB virtual serial port, the modules that depend on the serial port, such as the console and shell, need to be initialized.

```
virtual_serial_init("/dev/ttyGS0");  
system_console_init("/dev/serial");
```

If the board writes data to the USB host, directly call the **write** API. If the board reads the data from the USB host, open the serial port, call the **ioctl** API, and then call the **read** API. The **ioctl** API is used as follows:

```
ioctl(fd, CONSOLE_CMD_RD_BLOCK_SERIAL, 1);
```

NOTICE

- The virtual serial port and the UART port cannot be used at the same time.
- If multiple **open** operations are performed on a node, you need to close the node for multiple times before unmounting the protocol. That is, the number of **open** operations must be the same as that of **close** operations. Otherwise, unpredictable errors may occur.
- The virtual serial port supports a maximum of 4096-byte data read/write.
- After initialization in single-serial-port mode, the **/dev/ttyGS0** node is generated under the **dev** directory. In two-serial-port mode, the **/dev/ttyGS1** node is generated.

If the host is a Windows PC, place the **linux-cdc-acm.inf** driver file in **01.software/pc/usb_tools** on the PC.

Connect the board to the host by using a USB cable. The PC automatically loads the driver. The driver may fail to be installed for the first time. You need to install the driver by yourself. The method is as follows:

- Step 1** Right-click the **Computer** icon on the desktop and select **Management** from the shortcut menu.
- Step 2** Open the **Device Manager** window.
- Step 3** Click **Other** devices. Then **Gadget Serial(COMx)** is displayed. Double-click it.
- Step 4** In the displayed window, click the **Driver** tab. Open the **Driver** tab page, click **Update Driver**, and then click **Browse** my computer for driver software.
- Step 5** Set the path to the directory where the driver file **linux-cdc-acm.inf** is located, and click **Next**. The computer automatically installs the driver. After the driver is successfully installed, close the window.

NOTICE

- 01. The **linux-cdc-acm.inf** file exists in the **software/pc/usb_tools** directory. Generally, Windows 7 cannot automatically load the virtual serial port driver. Therefore, you need to manually load the virtual serial port driver. In Windows 10, the virtual serial port driver is automatically loaded.
- If the serial port driver fails to be installed, modify the **linux-cdc-acm.inf** file by adding **USB\VID_3361&PID_0102&REV_0318** to the **[DeviceList]** and **[DeviceList.NTamd64]** fields. **3361** is the **VID**, and **0102** is the **PID**, **0318** is the device version number. The three numbers are set by calling **usbd_set_device_info**. Change them based on the actual device and ensure that they are the same as the configured ones.

Power on and start the board again. Then the board can be used as a real serial port on the PC.

----End

7.4.3.2.6 Device Virtual Network Port

Step 1 Call the following initialization APIs to register the TCP/IP protocol stack and USB driver:

```
tcpip_init(NULL, NULL); // Initialize the TCP/IP protocol stack.  
usbd_set_device_info(DEV_ETHERNET, &str_manufacturer, &str_product, &str_serial_number, dev_id);  
usb_init(DEVICE, DEV_ETHERNET);
```

Step 2 By default, no IP address is configured for the USB virtual network port. Configure an IP address based on the network plan.

Step 3 If Windows 7 or Windows 10 is used on the host, place the **linux.inf** driver file in **01.software/pc/usb_tools** on the PC. Modify the **linux.inf** driver file. Add **USB\VID_3361&PID_0102&REV_0318** to the **[RndisDevices.NTx86]**, **[RndisDevices.NTamd64]**, and **[RndisDevices.NTia64]** fields. **3361** is the **VID**, **0102** is the **PID**, and **0318** corresponds to the device version number. The three numbers are set by calling **usbd_set_device_info**. Change them based on the actual device and ensure that they are the same as the configured ones.

Step 4 Connect the board to the host PC through a USB cable.

- If the host runs Windows 7, the driver is automatically loaded. In this case, you need to install the driver. The method is as follows:
 - a. Right-click the **Computer** icon on the desktop and select **Management** from the shortcut menu.
 - b. Open the **Device Manager** window.
 - c. Click **Other Devices**, and double-click **Linux USB Ethernet/RNDIS Gadget #3**.
 - d. In the displayed window, click the **Driver** tab. Open the **Driver** tab page, click **Update Driver**, and then click **Browse** my computer for driver software.
 - e. In the displayed window, click **Browse** and select the directory where **linux.inf** is located, and click **Next**. The PC will automatically install the driver. After the driver is successfully installed, close the GUI.
- If the host runs Windows 10, the serial port is identified as a virtual serial port by mistake. In this case, you need to manually update the driver as follows:
 - a. Right-click the **Computer** icon on the desktop and select **Management** from the shortcut menu.
 - b. Open the **Device Manager** window.
 - c. Click the port device. The USB serial device (COMX) is displayed.
 - d. In the displayed window, click the **Driver** tab. Open the **Driver** tab page, click **Update Driver**, and then click **Browse** my computer for driver software.
 - e. Select the path where the **linux.inf** driver file is located, and click **Next**. The PC will automatically install the driver. After the driver is successfully installed, a virtual network port device is generated and the corresponding virtual serial port device disappears. Then close the GUI. If

a message is displayed during the installation indicating that the third-party INF does not contain the digital signature information, you can search for the method of disabling the digital signature function on the Internet.

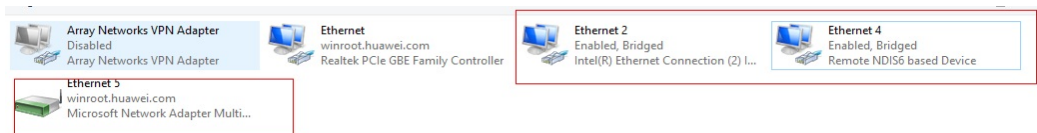
Step 5 On the board, configure an IP address and add a route.

Step 6 When the board is connected to the PC through a USB data cable, a USB network node is generated on the PC. Choose **Start > Control Panel**, click **Network and Sharing Center > Change adapter settings > Linux USB Ethernet/RNDIS Gadget #x**.

Step 7 Establish a network bridge.

1. Right-click **Linux USB Ethernet/RNDIS Gadget #x** and choose **Add to Bridge**.
2. Right-click the local connection node on the PC for connecting to the extranet and choose **Add to Bridge**.
3. Wait until the bridge is set up, as shown in the following figure:

Figure 7-7 Successful building the bridge



Then the board can access the network normally. The device virtual network port can be used as a real network port.

----End

NOTICE

- When a Windows 10 PC is used to bridge a network, if the bridge cannot obtain an IP address, set up a bridge by following the sequence: select a physical network adapter, a virtual network adapter, and then a bridge. The bridge may fail to be configured randomly. In this case, click **OK**, right-click the physical network adapter, and choose **Add to Bridge** from the shortcut menu. This step is important because the physical network adapter is not added to the bridge.
- If the bridge network environment requires access permission, the IP address may be pinged but the gateway and server cannot be pinged. In this case, check the MAC address of the board (the IP address is associated with the MAC address).

7.4.3.2.7 Composite Device (Serial Port and Network Port)

A composite device supports the combination of a single serial port and a network port as well as the combination of two serial ports and a network port. After the configuration item of the composite device is enabled, the combination of a single serial port and a network port is used by default. To combine two serial ports and a network port, select two serial ports in serial port mode.

The initialization processes of the preceding two composite devices are the same. The detailed process is as follows:

```
tcpip_init(NULL, NULL); // Initialize the TCP/IP protocol stack.
usbd_set_device_info(DEV_SER_ETH, &str_manufacturer, &str_product, &str_serial_number, dev_id);
usb_init(DEVICE, DEV_SER_ETH);
virtual_serial_init("/dev/ttyGS0");
system_console_init("/dev/serial");
```

The use of the serial port and network port in a composite device is the same as that of the [7.4.3.2.5 Device Virtual Serial Port](#) and [7.4.3.2.6 Device Virtual Network Port](#). For details, see the corresponding examples.

NOTICE

The virtual serial port and the UART port cannot be used at the same time.

7.4.3.2.8 Device DFU

Step 1 Call the following initialization API to register the USB driver:

```
usbd_set_device_info(DEV_DFU, &str_manufacturer, &str_product, &str_serial_number, dev_id);
usb_init(DEVICE, DEV_DFU);
```

Step 2 Before the upgrade, call **usb_dfu_init_env_entities** to set the upgrade API type and environment variables.

Step 3 Call **usb_dfu_update_status** to wait for the upgrade event.

Step 4 After an upgrade event is generated, call **usb_dfu_update_size_get** to obtain the upgrade file size, perform necessary verification on the upgrade file, and check whether the upgrade file carries the suffix field (such as the checksum and flag).

Step 5 Based on the size of the upgrade file, provide sufficient memory space for the DFU cache upgrade file, and obtain the actual upgrade file by calling **usb_dfu_read**.

Step 6 After obtaining the actual upgrade file, upgrade the firmware and write it to the specified flash address segment.

Step 7 Call the **usb_dfu_free_entities** API to release related resources.

NOTE

The memory space sufficiency for the upgrade file depends on the size of the upgrade image provided by the vendor.

----End

7.4.3.2.9 Device HID

The HID upper-layer application calls the **usbd_set_device_info** API to set the vendor ID, product ID, device serial number, and string information before the USB initialization. Then, it calls the **hid_report_descriptor_info** or **hid_add_report_descriptor** API to configure the report descriptor information so that the USB host can identify the HID device, such as the keyboard, mouse, and keyboard with touchpad.

Operation Example

- Method 1: Open the device node registered by the Device HID and call an API of the VFS module to transmit HID data streams.

Step 1 Call the **usb_set_device_info** and **hid_report_descriptor_info** APIs to fill in the descriptor information.

Step 2 Call the **usb_init** API to register the USB driver:

```
usb_init(DEVICE, DEV_HID); // Set dtype to DEV_HID.
```

Step 3 In the upper-layer application, call the **open** function to enable the specified HID device.

```
fd = open("/dev/hid", O_RDWR, 0);
```

Step 4 In the upper-layer application, write the data to the INPUT parameter buffer of the **write** function by following the input data format specified by the report descriptor. The length is also the INPUT length specified by the report descriptor. If the report descriptor defines the OUTPUT item, call the **read** function to read the data sent by the USB host to the device. The size of the read data is the OUTPUT length specified in the report descriptor. To improve the task execution efficiency, the poll mechanism can be called. The difference is that the **read** function uses the permanent blocking mode, while the poll mechanism uses the scheduled blocking mode.

Step 5 After the data transmission is complete, if the device is not used, call the **close** function to disable the device.

----End

- Method 2: Call the external API of the non-VFS module to transmit HID data streams.

Step 1 Call the **usb_set_device_info** and **hid_add_report_descriptor** APIs to fill in the descriptor information.

Step 2 Call the **usb_init** API to register the USB driver:

```
usb_init(DEVICE, DEV_HID); // Set dtype to DEV_HID.
```

Step 3 In the upper-layer application, call the **fhid_send_data** API to send the HID data based on the data format and length of the INPUT field specified by the report descriptor. If the OUTPUT item is defined by the report descriptor, call the **fhid_rcv_data** API to read the data sent from the USB host to the device. The length of the HID data is the same as the length of the OUTPUT item specified in the report descriptor.

----End

NOTICE

- The **hid_report_descriptor_info** API can be used to set only one report and work with the API of the VFS module. The **hid_add_report_descriptor** API can be used to set a maximum of two reports (by calling the API cyclically) and work with the API of the non-VFS module. The two methods cannot be used together. To reset the report, you need to load and uninstall the Device HID protocol.
- If the report descriptor does not contain the **Report ID** item, the default ID value is **0**. During data transmission, the first byte does not need to be filled with **0**. If more than one report ID is defined, the first byte in the transmission report is filled with the corresponding report ID. The output report and feature report can share the same report ID.
- When the USB host sends data to the device, it is recommended that the data be read in time. Otherwise, new data sent by the USB host will overwrite the previous data.
- If the **open** function is called, the protocol cannot be uninstalled. The protocol can be uninstalled only after the **close** function is called. Currently, the Device HID can be opened only once. If the Device HID is opened again, the operation fails.
- When upper-layer applications call the **read** and **write** APIs of the VFS module or the **fhid_send_data** and **fhid_rcv_data** APIs of the non-VFS module, ensure that the size of the transferred **buffer** is the same as that of the transferred **len**. Otherwise, the transmitted or read data is incorrect.
- Method 1 and method 2 cannot be used at the same time. If method 1 is used, you need to enable the VFS module. The HID protocol cannot be uninstalled when it is in use. The uninstall operation can be performed only after the service operations at the application layer are stopped.
- The APIs for receiving HID data in methods 1 and 2 can be called only in tasks and cannot be called during interrupt callback. The APIs for sending HID data can be called during interrupt callback.

7.4.4 API Reference

The following APIs are provided:

API	Description
usb_set_device_info	Sets the device information, including the VID, PID, version number, vendor, product, and serial number.
usb_init	Initializes the USB module.
usb_deinit	Uninstalls the USB module.
usb_is_devicemode	Checks whether the USB device is connected to the USB host.
fmass_register_notify	Registers the device notify callback function.

API	Description
fmass_unregister_notify	Deregisters the device notify callback function.
fmass_partition_startup	Starts the SD card, MMC, or RAM media partition node that is identified by the PC.
ram_mass_storage_init	Enables memory mapping for the FAT32 file system image and registers the /dev/uram device node.
ram_mass_storage_deinit	Disables memory mapping for the FAT32 file system.
fuvc_frame_descriptors_get	Sets the video transmission capability set of the UVC device.
uvc_open_device	Opens the UVC.
uvc_close_device	Closes the UVC.
uvc_wait_host	Defines the UVC wait for connecting to the USB host.
uvc_get_state	Obtains the running status of the UVC device.
uvc_rcv_pack	Starts the UVC video transmission in copy mode.
uvc_video_stop	Stops the UVC video transmission.
uvc_format_info_get	Obtains the detailed information about the new stream format.
uvc_format_status_check	Defines the stream format switching event.
hi_camera_register_cmd	Defines the UVC extension command for application registration, which is called by the underlying driver.
uac_wait_host	Defines the UAC wait for connecting to the USB host.
fuac_send_message	Sends the audio data.
fuac_receive_message	Receives the audio data.
fuac_opts_set	Sets UAC 1.0 audio parameters, including the number of channels.
fuac_get_opts	Obtains UAC 1.0 audio parameters, including the number of channels.
fuac_rate_get	Obtains the sampling rate of UAC 1.0 audio.
fuac2_get_opts	Obtains the parameters of the Audio 2.0 protocol, including the number of channels.
fuac2_set_opts	Sets the parameters of the Audio 2.0 protocol, including the number of channels.

API	Description
usb_dfu_init_env_entities	Initializes the DFU and configures environment variables.
usb_dfu_update_status	Obtains the upgrade event.
usb_dfu_update_size_get	Obtains the actual size of the updated file.
usb_dfu_get_entity	Obtains the DFU structure variable specified in the list.
usb_dfu_read	Obtains the buffer data of a certain size in the specified DFU list.
hid_report_descriptor_info	Configures the report descriptor information to set the report data format and function.
hid_add_report_descriptor	Configures multiple report table descriptors and sets the report table data format, function, and API protocol.
fhid_send_data	Sends the HID data.
fhid_rcv_data	Receives the HID data.
u3g_write_from_shell	Sends the serial port command to the 4G module.
uvc_get_frame_buf	Obtains the address of the frame buffer (self-developed API).
usb_dfu_free_entities	Releases DFU-related resources.

7.4.4.1 usbd_set_device_info

[Description]

Sets the device information, including the VID, PID, version number, vendor, product, and serial number.

[Syntax]

```
uint32_t usbd_set_device_info(device_type dtype,
                             const struct device_string *str_manufacturer,
                             const struct device_string *str_product,
                             const struct device_string *str_serial_number,
                             struct device_id dev_id);
```

[Parameters]

Parameter	Description	Input/Output
dtype	Device type - DEV_SERIAL: virtual serial port. - DEV_ETHERNET: virtual network port. - DEV_SER_ETH: composite device (serial port and network port). - DEV_DFU: USB firmware upgrade. - DEV_MASS: virtual USB flash drive. - DEV_UVC: USB video transmission. - DEV_UAC: USB audio transmission. - DEV_CAMERA: USB camera. - DEV_HID: USB HID device input.	Input
str_manufacturer	Manufacturer information. The first member variable of the structure is the string pointer, and the second member variable is the total length of the string.	Input
str_product	Product information. The structure members are the same as those in the preceding information.	Input
str_serial_number	Product serial number information. The structure members are the same as those in the preceding information.	Input
dev_id	Device ID. The first member variable is VID (vendor ID), the second member variable is PID (product ID), and the third member variable is release_num (device version).	Input

[Return Values]

Return Value	Description
LOS_OK	Successful
LOS_NOK	Failed

[Requirement]

- Header file: **usb_init.h**
- Library file: **libusb_base.a**

[Note]

- The vendor ID cannot be set randomly, and must be applied to the USB Implementers Forum (USB-IF). The product ID and device version number can

be set based on the actual product. If a different USB device is used on the board, the PID needs to be changed. Otherwise, the device driver identified by the USB host may be faulty.

- The length of the character string must be the same as that of the character string, and the length ranges from 2 to 252. Note that the length of the input parameter must be an even number. Otherwise, the USB host cannot identify the character string.

[Example]

```
const char manufacturer[38] = {
    'H', 0, 'i', 0, 'S', 0, 'i', 0, 'l', 0, 'i', 0, 'c', 0, 'o', 0, 'n', 0,
    '(', 0, 'S', 0, 'h', 0, 'a', 0, 'n', 0, 'g', 0, 'h', 0, 'a', 0, 'i', 0, ')', 0
};
struct device_string str_manufacturer = {
    .str = manufacturer,
    .len = 38
};

const char product[18] = {
    'U', 0, 'S', 0, 'B', 0, 'D', 0, 'E', 0, 'V', 0, 'I', 0,
    'C', 0, 'E', 0
};
struct device_string str_product = {
    .str = product,
    .len = 18
};

const char serial[16] = {
    '2', 0, '0', 0, '2', 0, '0', 0, '0', 0, '6', 0, '2', 0,
    '4', 0
};
struct device_string str_serial_number = {
    .str = serial,
    .len = 16
};
struct device_id dev_id = {
    .vendor_id = 0x3361,
    .product_id = 0x0102,
    .release_num = 0x0318
};

uint32_t ret;
ret = usbd_set_device_info(DEV_UVC, &str_manufacturer, &str_product, &str_serial_number, dev_id);
if (ret != 0) {
    dprintf("set fail!\n");
}
```

CAUTION

This example is for reference only and cannot be used directly. The **VID** must be changed to the customer's vendor information.

[Related Topics]

None.

7.4.4.2 usb_init

[Description]

Initializes the USB module.

[Syntax]

```
uint32_t usb_init(controller_type ctype, device_type dtype);
```

[Parameters]

Parameter	Description	Input/Output
ctype	Enumerated type, specifying whether the USB is of the host type or device type • HOST indicates the host type. • DEVICE indicates the device type.	Input
dtype	Enumerated type. • In host mode, this parameter is invalid and the input value is 0. • In device mode, this parameter indicates the device type. – DEV_SERIAL: virtual serial port – DEV_ETHERNET: virtual network port – DEV_SER_ETH: composite device (serial port and network port) – DEV_DFU: USB firmware upgrade – DEV_MASS: virtual USB flash drive – DEV_UVC: USB video transmission – DEV_UAC: USB audio transmission – DEV_CAMERA: USB camera – DEV_HID: USB HID device input	Input

[Return Values]

Return Value	Description
LOS_OK	Successful
LOS_NOK	Failed

[Requirement]

- Header file: **usb_init.h**
- Library file: **libusb_base.a**

[Note]

- For details about the dependency between the USB protocols and other modules, see Table [Programming Examples](#).

- This API cannot be called repeatedly. To switch the protocol, you need to call **usb_deinit** to uninstall the current protocol.

[Example]

For details, see [Programming Examples](#).

[Related Topics]

[usb_deinit](#)

7.4.4.3 usb_deinit

[Description]

Uninstalls the USB module.

[Syntax]

```
uint32_t usb_deinit(void);
```

[Parameters]

None.

[Return Values]

Return Value	Description
LOS_OK	Successful
LOS_NOK	Failed

[Requirement]

- Header file: **usb_init.h**
- Library file: **libusb_base.a**

[Note]

None.

[Example]

For details, see [Programming Examples](#).

[Related Topics]

[usb_init](#)

7.4.4.4 usb_is_devicemode

[Description]

Checks whether the USB device is connected to the USB host.

[Syntax]

```
bool usb_is_devicemode(void);
```

[Parameters]

None.

[Return Values]

Return Value	Description
TRUE	The USB device is connected to the USB host.
FALSE	The USB device is not connected to the USB host.

[Requirements]

- Header file: **usb_init.h**
- Library file: **libusb_base.a**

[Note]

This API can be called only after the USB device protocol is loaded.

[Example]

None.

[Related Topics]

None.

7.4.4.5 fmass_register_notify

[Description]

Registers the device notify callback function.

[Syntax]

```
int fmass_register_notify(void (*notify)(void *context, int status), void *context);
```

[Parameters]

Parameter	Description	Input/Output
notify	Pointer to the Notify callback function	Input
context	Private data pointer that needs to be transferred to the Notify callback function. The pointer is transferred to the context parameter of the Notify callback function.	Input

[Return Values]

Return Value	Description
0– MAX_NOIFY_NUM – 1	Successful
–1	Failed

[Requirement]

- Header file: **f_mass_storage.h**
- Library file: **libusb_device.a**

[Note]

Multiple notify callback functions can be registered, each of them will be invoked by the driver when the USB 2.0 device is inserted or removed.

[Example]

For details, see [Programming Examples](#).

[Related Topics]

- [fmass_partition_startup](#)
- [fmass_unregister_notify](#)

7.4.4.6 fmass_unregister_notify

[Description]

Deregisters the device notify callback function.

[Syntax]

```
int fmass_unregister_notify(int handle);
```

[Parameters]

Parameter	Description	Input/ Output
handle	Handle returned by fmass_register_notify .	Input

[Return Values]

Return Value	Description
0	Successful
–1	Failed

[Requirement]

- Header file: **f_mass_storage.h**
- Library file: **libusb_device.a**

[Note]

None.

[Example]

For details, see [Programming Examples](#).

[Related Topics]

[fmass_register_notify](#)

7.4.4.7 fmass_partition_startup

[Description]

Starts the SD card, MMC, or RAM media partition node that is identified by the PC.

[Syntax]

```
int fmass_partition_startup(const char *path);
```

[Parameters]

Parameter	Description	Input/Output
path	SD card, MMC, or RAM medium partition node must be valid.	Input

[Return Values]

Return Value	Description
0	Successful
-1	Failed

[Requirements]

- Header file: **f_mass_storage.h**
- Library file: **libusb_base.a**

[Note]

None.

[Example]

For details, see [7.4.3 Programming Examples](#).

[Related Topics]

[fmass_register_notify](#)

7.4.4.8 ram_mass_storage_init

[Description]

Enables memory mapping for the FAT32 file system image and registers the **/dev/uram** device node.

[Syntax]

```
int ram_mass_storage_init(void);
```

[Parameters]

None.

[Return Values]

Return Value	Description
0	Successful
-1	Failed

[Requirement]

- Header file: **f_mass_storage_ram.h**
- Library file: **libusb_base.a**

[Note]

None.

[Example]

For details, see [Programming Examples](#).

[Related Topics]

[ram_mass_storage_deinit](#)

7.4.4.9 ram_mass_storage_deinit

[Description]

Disables the memory mapping of the FAT32 file system image.

[Syntax]

```
int ram_mass_storage_deinit(void);
```

[Parameters]

None.

[Return Values]

Return Value	Description
0	Successful
-1	Failed

[Requirement]

- Header file: **f_mass_storage_ram.h**
- Library file: **libusb_base.a**

[Note]

None.

[Example]

For details, see [Programming Examples](#).

[Related Topics]

[ram_mass_storage_init](#)

7.4.4.10 fuvc_frame_descriptors_get

[Description]

Sets the video transmission capability set of the UVC device.

[Syntax]

```
void fuvc_frame_descriptors_get(const struct fuvc_format_info *format_info, uint32_t frames_count);
```

[Parameters]

Parameter	Description	Input/Output
format_info	Video stream formats and resolutions supported by the UVC	Input
frames_count	Total number of resolutions corresponding to the video format	Input

[Return Values]

None.

[Requirement]

- Header file: **f_uvc.h**
- Library file: **libusb_device.a**

[Note]

For details, see [Programming Examples](#).

[Example]

For details, see [Programming Examples](#).

[Related Topics]

None.

7.4.4.11 uvc_open_device

[Description]

Opens the UVC.

[Syntax]

```
int uvc_open_device(uvc_t *hdl, const struct uvc_open_param *param);
```

[Parameters]

Parameter	Description	Input/Output
hdl	Stores UVC device handles.	Output
param	Enables the initial capability of the UVC device. The value can be changed based on the actual requirements.	Input

[Return Values]

Return Value	Description
UVC_OK	Successful
UVC_ERROR_NOMATCH	The UVC status does not match.
UVC_ERROR_HANDLE	The UVC handle is invalid.
UVC_ERROR_PTR	The pointer is invalid.
UVC_ERROR_MEMORY	Insufficient memory
UVC_ERROR_VALUE	Invalid value

[Requirement]

- Header file: **f_uvc.h**
- Library file: **libusb_device.a**

[Note]

For details, see [Programming Examples](#).

[Example]

For details, see [Programming Examples](#).

[Related Topics]

[uvc_close_device](#)

7.4.4.12 uvc_close_device

[Description]

Closes the UVC.

[Syntax]

```
int uvc_close_device(uvc_t hdl);
```

[Parameters]

Parameter	Description	Input/Output
hdl	Stores UVC device handles.	Input

[Return Values]

Return Value	Description
UVC_OK	Successful
UVC_ERROR_FATAL	Fatal error

[Requirement]

- Header file: **f_uvc.h**
- Library file: **libusb_device.a**

[Note]

For details, see [Programming Examples](#).

[Example]

For details, see [Programming Examples](#).

[Related Topics]

[uvc_open_device](#)

7.4.4.13 uvc_wait_host

[Description]

Waits until the USB host is connected.

[Syntax]

```
int uvc_wait_host(uvc_t hdl, int wait_option, int *connected);
```

[Parameters]

Parameter	Description	Input/Output
hdl	Stores the UVC device handle.	Input
wait_option	Defines the status when waiting for connection: • UVC_WAIT_HOST_NOP: Returns the status immediately. • UVC_WAIT_HOST_FOREVER: The connection is blocked until the connection is established.	Input
connected	Connection status: • 0: disconnected • 1: connected	Output

[Return Values]

Return Value	Description
UVC_OK	Successful
UVC_ERROR_NOMATCH	The UVC status does not match.
UVC_ERROR_HANDLE	The UVC handle is invalid.
UVC_ERROR_PTR	The pointer is invalid.
UVC_ERROR_VALUE	Invalid value

[Requirement]

- Header file: **f_uvc.h**
- Library file: **libusb_device.a**

[Note]

For details, see [Programming Examples](#).

[Example]

For details, see [Programming Examples](#).

[Related Topics]

[uvc_close_device](#)

7.4.4.14 uvc_get_state

[Description]

Obtains the running status of the UVC device.

[Syntax]

```
int uvc_get_state(uvc_t hdl, uint32_t *state);
```

[Parameters]

Parameter	Description	Input/Output
hdl	Stores the UVC device handle.	Input
state	UVC device running status - UVC_STATE_IDLE: idle - UVC_STATE_CONN: connected - UVC_STATE_CLOSE: closed	Output

[Return Values]

Return Value	Description
UVC_OK	Successful
UVC_ERROR_HANDLE	The UVC handle is invalid.
UVC_ERROR_PTR	The pointer is invalid.
UVC_ERROR_VALUE	The value is invalid.

[Requirement]

- Header file: **f_uvc.h**
- Library file: **libusb_device.a**

[Note]

None.

[Example]

None.

[Related Topics]

None.

7.4.4.15 uvc_recv_pack

[Description]

Starts the UVC video transmission in copy mode.

[Syntax]

```
int uvc_recv_pack(const struct uvc_pack *pack);
```

[Parameters]

Parameter	Description	Input/Output
pack	Pointer to the UVC data packet.	Input

[Return Values]

Return Value	Description
UVC_OK	Successful
UVC_ERROR_MEMORY	Memory allocation failed.
UVC_ERROR_HANDLE	The UVC handle is invalid.
UVC_ERROR_PTR	The pointer is invalid.

[Requirement]

- Header file: **f_uvc.h**
- Library file: **libusb_device.a**

[Note]

For details, see [Programming Examples](#).

[Example]

For details, see [Programming Examples](#).

[Related Topics]

[uvc_video_stop](#)

7.4.4.16 uvc_video_stop

[Description]

Stops the UVC video transmission.

[Syntax]

```
int uvc_video_stop(uvc_t hdl);
```

[Parameters]

Parameter	Description	Input/Output
hdl	Stores the UVC device handle.	Input

[Return Values]

Return Value	Description
UVC_OK	Successful
UVC_ERROR_NOMATCH	The UVC status does not match.
UVC_ERROR_HANDLE	The UVC handle is invalid.
UVC_ERROR_PTR	The pointer is invalid.
UVC_ERROR_VALUE	Invalid value

[Requirement]

- Header file: **f_uvc.h**
- Library file: **libusb_device.a**

[Note]**[Related Topics]**[uvc_rcv_pack](#)

7.4.4.17 uvc_format_info_get

[Description]

Obtains the detailed information about the new stream format.

[Syntax]

```
int uvc_format_info_get(struct uvc_format_info *info);
```

[Parameters]

Parameter	Description	Input/Output
info	struct uvc_format_info{ uint32_t width; // Resolution, width uint32_t height; // Resolution, height uint32_t format; // Stream format uint32_t status; // 1 indicates that the format is being switched; 0 indicates that the format has been switched. } __attribute__((packed));	Output

[Return Values]

Return Value	Description
UVC_OK	Successful
UVC_ERROR_NOMATCH	The UVC status does not match.
UVC_ERROR_PTR	The pointer is invalid.
UVC_ERROR_FATAL	The error is fatal.

[Requirement]

- Header file: **f_uvc.h**
- Library file: **libusb_device.a**

[Note]

None.

[Example]

None.

[Related Topics]

[uvc_format_status_check](#)

7.4.4.18 uvc_format_status_check

[Description]

Defines the stream format switching event.

[Syntax]

```
enum format_switch_status uvc_format_status_check(void);
```

[Parameters]

None.

[Return Values]

Return Value	Description
FORMAT_SWITCH_FINISH	Format switching is complete.
FORMAT_SWITCH_PENDING	The format is being switched.

[Requirement]

- Header file: **f_uvc.h**
- Library file: **libusb_device.a**

[Note]

None.

[Example]

None.

[Related Topics]

[uvc_format_info_get](#)

7.4.4.19 hi_camera_register_cmd

[Description]

Defines the UVC extension command for application registration, which is called by the underlying driver.

[Syntax]

```
int hi_camera_register_cmd(const struct uvc_camera_cmd *cmd);
```

[Parameters]

Parameter	Description	Input/Output
cmd	UVC extended commands: <pre>struct uvc_camera_cmd { uint32_t id; // Event code uint8_t name[32]; // Name camera_control_func uvc_control_func; // Array for processing the function pointer information }; typedef uint32_t (*camera_control_func)(void *buf, uint32_t len, uint32_t event_id, uint32_t cmd_type);</pre>	Input

[Return Values]

Return Value	Description
0	Successful
-1	Failed

[Requirement]

- Header file: **f_uvc.h**
- Library file: **libusb_device.a**

[Note]

None.

[Example]

None.

[Related Topics]

None.

7.4.4.20 uac_wait_host

[Description]

Waits until the USB host is connected.

[Syntax]

```
int uac_wait_host(int wait_option);
```

[Parameters]

Parameter	Description	Input/Output
wait_option	Defines the status when waiting for connection: • UAC_WAIT_HOST_NOP: The status is returned immediately. • UAC_WAIT_HOST_FOREVER: Blocked until the connection is established.	Input

[Return Values]

Return Value	Description
UAC_OK	Successful
UAC_ERROR_NOMATCH	The status does not match.

Return Value	Description
UAC_ERROR_PTR	The pointer is invalid.
UAC_ERROR_VALUE	Invalid parameter

[Requirement]

- Header file: **f_uac.h**
- Library file: **libusb_device.a**

[Note]

None.

[Example]

None.

[Related Topics]

[fuac_send_message](#)

7.4.4.21 fuac_send_message

[Description]

Sends the audio data.

[Syntax]

```
int fuac_send_message(const void *buf,int len);
```

[Parameters]

Parameter	Description	Input/Output
buf	Audio data to be sent	Input
len	Length of the audio data to be sent	Input

[Return Values]

Return Value	Description
UAC_OK	Successful
UAC_ERROR_MEMORY	Insufficient memory
UAC_ERROR_PTR	The pointer is invalid.

Return Value	Description
UAC_ERROR_DISCONNECT	The device is disconnected.

[Requirement]

- Header file: **f_uac.h**
- Library file: **libusb_device.a**

[Note]

None.

[Example]

None.

[Related Topics]

None.

7.4.4.22 fuac_receive_message

[Description]

Receives the audio data.

[Syntax]

```
int fuac_receive_message(void *buf, int len, int timeout);
```

[Parameters]

Parameter	Description	Input/Output
buf	Buffer of audio data to be received.	Input
len	Length of the buffer for receiving audio data.	Input
timeout	Timeout interval for receiving the audio data.	Input

[Return Values]

Return Value	Description
int	Actual size of the received audio data.
-UAC_ERROR_MEMORY	The memory is insufficient.
-UAC_ERROR_PTR	The pointer is invalid.

Return Value	Description
- UAC_ERROR_DISCONNECT	The device is disconnected.
- UAC_ERROR_TIMEOUT	Waiting timedout.
-UAC_ERROR_EVENT	Error event.

[Requirement]

- Header file: **f_uac.h**
- Library file: **libusb_device.a**

[Note]

None.

[Example]

None.

[Related Topics]

None.

7.4.4.23 fuac_opts_set

[Description]

Sets UAC 1.0 audio parameters.

[Syntax]

```
int fuac_opts_set(const struct uac_opts *opts);
```

[Parameters]

Parameter	Description	Input/Output
opts	Audio parameter structure.	Input

[Return Values]

Return Value	Description
0	Successful
-1	Null pointer.

[Requirement]

- Header file: **f_uac.h**
- Library file: **libusb_device.a**

[Note]

None.

[Example]

None.

[Related Topics]

[fuac_get_opts](#)

7.4.4.24 fuac_get_opts

[Description]

Obtains the UAC 1.0 audio parameters.

[Syntax]

```
int fuac_get_opts(struct uac_opts *opts);
```

[Parameters]

Parameter	Description	Input/Output
opts	Audio parameter structure.	Output

[Return Values]

Return Value	Description
0	Successful.
-UAC_ERROR_PTR	The pointer is invalid.

[Requirement]

- Header file: **f_uac.h**
- Library file: **libusb_device.a**

[Note]

None.

[Example]

None.

[Related Topics]

[fuac_opts_set](#)

7.4.4.25 fuac_rate_get

[Description]

Obtains the sampling rate of UAC 1.0 audio.

[Syntax]

```
uint32_t fuac_rate_get(void);
```

[Parameters]

None.

[Return Values]

Return Value	Description
uint32_t	Current audio sampling rate of UAC 1.0.

[Requirement]

- Header file: **f_uac.h**
- Library file: **libusb_device.a**

[Note]

None.

[Example]

None.

[Related Topics]

None.

7.4.4.26 fuac2_get_opts

[Description]

Obtains the parameters of the audio 2.0 protocol, including the number of channels.

[Syntax]

```
int fuac2_get_opts(struct uac2_opts *opts);
```

[Parameters]

Parameter	Description	Input/Output
opts	Current audio parameter structure	Output

[Return Values]

Return Value	Description
0	Successful
-UAC_ERROR_PTR	Null pointer

[Requirement]

- Header file: **f_uac.h**
- Library file: **libusb_device.a**

[Note]

None.

[Example]

None.

[Related Topics]

[fuac2_set_opts](#)

7.4.4.27 fuac2_set_opts

[Description]

Sets the parameters of UAC 2.0 audio.

[Syntax]

```
int fuac2_set_opts(const struct uac2_opts *opts);
```

[Parameters]

Parameter	Description	Input/Output
opts	Audio parameter structure.	Input

[Return Values]

Return Value	Description
0	Successful.
-UAC_ERROR_PTR	The pointer is null.
-UAC_ERROR_VALUE	The parameter setting is not supported.

[Requirement]

- Header file: **f_uac.h**
- Library file: **libusb_device.a**

[Note]

None.

[Example]

None.

[Related Topics]

[fuac2_get_opts](#)

7.4.4.28 usb_dfu_init_env_entities

[Description]

Initializes the DFU API and configure environment variables (currently, only RAM is supported).

[Syntax]

```
int usb_dfu_init_env_entities(const char *type, char *envstr, const char *devstr);
```

[Parameters]

Parameter	Description	Input/Output
type	DFU function. Only RAM is supported.	Input
envstr	DFU environment parameter	Input
devstr	Not used	Input

[Return Values]

Return Value	Description
0	Successful
-1	Failed

[Requirement]

- Header file: **f_dfu.h**
- Library file: **libusb_device.a**

[Note]

None.

[Example]

None.

[Related Topics]

None.

7.4.4.29 usb_dfu_update_status

[Description]

Obtains the upgrade event.

[Syntax]

```
uint32_t usb_dfu_update_status(void);
```

[Parameters]

None.

[Return Values]

Return Value	Description
0	Upgrade not started
1	Upgrading

[Requirement]

- Header file: **f_dfu.h**
- Library file: **libusb_device.a**

[Note]

None.

[Example]

None.

[Related Topics]

None.

7.4.4.30 usb_dfu_update_size_get

[Description]

Obtains the actual size of the upgrade file.

[Syntax]

```
uint64_t *usb_dfu_update_size_get(void);
```

[Parameters]

None.

[Return Values]

Return Value	Description
uint64_t *	Address for saving the actual size of the upgrade file

[Requirement]

- Header file: **f_dfu.h**
- Library file: **libusb_device.a**

[Note]

None.

[Example]

None.

[Related Topics]

None.

7.4.4.31 usb_dfu_get_entity

[Description]

Obtains the DFU structure variable specified in the list.

[Syntax]

```
struct usb_dfu_entity *usb_dfu_get_entity(int alter);
```

[Parameters]

Parameter	Description	Input/Output
alter	Obtains the sequence number of a variable stored in the list.	Input

[Return Values]

Return Value	Description
struct usb_dfu_entity *	DFU structure variable, which stores the current DFU parameter values. For details, see the structure statement.

[Requirement]

- Header file: **f_dfu.h**
- Library file: **libusb_device.a**

[Note]

None.

[Example]

None.

[Related Topics]

None.

7.4.4.32 usb_dfu_read

[Description]

Obtains the buffer data of a certain size in the specified DFU list.

[Syntax]

```
int usb_dfu_read(struct usb_dfu_entity *dfu, void *buf, int size, uint32_t blk_seq_num);
```

[Parameters]

Parameter	Description	Input/Output
dfu	DFU list structure information where buffer data is located, which can be obtained by calling usb_dfu_get_entity .	Input
buf	RX buffer	Input
size	Size of received data	Input
blk_seq_num	Number of times that the value is obtained. The value must be a continuous self-added value.	Input

[Return Values]

Return Value	Description
int	Actual length of the obtained data

[Requirements]

- Header file: **f_dfu.h**
- Library file: **libusb_device.a**

[Note]

None.

[Example]

None.

[Related Topics]

None.

7.4.4.33 hid_report_descriptor_info

[Description]

Configures the report descriptor information to set the report data format and function.

[Syntax]

```
int hid_report_descriptor_info(const void *report_desc, size_t report_desc_len);
```

[Parameters]

Parameter	Description	Input/Output
report_desc	Report descriptor information.	Input
report_desc_len	Total size of the report descriptor.	Input

[Return Values]

Return Value	Description
0	Successful
-1	Failed

[Requirement]

- Header file: **f_hid.h**
- Library file: **libusb_device.a**

[Note]

The definition of the report descriptor must comply with the standard HID protocol and be used together with the VFS module API.

[Example]

For details, see the official HID documents: *Device Class Definition for Human interface Devices* and *HID Usage Tables*.

[Related Topics]

None.

7.4.4.34 hid_add_report_descriptor

[Description]

Configures multiple report table descriptors and sets the report table data format, function, and API protocol.

[Syntax]

```
int hid_add_report_descriptor(const uint8_t *report_desc, size_t report_desc_len, uint8_t protocol);
```

[Parameters]

Parameter	Description	Input/Output
report_desc	Report descriptor information.	Input
report_desc_len	Total size of the report descriptor.	Input
protocol	The default value is 0 . To support the startup API, the value 1 or 2 needs to be transferred. The value 1 indicates the keyboard, and the value 2 indicates the mouse.	Input

[Return Values]

Return Value	Description
report_index	Success (corresponding report index value)
-1	Failed

[Requirement]

- Header file: **f_hid.h**
- Library file: **libusb_device.a**

[Note]

The definition of the report descriptor must comply with the standard HID protocol and be used together with the **fhid_send_data** and **fhid_rcv_data** APIs.

[Example]

For details, see the official HID documents: *Device Class Definition for Human interface Devices* and *HID Usage Tables*.

[Related Topics]

None.

7.4.4.35 fhid_send_data

[Description]

Sends the HID data.

[Syntax]

```
ssize_t fhid_send_data (uint8_t report_index, const char *buf, size_t buflen);
```

[Parameters]

Parameter	Description	Input/Output
report_index	Report index, which is obtained from the return value of the hid_add_report_descriptor API.	Input
buf	HID data to be sent.	Input
buflen	Total size of HID data.	Input

[Return Values]

Return Value	Description
ssize_t	Actual size of the sent HID data
-1	Failed

[Requirement]

- Header file: **f_hid.h**
- Library file: **libusb_device.a**

[Note]

None.

[Example]

None.

[Related Topics]

None.

7.4.4.36 fhid_rcv_data

[Description]

Receives the HID data.

[Syntax]

```
ssize_t fhid_rcv_data(uint8_t report_index, char *buf, size_t buflen);
```


[Parameters]

Parameter	Description	Input/Output
report_index	Report index, which is obtained from the return value of the hid_add_report_descriptor API.	Input
buf	Buffer that receives the HID data	Input
buflen	Length of the buffer that receives the HID data	Input

[Return Values]

Return Value	Description
ssize_t	Actual length of the obtained data
-1	Failed

[Requirement]

- Header file: **f_hid.h**
- Library file: **libusb_device.a**

[Note]

This API can be called only in tasks and cannot be called during the interrupt callback because this API has the read data blocking mechanism.

[Example]

None.

[Related Topics]

None.

7.4.4.37 u3g_write_from_shell

[Description]

Sends the serial port command to the 4G module.

[Syntax]

```
uint32_t u3g_write_from_shell(int argc, const char **argv);
```

[Parameters]

Parameter	Description	Input/Output
argc	Number of parameters.	Input

Parameter	Description	Input/Output
argv	Pointer array. Each element points to a string.	Input

[Return Values]

Return Value	Description
LOS_OK	Successful
LOS_NOK	Failed

[Requirement]

- Header file: **usb_host.h**
- Library file: **libusb_base.a**

[Note]

The device nodes corresponding to each 4G module are different. Some device nodes are **/dev/ttyUSB0** and some are **/dev/ttyUSB1**. Therefore, you need to change the port number of the corresponding device based on the module model.

[Example]

```
static void usb_4g_modem(void)
{
    const char *apOpenCmd[] = {"open", NULL};
    const char *apSetModeCmd[] = {"at+qcfg=\"usbnet\",3", NULL};
    const char *apDialCmd[] = {"at+qnetdevctl=1,1", NULL}; // EC200T module performs dial up.
    sleep(10);
    u3g_write_from_shell(1, apOpenCmd);
    sleep(1);
    u3g_write_from_shell(1, apSetModeCmd);
    sleep(1);
    u3g_write_from_shell(1, apDialCmd);
    sleep(1);
}
```

[Related Topics]

None.

7.4.4.38 uvc_get_frame_buf

[Description]

Obtains the file handle of the character device by using the file descriptor **fd**.

[Syntax]

```
uint8_t *uvc_get_frame_buf(struct file *filep, uint32_t index, uint32_t offset);
```

[Parameters]

Parameter	Description	Input/Output
filep	Obtains the file handle of the character device by using the file descriptor fd .	Input
index	Index of the current buffer frame.	Input
offset	Queries and allocates the start point of the frame data by using the VIDIOC_QUERYBUF .	Input

[Return Values]

Return Value	Description
uint8_t *	Returns the address of the frame buffer.

[Requirements]

- Header file: **usb_v4l2.h**
- Library file: **libusb_base.a**

[Note]

None.

[Example]

None.

[Related Topics]

None.

7.4.4.39 usb_dfu_free_entities

[Description]

Releases DFU-related resources.

[Syntax]

```
void usb_dfu_free_entities(void)
```

[Parameters]

None.

[Return Values]

None.

[Requirements]

- Header file: **f_dfu.h**

- Library file: **libusb_device.a**

[Note]

None.

[Example]

None.

[Related Topics]

None.

7.5 UART Driver Development

7.5.1 Overview

UART is short for universal asynchronous receiver/transmitter. In the embedded design, UART is used to communicate with the PC, including with monitor debugger, EEPROM and other devices.

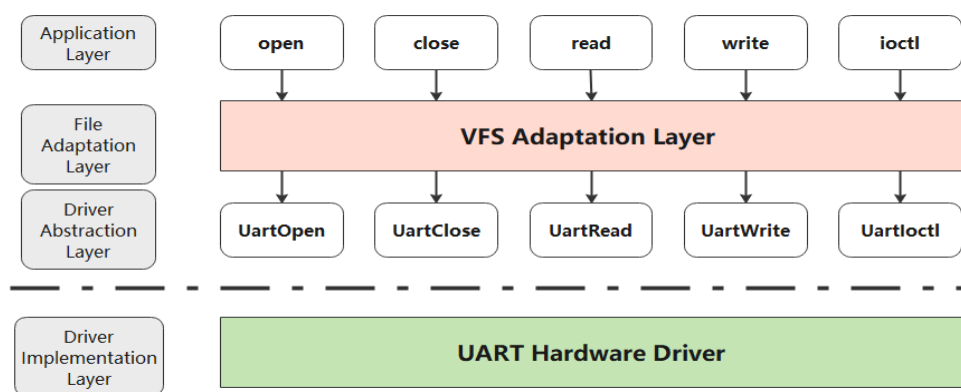
The basic framework of the Huawei_LiteOS UART driver module consists of the file adaptation layer, driver abstraction layer and driver implementation layer.

File adaptation layer: **LOSCFG_FS_VFS** is used for UART management and control through files, which can be tailored.

Driver abstraction layer: External APIs are provided to control the UART through **struct LosDevice** device files.

Driver implementation layer: Hardware implementation layer of the UART controller, which does not provide external API.

Figure 7-8 Basic framework of the UART driver



Operation Mechanism

The development process of the UART driver is as follows:

1. Load the UART driver.

The driver abstraction layer provides an API for loading the UART driver. The system registers the driver during startup.

2. Load the UART device.

The UART device is loaded by the system according to the actual situation.

3. Send and receive the UART data.

The driver abstraction layer provides raw read and write APIs, and the file adaptation layer provides read and write in VFS file mode.

7.5.2 Development Guidelines

7.5.2.1 Raw Read/Write UART

The UART driver abstraction layer provides the following APIs:

API	Description
struct LosDevice *UartDevGetByPort(int num)	Obtains the UART device through the serial port number.
int UartOpen(struct LosDevice *dev)	Opens a specified UART device.
int UartClose(struct LosDevice *dev)	Closes a specified UART device.
ssize_t UartRead(struct LosDevice *dev, char *buf, size_t count)	Reads the data of a specified UART device.
ssize_t UartWrite(struct LosDevice *dev, const char *buf, size_t count)	Writes the data to a specified UART device.
int UartIoctl(struct LosDevice *dev, int cmd, unsigned long arg)	Controls the specified UART device. (The control modes are provided by the driver implementation layer of the UART controller.)
int UartSuspend(struct LosDevice *dev)	Suspends a specified UART device.
int UartResume(struct LosDevice *dev)	Resumes a specified UART device.

7.5.2.2 UART Read/Write in File Mode

For details, see [VFS](#). The open, close, read, write, and ioctl operations are supported.

7.5.2.3 Module Configuration

Open the menu and choose **Driver > Uart Type** to configure the UART module.

Configuration Item	Description	Value Range	Default Value	Dependency
LOSCFG_DRIVERS_SIMPLE_UART	Simple UART. Only output is supported.	YES/NO	YES	None.
LOSCFG_DRIVERS_UART	Complete UART. Input and output are supported.	YES/NO	NO	LOSCFG_DRIVERS_BASE

7.5.3 Programming Example

7.5.3.1 Simple UART

Enable LOSCFG_DRIVERS_SIMPLE_UART. The simple UART only supports output.

7.5.3.2 General UART

Enable LOSCFG_DRIVERS_UART. The general UART supports both output and input.

7.5.3.2.1 UART Raw Read/Write

Example Description

Step 1 Obtain the UART device based on the serial port number.

Step 2 Enable the serial port.

Step 3 Write the serial port.

Step 4 Suspend the serial port.

Step 5 Resume the serial port.

Step 6 Disable the serial port.

----End

Example Code

```
#define BUF_SIZE 100

UINT32 TestCase(VOID)
{
    struct LosDevice *uartDev = NULL;
    int ret;
    ssize_t rtRet;
    char dataBuf[BUF_SIZE] = {"test uart\n"};

    uartDev = UartDevGetByPort(0);
    if (uartDev == NULL) {
```

```
    return LOS_NOK;
}

ret = UartOpen(uartDev);
if (ret != 0) {
    return LOS_NOK;
}

rtRet = UartWrite(uartDev, dataBuf, BUF_SIZE);
if (rtRet <= 0) {
    goto OUT_CLOSE;
}

ret = UartSuspend(uartDev);
if (ret != 0) {
    goto OUT_CLOSE;
}

ret = UartResume(uartDev);
if (ret != 0) {
    goto OUT_CLOSE;
}

ret = UartClose(uartDev);
if (ret != 0) {
    return LOS_NOK;
}

return LOS_OK;

OUT_CLOSE:
    (VOID)UartClose(uartDev);
    return LOS_NOK;
}
```

7.5.3.2.2 UART Read/Write in File Mode

Example Description

- Step 1** Enable **LOSCFG_FS_VFS**.
 - Step 2** Enable the serial port.
 - Step 3** Write the serial port.
 - Step 4** Suspend the serial ports.
 - Step 5** Resume the serial port.
 - Step 6** Disable the serial port.
- End

Example Code

```
#define BUF_SIZE 100
#define UART_DEV_PATH "/dev/uartdev-0"

UINT32 TestCase(VOID)
{
    int ret;
    int fd;
    char dataBuf[BUF_SIZE] = {"test uart\n"};
    ssize_t writeLen;

    fd = open(UART_DEV_PATH, O_RDWR);
```

```
    if (fd == -1) {  
        return LOS_NOK;  
    }  
  
    writeLen = write(fd, dataBuf, BUF_SIZE);  
    if (writeLen <= 0) {  
        goto OUT_CLOSE;  
    }  
  
    ret = ioctl(fd, PI011_IOCTL_SUSPEND, 0);  
    if (ret != 0) {  
        goto OUT_CLOSE;  
    }  
  
    ret = ioctl(fd, PI011_IOCTL_RESUME, 0);  
    if (ret != 0) {  
        goto OUT_CLOSE;  
    }  
  
    ret = close(fd);  
    if (ret != 0) {  
        return LOS_NOK;  
    }  
  
    return LOS_OK;  
OUT_CLOSE:  
    (VOID)close(fd);  
    return LOS_NOK;  
}
```


8 Configuration Tool

- [8.1 Overview](#)
- [8.2 Installation Tools](#)
- [8.3 Usage Guidelines](#)
- [8.4 menuconfig](#)
- [8.5 savemenuconfig](#)
- [8.6 make defconfig](#)
- [8.7 make allyesconfig](#)
- [8.8 make allnoconfig](#)
- [8.9 Configuration Item Description](#)

8.1 Overview

Huawei LiteOS uses Kconfig files to configure the system. The Kconfig language is a menu configuration language. Both config.in and Kconfig are compiled in this language. Huawei LiteOS uses python kconfiglib to parse and display Kconfig files. After a Kconfig file is parsed, the .config file is generated or updated in the root directory, and the **menuconfig.h** file is generated in the **include** folder of the development board. Before using menuconfig, install python and kconfiglib.

8.2 Installation Tools

- Install Python 2.7/3.2+.
 - Installation through CLI:

```
sudo apt-get install python3.10
```
 - Installation through source code compilation:
 - i. Download the [official Python source code package](#).
 - ii. Decompress the source code package.

Run the following command to decompress the package. Replace the package name with the downloaded source code package name.

```
tar -xf Python-3.10.2.tgz
```

iii. Check the dependency.

After the package is decompressed, go to the directory, and run the **./configure** command to check the dependency required for compiling and installing Python.

```
cd Python-3.10.2
./configure
```

If no error is reported, go to the next step. If an error is reported, install the dependency as prompted.

iv. Compile and install Python.

```
sudo make
sudo make install
```

v. Check the Python version and properly link the Python command.

```
python --version
```

If the displayed version is not the installed Python version, run the following command to properly link the Python command:

1)

```
which python3.10
```

2) Link the Python command to the newly installed Python package.

```
cd /usr/bin
sudo rm python3 && sudo ln -s "python3.10-path" python3
sudo rm python && sudo ln -s python3 python
```

3) Check the Python version again.

```
python --version
python3 --version
```

- Install the pip package management tool.

If no pip command is available, download the pip source code package and install it. pip depends on setuptools. Install pip even if setuptools does not exist.

- Installation through CLI:

```
sudo apt-get install python3-setuptools python3-pip -y
sudo pip3 install --upgrade pip
```

- Installation through the source code package:

- i. Install setuptools.

Click the [download address of the setuptools source code package](#) and run the following commands to install setuptools:

```
sudo unzip setuptools-50.3.2.zip
cd setuptools
sudo python setup.py install
```

NOTICE

The latest setuptools version does not support Python 2.7. To use Python 2.7, download [setuptools 45.0.0](#).

- ii. Install pip.

Click the [download address of the pip source code package](#) and run the following commands to install pip:

```
sudo tar -xf pip-20.2.4.tar.gz
cd pip-20.2.4
sudo python setup.py install
```

- Install the kconfiglib library.
 - If the server can be connected to the Internet:
Run the following command to install kconfiglib:

```
sudo pip install kconfiglib
```
 - If the server cannot be connected to the Internet:
Install kconfiglib in offline mode. Download **kconfiglib** from an environment that can be connected to the Internet. You can download either the wheel file **kconfiglib-14.1.0-py2.py3-none-any.whl** or the source code file **kconfiglib-14.1.0.tar.gz**. The following uses kconfiglib 14.1.0 as an example.
 - To install the wheel file, run the following command:

```
sudo pip install kconfiglib-14.1.0-py2.py3-none-any.whl
```
 - To install the source code file, run the following commands:

```
sudo tar -zxvf kconfiglib-14.1.0.tar.gz
cd kconfiglib-14.1.0
sudo python setup.py install
```

8.3 Usage Guidelines

Before using menuconfig, ensure that the cross compiler tool chain for compiling the Huawei LiteOS source code has been installed and environment variables have been added.

This section provides the commands that are supported by menuconfig to meet the requirements of different usage scenarios. Before running the commands, copy the default configuration file `${platform}.config` in **tools/build/config/** to the root directory and rename it **.config**. Another important configuration file **tools/menuconfig/config.in** is required, which contains the Kconfig file of each module. The **.config** file determines the default value of each configuration item. The **config.in** file determines the configuration items displayed on the GUI.

8.4 menuconfig

Run the **make menuconfig** command or the **./build/build_cmake.sh menuconfig** command in the root directory of Huawei_LiteOS to display the configuration GUI. You can tailor the module or modify the settings. After you save the settings and exit, the **.config** file in the root directory is updated.

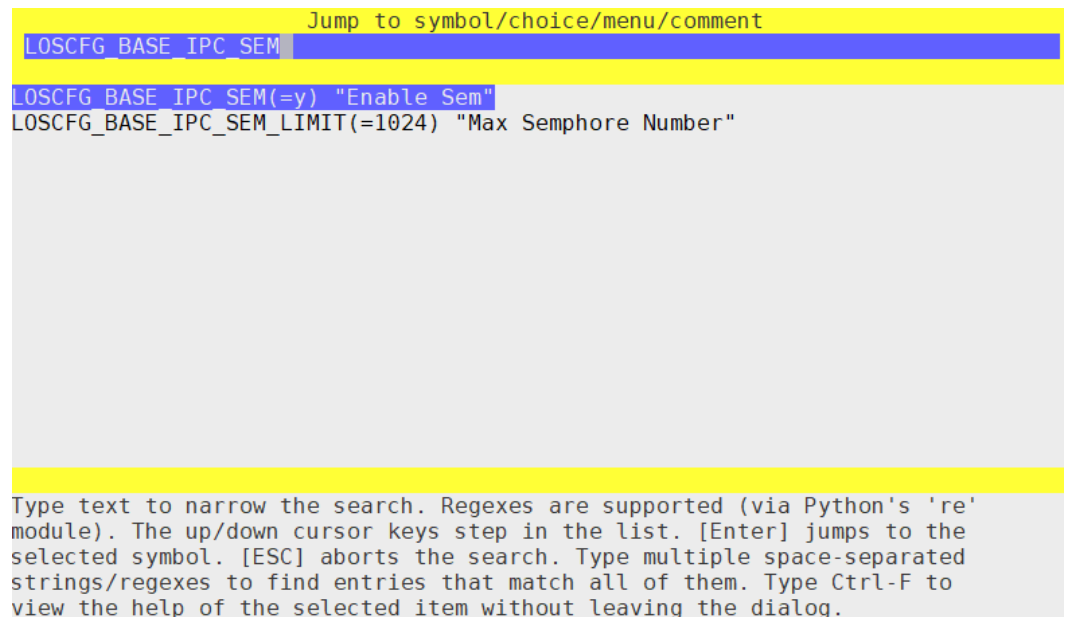
menuconfig provides the following operation methods:

- Up and down arrow keys: Select different rows and press the keys to move to different options.
- Space or Enter key: Enable or disable an option.
 - a. Enable an option: If there is an asterisk in the brackets [*] in front of an option, the option is enabled.
 - b. Disable an option: If only brackets [] are displayed in front of an option, the option is disabled.

- c. If ---> is displayed behind an option, the option contains sub-options. Press the space key or **Enter** key to go to the suboption.
- ESC key: Press this key to return to the upper-level menu or exit menuconfig and save the settings as prompted.
- Question mark (?) : Displays the help information about a configuration item.
- Slash (/): Go to the page for searching for configuration items.
- F: Enter the help mode. The help information about the configuration item is displayed in the lower part of the interface. Press **F** again to exit the help mode.
- C: Enter the name mode. The macro switch of the configuration item is displayed. Press **C** again to exit the name mode.
- A: Enter the all mode. All options in the menu are displayed. Press **A** again to exit the all mode.
- S: save configuration items.
- Q: Exit menuconfig and save the settings as prompted.

NOTE

- The letters are case-insensitive.
- You can press the slash key (/) to go to the search page, enter the macro definition of the configuration item to be searched for in the text box, for example, LOSCFG_BASE_IPC_SEM. The configuration items matching the keyword are automatically displayed. Select the required configuration item and press **Enter**.



8.5 savemenuconfig

For the Make compilation tool, run the **make savemenuconfig** command in the root directory of Huawei_LiteOS to parse the .config file in the root directory and generate the **menuconfig.h** file in the **include** directory of the corresponding Huawei_LiteOS development board project.

For the CMake compilation tool, run **./build/build_cmake.sh savemenuconfig** in the Huawei LiteOS root directory to parse the .config file in the root directory and

generate the **menuconfig.h** file in the **buildcmakeout/xxx/** subdirectory of the Huawei LiteOS development board project.

8.6 make defconfig

Run the **make defconfig** command in the root directory of Huawei_LiteOS to parse the .config file in the root directory (CMake compilation tool does not support this function.) to use the default values for all configuration items and update the .config file. In addition, the **menuconfig.h** file is generated in the **include** directory of the corresponding Huawei_LiteOS development board project.

8.7 make allyesconfig

Run the **make allyesconfig** command in the root directory of Huawei_LiteOS to parse the .config file in the root directory (CMake compilation tool does not support this function.) to enable all configuration items (the value is **Y**.) and update the .config file. In addition, the **menuconfig.h** file is generated in the **include** directory of the corresponding Huawei_LiteOS development board project.

8.8 make allnoconfig

Run the **make allnoconfig** command in the root directory of Huawei_LiteOS to parse the .config file in the root directory (CMake compilation tool does not support this function.) to disable all configuration items (the value is **is not set**.) and update the .config file. In addition, the **menuconfig.h** file is generated in the **include** directory of the corresponding Huawei_LiteOS development board project.

8.9 Configuration Item Description

8.9.1 Overview

Run make menuconfig to go to the **Huawei LiteOS Configuration** page. Currently, options such as Compiler, Targets, Kernel, Debug and SSP are available. This section describes Compiler, Targets, and SSP. For other options, see the module sections.

8.9.2 Compiler

Compiler contains two menu options: Compiler (cross-compiler tool chain) and Optimize Option (compilation optimization option).

```
(Top) → Compiler
Huawei LiteOS Configuration
Compiler (arm-himix410-linux) --->
Optimize Option (Optimize Speed) --->

[Space/Enter] Toggle/enter  [ESC] Leave menu      [S] Save
[O] Load                  [?] Symbol info      [/] Jump to symbol
[F] Toggle show-help mode  [C] Toggle show-name mode [A] Toggle show-all mode
[Q] Quit (prompts for save) [D] Save minimal config (advanced)
```

- Two cross compilers are available: arm-himix410-linux and arm-none-eabi-. Select the actual compiler.
- Three compilation optimization options are available: Optimize None, Optimize Speed, and Optimize Size.

8.9.3 Targets

Targets include **Family**, **Board**, **Enable Floating Pointer Unit**, and **Enable Access Permission Control**.

```

(Top) → Targets
Family (BVT) --->
Board (hi3556v200) --->
[*] Enable Floating Pointer Unit
[*] Enable Access Permission Control

[Space/Enter] Toggle/enter  [ESC] Leave menu          [S] Save
[O] Load                   [?] Symbol info           [J] Jump to symbol
[F] Toggle show-help mode  [C] Toggle show-name mode [A] Toggle show-all mode
[Q] Quit (prompts for save) [D] Save minimal config (advanced)

```

- Family: Select a product, such as BVT, DPT, and AIOT. SECRET indicates a confidential project, and QEMU and STM32 are used internally in Huawei LiteOS.
- Board: Select a board. The options are hi3556v200, hi3516ev200, hi3556av100, and hi3731v110.
- Enable Access Permission Control: The memory management units can be enabled based on the architecture, including the PMP under the RISC-V architecture, MPU under the Cortex-M architecture, and MMU under the Cortex-A/R architecture.

8.9.4 SSP

Select Stack Smashing Protector (SSP) Compiler Feature to enable or disable the stack protection function.

- `-fno-stack-protector`: disables stack protection.
- `-fstack-protector`: enables stack protection. If the local variable of the C or C++ function contains a char array (a character array whose size is greater than or equal to 4 bytes) and the array is not optimized, the compiler inserts protection code for the function.
- `-fstack-protector-strong`: enables stack protection. If a C or C++ function has an array definition and is not optimized, or an address reference of a local variable exists, the compiler inserts protection code (default option) for the function.
- `-fstack-protector-all`: inserts protection code for all C or C++ functions, which may greatly increase the performance overhead compared with `-fstack-protector`.

-fstack-protector is recommended to enhance security while ensuring performance.

- ArchStackGuardInit: The value provided on LiteOS is a pseudo-random number, specifically, a canary value. The vendor needs to adapt to the true random number to ensure security.

```

mashing Protector (SSP) Compiler Feature → Enable stack buffer overflow detection
Huawei LiteOS Configuration
( ) -fno-stack-protector
( ) -fstack-protector
(X) -fstack-protector-strong
( ) -fstack-protector-all

[Space/Enter] Toggle/enter  [ESC] Leave menu          [S] Save
[O] Load                  [?] Symbol info           [/] Jump to symbol
[F] Toggle show-help mode  [C] Toggle show-name mode [A] Toggle show-all mode
[Q] Quit (prompts for save) [D] Save minimal config (advanced)

```

CAUTION

LiteOS provides the weak function **ArchStackGuardInit** to initialize the **__stack_chk_guard** by using the pseudo-random number. To obtain the true random number, you need to redefine **ArchStackGuardInit**. Besides, some architectures do not have random number seeds and cannot generate the pseudo-random number. Therefore, you need to verify the randomness of **__stack_chk_guard** by yourself.

8.9.5 NX

Set **Enable Data Sec NX Feature** to enable or disable the function of not executing data segments.

- NX is recommended to be used together with the link script provided by the system. If you do not use the system link script, you need to compile a private link script by referring to the system link script.
- NX takes effect only on the architecture that supports the MMU. For the architecture that does not support the MMU, you need to implement the NX during initialization.


```

(Top) → Kernel
Huawei LiteOS Configuration
[*] Enable Mini Kernel
[ ] Enable Kernel SMP ----
    Basic Config --->
    Memory Management --->
    Interrupt Management --->
    Exception Management --->
    *- Enable Software Timer --->
    *- Enable Queue --->
    *- Enable Event ----
    *- Enable Mutex --->
    *- Enable Sem --->
[ ] Enable Data Sec NX Feature
[*] Enable Kernel Printf Frame
[*] Enable Kernel Printf Size Extend
[ ] Enable Ringbuf
[*] Enable Extend Kernel
[*] C++ Support
[*] Enable Cpu
[*] Enable Cpu include irq
[ ] Enable Dynamic Load Feature
[*] Enable Low Power Management Framework
    Low Power Management Configure --->
[ ] Enable Perf Feature
[*] Enable System Resource Check
+++++
[Space/Enter] Toggle/enter [ESC] Leave menu [S] Save
[O] Load [?] Symbol info [/] Jump to symbol
[F] Toggle show-help mode [C] Toggle show-name mode [A] Toggle show-all mode
[Q] Quit (prompts for save) [D] Save minimal config (advanced)

```

9 Maintenance and Debugging

[9.1 Telnet](#)

[9.2 Shell](#)

[9.3 Scheduling Statistics](#)

[9.4 Memory Debugging](#)

[9.5 MMC Debugging](#)

[9.6 Inter-Task Communication Debugging](#)

[9.7 Exception Interaction](#)

[9.8 Low Power Consumption Debugging Methods](#)

[9.9 Usage of Dying Gasp](#)

[9.10 Usage of Magic Keys](#)

[9.11 Usage of the Stack Estimation Tool](#)

[9.12 Debugging Case](#)

9.1 Telnet

9.1.1 Overview

Basic Concepts

Telnet, as part of TCP/IP protocol, is the standard protocol and major method for remote Internet login. Telnet enables you to operate a remote server on a local computer. Telnet program connects the local computer to the server. Telnet enables you to input commands just like direct input by using the server console.

Huawei LiteOS Telnet

Telnet uses commands to debug a development board. To achieve network connection by using Telnet, users must configure the IP address and gateway (which must be in the same network segment as the development board) for the

local computer and run cmd.exe in Windows to execute Telnet IP (IP address of the development board). In other words, Telnet is another debugging mode by using serial ports.

Huawei LiteOS Telnet adopts a simple Telnet protocol, which connects the computers to the development board without authentication on the user name and password. Huawei LiteOS Telnet is used to:

- Read the input characters and transmit them to the development board through TCP.
- Send back the processed data to terminals.

9.1.2 Development Guidelines

Development Procedure

1. Run telnet on to start telnet server in Huawei LiteOS Shell.

```
Huawei LiteOS # telnet on
Huawei LiteOS # init telnet.
```

2. In a Windows-based OS that has Telnet client installed, run cmd.exe and enter telnet + IP address of the development board to connect the computer to the board, as shown in the following figure.

```
C:\Users\Administrator>telnet 192.168.1.1
```

Press Enter. If Huawei LiteOS# (the prompt of Huawei LiteOS Shell) is displayed, it indicates that the computer is connected to the board successfully and you can run the shell commands. For example, you can run the task command to view the status of all tasks.

```
Huawei LiteOS # task
```

Name	PID	Priority	Status	StackSize	WaterLine	StackPoint
Swt_Task	0x0	0	QueuePend	0x6000	0x320	0x85121038
ui_MainApp	0x1	10	Pend	0x6000	0x16b0	0x85126f88
IdleCore000	0x2	31	Ready	0x400	0x154	0x851274a4
RecvAndDispatch	0x3	10	Ready	0x6000	0x25c	0x8522ad2c
SavePara2Flash	0x4	10	Ready	0x6000	0x244	0x8529fc94
RecMngMsgRcv	0x5	10	Ready	0x6000	0x784	0x852ba924
RecMngRecTimer	0x6	10	Ready	0x6000	0x284	0x852c08ec
RecMsgRcv	0x7	10	Ready	0x6000	0x23c	0x852e3d40
Isprun	0x8	10	Ready	0x6000	0x151c	0x85301424
ProcessEncStreamThread	0x9	2	PendTimeOut	0x6000	0xbac	0x85339cfc
enc_get	0xa	10	Pend	0x6000	0x3a4	0x8539d128
SendAFrameToEncThread	0xb	10	Ready	0x6000	0x4130	0x853a30e8

3. Run telnet off to stop telnet server in Huawei LiteOS Shell.

```
Huawei LiteOS # telnet off
telnetd_accept_loop[412] Software caused connection abort
close telnet.
```

```
Huawei LiteOS #
```

9.1.3 Precautions

Enabling the Telnet service in Windows:

- If **telnet is not recognized as an internal or external command** is returned after the input of telnet + IP in a Windows-based OS, the Telnet is not enabled. Choose **Control Panel > Programs > Programs and Features > Turn Windows features on or off** and select the **Telnet Server** and **Telnet Client**.

Preparations for starting Telnet on Huawei LiteOS:

- Ensure that the network driver of the board is initialized and opened before starting Telnet.
- Ensure that lwip started normally before starting Telnet. Initialized tcpip is needed while using lwip.

Telnet usage restrictions:

- After a client is connected to a development board using Telnet, setting the IP address of the development board using **ifconfig** will break the connection.
- Currently, only one client can be connected to a development board using telnet and an IP address at one time.
- Telnet only supports the commands supported by Shell.
- Telnet printout is prohibited in the following scenarios: locked tasks, interrupt callback function, and software timer callback function.
- Telnet is a debug function and is disabled by default. It must not be used in formal Huawei LiteOS.
- Disclaimer: Huawei shares no responsibilities for any risks caused in the use of Telnet in commercial products.

9.2 Shell

9.2.1 Overview

Basic Concept

Huawei LiteOS provides shell command lines, which can be used to access OS functions or services in command line interaction mode. The shell command lines receive and parse commands entered by users and process the output of the OS.

Huawei LiteOS Shell

As an online debugging tool, Huawei LiteOS Shell supports basic debugging commands for the system, files, network, dynamic loading, and trace. In addition, customized commands can be added to Huawei LiteOS Shell as required.

- Commands for Huawei LiteOS are used for checking system tasks, kernel semaphore, CPU usage, and current interrupts. For details, see [System Commands](#).

- Commands for files include **ls**, **cd**, and **sync**. The **sync** command synchronizes the cache data (data in the file system) to an SD card or flash memory. For details, see [File Commands](#).
- Commands for network are used to view the IP address of the local computer and other devices that connect to the development board, test the network connection, and set the IP address and station mode of development board. For details, see [Network Commands](#).
- Commands for dynamic loading are used to obtain the **.obj** file from a specified path and call related functions by searching for the addresses of functions that have loaded the **.obj** file. A user can unload the **.obj** file that has been loaded to a specific path. For details, see [Dynamic Loading Commands](#).
- Commands for trace are used to enable or disable the trace function, set trace events, and output trace buffer data. For details, see [Trace Commands](#).

9.2.2 Development Guidelines

Application Scenario

Shell commands can be entered through a serial port or Telnet. Customized commands can be executed only after their links are recompiled.

Functions

1. For details about the shell commands provided by Huawei LiteOS, see the sections of detailed command description.
2. The shell module of Huawei LiteOS provides the following APIs. For details about the APIs, see the API reference.

Function Category	API	Description
Registration command	SHELLCMD_ENTRY	Static registration command
	osCmdReg	Dynamic registration command

 NOTE

Normally, static registration is applicable to common system commands, and dynamic registration is applicable user commands.

The static registration command has five input parameters, and the dynamic registration command has four input parameters. The first input parameter is unique for static registration. The other four input parameters are the same for the two registration commands.

- First input parameter: Command variable name, which is used to set the link option (`LITEOS_TABLES_LDFLAGS` variable of `build/mk/liteos_tables_ldflags.mk`). For example, if the variable name is `ls_shellcmd`, the link option must be set to `LITEOS_TABLES_LDFLAGS += -uls_shellcmd`. This input parameter is unique to static registration and does not exist in dynamic registration.
- Second input parameter: Command type. Currently, two command types are supported.
 - **CMD_TYPE_EX**: Standard command parameters cannot be input. The command keywords entered by users are shielded. For example, if you enter `ls /ramfs`, only `/ramfs`, which corresponds to `argv[0]` in the command processing function, is transferred to the function. The keyword `ls` is not transferred.
 - **CMD_TYPE_STD**: Standard command parameter can be input. All entered characters are input after being parsed by the command. For example, if you enter `ls /ramfs`, `ls` and `/ramfs`, which correspond to `argv[0]` and `argv[1]` respectively, are transferred to the command processing function.
- Third input parameter: Command keyword, which is the name of the command processing function in the shell. The command keyword must be unique. That is, two different commands cannot share the same command keyword. Otherwise, only one is executed. During user command execution, if multiple commands share the same keyword, the shell executes only the first one among the **help** commands.
- Fourth input parameter: Maximum number of input parameters for the command processing function.
 - Currently, this parameter cannot be set for the static registration command.
 - For the dynamic registration command, this parameter can be set to **32**, which indicates a maximum of 32 input parameters, or **XARGS** (defined as `0xffffffff` in the code), which indicates no limit for the number of parameters.
- Fifth input parameter: Name of the command processing function, that is, the function called during the execution by the shell.

Configuration Items

Run the **make menuconfig** command and choose **Debug > Enable a Debug Version > Enable Shell** to configure the shell.

Configuration Item	Description	Value Range	Default Value	Dependency
LOSCFG_SHELL	Determines whether to enable the shell module.	YES/NO	YES	LOSCFG_DEBUG_VERSION(=y) && LOSCFG_DRIVER_S_UART(=y)

Configuration Item	Description	Value Range	Default Value	Dependency
LOSCFG_SHELL_CONSOLE (it is unavailable in the open-source version)	Determines whether to directly interact the shell with the console.	YES/NO	YES	LOSCFG_SHELL(=y)
LOSCFG_SHELL_UART	Determines whether to directly interact the shell with the UART driver.	YES/NO	NO	LOSCFG_DRIVER_S_UART(=y)

Development Process of Adding a Shell Command

The following uses the **ls** command as an example to describe the typical development process of adding a shell command.

1. Define the shell command processing function.

The shell command processing function is used to process registered commands. For example, define a command processing function **osShellCmdLs** to process the **ls** command, and declare the command processing function prototype in the header file.

```
int osShellCmdLs(int argc, const char **argv);
```

NOTICE

The parameters of the command processing function are similar to those of the **main** function in the C language, including the following two input parameters:

- **argc**: Number of parameters in the shell command. Whether the command keyword is included in the parameter number depends on the command type when the command is registered.
- **argv**: Pointer array. Each element points to a character string, which is the parameter transferred to the command processing function during the shell command execution. Whether the command keyword is included in the parameter value depends on the command type when the command is registered.

2. Register a command.

The command can be registered statically or dynamically when the OS is running.

- Static registration

```
#include "shcmd.h"
SHELLCMD_ENTRY(ls_shellcmd, CMD_TYPE_EX, "ls", XARGS, (CMD_CBK_FUNC)osShellCmdLs);
```

- Dynamic registration

```
#include "shell.h"
osCmdReg(CMD_TYPE_EX, "ls", XARGS, (CMD_CBK_FUNC)osShellCmdLs);
```

3. For static registration command mode, set the link option (LITEOS_TABLES_LDFLAGS variable) in **build/mk/liteos_tables_ldflags.mk**.
4. Open the menu to enable the shell. For details, see [Configuration Items](#).
5. After compiling and burning the system, you can run the added shell command.

9.2.3 Precautions

- English input is allowed in the default mode. If you enter Chinese characters in UTF8 format, you can delete them only by pressing the **Backspace** key for three times.
- You can press the **Tab** key to let the system automatically complete the shell command, file name, and directory you want to enter. If 2 to 24 matches are found, the system completes the characters shared by these matches and displays the matches for you to choose from. If more than 24 matches are found, the system displays "**Display all num possibilities? (y/n)**", prompting for your confirmation. Pressing **N** will exit the type-ahead input function. Pressing **Y** will display the first 24 matches and then the "**--More--**" prompt message. You can press **Enter** to display the rest matches, or **Q** or **Ctrl+C** to exit the type-ahead input function.
- Manipulating device files under the **/dev** directory using shell commands is not recommended because it may cause unpredictable results.
- Shell is a commissioning function and is disabled by default. It is forbidden to use Shell in commercial products.
- Disclaimer: Huawei shares no responsibilities for any risks caused in the use of Shell in commercial products.

9.2.4 Static Registration Programming Example

Description

This example shows how to add a shell command named **test** through static registration. The compilation tool **Make** is used as an example.

1. Define the command processing function **cmd_test** that will be called to add the new command.
2. Call the **SHELLCMD_ENTRY** function to add the new command.
3. Add the parameter of the new command to the link option **liteos_tables_ldflags.mk**.
4. Open the menu to enable the shell.
5. Recompile code and run Huawei LiteOS.

Example Code

1. Define the **cmd_test** function.

```
#include "shell.h"
#include "shcmd.h"

int cmd_test(void)
{
    printf("hello everybody!\n");
}
```



```
    return 0;
}
```

2. Add the new command.
SHELLCMD_ENTRY(test_shellcmd, CMD_TYPE_EX, "test", 0, (CMD_CBK_FUNC)cmd_test);
3. Add the command parameter **l** to the linker options.
Add **-utest_shellcmd** to **LITEOS_TABLES_LDFLAGS** in **build/mk/liteos_tables_ldflags.mk**.
4. Open the menu to enable the shell, that is, set **LOSCFG_SHELL** to **y**.
5. Recompile the commands.
make clean;make

Result Verification

Burn a new system image and restart the system. Run the **help** command to view all the registered commands in the current system. You can find that the **test** command is registered.

9.2.5 Dynamic Registration Programming Example

Description

This example shows how to add a shell command named **test** through dynamic registration. The compilation tool **Make** is used as an example.

1. Define the command processing function **cmd_test** that will be called to add the new command.
2. Call the **osCmdReg** function to add the new command.
3. Open the menu to enable the shell.
4. Recompile and run.

Example Code

1. Define the command processing function **cmd_test** that will be called to add the new command.

```
#include "shell.h"
#include "shcmd.h"

int cmd_test(void)
{
    printf("hello everybody!\n");
    return 0;
}
```
2. Call the **osCmdReg** function in the **app_init** function for dynamical registration.

```
void app_init(void)
{
    ....
    ....
    osCmdReg(CMD_TYPE_EX, "test", 0,(CMD_CBK_FUNC)cmd_test);
    ....
}
```
3. Open the menu to enable the shell, that is, set **LOSCFG_SHELL** to **y**.
4. Recompile the code.
make clean;make

Result Verification

Burn a new system image and restart the system. Run the **help** command to view all the registered commands in the current system. You can find that the **test** command is registered.

9.2.6 System Commands

9.2.6.1 Enabling System Commands

Before using system commands in the shell, you need to run the **menuconfig** command to enable the shell. For details, see [Configuration Items](#).

9.2.6.2 help

Function

The **help** command is used to view all shell commands in Huawei LiteOS.

Format

help

Example

Enter **help**.

Output

Run the **help** command to display all shell commands in the system.

```
Huawei LiteOS # help
*****shell commands:*****
cpup      date      dlock     dmesg     free      help      hwi
log        memcheck  mutex     queue     sem       stack     swtmr
systeminfo task      uname     watch
```

9.2.6.3 date

Function

The **date** command is used to query and set the system time.

Format

date
date --help
date +Format
date -s YY/MM/DD
date -s hh:mm:ss

`date -r Filename` (The open-source version does not support this command.)

Parameter Description

Table 9-1 Parameter Description

Parameter	Description	Value Range
<code>--help</code>	Get help info.	N/A
<code>+Format</code>	Print the current time in the configured format.	Placeholders listed using <code>--help</code>
<code>-s YY/MM/DD</code>	Set the system time. The value is in YY/MM/DD format. The year, month, and date are separated by slashes (/).	$\geq 1970/01/01$
<code>-s hh:mm:ss</code>	Set the system time. The value is in hh:mm:ss format. The hour, minute, and second are separated by colons (:).	N/A
<code>-r Filename</code>	Query the last modification time of a specified file. LOSCFG_FS_VFS needs to be enabled.	N/A

User Guide

- This command depends on LOSCFG_SHELL_EXTENDED_CMDS. You can run the **menuconfig** command to enable the macro switch in the menu item **Enable Shell Ext CMDs**.
Debug ---> Enable a Debug Version ---> Enable Shell ---> Enable Shell Ext CMDs
- If date is not specified, the current system time is displayed by default.
- The `--help`, `+Format`, `-s`, and `-r` commands cannot be used together.

Example

Enter **`date +%Y--%m--%d`**.

Output

Run the **`date +%Y--%m--%d`** command to display the current system time in the specified format.

```
Huawei LiteOS # date +%Y--%m--%d
2021--01--20
```

9.2.6.4 uname

Function

The **uname** command is used to display the current OS name, time of OS building, and version of Huawei LiteOS.

Format

uname [-a | -s | -t | -v | --help]

Parameter Description

Parameter	Description
-a	Display all information.
-s	OS name
-t	Displays the OS building time.
-v	Version
--help	Prompt of uname command format

User Guide

- If this parameter is not specified, the OS name is displayed by default.
- The **uname** parameters cannot be used together.

Example

Enter **uname -a**.

Output

Run the **uname -a** command to obtain the system information.

```
Huawei LiteOS # uname -a
Huawei LiteOS V200R002C00SPC050B011 Huawei LiteOS 3.2.2 Mar 30 2019 16:09:28
```

9.2.6.5 task

Function

The **task** command is used to query the information about tasks in Huawei LiteOS.

Format

task [/D]

Parameter Description

Table 9-2 Parameter description

Parameter	Description	Value Range
ID	Task ID. The value can be expressed in decimal or hexadecimal format.	[0, 0xFFFFFFFF]

User Guide

- If the parameter is left unspecified, all task information will be printed by default.
- If an ID added to the end of the **task** command is in the range of [0, 64], the task name, task ID, and call stack information (a maximum of 15 call stacks are supported) of the task with the specified ID are returned. If the task ID is not in the range of [0, 64], a parameter error message is returned. If the task corresponding to the specified ID is not created, the system displays a message.
- If a task is in the **Invalid** state in the **task** command output, perform either of the following operations on **pthread_create**. Otherwise, resources cannot be properly reclaimed.
 - Call the **pthread_join()** function in block mode.
 - Call the **pthread_detach()** function in non-block mode.
 - If you do not want to call the preceding two functions, set **pthread_attr_t** to **PTHREAD_STATE_DETACHED** and transfer the **attr** parameter to **pthread_create**. These operations are performed in non-block mode as well.

Examples

Example 1: Enter **task 6**.

Example 2: Enter **task**.

Output

Run the **task 0xb** command to query the information about the task whose ID is **b**.

```
Huawei LiteOS # task 0xb
TaskName = SerialEntryTask
TaskId = 0xb
*****backtrace begin*****
traceback 0 -- lr = 0x1d804   fp = 0xa86bc
traceback 1 -- lr = 0x1da40   fp = 0xa86e4
traceback 2 -- lr = 0x20154   fp = 0xa86fc
traceback 3 -- lr = 0x258e4   fp = 0xa8714
traceback 4 -- lr = 0x242f4   fp = 0xa872c
traceback 5 -- lr = 0x123e4   fp = 0xa8754
traceback 6 -- lr = 0x2a9d8   fp = 0xb0b0b0b
```

Run the **task** command to query information about all tasks:

Huawei LiteOS # task										
Name	TaskEntryAddr	TID	Priority	Status	StackSize	WaterLine	StackPoint	TopOfStack		
EventMask	SemID	CPUUSE	CPUUSE10s	CPUUSE1s	MEMUSE					

Swt_Task	0x40002770	0x0	0	QueuePend	0x6000	0x2cc	0x4015a318	0x401544e8		
0x0	0xffff	0.0	0.0	0.0	0					
IdleCore000	0x40002dc8	0x1	31	Ready	0x400	0x15c	0x4015a7f4	0x4015a550	0x0	
0xffff	98.6	98.2	99.9	0						
system_wq	0x400b80fc	0x3	1	Pend	0x6000	0x244	0x40166928	0x40160ab8	0x1	
0xffff	0.0	0.0	0.0	0						
SerialShellTask	0x40090158	0x5	9	Running	0x3000	0x55c	0x40174918	0x40171e70		
0xfff	0xffff	1.2	1.7	0.0	48					
SerialEntryTask	0x4008fe30	0x6	9	Pend	0x1000	0x2c4	0x40175c78	0x40174e88	0x1	
0xffff	0.0	0.0	0.0	72						

 NOTE

1. Description of output items

- **Name**: Task name.
- **TID**: Task ID.
- **Priority**: Priority of a task.
- **Status**: Status of the current task.
- **StackSize**: Size of a task stack.
- **WaterLine**: Occupied memory of a task stack.
- **StackPoint**: Start address of a task stack.
- **TopOfStack**: Address of stack top.
- **EventMask**: Event mask of current task, default even mask of task that is not used is 0. (If there are many events used in task, the recently used one will be displayed).
- **SemID**: Semaphore ID that current task owned, default semaphore ID that is not used is 0xFFFF. (If there are many semaphores used in task, the recently used one will be displayed).
- **CPUUSE**: CPU usage since startup.
- **CPUUSE10s**: CPU usage in the last 10 seconds.
- **CPUUSE1s**: CPU usage in the last second.
- **MEMUSE**: Size of memory that is allocated till now, with unit byte. The counting of **MEMUSE** refers to system memory pool. The memory used by operations that take place during an interrupt and any memory processed before task scheduling starts will not be counted.

If memory is allocated to tasks, the **MEMUSE** value increases. If tasks release memory, the **MEMUSE** value decreases. The value of **MEMUSE** can be positive or negative.

(1) If no memory is allocated to tasks, or the allocated memory equals the released memory, the **MEMUSE** value is 0.

(2) If the **MEMUSE** value is positive, some memory is not released by the task.

(3) If the **MEMUSE** value is negative, the released memory is larger than the allocated memory.

2. Description of initial system tasks in Huawei LiteOS

- **SwT_Task**: Software timer task, which is used to process the timeout callback function for software timers.
- **IdleCore000**: Task that is executed when Huawei LiteOS is idle.
- **system_wq**: Default work queue for processing tasks.
- **SerialEntryTask**: Reads user input from the lower-layer buf and preliminarily parses a command, such as arrow keys and command completion by pressing **Tab**.
- **SerialShellTask**: Further parses the command sent by cmdParesTask and calls the registration function that matches the command.

3. Description of task status

- **Ready**: The task is ready.
- **Pend**: The task is pending.
- **PendTime**: The pending task timed out.
- **Suspend**: The task is suspended.
- **Running**: The task is running.
- **Delay**: The task is delayed.
- **SuspendTime**: The suspended task timed out.

- **Invalid:** The task is not in any of the preceding states.

9.2.6.6 free

Function

The **free** command displays the usage of memory in Huawei LiteOS and the sizes of the text segment, data segment, rodata segment, and bss segment.

Format

`free [-k | -m]`

Parameter Description

Table 9-3 Parameter description

Parameter	Description	Value Range
No parameters	In the unit of byte	N/A
-k	In the unit of KB	N/A
-m	In the unit of MB	N/A

User Guide

- Enter **free** to display the memory usage. **total** indicates the total size of the dynamic memory pool, **used** indicates the size of the used memory, and **free** indicates the size of the free memory. **text** indicates the size of the code segment, **data** indicates the size of the data segment, **rodata** indicates the size of the read-only data segment, and **bss** indicates the size of the memory occupied by uninitialized global variables.
- The **free** command can be used to display the memory usage in three units: byte, KB, and MB.

Example

Enter **free**, **free -k**, and **free -m**.

Output

Display the memory usage in three units

```
Huawei LiteOS # free
      total      used      free
Mem: 117631744 31826864 85804880

      text      data      rodata      bss
Mem: 4116480 423656 1204224 6659316

Huawei LiteOS # free -k
```



```

      total      used      free
Mem: 114874    31080    83793

      text      data      rodata      bss
Mem:  4020      423      1176      6503

Huawei LiteOS # free -m

      total      used      free
Mem:    112         30         81

      text      data      rodata      bss
Mem:     3         0         1         6

```

9.2.6.7 memcheck

Function

The **memcheck** command is used to check whether the dynamically allocated memory is complete and whether a node is damaged due to memory overwriting.

Format

memcheck

User Guide

- Enable the memory integrity check function.
 - Currently, only the **bestfit** memory management algorithm supports this command. You need to enable **LOSCFG_KERNEL_MEM_BESTFIT**.
Kernel ---> Memory Management ---> Dynamic Memory Management Algorithm ---> bestfit
 - This command depends on **LOSCFG_BASE_MEM_NODE_INTEGRITY_CHECK**. Before running this command, enable the **Enable integrity check or not** menu item as follows:
Debug ---> Enable a Debug Version ---> Enable MEM Debug ---> Enable integrity check or not
- If memory leak does not occur, the output of memcheck is "memcheck over, all passed! "
- If nodes are not completed memory pool, the output is the information about the memory of the node that destroyed.

Example

Enter **memcheck**.

Output

- If no memory overwriting occurs, the output of the **memcheck** command is as follows.
Huawei LiteOS # memcheck
system memcheck over, all passed!
- If memory overwriting occurs, the output of the **memcheck** command is as follows.
Huawei LiteOS # memcheck
[ERR][OsMemIntegrityCheck], 1145, memory check error!
freeNodeInfo.pstPrev:(nil) is out of legal mem range[0x80353540, 0x84000000]

```

broken node head: (nil) (nil) (nil) 0x0, pre node head: 0x7fc6a31b 0x8 0x80395ccc 0x80000110
-----
dump mem tmpNode:0x80395df4 ~ 0x80395e34

0x80395df4 :00000000 00000000 00000000 00000000
0x80395e04 :cacacaca cacacaca cacacaca cacacaca
0x80395e14 :cacacaca cacacaca cacacaca cacacaca
0x80395e24 :cacacaca cacacaca cacacaca cacacaca
-----

dump mem :0x80395db4 ~ tmpNode:0x80395df4

0x80395db4 :00000000 00000000 00000000 00000000
0x80395dc4 :00000000 00000000 00000000 00000000
0x80395dd4 :00000000 00000000 00000000 00000000
0x80395de4 :00000000 00000000 00000000 00000000
-----

cur node: 0x80395df4
pre node: 0x80395ce4
pre node was allocated by task:SerialShellTask
cpu0 is in exc.
cpu1 is running.
excType:software interrupt
taskName = SerialShellTask
taskId = 8
task stackSize = 12288
system mem addr = 0x80353540
excBuffAddr pc = 0x80210b78
excBuffAddr lr = 0x80210b7c
excBuffAddr sp = 0x803b2d50
excBuffAddr fp = 0x80280368
R0      = 0x59
R1      = 0x600101d3
R2      = 0x0
R3      = 0x8027a300
R4      = 0x1
R5      = 0xa0010113
R6      = 0x80395e04
R7      = 0x80317254
R8      = 0x803b2de4
R9      = 0x4
R10     = 0x803afca4
R11     = 0x80280368
R12     = 0x1
CPSR    = 0x600101d3

```

The key output items are described as follows:

- "mem check error" indicates that the memory node is damaged.
- "cur node" indicates the node where the memory is illegally overwritten and the memory address is printed.
- "pre node" indicates the node preceding the node where the memory is illegally overwritten, and the node address is printed.
- "pre node was allocated by task" indicates that memory overwriting occurs in **SerialShellTask**.
- "taskName" and "taskId" indicate the name and ID of an abnormal task.

9.2.6.8 memused

Function

The **memused** command is used to view the function call stack LR information saved in the current system used node. Memory leaks can be detected by analyzing data.

Format

memused

User Guide

Run the **menuconfig** command to enable memory leak detection.

- Currently, only the **bestfit** memory management algorithm supports this function. You need to enable **LOSCFG_KERNEL_MEM_BESTFIT**.

Kernel ---> Memory Management ---> Dynamic Memory Management Algorithm ---> bestfit

- This command depends on **LOSCFG_MEM_LEAKCHECK**. You can enable the **Enable Function call stack of Mem operation recorded** menu item.

Debug ---> Enable a Debug Version ---> Enable MEM Debug ---> Enable Function call stack of Mem operation recorded

While opening this configuration, the function call relationship LR is recorded in the memory node control block when the memory operation is performed. If the memory node of the same call stack grows with time, there is a possibility of leakage. The location of the memory application can be traced through the LR. Focus on LR duplicate nodes.

NOTICE

Memory usage and memory operation performance will be affected when this configuration is turned on, so it is only turned on when detecting memory leaks.

Example

Enter **memused**.

Output

```
Huawei LiteOS # memused
node      LR[0]    LR[1]    LR[2]
0x802d79e4: 0x8011d740 0x8011a990 0x00000000
0x802daa0c: 0x8011d5ec 0x8011d740 0x8011a990
0x802dca28: 0x8006e6f4 0x8006e824 0x8011d5ec
0x802e2a48: 0x8014daac 0x8014db4c 0x800f6da0
0x802e8a68: 0x8011d274 0x8011d654 0x8011d740
0x802e8a94: 0x8014daac 0x8014db4c 0x8011d494
0x802eeab4: 0x8011d4e0 0x8011d658 0x8011d740
0x802f4ad4: 0x8015326c 0x80152a20 0x800702bc
0x802f4b48: 0x8015326c 0x80152a20 0x800702bc
0x802f4bac: 0x801504d8 0x801505d8 0x80150834
0x802f4c08: 0x801504d8 0x801505d8 0x80150834
0x802f4c74: 0x801504d8 0x801505d8 0x80150834
0x802f4e08: 0x801504d8 0x801505d8 0x80150834
```

```
0x802f4e60: 0x801030e8 0x801504d8 0x801505d8
0x802f4e88: 0x80103114 0x801504d8 0x801505d8
0x802f4eb4: 0x801504d8 0x801505d8 0x80150834
0x802f4f7c: 0x801504d8 0x801505d8 0x80150834
0x802f5044: 0x801504d8 0x801505d8 0x80150834
0x802f510c: 0x800702bc 0x80118f24 0x00000000
```

9.2.6.9 hwi

Function

The **hwi** command is used to query the information about current interruptions.

Format

hwi

hwi status all

hwi status hwiNum [devId]

Parameter Description

Table 9-4 Parameter description

Parameter	Description	Value Range
status all	Prints the status of all created interrupts.	None.
hwiNum	Indicates the interrupt ID. The value is a decimal number.	[0,0xFFFFFFFF]
devId	Indicates the ID of the shared interrupt device. The value is a hexadecimal number. The default value indicates that the queried interrupt is a non-shared interrupt.	[0,0xFFFFFFFF]

User Guide

- Enable the **hwi** command and enable the macro switch **LOSCFG_DEBUG_HWI** in the menu.
Debug ---> Enable a Debug Version ---> Enable Debug LiteOS Kernel Resource ---> Enable Hwi Debugging
- Enter **hwi** to display the interruption number, the number of occurrences, and the name of each current interruption.
- Enter **hwi status all** to display the status of all created interrupts.
- Enter **hwi status hwiNum [devId]** to query the status of a specific interrupt. For a shared interrupt, the **devId** parameter is required. For a non-shared interrupt, the **devId** parameter is not required.
- If **LOSCFG_CPUP_INCLUDE_IRQ** is enabled, the **hwi** command also displays the processing time (cycles), CPU usage, and interruption type of each

interruption. You can enable the **Enable Cpu include irq** menu item to enable the macro.

Kernel ----> Enable Extend Kernel ----> Enable Cpu ----> Enable Cpu include irq

Example

Enter **hwi**.

Enter **hwi status hwiNum devId**.

Enter **hwi status hwiNum**.

Enter **hwi status all**.

Output

1. Enter **hwi** to display the interrupt information (**LOSCFG_CPUP_INCLUDE_IRQ** is disabled).

```
Huawei LiteOS # hwi
InterruptNo  Count  Name
35:         1364:
36:          0:
40:          79:  uart_pl011
```

2. Enter **hwi** to display the interrupt information (**LOSCFG_CPUP_INCLUDE_IRQ** is enabled).

```
Huawei LiteOS # hwi
InterruptNo Count  Name  CYCLECOST CPUUSE CPUUSE10s CPUUSE1s mode
3: 1333      122   0.0  0.0    0.0  normal
4: 0         0    0.0  0.0    0.0  normal
5: 59  uart_pl011 305   0.0  0.0    0.0  normal
12: 96  ETH      131   0.0  0.0    0.0  normal
```

3. Enter **hwi status 3 0x1** to display the status of the shared interrupt whose interrupt ID is 3 and device ID is 0x1.

```
Huawei LiteOS # hwi status 3 0x1
InterruptNo  DevId  Priority  Affinity  Created  Enable  Pending
-----
3           0x1    20       0x0       0        1       0
```

4. Enter **hwi status 3** to display the status of non-shared interrupt whose interrupt ID is 3.

```
Huawei LiteOS # hwi status 3
InterruptNo  DevId  Priority  Affinity  Created  Enable  Pending
-----
3           0x0    20       0x0       0        1       0
```

5. Enter **hwi status all** to display the status of all created interrupts.

```
Huawei LiteOS # hwi status all
InterruptNo  DevId  Priority  Affinity  Created  Enable  Pending
-----
0           0x0    20       0x0       1        1       0
1           0x0    20       0x0       1        1       0
2           0x0    20       0x0       1        1       0
30          0x0    20       0x0       1        1       0
33          0x0     0       0x1       1        1       0
```

9.2.6.10 queue

Function

The **queue** command is used to view the queue usage.

Format

queue

User Guide

This command depends on **LOSCFG_DEBUG_QUEUE**. You can enable the **Enable Queue Debugging** menu item.

Debug ---> Enable a Debug Version ---> Enable Debug LiteOS Kernel Resource ---> Enable Queue Debugging

Example

For example:

Enter **queue**.

Output

After the **queue** command is executed, the queue usage is obtained.

```
Huawei LiteOS # queue
used queues information:
Queue ID <0x0> may leak, queue len is 0x10, readable cnt:0x0, writeable cnt:0x10, TaskEntry of creator:
0x0x80007d5, Latest operation time: 0x614271
```

The output items are described as follows:

Parameter	Description
Queue ID	A queue ID
queue len	Number of queue message nodes
readable cnt	Number of readable messages in the queue
writeable cnt	Number of writable messages in the queue
TaskEntry of creator	Interface IP address for creating a queue
Latest operation time	Last operation time of a queue

9.2.6.11 sem

Function

The **sem** command is used to query the information about semaphores of Huawei LiteOS Kernel.

Format

sem [/D]

sem fulldata

Parameter Description

Table 9-5 Parameter description

Parameter	Description	Value Range
ID	Semaphore ID. The value can be expressed in decimal or hexadecimal format.	[0, 0xFFFFFFFF]
fulldata	Print all semaphore information in use: SemID, Count, Original Count, Creator TaskEntry, Last Access Time	N/A

User Guide

- Parameter displays the usage number and total number of semaphore.
- If an ID is added, when the **ID** value falls in the range [0,1023], the usage number of the specified semaphore will be displayed. (A prompt will be displayed if the semaphore specified by the ID is not in use.) A prompt indicating a parameter error will be displayed if the parameter value is outside the range [0,1023].
- The parameter ID and fulldata cannot be used together.
- The **fulldata** parameter depends on **LOSCFG_DEBUG_SEMAPHORE**. You can enable the **Enable Semaphore Debugging** menu item.
Debug ---> Enable a Debug Version ---> Enable Debug LiteOS Kernel Resource ---> Enable Semaphore Debugging

Examples

Example 1: enter **sem 1**.

Example 2: enter **sem** and **sem fulldata**.

Output

Run **sem 1** to query the command output.

```
Huawei LiteOS # sem 1
SemID    Count
-----
0x00000001  0x1
No task is pended on this semaphore!
```

Run the **sem** and **sem fulldata** commands to query information about all semaphores in use.

```
Huawei LiteOS # sem
SemID    Count
-----
0x00000000  1
SemID    Count
-----
0x00000001  1
SemID    Count
-----
```

```
0x00000002 1
SemUsingNum : 3

Huawei LiteOS # sem fulldata
Used Semaphore List:
SemID Count OriginalCout Creator(TaskEntry) LastAccessTime
-----
0x2 0x1 0x1 0x80164d70 0x3
0x0 0x1 0x1 0x0 0x3
0x1 0x1 0x1 0x0 0x3
```

9.2.6.12 mutex

Function

The **mutex** command is used to view the mutex usage.

Format

mutex

User Guide

This command depends on **LOSCFG_DEBUG_MUTEX**. You can enable the **Enable Mutex Debugging** menu item.

Debug ---> Enable a Debug Version ---> Enable Debug LiteOS Kernel Resource ---> Enable Mutex Debugging

Example

Enter **mutex**.

Output

Run the **mutex** command to display the mutex usage.

```
Huawei LiteOS # mutex
used mutexs information:
Mutex ID <0x0> may leak, Owner is null, TaskEntry of creator: 0x8000711, Latest operation time: 0x0
```

The main output items are described as follows:

Output Item	Description
Mutex ID	Mutex ID
TaskEntry of creator	Interface IP address for creating a mutex
Latest operation time	Last scheduling time of a task

9.2.6.13 swtmr

Function

The **swtmr** command is used to query the information about system software timers.

Format

swtmr [*ID*]

Parameter Description

Table 9-6 Parameter description

Parameter	Description	Value Range
ID	ID of a software timer. The value can be expressed in decimal or hexadecimal format.	[0,0xFFFFFFFF]

User Guide

- Enable the **swtmr** command and enable the macro switch **LOSCFG_DEBUG_SWTMR** in the menu.
Debug ---> Enable a Debug Version ---> Enable Debug LiteOS Kernel Resource ---> Enable Swtmr Debugging
- If the parameter is left unspecified, information about all software timers will be displayed.
- If an ID is added after the **swtmr** command, when the **ID** value falls in the range [0,current number of software timers –1], the status of the software timer specified by the ID is returned. A prompt indicating a parameter error will be displayed if the parameter value is outside the range.

Example

Enter **swtmr** and **swtmr 0**.

Output

Run the **swtmr** command to query information about the software timer. The command output is as follows:

```
Huawei LiteOS # swtmr
SwTmrID   State   Mode   Interval Arg      handlerAddr
-----
0x00000000 Ticking Period 1000 0x00000000 0x800442d
```

Run the **swtmr 0** command to query information about the software timer whose ID is **0**. The command output is as follows:

```
Huawei LiteOS # swtmr 0
```

SwTmrID	State	Mode	Interval	Arg	handlerAddr
0x00000000	Ticking	Period	1000	0x00000000	0x800442d

Table 9-7 Output

Parameter	Description
SwTmrID	ID of a software timer
State	State of a software timer
Mode	Mode of a software timer
Interval	Number of ticks used by a software timer
Arg	Number of input parameters
handlerAddr	Address of a callback function

9.2.6.14 systeminfo

Function

The **systeminfo** command is used to view the usage of resources including tasks, semaphores, mutexes, queues, and timers in Huawei LiteOS.

If the semaphore, mutex, queue, and software timer are tailored in the system image, the system does not use these resources. In this case, this command does not display the information about these resources.

Format

systeminfo

Example

Enter **systeminfo**.

Output

Run the **systeminfo** command to check the system resource usage. The following information is displayed:

Huawei LiteOS # systeminfo		
Module	Used	Total
Task	5	16
Sem	0	20
Mutex	1	20
Queue	1	10
SwTmr	1	16

The output items are described as follows:

Output Item	Description
Module	Module name
Used	Number of used resources
Total	Maximum number of usable resources

9.2.6.15 log

Function

The **log** command is used to query & modify log print level.

Format

log level [*levelNum*]

Parameter Description

Table 9-8 Parameter Description

Parameter	Description	Value Range
levelNum	config log print level	[0x0,0x5]

NOTICE

The help information of the log command also displays the module and path parameters. Currently, the two parameters are not supported.

User Guide

- This command depends on **LOSCFG_SHELL_LK**. You can enable **Enable Shell lk**.
Debug ---> Enable a Debug Version ---> Enable Shell ---> Enable Shell lk
- log level** command is used to config log print level which includes the following 6 levels:
TRACE_EMG = 0
TRACE_COMMOM = 1
TRACE_ERROR = 2
TRACE_WARN = 3
TRACE_INFO = 4
TRACE_DEBUG = 5
- If the log level command does not contain the **levelNum** parameter, the system displays the command usage.

Example

Enter log level 4.

Output

Run the **log level 4** command to set the log level to **INFO**.

```
Huawei LiteOS # log level 4
Set current log level INFO
```

9.2.6.16 dmesg

Function

The **dmesg** command is used to control the kernel dmesg buffer.

Format

```
dmesg
dmesg [-c / -C / -D / -E / -L / -U]
dmesg -s <size>
dmesg -l <level>
dmesg > file (The open-source version does not support this command.)
```

Parameter Description

Table 9-9 Parameter Description

Parameter	Description	Value Range
-c	Prints the contents in the buffer and clears the buffer. The printing is not affected by -D and -L .	N/A
-C	Clean the dmesg buffer.	N/A
-D -E	Specifies whether to enable the console printing function. This parameter is not supported in the open-source version.	N/A
-L -U	Specifies whether to enable the serial port printing function. This parameter is not supported in the open-source version.	N/A
-s <size>	Sets the size of the dmesg buffer.	N/A
-l <level>	Sets the dmesg level.	0 -- 5

Parameter	Description	Value Range
> file	Writes the content in the buffer to a file. The open-source version does not support this parameter.	N/A

NOTICE

1. Before writing a file, ensure that the file system has been mounted.
2. Disabling the serial port print function affects the use of the shell. You are advised to connect to the telnet and then disable the serial port.

User Guide

- This command depends on **LOSCFG_SHELL_DMESG**. You can enable **Enable Shell dmesg**.
Debug ---> Enable a Debug Version---> Enable Shell---> Enable Shell dmesg
- If the parameter is left unspecified, logs in the dmesg buffer will be printed by default.
- The parameters cannot be used together.

Example

Enter **dmesg** first, and then enter **dmesg -C** and **dmesg**.

Output

In the above figure, in the blue box is the input command. After the **dmesg** command is executed for the first time, the buffer content is displayed. After running the **dmesg -C** command to clear the cache, run the **dmesg** command again. We can see that the cache is cleared.

```
Huawei LiteOS # dmesg
Huawei LiteOS # log level 4
Set current log level INFO
Huawei LiteOS # dmesg

Huawei LiteOS # dmesg -C
Huawei LiteOS # dmesg

Huawei LiteOS # dmesg
```

NOTICE

dmesg -C is a log maintenance and test command. It supports only serial port output and does not support output to other output ports (such as Telnet and USB) through the console port.

9.2.6.17 stack

Function

The **stack** command is used to view information about all the stacks in the current OS.

Format

stack

User Guide

This function is enabled by default.

Example

Enter **stack**.

Output

Run the **stack** command to view information about all stacks in the system.

Huawei LiteOS # stack

stack name	cpu id	stack addr	total size	used size
udf_stack	3	0x3c800	0x28	0x0
udf_stack	2	0x3c828	0x28	0x0
udf_stack	1	0x3c850	0x28	0x0
udf_stack	0	0x3c878	0x28	0x0
abt_stack	3	0x3c8a0	0x28	0x0
abt_stack	2	0x3c8c8	0x28	0x0
abt_stack	1	0x3c8f0	0x28	0x0
abt_stack	0	0x3c918	0x28	0x0
fiq_stack	3	0x3ca40	0x40	0x0
fiq_stack	2	0x3ca80	0x40	0x0
fiq_stack	1	0x3cac0	0x40	0x0
fiq_stack	0	0x3cb00	0x40	0x0
svc_stack	3	0x3cb40	0x2000	0x524
svc_stack	2	0x3eb40	0x2000	0x524
svc_stack	1	0x40b40	0x2000	0x524
svc_stack	0	0x42b40	0x2000	0x528
irq_stack	3	0x3c940	0x40	0x0
irq_stack	2	0x3c980	0x40	0x0
irq_stack	1	0x3c9c0	0x40	0x0
irq_stack	0	0x3ca00	0x40	0x0
exc_stack	3	0x44b40	0x1000	0x0
exc_stack	2	0x45b40	0x1000	0x0
exc_stack	1	0x46b40	0x1000	0x0
exc_stack	0	0x47b40	0x1000	0x0

NOTICE

The **stack** command is a maintenance and test command applicable to exception scenarios. It supports only serial port output and does not support console output to other output ports (such as Telnet and USB).

9.2.6.18 cpup

Function

The **cpup** command is used to query the CPU usage in a percentage.

Format

cpup [*mode*] [*taskID*]

Parameter Description

Table 9-10 Parameter description

Parameter	Description	Value Range
mode	- Default: displays the CPU usage in the last 10 seconds. 0: displays the CPU usage in the last 10 seconds. 1: displays the CPU usage in the last second. - Other value: displays the CPU usage since Huawei LiteOS is started.	[0,0xFFFF] or 0xFFFFFFFF
taskID	Task ID	[0,0xFFFF] or 0xFFFFFFFF

User Guide

- By default, the system displays the CPU usage in the latest 10 seconds.
- If parameter is only one, and the parameter is mode, the CPU usage percent of system corresponding time ago will be displayed.
- If two parameters are passed in, and the first one is mode, the second one is taskID. The CPU usage percent of system with the specific task ID corresponding time ago will be displayed.
- To enable this function, you need to enable the CPU usage module. The menu path is as follows:
Kernel ---> Enable Extend Kernel ---> Enable Cpu
For more information about the CPU usage module, see [CPUP](#).

Example

Enter **cpup 1 1**.

Output

Run the **cpup 1 1** command to display the CPU usage of the task whose ID is 1 in the last 1s.

```
Huawei LiteOS # cpup 1 1
TaskId 1 CpuUsage in 1s: 78.7
```

9.2.6.19 watch

Function

The **watch** command is used to periodically monitor the command output.

Format

```
watch [-c / -n / -t / --count / --interval / -no-title] <command>
watch --over
```

Parameter Description

Table 9-11 Parameter description

Parameter	Description	Default Value	Value Range
-c --count	Total number of command executions	0xFFFFFFFF	(0,0xFFFFFFFF]
-n --interval	Time interval of executing commands, in seconds	1s	(0,0xFFFFFFFF]
-t -no-title	Close the time display on the top.	N/A	N/A
command	Shell commands to be listened on.	N/A	N/A
--over	Disable the current monitor.	N/A	N/A

User Guide

- This command depends on LOSCFG_SHELL_EXTENDED_CMDDS. You can enable the **Enable Shell Ext CMDs** menu item.
Debug ---> Enable a Debug Version ---> Enable Shell ---> Enable Shell Ext CMDs
- The parameter **command** must be a shell command. For a non-shell command, the error message "command is not fount" will be displayed.
- The parameter **command** is mandatory if you want to listen on command.
- The parameter **--over** cannot be used together with other parameters.

Examples

- Example 1: Enter **watch -c 5 task 1**.
- Example 2: If the **watch** command is not required for listening, run the **watch --over** command.

Output

The **task 1** command is executed for five times at an interval of 1 second. The following information is displayed after the **watch** command is executed:

```
Huawei LiteOS # watch -c 3 task 1

Huawei LiteOS # Thu Jan  1 16:26:26 1970

TaskName = Swt_Task
TaskId = 0x1
*****backtrace begin*****
traceback 1 -- lr = 0x08004006 -- fp = 0x0800045e
traceback 2 -- lr = 0x08004000 -- fp = 0x0800194c
traceback 3 -- lr = 0x080040da -- fp = 0x08003e50
traceback 4 -- lr = 0x080015c2 -- fp = 0x080040a8
traceback 5 -- lr = 0x0800396e -- fp = 0x08001598

Thu Jan  1 16:26:27 1970

TaskName = Swt_Task
TaskId = 0x1
*****backtrace begin*****
traceback 1 -- lr = 0x08004006 -- fp = 0x0800045e
traceback 2 -- lr = 0x08004000 -- fp = 0x0800194c
traceback 3 -- lr = 0x080040da -- fp = 0x08003e50
traceback 4 -- lr = 0x080015c2 -- fp = 0x080040a8
traceback 5 -- lr = 0x0800396e -- fp = 0x08001598

Thu Jan  1 16:26:28 1970

TaskName = Swt_Task
TaskId = 0x1
*****backtrace begin*****
traceback 1 -- lr = 0x08004006 -- fp = 0x0800045e
traceback 2 -- lr = 0x08004000 -- fp = 0x0800194c
traceback 3 -- lr = 0x080040da -- fp = 0x08003e50
traceback 4 -- lr = 0x080015c2 -- fp = 0x080040a8
traceback 5 -- lr = 0x0800396e -- fp = 0x08001598
```

9.2.6.20 schedstatstart

Function

The **schedstatstart** command is used to enable the scheduling statistics function.

Format

```
schedstatstart
```

User Guide

- Currently, the scheduling statistics function only performs in platforms with floating-point arithmetic, and this command depends on **LOSCFG_DEBUG_SCHED_STATISTICS**. You can enable the macro switch in the menu item **Enable Scheduler Statistics Debugging**.
Debug > Enable a Debug Version > Enable DebugLiteOS Kernel Resource > Enable Scheduler Statistics Debugging.
- schedstatstart**, **schedstatstop**, and **schedstatinfo** commands are used together.

Example

Enter **schedstatstart**.

Output

For details, see the [Output](#).

9.2.6.21 schedstatstop

Function

The **schedstatstop** command is used to disable the scheduling statistics function and export the scheduling statistics information of each CPU.

Format

schedstatstop

User Guide

See [User Guide](#).

Example

Enter **schedstatstop**.

Output

For details, see the [Output](#).

9.2.6.22 schedstatinfo

Function

The **schedstatinfo** command is used to export the CPU scheduling statistics of each task.

Format

schedstatinfo

User Guide

See [User Guide](#).

Example

For example:

Enter **schedstatinfo**.

Output

For details, see the [Output](#).

9.2.7 File Commands

9.2.7.1 Command for Enabling the File System

Before using system commands in the shell, you need to run the **menuconfig** command to enable the shell. For details, see [Configuration Items](#).

9.2.7.2 cd

Function

The **cd** command is used to change the current directory.

Format

`cd [path]`

Parameter Description

Table 9-12 Parameter description

Parameter	Description	Value Range
path	File path	You must have the execution (search) permission of the specified directory.

User Guide

- If the directory parameter is not configured, the **cd** command will jump to the root directory.
- If a complete file path is configured, it will jump to the file path.
- A complete file path starts with a slash (/), which indicates the root directory.
- One point (.) indicates the current directory.
- Two points (..) indicates the parent directory.

Example

For example:

cd..

Output

Figure 9-1 The command output is as follows:

```
Huawei LiteOS# cd ..  
  
Huawei LiteOS# ls  
Directory /:  
  
bin          <DIR>  
dev          <DIR>  
ramfs        <DIR>  
yaffs0       <DIR>
```

9.2.7.3 pwd

Function

The **pwd** command is used to display the current path.

Format

pwd

User Guide

The **pwd** command writes the full path name (absolute path starting from root directory) of the current directory to the standard output. All directories are separated by slash (/). The first slash indicates the root directory and the last indicates the current directory.

Example

For example,
enter **pwd**.

Output

Figure 9-2 View current path

```
Huawei LiteOS# pwd  
/bin/vs
```

9.2.7.4 mkdir

Function

The **mkdir** command is used to create a directory.

Format

mkdir <dir>

Parameter Description

Table 9-13 Parameter description

Parameter	Description	Value Range
dir	Directory to be created	N/A

User Guide

- Add a directory name to the end of the **mkdir** command to create a directory.
- Add a path name and a directory name to the end of the **mkdir** command to create a directory under the specified directory.

Example

Enter **mkdir share**.

Output

Figure 9-3 Create the **share** directory in current directory

```
Huawei LiteOS# mkdir share
Huawei LiteOS# ls
Directory /bin/vs:
share          <DIR>
sd             <DIR>
```

9.2.7.5 rmdir

Function

The **rmdir** command is used to delete a directory.

Format

rmdir <dir>

Parameter Description

Table 9-14 Parameter description

Parameter	Description	Value Range
dir	Name of the directory to be deleted. The directory must be empty and the name can be a file path.	N/A

User Guide

- The **rmdir** command only deletes a directory.
- The **rmdir** command deletes only one directory each time.
- The **rmdir** command deletes only empty directory.

Example

For example, enter **rmdir dir**.

Output

Figure 9-4 Delete the **dir** directory

```
Huawei LiteOS# ls
Directory /bin/vs:
dir          <DIR>
sd           <DIR>

Huawei LiteOS# rmdir dir

Huawei LiteOS# ls
Directory /bin/vs:
sd           <DIR>
```

9.2.7.6 rm

Function

The **rm** command is used to delete a file or a non-empty directory.

Format

rm -r <dir>

rm <file>

Parameter Description

Table 9-15 Parameter description

Parameter	Description	Value Range
-r <dir>	dir indicates the directory to be deleted. Non-empty directories and empty directories can be deleted.	N/A
file	Name of the file to be deleted	N/A

User Guide

- The **rm** command deletes only one file each time.
- The **rm -r** command deletes a non-empty directory.

NOTICE

Deleting important system resources such as **/dev** by running the **rm** command will cause unknown problems such as crashes.

Example

For example:

1. Enter **rm 1.c**.
2. Enter **rm -r dir**.

Output

Figure 9-5 The **rm** command deletes the **1.c** file.

```
Huawei LiteOS# ls
Directory /ramfs:
1.c                0

Huawei LiteOS# rm 1.c

Huawei LiteOS# ls
Directory /ramfs:
```

Figure 9-6 Delete the **dir** directory by using the **rm -r** command

```
Huawei LiteOS# ls
Directory /ramfs:
dir                <DIR>

Huawei LiteOS# rm -r dir

Huawei LiteOS# ls
Directory /ramfs:
```

9.2.7.7 touch

Function

The **touch** command is used to create a nonexistent file in the current directory.

Format

touch <filename>

Parameter Description

Table 9-16 Parameter description

Parameter	Description	Value Range
filename	Name of the file to be created.	N/A

User Guide

- The **touch** command creates a readable and writable empty file.
- The **touch** command creates only one file each time.
- If the touch command is used to create an existing file, no file will be created and the timestamp will not be updated.

NOTICE

Creating a file by running the **touch** command in an important system resource path will cause unknown problems such as crashes. For example, running the **touch uartdev-0** command in the **/dev** path will cause system crashes.

Example

For example, enter **touch file.c**.

Output

Figure 9-7 Create a file named file.c in current directory

```
Huawei LiteOS# touch file.c

Huawei LiteOS# ls
Directory /bin/vs:
file.c          0
sd              <DIR>

Huawei LiteOS#
```

9.2.7.8 cp

Function

The **cp** command is used to copy files.

Format

cp <source path> <dest path>

Parameter Description

Table 9-17 Parameter description

Parameter	Description	Value Range
source path	Source file	Currently, only files are supported. Directories are not supported.
dest path	Path to the destination file	Directories and files are supported.

User Guide

- The name of the source file cannot be the same as that of the destination file in the same path.
- The source file must exist and cannot be a directory.
- The source file path supports asterisks (*) and question marks (?). Wildcard. The asterisk (*) indicates any number of characters, and the question mark (?) indicates any single character. The destination path does not support wildcards. While the source path matches multiple files, the destination path must be a directory.
- If the destination path is a directory, the directory must exist. In this case, the destination file is named after the source file name.
- If the destination path is a file, the directory must exist. The destination file name is the file name specified in the destination path.
- Currently, multiple files (more than two files) cannot be copied concurrently. If there are more than two source path parameters, files specified by the first two parameters are copied.
- If the destination file does not exist, a destination file will be created. If the destination file already exists, it will be overwritten after the copy operation.

NOTICE

Copying important system resources will cause unknown serious problems such as crashes. For example, copying the **/dev/uartdev-0** file by running the **cp** command will cause system crashes.

Example

For example, enter **cp 100HSCAM/FILE0087.MP4**.

Output

Figure 9-8 The command output is as follows:

```
Huawei LiteOS# ls
Directory /bin/vs/sd/dcim:
100HSCAM                <DIR>

Huawei LiteOS# cp 100HSCAM/FILE0087.MP4 .

Huawei LiteOS# ls
Directory /bin/vs/sd/dcim:
100HSCAM                <DIR>
FILE0087.MP4            11332370
```

9.2.7.9 dd

Function

The **dd** command is used to collect file read and write performance statistics.

Format

```
dd file=[PATHNAME] mode=[OPS] bs=[SIZE] count=[N]
```

Parameter Description

Table 9-18 Parameter description

Parameter	Description	Value Range
PATHNAME	File name with the file path	Not empty
OPS	Type of the file operation to be tested, including read and write. The value 1 indicates read and the value 2 indicates write.	[1, 2]
SIZE	Number of block bytes read or written at a time	[0, 0xFFFFFFFF]
N	Number of read/write blocks	[0, 0xFFFFFFFF]

Usage Guide

1. The **dd** command can be executed concurrently. One **dd** command corresponds to one thread. Therefore, multi-thread read/write tests can be performed.
2. When the **dd** command is used to test the read and write operations on a large file, it takes a long time and the performance result may not be output immediately. This is a normal.
3. If you do not specify the **bs** and **count** fields, the system uses the default values **LOSCFG_DD_DEFAULT_BS 1024** and **LOSCFG_DD_DEFAULT_CNT 1**.
4. The size of the **bs** field cannot be too large. The system allocates the dynamic memory of the size. If the size is too large, the memory allocation fails. As a result, the **dd** command cannot be executed.
5. In the **dd** process, only the open, read/write, and close operations are performed. You need to ensure that the corresponding file system has been correctly mounted before the **dd** operation is performed. If other operations are required, you also need to perform them.

Example

The test file write performance is as follows:

```
dd file=/app/sd/testfile mode=2 bs=32768 count=32768
```

Output

```
Huawei LiteOS # dd file=/app/sd/testfile mode=2 bs=32768 count=32768
filePath  : /app/sd/testfile
mode      : write
blockSize : 32768
blockCount : 32768

Huawei LiteOS # total 1073741824 bytes (1.00 GB, 1024.00MB) write, time used: 48.415982(s), 21.15MB/S
```

The total number of bytes written to the file, total time consumed, and rate are displayed.

9.2.7.10 ls

Function

The **ls** command is used to display the contents of the current directory.

Format

ls [*path*]

Parameter Description

Table 9-19 Parameter description

Parameter	Description	Value Range
path	- If the path parameter is null, the current contents will be displayed. - If the value of the path parameter is an invalid file name, no content will be displayed. "No such directory" will be prompted. - If the value of the path parameter is a valid directory, content under the directory will be displayed.	- Null - Valid directory

User Guide

- The **ls** command displays the size of files.
- The **ls** command cannot count the size of files in proc, displaying 0.

Example

For example, enter **ls**.

Output

Figure 9-9 Check the contents under the current directory. The displayed content is as follows:

```
Huawei LiteOS# ls
Directory /:
. flash          153
bin              <DIR>
font             <DIR>
etc              <DIR>
lost+found      <DIR>
```

9.2.7.11 cat

Function

The **cat** command is used to display content of text files.

Format

cat <pathname>

Parameter Description

Table 9-20 Parameter description

Parameter	Description	Value Range
pathname	File name with the path. If the file is in the current path, can directly enter the file name.	Existed files

Example

For example: **cat w**.

Output

Figure 9-10 View information about the **w** file

```
Huawei LiteOS # cat test0.txt
Huawei LiteOS # 123abctest
```

9.2.7.12 lsfd

Function

The **lsfd** command is used to display the opened file descriptors and their corresponding file names.

Format

lsfd

User Guide

Run the **lsfd** command to view opened file descriptors and their corresponding file names.

Example

Enter **lsfd**.

Output

```
Huawei LiteOS # lsfd
fd  filename
3  /dev/shell
```

9.2.7.13 format

Function

The **format** command is used to format disks.

Format

format <dev_inodename> <sectors> <option> [*label*]

Parameter Description

Parameter	Description
dev_inodename	Name of a device
sectors	Size of the allocated unit memory or sector. The value must be a power of 2. In FAT32, the maximum value is 128 . The value 0 indicates that a proper cluster size is automatically selected. The available cluster size range varies according to the partition size. If the cluster size is incorrectly specified, the formatting may fail.
option	Format option, which is used to select the type of the file system. The options are as follows: • 0x01: FMT_FAT • 0x02: FMT_FAT32 • 0x07: FMT_ANY • 0x08: FMT_ERASE (not supported by USB flash drives) Other values are invalid. The system automatically selects the formatting mode. If the low-level formatting bit is 1 when the USB flash drive is formatted, an error message is displayed.
label	(Optional) Volume label name. The value of this parameter is a string. When the value is set to null , the volume label name set earlier is cleared.

User Guide

- The **format** command formats disks. The device name can be searched for under the **dev** directory. A storage card must be installed before formatting.
- The **format** command can only be used to format USB flash disk, SD cards and EMMC cards and is not valid to the NAND flash or NOR flash memory.

- The **sectors** parameter value must be valid. Otherwise, errors may occur.

Example

For example, enter **format /dev/mmcblk0p0 128 2**.

Output

```
Huawei LiteOS # format /dev/mmcblk0p0 128 2
Format to FAT32, 128 sectors per cluster.
format /dev/mmcblk0p0 Success
```

9.2.7.14 mount

Function

The **mount** command is used to mount a device to a specified directory.

Format

mount <device> <path> <name>

Parameter Description

Table 9-21 Parameter description

Parameter	Description	Value Range
device	The device to be mounted (the format is the path of the device).	Devices of Huawei LiteOS.
path	Directory to which the device is mounted. You must have the execution (search) permission of the specified directory.	N/A
name	Type of the file system.	vfat, yaffs, jffs, ramfs, nfs, procfs and romfs.

User Guide

Add device information, the specified directory, and type of the file system to mount the file system to a specified directory.

Example

For example, enter **mount /dev/mmc0 /bin/vs/sd vfat**.

Output

Mount **/dev/mmc0** to the **/bin/vs/sd** directory:

```
Huawei LiteOS # mount /dev/mmc0 /bin/vs/sd vfat
mount ok
```

9.2.7.15 umount

Function

The **umount** command is used to uninstall a specified file system.

Format

umount <dir>

Parameter Description

Table 9-22 Parameter description

Parameter	Description	Value Range
dir	Directory of the file system that needs to be unmounted.	Directory of the mounted file system

User Guide

- Add the directory to be uninstalled (of the specified directory) to the end of the **umount** command.

Example

For example, enter **umount /bin/vs/sd**.

Output

Uninstall the file system that is mounted to **/bin/vs/sd**:

```
Huawei LiteOS # umount /bin/vs/sd
umount ok
```

9.2.7.16 partinfo

Function

The information about the identified partitions of hard disk, USB disk and SD card can be queried by running the **partinfo** command.

Format

partinfo <dev_inodename>

Parameter Description

Table 9-23 Parameter description

Parameter	Description	Value Range
dev_inodename	The name of the partition to check.	Legal partition name.

Example

Enter **partinfo /dev/sdap0**.

Output

```
Huawei LiteOS # partinfo /dev/sdap0
part info :
disk id      : 3
part_id in system: 0
part no in disk : 0
part no in mbr  : 1
part filesystem : 0C
part dev name   : sdap0
part sec start  : 2048
part sec count  : 167794688
```

9.2.7.17 partition

Function

The **partition** command is used to query the information about flash partitions.

Format

partition <nand | spinor>

Parameter Description

Table 9-24 Parameter description

Parameter	Description	Value Range
nand	Display partition information of nand flash system	N/A
spinor	Display partition information of spinor flash system	N/A

User Guide

- You can view the information about the NAND flash partition only when the yaffs file system is enabled.

- When the JFFS or romfs file system is enabled, you can view the information about the spinor flash partition.

Example

For example, enter **partition nand**.

Output

Information of nand flash partition

```
Huawei LiteOS # partition nand
nand partition num:0, blkdev name:/dev/nandblk0, mountpt:/yaffs0, startaddr:0x00e00000, length:
0x00200000
```

9.2.7.18 statfs

Function

The **statfs** command is used to print the information about a file system, such as type, total size, and available size.

Format

statfs <directory>

Parameter Description

Parameter	Description	Value Range
directory	Mount path of the file system	The existing file system that supports the statfs command

User Guide

The printed information differs with different systems.

Example

Example of printing the information about the YAFFS file system:

statfs yaffs0

Output

```
statfs got:
f_type      = 1497497427
cluster_size = 2048
total_clusters = 704
free_clusters = 640
avail_clusters = 640
f_namelen    = 255
/yaffs0
```

```
total size: 9830400 Bytes
free size: 5890048 Bytes
```

9.2.7.19 virstatfs

Function

The **virstatfs** command is used to display file system information in a virtual partition, such as the total size and available size of the file system.

Format

```
virstatfs <directory>
```

Parameter Description

Parameter	Description	Value Range
directory	Path or subpath of the virtual partition directory entry.	N/A

User Guide

- The **virstatfs** command is available only when the virtual partition feature is enabled on the menuconfig page, that is `LOSCFG_FS_FAT_VIRTUAL_PARTITION=y`.
- The input path of **virstatfs** must be the virtual partition entry directory or subdirectory of the physical partition. If the input path is the root directory of the physical partition, the physical partition fails to use the virtual partitioning feature, or the path points to a non-FAT32 file system partition, the **virstatfs** command will be rejected.
- For details about the virtual partitioning feature, see [Description of Virtual Partition APIs](#) and [Development Procedure](#) in the *FAT Development Guide*.

Example

For example, enter **virstatfs /bin/vs/sd/virpart0**.

Output

```
Huawei LiteOS # mount /dev/mmcblk0p0 /bin/vs/sd vfat
mount ok

Huawei LiteOS # ls /bin/vs/sd
Directory /bin/vs/sd:
virpart0      <DIR>
virpart1      <DIR>
virpart2      <DIR>

Huawei LiteOS # virstatfs /bin/vs/sd/virpart0
Virtual partition "/bin/vs/sd/virpart0" Info:
f_type        = 19780
cluster_size  = 32768
total_clusters = 0x23b73f
free_clusters = 0x23b73e
avail_clusters = 0x23b73e
f_namelen     = 255

/bin/vs/sd/virpart0
total size: 76699107328 Bytes
free  size: 76699074560 Bytes

Huawei LiteOS #
```

9.2.7.20 sync

Function

The **sync** command is used to synchronize the cache data (data in the FAT file system) to an SD card.

Format

sync

User Guide

- The **sync** command is used to refresh the cache. If no SD card is inserted, no operation is performed. If an SD card is inserted, the cache information is synchronized to the SD card. If the operation is successful, no information is displayed.
- Before using the **sync** command, need to enable the FAT file system, that is setting LOSCFG_FS_FAT = y.

Example

For example: after **sync** is input, the synchronization will succeed if there is an SD card and no operation will be done if there is not.

Output

None.

9.2.7.21 writeproc

Function

The **writeproc** command is used to write data to a specified proc file system.

Format

writeproc <val> <operational mark> <path>

Parameter Description

Table 9-25 Parameter description

Parameter	Description	Value Range
val	Data to be written to the file.	A string.
operational mark	If the operational mark is >>, data is written to an existing file.	Only >> is valid.
path	Path of the file to which data will be written.	Absolute path.

User Guide

- The **writeproc** command writes data to a file in the proc file system, and the file must be an existing file.
- If the operational mark is >> and the file to which the data is written exists, the data is written to the file.
- Before using the **writeproc** command, need to enable the proc file system, that is setting LOSCFG_FS_PROC = y.

Example

Enter **writeproc 'sys=2' >> /proc/umap/logmpp**.

Output

Figure 9-11 Modify the level of sys in logmpp

```
Huawei LiteOS # writeproc 'sys=2' >> /proc/umap/logmpp
sys=2 >> /proc/umap/logmpp

Huawei LiteOS # cat /proc/umap/logmpp

Huawei LiteOS # LOG BUFFER STATE-----
MaxLen  ReadPos WritePos ButtPos
64(KB)   0      117    65536

-----CURRENT LOG LEVEL-----
vb      : 3
sys     : 2
chnl    : 3
vpss    : 3
venc    : 3
h264e   : 3
jpege   : 3
viu     : 3
rc      : 3
isp     : 3
vgs     : 3
```

9.2.8 Network Commands

9.2.8.1 Commands for Enabling the Network

- The network commands in the shell take effect only after tcpip_init is initialized and the Telnet connection is set up. Huawei LiteOS does not initialize tcpip_init by default.
- Before using network commands in the shell, you need to run the **menuconfig** command to enable the shell. For details, see [Configuration Items](#).

9.2.8.2 ifconfig

Function

The **ifconfig** command is used to query and configure the parameters such as IP address, network mask, gateway, and the MAC address. The command also enables or disables the data processing function of the NIC.

Format

```
ifconfig  
[-a]  
<interface> [<address>] [netmask <address>] [gateway <address>]  
[hw ether <address>] [mtu <size>]  
[add <address>]  
[del <address>]  
[up | down]
```

Parameter Description

Table 9-26 Parameter description

Parameter	Description	Value Range
No parameter	Print the information about all NICs, such as IP address, network mask, gateway, MAC address, MTU, and running status.	N/A
-a	Print the sending and receiving statistics of protocol stack data.	N/A
interface	Name of specified NIC, for example, eth0.	N/A
address	Configure the IP address, for example 192.168.1.10.	N/A

Parameter	Description	Value Range
netmask	Configure the mask of subnet. Next to the command is the mask parameter, for example, 255.255.255.0.	N/A
gateway	Configure the gateway. Next to the command is the gateway parameter, for example, 192.168.1.1.	N/A
hw ether	Configure the MAC address. Next to the command is the MAC address, for example, 00:11:22:33:44:55. Only support ether hard type currently.	N/A
mtu	Configure the size of mtu. Next to the command is the mtu size, for example, 1000.	[68,1500]
add	Configure the IPv6 address. For example, if the IPv6 address is 2001:db8:2:3:4:5:6:7 , you need to specify inet6 .	NA
del	To delete an IPv6 address, you need to specify inet6 .	NA
up	Enable NIC data processing.	N/A
down	Disable NIC data processing.	N/A

NOTICE

You need to specify the NIC name when running the **ifconfig** command except that the command contains the **-a** parameter and does not contain any parameter.

User Guide

- The **ifconfig** command queries and configures IP address, network mask, gateway, and MAC address.
- Use the command after TCP/IP protocol takes effect.
- Because the IP address collision detection requires response time, IP address configuration by using the **ifconfig** command each time has a delay of about 2 seconds.

Example

1. Enter **ifconfig eth0 192.168.100.31 netmask 255.255.255.0 gateway 192.168.100.1 hw ether 00:11:22:33:44:55**.

Set the eth0 NIC IP address of the development board to **192.168.100.31**, the mask to **255.255.255.0**, the gateway to **192.168.100.1**, and the MAC address to **00:11:22:33:44:55**.

2. Run the **ifconfig -a** command to obtain protocol stack statistics.
3. Enter **ifconfig eth0 inet6 add 2001:db8:2:3:4:5:6:7** and set the IPv6 address of the development board to 2001:db8:2:3:4:5:6:7 (the prefix length is fixed to 64).
4. Enter **ifconfig eth0 inet6 del 2001:db8:2:3:4:5:6:7** to delete the IPv6 address of the development board.

Output

1. Set network parameters.

```
Huawei LiteOS # ifconfig
eth0  ip:192.168.1.2 netmask:255.255.255.0 gateway:192.168.1.1
      HWaddr 00:11:22:33:44:55 MTU:1500 Running Default Link UP
lo     ip:127.0.0.1 netmask:255.0.0.0 gateway:127.0.0.1
      HWaddr 00 MTU:0 Running Link Down

Huawei LiteOS # ifconfig eth0 192.168.100.31 netmask 255.255.255.0 gateway 192.168.100.1 hw
ether 00:11:22:33:44:55

Huawei LiteOS # ifconfig
eth0  ip:192.168.100.31 netmask:255.255.255.0 gateway:192.168.100.1
      HWaddr 00:11:22:33:44:55 MTU:1500 Running Default Link UP
lo     ip:127.0.0.1 netmask:255.0.0.0 gateway:127.0.0.1
      HWaddr 00 MTU:0 Running Link Down
```

The following table lists the output parameters.

Parameter	Description
ip	IP address of the board
netmask	Network mask
gateway	Gateway
HWaddr	MAC address of the board
MTU	Maximum transmission units of network
Running/Stop	Whether NIC is running
Default	Explain connecting to the default gateway
Link UP/Down	Connect status to NIC

2. Obtain protocol stack statistics.

```
Huawei LiteOS # ifconfig -a
RX packets:23128 error:0 dropped:0 overrun:0 bytes:10390(10.1KB)
TX packets:40921 error:0 dropped:0 overrun:0 bytes:64008(62.5KB)
```

The following table describes the output items of the **ifconfig -a** command:

Parameter	Description
RX packets	The number of normal packets that the IP layer has received.

Parameter	Description
RX error	The number of packets with errors that the IP layer has received. The error types include length error, verification error, IP option error, and error of the protocol field in an IP header.
RX dropped	The number of packets that the IP layer has dropped. The packets are dropped because the packets have errors, the packets cannot be forwarded, or the local NIC that receives the packets is disabled.
RX overrun	The number of packets that the MAC layer fails to deliver to the upper-layer protocol stack. The main failure cause is that the protocol stack resources are insufficient.
RX bytes	The total number of bytes in all normal packets that the IP layer has received, excluding the bytes in the fragments that have not been completely reassembled.
TX packets	The number of packets that the IP layer has successfully sent or forwarded.
TX error	The number of packets that the IP layer fails to send. The packets fail to be sent because the packets cannot be routed, or the packets fail to be processed in the protocol stack.
TX dropped	The number of packets that the MAC layer drops due to failure to send the packets. The packets fail to be sent because the network adapter driver fails to process the packets.
TX overrun	Not in use.
TX bytes	The total number of bytes in the packets that the IP layer has successfully sent or forwarded.

3. Configure the IPv6 address.

```
Huawei LiteOS # ifconfig eth0 inet6 add 2001:db8:2:3:4:5:6:7
```

```
Huawei LiteOS # ifconfig
eth1 ip:192.168.3.60 netmask:255.255.255.0 gateway:0.0.0.0
HWaddr 00:11:22:33:44:55 MTU:1500 Running Link UP
eth0 ip:192.168.2.60 netmask:255.255.255.0 gateway:0.0.0.0
ip6: 2001:db8:2:3:4:5:6:7/64
HWaddr 00:11:22:33:44:55 MTU:1500 Running Link UP
lo ip:127.0.0.1 netmask:255.0.0.0 gateway:127.0.0.1
ip6: ::*/64
HWaddr 00 MTU:16436 Running Link UP
```

4. Delete the IPv6 address.

```
Huawei LiteOS # ifconfig eth0 inet6 del 2001:db8:2:3:4:5:6:7
```

```
Huawei LiteOS # ifconfig
eth1 ip:192.168.3.60 netmask:255.255.255.0 gateway:0.0.0.0
```



```
HWaddr 00:11:22:33:44:55 MTU:1500 Running Link UP
eth0 ip:192.168.2.60 netmask:255.255.255.0 gateway:0.0.0.0
HWaddr 00:11:22:33:44:55 MTU:1500 Running Link UP
lo ip:127.0.0.1 netmask:255.0.0.0 gateway:127.0.0.1
ip6: ::*/64
HWaddr 00 MTU:16436 Running Link UP
```

9.2.8.3 dns

Function

The **dns** command is used to configure the DNS server address of the board.

Format

dns <1-2> IP

dns -a

Parameter Description

Table 9-27 Parameter description

Parameter	Description	Value Range
1-2	Choose to configure the first or second DNS server.	1~2
IP	IP address of the DNS server.	-
-a	Display the current configuration state of the DNS server.	-

Example

For example:

1. Run the **dns -a** command to check the current DNS settings.
2. Enter **dns 2 192.168.1.2** to set the IP address of the second DNS server.
3. Run the **dns -a** command to check whether the DNS is set successfully.

Output

1. View the configuration information about the current DNS.

```
Huawei LiteOS # dns -a
dns1: 208.**.**.*
dns2: 0.0.0.0
```

2. Configure the IP of the second DNS.

```
Huawei LiteOS # dns 2 192.168.1.2
```

3. Check whether the configuration of DNS is successful.

```
Huawei LiteOS # dns -a
dns1: 208.**.**.*
dns2: 192.168.1.2
```

9.2.8.4 arp

Function

On an Ethernet (LAN), hosts use MAC addresses (not IP addresses) for communication. Therefore, IP addresses must be converted into MAC addresses for communication on the LAN. To transform the IP address, store the ARP cache, a table that records the mapping between the MAC address and IP address, in the host. When sending IP package to the target IP address in a LAN, the host checks the table for the target MAC address. The ARP cache is maintained by TCP/IP protocol stack. You can view or modify the ARP table by using **ARP** commands.

Format

```
arp
arp [-i /F] -s IPADDR HWADDR
arp [-i /F] -d IPADDR
```

Parameter Description

Table 9-28 Parameter description

Parameter	Description	Value Range
None.	Print the contents of the whole ARP cache.	N/A
-i IF	Specify a network API (optional).	N/A
-s IPADDR HWADDR	Add an ARP entry. The parameters next to the command are the IP address and MAC address of another host in LAN.	N/A
-d IPADDR	Delete an ARP entry.	N/A

User Guide

- The **arp** command queries and modifies the ARP cache table of TCP/IP protocol stack. It is meaningless to add the ARP entry in non-LAN networks, and protocol stack will return fail.
- Use the command after TCP/IP protocol takes effect.

Example

```
For example:
1. Enter arp.
2. Enter arp -s 192.168.1.1 00:11:22:33:44:55.
```

Output

Figure 9-12 Print the whole ARP cache table

Huawei LiteOS # arp			
Address	HWaddress	Iface	Type
192.168.1.3	00:D8:61:3E:49:01	eth0	dynamic
192.168.1.100	00:00:00:11:11:11	eth0	static

Table 9-29 Parameter description

Parameter	Description
Address	The IP address of the network device that connects to the board.
HWaddress	The MAC address of the network device that connects to the board.
Iface	Name of the API used by ARP entry.
Type	Indicate ARP entry is static or dynamic. The ARP entry added by ARP protocol when processing ARP request or reply is dynamic, while the entry added by user is static.

9.2.8.5 ntpdate

Function

The **ntpdate** command is used to synchronize the system time from the NTP server.

Format

ntpdate <SERVER_IP1> <SERVER_IP2>...

Parameter Description

Table 9-30 Parameter description

Parameter	Description	Value Range
SERVER_IP	IP of NTP server	-

User Guide

Run ntpdate [*SERVER_IP1*] [*SERVER_IP2*]...

The time of the first valid server IP will be obtained and displayed by running the **ntpdate** command.

Example

For example:

Use the **ntpdate** command to update the time of system.

Output

Use the **ntpdate** command to update the time of system.

```
Huawei LiteOS # ntpdate 192.168.1.3  
time server 192.168.1.3: Mon Jun 13 09:24:25 2016
```

The time zone of the development board is different from that of the server. Therefore, the time displayed on the development board may be several hours different from that on the server.

9.2.8.6 ping

Function

The **ping** command is used to check the network connectivity.

Format

ping [-n <cnt>] [-w <interval>] [-l <data_len>] IP

ping [-t] [-w <interval>] IP

ping -k

Parameter Description

Table 9-31 Parameter description

Parameter	Description	Value Range
IP	IP address of the network to be tested.	-
-n <cnt>	Times of execution. The default value is 4.	1~65535
-w <interval>	Interval between sending each ping packet. (Unit: ms)	-
-l <data_len>	Data length of each ping packet (ICMP ECHO request packet) excluding the header of ICMP packet.	0~65500
-t	Ping the target until the ping thread is killed by ping -k command.	-
-k	Kills the ping thread.	-

User Guide

- The **ping** command tests the connectivity of the target IP network. The parameter is the destination IP address.
- If the destination IP address is not reachable, a message indicating request timeout will be displayed.
- If displaying sends an error, it explains the destination IP route is not reachable.
- Use the command after TCP/IP protocol takes effect.

Example

For example, enter **ping 192.168.0.2**.

Output

Figure 9-13 ping 192.168.0.2

```
Huawei LiteOS# ping 192.168.0.2
Reply from 192.168.0.2: time=2ms TTL=128
Reply from 192.168.0.2: time=1ms TTL=128
Reply from 192.168.0.2: time=5ms TTL=128
Reply from 192.168.0.2: time=2ms TTL=128
```

9.2.8.7 ping6

Function

ping6 command is used to test whether the IPv6 network connection is normal.

Format

ping6 [-c <count>] [-I <interface/sourceAddress>] <destination>

Parameter Description

Table 9-32 Parameter description

Parameter	Description	Value Range
-c <count>	The number of executions, the default value is 4.	1~65535
-I <interface>	Execute the specified IPv6 network card interface.	NA
-I <sourceAddress>	Execute the specified IPv6 source address.	NA

User Guide

- **ping6** command is used to check whether the network connection to destination IPv6 is normal. The parameter is the destination IPv6 address.
- If the destination IPv6 address is unreachable, the request timeout will be displayed.
- If a transmission error is displayed, there is no route to destination IPv6 address.
- This command could be used while TCP/IP stack is started.

Examples

- **ping6 destination:** Test the IPv6 network with default options.
- **ping -c destination:** Test the IPv6 network for specified times.
- **ping -I interface destination:** Test the IPv6 network with a specified NIC.
- **ping -I sourceAddress destination:** Test the IPv6 network with a specified source IPv6 address.

Output

1. Enter **ping6 2001:db8:2:3:4:5:6:7**.
Huawei LiteOS # ping6 2001:db8:2:3:4:5:6:7
PING 2001:db8:2:3:4:5:6:7 with 56 bytes of data.
56 bytes from 2001:db8:2:3:4:5:6:7 : icmp_seq=1 time<1 ms
56 bytes from 2001:db8:2:3:4:5:6:7 : icmp_seq=2 time<1 ms
56 bytes from 2001:db8:2:3:4:5:6:7 : icmp_seq=3 time<1 ms
56 bytes from 2001:db8:2:3:4:5:6:7 : icmp_seq=4 time<1 ms
--- 2001:db8:2:3:4:5:6:7/64 ping statistics ---
4 packets transmitted, 4 received, 0.00% packet loss, time 20ms
rtt min/avg/max = 0/0.00/0 ms
2. Enter **ping6 -c 3 2001:db8:2:3:4:5:6:7**.
Huawei LiteOS # ping6 -c 3 2001:db8:2:3:4:5:6:7
PING 2001:db8:2:3:4:5:6:7 with 56 bytes of data.
56 bytes from 2001:db8:2:3:4:5:6:7 : icmp_seq=1 time<1 ms
56 bytes from 2001:db8:2:3:4:5:6:7 : icmp_seq=2 time<1 ms
56 bytes from 2001:db8:2:3:4:5:6:7 : icmp_seq=3 time<1 ms
--- 2001:db8:2:3:4:5:6:7/64 ping statistics ---
3 packets transmitted, 3 received, 0.00% packet loss, time 20ms
rtt min/avg/max = 0/0.00/0 ms
3. Enter **ping6 -I eth0 2001:db8:2:3:4:5:6:7 2001:db8:2:3:4:5:6:7**.
Huawei LiteOS # ping6 -I eth0 2001:db8:2:3:4:5:6:7
PING 2001:db8:2:3:4:5:6:7 with 56 bytes of data.
56 bytes from 2001:db8:2:3:4:5:6:7 : icmp_seq=1 time=10 ms
56 bytes from 2001:db8:2:3:4:5:6:7 : icmp_seq=2 time<1 ms
56 bytes from 2001:db8:2:3:4:5:6:7 : icmp_seq=3 time<1 ms
56 bytes from 2001:db8:2:3:4:5:6:7 : icmp_seq=4 time<1 ms
--- 2001:db8:2:3:4:5:6:7/64 ping statistics ---
4 packets transmitted, 4 received, 0.00% packet loss, time 30ms
rtt min/avg/max = 0/2.50/10 ms
4. Enter **ping6 -I 2001:db8:2:3:4:5:6:7 2001:db8:2:3:4:5:6:7**.
Huawei LiteOS # ping6 -I 2001:db8:2:3:4:5:6:7 2001:db8:2:3:4:5:6:7
PING 2001:db8:2:3:4:5:6:7 with 56 bytes of data.
56 bytes from 2001:db8:2:3:4:5:6:7 : icmp_seq=1 time<1 ms
56 bytes from 2001:db8:2:3:4:5:6:7 : icmp_seq=2 time<1 ms
56 bytes from 2001:db8:2:3:4:5:6:7 : icmp_seq=3 time<1 ms
56 bytes from 2001:db8:2:3:4:5:6:7 : icmp_seq=4 time<1 ms
--- 2001:db8:2:3:4:5:6:7/64 ping statistics ---
4 packets transmitted, 4 received, 0.00% packet loss, time 20ms
rtt min/avg/max = 0/0.00/0 ms

9.2.8.8 tftp

Function

Trivial file transfer protocol (TFTP) is a simple file transfer protocol in the TCP/IP protocol suite. It provides simple and low-overhead file transfer services between clients and servers. The port number is 69.

Format

```
tftp <-g | -p> -l <FullPathLocalFile> -r <RemoteFile> <Host>
```

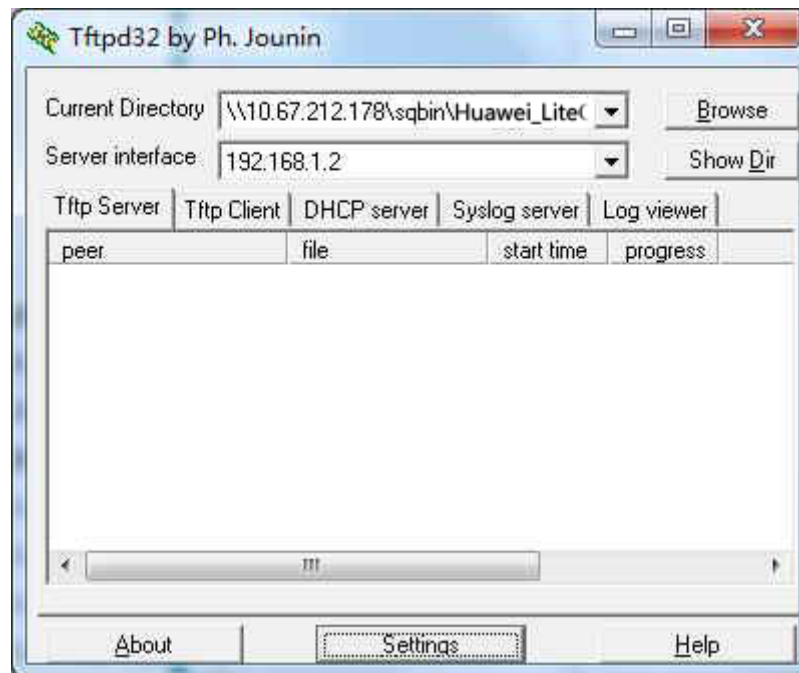
Parameter Description

Table 9-33 Parameter description

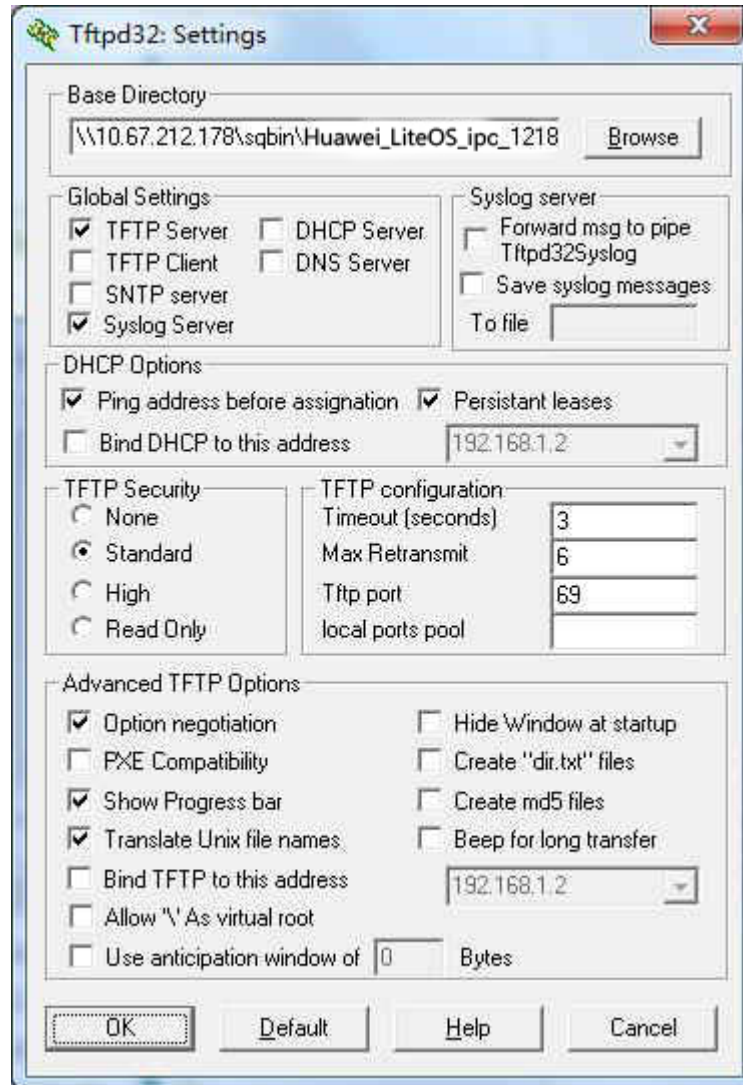
Parameter	Description	Value Range
-g	Obtain files from the server	Choose one from -g and -p
-p	Upload files to the server	Choose one from -g and -p
-l	Name of a local file (full path should be opened in Huawei LiteOS)	-
-r	Name of a file on the server	-
Host	Server IP	-

User Guide

1. Set up TFTP server. Firstly, you need to ensure that the server has been installed TFTP client, and then configure it according to following figures.



Click Setting to set the TFTP server.



Set Base Directory to the TFTP directory and then click OK to exit.

2. Huawei LiteOS board uses the tftp command to upload or download files.
3. The size of a file that is transmitted must not be greater than 32 MB.
4. TFTP is a commissioning function and is disabled by default. It is forbidden to use Shell in commercial products.
5. Disclaimer: Huawei shares no responsibilities for any risks caused in the use of TFTP in commercial products.

Example

For example, download the **vs_server.bin** file from the server.

Output

```
Huawei LiteOS # tftp -g -l /nfs/vs_server.bin -r vs_server.bin 192.168.1.2
TFTP transfer finish
```

If the transfer succeeds, the message TFTP transfer finish will be displayed. If the transfer fails, other printed information will be displayed to help locate the problem.

9.2.8.9 netstat

Function

The **netstat** command is a console command and a useful tool for monitoring the TCP/IP network. It displays the actual network connection and the status of each network API device. The **netstat** command displays the statistics of TCP and UDP protocols to check the connectivity of all APIs of the board.

Format

netstat

Example

For example, enter **netstat**.

Output

Figure 9-14 Information printed using netstat

```
Huawei LiteOS # netstat
===== total sockets 128 ===== unused sockets 122 === BootTime 27 s=====
Proto Recv-Q Send-Q Local Address Foreign Address State Time
tcp 0 0 192.168.1.102:63758 192.168.1.2:5001 ESTABLISHED 0

Proto Recv-Q Send-Q Local Address Foreign Address
udp 0 0 0.0.0.0:1337 0.0.0.0:0

Type Recv-Q Send-Q Local Address Foreign Address Protocol HDRINCL
raw 88 0 0.0.0.0 0.0.0.0 6 1
```

Table 9-34 Parameter description

Parameter	Description
Proto	Protocol type
Recv-Q	Amount of data that is not read by the user. For TCP socket in Listen state, the value represents the number of connections that have been done 3-way hand shake, but not accepted by upper layer.
Send-Q	When TCP is used, the parameter specifies the amount of data that is sent and not acknowledged. When UDP is used, the parameter specifies the amount of data that is cached because IP address parsing is not complete.
Local Address	Local address and port
Foreign Address	Remote address and port
State	Connectivity of TCP (the parameter is meaningless to UDP)

Parameter	Description
Type	Raw socket type: • raw indicates the AF_INET type of raw socket. • pkt-raw indicates the PF_PACKET type of raw socket.
Protocol	• For an AF_INET raw socket, protocol indicates the type of the protocol in the IP header. • For a PF_PACKET raw socket, protocol indicates the type of the protocol in the Ethernet header.
HDRINCL	Whether the user has enabled the IP_HDRINCL socket option on the AF_INET raw socket. • 1 indicates that the user has enabled this option. • 0 indicates that the user has not enabled this option.
netif	NIC that is bound to the PF_PACKET raw socket. None indicates that no NIC is bound to the raw socket.
Time	When the TCP socket is in the CLOSED state, it indicates the time when the socket is created. While in the CLOSE_WAIT or LAST_ACK state, it indicates the time when the socket receives the FIN. Other states have no meaning.

 NOTE

```
"===== total sockets 32 ===== unused sockets 22 BootTime 27 s ===== "
```

The preceding printed information indicates that there are 32 sockets in total, and 22 out of them are not in use, 27 seconds since the system was started.

9.2.8.10 ipdebug

Function

ipdebug is a console command that is a very useful tool for debugging IPv6 information. It can display IPv6 address prefixes, neighbor entry information, destination cache entries, and default route entries.

Format

```
ipdebug
```

Example

Enter **ipdebug**.

Output

The following information is displayed:

```

Huawei LiteOS # ipdebug
=====
|| Prefix List ||
=====
Prefix          validLifetime    Flags
=====
0000::          86384          1
-----

=====
|| Neighbor Cache Entries ||
=====
0000::1 dev eth0 lladr 00:11:22:33:44:55 STALE Router
FE80::4639:C4FF:FE94:5D44 dev eth0 lladr 00:11:22:33:44:55 REACHABLE Router
-----

=====
|| Destination Cache Entries ||
=====
0000::1 0000::1 pmtu 1500 age 152
FE80::4639:C4FF:FE94:5D44 FE80::4639:C4FF:FE94:5D44 pmtu 1500 age 6
0000::1 FE80::4639:C4FF:FE94:5D44 pmtu 1500 age 17
:: :: pmtu 0 age 219
:: :: pmtu 0 age 219
:: :: pmtu 0 age 219
:: :: pmtu 0 age 219
:: :: pmtu 0 age 219
:: :: pmtu 0 age 219
:: :: pmtu 0 age 219
-----

=====
|| Default Router Entries ||
=====
FE80::4639:C4FF:FE94:5D44 invalidation_timer 1784 flags 0
-----

```

9.2.8.11 netdebug

Function

1. Displays the protocol stack memory resource status and key memory information.
2. Queries the usage of global socket resources or associated data.

Format

```

netdebug memp [-i / -d <type> / -udp / -tcp / -raw / -conn]
netdebug sock [-i / -d <idx>]

```

Parameter Description

Table 9-35 Parameter description

Parameter		Description	Value Range
memp	-i	Queries all memory information.	
	-d <type>	Queries the linked list of a specified type of memory.	
	-udp	Queries the UDP PCB information.	
	-tcp	Queries the TCP PCB information.	
	-raw	Queries the RAW PCB information.	
	-conn	Queries the NETCONN information.	
sock	-i	Queries the SOCKET information.	
	-d <idx>	Queries information about the socket with a specified index, including NETCONN and PCB information.	0~127

User Guide

This command is used to view the resource usage and query related information during fault location.

Example

1. Run the **netdebug mem -i** command to query memory information.
2. Run the **netdebug mem -d 12** command to query information about the memory with a type of 12.
3. Run the **netdebug mem -udp** command to query the **UDP PCB** brief information.
4. Run the **netdebug mem -tcp** command to query the **TCP PCB** brief information.
5. Run the **netdebug mem -raw** command to query the **RAW PCB** brief information.
6. Run the **netdebug mem -conn** command to query the **NETCONN** brief information.
7. Run the **netdebug sock -i** command to query the **SOCKET** information.

8. Run the **netdebug sock -d 2** command to query brief information about the socket with an index of 2.

Output

Figure 9-15 Command output displayed after running the netdebug memp -i command

```
Huawei LiteOS # netdebug memp -i
```

[MEMP]	TYPE	START	END	TAB	NUM	USED	AVAILABLE
[MEMP] 0		0x801c8508	0x801c9adc	0x801c9adc	128	0	128
[MEMP] 1		0x801c9b08	0x801cb0dc	0x801cb084	128	2	126
[MEMP] 2		0x801cb108	0x801d3400	0x801d3400	128	0	128
[MEMP] 3		0x801d3508	0x801d46e4	0x801d46e4	128	0	128
[MEMP] 4		0x801d4708	0x80261db0	0x80261db0	14482	0	14482
[MEMP] 5		0x80261dd8	0x802641b4	0x802641b4	256	0	256
[MEMP] 6		0x802641d8	0x802b7964	0x802b7964	17096	0	17096
[MEMP] 7		0x802b7978	0x802c6900	0x802c6810	512	2	510
[MEMP] 8		0x802c6978	0x802c6e64	0x802c6e64	64	0	64
[MEMP] 9		0x802c6e78	0x802c8264	0x802c8250	256	0	256
[MEMP] 10		0x802c8278	0x802d3480	0x802d3480	5698	0	5698
[MEMP] 11		0x802d3488	0x802d3514	0x802d3500	8	1	7
[MEMP] 12		0x802d3528	0x802d3598	0x802d3538	8	6	2
[MEMP] 13		0x802d35a8	0x802d3e14	0x802d3e14	8	0	8
[MEMP] 14		0x802d3f48	0x802d5b2c	0x802d5b2c	256	0	256
[MEMP] 15		0x802d5b48	0x802ee01c	0x802ee01c	64	0	64

Figure 9-16 Command output displayed after running the netdebug memp -i -d 12 command

```
Huawei LiteOS # netdebug memp -i -d 12
```

[MEMP]	TYPE	INDEX	ADDR	NEXT
[MEMP] 12		0	0x802d3528	0x0
[MEMP] 12		1	0x802d3538	0x802d3528
[MEMP] 12		2	0x802d3548	0x802d3598
[MEMP] 12		3	0x802d3558	0x802d3568
[MEMP] 12		4	0x802d3568	0x802d3548
[MEMP] 12		5	0x802d3578	0x0
[MEMP] 12		6	0x802d3588	0x802d3578
[MEMP] 12		7	0x802d3598	0x802d3588

Figure 9-17 Command output displayed after running the **netdebug memp -udp** command

```
Huawei LiteOS # netdebug memp -udp
BootTime 112 s
-----IMPPCB
pcb      :0x8031a96c
l_ip     :0.0.0.0
r_ip     :0.0.0.0
next     :0x0
flags    :0
l_port   :1337
r_port   :0
```

Figure 9-18 Command output displayed after running the **netdebug memp -tcp** command

```
Huawei LiteOS # netdebug memp -tcp
BootTime 135 s
-----TCPPCB
pcb      :0x80323368
l_ip     :0.0.0.0
r_ip     :0.0.0.0
cb_arg   :0x80417b40
l_port   :0
r_port   :0
state    :0
flags    :64
```

Figure 9-19 Command output displayed after running the **netdebug memp -raw** command

```
Huawei LiteOS # netdebug memp -raw
BootTime 112 s
-----RAWPCB
pcb      :0x8031936c
l_ip     :0.0.0.0
r_ip     :0.0.0.0
hdrincl  :1
next     :0x0
netifidx:0
proto    :6
e_proto  :6
```

Figure 9-20 Command output displayed after running the **netdebug memp -conn** command

```
Huawei LiteOS # netdebug memp -conn
BootTime 147 s

-----NETCONN
conn      :0x80417ce4
pcb       :0x8031a96c
socket    :256
r_buf_s   :2000
r_avail   :0
l_err     :0
is_fatal  :0
type      :32
state     :0
shutdown  :0
```

Figure 9-21 Command output displayed after running the **netdebug sock -i** command

```
Huawei LiteOS # netdebug sock -i
-----SOCKET
idx       :0
conn      :0x802c6900
r_event   :0
s_event   :1
e_event   :0
err       :0
events    :1
select_w  :0
sock_flag :1
poll_prev :0x802ee6a4
poll_next :0x802ee6a4

-----SOCKET
idx       :1
conn      :0x802c6888
r_event   :0
s_event   :1
e_event   :0
err       :0
events    :0
select_w  :0
sock_flag :1
poll_prev :0x802ee6f8
poll_next :0x802ee6f8
```


Figure 9-22 Command output displayed after running the **netdebug sock -d 0** command

```
Huawei LiteOS # netdebug sock -d 0
BootTime 165 s

-----SOCKET
idx      :0
conn     :0x80417ce4
r_event  :0
s_event  :1
e_event  :0
err      :0
sock_flag:1

-----NETCONN
conn     :0x80417ce4
pcb      :0x8031a96c
socket   :256
r_buf_s  :2000
r_avail  :0
l_err    :0
is_fatal:0
type     :32
state    :0
shutdown:0

-----UDPPCB
pcb      :0x8031a96c
l_ip     :0.0.0.0
r_ip     :0.0.0.0
next     :0x0
flags    :0
l_port   :1337
r_port   :0
```

9.2.8.12 telnet

Function

telnet is used to access servers through networks from computers of terminal users.

Format

telnet [*on / off*]

Parameter Description

Table 9-36 Parameter description

Parameter	Description	Value Range
on	Turning on a server	N/A
off	Turning off a server	N/A

User Guide

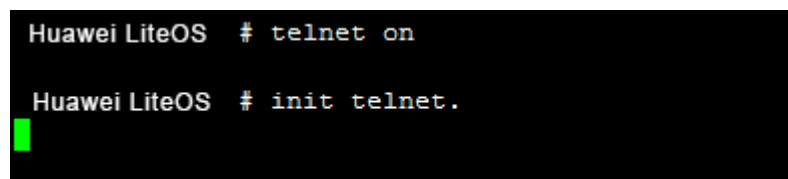
- **telnet** is used to access servers through networks from computers of terminal users.
- Before starting Telnet, ensure that the network driver and network protocol stack have been initialized and the NIC of the development board is in the **link up** state.
- Currently, multiple clients (Telnet + IP) cannot connect to the development board.
- Telnet is a commissioning function and is disabled by default. It is forbidden to use Telnet in commercial products.
- Disclaimer: Huawei shares no responsibilities for any risks caused in the use of Telnet in commercial products.

Example

For example, enter **telnet on**.

Output

Figure 9-23 Entering telnet on

A terminal window with a black background. The prompt 'Huawei LiteOS #' is shown in green. The command 'telnet on' is entered in red. The prompt 'Huawei LiteOS #' is shown again in green. The command 'init telnet.' is entered in red. A green cursor is visible at the end of the second line.

```
Huawei LiteOS # telnet on
Huawei LiteOS # init telnet.
```

9.2.8.13 iperf

Function

Iperf is a network performance test tool. It can be used to test the TCP and UDP bandwidth performance, delay jitter, and packet loss.

Format

```
iperf -s [options]
iperf [-c <host>] [options]
iperf -k
```

Description

Table 9-37 Common parameters

Parameter	Description	Value Range
-V	Use an IPv6 address. (If an IPv6 address is required, it must be used as the first parameter and placed before -s.)	None
-u	Set the connection type to UDP.	None
-i <n>	Set the data report interval to <i>n</i> seconds (1 second by default).	<i>n</i> is an unsigned integer, and its unit is second.
-l <n>	Set the size of the read/write buffer to <i>n</i> . The default value is 8 KB (TCP) or 1470 B (UDP).	<i>n</i> is an unsigned integer, in bytes.
-p <n>	Set the network port number of the server to be listened on to <i>n</i> . The default value is 5001 .	<i>n</i> is an unsigned integer and has no unit.
-S <n>	Set the IPv4 service type to <i>n</i> . The default value is 0 .	<i>n</i> is an unsigned integer ranging from 0 to 255.
-B <ip>	Bind host address: - Bind the client from which data is received when running as a server. - Server from which data is received when the system is running as a client.	The IP address can be an IPv4 or IPv6 address.
-k	Stop all iperf threads.	None
-h -help --help	Display help information.	None

Table 9-38 Server-specific parameters

Parameter	Description	Value Range
-s	Set as a server.	None

Table 9-39 Client-specific parameters

Parameter	Description	Value Range
-c <host>	Set as a client and connect to the specified host.	<i>host</i> indicates the IPv4/IPv6 address of the server.
-t <n>	Set the total time for sending data to <i>n</i> seconds. The default value is 30 seconds.	<i>n</i> is an unsigned integer, and its unit is second.
-n <m>	Set the total number of bytes to be sent to <i>m</i> .	<i>m</i> is an unsigned integer, and its unit is byte.
-b <n>	Set the sending bandwidth in the UDP connection scenario to <i>n</i> bit rate. The default value is 1 Mbit/s.	<i>n</i> is an unsigned integer, and its unit is bit rate.

Usage Guide

- The iperf tool can be used only after shell, lwIP, and iperf are enabled. To collect statistics on the packet loss rate and jitter delay in the UDP connection, you need to enable the UDP jitter function. You can enable them by using the following menu items.
 Debug --> Enable a Debug Version --> Enable Shell
 Net --> Enable Lwip
 Net --> Enable Lwip --> Enable Iperf
 Net --> Enable Lwip --> Enable Iperf --> Enable Udp Jitter
- When the **-p** and **-l** parameters are set, the values of these parameters on the server and client must be the same.
- When the **iperf -k** command is terminated, there is a 1-second blocking delay due to thread synchronization and socket connection problems.
- Only one iperf instance can run on a device at a time. If an iperf instance is running, the message "Iperf is running" is displayed when you start the iperf instance.

Example

- IPv4 TCP bandwidth test
 Run the **iperf -s** command on the server.
 Run the **iperf -c 192.168.1.1 -i 1 -t 10** command on the client.
- IPv6 UDP bandwidth test
 Run the **iperf -V -s -p 5001 -i 5 -u** command on the server.
 Run the **iperf -V -c FFFF::ABCD:6666:1234:5678 -p 5001 -i 1 -t 10 -u -b 10M** command on the client.

Output

- TCP bandwidth test in IPv4 networking
 The following information is displayed on the server. The data transmission volume and bandwidth are output every second. The total test duration is 10

seconds. After the test is complete, the statistics of the entire process are output:

```
+IPERF:
Interval      Transfer      Bandwidth
0.0- 1.0 sec  11.26 MBytes  94.42 Mb/s
1.0- 2.0 sec  11.27 MBytes  94.57 Mb/s
2.0- 3.0 sec  11.27 MBytes  94.57 Mb/s
3.0- 4.0 sec  11.28 MBytes  94.58 Mb/s
4.0- 5.0 sec  11.27 MBytes  94.57 Mb/s
5.0- 6.0 sec  11.28 MBytes  94.58 Mb/s
6.0- 7.0 sec  11.27 MBytes  94.57 Mb/s
7.0- 8.0 sec  11.27 MBytes  94.57 Mb/s
8.0- 9.0 sec  11.27 MBytes  94.57 Mb/s
9.0-10.0 sec  11.28 MBytes  94.58 Mb/s
0.0-10.0 sec  113.00 MBytes 94.56 Mb/s
OK
```

The output data of the client is basically the same as that of the server.

- IPv6 UDP bandwidth test

The following information is displayed on the server. The data transmission volume and bandwidth are output every 5 seconds. The total test duration is 10 seconds. After the test is complete, the statistics of the entire process are output: The transmit bandwidth of the client is set to 10 Mbit/s. Therefore, the bandwidth rate is close to 10 Mbit/s in the output information:

```
+IPERF:
Interval      Transfer      Bandwidth
0.0- 5.0 sec   5.57 MBytes   9.34 Mb/s
5.0-10.0 sec   5.57 MBytes   9.34 Mb/s
0.0-10.0 sec   11.14 MBytes  9.34 Mb/s
OK
```

The output on the client is as follows. Because **-i 1** is set, the data transmission volume and bandwidth are output per second:

```
+IPERF:
Interval      Transfer      Bandwidth
0.0- 1.0 sec   1.13 MBytes   9.48 Mb/s
1.0- 2.0 sec   1.09 MBytes   9.14 Mb/s
2.0- 3.0 sec   1.11 MBytes   9.31 Mb/s
3.0- 4.0 sec   1.12 MBytes   9.40 Mb/s
4.0- 5.0 sec   1.12 MBytes   9.40 Mb/s
5.0- 6.0 sec   1.11 MBytes   9.31 Mb/s
6.0- 7.0 sec   1.13 MBytes   9.48 Mb/s
7.0- 8.0 sec   1.12 MBytes   9.40 Mb/s
8.0- 9.0 sec   1.12 MBytes   9.40 Mb/s
9.0-10.0 sec   1.09 MBytes   9.14 Mb/s
0.0-10.0 sec   11.14 MBytes  9.34 Mb/s
OK
```

9.2.9 Dynamic Loading Commands

9.2.9.1 Commands for Enabling Dynamic Loading

- To run shell commands related to dynamic loading, you need to initialize the dynamic loading module in advance. For details, see [Dynamic Loading](#).
- Before using the dynamic loading commands in the shell, you need to open menuconfig to enable the shell. For details, see [Configuration Items](#). Then, configure the dynamic loading module. For details, see [Step 1](#).

9.2.9.2 mopen

Function

The **mopen** command is used to load a user module.

Format

mopen <module_path>

Parameter Description

Table 9-40 Parameter description

Parameter	Description
module_path	Path of the user module, and the path must be an absolute path

User Guide

module_path can only be set to a .o file, cannot be set to a .so file.

Example

For example, load **/yaffs/bin/dynload/foo.o**.

Enter **mopen /yaffs/bin/dynload/foo.o**.

Output

```
Huawei LiteOS# mopen /yaffs/bin/dynload/foo.o  
module handle: 0x80391928
```

```
Huawei LiteOS#
```

Module handle will be returned if the loading succeeds. The returned module handle in this example is 0x80391928.

9.2.9.3 mclose

Function

The **mclose** command is used to uninstall a module.

Format

mclose <module_handle>

Parameter Description

Table 9-41 Parameter description

Parameter	Description
module_handle	The module handle which to be unloaded. The handle value returned by the module that is opened in mopen.

User Guide

If a module with specified handle is uninstalled, it cannot be searched for symbols.

Example

For example: uninstall the user module whose handle is 0x80391928.

Enter **mclose 0x80391928**.

Output

```
Huawei LiteOS# mclose 0x80391928
```

```
Huawei LiteOS#
```

If no error information is displayed, the uninstallation is successful.

9.2.9.4 findsym

Function

The **findsym** command is used to query the symbol address in the global symbol table or the specified module.

Format

findsym <handle> <symbol_name>

Parameter Description

Table 9-42 Parameter description

Parameter	Description
hanlde	Module handle. The value 0 indicates the global symbol table.
symbol_name	Name of the symbol to be searched.

User Guide

- If the handle is 0, the command searches the symbol address in the global symbol table. (The global symbol table contains the kernel symbol and other symbols provided by user module).
- If the handle is not 0 and is valid, the command searches the symbol in the module specified by the handle.

Example

For example:

1. To search the global symbol table for the printf symbol, enter **findsym 0 printf**.
2. To search for the test_0 symbol address in the user module whose handle loaded in mopen is 0x80391928, enter **findsym 0x80391928 test_0**.

Output

```
Huawei LiteOS# findsym 0 printf
symbol address:0x8004500c

Huawei LiteOS#
Huawei LiteOS# findsym 0x80391928 test_0
symbol address:0x8030f241

Huawei LiteOS#
```

9.2.9.5 call

Function

The **call** command is used to call a function with no parameters.

Format

call <func_address>

Parameter Description

Table 9-43 Parameter description

Parameter	Description
func_address	Memory address of the function

User Guide

Call the function after the memory address of a function symbol has been searched in findsym.

NOTICE

Arbitrary memory operations by running the call command will cause system crashes.

Example

For example: call test_0 function (address: 0x8030f241) that is searched in findsym.

Output

```
Huawei LiteOS# call 0x8030f241
test_0
```

9.2.9.6 lddrop

Function

The **lddrop** command is used to destroy the dynamic loading module.

Format

lddrop

User Guide

The dynamic loading feature cannot be used after the module is destroyed. To use the dynamic loading feature, the module needs to be reloaded for initialization.

Example

For example: uninstall the dynamic loading module.

Output

```
Huawei LiteOS# lddrop
Huawei LiteOS#
```

If no error information is displayed, the uninstallation is successful.

9.2.10 Trace Commands

9.2.10.1 Enabling the Trace Function

Configure the trace module. For details, see [1](#).

9.2.10.2 trace_mask

Function

The **trace_mask** command is used to set the event filtering mask.

Format

trace_mask [*MASK*]

Parameter Description

Table 9-44 Parameter description

Parameter	Description	Value Range
MASK	Trace event mask	[0, 0xFFFFFFFF]

User Guide

- If the event mask is not set or the default parameter is used when this command is executed, only the task and interrupt event recording is enabled by default.
- If **MASK** is added after **trace_mask**, the event recording function of the corresponding module is enabled.
- For details about the module mask, see **LOS_TRACE_MASK** defined in **los_trace.h**.

Examples

1. Enter **trace_mask 0**.
2. Enter **trace_mask 0xFFFFFFFF**.

Output

Run the **trace_mask 0** command to set the events of all modules not to be recorded. After the command is executed successfully, no information is displayed.

```
Huawei LiteOS # trace_mask 0
```

```
Huawei LiteOS #
```

Run the **trace_mask 0xFFFFFFFF** command to set the events of all modules to be recorded. After the command is executed successfully, no information is displayed.

```
Huawei LiteOS # trace_mask 0xFFFFFFFF
```

```
Huawei LiteOS #
```

9.2.10.3 trace_start

Function

Enable the trace function.

Format

trace_start

User Guide

If **trace_start** is entered, the trace function of the system is enabled. In recording mode, events that occur in the system are recorded and saved in the specified buffer.

Example

To enable the trace function, enter **trace_start**.

Output

In offline mode, no information is displayed after the **trace_start** command is executed successfully.

```
Huawei LiteOS # trace_start
Huawei LiteOS #
```

9.2.10.4 trace_stop

Function

The **trace_stop** command is used to stop the trace function.

Format

trace_stop

User Guide

If **trace_stop** is entered, the trace function stops recording events.

Example

To stop the trace function of the system, enter **trace_stop**.

Output

No information is displayed after the **trace_stop** command is executed successfully.

```
Huawei LiteOS # trace_stop
Huawei LiteOS #
```

9.2.10.5 trace_dump

Function

This command is used to dump the trace buffer information in offline mode.

Format

trace_dump [1 | 0]

Parameter Description

Table 9-45 Parameter Description

Parameter	Parameter Description	Value Range
1	Export trace data to the client.	NA
0	Format and print trace data.	NA

User Guide

- The **trace_dump** command can be used only in offline mode.
- By default, the displayed trace data is formatted.
- The **trace_dump** command displays the data between **trace_start** and **trace_stop**. Therefore, you need to run the **trace_stop** command to stop the trace, and then run the **trace_dump** command to display the information about the trace buffer.

Example

Enter **trace_dump**.

Output

Run the **trace_dump** command to format and display the buffer data.

```
Huawei LiteOS # trace_dump
*****TraceInfo begin*****
clockFreq = 180000000
CurEvtIndex = 19
Index  Time(cycles)  EventType  CurTask  Identity  params
0      0x7da8da5180   0x45      0x5     0x2     0x9     0x20     0x1f
1      0x7dde8c6980   0x45      0x2     0x5     0x1f     0x4     0x9
2      0x7e1431df20   0x45      0x5     0x2     0x9     0x20     0x1f
3      0x7e49e3f720   0x45      0x2     0x5     0x1f     0x4     0x9
4      0x7e7f896cc0   0x45      0x5     0x2     0x9     0x20     0x1f
5      0x7eb53b84c0   0x45      0x2     0x5     0x1f     0x4     0x9
6      0x7eeae0fa60   0x45      0x5     0x2     0x9     0x20     0x1f
7      0x7f20931260   0x45      0x2     0x5     0x1f     0x4     0x9
```

```

8  0x7f56388800  0x45  0x5  0x2  0x9  0x20  0x1f
9  0x7f8beaa000  0x45  0x2  0x5  0x1f  0x4  0x9
10 0x7fc19015a0  0x45  0x5  0x2  0x9  0x20  0x1f
11 0x7ff7422da0  0x45  0x2  0x5  0x1f  0x4  0x9
12 0x802ce7a340  0x45  0x5  0x2  0x9  0x20  0x1f
13 0x806299bb40  0x45  0x2  0x5  0x1f  0x4  0x9
14 0x80983f30e0  0x45  0x5  0x2  0x9  0x20  0x1f
15 0x80cdf148e0  0x45  0x2  0x5  0x1f  0x4  0x9
.....
24 0x6c560a8d00  0x24  0x2  0x2d  0x0  0x0  0x0
25 0x6c8baf7600  0x25  0x2  0x2d  0x0  0x0  0x0
.....
36 0x71fe6f2000  0x24  0x2  0x2d  0x0  0x0  0x0
37 0x7234140900  0x25  0x2  0x2d  0x0  0x0  0x0
38 0x7269c20250  0x45  0x2  0x1  0x1f  0x4  0x0
39 0x734055a650  0x45  0x1  0x2  0x0  0x8  0x1f
40 0x7380b52450  0x45  0x2  0x1  0x1f  0x4  0x0
.....
48 0x77a6d3b300  0x24  0x2  0x2d  0x0  0x0  0x0
49 0x77dc789c00  0x25  0x2  0x2d  0x0  0x0  0x0
50 0x7812269550  0x45  0x2  0x1  0x1f  0x4  0x0
.....
*****TraceInfo end*****

```

9.2.10.6 trace_reset

Function

This command is used to clear the event data in the trace buffer in offline mode.

Format

trace_reset

User Guide

The **trace_reset** command can be used only in offline mode.

Example

Enter **trace_reset**.

Output

Run the **trace_reset** command to clear the event data.

```

Huawei LiteOS # trace_reset
Huawei LiteOS #

```

9.3 Scheduling Statistics

Function Description

Huawei LiteOS provides the scheduling statistics maintenance and debugging function. This function is used to collect some CPU scheduling information, including the idle task start time, idle task running duration, number of scheduling

switching times, number of interruption times, and number of inter-core migration times (for SMP).

How to Use

- Step 1** Choose **Debug > Enable a Debug Version > Enable DebugLiteOS Kernel Resource > Enable Scheduler Statistics Debugging** in the menu to enable the scheduling statistics maintenance and debugging function (only the platform with the floating-point operation function is supported).
- Step 2** Enter the shell command **schedstatstart** to enable the scheduling statistics function.
- Step 3** Run the shell command **schedstatstop** to disable the scheduling statistics function. The system displays the scheduling statistics of each CPU.
- Step 4** Run the shell command **schedstatinfo**. The system displays the CPU scheduling statistics of each task.

----End

Precautions

Shell commands are executed in the following sequence: **schedstatstart** > **schedstatstop** > **schedstatinfo**.

Output

- After **schedstatstart** is executed, the following information is displayed:

```
Huawei LiteOS # schedstatstart
schedule statistic start
```

- After **schedstatstop** is executed, the following information is displayed:

```
Huawei LiteOS # schedstatstop
Passed Time: +15694.394940 ms
-----
CPU      Idle(%)  ContextSwitch  HwiNum  Avg Pri  HiTask(%)  HiTask SwiNum  HiTask P(ms)  MP Hwi  MGR
-----
CPU0    +99.238067  317           5      +0.225000  +0.021202      160      +0.020797      1      2
CPU1    +98.706596  0            162    +31.000000  +0.000000      0        +0.000000      162     0
CPU2    +98.988885  3            161    +9.000000   +0.003726      2        +0.292425      161     2
CPU3    +98.736067  2            162    +9.000000   +0.001431      1        +0.224580      162     1
```

The following table describes the parameters.

Parameter	Description
Passed Time	Running duration of the scheduling statistics function.
CPU	CPU ID.
Idle(%)	Percentage of idle task running duration.
ContextSwitch	Number of task scheduling switching times.
HwiNum	Number of times that interrupts are triggered.

Avg Pri	Average priority of switch-in tasks that are not idle tasks.
HiTask(%)	Percentage of the running duration of high-priority tasks. A task whose priority is less than 16 is defined as a high-priority task.
HiTask SwiNum	Number of switching times that a high-priority task is switched in. A task with a priority lower than 16 is high-priority.
HiTask P(ms)	Average running duration of high-priority tasks. A task whose priority is less than 16 is high-priority.
MP Hwi	Number of times that inter-core interrupts are triggered. This item is used only for SMP.
MGR	Number of inter-core migration times. This parameter is used only for SMP.

- After **schedstatinfo** is executed, the following information is displayed:

```
Passed Time: +15694.394940 ms
-----
Task      TID      Total Time  Total CST  Total MGR  CPU      Time      CST      MGR
-----
Swt_Task  0x0      +2.652840 ms  156      0      CPU0      +2.652840 ms  156      0
IdleCore000  0x1      +15574.814130 ms  158      0      CPU0      +15574.814130 ms  158      0
system_wq  0x2      +0.000000 ms  0      0      CPU0      +0.000000 ms  0      0
           CPU1      +0.000000 ms  0      0
           CPU2      +0.000000 ms  0      0
           CPU3      +0.000000 ms  0      0
Swt_Task  0x4      +0.000000 ms  0      0      CPU2      +0.000000 ms  0      0
Swt_Task  0x5      +0.000000 ms  0      0      CPU3      +0.000000 ms  0      0
IdleCore000  0x6      +15522.925190 ms  1      0      CPU2      +15522.925190 ms  1      0
IdleCore000  0x7      +15495.945590 ms  1      0      CPU3      +15495.945590 ms  1      0
Swt_Task  0x8      +0.000000 ms  0      0      CPU1      +0.000000 ms  0      0
IdleCore000  0x9      +15491.321150 ms  0      0      CPU1      +15491.321150 ms  0      0
SerialShellTask  0xa      +0.804040 ms  2      2      CPU0      +0.316340 ms  1      1
           CPU1      +0.000000 ms  0      0
           CPU2      +0.487700 ms  1      1
           CPU3      +0.000000 ms  0      0
SerialEntryTask  0xb      +0.576100 ms  4      3      CPU0      +0.254370 ms  2      1
           CPU1      +0.000000 ms  0      0
           CPU2      +0.097150 ms  1      1
           CPU3      +0.224580 ms  1      1
```

The following table describes the parameters.

Parameter	Description
Passed Time	Running duration of the scheduling statistics function.
Task	Task name.
TID	Task ID.
Total Time	Total running duration of all CPU tasks.
Total CST	Number of context switching times of all CPU tasks.

Total MGR	Number of inter-core migration times of all CPUs. This parameter is used only for SMP.
CPU	ID of the CPU where the task is running.
Time	Running duration of a task on a CPU.
CST	Number of context switches of a task on a CPU.
MGR	Number of inter-core task migration times on a CPU. This parameter is used only for SMP.

9.4 Memory Debugging

9.4.1 Overview

Huawei LiteOS provides the memory maintenance and test function. You can choose **Debug > Enable a Debug Version > Enable MEM Debug** in the menu to enable the memory maintenance and test function.

After the memory maintenance and test function are enabled, the LOS_MemFree and LOS_MemRealloc APIs check the validity of the target node and the nodes before and after the target node based on the linked list. If an exception is detected, the system is suspended and an exception is thrown. In addition, the memory maintenance and test sub-functions are provided. Each function can be enabled independently. For details, see the following sections.

9.4.2 Multi-Memory Pool Mechanism

Application Scenario

If multiple dynamic memory pools are used in the system, each memory pool shall be managed and the usage statistics of each memory pool shall be collected.

Function Description

In the system memory mechanism, multiple memory pools are managed using a linked list. When the memory pool needs to be recycled, call the corresponding interface to deinitialize the memory pool.

The multi-memory pool mechanism can be used to obtain the information and usage of each memory pool in the system and check the space allocation of memory pools. When the space of two memory pools overlaps, the initialization of the second memory pool fails and a message indicating space overlapping is displayed.

Function Category	API	Description
Initialize memory	LOS_MemInit	Initializes a dynamic memory pool of the required size.
Delete memory pools	LOS_MemDelnit	Deletes a memory pool. This function takes effect only when LOSCFG_MEM_MUL_POOL is enabled.
Display system memory pools	LOS_MemPoolList	Prints information about all initialized memory pools in the system, including the start addresses of the memory pools, size of the memory pools, total size of idle memory, total size of used memory, maximum size of idle memory blocks, number of idle memory blocks, and number of used memory blocks. This function takes effect only when LOSCFG_MEM_MUL_POOL is enabled.

How to Use

1. Open the menu to enable the multi-memory pool mechanism.
This function depends on **LOSCFG_MEM_MUL_POOL**. Before using this function, enable **Enable Memory multi-pool control**.
Debug ---> Enable a Debug Version---> Enable MEM Debug---> Enable Memory multi-pool control
2. Call the LOS_MemInit API to initialize the memory pool and call the LOS_MemDelnit API to deinitialize the memory pool when the memory pool needs to be recycled.
3. Call the LOS_MemInfoGet API to obtain the information and usage of the specified memory pool.
4. Call the LOS_MemPoolList API to obtain the information and usage of all memory pools in the system.

Precautions

- Before initializing a memory pool, ensure that the memory pool spaces are not overlapped. If the memory pool spaces are overlapped, the memory pool fails to be initialized.
- By default, the malloc/free series APIs allocate and release memory from the OS memory pool. For other memory pool operations, the Huawei LiteOS memory APIs (such as LOS_MemAlloc) must be called, and the malloc/free series APIs and related encapsulation interfaces cannot be called.
- The LOS_MemDelnit API must be called to deinitialize the memory pool for recycling (ensure that all memory blocks in the memory pool have been released before the recycling). Otherwise, the memory pool space fails to be initialized again, and the memory pool cannot be used again.
- The memory pool size must be properly allocated based on site requirements.

Programming Example

```
void test(void)
{
    UINT32 ret = 0;
    UINT32 size = 0x100000;

    VOID *poolAddr1 = LOS_MemAlloc(OS_SYS_MEM_ADDR, size);
    ret = LOS_MemInit(poolAddr1, size);
    if (ret != 0) {
        PRINTK("LOS_MemInit failed\n");
        return;
    }

    VOID *poolAddr2 = LOS_MemAlloc(OS_SYS_MEM_ADDR, size);
    ret = LOS_MemInit(poolAddr2, size);
    if (ret != 0) {
        PRINTK("LOS_MemInit failed\n");
        return;
    }

    PRINTK("\n*****step1 list the mem poll\n");
    LOS_MemPoolList();

    LOS_MemDeInit(poolAddr1);
    if (ret != 0) {
        PRINTK("LOS_MemDeInit failed\n");
        return;
    }

    PRINTK("\n*****step2 list the mem poll\n");
    LOS_MemPoolList();

    LOS_MemDeInit(poolAddr2);
    if (ret != 0) {
        PRINTK("LOS_MemDeInit failed\n");
        return;
    }

    PRINTK("\n*****step3 list the mem poll\n");
    LOS_MemPoolList();
}
```

Logs:

*****step1 list the mem poll

pool0 :	pool size	used size	free size	max free node size	used node num	free node num
pool addr						
0x8017b2c0	0x100000	0x2e1fc	0xd1d20	0xd1d20	0x2b	0x1
pool1 :						
pool addr	pool size	used size	free size	max free node size	used node num	free node num
0x8027b2c0	0x7d84d40	0x7070c8	0x767db94	0x767db94	0x1026	0x1
pool2 :						
pool addr	pool size	used size	free size	max free node size	used node num	free node num
0x8078244c	0x100000	0x10	0xfff0c	0xfff0c	0x1	0x1
pool3 :						
pool addr	pool size	used size	free size	max free node size	used node num	free node num
0x8088245c	0x100000	0x10	0xfff0c	0xfff0c	0x1	0x1

*****step2 list the mem poll

pool0 :	pool size	used size	free size	max free node size	used node num	free node num
pool addr						
0x8017b2c0	0x100000	0x2e1fc	0xd1d20	0xd1d20	0x2b	0x1
pool1 :						

pool addr	pool size	used size	free size	max free node size	used node num	free node num
0x8027b2c0	0x7d84d40	0x7070c8	0x767db94	0x767db94	0x1026	0x1
pool2 :						
pool addr	pool size	used size	free size	max free node size	used node num	free node num
0x8088245c	0x100000	0x10	0xffff0c	0xffff0c	0x1	0x1
*****step3 list the mem poll						
pool0 :						
pool addr	pool size	used size	free size	max free node size	used node num	free node num
0x8017b2c0	0x100000	0x2e1fc	0xd1d20	0xd1d20	0x2b	0x1
pool1 :						
pool addr	pool size	used size	free size	max free node size	used node num	free node num
0x8027b2c0	0x7d84d40	0x7070c8	0x767db94	0x767db94	0x1026	0x1

9.4.3 Memory Backup Mechanism

Application Scenario

If an issue such as memory overwriting or releasing of wild pointer occurs, it is difficult to locate the illegitimate operation position of the memory by using the abnormal dump information.

Function Description

Back up the control header information of the dynamic memory node: Back up the control header information of the current node in the control header of the previous memory node. When allocating and releasing memory, detect the control header information and backup information of the current node. When the node control header information is overwritten but the backup information is not overwritten, output the node control header backup information and the information of the previous memory node of the overwritten node, to further analyze whether the memory overwriting is caused by overflow. When the wild pointer is released, promptly output the prompt information and call stack information to quickly locate the position where the wild pointer is released.

How to Use

Open the menu and enable the memory backup mechanism.

- Currently, only the **bestfit** memory management algorithm supports this function. You need to enable **LOSCFG_KERNEL_MEM_BESTFIT**.
Kernel ---> Memory Management ---> Dynamic Memory Management Algorithm ---> bestfit
- In addition, this function depends on **LOSCFG_MEM_HEAD_BACKUP**. You can enable the macro by enabling **Enable Node Head Backup**.
Debug ---> Enable a Debug Version ---> Enable MEM Debug ---> Enable Node Head Backup

Precautions

After this function is enabled, the system memory usage increases (Size of occupied memory = Number of nodes x Size of the node control header) and the memory operation performance is affected. It is recommended that this function be enabled only for detecting issues. By default, this function is disabled.

9.4.4 Memory Validity Check

Application Scenario

Overwriting of the service memory will cause overwriting of the control header of the memory node. However, service exceptions will not occur immediately, but after a long period of time. Therefore, the service logic is so complex that the location where memory is overwritten cannot be located easily.

Function Description

When this function is enabled, memory validity check is added to the dynamic memory allocation API to check the validity of all node control headers in the memory pool. If the dynamic memory node is overwritten, an exception is promptly triggered to display an error message and narrow down the fault location scope.

How to Use

1. Open the menu and enable memory validity check.
This function depends on **LOSCFG_BASE_MEM_NODE_INTEGRITY_CHECK**. Before using this function, enable **Enable integrity check or not**.
Debug ---> Enable a Debug Version ---> Enable MEM Debug ---> Enable integrity check or not
2. After memory overwriting occurs, the next memory allocation operation triggers an exception and the information about the overwritten node and the previous node will be provided. A preliminary analysis can be performed to determine whether the memory overwriting is caused by the overflow of the previous node. The memory overwriting occurs between the previous and the current memory allocation. You can run the shell command **memcheck** to view exception information.

Precautions

Enabling this function greatly degrades the system performance when you allocate memory. Therefore, you are advised to enable this function only for fault locating. By default, this function is disabled.

Example for Locating Memory Overwriting

Construct a **memset** operation that exceeds the memory length to construct memory overwriting. As a result, the memory node is damaged. The code is constructed as follows:

```
VOID SampleFunc(VOID *p)
{
    memset(p, 0, 0x110); // memset that exceeds the length, which is used to construct memory overwriting
}

UINT32 Test(UINT32 argc, CHAR **args)
{
    void *p1, *p2;

    p1 = LOS_MemAlloc((void*)OS_SYS_MEM_ADDR, 0x100);
    p2 = LOS_MemAlloc((void*)OS_SYS_MEM_ADDR, 0x100);
    dprintf("p1 = %p, p2 = %p \n", p1, p2);
    SampleFunc(p1); // As p1 and p2 are adjacent, when the length of memset exceeds the memory size of
```

p1, memory overwriting occurs on p2.

```

LOS_MemFree(OS_SYS_MEM_ADDR, (void *)p1);
LOS_MemFree(OS_SYS_MEM_ADDR, (void *)p2);
return 0;
}

```

After the preceding code is executed, run the shell command **memcheck**. The command output is shown in the following figure.

```

Huawei_LiteOS # test
p1 = 0x8034fafc, p2 = 0x8034fc0c 1
[ERR][OsMemIntegrityCheck], 1145, memory check error!
freeNodeInfo.pstPrev:(nil) is out of legal mem range[0x80304f80, 0x84000000]

broken node head: (nil) (nil) (nil) 0x0, pre node head: 0x7fcb0513 0x8 0x80349ad8 0x80000110

-----
dump mem tmpNode:0x8034fbfc ~ 0x8034fc3c

0x8034fbfc :00000000 00000000 00000000 00000000
0x8034fc0c :cacacaca cacacaca cacacaca cacacaca
0x8034fc1c :cacacaca cacacaca cacacaca cacacaca
0x8034fc2c :cacacaca cacacaca cacacaca cacacaca

-----
dump mem :0x8034fbbc ~ tmpNode:0x8034fbfc

0x8034fbbc :00000000 00000000 00000000 00000000
0x8034fbcc :00000000 00000000 00000000 00000000
0x8034fbdc :00000000 00000000 00000000 00000000
0x8034fbec :00000000 00000000 00000000 00000000

-----
cur node: 0x8034fbfc
pre node: 0x8034faec 2
pre node was allocated by task:SerialShellTask 3
cpu0 is in exc.
cpu1 is running.
excType:software interrupt
taskName = SerialShellTask
taskId = 8
task stackSize = 12288
system mem addr = 0x80304f80
excBuffAddr pc = 0x8020f8b4
excBuffAddr lr = 0x8020f8b8
excBuffAddr sp = 0x8036cb88
excBuffAddr fp = 0x802539d0
R0      = 0x59
R1      = 0x600f01d3
R2      = 0x0
R3      = 0x8024dc38
R4      = 0x1
R5      = 0xa00f0113
R6      = 0x8034fc0c
R7      = 0x802cc7f4
R8      = 0x8036cc1c
R9      = 0x4
R10     = 0x80369ae0
R11     = 0x802539d0
R12     = 0x1
CPSR    = 0x600f01d3

```

The preceding figure shows that the printed information is incorrect.

- As shown in mark 2, "cur node: 0x8034fbfc" indicates the node where the memory is illegally overwritten, and "pre node: 0x8034faec" indicates the node before the illegally overwritten node. As shown in mark 3, "pre node was allocated by task:SerialShellTask" indicates that memory overwriting occurs in the SerialShellTask task.
- Mark 1 displays the start addresses of the P1 and P2 memories. "p2 = 0x8034fc0c" minus the control header size 0x10 equals the address printed by the cur node (p2 - 0x10 = 0x8034fbfc), and the p2 memory is illegally overwritten. According to the code, p1 and p2 are two adjacent nodes (which can also be seen from the printed p1 and p2 addresses, that is, p1 + p1 size + control header size = p2, 0x8034fafc + 0x100 + 0x10 = 0x8034fc0c). Therefore, "pre node: 0x8034faec" should be the address of p1. The p1 address shown in

mark 1 is "p1 = 0x8034fadc", that is, the pre node plus the control header size 0x10 (0x8034faec + 0x10 = 0x8034fadc).

9.4.5 Memory Size Check

Application Scenario

When the **memset** and **memcpy** functions are called for operations on the dynamic memory, memory overwriting caused by overflow occurs.

Function Description

For the **memset** and **memcpy** operations, when the input parameter is a dynamic memory node, the system checks the actual size of the memory node and the size specified by the input parameter. If the specified size of the node is greater than the actual size, the system displays an error message and cancels the **memset** or **memcpy** operation to prevent operation violation. When dynamic memory overwriting occurs, you can enable this function to locate the fault.

API	Description
LOS_MemCheckLevelSet	Checks the memory check level.
LOS_MemCheckLevelGet	Obtains the memory check level.
LOS_MemNodeSizeCheck	Obtains the total size and available size of a specified memory block.

Error Codes

No.	Definition	Error Code	Description	Reference Solution
1	LOS_ERRNO_MEMCHECK_PARA_NULL	0x0200 0101	The LOS_MemNodeSizeCheck input parameter contains a null pointer.	Input a valid pointer.
2	LOS_ERRNO_MEMCHECK_OUTSIDE	0x0200 0102	The memory address is not in the valid range.	The input memory address is not within the memory management range. Ignore this error code.
3	LOS_ERRNO_MEMCHECK_NO_HEAD	0x0200 0103	The memory address has been released or is a wild pointer.	Verify that the input memory address is valid.

No.	Definition	Error Code	Description	Reference Solution
4	LOS_ERRNO_MEMCHECK_WRONG_LEVEL	0x02000104	The memory detection level is invalid.	Use <code>LOS_MemCheckLevelGet</code> to check the memory check level and use <code>LOS_MemCheckLevelSet</code> to configure a valid memory check level.
5	LOS_ERRNO_MEMCHECK_DISABLED	0x02000105	The memory check is disabled.	Use <code>LOS_MemCheckLevelSet</code> to enable memory check.

NOTICE

For details about the error code definition, see [Error Codes](#). Bits 8 to 15 belong to the dynamic memory module, and the value is **0x01**.

How to Use

Open the menu and enable the memory size check.

- Currently, only the **bestfit** memory management algorithm supports this function. Therefore, you need to enable **LOSCFG_KERNEL_MEM_BESTFIT**.
Kernel ---> Memory Management ---> Dynamic Memory Management Algorithm ---> bestfit
- In addition, this function depends on **LOSCFG_BASE_MEM_NODE_SIZE_CHECK**. You can enable the macro by enabling **Enable size check or not**.
Debug ---> Enable a Debug Version ---> Enable MEM Debug ---> Enable size check or not

Precautions

After this function is enabled, the performance of **memset** and **memcpy** degrades. You are advised to enable this function only for locating memory overwriting issues. By default, this function is disabled.

Example for Locating Memory Overwriting

Construct the **memset** and **memcpy** operations that exceed the memory length to construct memory overwriting. The code is constructed as follows:

```
VOID test(VOID)
{
    UINT32 size = 0x100;

    VOID *poolAddr1 = LOS_MemAlloc((VOID *)OS_SYS_MEM_ADDR, size);
    if (poolAddr1 == NULL) {
        PRINTK("malloc poolAddr1 failed\n");
        return;
    }
}
```

```

    } else {
        PRINTK("malloc poolAddr1 %p succeeded\n", poolAddr1);
    }

    VOID *poolAddr2 = LOS_MemAlloc((VOID *)OS_SYS_MEM_ADDR, size);
    if (poolAddr2 == NULL) {
        PRINTK("malloc poolAddr2 failed\n");
        return;
    } else {
        PRINTK("malloc poolAddr2 %p succeeded\n", poolAddr2);
    }

    LOS_MemCheckLevelSet(LOS_MEM_CHECK_LEVEL_LOW);    // Used to enable the check of the length
of memset and memcpy.

    PRINTK("memset poolAddr1 overflow\n");
    memset(poolAddr1, 0x10, size * 2);                // memset that exceeds the length
    PRINTK("memset poolAddr1\n");
    memset(poolAddr1, 0x10, size);                    // memset with a proper length

    PRINTK("memcpy poolAddr2 overflow\n");
    memcpy(poolAddr2, poolAddr1, size * 2);           // memcpy that exceeds the length
    PRINTK("memcpy poolAddr2\n");
    memcpy(poolAddr2, poolAddr1, size);               // memcpy with a proper length

    LOS_MemCheckLevelSet(LOS_MEM_CHECK_LEVEL_DISABLE); // Used to disable the check of the length
of memset and memcpy.

    LOS_MemFree((VOID *)OS_SYS_MEM_ADDR, (VOID *)poolAddr1);
    LOS_MemFree((VOID *)OS_SYS_MEM_ADDR, (VOID *)poolAddr2);

    return 0;
}

```

Logs:

```

malloc poolAddr1 0x80349514 succeeded
malloc poolAddr2 0x80349624 succeeded
LOS_MemCheckLevelSet: LOS_MEM_CHECK_LEVEL_LOW
memset poolAddr1 overflow
[ERR] -----
memset: dst inode availSize is not enough availSize = 0x100, memcpy length = 0x200
runTask->taskName = osMain
runTask->taskId = 64
*****backtrace begin*****
traceback 0 -- lr = 0x80209798   fp = 0x802c6930
traceback 1 -- lr = 0x80210fc4   fp = 0x802c6954
traceback 2 -- lr = 0x8020194c   fp = 0x802c6994
traceback 3 -- lr = 0x80201448   fp = 0x802c699c
traceback 4 -- lr = 0x802012fc   fp = 0x0
[ERR] -----
memset poolAddr1
memcpy poolAddr2 overflow
[ERR] -----
memcpy: dst inode availSize is not enough availSize = 0x100, memcpy length = 0x200
runTask->taskName = osMain
runTask->taskId = 64
*****backtrace begin*****
traceback 0 -- lr = 0x80209798   fp = 0x802c6930
traceback 1 -- lr = 0x8020dbc4   fp = 0x802c6954
traceback 2 -- lr = 0x8020194c   fp = 0x802c6994
traceback 3 -- lr = 0x80201448   fp = 0x802c699c
traceback 4 -- lr = 0x802012fc   fp = 0x0
[ERR] -----
memcpy poolAddr2

```

As the size check is enabled, invalid **memset** and **memcpy** operations are canceled, and error information is provided. "runTask->taskName = osMain"

indicates that the illegal operation occurs in the `osMain` function, and the values of registers **lr** and **fp** are printed. You can open the `.asm` disassembly file generated after compilation. By default, the file is generated in the **Huawei_LiteOS/out/<Platform name>** directory. Compare the value of the **lr** register to check the nested function calling.

9.4.6 Memory Leakage Check

Application Scenario

Memory leakage occurs during service running if the service logic is complex or runs for a long time.

Function Description

During memory allocation and release, the function call stack is recorded in the control header of the memory node. When a memory leakage occurs, the suspected memory leakage location can be located by analyzing the used node information.

How to Use

Step 1 Open the menu and enable memory leakage check.

- Currently, only the **bestfit** memory management algorithm supports this function. You need to enable **LOSCFG_KERNEL_MEM_BESTFIT**.
Kernel ---> Memory Management ---> Dynamic Memory Management Algorithm ---> bestfit
- In addition, this function depends on **LOSCFG_MEM_LEAKCHECK**. You can enable the macro by enabling **Enable Function call stack of Mem operation recorded**.
Debug ---> Enable a Debug Version ---> Enable MEM Debug ---> Enable Function call stack of Mem operation recorded

Step 2 Configure the call stack backtracking information in this menu path:

Debug ---> Enable a Debug Version ---> Enable MEM Debug ---> Enable Function call stack of Mem operation recorded

- LOSCFG_MEM_OMIT_LR_CNT**: Layer without call stack backtracking. The default value is 2.
- LOSCFG_MEM_RECORD_LR_CNT**: Number of call stack backtracking records. The default value is 3.

By default, the system obtains the LR information from layer 0 to 2. The system ignores layer 0 and layer 1 (the LR information at layer 0 and layer 1 of the node that calls the encapsulation interface is the same), and records LR information at layer 2.

Step 3 Run the shell command **memused** to obtain used node data.

After the system runs stably, if the number of used nodes keeps increasing, there is a very large probability that memory leaks. Compare and analyze the data, and check whether nodes with repeated LR information have memory leakage. The memory leakage can be backtracked based on the LR information.

The log information is displayed as follows (for details about the **memused** command, see [memused](#)):

```
Huawei LiteOS # memused
node      LR[0]      LR[1]      LR[2]
0x802d7b34: 0x8006d86c 0x8011c604 0x8011c758
0x802dab6c: 0x8006d16c 0x8006d8a0 0x8011c604
```

----End

Precautions

Enabling this function increases the system memory usage and has an impact on the system performance during memory operations. Therefore, you are advised to enable this function only for fault locating. By default, this function is disabled.

Memory usage size = Number of memory nodes x
(LOSCFG_MEM_RECORD_LR_CNT - LOSCFG_MEM_OMIT_LR_CNT) x
sizeof(pointer)

9.5 MMC Debugging

9.5.1 Debugging for Command Failure

Function Description

During the development of an MMC module, a command may fail to be sent. That is probably caused by incorrect configuration of the MMC protocol register. In this case, you can enable the related configuration and observe the register value to check for exceptions.

How to Use

Enable the macro **LOSCFG_DRIVERS_MMC_CMD_DUMP** and change the value of **LOSCFG_DRIVERS_MMC_OPCODE_DUMP** to select the command to be observed.

Debug ---> Enable a Debug Version ---> Enable MMC Debug ---> Enable mmc command dump

Precautions

Enabling this function increases the time required for sending MMC commands. Therefore, you are advised to enable this function only when you want to analyze the cause of a command sending failure. By default, this function is disabled.

9.5.2 Debugging for DMA Transmission

Function Description

The data receiving and sending of MMC module depend on the DMA transmission. Therefore, there are special requirements on the address and length of data buffer. If the data receiving and sending encounter exceptions due to buffer problems, you can enable related configurations and observe the value of the DMA descriptor register to check for exceptions of DMA transmission.

How to Use

Enable the macro **LOSCFG_DRIVERS_MMC_DMA_DES_DUMP**.

Debug ---> Enable a Debug Version ---> Enable MMC Debug ---> Enable mmc dma descriptors dump

Precautions

Enabling this function increases the time required for sending MMC commands. Therefore, you are advised to enable this function only when you need to analyze the cause of a command sending failure. By default, this function is disabled.

9.6 Inter-Task Communication Debugging

9.6.1 Queue Debugging

Function Description

A queue is a producer-consumer model in which a producer produces messages and puts them into a queue for consumers to use. If the queue is full, the producer is suspended. If the queue is empty, the consumer is suspended. When queues are used to transmit messages in Huawei LiteOS, you can set the timeout interval. Queues are used to implement asynchronous communication between tasks. You can run the shell command **queue** to view the queue usage.

How to Use

The **queue** command depends on **LOSCFG_DEBUG_QUEUE**. When using the **queue** command, you need to enable **Enable Queue Debugging** in **menuconfig**.

Debug ---> Enable a Debug Version ---> Enable Debug LiteOS Kernel Resource ---> Enable Queue Debugging

Example

In the shell window, run the **queue** command to display the following queue information:

```
Huawei LiteOS # queue
used queues information:
Queue ID <0x8> may leak, queue len is 0x5, readable cnt:0x0, writeable cnt:0x5, TaskEntry of creator:0x0x80242df8, Latest operation time: 0x4ee
Queue ID <0x7> may leak, queue len is 0x5, readable cnt:0x0, writeable cnt:0x5, TaskEntry of creator:0x0x80242df8, Latest operation time: 0x4ee
Queue ID <0xb> may leak, queue len is 0x5, readable cnt:0x0, writeable cnt:0x5, TaskEntry of creator:0x0x80242df8, Latest operation time: 0x4ee
Queue ID <0xa> may leak, queue len is 0x5, readable cnt:0x0, writeable cnt:0x5, TaskEntry of creator:0x0x80242df8, Latest operation time: 0x4ee
Queue ID <0x9> may leak, queue len is 0x5, readable cnt:0x0, writeable cnt:0x5, TaskEntry of creator:0x0x80242df8, Latest operation time: 0x4ee
Queue ID <0x2> may leak, queue len is 0x5, readable cnt:0x0, writeable cnt:0x5, TaskEntry of creator:0x0x80242df8, Latest operation time: 0x4ee
Queue ID <0x4> may leak, queue len is 0x5, readable cnt:0x0, writeable cnt:0x5, TaskEntry of creator:0x0x80242df8, Latest operation time: 0x4ee
Queue ID <0x3> may leak, queue len is 0x5, readable cnt:0x0, writeable cnt:0x5, TaskEntry of creator:0x0x80242df8, Latest operation time: 0x4ee
Queue ID <0x6> may leak, queue len is 0x5, readable cnt:0x0, writeable cnt:0x5, TaskEntry of creator:0x0x80242df8, Latest operation time: 0x4ee
Queue ID <0x5> may leak, queue len is 0x5, readable cnt:0x0, writeable cnt:0x5, TaskEntry of creator:0x0x80242df8, Latest operation time: 0x4ee
```

For the description of the output items, see **Output**. During the debugging, the identification item **TaskEntry of creator** in the preceding figure is used. **TaskEntry of creator** is the address (0x0x80242df8) of the API function for creating a queue. Find the address in the .asm disassembly file. By default, the disassembly file is in the **Huawei_LiteOS/out/Platform name** directory. You can view the name of the function for creating a queue, for example, app_init (0x0x80242df8) is shown in the following figure.

```

56613
56614 80242df8 <app_init>:
56615 80242df8: e3093a04 movw r3, #39428 ; 0x9a04
56616 80242dfc: e92d4070 push {r4, r5, r6, lr}
56617 80242e00: e24dd038 sub sp, sp, #56 ; 0x38
56618 80242e04: e3483020 movt r3, #32800 ; 0x8020
56619 80242e08: e5933000 ldr r3, [r3]
56620 80242e0c: e58d3034 str r3, [sp, #52] ; 0x34
56621 80242e10: e3a03000 mov r3, #0
56622 80242e14: ebff315b bl 8020f388 <uart_dev_init>
56623 80242e18: e3500000 cmp r0, #0
56624 80242e1c: 1a000059 bne 80242f88 <app_init+0x190>
56625 80242e20: e3025d78 movw r5, #11640 ; 0x2d78
56626 80242e24: ebffcf77 bl 80242208 <spinor_init>
56627 80242e28: e3485024 movt r5, #32804 ; 0x8024
56628 80242e2c: e30504e8 movw r0, #21736 ; 0x54e8
56629 80242e30: e3480020 movt r0, #32800 ; 0x8020
56630 80242e34: e3a04001 mov r4, #1
56631 80242e38: e3a06601 mov r6, #1048576 ; 0x100000
56632 80242e3c: ebffd2cd bl 80237978 <get_mtd>
56633 80242e40: e30c3028 movw r3, #49192 ; 0xc028
56634 80242e44: e3a02028 mov r2, #40 ; 0x28
56635 80242e48: e3483039 movt r3, #32825 ; 0x8039
56636 80242e4c: e3a01000 mov r1, #0
56637 80242e50: e5830000 str r0, [r3]
56638 80242e54: e28d000c add r0, sp, #12
56639 80242e58: ebff2cfa bl 8020e248 <memset>
56640 80242e5c: e58d5018 str r5, [sp, #24]

```

9.6.2 Deadlock Debugging Mechanism

Function Description

Resource access is performed in an exclusive way in many application processes. If multiple processes execute concurrently, a process may be blocked forever. When two processes wait for the resources occupied by each other, neither of them can be executed and both are in the waiting state forever, that is, the two processes are in deadlock. The deadlock detection module debugs the deadlock of spinlock, mutex, binary semaphore and pthread mutex and supports the debugging of repeated locking, incorrect lock release and lock overflow.

How to Use

Open the menu, choose **Debug > Enable a Debug Version > Enable Debug LiteOS Kernel Resource > Enable Lockdep Debugging**, and configure the deadlock detection module.

Configuration Item	Description	Value Range	Default Value	Dependency
LOSCFG_KERNEL_LOCKDEP	Determines whether to enable the deadlock debugging.	YES/NO	NO	LOSCFG_KERNEL_MM_ALLOC

Configuration Item	Description	Value Range	Default Value	Dependency
LOSCFG_KERNEL_SPINDEP	Determines whether to enable the deadlock debugging of spinlock.	YES/NO	NO	LOSCFG_KERNEL_SMP
LOSCFG_KERNEL_MUXDEP	Determines whether to enable the mutex deadlock debugging.	YES/NO	NO	LOSCFG_BASE_IPC_MUX
LOSCFG_KERNEL_SEMDEP	Determines whether to enable the deadlock debugging of binary semaphore.	YES/NO	NO	LOSCFG_BASE_IPC_SEM
LOSCFG_PTHREAD_MUXDEP	Determines whether to enable the deadlock debugging of thread lock.	YES/NO	NO	LOSCFG_COMPAT_POSIX

Example for Locating Spinlock

Step 1 After spinlock debugging function is enabled, deadlock information is printed when a deadlock is debugged. The following is an example of the information printed during deadlock debugging:

Figure 9-24 Information printed by the deadlock module

```
ERR!lockdep check failed
ERR!error type: dead lock!
ERR!request addr: 0x80204028
ERR!task name: it_smpl_lockdep_004_2
ERR!task id: 7
ERR!cpu num: 1
ERR!start dumping lockdep information
ERR!0) a testSpin2
ERR!1) a testSpin2
ERR!task name: it_smpl_lockdep_004_1
ERR!task id: 6
ERR!cpu num: 0
ERR!start dumping lockdep information
ERR!0) a testSpin2
ERR!1) a testSpin2
ERR!2) a testSpin2
OshackTrace fp = 0x8034500
runTask(taskName = it_smpl_lockdep_004_2)
runTask(taskId = 7)
*****backtrace begins*****
backtrace 0 - Ir = 0x8020404c fp = 0x80345014
backtrace 1 - Ir = 0x8020404c fp = 0x80345044
backtrace 2 - Ir = 0x80204028 fp = 0x8034505c
backtrace 3 - Ir = 0x80204030 fp = 0x80345064
backtrace 4 - Ir = 0x8020402c fp = 0x8034506b
```

Name	TaskEntryAddr	TID	Affinity	CPU	Priority	Status	StackSize	WaterLine	StackPoint	TopOfStack	EventMask	SenID	CPURISE	CPURISE10s	CPURISE1s	MEMUSE
IdleCore000	0x80204054	0x0	0x0002	-1	31	Ready	0x800	0x238	0x80345038	0x80345070	0x0	0xffffffff	0.0	0.0	0.0	0
Test_Task	0x8020407c	0x1	0x0001	-1	0	Pend	0x6000	0x238	0x80397440	0x80397468	0x0	0xffffffff	0.0	0.0	0.0	0
IdleCore000	0x80204084	0x2	0x0001	-1	31	Ready	0x800	0x148	0x80397480	0x80397738	0x0	0xffffffff	0.0	0.0	0.0	0
devtrace_wd	0x8024016c	0x3	0x0003	-1	1	Pend	0x6000	0x22c	0x80397480	0x80397498	0x1	0xffffffff	0.0	0.0	0.0	0
it_smpl_lockdep_004_2	0x80204090	0x4	0x0001	-1	25	Delay	0x3000	0x244	0x80345038	0x80397498	0x0	0xffffffff	0.0	0.0	0.0	3112
Test_Task	0x8020407c	0x5	0x0002	-1	0	Pend	0x6000	0x238	0x80345038	0x80340010	0x0	0xffffffff	0.0	0.0	0.0	0
it_smpl_lockdep_004_1	0x80204084	0x6	0x0001	0	25	Running	0x600	0x148	0x80345038	0x80345050	0x0	0xffffffff	0.0	0.0	0.0	0
it_smpl_lockdep_004_2	0x80204090	0x7	0x0002	1	25	Running	0x600	0x148	0x80345038	0x80345050	0x0	0xffffffff	0.0	0.0	0.0	0

- Step 2** Copy the value of **request addr** (0x802a6528 in this example) and find the corresponding address in the **.asm** disassembly file of the system image. By default, the file compiled by the Make tool is generated in the **Huawei_LiteOS/out/<platform name>** directory and the file compiled by the CMake tool is generated in the **Huawei_LiteOS/buildcmakeout/<platform name>** directory. As shown in the following figure, the location where spinlock is called and the called function (task_fx02 in this example) can be located.

Figure 9-25 Finding corresponding addresses in the disassembly file

```

802a64c4 <task_fx02>:
802a64c4: e92d48f0      push    {r4, r5, r6, r7, fp, lr}
802a64c8: e28db014      add     fp, sp, #20
802a64cc: e10f3000      mrs     r3, CPSR
802a64d0: f10c00c0      cpsid   if
802a64d4: e30d7070      movw    r7, #53360      ; 0xd070
802a64d8: e30d6074      movw    r6, #53364      ; 0xd074
802a64dc: e3487034      movt    r7, #32820      ; 0x8034
802a64e0: e3095514      movw    r5, #38164      ; 0x9514
802a64e4: e3486034      movt    r6, #32820      ; 0x8034
802a64e8: e3094504      movw    r4, #38148      ; 0x9504
802a64ec: e3485033      movt    r5, #32819      ; 0x8033
802a64f0: e3484033      movt    r4, #32819      ; 0x8033
802a64f4: ebfd896c      bl      80208aac <LOS_TaskLock>
802a64f8: e1a00005      mov     r0, r5
802a64fc: ebfd9f55      bl      8020e258 <OsLockDepCheckIn>
802a6500: e1a00005      mov     r0, r5
802a6504: eb000dfc      bl      802a9cfc <OsSpinLock>
802a6508: e1a00005      mov     r0, r5
802a650c: ebfd9f9a      bl      8020e37c <OsLockDepRecord>
802a6510: e5973000      ldr     r3, [r7]
802a6514: e2833001      add     r3, r3, #1
802a6518: e5873000      str     r3, [r7]
802a651c: ebfd8962      bl      80208aac <LOS_TaskLock>
802a6520: e1a00004      mov     r0, r4
802a6524: ebfd9f4b      bl      8020e258 <OsLockDepCheckIn>
802a6528: e1a00004      mov     r0, r4
802a652c: eb000df2      bl      802a9cfc <OsSpinLock>
802a6530: e1a00004      mov     r0, r4
802a6534: ebfd9f90      bl      8020e37c <OsLockDepRecord>
802a6538: e5973000      ldr     r3, [r7]
802a653c: e1a00004      mov     r0, r4
802a6540: e2833001      add     r3, r3, #1
802a6544: e5873000      str     r3, [r7]
802a6548: ebfd9fc7      bl      8020e46c <OsLockDepCheckOut>
  
```

- Step 3** Based on the spinlock in the second blue box of the deadlock information shown in Figure 1, find the spinlock owned by another task based on the code logic, and then adjust the spinlock calling time sequence based on the code to debug the deadlock.

----End

NOTICE

The spinlock deadlock occurs when a CPU is suspended and tasks can no longer be scheduled. If the spinlock debugging is not enabled, you can use a debugging tool such as JLink to halt the CPU or trigger a watchdog exception. Then, check whether the PC value is the spinlock code to determine whether the spinlock occurs.

Example for Mutex Deadlock Locating

Construct the ABBA mutex deadlock scenario as follows:

The system has two tasks **Task_A** and **Task_B**. In addition, there are other default initial tasks in the system. Allocate **Mutex1** and **Mutex0** in **Task_A** respectively, and then allocate **Mutex1** and **Mutex0** in **Task_B** to construct the ABBA deadlock scenario.

The code is as follows:

```
#include "los_mux.h"

static UINT32 mutexTest[2];
UINT32 g_TestTaskID01, g_TestTaskID02;

static VOID Task_A(VOID)
{
    UINT32 ret;
    ret = LOS_MuxPend(mutexTest[0], LOS_WAIT_FOREVER);
    if (ret == LOS_OK) {
        dprintf("%s pend muxid %d\n", __func__, mutexTest[0]);
    }

    LOS_TaskDelay(0x2);

    ret = LOS_MuxPend(mutexTest[1], 6000); // Scheduled blocking mode (6000 ticks)
    if (ret == LOS_ERRNO_MUX_TIMEOUT) {
        dprintf("%s pend muxid %d\n", __func__, mutexTest[1]);
    }

    return;
}

static VOID Task_B(VOID)
{
    UINT32 ret;
    ret = LOS_MuxPend(mutexTest[1], LOS_WAIT_FOREVER);
    if (ret == LOS_OK) {
        dprintf("%s pend muxid %d\n", __func__, mutexTest[1]);
    }
    LOS_TaskDelay(0x2);
    ret = LOS_MuxPend(mutexTest[0], 6000); // Scheduled blocking mode (6000 ticks)
    if (ret == LOS_ERRNO_MUX_TIMEOUT) {
        dprintf("%s pend muxid %d\n", __func__, mutexTest[0]);
    }
    return;
}

UINT32 mutexDeadlock(VOID)
{
    UINT32 ret;
    TSK_INIT_PARAM_S stTask1 = {0};
    stTask1.pfnTaskEntry = (TSK_ENTRY_FUNC)Task_A;
    stTask1.uwStackSize = LOSCFG_BASE_CORE_TSK_MIN_STACK_SIZE;
    stTask1.pcName = "Task_A";
    stTask1.usTaskPrio = 9;

    TSK_INIT_PARAM_S stTask2 = {0};
    stTask2.pfnTaskEntry = (TSK_ENTRY_FUNC)Task_B;
    stTask2.uwStackSize = LOSCFG_BASE_CORE_TSK_MIN_STACK_SIZE;
    stTask2.pcName = "Task_B";
    stTask2.usTaskPrio = 9;

    ret = LOS_MuxCreate(&mutexTest[0]);
    ret |= LOS_MuxCreate(&mutexTest[1]);
    if (ret != LOS_OK) {
        PRINT_ERR("create mutex error %u\n", ret);
    }

    ret = LOS_TaskCreate(&g_TestTaskID01, &stTask1);
    ret |= LOS_TaskCreate(&g_TestTaskID02, &stTask2);
    if (ret != LOS_OK) {
```

```

    PRINT_ERR("create task error %u\n", ret);
}

LOS_TaskDelay(3000); // Enable the task to enter sleep mode for 3000 ticks.

ret = LOS_MuxDelete(mutexTest[0]);
ret = LOS_MuxDelete(mutexTest[1]);

LOS_TaskDelete(g_TestTaskID01);
LOS_TaskDelete(g_TestTaskID02);
return LOS_OK;
}

```

If the problem occurs in the preceding scenario, the following information is dumped:

```

Task_A pend muxid 3
Task_B pend muxid 4
lockdep check failed
error type : dead lock
request addr : 0x4200a08c
task name : Task_B
task id : 11
cpu num : 3
start dumping lockdep information
[0] Mutex ID: 0004
[1] Mutex ID: 0003 <-- now
task name : Task_A
task id : 10
cpu num : 0
start dumping lockdep information
[0] Mutex ID: 0003
[1] Mutex ID: 0004 <-- now
[ERR] The current exception level is EL1
/*****Exception Information*****/
ExcClass:0x25 => Data Abort.
Abort caused by a read instruction.
OMG!UNKNOWN fault, check ISS encoding for an exception from a Data Abort in AARM for
PC at 0x420083b4,FAR 0x0,ISS 0x6
ESR 0x96000006: ec 0x25, il 0x2000000, iss 0x6
taskName = Task_B
TaskId = 11
Task StackSize = 2048
system mem addr = 0x422938c0
PC = 0x420083b4

```


Name	TaskEntryAddr	TID	Affi	CPU	Priority	Status
----	-----	---	-----	----	-----	-----
Swt_Task	0x0000000042008e80	0x0	0x0001	0	0	Suspend
IdleCore000	0x000000004200aa40	0x1	0x0001	0	31	Running
system_wq	0x0000000042012de0	0x2	0x000f	0	1	Pend
app_Task	0x0000000042006f20	0x3	0x0001	0	10	Delay
Swt_Task	0x0000000042008e80	0x4	0x0008	3	0	Suspend
Swt_Task	0x0000000042008e80	0x5	0x0004	2	0	Suspend
IdleCore000	0x000000004200aa40	0x6	0x0008	3	31	Running
Swt_Task	0x0000000042008e80	0x7	0x0002	1	0	Suspend
IdleCore000	0x000000004200aa40	0x8	0x0004	2	31	Ready
IdleCore000	0x000000004200aa40	0x9	0x0002	1	31	Running
Task_A	0x0000000042006fd0	0xa	0x000f	2	9	PendTime
Task_B	0x0000000042006f30	0xb	0x000f	2	9	Running

The operations are as follows:

- Step 1** According to the information, a deadlock occurs between **Task_A(0x42006fd0)** and **Task_B(0x42006f30)**.
- Step 2** Find the corresponding task function in the disassembly file.

Take **Task_A(0x42006fd0)** as an example. The following figure shows the operation of locating the MuxPend and the called API.

```

0000000042006fd0 <Task_A>:
42006fd0: a9be7bfd stp x29, x30, [sp, #-32]!
42006fd4: 12800001 mov w1, #0xffffffff // #-1
42006fd8: 910003fd mov x29, sp
42006fdc: f9000bf3 str x19, [sp, #16]
42006fe0: f0001313 adrp x19, 42269000 <g_taskCBArray+0xb4a0>
42006fe4: b9444a60 ldr w0, [x19, #1096]
42006fe8: 940004de bl 42008360 <LOS_MuxPend>
4200fec: 340001e0 cbz w0, 42007028 <Task_A+0x58>
4200ff0: 91112273 add x19, x19, #0x448
4200ff4: 52800040 mov w0, #0x2 // #2
4200ff8: 94000df2 bl 4200a7c0 <LOS_TaskDelay>
4200ffc: 5282ee01 mov w1, #0x1770 // #6000
42007000: b9400660 ldr w0, [x19, #4]
42007004: 940004d7 bl 42008360 <LOS_MuxPend>
42007008: 5283a0e1 mov w1, #0x1d07 // #7431
4200700c: 72a04001 movk w1, #0x200, lsl #16
42007010: 6b01001f cmp w0, w1
42007014: 540001a0 b.eq 42007048 <Task_A+0x78> // b.none
42007018: f9400bf3 ldr x19, [sp, #16]
4200701c: a8c27bfd ldp x29, x30, [sp], #32
42007020: d65f03c0 ret

```

Step 3 Confirm the deadlock cause based on the context and adjust the time sequence for the allocated lock.

----End

Complete Code

sample_mux_dlock.zip

9.7 Exception Interaction

Application Scenario

If an exception occurs during the OS running, you can run the shell command to further locate the exception cause and check the system status.

Function Description

When an exception occurs in the system, the basic interaction function of the shell is retained.

Configuration Item

Configuration Item	Description	Value Range	Default Value	Dependency
LOSCFG_EXC_INTERACTION	Determines whether to enable the exception interaction function.	YES/NO	NO	LOSCFG_SHELL && LOSCFG_KERNEL_CONSOLE

How to Use

Step 1 Open the menu to enable the exception interaction function.

To enable this function, set **LOSCFG_EXC_INTERACTION** to **y**. This function is disabled by default. If this function is disabled, exception interaction becomes invalid.

The menu items for enabling this macro are as follows:

Debug ---> Enable a Debug Version ---> Enable Shell ---> Enable exc interaction

Step 2 Enter **help** to view the supported shell commands.

----End

Precautions

- When the system is abnormal, the exception interaction function enables only the serial port interrupt and disables other interrupts. The tasks created by users are also deleted, and only the basic tasks used for exception interaction are retained.
- The exception interaction is a debug function and is disabled by default. It must not be used in formal Huawei LiteOS.
- Disclaimer: Huawei is not liable for any risks caused by the use of this function in commercial products.

9.8 Low Power Consumption Debugging Methods

9.8.1 Enabling Low-power Commissioning

Huawei LiteOS provides the low-power maintenance and test function. You can choose **Debug > Enable a Debug Version > Enable Debug LiteOS Kernel Resource > Enable Lowpower Sleep Debugging** on the menu to enable the low-power maintenance and test function. Before enabling the low-power maintenance and test function, you need to open the **menuconfig** menu to enable the shell. For details, see [Configuration Items](#). At the same time, you need to enable the low-power framework. For details, see section 4.3. This function is used only in the development phase. Disable this function before the commercial use of the official version.

9.8.2 Querying System Status

Function Description

The system status query command displays detailed information about interrupts, timers, and tasks. This command is used together with the command for querying blocked task deep sleep events and the system RunStop dotting command to facilitate fault locating. For details about the effect of the **hwi**, **swtmr**, and **task** commands, see [System Commands](#).

How to Use

```
lpm sysinfo
```

```

Huawei LiteOS # lpm sysinfo
InterruptNo      Share      ResponseCount      Name      DevId
-----
0                N        0
1                N        44
2                N        0
29               N        26
36               N        0
38               N        16                uart_pl011      0x802b

SwTmrID      State      Mode      Interval      Arg      handlerAddr
-----
0x00000000    Ticking    Period    100          0x00000000    0x80204688

Name      TaskEntryAddr      TID      Affi      CPU      Pr
----
Swt_Task      0x802062fc          0x0      0x0001     -1      0
IdleCore000    0x80207064          0x1      0x0001     -1      31
system_wq      0x8022066c          0x2      0x0003     -1      1
Swt_Task      0x802062fc          0x4      0x0002     -1      0
IdleCore000    0x80207064          0x5      0x0002      1      31
SendToSer      0x8020e3b0          0x6      0x0003     -1      10
SerialShellTask 0x802689c0          0x7      0x0003      0      9
SerialEntryTask 0x802682f0          0x8      0x0003     -1      9

```

Precautions

The **hwi** and **swtmr** commands must be supported for displaying the interrupt and timer information. For details, see [System Commands](#).

9.8.3 Controlling Dynamic Low-power Consumption

Function Description

The dynamic low-power consumption can be manually enabled, disabled, or queried. It is enabled by default.

How to Use

1. Run the **lpm enable** command to enable the dynamic low-power consumption.
2. Run the **lpm disable** command to disable the dynamic low-power consumption.
3. Run the **lpm status** command to query the status of the dynamic low-power consumption.

Precautions

None.

9.8.4 Blocking System Deep Sleep Event Query

Function Description

Blocking system deep sleep event refers to the event that the system cannot enter the deep sleep mode after the user call **getSleepMode** to return to the deep sleep mode. The possible causes are as follows: The remaining time before the timer or task expires does not reach the minimum time threshold required for deep sleep. The task is blocked upon voting.

How to Use

1. Run the **lpm blockinfo cpuid** command to query the events that blocks the system from entering the deep sleep mode after startup. **cpuid** indicates the CPU ID.
2. Run the **lpm resetblockinfo cpuid** command to clear statistics about events that block the system deep sleep.

Example:

The information can be obtained from the following figure:

1. The task whose **id** is **3** and **cpuid** is **0** blocks the system from entering the deep sleep mode twice, and the task address is 0x8008ced.
2. The swtmr whose **id** is **2** and **cpuid** is **0** blocks the system from entering the deep sleep mode once, and the timer task address is 0x800fc91.
3. (Normal, 3), (Light, 3), and (Deep, 3) each blocks a deep sleep.

```
Huawei LiteOS # lpm blockinfo 0
deep sleep block info:
-----
cpuId      type      id      blockCount  handler
-----
0          task      3        2          0x8008ced
0          swtmr     2        1          0x800fc91
user veto block deep sleep info:
-----
sleepMode  requestId  blockCount
-----
Normal     3          1
Light      3          1
Deep       3          1
```

Precautions

1. Currently, only UP is supported. Therefore, the value of **cpuid** can only be **0**, that is, **lpm blockinfo 0** or **lpm resetblockinfo 0**.
2. Statistics are recorded based on the timer ID or task ID. If the ID of a task or timer to be destroyed is the same as that of a task or timer to be created, the task or timer may be counted as one task or timer. Therefore, avoid this scenario. For example, the ID of task A is 8. After task A is executed, task A is

destroyed and blocks deep sleep for three times. After a period of time, task B is created and its ID is 8. It blocks deep sleep for twice. In the statistics, the number of blocking times is 3 for the task whose type is task and ID is 8, and the handler is the handler of task B.

3. If a deep sleep is blocked, all factors that do not meet the deep sleep conditions at the moment are counted.

9.8.5 System RunStop Dotting

Function Description

The system RunStop dotting function collects statistics on sleep timestamps in normal, light, and deep modes. The information includes the CPU ID, sleep mode, sleep start time, sleep end time, true sleep time, and wakeup interrupt. The default number of records is 10. The value of the wakeup interrupt during deep sleep is fixed at **4294967295**.

How to Use

1. Run **lpm timestamp cpuid** to query the dotting information about system RunStop. **cpuid** indicates the CPU ID.
2. You can change the maximum number of records by setting the **LOSCFG_LOWPOWER_RECORD_COUNT** parameter. The maximum value is **1000**.

Example:

```
Huawei LiteOS # lpm timestamp 0
cpuId    sleepMode    startTicks    endTicks    sleepTicks    InterruptNo
-----
0         Light        5300          5399        99            29
0         Light        5400          5499        99            29
0         Light        5500          5599        99            29
0         Light        5600          5699        99            29
0         Light        5700          5781        81            38
0         Normal       5788          5799        11            29
0         Light        5800          5875        75            38
0         Normal       5882          5899        17            29
0         Light        5900          5938        38            38
0         Light        5945          5996        51            38
```

Precautions

1. Currently, only UP is supported. Therefore, the value of **cpuid** can only be **0**, that is, **lpm timestamp 0**.
2. The system RunStop dotting statistics are the actual sleep timestamp information. As shown in the preceding figure, the configured light sleep time is 100, but the actual light sleep time is 99.

9.9 Usage of Dying Gasp

Function Description

The dying gasp log storage hook function is used to register the read and write functions for log storage. For details about the APIs, see the API reference.

API	Description
LOS_ExcInfoRegHook	Registers the hook function for reading and writing dying gasp logs, and sets the start address, size, and buffer for recording dying gasp logs.

NOTE

LOS_ExcInfoRegHook has four parameters:

- **startAddr**: indicates the start address for storing dying gasp logs.
- **space**: indicates the size of the storage space for dying gasp logs.
- **buf**: indicates the buffer address of the dying gasp logs in the memory. The size of **buffer** is the same as that of **space**.
- **LogReadWriteFunc**: indicates the hook function for reading and writing exception information. The input parameters of this function include **startAddr**, **space**, **rwFlag**, and **buf**. **rwFlag** is the Read/Write flag. The value **0** indicates write, and the value **1** indicates read. The other three input parameters are the same as those of LOS_ExcInfoRegHook.

How to Use

Step 1 Open the menu and enable the dying gasp recording function, that is, set **LOSCFG_SHELL_EXCINFO_DUMP** to **y**. This function is disabled by default. The menu path is **Kernel > Exception Management > Enable Shell excInfo**.

Step 2 Register a read/write hook function.

1. Compile the hook function for reading and writing dying gasp logs. The following codes are as an example:

```
#include "los_base.h"
#ifdef (LOSCFG_SHELL_EXCINFO) && defined(LOSCFG_DRIVERS_MTD_SPI_NOR)
#include "linux/mtd/mtd.h"
#include "linux/module.h"
#include "linux/mtd/mtd_list.h"
#include "spinor.h"
#endif
#include "los_hwi.h"
#ifdef LOSCFG_FS_VFS
#include "fs/fs.h"
#endif

#ifdef LOSCFG_SHELL_EXCINFO_DUMP
STATIC struct mtd_info *g_mtdSpinor = NULL;
STATIC VOID OsSpiflashErase(UINT32 start, size_t size)
{
    struct erase_info eraseInfo;
    (VOID)memset_s(&eraseInfo, sizeof(struct erase_info), 0, sizeof(struct erase_info));
```

```

    eraseInfo.mtd = g_mtdSpinor;
    eraseInfo.callback = NULL;
    eraseInfo.fail_addr = (UINT64)MTD_FAIL_ADDR_UNKNOWN;
    eraseInfo.addr = start;
    eraseInfo.len = size;
    eraseInfo.time = 1;
    eraseInfo.retries = 1;
    eraseInfo.dev = 0;
    eraseInfo.cell = 0;
    eraseInfo.priv = 0;
    eraseInfo.state = 0;
    eraseInfo.next = NULL;
    eraseInfo.scrub = 0;
    (VOID)g_mtdSpinor->erase(g_mtdSpinor, &eraseInfo);
}
STATIC INT32 OsWriteExcInfoToSpiFlash(UINT32 startAddr, UINT32 space, const CHAR *buf)
{
    UINT32 outLen;
    OsSpiFlashErase(startAddr, LOS_Align(space, g_mtdSpinor->erasesize));
    return g_mtdSpinor->write(g_mtdSpinor, (loff_t)startAddr, space, &outLen, buf);
}
STATIC INT32 OsReadExcInfoForSpiFlash(UINT32 startAddr, UINT32 space, CHAR *buf)
{
    UINT32 outLen;
    return g_mtdSpinor->read(g_mtdSpinor, (loff_t)startAddr, space, &outLen, buf);
}

/* Be called in the exception. */
VOID OsReadWriteExceptionInfo(UINT32 startAddr, UINT32 space, UINT32 flag, CHAR *buf)
{
    if ((buf == NULL) || (space == 0)) {
        PRINT_ERR("buffer is null or space is zero\n");
        return;
    }

    g_mtdSpinor = get_mtd("spinor");
    if (g_mtdSpinor == NULL) {
        (VOID)spinor_init();
        g_mtdSpinor = get_mtd("spinor");
        if (g_mtdSpinor == NULL) {
            PRINT_ERR("Init spinor is failed\n");
            return;
        }
    }

    if (flag == 0) {
        if (OsWriteExcInfoToSpiFlash(startAddr, space, buf) != LOS_OK) {
            PRINT_ERR("Exception information written to 0x%x flash failed\n", startAddr);
        }
    }
#ifdef LOSCFG_EXC_INTERACTION
    /*
     * When system is in the exception interaction, this buf was free,
     * but this feature is still running. This buffer may be used again
     * without malloc.
     * So, consider whether or not the "buf" is released according to actual use.
     */
    free(buf);
#endif
    } elseif (flag == 1) {
        if (OsReadExcInfoForSpiFlash(startAddr, space, buf) != LOS_OK) {
            PRINT_ERR("Exception information read from 0x%x flash failed\n", startAddr);
        }
    } else {
        PRINT_ERR("flag is error\n");
    }
}
#endif
#endif

```


2. Register the hook function in the initialization function such as `app_init`. The following codes are as an example:

```
#ifndef LOSCFG_SHELL_EXCINFO_DUMP
#define EXCINFO_RECORD_BUF_SIZE    (16 * 1024) /* user-defined */
#define EXCINFO_RECORD_ADDR        (0xffffffff) /* The value is invalid and needs to be defined by
users. */
CHAR *buf = (CHAR *)malloc(EXCINFO_RECORD_BUF_SIZE);
if (buf != NULL) {
    (VOID)memset_s(buf, EXCINFO_RECORD_BUF_SIZE, 0, EXCINFO_RECORD_BUF_SIZE);
    LOS_ExcInfoRegHook(EXCINFO_RECORD_ADDR, EXCINFO_RECORD_BUF_SIZE, buf,
OsReadWriteExceptionInfo);
} else {
    PRINTK("shell excinfo malloc failed!\n");
}
#endif
#ifdef LOSCFG_FS_VFS
    los_vfs_init();
#endif
#endif
```

NOTICE

- The `OsReadWriteExceptionInfo` function is a NOR flash read/write function. The `EXCINFO_RECORD_BUF_SIZE` and `EXCINFO_RECORD_ADDR` macros need to be configured by users based on site requirements. It is recommended that the code be placed in front of the user service code.
- Before data is written to the NOR flash, the block needs to be erased. The block size is 64 KB. Therefore, the 64 KB area behind the start address for the dying gasp storage is erased when the dying gasp information is written. You are advised not to save data in this area.

Step 3 View error information.

After the system is reset and restarted, run the **exclInfo** command in the shell window to print the recorded dying gasp logs.

----End

Precautions

You need to maintain the buffer for storing dying gasp information. When calling a registration function repeatedly, you need to release the buffer generated during the previous registration.

9.10 Usage of Magic Keys

Application Scenario

When the system does not display crash information and does not respond, use magic keys to check whether the system responds to an interrupt. If the system has no response, check the CPU usage (`cpup`) in the task information and identify the task that occupies the CPU for a long period to cause the system crash. If high-priority tasks are using the CPU, low-priority tasks will not respond.

Function Description

Magic keys can be used to identify special keys and output relevant information during the UART and USB-to-virtual serial port interrupts.

How to Use

Step 1 Open the menu and enable the magic key function.

The magic key is disabled by default. To enable this function, set **LOSCFG_ENABLE_MAGICKEY** to **y**. Otherwise, the magic key becomes invalid. To enable the macro, choose the following menu item:

Debug ---> Enable MAGIC KEY

NOTE

You can move the cursor to this function in **menuconfig**, and enter **?** to view help information.

Answer **Y** to enable LiteOS Magic key.

ctrl + r : Magic key check switch;
ctrl + z : Show all magic op key;
ctrl + t : Show task information;
ctrl + p : System panic;
ctrl + e : Check system memory pool.

Step 2 Press the combined key **Ctrl+r** to enable the detection function of a magic key.

When the UART or USB-to-virtual serial port is connected, press the combined key **Ctrl+r**. The detection function of a magic key is enabled and output "Magic key on". If you press the combined key once again, the detection function is disabled and output "Magic key off". Magic keys can achieve the following functions:

- **Ctrl+z**: Help key, which is used to display the brief introduction to related magic keys.
- **Ctrl+t**: displays task information.
- **Ctrl+p**: The system proactively enters the panic state. After the panic-related information is printed, the system is suspended.
- **Ctrl+e**: The system checks the integrity of the memory pool. If an error is detected, the system displays an error message. If no error is detected, the system displays "system memcheck over, all passed!".

----End

Precautions

When the detection function of a magic key is enabled, you need to ensure that special characters entered through UART or USB-to-virtual serial ports differ from the magic key values; otherwise, a magic key is incorrectly triggered, and existing design functions may have exceptions.

9.11 Usage of the Stack Estimation Tool

Application Scenario

For embedded devices with limited memory resources, you can use the stack estimation tool to analyze the task stack for accuracy in use of the stack space.

- When creating a task, you can use the tool to analyze the task stack for accuracy in setting its size.
- When allocating the memory, you can use the tool to estimate the interrupt stack for accuracy in setting its size.
- You can use the tool to analyze task stack overflows.

Function Description

The stack estimation tool is a Python-based offline static analysis tool for function stacks. By parsing the disassembly file, it calculates the maximum stack space usage during task running to provide reference data for static analysis in scenarios such as task stack size setting, stack overflow analysis, and stack memory optimization. In addition, the tool reports recursive function alarms and displays function calling relationship trees.

Operation Mechanism

The stack estimation tool parses the disassembly file row by row, converts the jump instruction into a function calling relationship, parses the stack opening instruction into the local space overhead of the function stack, and finally traverses the task entry function and all the subfunctions through depth-first search to obtain the peak value of the task stack memory usage.

How to Use

The name of the stack estimation tool is `stackusage.py`. Before using the tool, you can run the **python3 stackusage.py -h** command to view help information. The stack estimation tool supports Python 3.6 or later. If Python 3.6 or a later version is not installed, install it by referring to [Python installation guide](#) on the official website.

```
00451438@ubuntu:~/liteos_test/smp_mainline/Huawei_LiteOS/tools/stackusage$ python3.8 stackusage.py -h
usage: stackusage.py [-h] [-m N] [-sp] [-t taskEntryName [taskEntryName ...]] [-out] [-j] [-r N] HuaweiLiteOS.asm

StackUsage tools. Estimate task stack size by disassembly file. Already support Arm and Risc-V platform

positional arguments:
  HuaweiLiteOS.asm      Disassembly file path

optional arguments:
  -h, --help            show this help message and exit
  -m N, --analyze_mode N
                        analyze_mode, 0:Only analyze assembly code; 1:Analyze source code line number. 2:Analyze source code in disassembly file.
  -sp, --specify        Analyze specified task entry. (default: analyze all function)
  -t taskEntryName [taskEntryName ...], --taskname taskEntryName [taskEntryName ...]
                        Specified task entry name. Can be omitted if analyze all.
  -out, --outputconsole
                        Output result to console.
  -j, --outputjson      Output result to json file.
  -r N, --rec-stats N   Recursive function loop cycles.
```

Step 1 (Optional) Generate a disassembly file.

If no disassembly file is available, you can use the **elf** file to generate one. For example, if the **Huawei_LiteOS.elf** file is available, run the following command to generate a disassembly file named **Huawei_LiteOS.asm**:

```
arm-none-eabi-objdump -d Huawei_LiteOS.elf > Huawei_LiteOS.asm
```

Step 2 (Optional) Modify the configuration file.

Sometimes, statistics about the stack overhead of the troubleshooting method in the error branch are worthless. The stack estimation tool supports optionally ignoring some assertion function, exception handling functions, and print functions. During statistics collection, the stack overhead of these functions is set to **0**.

Add the functions to be ignored to the end of the **Ignore_function** line. If there are multiple functions when modifying the **cfg/config.ini** file, separate multiple functions with spaces. The tool automatically parses the file during execution.

```
# Ignore some assertions, exception methods, or printing methods. Set their stack cost to 0.
Ignore_function: __stack_chk_fail LOS_Panic LOS_BackTrace PrintExcInfo
Ignore_function: dprintf OsVprintf
```

Step 3 Run the stack estimation tool.

If the disassembly file is **Huawei_LiteOS.asm** and you want to use the tool to estimate the task stack size of the **OsIdleTask** task, run the following command:

```
python3 stackusage.py -sp -t OsIdleTask -out Huawei_LiteOS.asm
```

The following figure shows the command output.

The screenshot displays the output of the stack usage tool. It includes a function calling relationship configuration file, a recursive function alarm, a calculation result, and a calling relationship tree. The tree shows the stack cost for the **OsIdleTask** and its sub-tasks, with a recursion mark for ***Rec**.

```
Read manual funcPtr configuration file! ./cfg/funcptr.txt
[Recursive function]: Recursive function alarm
<LOS_TaskUnlock>:
<LOS_Schedule>:
<OsSchedPreempt>:
<OsSchedResched>:
<OsTaskSwitchCheck>:
<LOS_TickCountGet>:
<LOS_SpinUnlockRestore>:
<LOS_SpinUnlock>:

function <OsIdleTask>: stack cost = 476
symbol task_entry task_stack_max_cost local_cost
symbol +---<OsIdleTask>: 476 8
+---<LOS_TaskResRecyle>: 468 20
+---<LOS_SpinUnlockRestore>: 400 20
+---<LOS_SpinUnlock>: 380 20
+---<ArchSpinUnlock>: 0 0
+---<LOS_TaskUnlock>: 360 28 *Rec
+---<LOS_IntRestore>: 36 20
+---<ArchIntRestore>: 16 16
+---<LOS_SpinLockSave>: 96 20
+---<LOS_SpinLock>: 76 20
+---<LOS_TaskLock>: 56 20
+---<OsPercpuGet>: 24 8
+---<ArchCurrCpuId>: 16 16
+---<LOS_IntRestore>: 36 20
+---<ArchIntRestore>: 16 16
+---<LOS_IntLock>: 24 8
+---<ArchIntLock>: 16 16
+---<ArchSpinLock>: 0 0
+---<LOS_IntLock>: 24 8
```

The tool output consists of four parts: function calling relationship configuration file, recursive function alarm, calculation result, and calling relationship tree. The calling relationship tree displays the calling relationship of tasks, the maximum cost of the function stack at each layer, and the cost of the local stack. In addition, ***Rec** is used to mark the functions that are recursively called and ***Ignore** is used to mark the function that ignores statistics.

Step 4 (Optional) Mark the function pointer.

The tool saves the function calling relationships that cannot be automatically parsed to the function calling relationship configuration file named **cfg/config.ini**. As shown in the following figure, the **LOS_LkPrint** method and **OsIdleTask** task entry function have function calling relationships that cannot be automatically parsed. You need to manually add the names of the functions called in the methods to the end of the **Remarks_here** line. If there are multiple functions, separate them with spaces; if no function needs to be added, mark **NULL**.

```
Miss_funcptr: <OsIdleTask>: 0x187f4
Remarks_here: <OsIdleTask>: NULL

Miss_funcptr: <LOS_LkPrint>: 0x2838c
Remarks_here: <LOS_LkPrint>: OsLkDefaultFunc
~
```

After editing the function calling relationship configuration file, perform step 2 to obtain the updated calculation result. The function calling relationship added to the configuration file has been added to the calling relationship tree.

Step 5 Calculate the size of a task stack.

The tool output shows the estimated size of the task stack. For example, the size of the **OsIdleTask** task stack displayed in step 2 is 476 bytes. When the task stack is set, the overheads for initializing the task stack and pushing the task switching register need to be added. It is recommended that the overhead be increased by 300 bytes. Therefore, the final task stack size of **OsIdleTask** is 776 bytes.

Step 6 (Optional) Estimate the interrupt stack.

The stack estimation tool cannot automatically identify or parse the interrupt entry function and interrupt callback function. You need to enter them in the **cfg/config.ini** file. The configuration format is as follows. **Interrupt_entry** indicates the interrupt entry function, and **Interrupt_isr** indicates the interrupt callback function, the interrupt ID and interrupt callback function entry need to be added after the colon (:).

```
Interrupt_entry: TrapVector OsIrqHandler
Interrupt_isr: 56 HwiStack001
Interrupt_isr: 57 HwiStack002
Interrupt_isr: 58 HwiStack003
```

Run the following command to estimate the interrupt stack:

```
python3 stackusage.py -irq -out Huawei_LiteOS.asm
```

The estimation result is as follows:

```
interrupt: stack cost = 444
           irq entry cost = 72
           irq 58 stack cost = 372
```

The overhead of the interrupt entry stack is 72 bytes. The called function of interrupt number 58 has the largest stack overhead 372 bytes. Therefore, the total overhead of the interrupt stack is 444 bytes.

----End

Tool Commands

Table 9-46

Command	Function
-h, --help	Displays help information.

Command	Function
-m N	Statistical mode. The options are as follows: 0 : Only assembly code is analyzed. 1 : The line number of the source code is analyzed. 2 : The source code in the disassembly file is analyzed.
-sp	Analyzes a specified task item. If -sp is not specified, all functions are counted.
-t taskEntry	Specifies the task entry. The default value is OsIdleTASK . If -sp is not specified, this command will be ignored.
-out	Exports the estimation result to the interface.
-j	Exports the estimation result to the JSON file.
-r N	Specifies the number of recursive layers <i>N</i> of a recursive function. The default value is 1 .
-irq	Calculates the interrupt stack. Only the interrupt stack is calculated if this command is set.

Precautions

- Through the function calling relationship parsed based on the static disassembly file, the dynamically assigned function pointer value cannot be accurately obtained. Therefore, the missing function calling relationship may cause calculation deviation. Supplement the pointer value in the function pointer configuration file to obtain more accurate calculation results.
- The recursion depth of the recursive function cannot be determined during static parsing. Therefore, the tool collects statistics based on the recursion depth 1 by default. If you need to set the recursion depth, add the **-r** input parameter when using the tool.
- The task stack initialization overhead and register push stack overhead during task switching need to be added to the parsing result. Generally, an overhead is related to the number of registers that can be used by the system. It is recommended that the overhead be increased by 300 bytes.
- The stack estimation tool can parse the disassembly files of the Arm and RISC-V instruction sets only.
- The interrupt stack increases with the interrupt nesting level, and interrupt nesting is related to the configured interrupt priority and actual system running status. Therefore, the stack estimation tool cannot statically estimate the stack overhead of interrupt nesting. The stack estimation tool allows you to configure multiple interrupt callback function entries. You can set the interrupt stack size by referring to the calculated maximum interrupt stack overhead based on the actual service nesting.

9.12 Debugging Case

9.12.1 Methods for Locating Memory Overwriting

9.12.1.1 Exception Information

Locate the fault based on the exception information. For details, see [Problem Locating Example](#).

9.12.1.2 Memory Backup Mechanism

For details about how to locate a problem through the memory backup mechanism. For details, see [Memory Backup Mechanism](#).

9.12.1.3 Memory Validity Check

For details about how to locate a problem by checking the memory validity. For details, see [Memory Validity Check](#).

9.12.1.4 Memory Size Check

For details about how to locate a problem by checking the memory size. For details, see [Memory Size Check](#).

9.12.1.5 Global Variable Check

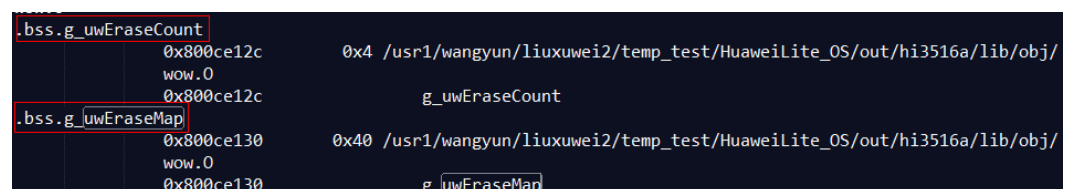
During debugging, it is found that a global variable is set to **0** in only one place. However, the value becomes a non-zero abnormal value in the displayed information. There is a high probability that the global variable is overwritten.

If the memory of a global variable is overwritten, you can find the address of the global variable in the **Huawei_LiteOS/out/<platform>/map** file. Pay attention to the recently used variable before the address and check whether the memory overwriting is caused by improper operations on the variable. For example, when **memcpy** or **memset** is performed on the variable, the memory overwriting occurs and current global variable is overwritten due to overflow.

Two global variables are defined in the file and are initialized.

```
UINT32 g_uwEraseMap[16] = {0};
UINT32 g_uwEraseCount = 0;
```

You can find the locations of the global variables in the bss section in the .map file.



.bss.g_uwEraseCount	
0x800ce12c	0x4 /usr1/wangyun/liuxuwei2/temp_test/HuaweiLite_OS/out/hi3516a/lib/obj/
wow.O	
0x800ce12c	g_uwEraseCount
.bss.g_uwEraseMap	
0x800ce130	0x40 /usr1/wangyun/liuxuwei2/temp_test/HuaweiLite_OS/out/hi3516a/lib/obj/
wow.O	
0x800ce130	g_uwEraseMap

If **g_uwEraseMap** is overwritten, find the address in the .map file, and then search for the variable before the address, that is, **g_uwEraseCount**. Analyze the usage of the **g_uwEraseCount** variable and check whether the **g_uwEraseCount** variable is overwritten.

9.12.1.6 Task Status Check

After Huawei LiteOS runs normally, run the **task** command to view status of all tasks. Values of **stackSize**, **WaterLine**, **StackPoint**, and **TopOfStack** can be used to determine whether a task stack causes illegal memory access.

The following use a task named **shellTask** as an example describes how to use the **task** command to determine whether memory is overwritten:

HuaweiLite OS # task													
Name	PID	Priority	Status	StackSize	WaterLine	StackPoint	TopOfStack	EventMask	SemID	CPURSE	CPURSE10s	CPURSE1s	MEMUSE
Set_Task	0x0	0	QueuePend	0x5000	0x360	0x80cfb4d0	0x80cf5c70	0x0	0xffff	1	1	1	-120080
IdleCore000	0x1	31	Ready	0x400	0x22c	0x80cfb22c	0x80cfbc90	0x0	0xffff	77	78	78	-7566584
system_wq	0x2	10	Pend	0x6000	0x23c	0x80d01f90	0x80cfcl38	0x0	0x2	0	0	0	0
app_Task	0x3	10	Delay	0x6000	0x3f50	0x80d01f38	0x80d02150	0x1	0xffff	0	0	0	21010728
shellTask	0x4	9	Running	0x3000	0x2810	0x80d10084	0x80d0d768	0xffff	0xffff	0	0	0	-52148
cmdParseTask	0x5	9	Pend	0x1000	0x28c	0x80d11540	0x80d10818	0x1	0xffff	0	0	0	218
pth01	0x6	10	Delay	0x6000	0x26b0	0x80d176c8	0x80d11868	0x0	0xffff	0	0	0	-340
RevAndDispatch	0x7	10	Ready	0x6000	0x25c	0x80d16318	0x80d12e40	0x0	0xffff	0	0	0	-2704
SavePara2Flash	0x8	10	Ready	0x6000	0x234	0x80d1fe40	0x80d1f900	0x0	0xffff	0	0	0	-340
ProcessEncStreamThread	0x9	10	Ready	0x10000	0xc40	0x80e11440	0x80e01b38	0x1	0xffff	3	4	4	8533536
UpdateTimeOsdThread	0xa	10	Delay	0x6000	0x860	0x80e17448	0x80e11640	0x0	0xffff	0	0	0	3008
ispRun	0xb	10	Ready	0x6000	0x1604	0x80e20648	0x80e14940	0x1	0xffff	11	9	10	-465952
tsmc_ret	0xc	10	Pend	0x12000	0x3f4	0x80e3fe48	0x80e3a238	0x1	0xffff	0	0	0	0
SendAFrameToAencThread	0xd	10	Ready	0x6000	0x5a3c	0x80e45f70	0x80e40250	0x0	0xffff	4	4	4	-345056
WORKSTATE_CHECK	0xe	10	Ready	0x6000	0x234	0x81cf23b0	0x81cece50	0x0	0xffff	0	0	0	-3136
hinci_Task	0xf	10	Ready	0x600	0x444	0x81cf4266	0x81cf3a20	0x1	0xffff	0	0	0	853676
hinci_Task	0x10	10	Ready	0x800	0x480	0x81cf5a38	0x81cf5490	0x0	0xffff	0	0	0	-40
TIMER_CHECK	0x11	10	Delay	0x6000	0x7cc	0x81cf6a60	0x81cf5c40	0x1	0xffff	0	0	0	-1060

StackSize = 0x3000 (size of the stack allocated to the task when the task is created)

WaterLine = 0x2810 (size of the used memory of the stack)

StackPoint = 0x80d10084 (stack pointer that points to the address of the task)

TopOfStack = 0x80d0d768 (top of the stack)

MaxStackPoint = TopOfStack + StackSize = 0x80d10768 (maximum range of accessible stack)

- If WaterLine > StackSize, the task causes memory overwriting.
- If StackPoint > MaxStackPoint or StackPoint < TopOfStack, the task stack causes memory overwriting.

10 Security

10.1 Address Space Layout Randomization

10.1 Address Space Layout Randomization

10.1.1 Overview

Basic Concept

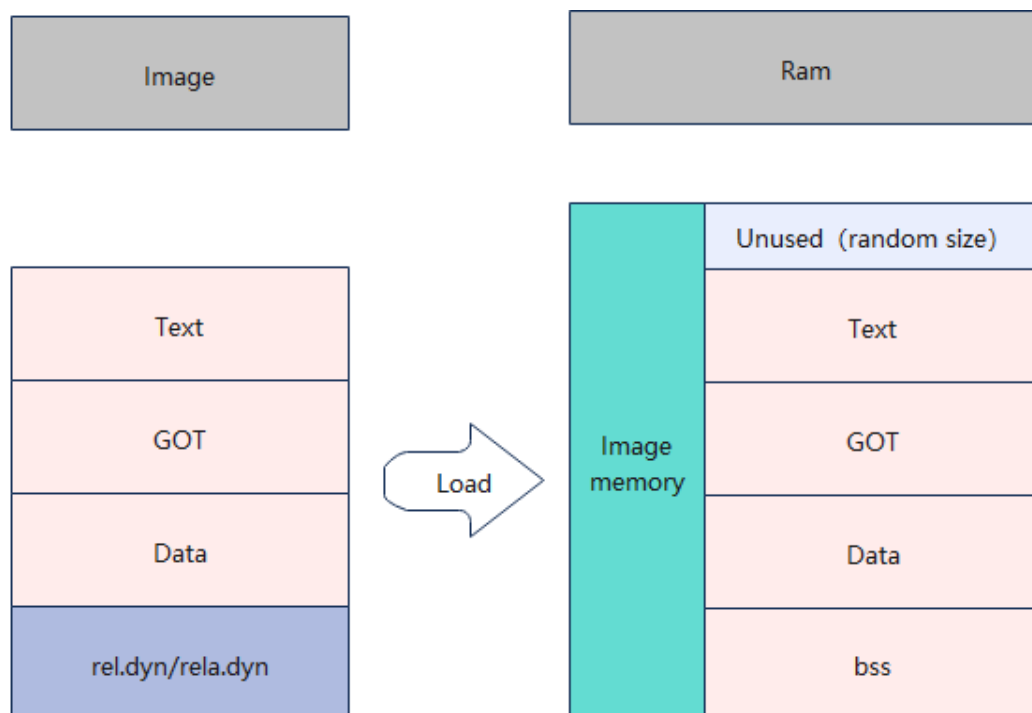
Address space layout randomization (ASLR) is a security technology against buffer overflow. It randomizes fixed running addresses such as code segments, data segments, and heaps to make it more difficult for attackers to locate target addresses and prevent attackers from directly locating and attacking code positions to hinder overflow attacks.

Operation Mechanism

Considering different application scenarios of the product, Huawei LiteOS currently supports two ASLR solutions: global offset table (GOT) and global symbol patching randomization (GSPR). The two solutions have different implementation mechanisms with their own advantages and disadvantages. You need to select a solution based on product requirements.

- **GOT**

A GOT is generated by using the **-pie/-fpie** option of GCC, so that the executable file becomes the position independent code (PIC). After the system is started, the bootloader transfers the random offset value to the relocation module, and then the relocation module corrects the address of the corresponding item in the GOT according to the location in the actual memory. The segments in the GOT are generated for accessing most global variables. Therefore, the access efficiency decreases and the running performance is affected because all global variable accesses are changed to the secondary redirection mode. Here is the image layout for the GOT solution.

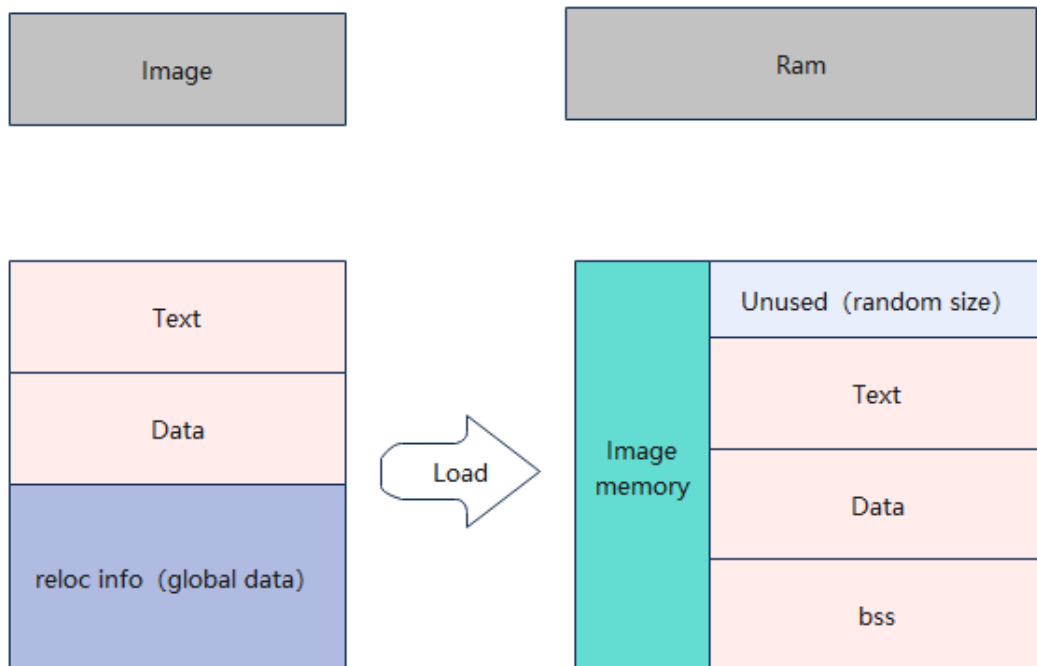


⚠ CAUTION

Add the **-fpie** option in the GOT solution for image compilation and the **-pie** option for connection. Two options both depend on the compiler, but some RISC-V compilers do not support **-fpie** and **-pie**. Therefore, you need to check whether the compiler supports those two options first.

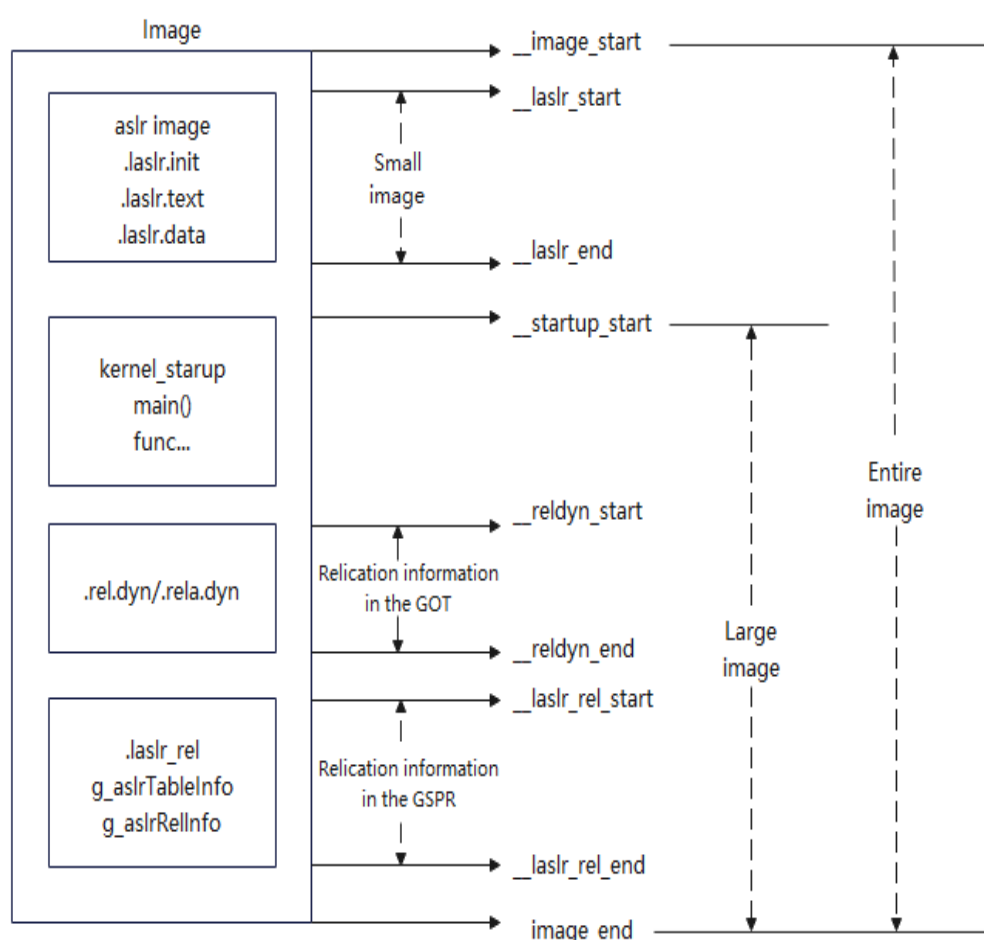
• GSPR

The relocation information segments are generated by using the **--emit-relocs** option of GCC. After the system is started, the bootloader transfers the random offset value to the relocation module, and then the relocation module corrects the instruction of the code segments in the GOT according to the offset location in the actual memory. The GSPR solution generates a segment of relocation information to relocate all instructions related to absolute address access. After relocation, the running effect is the same as that before randomization. However, a large number of symbols need to be relocated in the startup phase, which affects the startup performance. The specific impact on the performance depends on the actual test result of the platform. Here is the image layout for the GSPR solution.

**CAUTION**

Please ensure that the compiler supports the **--emit-relocs** option before selecting the GSPR solution.

Generally, no matter which ASLR is used, the compiled image layout is roughly as shown below:



The small image used for relocating symbols in the large image is aligned at the beginning of the entire image. The main function of the small image is to correct absolute addresses in the user's main program to the actual addresses with random offset symbols due to address randomization. The large image following the small image is the user's main program. The segments in the large image that stores relocation information are reused with the heap segments in OVERLAY mode and traversed by the small image to obtain the symbols to be relocated. The execution of the small image program is completed after relocation and then the start address of the large image continues to execute. During the kernel initialization, the memory is initialized. In this case, the segments of relocation information are overwritten and used as the system heap space to save the memory space.

The items in relocation information segments are in REL or RELA format. Generally, there is only one relocation type, which must be in the REL or RELA format, in the image compiled by a specific architecture. Each relocation item represents a symbol reference to be relocated and indicates how the modified symbol reference is calculated. The relocation items in the GOT solution and GSPR solution are generated by the assembler and the ELF file parsed by the script, respectively. The symbols in the preceding figure are the main symbols in the link script to adapt the address randomization function of platform. The following table shows the meanings and functions of symbols.

Table 10-1

Symbol	Description	Function
__image_start	Specifies the start address of the entire image.	Used to check whether the symbols in the relocation information segments are within the valid address range in the small image.
__image_end	Specifies the end address of the entire image.	Used to check whether the symbols in the relocation information segments are within the valid address range in the small image.
__laslr_start	Specifies the start address of the small image.	Used to ignore symbols in the small image based on the address range when the small image relocates the symbols.
__laslr_end	Specifies the end address of the small image.	Used to ignore symbols in the small image based on the address range when the small image relocates the symbols.
__reldyn_start	Specifies the start address of the relocation information segments in the GOT.	Used as the start address for traversing relocation items in the small image (GOT).
__reldyn_end	Specifies the end address of the relocation information segments in the GOT.	Used as the end address for traversing relocation items in the small image (GOT).
__laslr_rel_start	Specifies the start address of the relocation information segments in the GSPR.	Used as the start address for traversing relocation items in the small image (GSPR).
__laslr_rel_end	Specifies the end address of the relocation information segments in the GSPR.	Used as the end address for traversing relocation items in the small image (GSPR).

Symbol	Description	Function
__startup_start	Specifies the start address of the large image.	Used to specify the entry address of the large image after the small image relocates symbols in the large image to access the startup system of the large image.

10.1.2 Usage Guide

Application Scenario

According to Huawei requirements for security compilation options, the ASLR function is enabled to reduce the success rate of fixed address attacks and buffer overflow attacks as well as increase the difficulty for attackers to locate code addresses. You need to compile a randomizable image, and then correct the image symbol address based on the random loading address during loading.

Development Procedure

This part uses the RISC-V architecture as an example to describe the development process.

Step 1 Open the menu, choose **Kernel > Enable ASLR** and configure the task module.

Table 10-2

Configuration Item	Description	Value Range	Default Value	Dependency
LOSCFG_LASLR_GOT	Specifies whether to select GOT solution to enable ASLR.	YES/NO	YES	! LOSCFG_KERNEL_DYNLOAD && ! LOSCFG_KERNEL_RUNSTOP && ! LOSCFG_KERNEL_SCATTER && ! LOSCFG_FS_VFS && ! LOSCFG_KERNEL_PERF

Configuration Item	Description	Value Range	Default Value	Dependency
LOSCFG_LASLR_GSPR	Specifies whether to select GSPR solution to enable ASLR.	YES/NO	NO	! LOSCFG_KERNEL_DYNLOAD && ! LOSCFG_KERNEL_RUNSTOP && ! LOSCFG_KERNEL_SCATTER && ! LOSCFG_FS_VFS && ! LOSCFG_KERNEL_PERF

Step 2 Modify the platform link script.

Take the most basic and common link script of the RISC-V architecture in Huawei LiteOS as an example. (The key points of the link script are marked in bold.)

```
ENTRY(reset_vector)

PHDRS
{
  code      PT_LOAD;
  data      PT_LOAD;
}

INCLUDE board.ld
SECTIONS
{
  .laslr : ALIGN(0x20) {
    __image_start = .; /* must be the first address */
    __laslr_start = .;
    KEEP (*(SORT(.laslr.init)))
    KEEP (*(SORT(.laslr.text)))
    KEEP (*(SORT(.laslr.data)))
    /* If the GOT solution is selected and the relocation item is of the RELA type, access the character
    table of relocation information through the ID script link and open the following comments. */
    /* Take the GOT solution as an example. */
    //ImageInfo = .;
    //LONG(__image_start)
    //LONG(__image_end)
    //LONG(__laslr_start)
    //LONG(__laslr_end)
    //LONG(__reldyn_start)
    //LONG(__reldyn_end)
    //LONG(__startup_start)
    //LONG(0x5A5A5C5C)
    __laslr_end = .;
  } > ram : code

  /* text/read-only data */
  .text ALIGN(0x20) : ALIGN(0x20) {
    . = ALIGN(0x20);
    __text_start = .;
    __startup_start = .;
    KEEP(*(entry.text))
  }
```

```

__startup_end = .;
__itcm_text_start = .;
KEEP*(.itcm.text))
__itcm_text_end = .;
*(.text .text*)
*(.gnu.linkonce.t.*)
*(.plt)
*(.eh_frame_hdr)
*(.eh_frame)
} > ram : code

.initcall.init ALIGN(0x20) : ALIGN(0x20) {
__initcall0_start = ABSOLUTE(.);
KEEP*(SORT (.initcall0.init))
__initcall0_end = ABSOLUTE(.);

__initcall1_start = ABSOLUTE(.);
KEEP*(SORT (.initcall1.init))
__initcall1_end = ABSOLUTE(.);

__initcall2_start = ABSOLUTE(.);
KEEP*(SORT (.initcall2.init))
__initcall2_end = ABSOLUTE(.);

__initcall3_start = ABSOLUTE(.);
KEEP*(SORT (.initcall3.init))
__initcall3_end = ABSOLUTE(.);

__initcall4_start = ABSOLUTE(.);
KEEP*(SORT (.initcall4.init))
__initcall4_end = ABSOLUTE(.);

__initcall5_start = ABSOLUTE(.);
KEEP*(SORT (.initcall5.init))
__initcall5_end = ABSOLUTE(.);

__initcall6_start = ABSOLUTE(.);
KEEP*(SORT (.initcall6.init))
__initcall6_end = ABSOLUTE(.);

__initcall7_start = ABSOLUTE(.);
KEEP*(SORT (.initcall7.init))
__initcall7_end = ABSOLUTE(.);
} > ram : code

.gcc_except_table : {
KEEP*(SORT (.gcc_except_table))
KEEP*(SORT (.gcc_except_table.*))
} > ram : code
.exception_ranges ALIGN (0x8) : ONLY_IF_RW { *(.exception_ranges .exception_ranges*) } > ram : code

.dummy_post_text ALIGN(0x20) : ALIGN(0x20) {
. = ALIGN(0x20);
__text_end = .;
} > ram : code

.dynamic : { *(.dynamic) } > ram : code
.interp : { *(.interp) } > ram : code
.hash : { *(.hash) } > ram : code
.dynsym : { *(.dynsym) } > ram : code
.dynstr : { *(.dynstr) } > ram : code
.plt : { *(.plt) } > ram : code
.got : { *(.got.plt) *(.got) } > ram : data

.rodata ALIGN(0x20) : ALIGN(0x20) {
__rodata_start = .;
*(.srodata.cst16) *(.srodata.cst8) *(.srodata.cst4) *(.srodata.cst2)
*(.srodata* .srodata*)
*(.rodata .rodata* .gnu.linkonce.r*)

```



```

} > ram : data

/*
 * extra linker scripts tend to insert sections just after .rodata,
 * so we want to make sure this symbol comes after anything inserted above,
 * but not aligned to the next section necessarily.
 */
.dummy_post_rodata ALIGN(0x20) : ALIGN(0x20) {
    . = ALIGN(0x20);
    __rodata_end = ;
} > ram : data

.data ALIGN(0x20) : ALIGN(0x20) {
    /* writable data */
    __ram_data_start = ;
    *(.sdata .sdata.* .gnu.linkonce.s.*)
    *(.data .data.* .gnu.linkonce.d.*)
    . = ALIGN(0x4);
    KEEP(*( SORT (.liteos.table.*)));
} > ram : data

.ctors : ALIGN(0x4) {
    __ctor_list = ;
    KEEP (*(ctors .init_array))
    __ctor_end = ;
} > ram : data

.dtors : ALIGN(0x4) {
    __dtor_list = ;
    KEEP (*(dtors .fini_array))
    __dtor_end = ;
} > ram : data

/*
 * extra linker scripts tend to insert sections just after .data,
 * so we want to make sure this symbol comes after anything inserted above,
 * but not aligned to the next section necessarily.
 */
.dummy_post_data ALIGN(0x20) : ALIGN(0x20) {
    . = ALIGN(0x20);
    __ram_data_end = ;
    . = ALIGN(64);
} > ram : data

OVERLAY : {
    .laslr_rel {
        __laslr_rel_start = ;
        KEEP (*(SORT(.laslr_rel.*)))
        __laslr_rel_end = ;
    }
    .rela.dyn {
        __reldyn_start = ;
        *(.rela.*)
        __reldyn_end = ;
    }
    .heap {
        __heap_start = ;
        KEEP(*(heap));
        __heap_end = ;
    }
} > ram : data
ASSERT((__laslr_rel_end - __laslr_rel_start) <= (__heap_end - __heap_start),
    "check liteos.ld, region laslr_rel size bigger than heap, rel address will change")

/* uninitialized data (in same segment as writable data) */
.bss ALIGN(0x20) : ALIGN(0x20) {
    __int_stack_start = ;
    *(.int_stack);
    . = ALIGN(0x4);

```

```

__int_stack_end = .;
KEEP(*(.bss.prebss.*))
. = ALIGN(0x20);
__bss_start = .;
__global_pointer$ = .;
*(.sbss .sbss.* .gnu.linkonce.sb.*)
*(.scommon)
*(.gnu.linkonce.b.*)
*(.bss .bss.*)
*(COMMON)
. = ALIGN(0x20);
__bss_end = .;
} > ram : data

. = ALIGN(0x20);
__image_end = .;
/* Strip unnecessary stuff */
/DISCARD/ 0 : { *(.comment .note) } > ram
}

```

CAUTION

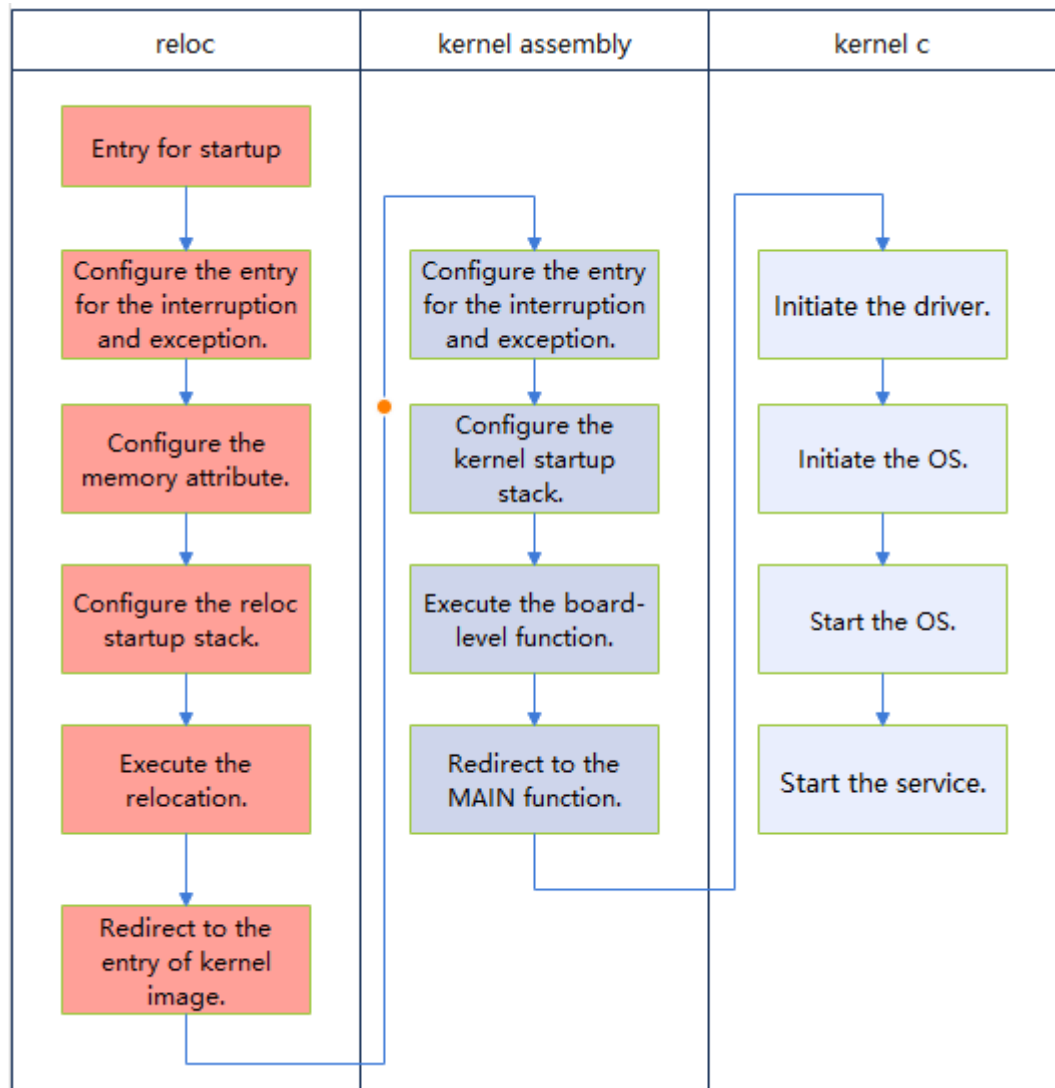
1. Segments are injected to the system heap in the form of static arrays because relocation information reuses the heap segments.
2. If the GOT solution is selected and the relocation items are of the RELA type, the character pool for constructing the relocation information cannot adopt static value assignment because the global address variables exist in the ADDEND variable of the relocation table. Therefore, the segments are injected by using the link script.

Step 3 Compile the startup file of the small image.

Take the basic startup process of an RISC-V architecture platform in Huawei LiteOS as an example. The process is as follows:

1. (Optional) Configure the cache as required.
2. Configure the PMP. (The PMP is required if the GSPR solution is selected, because the unaligned address needs to be operated when the unaligned address symbol is relocated. Otherwise, the system is suspended.)
3. (Optional) Set the general exception entry for debugging.
4. Set the startup stack for the small image. (In principle, the startup stack address cannot overlap the image loading address. The end part of the system memory space is used as the startup stack of the small image. After the large image is executed, the memory can be reused as the system heap space.)
5. Relocate the large image symbols (by directly calling the encapsulated function).
6. Clear the relocation information in the small image and start the system through the start address of the large image (by directly calling the encapsulated function).

The following figure shows the basic system startup process during address randomization.



Step 4 Compile the image.

1. Use the **Make** or **CMake** compilation framework to compile the BIN image file.
2. After the image is compiled, the operations performed in the GOT solution are different from those performed in the GSPR solution.
 - For the GOT solution, go to the **Huawei_LiteOS/build/scripts/aslr** directory and run the **check_aslr_rel_type.py** script. The following is an example:

```
$ ./check_aslr_rel_type.py GOT riscv32-linux-musl- out/vs_server
```
 - For the GSPR solution, go to the **Huawei_LiteOS/build/scripts/aslr** directory and run the **generate_gspr_rel_info.py** script. The following is an example:

```
$ ./generate_gspr_rel_info.py GSPR riscv32-unknown-elf- out/vs_server
```
3. Go to the **Huawei_LiteOS/build/scripts/aslr** directory and run the **check_aslr_image.py** script. The following is an example:

```
$ ./check_aslr_image.py out/vs_server out/vs_server.map riscv32-unknown-elf- ${LITEOS_LDFLAGS}
```

 NOTE

out/vs_server indicates the generated ELF file, and **\$(LITEOS_LDFLAGS)** is the link option in the Huawei LiteOS compilation framework.

----End

Solution Comparison

In addition to the compiler options and implementation mechanisms, the GOT solution and the GSPR solution have distinct advantages and disadvantages. The following table lists their differences.

Table 10-3

Item	GOT	GSPR
Startup Time	Minor impacts	Major impact
Size of the text segment during running	Major impact	Minor impacts
Size of the relocation information segment	Minor impacts	Major impact
Runtime performance	Major impact	Minor impacts

Compared with the GSPR solution, the GOT solution has fewer relocation items and small segments. Therefore, the time is short for the small image to perform symbol relocation. However, because the GOT solution needs to indirectly access symbols through the GOT, its runtime performance is poor, and the text segment is large during the runtime. The relocation information segments are displayed in the ELF file. Therefore, the size of the compiled image in the GOT solution is smaller than that in the GSPR solution. However, the subsequent relocation information segment can be reused by the system heap space, which is large enough to avoid extra running memory occupation.

Product Dependency

- The compiler must support the corresponding compilation link options. For the GOT solution, the compiler must support the **-pie** and **-fpie** options. For the GSPR solution, the compiler must support the **--emit-relocs** option.
- The product must provide a secure random number that meets Huawei's trustworthiness requirements for random address loading. For details, see the *Cryptographic Algorithms Application Standards*. A true random number generator (TRNG) must obtain sufficient entropy values from the entropy source for full entropy output. The product needs to provide entropy values based on the memory layout of the platform. The reference values are as follows:
 - a. In Linux, there are 256 8-bit entropy values for a 32-bit platform, and 1024 10-bit entropy values for a 64-bit platform.

- b. There are more than 7 thousands entropy values within 1 MB space of the system.
- The platform without a virtual address uses the physical address to load the image. A certain memory range must be reserved for random offset of the image.
- The bootloader supports random loading. It can load the image to the random address space that complies with the platform constraints. In addition, Huawei LiteOS can obtain the random value, which is cleared if the small image completes the symbol relocation.

Constraints

- Use **check_aslr_rel_type.py** to check the relocation type. This method applies only to the GOT solution in the trustlist verification mode. Currently, only the Arm, AArch64, and RISC-V architectures are supported for the check. The relationship between the architecture and the corresponding relocation type is as follows:

Table 10-4

Architecture	Relocation Type
Arm	R_ARM_RELATIVE
AArch64	R_AARCH64_RELATIVE
RISC-V	R_RISCV_RELATIVE

When the GOT solution is selected, if one of the preceding three architectures is used, the relocation information segment contains only the relocation algorithm listed in the preceding table. Otherwise, the script reports an error. In this case, check whether the symbol implementation corresponding to the invalid relocation algorithm is correct. If the adaptation platform architecture is not one of the three, you need to modify the script based on the site requirements to expand the verification trustlist.

- The **generate_gspr_rel_info.py** script is used to construct relocation information segment data. This script is used only in the GSPR solution in trustlist verification mode. Currently, only the Arm and RISC-V architectures are supported for the construction. The GSPR solution generates the relocation information of all segments. The relocation type corresponding to the relocation item in a code segment is not only related to the architecture, but also related to the compiler and compilation options. Currently, the relationship between the two architectures and the relocation types is as follows:

Table 10-5

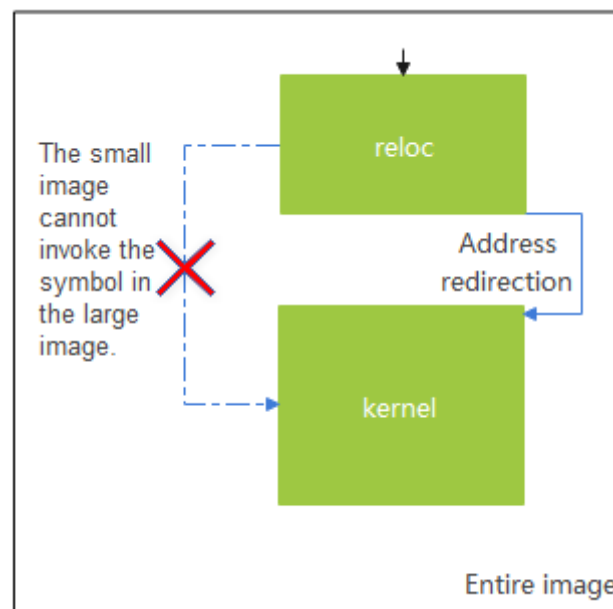
Architecture	Relocation Type
Arm	R_ARM_ABS32

Architecture	Relocation Type
	R_ARM_MOVW_ABS_NC
	R_ARM_MOVT_ABS
	R_ARM_CALL
	R_ARM_JUMP24
	R_ARM_V4BX
	R_ARM_NONE
RISCV-V	R_RISCV_RVC_JUMP
	R_RISCV_RVC_BRANCH
	R_RISCV_JAL
	R_RISCV_BRANCH
	R_RISCV_BRANCHI
	R_RISCV_CALL
	R_RISCV_RELAX
	R_RISCV_NONE
	R_RISCV_PCREL_HI20
	R_RISCV_PCREL_LO12_I
	R_RISCV_GPREL_I
	R_RISCV_GPREL_S
	R_RISCV_HI20
	R_RISCV_LO12_I
	R_RISCV_LO12_S
	R_RISCV_32
	R_RISCV_BEQ_X
	R_RISCV_RVC_LUI
	R_RISCV_BEQI_YK
	R_RISCV_GPREL17_S
	R_RISCV_GPREL17_I
	R_RISCV_GP_HI20
	R_RISCV_GP_LO12_I

When the GSPR solution is used, if a new relocation type is not in the trustlist during the relocation of the parsed ELF file, a compilation error is reported. In this case, add the untrusted relocation type to the trustlist and determine whether the type needs to be relocated. When adapting to the new architecture platform, you need to implement a Python script that contains the trustlist of relocation type and relocation information parsing function set, as well as the relocation algorithm related to the architecture.

10.1.3 Precautions

- The fixed address is used in the code and must be changed to the symbol in the link script. For example, if the **IMAGE_START** macro address is used to determine the start address of the function, the symbol needs to be changed to **__image_start**, which changes with the loading address. If the symbol in the link script is used to indicate the memory size instead of the address in the code, change the value to a fixed value. Otherwise, the symbol will be relocated. For example, if **__heap_size** is used to indicate the heap memory size, the value will be changed during relocation. In this case, you need to change **__heap_size** to the macro **HEAP_SIZE**.
- The current address randomization supports only backward loading, that is, the actual loading address is greater than the entry address in the image.
- If ASLR is enabled, the following functions cannot be enabled: low-power framework (power-off restoration), VFS, scatter loading, and Perf.
- Ensure that the small image used to relocate the large image cannot invoke the symbols in the large image because the symbols have not been relocated and the symbol address has not been corrected. In addition, address independent code needs to be implemented for the small image. The following figure shows the details.

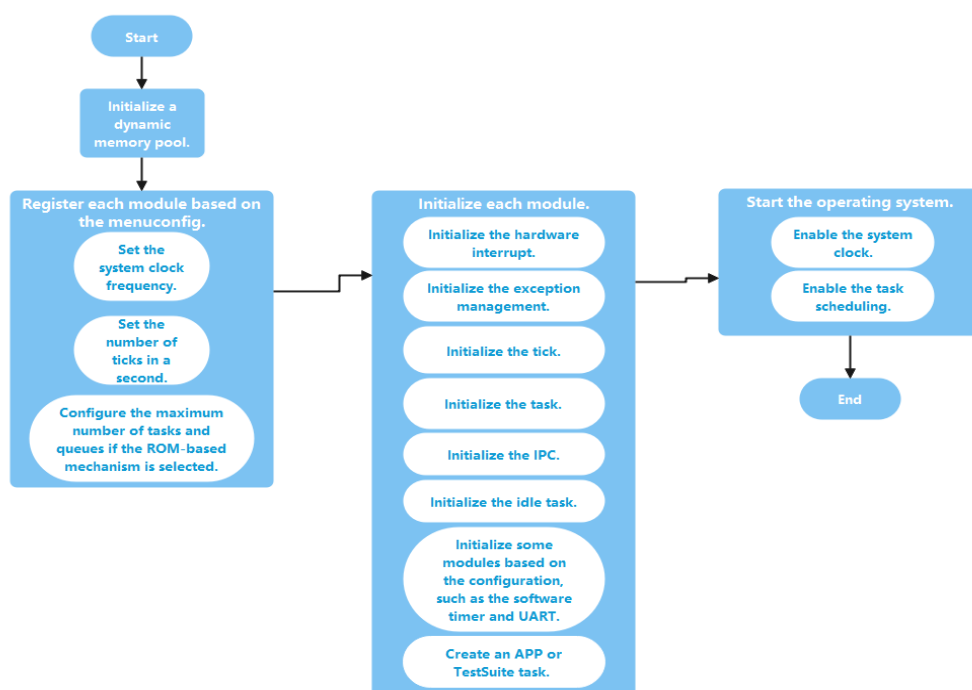


11 Appendix

11.1 Startup Process of Huawei LiteOS Kernel

11.2 Key Modification Points for Platform Compatibility Adaptation

11.1 Startup Process of Huawei LiteOS Kernel



The preceding figure shows the system startup process. Some modules are initiated only when the module macro is enabled. You can choose whether to create an app task (app_init). The menu configuration path is as follows:

Debug > Enable TestSuite or AppInit > Choose TestSuite or AppInit.

After `LOSCFG_APPINIT_TESTSUITE` is enabled, you can create an app or testsuite task. If `LOSCFG_APPINIT_TESTSUITE` is disabled, neither the app nor testsuite task is created.

11.2 Key Modification Points for Platform Compatibility Adaptation

Differences

The sizes of some data types in a 32-bit environment are different from those in a 64-bit environment. Table 1 compares data models and data type sizes of the 32-bit and 64-bit environments. During development, you need to pay attention to the changes in data type sizes.

Table 11-1 Data type size differences

Programming type	Size in A32	Size in A64-LP64
char	8-bit	8-bit
short	16-bit	16-bit
int	32-bit	32-bit
int *(pointer)	32-bit	64-bit
long	32-bit	64-bit
long long	64-bit	64-bit
float	32-bit	32-bit
double	64-bit	64-bit
size_t	32-bit	64-bit

The 64-bit compiler of Huawei LiteOS uses the LP64 instruction mode. As shown in the preceding table, the pointer, long, and size_t types are upgraded to 64-bit.

Compared with the 32-bit mode, the 64-bit mode has some usage differences due to the bit width difference. Check the differences in the application code by referring to the following seven modification points:

Usage Differences

1. Process pointers.

You are not advised to forcibly convert a pointer to a non-pointer type, such as long and int. Pointers can be forcibly converted to other types of pointers and void*. If a pointer is forcibly converted to a non-pointer type, the 64-bit compiler will report an alarm, which needs to be cleared.

NOTICE

You are not advised to forcibly convert a pointer to unsigned long or int temporarily when the pointer is used for mathematical calculation. You can convert the pointer to char* and then add or subtract a constant.

2. Change long to int.

You need to change long to int, unless you use long in public kernel modules. Do not use long or forcibly convert other types to long.

NOTICE

In some 64-bit programming standards, the use of the long type is not allowed. If the long type is used, errors are very likely to occur.

3. Check all assignments and comparison statements that are related to ints, longs, and pointers.

Although long is not allowed in principle, the referenced data type may be defined as long in the kernel. In this case, if the internal native data definition cannot be modified, you need to check the reference.

In new compilers, data types are automatically converted according to the following rules:

int + long = long

unsigned + signed = unsigned

Here is an example:

```
long long a;
int b;
unsigned int c;
b = -2;
c = 1;
a = b + c;
```

The value of **a** is **0x00000000FFFFFFFF** rather than -1. The cause of this result is that b+c indicates signed+unsigned, of which the result is an unsigned integer, and to convert the unsigned integer to a long long, 0 must be inserted from the left of FFFFFFFF. Here is one more example:

```
long long a;
int b;
unsigned int c;
b = -2;
c = 1;
a = (long long) b + c;
```

The value of **a** is **0xfffffffffffffffe**. The value of **b** becomes **0xfffffffffffffffe** after **b** is converted to a long. The result of 0xfffffffffffffffe plus the unsigned int c is the value of a.

4. Specify data types in constant expressions.

If you want to use a constant as 64 bits, append ll or LL to the end of the constant. If the constant is not followed by ll or LL, the compiler may process it as an int. For example:

```
long long SetBitN(long long value, unsigned bitNum)
{
    long long mask;
```

```
mask = 1 << bitNum;
return value | mask;
}
```

In this example code, 1 has been processed as an int. If bitNum is a number exceeding 32, the result of 1 << bitNum is invalid because the part that is higher than the first 32 bits is truncated. Then the result of 1 << bitNum becomes 0. You need to write the code in the following way:

```
long long SetBitN(long long value, unsigned bitNum)
{
    long long mask;
    mask = 1LL << bitNum;
    return value | mask;
}
```

In the following code, the two operations will clear the upper 32 bits of the pointers.

```
(void*) ((long long)p & ~(16 - 1))
(void*) ((long long)p & ~(alain - 1)) /* alain is u32.*/
```

You can modify the code as follows:

```
(void*) ((long long)p & ~((long long)16 - 1))
(void*) ((long long)p & ~((long long)alain - 1)) /* alain is u32.*/
```

Alternatively, you can use the following definitions to avoid the problem:

```
1LL // (long long)
1ULL // (unsigned long long)
```

5. Formatted input/output problems and solutions

Compared with a 32-bit system, the following problems exist in a 64-bit system:

- In a 32-bit system, %x can be used to print pointers. However in a 64-bit system, truncation occurs when %x is used to print pointers.
Solution: Use %p to input and output pointers.
- In a 32-bit system, %d can be used to print a value of type int or long. However in a 64-bit system, this usage of %d results in truncation.
Solution: Use %ld to input and output long.

On a 32-bit platform, a size_t is actually an unsigned int. On a 64-bit platform, a size_t is actually an unsigned long. As a result, when functions such as printf are used, a warning is reported regardless of whether %u or %llu is used for printing. To avoid forcible data type conversion caused by formatted print of data types, you can use %lu to print a size_t.

The buffer required in a 64-bit is larger than that required in a 32-bit system. During input, ensure that the buffer size is sufficient. Here is an example code:

```
char * buf[9];
int a = 0;
Sprintf(buf, "%p", &a);
```

This code is correct in a 32-bit system. A pointer value is c0001a38. After the value is input, it has 8 characters. However in a 64-bit system, this code is incorrect because &a has 8 bytes in length.

6. Macro definitions of maximum numbers and invalid numbers

The macro definitions of maximum numbers and invalid numbers may have changed in a 64-bit system. For example:

```
#define BYTES_IN_WORD 4
#define INVALID_DMA_ADDRESS 0xffffffff
```

Check whether the values are invalid or maximum in 64-bit mode. For example:

```
size_t count = BIG_NUMBER;
for (unsigned int index = 0; index != count; index++)
.....
```

The size_t is 64-bit, but the index is 32-bit. Therefore, the for statement will be repeated.

7. Check the size of the member variable in the union.

All member variables in a union must have the same size. In a 64-bit environment, some variables may become larger in size than they are in a 32-bit environment. This breaks the balance between member variables. Ensure that the all member variables in a union have the same size.

The following example is a common open source code package, which can run in an ILP32 environment but cannot run in an LP64 environment. In this example code, the length of the unsigned short array is 2, and the unsigned short array shares the same space with the long. This code is incorrect in an LP64 environment.

```
union{
    unsigned long bytes;
    unsigned short len[2];
} size;
```

Check whether there are special applications of unions. If there are special applications, check unions in all code to ensure that all members of each union have the same size in an LP64 environment.

Suggestions

1. Data types

- If 64-bit computing and storage are involved, the data type is long long. If 32 bits are sufficient, you are advised to use the int or unsigned int data type to reduce the memory. Do not define the data type as long.
- Type definitions use architecture-independent type definitions: u32, u16, u64, s16, s32 and s64.

2. Structure definition: Place commonly used data fields at adjacent positions so that they can be fetched to the cache at once. This improves operating performance and efficiency.

3. In a 64-bit system, the 8-byte alignment is used. Note that you need to place structures according to the new alignment mode.

```
struct MyStruct
{
    bool m_bool;
    char *m_pointer;
    int m_int;
};
```

In the preceding example, 4 bytes will be inserted between bool and char* to align them on an 8-byte boundary. To reduce memory use, you need to arrange the structures as follows:

```
struct MyStructOpt
{
    char *m_pointer;
    int m_int;
    bool m_bool;
};
```

Other Suggestions

1. Non-alignment operations

Different chip platforms have different degrees of support for non-alignment operations of different media. Before using non-alignment, ensure that the target platform supports non-alignment access.

2. Use of unions and bit fields

The bit order of the union and bit field is not defined in the C language standard. During compilation, the bit order of the union and bit field is determined by the compiler implementation. It is recommended to reduce the use of union and bit fields in program development.